BALLUFF

A brand of Balluff MATRIX VISION



mvBlueCOUGAR-X/-XD

Technical Manual

English - Version 6.07

1.1 About this manual . . . 1
1.1.1 Goal of the manual . . . 1
1.1.2 Contents of the manual . . . 1
1.2 Migration Guide . . . 2
1.2.1 Introduction . . . 2
1.2.2 Hardware Revision 5 . . . 2
1.3 Imprint . . . 3
1.4 Legal notice . . . 5
1.4.1 Introduction . . . 5
1.4.2 cJSON . . . 5
1.4.3 Unity . . . 5
1.5 Revisions . . . 6
1.6 Symbols and Conventions . . . 7
1.6.1 Explanation of the warnings . . . 7
1.7 Important Information . . . 8
1.7.1 Important Safety Instructions . . . 8
1.7.2 Operating considerations . . . 8
1.7.2.1 Important Safety Notes . . . 8
1.7.2.2 Handling And Cleaning . . . 9
1.7.2.3 Installing . . . 9
1.7.2.4 Optimizing performance and life time . . . 9
1.7.2.5 Connectors . . . 9
1.7.2.6 Cleaning . . . 9
1.7.2.7 Additional temperature and power consumption notes concerning mvBlueCOUGAR-XD and mvBlueCOUGAR-X2xx . . . 11
1.7.3 Conformity . . . 12
1.7.4 Additional notices . . . 17
1.7.4.1 For customers in the U.S.A. . . . 17
1.7.4.2 For customers in Canada . . . 17
1.7.4.3 Pour utilisateurs au Canada . . . 17
1.8 Introduction . . . 18
1.8.1 Software concept . . . 19
1.8.2 Order code nomenclature . . . 20
1.8.2.1 mvBlueCOUGAR-X . . . 20
1.8.2.2 mvBlueCOUGAR-XD . . . 22
1.8.2.3 mvBlueCOUGAR-3X . . . 23
1.8.2.4 Ordering code samples . . . 24
1.8.3 What's inside and accessories . . . 24
1.8.3.1 Accessories for the mvBlueCOUGAR . . . 26
1.9 Quickstart . . . 27
1.9.1 System Requirements . . . 27
1.9.1.1 Host System . . . 27

1.9.1.2 Network interface card (NIC) settings . . . 27
1.9.1.3 Supported Operating Systems . . . 28
1.9.2 Installing The mvGenTL-Acquire Package . . . 29
1.9.2.1 Windows . . . 29
1.9.2.2 Linux . . . 32
1.9.3 Connecting The Camera . . . 35
1.9.3.1 Communicating With The Camera . . . 36
1.9.3.2 Communicating With A Camera In A Different Subnet . . . 37
1.9.3.3 Setting Up The Camera . . . 37
1.9.3.4 About Settings . . . 37
1.9.4 Driver concept . . . 40
1.9.4.1 NeuroCheck Support . . . 41
1.9.4.2 VisionPro Support . . . 42
1.9.4.3 HALCON Support . . . 42
1.9.4.4 LabVIEW Support . . . 42
1.9.4.5 DirectShow Support . . . 42
1.9.4.6 Micro-Manager Support . . . 42
1.9.5 Relationship Between Driver, Firmware And SDK . . . 43
1.9.6 Optimizing Network Performance . . . 46
1.9.6.1 Checklist for Windows . . . 46
1.9.6.2 Checklist for Linux . . . 57
1.9.6.3 Multi-Core Acquisition Optimizer . . . 64
1.9.7 Using GigE Vision™ Devices In A Docker Container . . . 79
1.9.7.1 Host Preparation . . . 80
1.9.7.2 Building A Docker Image . . . 80
1.9.7.3 Starting The Docker Container . . . 81
1.9.7.4 Validation . . . 82
1.10 Technical Data . . . 82
1.10.1 mvBlueCOUGAR-X / -POE . . . 82
1.10.1.1 Dimensions . . . 82
1.10.1.2 Connectors . . . 84
1.10.1.3 Signal LED . . . 92
1.10.2 mvBlueCOUGAR-X-POE-I / -IP67 . . . 94
1.10.2.1 Dimensions . . . 94
1.10.2.2 Connectors . . . 95
1.10.3 mvBlueCOUGAR-XD . . . 99
1.10.3.1 Dimensions . . . 99
1.10.3.2 Connectors . . . 102
1.10.3.3 Signal LED . . . 108
1.10.4 mvBlueCOUGAR-3X . . . 110
1.10.4.1 Dimensions . . . 110
1.10.4.2 Model specific features . . . 112

1.10.5 Summary of components . . . 113
1.10.6 Summary of advanced features . . . 114
1.11 Sensor Overview . . . 116
1.11.1 Image data flow . . . 116
1.11.2 mvBlueCOUGAR-X specific CMOS sensors . . . 117
1.11.2.1 Sony Pregius S . . . 117
1.11.2.2 Sony Pregius . . . 118
1.11.2.3 Sony Starvis . . . 121
1.11.2.4 Sony Polarsens . . . 121
1.11.2.5 Aptina, CMOSIS, e2v . . . 122
1.11.2.6 mvBlueCOUGAR-XD specific CMOS sensors . . . 125
1.11.3 CCD sensors . . . 128
1.11.3.1 mvBlueCOUGAR-XD specific CCD sensors . . . 130
1.11.3.2 Negative gain . . . 132
1.11.4 Output sequence of color sensors (RGB Bayer) . . . 133
1.11.5 Bilinear interpolation of color sensors (RGB Bayer) . . . 133
1.11.6 Supported image formats . . . 134
1.12 Filters . . . 136
1.12.1 Hot Mirror Filter . . . 136
1.12.2 Cold mirror filter . . . 137
1.12.3 Glass filter . . . 138
1.13 GUI tools . . . 139
1.13.1 Introduction . . . 139
1.13.2 wxPropView . . . 139
1.13.3 mvDeviceConfigure . . . 139
1.13.4 mvIPConfigure . . . 139
1.13.5 mvGigEConfigure . . . 139
1.14 GenICam and advanced features . . . 140
1.14.1 Introduction . . . 140
1.14.2 Device Control . . . 141
1.14.3 Image Format Control . . . 143
1.14.4 Acquisition Control . . . 144
1.14.5 Counter And Timer Control . . . 148
1.14.6 Analog Control . . . 150
1.14.7 Color Transformation Control . . . 152
1.14.8 Event Control . . . 153
1.14.9 Chunk Data Control . . . 154
1.14.10 File Access Control . . . 155
1.14.11 Digital I/O Control . . . 155
1.14.12 Encoder Control . . . 156
1.14.13 Sequencer Control . . . 157
1.14.13.1 Sequencer overview . . . 157

1.14.13.2 Configuration of a sequencer set . . . 157
1.14.14 Transport Layer Control . . . 162
1.14.15 User Set Control . . . 163
1.14.16 Action Control . . . 164
1.14.17 mv Logic Gate Control . . . 165
1.14.18 mv Flat Field Correction Control . . . 166
1.14.19 mv Serial Interface Control . . . 167
1.14.20 mv Defective Pixel Correction Control . . . 167
1.14.21 mv Frame Average Control (only with specific models) . . . 168
1.14.22 mv Auto Feature Control . . . 169
1.14.23 mv High Dynamic Range Control (only with specific sensor models) . . . 169
1.14.24 LUT Control . . . 170
1.14.24.1 mvLUTType . . . 171
1.14.24.2 mvLUTInputData . . . 171
1.14.24.3 mvLUTMapping . . . 171
1.14.24.4 LUT support in MATRIX VISION cameras . . . 172
1.15 Developing applications using the mvIMPACT Acquire SDK . . . 177
1.16 DirectShow interface . . . 178
1.16.1 Supported interfaces . . . 178
1.16.1.1 IAMCameraControl . . . 178
1.16.1.2 IAMDroppedFrames . . . 178
1.16.1.3 IAMStreamConfig . . . 178
1.16.1.4 IAMVideoProcAmp . . . 178
1.16.1.5 IKsPropertySet . . . 178
1.16.1.6 ISpecifyPropertyPages . . . 178
1.16.2 Logging . . . 178
1.16.3 Setting up devices for DirectShow usage . . . 178
1.16.3.1 Registering devices . . . 179
1.16.3.2 Renaming devices . . . 181
1.16.3.3 Using regsvr32 . . . 182
1.17 Troubleshooting . . . 183
1.17.1 Error code list . . . 183
1.17.2 Accessing log files . . . 196
1.17.2.1 Windows . . . 196
1.17.2.2 Linux . . . 196
1.17.3 General Issues . . . 198
1.17.3.1 I can see the camera in mvIPConfigure, but I cannot access the camera . . . 198
1.17.3.2 Image Data Is Lost (Incomplete Frames) . . . 199
1.17.3.3 The error counter increases . . . 200
1.17.3.4 I get an exception when setting OffsetX or OffsetY . . . 202
1.17.3.5 Cognex VisionPro . . . 203
1.17.3.6 I cannot focus on distant objects . . . 208

1.17.3.7 mvGigEConfigure freezes when trying to Activate/Deactivate the filter driver a Link Aggregation Group(LAG) . . . 208
1.17.3.8 mvBlueCOUGAR-X105G with frame rates < 5.75 Hz . . . 208
1.17.3.9 I get an oscillating frame rate . . . 209
1.17.3.10 Why does updating the device list take so long . . . 209
1.17.4 Windows . . . 210
1.17.4.1 The 'LostImagesCount' increases . . . 210
1.17.5 Linux . . . 211
1.17.5.1 No GenICam devices are detected on a Linux system . . . 211
1.17.5.2 No GigE Vision™ devices are detected on a Linux system . . . 212
1.18 Glossary . . . 213
1.19 Use Cases . . . 222
1.19.1 Introducing acquisition / recording possibilities . . . 223
1.19.1.1 Acquiring a number of images . . . 223
1.19.1.2 Recording sequences in the camera . . . 224
1.19.1.3 Recording sequences with pre-trigger . . . 225
1.19.1.4 Creating acquisition sequences (Sequencer Control) . . . 226
1.19.1.5 Generating very long exposure times . . . 239
1.19.1.6 Working with multiple AOIs (mv Multi Area Mode) . . . 241
1.19.1.7 Working with burst mode buffer . . . 245
1.19.1.8 Using the SmartFrameRecall feature . . . 249
1.19.1.9 Using The Linescan mode . . . 252
1.19.1.10 Using the mvBlockscan feature . . . 256
1.19.1.11 Working with Event Control . . . 258
1.19.1.12 Polarized Data Extraction . . . 261
1.19.1.13 Using Video Stream Recording . . . 265
1.19.1.14 Introducing multicasting . . . 276
1.19.2 Improving the acquisition / image quality . . . 284
1.19.2.1 Correcting image errors of a sensor . . . 284
1.19.2.2 Optimizing the color/luminance fidelity of the camera . . . 297
1.19.2.3 Reducing noise by frame averaging . . . 312
1.19.2.4 Setting a flicker-free auto expose and auto gain . . . 315
1.19.2.5 Working with binning / decimation . . . 318
1.19.2.6 Working with Sony's 4 Tap CCD sensors . . . 321
1.19.2.7 Minimizing sensor pattern of mvBlueCOUGAR-X1010G . . . 324
1.19.2.8 Working with the dual exposure feature ("mvMultiZone") of mvBlueCOUGAR-XD107 327
1.19.2.9 Working with the dual gain feature of mvBlueCOUGAR-XD107/XD1031 and mvBlueCOUGAR-X102m/X107 . . . 330
1.19.2.10 Working with the dual ADC feature of mvBlueCOUGAR-XD107 . . . 334
1.19.2.11 Working With Gain And Black-Level Values Per Color Channel . . . 335
1.19.3 Improving the communication . . . 343
1.19.3.1 Optimizing the bandwidth . . . 343
1.19.3.2 Discovering Devices In Different Subnets . . . 346

1.19.4 Working with triggers . . . 347
1.19.4.1 Processing triggers from an incremental encoder . . . 348
1.19.4.2 Generating a pulse width modulation (PWM) . . . 351
1.19.4.3 Outputting a pulse at every other external trigger . . . 354
1.19.4.4 Creating different exposure times for consecutive images . . . 355
1.19.4.5 Detecting overtriggering . . . 358
1.19.4.6 Triggering of an indefinite sequence with precise starting time . . . 362
1.19.4.7 Low latency triggering . . . 365
1.19.5 Working with I/Os . . . 366
1.19.5.1 Controlling strobe or flash at the outputs . . . 367
1.19.5.2 Creating a debouncing filter at the inputs . . . 370
1.19.5.3 Using motorized lenses with mvBlueCOUGAR-XD . . . 373
1.19.6 Working with HDR (High Dynamic Range Control) . . . 378
1.19.6.1 Adjusting sensor of camera models -x00w . . . 378
1.19.6.2 Adjusting sensor of camera models -x02d (-1012d) . . . 381
1.19.6.3 Adjusting sensor of camera models -x02e (-1013) / -x04e (-1020) . . . 383
1.19.7 Working with LUTs . . . 386
1.19.7.1 Introducing LUTs . . . 386
1.19.7.2 Working with LUTValueAll . . . 390
1.19.7.3 Implementing a hardware-based binarization . . . 392
1.19.8 Saving data on the device . . . 394
1.19.8.1 Creating user data entries . . . 394
1.19.8.2 Creating user set entries . . . 396
1.19.8.3 Working with the UserFile section (Flash memory) . . . 400
1.19.9 Working with device features . . . 404
1.19.9.1 Working with the temperature sensors . . . 404
1.19.9.2 Disabling the heartbeat . . . 406
1.19.9.3 Reset timestamp by hardware . . . 407
1.19.9.4 Synchronizing camera timestamps . . . 408
1.19.9.5 Working with the 3 head model . . . 413
1.19.9.6 Working With The Serial Interface (mv Serial Interface Control) . . . 414
1.19.10 Working with several cameras simultaneously . . . 418
1.19.10.1 Using Action Commands . . . 418
1.19.10.2 Creating synchronized acquisitions using timers . . . 424
1.19.10.3 Using the primary application switchover functionality . . . 430
1.19.11 Working with 3rd party tools . . . 434
1.19.11.1 Using VLC Media Player . . . 434
1.19.11.2 Working with ROS (Robot Operating System) . . . 436
1.19.11.3 Using GigE Vision™ Devices In A Docker Container . . . 439
1.20 Appendix A. Specific Camera / Sensor Data . . . 442
1.20.1 A.1 Pregius CMOS . . . 442
1.20.1.1 Pregius S . . . 442

1.20.1.2 Pregius . . . 467
1.20.2 A.2 Starvis CMOS . . . 526
1.20.2.1 mvBlueCOUGAR-X106 (6.4 Mpix [3096 x 2080]) . . . 526
1.20.2.2 mvBlueCOUGAR-X1012r (12.4 Mpix [4072 x 3044]) . . . 530
1.20.2.3 mvBlueCOUGAR-X1020 (20.5 Mpix [5544 x 3692]) . . . 533
1.20.3 A.3 Polarsens CMOS . . . 537
1.20.3.1 mvBlueCOUGAR-X105p (5.1 Mpix [2464 x 2056]) . . . 537
1.20.4 A.4 CMOS . . . 541
1.20.4.1 mvBlueCOUGAR-X100w (0.4 Mpix [752 x 480]) . . . 542
1.20.4.2 mvBlueCOUGAR-X102b (1.2 Mpix [1280 x 960]) . . . 546
1.20.4.3 mvBlueCOUGAR-X102d (1.2 Mpix [1280 x 960]) . . . 549
1.20.4.4 mvBlueCOUGAR-X102e (1.3 Mpix [1280 x 1024]) . . . 554
1.20.4.5 mvBlueCOUGAR-X102eGE (1.3 Mpix [1280 x 1024]) . . . 560
1.20.4.6 mvBlueCOUGAR-X104e (1.9 Mpix [1600 x 1200]) . . . 567
1.20.4.7 mvBlueCOUGAR-X[D]x04 (2.2 Mpix [2048 x 1088]) . . . 573
1.20.4.8 mvBlueCOUGAR-X[D]x04b (4.2 Mpix [2048 x 2048]) . . . 577
1.20.4.9 mvBlueCOUGAR-X105 (5 Mpix [2592 x 1944]) . . . 581
1.20.4.10 mvBlueCOUGAR-X1010 (10 Mpix [3856 x 2764]) . . . 586
1.20.5 A.5 CCD . . . 592
1.20.5.1 mvBlueCOUGAR-X120a (0.3 Mpix [640 x 480]) . . . 593
1.20.5.2 mvBlueCOUGAR-X120b (0.3 Mpix [640 x 480]) . . . 599
1.20.5.3 mvBlueCOUGAR-X120d (0.5 Mpix [776 x 580]) . . . 605
1.20.5.4 mvBlueCOUGAR-X122 (1.2 Mpix [1280 x 960]) . . . 611
1.20.5.5 mvBlueCOUGAR-X123 (1.4 Mpix [1360 x 1024]) . . . 617
1.20.5.6 mvBlueCOUGAR-X124 (1.9 Mpix [1600 x 1200]) . . . 624
1.20.5.7 mvBlueCOUGAR-XD124a (3 Mpix [1936 x 1460]) . . . 630
1.20.5.8 mvBlueCOUGAR-X225 (5.1 Mpix [2448 x 2050]) . . . 637
1.20.5.9 mvBlueCOUGAR-X125a (5.1 Mpix [2448 x 2050]) . . . 642
1.20.5.10 mvBlueCOUGAR-XD126 (6 Mpix [2752 x 2208]) . . . 649
1.20.5.11 mvBlueCOUGAR-XD126a (6 Mpix [2752 x 2208]) . . . 655
1.20.5.12 mvBlueCOUGAR-XD129 (9.2 Mpix [3384 x 2712]) . . . 661
1.20.5.13 mvBlueCOUGAR-XD129a (9.2 Mpix [3384 x 2712]) . . . 667
1.20.5.14 mvBlueCOUGAR-XD1212a (12.1 Mpix [4248 x 2836]) . . . 673
1.21 Appendix C. Tested ARM platforms . . . 679
1.21.1 C.1 ARM64 based devices . . . 681
1.21.1.1 NVIDIA Jetson AGX Xavier . . . 681
1.21.1.2 NVIDIA Jetson Xavier NX . . . 683
1.21.1.3 NVIDIA Jetson Nano . . . 685
1.21.1.4 NVIDIA Jetson TX2 . . . 687
1.21.1.5 i.MX8M Mini . . . 689
1.21.2 C.2 ARMhf based devices . . . 692
1.21.2.1 Raspberry Pi 4 . . . 692

1.1 About this manual

1.1.1 Goal of the manual

This manual gives you an overview of the mvBlueCOUGAR-X/-XD, MATRIX VISION's compact Gigabit Ethernet industrial camera family compliant to GigE Vision, its technical data and basic operation of the mvBlueCOUGAR-X/-XD. Programming the device is detailed in a separate documentation, which will be available in an online format.

1.1.2 Contents of the manual

At the beginning of the manual, you will get an **introduction** (p. 18) to the possible usages of the camera. The following chapters contain general information about the camera including:

- **Quickstart** (p. 27) followed by
- **Technical Data** (p. 82)
- **Sensor Overview** (p. 116)
- **Filters** (p. 136)

The general information is followed by the description of the

- **Software tools for the camera** (p. 139) including the tools
- **GenICam and advanced features** (p. 140) introduces the GenICam and the advanced features of the camera (the cameras are GenICam compliant devices).
- **Developing applications using the mvIMPACT Acquire SDK** (p. 177)
- **DirectShow developers** (p. 178) documents MATRIX VISION's mvIMPACT Acquire to DirectShow interface(DirectShow_acquire).
- **Troubleshooting** (p. 183) shows how to detect damages and other inconveniences.
- **Use Cases** (p. 222) describes solutions for general tasks and
- A **Glossary** (p. 213) explains abbreviations and technical terms.
  - **Appendix A. Specific Camera / Sensor Data** (p. 442) contains all data of the sensors like timings, details of operation, etc.
  - **A.1 Pregius CMOS** (p. 442) contains all data of the Pregius CMOS sensors like timings, details of operation, etc.
  - **A.2 Starvis CMOS** (p. 526) contains all data of the Starvis CMOS sensors like timings, details of operation, etc.
  - **A.3 Polarsens CMOS** (p. 537) contains all data of the Polarsens CMOS sensors like timings, details of operation, etc.
  - **A.4 CMOS** (p. 541) contains all data of the other CMOS sensors like timings, details of operation, etc.
  - **A.5 CCD** (p. 592) contains all data of the other CCD sensors like timings, details of operation, etc.
- **Appendix C. Tested ARM platforms** (p. 679) contains a list of ARM platforms tested with this product and information on how to setup these systems for achieving optimal results

1.2 Migration Guide

1.2.1 Introduction

Since

Firmware 2.27.0 for mvBlueCOUGAR-X(D) devices (released 9th of February 2018)

In general all firmware versions are meant to operate on every device. Whenever this is not the case this page will provide some guidelines on what to do or what to consider. Apart from that all known compatibility issues are listed here.

1.2.2 Hardware Revision 5

Hardware revision 5 for mvBlueCOUGAR-X(D) devices are shipped with a different network interface chip since the chip used before is no longer available. As a consequence devices with this hardware revision cannot be operated with previous firmware releases:

Device version / Hardware revision	Supported firmware versions
$<$ 5.0	All
5.0	$>$= 2.27.0 (2.28.0 and greater is recommended)

Attention

"Risk of equipment damage"

Downgrading to a firmware version smaller than the versions listed in the table above may result in device malfunction!

To verify the compatibility of you application with new firmware / mvIMPACT Acquire versions you can use cameras with smaller hardware revisions. Just update them to firmware version 2.28.0 or greater. If your applications runs without any problems afterwards you can use the new hardware revisions without any problems as well.

For a detailed migration guide for your application please refer to `https://www.matrix-vision.com/manuals/SDK_CPP/MigrationGuide.html`

1.3 Imprint

MATRIX VISION GmbH
Talstrasse 16
DE - 71570 Oppenweiler
Telephone: +49-7191-9432-0
Fax: +49-7191-9432-288
Website: http://www.matrix-vision.de
E-Mail:

- info@matrix-vision.de
- jobs@matrix-vision.de

Author

U. Lansche

Date

2020

This document assumes a general knowledge of PCs and programming.

Since the documentation is published electronically, an updated version may be available online. For this reason we recommend checking for updates on the MATRIX VISION website.

MATRIX VISION cannot guarantee that the data is free of errors or is accurate and complete and, therefore, assumes no liability for loss or damage of any kind incurred directly or indirectly through the use of the information of this document.

MATRIX VISION reserves the right to change technical data and design and specifications of the described products at any time without notice.

Copyright

MATRIX VISION GmbH. All rights reserved. The text, images and graphical content are protected by copyright and other laws which protect intellectual property. It is not permitted to copy or modify them for trade use or transfer. They may not be used on websites.

- Windows® XP, Windows® Vista, Windows® 7, 8, 10 are trademarks of Microsoft, Corp.
- Linux® is a trademark of Linus Torvalds.
- Jetson is a registered trademark of NVIDIA Corporation.
- NVIDIA and Jetson are trademarks and/or registered trademarks of NVIDIA Corporation in the U.S. and other countries.
- Arm and Cortex are registered trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere.
- GenICam™ is a trademark of the GenICam™ standard group.

- GigE Vision™ and the distinctive logo are trademarks owned by the Automated Imaging Association and may only be used under license for compliant products registered with the AIA.

  All other product and company names in this document may be the trademarks and tradenames of their respective owners and are hereby acknowledged.

  The manual has been generated with Doxygen (Website: `http://www.doxygen.org`).

  Parts of the log file creation and the log file display make use of Sarissa (Website: `http://dev.↩abiss.gr/sarissa`) which is distributed under the GNU GPL version 2 or higher, GNU LGPL version 2.1 or higher and Apache Software License 2.0 or higher. The Apache Software License 2.0 is part of this driver package.

1.4 Legal notice

1.4.1 Introduction

The firmware running on MATRIX VISION devices make use of a couple of third party software packages that come with various licenses. This section is meant to list all these packages and to give credit to those whose code helped in the creation of this software:

1.4.2 cJSON

A slightly modified version of cJSON is used inside some of the modules that eventually build up the firmware.


```
Copyright (c) 2009 Dave Gamble

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
THE SOFTWARE.
```


1.4.3 Unity

A slightly modified version of Unity (`https://github.com/ThrowTheSwitch/Unity`) is used for unit testing various modules that eventually build up the firmware.

1.5 Revisions

Date	Rev.	Author	Description	Driver / Firmware version
27. June 2022	V6.07	LAN	Added **mvBlueCOUGAR-X106 (6.4 Mpix [3096 x 2080])** (p. 526) .	
31. May 2022	V6.06	LAN	Updated **mvBlueCOUGAR-X1012r (12.4 Mpix [4072 x 3044])** (p. 530) and **mvBlueCOUGAR-X108u (8.1 Mpix [2848 x 2848])** (p. 449) .	
17. May 2022	V6.05	LAN	Updated **mvBlueCOUGAR-X102k (1.8 Mpix [1600 x 1104])** (p. 475) and **mvBlueCOUGAR-X102m (1.8 Mpix [1600 x 1104])** (p. 478) .	
17. September 2021	V6.04	LAN	New control **mv Auto Feature Control** (p. 169) .	FW Revision 2.40.2546.0
24. August 2021	V6.03	LAN	Updated t_{ON} of digital input delay in • **mvBlueCOUGAR-X / -POE** (p. 82) • **mvBlueCOUGAR-XD** (p. 99) .	
19. August 2021	V6.02	LAN	Updated power consumption information.	
01. July 2021	V6.01	LAN	Added use case **Low latency triggering** (p. 365).	
03. May 2021	V6.00	LAN	Corrected **Symbols and Conventions** (p. 7).	
24. March 2021	V5.03	LAN	Added use case section **Improving the communication** (p. 343).	
08. March 2021	V5.02	LAN	Added sensors **mvBlueCOUGAR-X[D]105d (5.1 Mpix [2472 x 2064])** (p. 442) **mvBlueCOUGAR-X[D]108a (8.1 Mpix [2856 x 2848])** (p. 446) **mvBlueCOUGAR-X[D]1012d (12.4 Mpix [4128 x 3008])** (p. 452).	
01. February 2021	V5.01	LAN	Added **No GigE Vision™ devices are detected on a Linux system** (p. 212).	
13. January 2021	V5.00	LAN	Separated **GUI tools** (p. 139).	

1.6 Symbols and Conventions

Note

This symbol indicates general notes.

1.6.1 Explanation of the warnings

Always observe the warnings in these instructions and the measures described to avoid hazards. The warnings used here contain various signal words and are structured as follows:

Attention

SIGNAL WORD
"Type and source of the hazard"

Consequences if not complied with

→ Measures to avoid hazards.

The individual signal words mean:

Attention

Indicates a **danger** that can lead to **damage** or **destruction** of the product.

All due care and attention has been taken in preparing this manual. In view of our policy of continuous product improvement, however, we can accept no liability for completeness and correctness of the information contained in this manual. We make every effort to provide you with a flawless product.

In the context of the applicable statutory regulations, we shall accept no liability for direct damage, indirect damage or third-party damage resulting from the acquisition or operation of a MATRIX VISION product. Our liability for intent and gross negligence is unaffected. In any case, the extend of our liability shall be limited to the purchase price.

1.7 Important Information



1.7.1 Important Safety Instructions

- We cannot and do not take any responsibility for the damage caused to you or to any other equipment connected to a MATRIX VISION device. Similarly, warranty will be void, if a damage is caused by not following the manual.
- Handle your MATRIX VISION device with care. Do not misuse it. Avoid shaking, striking, etc.. Your MATRIX VISION device could be damaged by faulty handling or shortage.
- Do not use accessories not recommended by the product manufacturer as they may cause hazards.
- The product should be situated away from heat sources such as radiators, heat registers, stoves, or other products (including amplifiers) that produce heat.
- Using the board-level version:
  - Provide sufficient cooling because single components can reach high temperatures.
  - Handle with care and avoid damage of electrical components by electrostatic discharge (ESD):
    - Discharge body static (contact a grounded surface and maintain contact).
    - Avoid all plastic, vinyl, and Styrofoam (except anti-static versions) around printed circuit boards.
    - Do not touch components on the printed circuit board with your hands or with conductive devices.

1.7.2 Operating considerations

1.7.2.1 Important Safety Notes

- Use this camera with a suitable power supply.
- Using the **Model without housing (-xx9x) (on demand)** (p. 82) :
  - Provide sufficient cooling because single components can reach high temperatures. Inadequate and incorrect cooling invalidate the guarantee.
- **For extended** temperature (-ET) **models:**

  Attention

  – **"Overheating"**

  The mainboards temperature may not exceed 65°C otherwise the device can be damaged.
- Observe that flammable objects, water or metal do not enter the camera interior. These may lead to failure or accident.
- Do not modify the camera or use the camera with external covers removed. These may cause failure, void any warranties and pose a safety hazard.
- Stop using the camera at the approach of electrical storm (thunder audible). Protect the camera from rain if using it outdoors.
- In the event of abnormal functioning, switch off the camera and disconnect the power cord. Contact MATRIX VISION.

1.7.2.2 Handling And Cleaning

- Do not attempt to disassemble camera.
- When installing or removing a lens, take care that water or dust does not enter the inside of the camera.

1.7.2.3 Installing

Avoid installing or storing the camera in the following environments:

- Environments exposed to direct sunlight, rain or snow.
- Environments where combustible or corrosive gas exists.
- Excessively warm or cold environment (Operating ambient temperature: 0 to 45 °C)
- Humid or dusty environment.
- Place subjected to excessive vibration or shock.
- Environment exposed to strong electric or magnetic field.
- It is recommended to mount the camera on a thermoconducting surface such as aluminum or other metals rather than plastic or wood.
- Please contact manufacturer or local distributor if you want to use additional enclosures for higher ingress protection.
- Do not aim the camera at the sun or other very strong light sources, otherwise you can destroy the image sensor.

"CCD phenomenon: Smearing": Vertical smearing may appear, in the image. However, this is not due to failure of the camera.

1.7.2.4 Optimizing performance and life time

When the camera is used continuously for long time under high ambient temperature, the inside electrical parts may deteriorate, resulting in shorter life span. Additional cooling by e.g. air convection is recommended.

1.7.2.5 Connectors

Confirm the power is off before connecting or disconnecting a signal cable. Grap connectors by the body, not the attached wires.

1.7.2.6 Cleaning

- Use a blower or a lens brush to remove dust on the lens or the optical filter.
- Do not disassemble front flange.
- Clean case with dry soft cloth. Use neutral detergent liquid if needed; wipe the cover with dry cloth.
- Do not use benzene, thinner, alcohol, liquid cleaner or spray-type cleaner.
- If dust or other debris is located between the CCD and optical filter, consult dealer for cleaning by an optical technician.

1.7.2.6.1 Adjusting the C-mount The mvBlueCOUGAR-X cameras allow a precise adjustment of the back focus of the C-mount by means of a back focus ring which is threaded into the C-mount and is secured by a lock nut ring which itself is secured by two screws. The mechanical adjustment of the imaging device is important in order to achieve a perfect alignment with the focal point of the lens. This adjustment is made before leaving the factory to conform to the standard of 17.526 mm (in air) and should normally not require adjustment in the field. However, if the back focal plane of your lens does not conform to the C-mount back focus specification or if you have e.g. removed the **IR-CUT filter** (p. 136), renewed adjustment may be required.

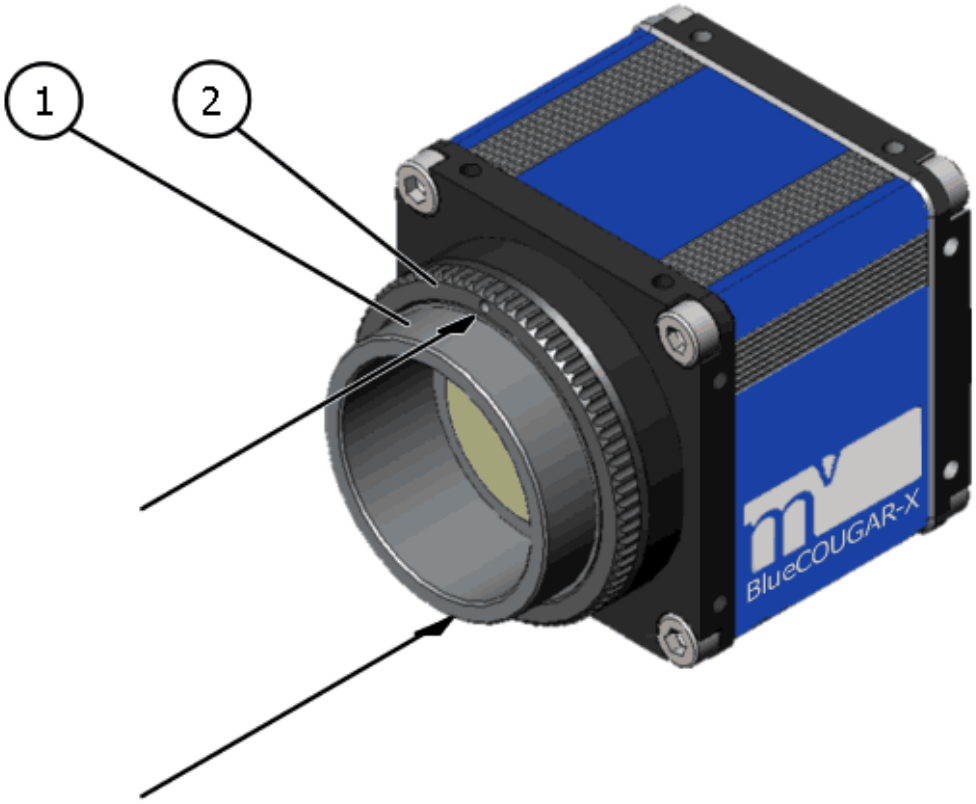


Figure 1: Lensholder with C-mount ring (1) and lock nut ring (2)

How to proceed:

- Loosen screws (location as shown above by arrows) of the lock nut ring with an Allen key (0.9 x 50).
- Loosen the lock nut ring.
- With the lens set to infinity or a known focus distance, set the camera to view an object located at "infinity" or the known distance.
- Rotate the C-mount ring and lens forward or backwards on its thread until the object is in sharp focus.

  Note

  Be careful that the lens remains seated in the C-mount.

- Once focus is achieved, tighten the lock nut ring, then tighten the two locking screws of the lock ring without applying excessive torque.

1.7.2.7 Additional temperature and power consumption notes concerning mvBlueCOUGAR-XD and mvBlueCOUGAR-X2xx

mvBlueCOUGAR-XD and mvBlueCOUGAR-X2xx cameras have very different power consumptions depending on sensor technology and connection and operation modes. As a rule of thumb:

- CMOS sensor camera with single network interface controller (NIC) output consumes less around 4W
- CCD equipped cameras with quad tap use and dual NIC output and max. pixel clock may consume up to 9W
- CCD sensor dual tap mode consumes 1.7W less than quad tap
- For mvBlueCOUGAR-XD: Dual NIC mode (so-called **link aggregation [LAG] or bonding** (p. 216)) consumes 0.8W more than single NIC used

Data sheets of cameras commonly specify the allowed temperature range by indicating the min/max. ambient temperature of the air (e.g in a room) where the camera is installed and operated. Whereas this is good for a first indication and comparison it presents a fuzziness for the temperature of the components inside because it is not defined how the camera is mounted and thus the overtemperature inside the camera is not clear.

And since each installation is different, MATRIX VISION cannot supply a strictly required procedure for proper heat dissipation. Instead, we provide the following general guidelines:

In all cases,

1. you should monitor the temperature of the camera by using the two built-in temperature sensors (separate for sensor board and mainboard) and
2. make sure that neither the sensor temperature nor the mainboard temperature exceed 75 °C.

Keep in mind that the camera will gradually heat up during the first hour of operation. After one hour, the housing temperature should have stabilized and increases no longer.

- If your camera is mounted on a substantial metal holder or component in your system, this may already provide sufficient heat dissipation because this metal component presents a low thermal resistance for the heat path.
- You may use additional heat conductors between the camera and the holder to bridge gaps.
- The use of a fan to provide air flow over the camera is an extremely efficient method of heat dissipation.

The fan can be driven directly by one of the robust camera outputs and controlled by the camera itself (temp. settings to switch fan on and off with hysteresis) as long as

- the switched voltage at the output is suitable for the used fan (12V up to 24V) and that
- the fan does not consume more than 700 mA which is the maximum one output of the camera can deliver.

Available CPU Socket 7 (370 or socket A) active cooler fit very well directly on any side of the camera by means of thermal glue. Ask for support if you need further assistance.

1.7.3 Conformity

CE	The mvBlueCOUGAR-X is in conformity with all applicable essential requirements necessary for CE marking. It corresponds to the EU EMC guideline 2014/30/EU based on the following harmonized standards Electromagnetic compatibility (EMC) • Interference emmision EN 61000-6-3 / 2007 + A1:2011 • Interference immunity EN 61000-6-2 / 2005 EN 61000-6-3 / 2007 requires an I/O cable with an retrofittable ferrite to be used such as – Company: Würth Elektronik Type: WE No. 742 711 31 • Restriction of certain Hazardous Substances EN 50581:2012
UK CA	
	MATRIX VISION corresponds to the EU guideline WEEE 2002/96/EG on waste electrical and electronic equipment and is registered under WEEE-Reg.-No. DE 25244305.
RoHS	All units delivered are RoHS compliant.
IP30 [1]	mvBlueCOUGAR-XD
IP40 [1]	mvBlueCOUGAR-X / **mvBlueCOUGAR-X -POE** (p. 20) / **mvBlueCOUGAR-X -POE-I** (p. 20)
IP67 [1]	**mvBlueCOUGAR-X -POE-IP67** (p. 20) / **mvBlueCOUGAR-X -POE-IP67C** (p. 20)
PoE 802.3at (p. 220)	The option **-POE** (p. 20) is compliant to **IEEE** 802.3af (IEEE 802.3at Type 1).

[1] not evaluated by UL

A brand of Balluff

MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueCOUGAR-X (BVS CA-GX0)**
Type: ***mvBlueCOUGAR-X (BVS CA-GX0)***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-3:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2019

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of high-quality, shielded connecting cables is required.

A brand of Balluff

MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueCOUGAR-X-POE (BVS CA-GX0)**
Type: ***mvBlueCOUGAR-X-POE (BVS CA-GX0)***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-4:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2005

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of high-quality, shielded connecting cables is required.

A brand of Balluff

MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueCOUGAR-X-POE-I/IP (BVS CA-GX0)**
Type: ***mvBlueCOUGAR-X-POE-I/IP (BVS CA-GX0)***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-4:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2005

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of high-quality, shielded connecting cables is required.

A brand of Balluff

MATRIX VISION

EG-Konformitätserklärung
Declaration of conformity

Der Hersteller
The Manufacturer

MATRIX VISION GmbH
Talstraße 16
71570 Oppenweiler
Germany

erklärt hiermit, dass sein Produkt
herewith declares, that his product

Typbezeichnung: **mvBlueCOUGAR-XD (BVS CA-GX2)**
Type: ***mvBlueCOUGAR-XD (BVS CA-GX2)***

mit den Bestimmungen folgender Europäischer Richtlinien übereinstimmt:
complies with the provisions of the following European Directives:

2014/30/EU EMC
2011/65/EU RoHS II

auf Grundlage folgender harmonisierter Normen:
based on the following harmonized standards:

Elektromagnetische Verträglichkeit (EMV) / *Electromagnetic compatibility (EMC)*
Störaussendung / *Interference emission*
EN 61000-6-3:2007 + A1:2011

Störfestigkeit / *Interference immunity*
EN 61000-6-2:2005

Beschränkung gefährlicher Stoffe / Restriction of certain Hazardous Substances
EN IEC 63000:2018

Oppenweiler, 10.06.2021

Uwe Hagmaier
Vice President R & D

Bitte beachten: Um die Einhaltung obiger Normen sicherzustellen, ist die Verwendung hochwertiger, vollständig geschirmter Anschlusskabel unbedingt erforderlich.
Please note: In order to fulfill the above standards, the use of high-quality, shielded connecting cables is required.

1.7.4 Additional notices

1.7.4.1 For customers in the U.S.A.

FC	Class B Class A (mvBlueCOUGAR-X **PoE** (p. 220) and POE-I version)

This equipment has been tested and found to comply with the limits for a Class B digital device, pursuant to Part 15 of the FCC Rules. These limits are designed to provide reasonable protection against harmful interference when the equipment is operated in a residential environment. This equipment generates, uses, and can radiate radio frequency energy and, if not installed and used in accordance with the instruction manual, may cause harmful interference to radio communications. However there is no guarantee that interferences will not occur in a particular installation. If the equipment does cause harmful interference to radio or television reception, the user is encouraged to try to correct the interference by one or more of the following measures:

- Reorient or relocate the receiving antenna.
- Increase the distance between the equipment and the receiver.
- Use a different line outlet for the receiver.
- Consult a radio or TV technician for help.

You are cautioned that any changes or modifications not expressly approved in this manual could void your authority to operate this equipment. The shielded interface cable recommended in this manual must be used with this equipment in order to comply with the limits for a computing device pursuant to Subpart B of Part 15 of FCC Rules.

- FCC Class B requires an I/O cable with an retrofittable ferrite to be used such as
  - Company: Würth Elektronik Type: WE No. 742 711 31

1.7.4.2 For customers in Canada

This apparatus complies with the Class B limits for radio noise emissions set out in the Radio Interference Regulations.

1.7.4.3 Pour utilisateurs au Canada

Cet appareil est conforme aux normes classe B pour bruits radioélectriques, spécifiées dans le Règlement sur le brouillage radioélectrique.

1.8 Introduction



Figure 1: mvBlueCOUGAR-X

The mvBlueCOUGAR is an extreme compact and flexible industrial Gigabit Ethernet camera family compliant to **GigE Vision** (p. 215) and **GenICam** (p. 213). The mvBlueCOUGAR offers

- a wide range of sensors,
- high frame rates over long distances using inexpensive standard network parts,
- I/Os suitable for industrial applications, and
- wide range of resolutions.

The image memory of the camera enables a high-speed-buffer mode (a.k.a. burst mode) which writes images faster in the camera's memory as they are transferred. With this mode, image losses are a thing of the past.

The mvBlueCOUGAR is ideally suited for all classical areas of machine vision and allows an easy integration in the environment and support the application with helpful features to deliver faster and better results. The hardware capabilities can be teamed with machine vision libraries which are compliant to GigE Vision.

The Dual-GigE version named mvBlueCOUGAR-XD is also available.

1.8.1 Software concept

The mvBlueCOUGAR is a **GigE Vision** (p. 215) compliant device, using a **GenICam** (p. 213) XML-file describing the device capabilities. Within this XML-file it uses the names, units, data types etc. recommended by the **SFNC** (p. 221) to describe the devices features. Whenever custom features have been added to the mvBlueCOUGAR firmware these can clearly be spotted by the leading 'mv' in the features name. The device can communicate with every third party compliant **GigE Vision** (p. 215) capture driver that can interpret **GenICam** (p. 213) XML-files.

The following figure shows the software concept of MATRIX VISION's camera devices:

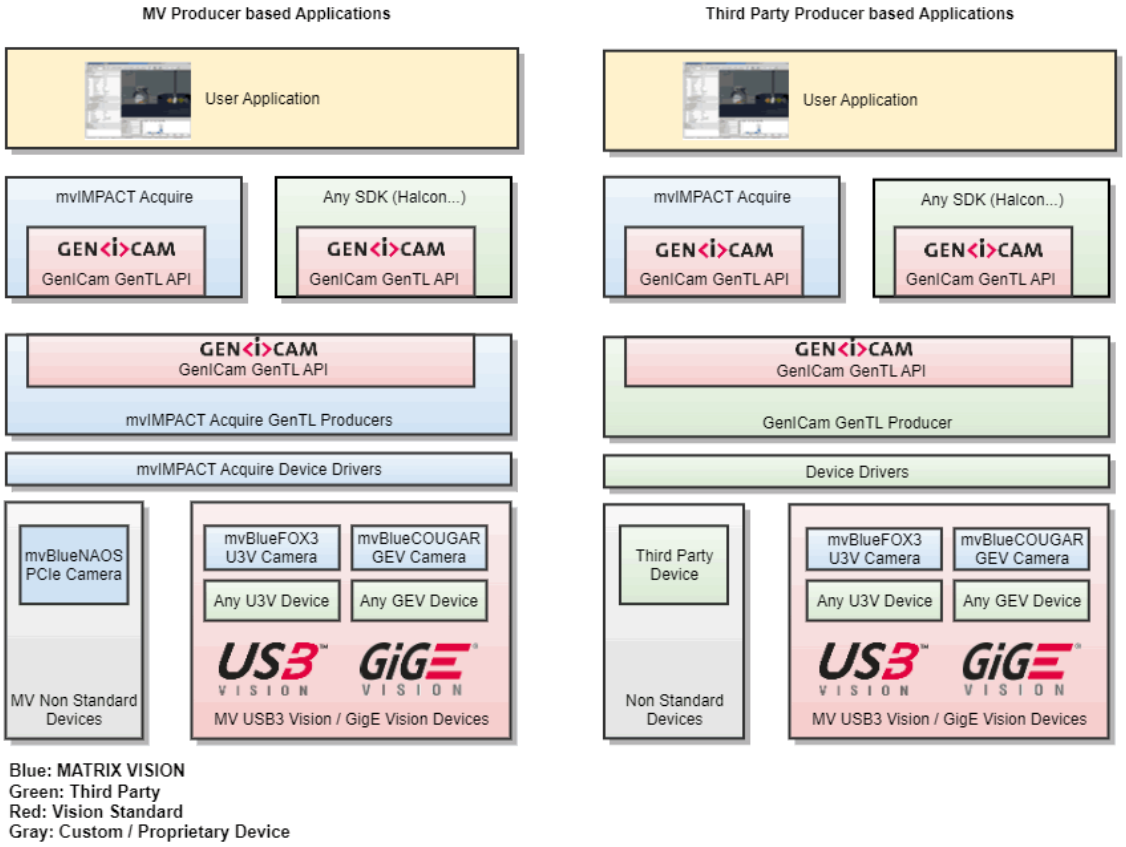


Figure 2: Software concept

As shown in figure 2, for the mvBlueCOUGAR the **mvIMPACT_Acquire** (p. 219) interface is stacked on the **GigE Vision** (p. 215) and **Genicam** (p. 213) layers. The **mvIMPACT_Acquire** (p. 219) interface internally uses the **GenICam** (p. 213) runtime libraries, so that it can be considered as a user application written with the **GenICam** (p. 213) interface.

1.8.2 Order code nomenclature

1.8.2.1 mvBlueCOUGAR-X

The mvBlueCOUGAR-X nomenclature scheme is as follows:

Sensor	Balluff model name	Model name
0.3 Mpix, 640 x 480, 1/3", ICX424, CCD	BVS CA-GX0-0003AG/C	mvBlueCOUGAR-X120aG/C
0.3 Mpix, 640 x 480, 1/3", ICX414, CCD	BVS CA-GX0-0003ZG/C	mvBlueCOUGAR-X120bG/C
0.4 Mpix, 728 x 544, 1/2.9", IMX287, CMOS	BVS CA-GX0-0004FG/C	mvBlueCOUGAR-X100fG/C
0.4 Mpix, 752 x 480, 1/3", MT9V034, CMOS	BVS CA-GX0-0004ZG/C	mvBlueCOUGAR-X100wG/C
0.5 Mpix, 776 x 580, 1/2", ICX415, CCD	BVS CA-GX0-0004BG/C	mvBlueCOUGAR-X120dG/C
1.2 Mpix, 1280 x 960, 1/3", MT9M021, CMOS	BVS CA-GX0-0012BG/C	mvBlueCOUGAR-X102bG/C
1.2 Mpix, 1280 x 960, 1/3", MT9M034, CMOS	BVS CA-GX0-0012DG/C	mvBlueCOUGAR-X102dG/C
1.2 Mpix, 1280 x 960, 1/3", ICX445, CCD	BVS CA-GX0-0012EG/C	mvBlueCOUGAR-X122G/C
1.3 Mpix, 1280 x 1024, 1/1.8", EV76C560, CMOS	BVS CA-GX0-0013ZG/C	mvBlueCOUGAR-X102eG/C
1.6 Mpix, 1456 x 1088, 1/2.9", IMX273, CMOS	BVS CA-GX0-0016ZG/C	mvBlueCOUGAR-X102fG/C
1.7 Mpix, 1600 x 1104, 1.1", IMX425, CMOS	BVS CA-GX0-0017ZG/C	mvBlueCOUGAR-X102mG/C
1.7 Mpix, 1600 x 1104, 1.1", IMX432, CMOS	BVS CA-GX0-0017BG/C	mvBlueCOUGAR-X102kG/C
2.0 Mpix, 1632 x 1248, 1/1.2", IMX430, CMOS	BVS CA-GX0-0020DG/C	mvBlueCOUGAR-X102nG/C
1.4 Mpix, 1360 x 1024, 1/2", ICX267, CCD	BVS CA-GX0-0014ZG/C	mvBlueCOUGAR-X123G/C
1.9 Mpix, 1600 x 1200, 1/1.8", ICX274, CCD	BVS CA-GX0-0020BG/C	mvBlueCOUGAR-X124G/C
1.9 Mpix, 1600 x 1200, 1/1.8", EV76C570, CMOS	BVS CA-GX0-0020ZG/C	mvBlueCOUGAR-X104eG/C
2.4 Mpix, 1936 x 1216, 1/1.2", IMX249, CMOS	BVS CA-GX0-0024AG/C	mvBlueCOUGAR-X104fG/C
3.2 Mpix, 2064 x 1544, 1/1.8", IMX265, CMOS	BVS CA-GX0-0032AG/C	mvBlueCOUGAR-X104iG/C
4.2 Mpix, 2048 x 2048, 1", CMV4000, CMOS	BVS CA-GX0-0042ZG/C	mvBlueCOUGAR-X104bG/C
5.0 Mpix, 2592 x 1944, 1/2.5", MT9P031, CMOS	BVS CA-GX0-0050ZG/C	mvBlueCOUGAR-X105G/C
5.1 Mpix, 2464 x 2056, 2/3", IMX264, CMOS	BVS CA-GX0-0051AG/C	mvBlueCOUGAR-X105bG/C
5.1 Mpix, 2472 x 2064, 1/1.8", IMX547, CMOS	BVS CA-GX0-0051DG/C	mvBlueCOUGAR-X105dG/C
5.1 Mpix, 2464 x 2056, 2/3", IMX250_POL, CMOS	BVS CA-GX0-0051PG/C	mvBlueCOUGAR-X105pG/C
5.1 Mpix, 2448 x 2050, 2/3", ICX655, CCD	BVS CA-GX0-0050BG/C	mvBlueCOUGAR-X125aG/C
5.1 Mpix, 2448 x 2050, 2/3", ICX625, CCD	BVS CA-GX1-0050AG/C	mvBlueCOUGAR-X225G/C
6.4 Mpix, 3096 x 2080, 1/1.8", IMX178, CMOS	BVS CA-GX0-0064ZG/C	mvBlueCOUGAR-X106G/C
7.1 Mpix, 3216 x 2208, 1.1", IMX428, CMOS	BVS CA-GX0-0071AG/C	mvBlueCOUGAR-X107bG/C
8.1 Mpix, 2856 x 2848, 2/3", IMX546, CMOS	BVS CA-GX0-0081AG/C	mvBlueCOUGAR-X108aG/C
8.1 Mpix, 2848 x 2848, 2/3", IMX487, CMOS	BVS CA-GX0-0081UG/C	mvBlueCOUGAR-X108uG/C
8.9 Mpix, 4112 x 2176, 1", IMX267, CMOS	BVS CA-GX0-0089AG/C	mvBlueCOUGAR-X109bG/C
10.7 Mpix, 3856 x 2764, 1/2.35", MT9J003, CMOS	BVS CA-GX0-0107ZG/C	mvBlueCOUGAR-X1010G/C
12.4 Mpix, 4112 x 3008, 1.1", IMX304, CMOS	BVS CA-GX0-0124AG/C	mvBlueCOUGAR-X1012bG/C
12.4 Mpix, 4128 x 3008, 1/1.1", IMX545, CMOS	BVS CA-GX0-0124DG/C	mvBlueCOUGAR-X1012dG/C
12.4 Mpix, 4072 x 3044, 1/1.7", IMX226, CMOS	BVS CA-GX0-0124RG/C	mvBlueCOUGAR-X1012rG/C
20.5 Mpix, 5544 x 3692, 1.1", IMX183, CMOS	BVS CA-GX0-0205ZG/C	mvBlueCOUGAR-X1020G/C

	Series	Sensor	Color		Options		HW Variant		CUP/STD
Balluff model name	BVS CA-↩ GX0-	xxxxx	G/C			-	see legend below	-	see legend below
Model name	mvBlue↩ COUGAR-X	xxx	G/C/GE	-	see legend below	-	see legend below	-	see legend below

Legend	Balluff model name	Model name
Options		**POE:** Power over Ethernet, PoE class 2 **PLC:** Inputs for PLC levels only **POE-I**: Power over Ethernet, PoE class 2 with a modular M12 industrial connector concept **POE-IP67**: Power over Ethernet, PoE class 2 with a modular M12 industrial connector concept and IP67 protection class **POE-IP67C**: Compact IP67 version
HW Variant	(1)(2)(3)(4)(5)(6) (1): Handling **1**: Standard handling **2**: Extended temperature range test (2): Lensholder **0**: None **1**: C-mount Type 2 with high durability and factory-set backfocus **2**: C-mount Type 1 with adjustable backfocus (standard) **4**: CS-mount Type 1 with adjustable backfocus **A:** C-Mount Type 2 + center position adjustable (3): Filter **0**: None **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut (only without backfocus adjustment) (4): Housing **0**: None **1**: Color blue (standard) **2**: Color black, no logo, no label MATRIX VISION **4**: IP67 enclosure for POE-I + lens, black anodized aluminium **6**: IP67 enclosure for POE-I + lens, stainless steel **C:** IP67 grade housing, blue with logo (available for POE-I) (5): I/O **2**: Default I/Os (standard) **3**: Power over Ethernet and isolated inputs / outputs **4**: Power over Ethernet with M12 industrial grade I/O connectors **6**: PLC logic level digital inputs (6): Software Adjustments **0**: None	(1)(2)(3)(4) (1): Lensholder **9**: None **6**: C-mount Type 2 with high durability and factory-set backfocus **1**: C-mount Type 1 with adjustable backfocus (standard) **2**: CS-mount Type 1 with adjustable backfocus **A:** C-Mount Type 2 + center position adjustable (2): Filter **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut (only without backfocus adjustment) **9**: None (3): Housing **1**: Color blue (standard) **2**: Color black, no logo, no label MATRIX VISION **4**: IP67 enclosure for POE-I + lens, black anodized aluminium **6**: IP67 enclosure for POE-I + lens, stainless steel **9**: None **C:** IP67 grade housing, blue with logo (available for POE-I) (4): I/O **1**: Default I/Os (standard)
CUP/STD	**001**	**ET:** With extended temperature behavior

1.8.2.2 mvBlueCOUGAR-XD

The mvBlueCOUGAR-XD nomenclature scheme is as follows:

Sensor	Balluff model name	Model name
1.6 Mpix, 1456 x 1088, 1/2.9", IMX273, CMOS	BVS CA-GX2-0016ZG/C	mvBlueCOUGAR-XD102fG/C
2.2 Mpix, 2048 x 1088, 2/3", CMV2000, CMOS	BVS CA-GX2-0022ZG/C	mvBlueCOUGAR-XD204G/C
2.4 Mpix, 1936 x 1214, 1/1.2", IMX174, CMOS	BVS CA-GX2-0024ZG/C	mvBlueCOUGAR-XD104dG/C
2.8 Mpix, 1936 x 1460, 2/3", ICX674, CCD	BVS CA-GX2-0028ZG/C	mvBlueCOUGAR-XD124aG/C
3.2 Mpix, 2064 x 1544, 1/1.8", IMX252, CMOS	BVS CA-GX2-0032ZG/C	mvBlueCOUGAR-XD104hG/C
4.2 Mpix, 2048 x 2048, 1", CMV4000, CMOS	BVS CA-GX2-0042ZG/C	mvBlueCOUGAR-XD204bG/C
5.1 Mpix, 2472 x 2064, 1/1.8", IMX547, CMOS	BVS CA-GX2-0051DG/C	mvBlueCOUGAR-XD105dG/C
5.1 Mpix, 2464 x 2056, 2/3", IMX250, CMOS	BVS CA-GX2-0051ZG/C	mvBlueCOUGAR-XD105aG/C
6.1 Mpix, 2752 x 2208, 1", ICX694, CCD	BVS CA-GX2-0060ZG/C	mvBlueCOUGAR-XD126G/C
6.1 Mpix, 2752 x 2208, 1", ICX695, CCD	BVS CA-GX2-0060AG/C	mvBlueCOUGAR-XD126aG/C
7.1 Mpix, 3216 x 2208, 1.1", IMX420, CMOS	BVS CA-GX2-0071ZG/C	mvBlueCOUGAR-XD107G/C
7.1 Mpix, 3216 x 2208, 1.1", IMX428, CMOS	BVS CA-GX2-0071AG/C	mvBlueCOUGAR-XD107bG/C
8.1 Mpix, 2856 x 2848, 2/3", IMX546, CMOS	BVS CA-GX2-0081AG/C	mvBlueCOUGAR-XD108aG/C
8.9 Mpix, 4112 x 2176, 1", IMX267, CMOS	BVS CA-GX2-0089AG/C	mvBlueCOUGAR-XD109bG/C
9.2 Mpix, 3384 x 2712, 1", ICX814, CCD	BVS CA-GX2-0092ZG/C	mvBlueCOUGAR-XD129G/C
9.2 Mpix, 3384 x 2712, 1", ICX815, CCD	BVS CA-GX2-0092AG/C	mvBlueCOUGAR-XD129aG/C
12.1 Mpix, 4250 x 2838, 1", ICX834 _2T, CCD	BVS CA-GX2-0121ZG/C	mvBlueCOUGAR-XD1212aG/C
12.4 Mpix, 4112 x 3008, 1.1", IMX304, CMOS	BVS CA-GX2-0124AG/C	mvBlueCOUGAR-XD1012bG/C
12.4 Mpix, 4128 x 3008, 1/1.1", IMX545, CMOS	BVS CA-GX2-0124DG/C	mvBlueCOUGAR-XD1012dG/C
16.2 Mpix, 5328 x 3040, 1.1", IMX542, CMOS	BVS CA-GX2-0162AG/C	mvBlueCOUGAR-XD1016G/C
20.4 Mpix, 4512 x 4512, 1.1", IMX541, CMOS	BVS CA-GX2-0204AG/C	mvBlueCOUGAR-XD1020aG/C
24.6 Mpix, 5328 x 4608, 1.2", IMX540, CMOS	BVS CA-GX2-0246AG/C	mvBlueCOUGAR-XD1025G/C
31.5 Mpix, 6480 x 4856, APS-C", IMX342 CMOS	BVS CA-GX2-0315ZG/C	mvBlueCOUGAR-XD1031G/C

	Series	Sensor	Color		Options		HW Variant		CUP/STD
Balluff model name	BVS CA-↩ GX2-	xxxxx	G/C			-	see legend below	-	see legend below
Model name	mvBlue↩ COUGAR-↩ XD	xxx	G/C	-	see legend below	-	see legend below	-	see legend below

Legend	Balluff model name	Model name

HW Variant	(1)(2)(3)(4)(5)(6) (1): Handling **1**: Standard handling (2): Lensholder **0**: None **1**: C-Mount, Type2, with high durability and factory-set backfocus, fixed square filter **2**: C-Mount, Type1, with adjustable backfocus, round filter with screw ring, Ø23.2 mm **4**: CS-mount Type1, with adjustable backfocus, round filter glued, Ø20 mm **A:** C-mount Type 2 + center position adjustable **B:** C-Mount Type 2 + D23.2mm Filter with screwring **N:** M42x1 10mm no filter **P:** M42x1 12mm focal distance with filter **Q:** M42 to F-Mount adapter 46mm focal distance with filter **T:** Lensholder with TFL-Mount or Adapter for M42 to TFL-Mount (3): Filter **0**: None **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut (only without backfocus adjustment) (4): Housing **0**: None **1**: Color blue (standard) (5): I/O **1**: Default I/Os (standard) (6): Software Adjustments **0**: None	(1)(2)(3)(4) (1): Lensholder **9**: None **6**: C-Mount, Type2, with high durability and factory-set backfocus, fixed square filter **1**: C-Mount, Type1, with adjustable backfocus, round filter with screw ring, Ø23.2 mm **2**: CS-mount Type1, with adjustable backfocus, round filter glued, Ø20 mm **A:** C-mount Type 2 + center position adjustable **B:** C-Mount Type 2 + D23.2mm Filter with screwring **N:** M42x1 10mm no filter **P:** M42x1 12mm focal distance with filter **Q:** M42 to F-Mount adapter 46mm focal distance with filter **T:** Lensholder with TFL-Mount or Adapter for M42 to TFL-Mount (2): Filter **1**: IR-CUT (standard) **2**: Glass **3**: Daylight cut (only without backfocus adjustment) **9**: None (3): Case **1**: Color blue (standard) **2**: Color black, no logo, no label MATRIX VISION **9**: None (4): Misc **1**: Default I/Os (standard)
CUP/STD	**001**	**ET:** With extended temperature behavior

1.8.2.3 mvBlueCOUGAR-3X

The mvBlueCOUGAR-3X nomenclature scheme is as follows:

```
mvBlueCOUGAR-3X - A B - D - (1) (2) (3) (4)

  - A: Sensor model
       120a: 0.3 Mpix, 640 x 480, 1/3, CCD
       123: 1.4 Mpix, 1360 x 1024, 1/2, CCD

  - B: Sensor color
       G: Gray scale version
       C: Color version

  - D: Options
       POE-I: Power over Ethernet, PoE class 2 with a modular M12 industrial connector concept
```

```
- (1): Lensholder
    1: S-mount (standard)
    2: C-mount
    3: CS-mount

- (2): Filter
    1: IR-CUT (standard)
    2: Glass
    3: Daylight cut (only without backfocus adjustment)
    9: None

- (3): Case
    9: None

- (4): Misc
    1: None (standard)
```

1.8.2.4 Ordering code samples

Balluff model name	Model name	Description
BVS CA-GX0-0020DC-111120	mvBlueCOUGAR-X102nC[1]	1632 x 1248, CMOS 1/1.7", color, Gigabit Ethernet, IR-CUT filter, C-mount, etc.
BVS CA-GX0-0032AG-112130	mvBlueCOUGAR-X104iG-POE-1211	2064 x 1544, CMOS 1/1.8", gray scale, Gigabit Ethernet with Power over Ethernet, C-mount, glass filter, etc.
BVS CA-GX2-0081AC-143120	mvBlueCOUGAR-XD108aC-2111	2856 x 2848, CMOS 2/3", color, Dual-Gigabit Ethernet, CS-mount, IR-CUT filter, etc.

[1]: `-1111` is the standard delivery variant and for this reason it is not mentioned.

1.8.3 What's inside and accessories

Due to the varying fields of application the mvBlueCOUGAR is shipped without accessories. The package contents:

- **mvBlueCOUGAR**

For the first use of the mvBlueCOUGAR we recommend the following accessories to get the camera up and running:

- **network cable** and
- **power supply** via
  - MV-DC1201 BCX.

Note

To be able use the mvBlueCOUGAR-XD it is required that the GigE ports (the two GigE interfaces at the host; either PC or a switch) support the LAG (Link Aggregation Group) functionality. Windows 7 and Linux support this functionality by default. Although the mvBlueCOUGAR-XD is connected via two cables it will act like a single device.

1.8.3.1 Accessories for the mvBlueCOUGAR

Part code	Description
MV-Tripod Adapter	**Tripod adapter for mvBlueCOUGAR-X** (p. 83)
MV-DC1201 BCS	Power supply 12 V for mvBlueCOUGAR-X
MV-DC1201 BCSX IO REV2	Power supply 12 V for mvBlueCOUGAR-X with digital I/O on separate cable length Power 3 m, I/O: 0.5 m
MV-DC2401-M12-IO	Power supply 24 V / 1.5 A to M12 female with dig. I/O on separate cable, length Power 1.85 m, I/O 1 m
MV-DC2430-M12 05.0	Power supply 24 V / 3 A, M12 connector female, length approx. 5 m, no I/O
KS-RJ45-M12-8X	Connecting cable RJ45/M12 8 pin X-coded, Cat 6A, length up to 20 m
KS-GIGE LS SLK x.0	Gigabit Ethernet cable RJ45 to RJ45 with horizontal screw lock 20 mm, M2x0.4, drag chain capable, Cat 6, length up to 20 m
KS-GIGE VLS SLK x.0	Gigabit Ethernet cable RJ45 to RJ45 with vertical screw lock 20mm for mvBlueCOUGAR-XD, M2x0.4, drag chain capable, Cat 6, length up to 20m
KS-GIGE LS SLK AK90L x.0	Gigabit Ethernet cable RJ45 to RJ45 with horizontal screw lock 20 mm, M2x0.4, drag chain capable, Cat 6 Camera plug 90° angled left direction,length up to 20 m
KS-GIGE LS SLK AK90R x.0	Gigabit Ethernet cable RJ45 to RJ45 with horizontal screw lock 20 mm, M2x0.4, drag chain capable, Cat 6 Camera plug 90° angled right direction,length up to 20 m
KS-BCX-HR12	Hirose I/O cable up to 20 m
KS-BCXD-HR12-MZ 00.50	Hirose I/O cable adapter for motor zoom (0.5 m)
KS-M12-PWR-IO	Shielded connection, M12 male connector to prepared wires, UL/CSA, drag chain, length up to 20 m
MV-X I/O-BOX	IO-Box to connect and test a mvBlueCOUGAR-X camera, to loop the camera into an existing system, and to mix both connection variants selectively
LAN KARTE 1000 PCIE REV2	PCI Express extension card with Gigabit Ethernet interface
LAN KARTE 1000 QUAD PCIE REV2	PCI Express extension card with 4 Gigabit Ethernet interfaces
BAM AD-VS-018-M42x0,75-1	Adapter for assembly of BAM PC-VC-023
BAM PC-VS-023-1	Lens protection tube, Ø 45 x 54 mm, Length: 54 mm
BAM PC-VS-023-1-01	Lens protection tube, Ø 45 x 83 mm, Length: 83 mm
BAM PC-VS-023-1-02	Lens protection tube, Ø 45 x 100 mm, Length: 100 mm

1.9 Quickstart

- **System Requirements** (p. 27)
- **Installing The mvGenTL-Acquire Package** (p. 29)
- **Connecting The Camera** (p. 35)
- **Driver concept** (p. 40)
- **Relationship Between Driver, Firmware And SDK** (p. 43)
- **Optimizing Network Performance** (p. 46)
- **Using GigE Vision™ Devices In A Docker Container** (p. 439)

1.9.1 System Requirements

1.9.1.1 Host System

The mvBlueCOUGAR-X / -XD is a high-performance camera which requires a high data throughput on the host system, for example, when processing large amount of image data and high CPU resources if processing color images on the host PC.

For this reason we recommend at least the following components:

Component	Recommendation
Processor	Preferably multi core Intel or ARM CPUs
RAM	4 GB in 32-bit OS; 8 GB in 64-bit OS
Mainboard	Latest PC architecture and chipset
PCIe (for mvBlueCOUGAR-XD)	x1 or x4 slot for the **Network interface card** (p. 219)
Network interface card (p. 219) (for mvBlue↩ COUGAR-XD)	With the latest **"Link Aggregate Group"** (LAG) capable drivers

Several **network interface cards** (p. 219) have been tested successfully by MATRIX VISION. Please ask our support team.

See also

`https://servicedesk.matrix-vision.com`

Please ask your system vendor for further advice and consult our technical documentation.

1.9.1.2 Network interface card (NIC) settings

We recommend following network interface card (NIC) settings:

Parameter	Setting
Receive buffer	Maximum
Transmitter buffer	Maximum
MTU	Maximum; the network interface card should support at least 1.5 KB jumbo frames, preferably 4 KB

Interrupt moderation	Active
Interrupt moderation rate	Extreme / Aggressive
RSS	Active
Number of RSS queues	Number of physical CPU cores (Hyper-Threading cores do not count. E.g. 8 cores and Hyper-Threading = 16 logical CPU cores; for RSS you should select 8, because the CPU does only have 8 physical cores
Link Aggregation / Teaming / Bonding (for mvBlue↩ COUGAR-XD)	Active

For more information about network settings, please have a look at the troubleshooting chapter **Optimizing Network Performance** (p. 46) .

1.9.1.3 Supported Operating Systems

The following operating system are supported officially:

1.9.1.3.1 Windows

- Microsoft Windows 7 (32-bit, 64-bit)
- Microsoft Windows 8.1 (32-bit, 64-bit)
- Microsoft Windows 10 (32-bit, 64-bit)
- Microsoft Windows 11

Other Windows versions might work as well but will not be tested on a regular basis.

Note

Since mvIMPACT Acquire version 2.8.0 it could be possible that you have to update your Windows installer at least using Windows XP. The necessary packages are available from Microsoft's website: `http://www.microsoft.com/en-US/download/details.aspx?id=8483`

1.9.1.3.2 Linux Please check the the 'Support' section of the MATRIX VISION website for the availability of the latest Linux driver package.

See also

`https://matrix-vision.com/software-drivers-en.html`

Currently supported Kernel versions are:

- Kernel 3.5.x or greater

In case the target system runs an older Linux kernel, **it is absolutely necessary to update the kernel to at least version 3.5.0** . Please refer to the documentation of your Linux distribution for information on how to update your system's Linux kernel.

All Kernels starting from version 3.5.0 should work without problems.

1.9.2 Installing The mvGenTL-Acquire Package

All necessary drivers are available from the MATRIX VISION website at `https://www.matrix-vision.de`, section "`Products -> Cameras -> your interface -> your product -> Downloads`".

1.9.2.1 Windows

Starting the installer application

- **mvGenTL_Acquire-x86-n.n.n.msi** (for 32-bit systems) or
- **mvGenTL_Acquire-x86_64-n.n.n.msi** (for 64-bit systems): will display the following dialog:

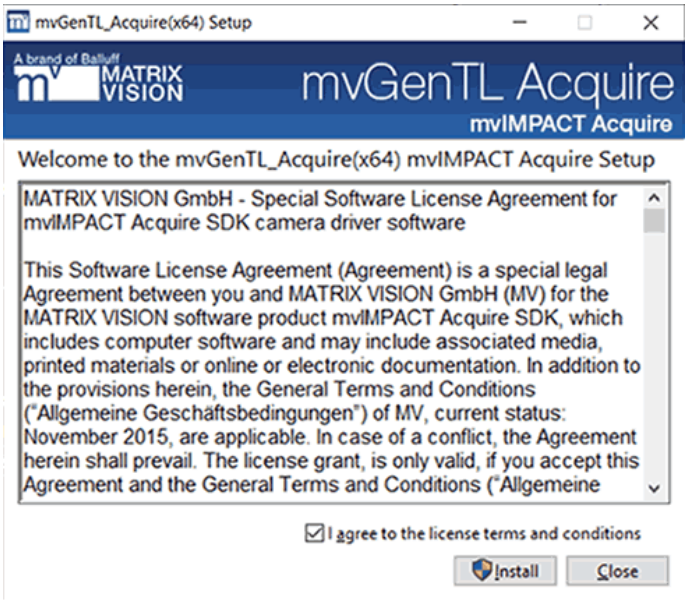


Figure 1:Driver installation - Start Window

- Now, follow the instructions of installation program and adjust the settings to your needs:

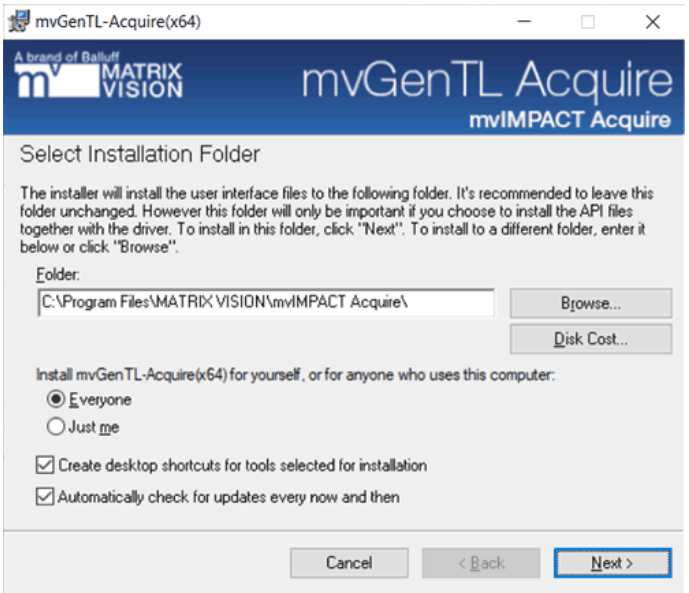


Figure 2:Driver installation - Select Installation Folder

Since

mvIMPACT Acquire 2.25.0

wxPropView (p. 139) is able to check if new driver version became available on a weekly basis. Deactivate the check box if wxPropView should not check for updates. You can activate this again in **wxPropView** (p. 139) via the help menu.

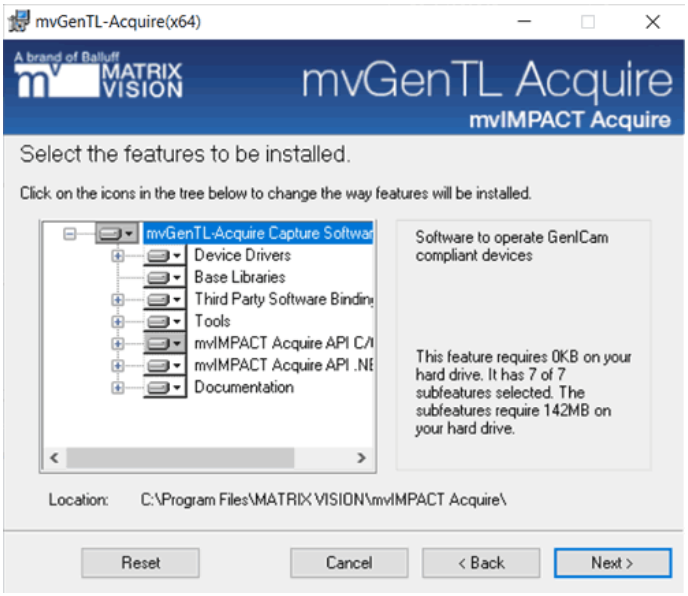


Figure 3:Driver installation - Select Features

- After confirmation, the installation will start and copy files and install device drivers.

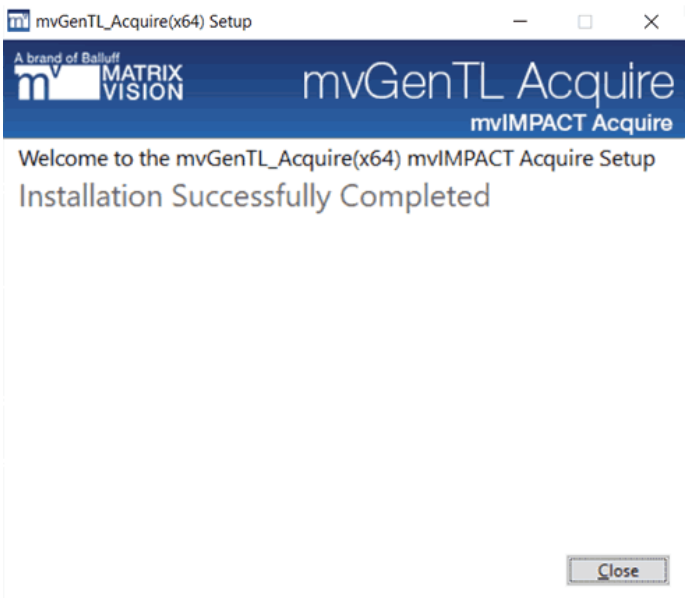


Figure 4:Driver installation - Installation Completed

You will find all tools like

- **wxPropView** (p. 139) and
- **mvDeviceConfigure** (p. 139)

either as shortcuts on the desktop or in the Windows start menu under **"MATRIX VISION ->** **mvIMPACT Acquire"**.

Afterwards, you can use **mvDeviceConfigure** (p. 139) to update the firmware if needed. The latest firmware image is available on the web - please check for updates. The current firmware version can be read out using **wxPropView** (p. 139).

1.9.2.2 Linux

The following (additional) packages will be needed to use all features of mvIMPACT Acquire:

- libwxbase3.0-0v5
- libwxbase3.0-dev
- libwxgtk3.0-gtk3-0v5
- libwxgtk3.0-gtk3-dev
- libwxgtk-webview3.0-gtk3-0v5
- libwxgtk-webview3.0-gtk3-dev
- wx3.0-headers
- build-essential (meta package)
- libgtk2.0-dev
- gcc 4.8.5 (4.9.4 for ARM versions) environment or newer

Note

The names of the packages as mentioned above are the Ubuntu specific ones. Other distributions (e.g. Debian, Arch, Redhat, ...) might use different names.

The installation script will ask if the packages should be downloaded during the installation process. If some of the packages are not installed some features might not be available. If the e.g. wxWidgets related packages are missing on the target system then all GUI application coming as part of the mvIMPACT Acquire installation won't be available.

Note

If you are going to install the mvGenTL-Acquire package on an ARM device, please read **this** (p. 57) section first.

To use a (camera) device in Linux (capture images from it and change its settings), a driver is needed, consisting of several libraries and several configuration files. These files are required during run time.

To develop applications that can use the device an API is needed, containing header files, makefiles, samples, and a few libraries. These files are required at compile time.

Both file collections are distributed in a single package which is available in the `Support` section of the MATRIX VISION website. In addition to that an installation script is provided which can be downloaded from the same location. Using this script makes installing the driver package a lot easier.

Note

The following table shows the supported platforms and the corresponding package and installation script name:

Architecture	Package	Installation Script
ARM64	mvGenTL_Acquire-ARM64_gnu-n.n.n.tgz	install_mvGenTL_Acquire_ARM.sh
ARMhf	mvGenTL_Acquire-ARMhf_gnueabi-n.n.n.tgz	install_mvGenTL_Acquire_ARM.sh
x86_64	mvGenTL_Acquire-x86_64_ABI2-n.n.n.tgz	install_mvGenTL_Acquire.sh

The following example explains the installation process for the x86_64 package. **The installation process for other packages will work almost identical except different names as mentioned in the previous table.**

- Please start a console and change into the directory where the installation script and the installation package are located e.g. `/home/username/Downloads` :

```
cd /home/username/Downloads
```

> **Note**
>
> If root permissions are needed, the script will ask for the permissions. **There is no need to call it with root permissions.**

- You might need to enable the execute flag with:

```
chmod a+x install_mvGenTL_Acquire.sh
```

- Run the install script:

```
./install_mvGenTL_Acquire.sh
```

During installation the script will ask, if it should build all tools and samples.

> **Note**
>
> The installation scripts is developed for Ubuntu/Debian, SUSE Linux and Red Hat Linux based distributions. On other distributions some features of the installation script may or may not work. Get in touch with us if you encounter any problems!

The installation script checks for package dependencies described above and installs them with the respective standard package manager (e.g. **apt-get**) if necessary. So an Internet connection is recommended.

> **Note**
>
> The installation script (`install_mvGenTL_Acquire.sh`) and the archive (`mvGenTL_Acquire-x86_64_ABI2-n.n.n.tgz`) must reside in the same directory. Nothing is written to this directory during script execution, so no write access to the directory is needed in order to execute the script.

You need Internet access in case one or more of the packages on which the **GenICam** (p. 213) libs depend are not yet installed on your system. In this case, the script will install these packages, and for that, Internet access is required.

The script supports various arguments, which allow to customize the installation, the desired functionalities and the installation process itself. All arguments are optional:

Argument	Function
`-h` or `-help`	Display the help.
`-p` or `-path`	Define a custom installation directory.
`-gev` or `-gev_support`	Install the GigE Vision related features of the driver. Default: **yes**
`-u3v` or `-u3v_support`	Install the USB3 Vision related features of the driver. Default: **yes**
`-pcie` or `-pcie_support`	Install the PCI Express related features of the driver. Default: **yes**
`-ogev` or `-only_gev`	Install only the GigE Vision related features of the driver (deprecated, favor flavors without the 'only' instead). Default: **no**

`-ou3v` or `-only_u3v`	Install only the USB3 Vision related features of the driver (deprecated, favor flavors without the 'only' instead). Default: **no**
`-onaos` or `-only_naos`	Install only the PCI Express related features of the driver (deprecated, favor flavors without the 'only' instead). Default: **no**
`-u` or `-unattended`	Unattended installation with default settings. By using this parameter you explicitly accept the EULA.
`-m` or `-minimal`	Minimal installation. No tools or samples will be built, and no automatic configuration and/or optimizations will be done. By using this parameter you explicitly accept the EULA.

The target directory name specifies where to place the driver. If the directory does not yet exist, it will be created. The path can be either absolute or relative; i.e. the name may but need not start with `/` .

If no path is specified, the package will be installed to **/opt/mvIMPACT_Acquire**.

1.9.3 Connecting The Camera

To start the device please accomplish following steps:

1. **Connect the mvBlueCOUGAR** with a
   - Cat 5e / Cat 6 or higher peer-to-peer-cable to a network (e.g. a switch) or PC directly

   Note

   With mvBlueCOUGAR-XD you can either connect both LAN connectors (RJ 45) using **Link Aggregation** (p. 54) or one LAN connector, which requires to use the **connector LAN 1** (p. 99) only.

2. **Power the mvBlueCOUGAR** via power supply.
   The mvBlueCOUGAR will boot immediately.

Note

If you're using **Power over Ethernet (PoE)** (p. 220) connecting and powering are one step.
If both power possibilities are connected (local power supply and **Power over Ethernet (PoE)** (p. 220)), the camera will use the local power supply.
If you change from one supply to the other, the camera will reset and reboot.

After energizing mvBlueCOUGAR via power supply or **Power over Ethernet (PoE)** (p. 220) the camera will start to boot.

Depending on the configuration of the network or the mvBlueCOUGAR (more precisely: if **DHCP** (p. 213) is used or not), following IP addresses are assigned:

- If the mvBlueCOUGAR is configured to use a static IP address, the mvBlueCOUGAR will use this one. In this case, the mvBlueCOUGAR will only be accessible, if the IP address is valid for the current network.
- With **DHCP** (p. 213), the mvBlueCOUGAR will get an IP address from the **DHCP** (p. 213) server (default setting).
- Without **DHCP** (p. 213), the mvBlueCOUGAR will start with a logical link address (**LLA** (p. 217)) / zero configuration class B IP address (**"169.254.X.X"**, netmask 255.255.0.0).

Note

Please have a look at **LLA** (p. 217) in the **Glossary** (p. 213) when using Linux and **LLA** (p. 217).

Attention

"Network misconfiguration"

The camera will not be reachable if you are using a wrong static IP address.

→ Please be sure that that the IP address
- does exist
  - is not used by anyone else in the same network
  - is a valid one

Note

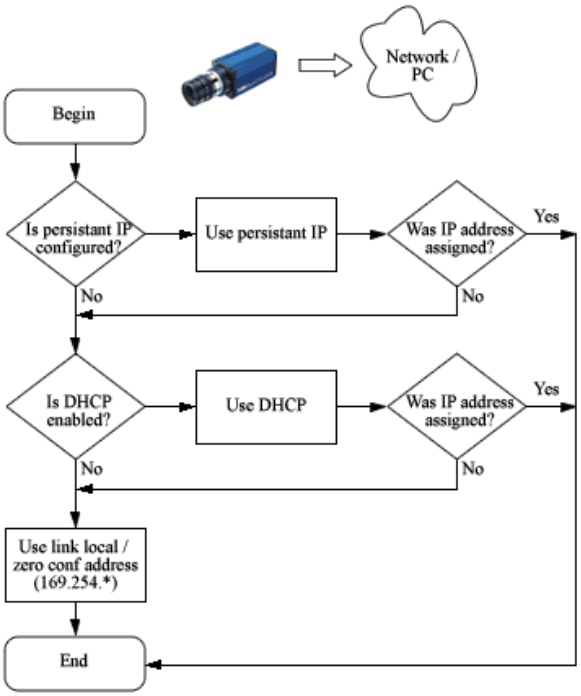
If you have two **NICs** (p. 219) in your PC, please be sure that they are using different subnets, otherwise the systems does not know which Ethernet interface is used to exchange the data.

```
Working example:

 192.168.0.1: PC Port1
 172.16.1.1:  PC Port2
 192.168.0.3: Cam1
 172.16.1.4:  Cam2
```

You can change settings like the static IP address to your needs (please have a look at: **mvIPConfigure** (p. 139)).

Further the mvBlueCOUGAR starts the **GigE Vision** (p. 215) control server making the device actually discoverable using standard **GigE Vision** (p. 215) mechanism.



Boot sequence of mvBlueCOUGAR

1.9.3.1 Communicating With The Camera

You can communicate with the camera the following way

1. **Via wxPropView** (p. 139) :
   Since driver version 2.11.3, starting **wxPropView** (p. 139) the first time, the so called Quick Setup Wizard will be launched.

   Read more about how to make optimal use of it in the mvIMPACT Acquire GUI manual: `https://www.matrix-vision.com/manuals/SDK_GUI_Tools/index.html`

> **Note**
>
> The current IP address can be found in the *"Device"* properties under **"DeviceIPAddress"**.

1.9.3.2 Communicating With A Camera In A Different Subnet

When a **GigE Vision** (p. 215) device is not connected to the same subnet than the host an advanced discovery mechanism can be used, which is called **"Unicast Device Discovery"**. This mechanism allows to find devices which are connected to a different subnet than the host if they are reachable over a specific gateway and the IP address of that device is known upfront.

> **Note**
>
> Discovering devices in different subnets requires to know the IPv4 address of each device which shall be discovered upfront as a network packet must be sent to these devices.

This special discovery mechanism can be configured using the mvIMPACT Acquire API or the GUI tools coming as part of the mvIMPACT Acquire driver stack. Please have a look at the **"Detecting Devices Residing In A Different Subnet"** chapter of the: **mvIPConfigure** (p. 139) manual for a detailed description. More information can also be found here: **Discovering Devices In Different Subnets** (p. 346)

1.9.3.3 Setting Up The Camera

MATRIX VISION offers several **GUI tools** (p. 139) to work with the camera. Please have a look at the specific chapter.

1.9.3.4 About Settings

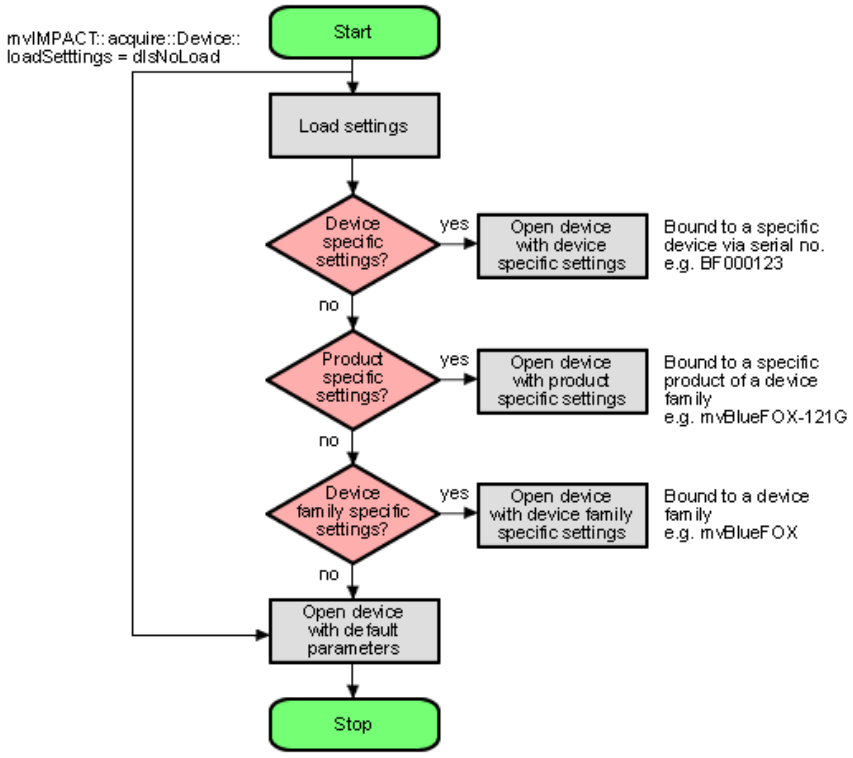
A setting contains all parameters that are needed to configure the device to a state it was in when the setting was created. Every image can be captured with a completely different set of parameters. In almost every case, these parameters are accessible via a property offered by the device driver. A setting e.g. might contain

- The gain to be applied to the analog to digital conversion process for analog video sources or
- The AOI to be captured from the incoming image data.

So for the user a setting is the one and only place where all the necessary modifications can be applied to achieve the desired data acquisition mode. There is however an important difference in behaviour between different interface layouts. Please have a look at the **"mvIMPACT Acquire SDK GUI Applications"** chapter **"wxPropView ->** **Device Configuration ->** **General Device Configuration ->** **Changing The Interface Layout To GenICam Or Device↩ Specific"** to find out how to modify the interface layout or check in the API documentation for the **interfaceLayout** property of the class **Device**.

- When working with the **DeviceSpecific** interface layout, each frame will be captured with the settings as present when requesting the image. Every parameter can be modified at any time. When requesting another image the settings valid at that moment will be used to fill this buffer with data
- For the **GenICam** interface layout all device properties modified during a continuous acquisition will be applied at once so might affect this or the next image transmitted by the device. Depending on various parameters (the number of buffer already captured but not collected by the application, the way the device internally operates(e.g. has already captured a couple of images that await transmission), etc.) this will have impact on that captured images somewhere in the near future thus when a precise moment to change settings is needed, continuous acquisition must be stopped and then restarted after modifying the features. Certain features (typically those affecting the buffer layout/size) cannot be changed while a continuous acquisition is running in GenICam interface layout anyway.

Now, whenever a device is opened, the driver will execute following procedure:



wxPropView - Device setting start procedure

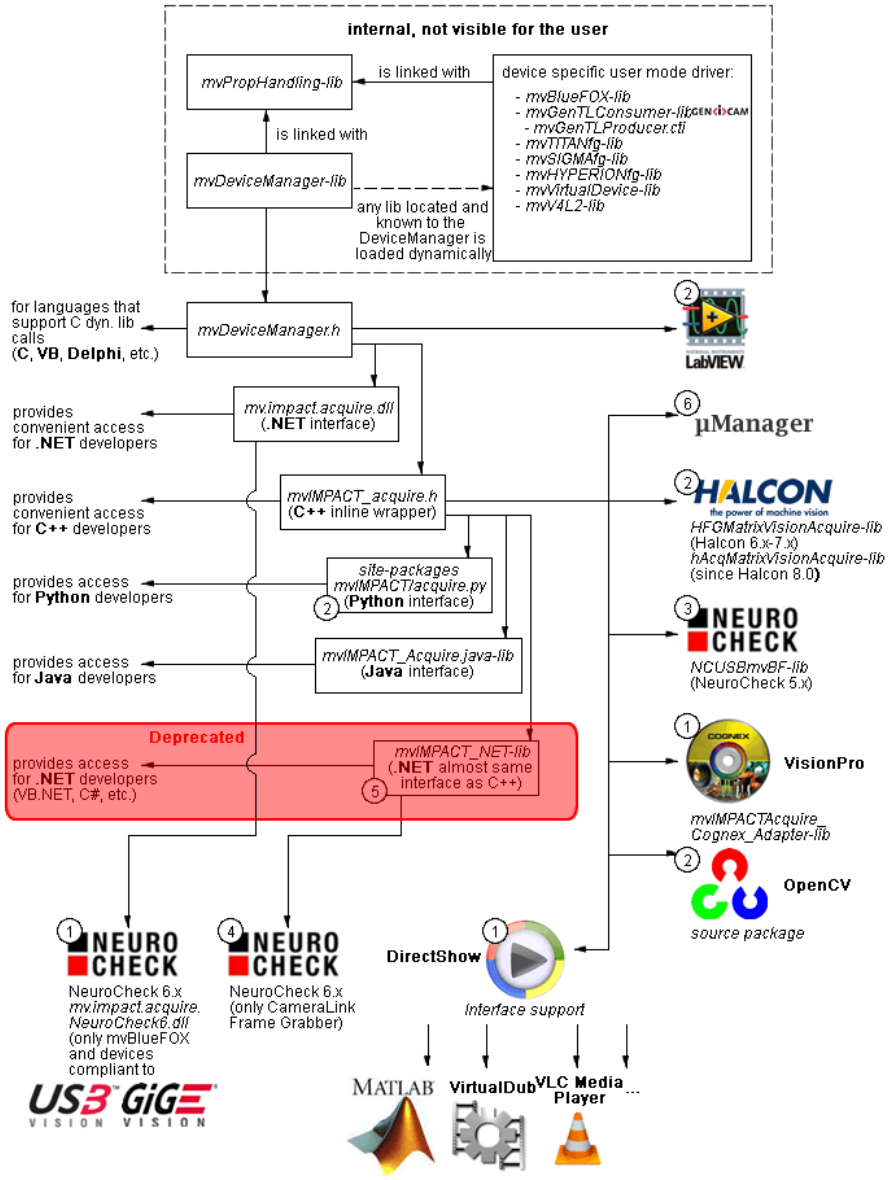
- Please note that each setting location step in the figure from above internally contains two search steps. First the framework will try to locate a setting with user scope and if this can't be located, the same setting will be searched with global (system-wide) scope. On Windows this e.g. will access either the **HKEY_CURRENT_USER** or (in the second step) the **HKEY_LOCAL_MACHINE** branch in the Registry.
- Whenever storing a product specific setting, the device specific setting of the device used for storing will be deleted (if existing). E.g. you have a device "VD000001" which belongs to the product group "VirtualDevice" with a setting exclusively for "VD000001". As soon as you store a product specific setting **using THIS device**, the (device specific) setting for "VD000001" will be deleted. Otherwise a product specific setting would never be loaded as a device specific setting will always be found first. Storing a product specific setting with a different device belonging to the same family however will **NOT** delete device specific settings for other devices.
- The very same thing will also happen when opening a device from any other application! mvPropView does not behave in a special way but only acts as an arbitrary user application.
- Whenever storing a device family specific setting, the device specific or product specific setting of the device used for storing will be deleted (if existing). See above to find out why.
- On Windows the driver will **not** look for a matching XML file during start-up automatically as the native storage location for settings is the Windows Registry. This must be loaded explicitly by the user by using the appropriate API function offered by the SDK. However, under **Linux** XML files are the only setting formats understood by the driver framework thus here the driver will also look for them at start-up. The device specific setting will be an XML file with the serial number of the device as the file name, the product specific setting will be an XML file with the product string as the filename, the device family specific setting will be an XML file with the device family name as the file name. All other XML files containing settings will be ignored!
- Restoring of settings previously stored works in a similar way. After a device has been opened the settings will be loaded automatically as described above.

- A detailed description of the individual properties offered by a device will **not** be provided here but can be found in the C++ API reference, where descriptions for all properties relevant for the user (grouped together in classes sorted by topic) can be found. As mvPropView doesn't introduce new functionality but simply evaluates the list of features offered by the device driver and lists them any modification made using the GUI controls just calls the underlying function needed to write to the selected component. mvPropView also doesn't know about the type of component or e.g. the list of allowed values for a property. This again is information delivered by the driver and therefore can be queried by the user as well without the need to have special *inside information*. One version of the tool will always be delivered in source so it can be used as a reference to find out how to get the desired information from the device driver.

1.9.4 Driver concept

The driver supplied with the MATRIX VISION product represents the port between the programmer and the hardware. The driver concept of MATRIX VISION provides a standardized programming interface to all image processing products made by MATRIX VISION GmbH.
The advantage of this concept for the programmer is that a developed application runs without the need for any major modifications to the various image processing products made by MATRIX VISION GmbH. You can also incorporate new driver versions, which are available for download free of charge on our website:
`https://www.matrix-vision.com`.

The following diagram shows a schematic structure of the driver concept:



Driver concept

- **1** Part of any mvIMPACT Acquire driver installation package (Windows).
- **2** Separately available for 32 bit and 64 bit. Requires at least one installed driver package.
- **3** See 2, but requires an installed version of the mvBlueFOX driver.

- **4** Part of the NeuroCheck installer but requires at least one installed frame grabber driver.
- **5** Part of the mvIMPACT SDK installation. However, new designs should use the .NET libs that are now part of mvIMPACT Acquire (`"mv.impact.acquire.dll"`). The namespace `"mv.impact.acquire"` of `"mv.impact.acquire.dll"` provides a more natural and more efficient access to the same features as contained in the namespace `"mvIMPACT_NET.acquire"` of `"mvIMPACT_NET.dll"`, which is why the latter one should only be used for backward compatibility but **NOT** when developing a new application.
- **6** Part of Micro-Manager.

1.9.4.1 NeuroCheck Support

A couple of devices are supported by NeuroCheck. However between NeuroCheck 5.x and NeuroCheck 6.x there has been a breaking change in the internal interfaces. Therefore also the list of supported devices differs from one version to another and some additional libraries might be required.

For NeuroCheck 5.x the following devices are supported:

Device	Additional software needed
mvTITAN-G1	mvSDK driver for mvTITAN/mvGAMMA devices
mvTITAN-CL	mvSDK driver for mvTITAN/mvGAMMA devices
mvGAMMA-CL	mvSDK driver for mvTITAN/mvGAMMA devices
mvBlueFOX	mvIMPACT Acquire driver for mvBlueFOX devices, `"NCUSBmvBF.dll"`

For NeuroCheck 6.0 the following devices are supported:

Device	Additional software needed
mvTITAN-G1	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvTITAN-CL	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvGAMMA-CL	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvHYPERION-CLb	mvIMPACT Acquire driver for mvHYPERION devices
Every other mvIMPACT Acquire compliant device	mvIMPACT Acquire driver for the corresponding device family, `"mv.impact.acquire.NeuroCheck6.dll"` (comes with the driver package, but the driver package must be installed **AFTER** installing NeuroCheck 6

For NeuroCheck 6.1 the following devices are supported:

Device	Additional software needed
mvTITAN-G1	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvTITAN-CL	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvGAMMA-CL	mvIMPACT Acquire driver for mvTITAN/mvGAMMA devices
mvHYPERION-CLb	mvIMPACT Acquire driver for mvHYPERION devices
Every other mvIMPACT Acquire compliant device	mvIMPACT Acquire driver for the corresponding device family, `"mv.impact.acquire.NeuroCheck6_1.dll"` (comes with the driver package, but the driver package must be installed **AFTER** installing NeuroCheck 6.1

1.9.4.2 VisionPro Support

Every *mvIMPACT Acquire* driver package on Windows comes with an adapter to VisionPro from Cognex. The installation order does not matter. After the driver package and VisionPro has been installed, the next time VisionPro is started it will allow selecting the *mvIMPACT Acquire* device. No additional steps are needed.

MATRIX VISION devices that also comply with the GigE Vision or USB3 Vision standard don't need any software at all, but can also use VisionPro's built-in GigE Vision or USB3 Vision support.

1.9.4.3 HALCON Support

HALCON comes with built-in support for *mvIMPACT Acquire* compliant devices, so once a device driver has been installed for the *mvIMPACT Acquire* device, it can also be operated from a HALCON environment using the corresponding acquisition interface. No additional steps are needed.

MATRIX VISION devices that also comply with the GigE Vision standard don't need any software at all, but can also use HALCON's built-in GigE Vision support.

As some *mvIMPACT Acquire* device driver packages also come with a GenTL compliant interface, these can also be operated through HALCON's built-in GenTL acquisition interface.

1.9.4.4 LabVIEW Support

Every *mvIMPACT Acquire* compliant device can be operated under LabVIEW through an additional set of VIs which is shipped by MATRIX VISION as a separate installation (`"mvLabVIEW Acquire"`).

MATRIX VISION devices that also comply with the GigE Vision or USB3 Vision standard don't need any additional software at all, but can also be operated through LabVIEW's GigE Vision or USB3 Vision driver packages.

1.9.4.5 DirectShow Support

Every *mvIMPACT Acquire* compliant device driver package comes with an interface to DirectShow. In order to be usable from a DirectShow compliant application, devices must first be registered for DirectShow support. How to this is explained **here** (p. 178).

1.9.4.6 Micro-Manager Support

Every *mvIMPACT Acquire* compliant device can be operated under `https://micro-manager.org` when using mvIMPACT Acquire 2.18.0 or later and at least *Micro-Manager* 1.4.23 build **AFTER** 15.12.2016. The adapter needed is part of the *Micro-Manager* release. Additional information can be found here: `https://micro-manager.org/wiki/MatrixVision`.

1.9.5 Relationship Between Driver, Firmware And SDK

To operate a **GenICam** (p. 213) based device apart from the physical hardware itself 2 pieces of software are needed:

- A firmware running on the device. This firmware consists of
  - A **GenICam** (p. 213) compliant XML file exposing the features in a generic and standard compliant way
  - A **FPGA** file
  - The actual **micro-code** making the device operational
- A **device driver** (this is the `mvGenTLConsumer.dll` and the `mvGenTLProducer.cti` on Windows and the `libmvGenTLConsumer.so` and the `libmvGenTLProducer.so` on Linux when using mv↩ IMPACT Acquire, but can be any other **USB3 Vision** (p. 221) / **GigE Vision** (p. 215) compliant driver package from a third party vendor) running of the host system (provides control over the device from an application running on the host system).

The physical **GenICam** (p. 213) compliant device has a firmware programmed into the device's non-volatile memory, thus allowing the device to boot to a fully functional state without the need of any additional software running on the host. The firmware version that will be used when operating the device does NOT depend on the driver version that is used to communicate with the device. This will allow any piece of compliant third party software to operate the device without the need to have special knowledge about the firmware structure. This shall be illustrated by the following figure:

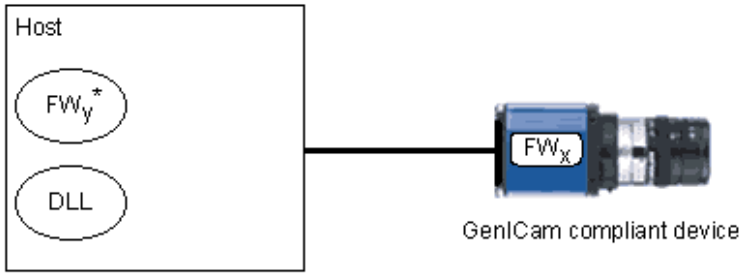


Figure 8: The firmware is not a part of the device driver

Note

As it can be seen in the image the firmware file is NOT part of the device driver but comes as a separate archive. It is important to notice that a firmware file that may be present on the host system will *not* be used automatically but only when the user or an application explicitly updates the firmware on the device and will only become active after power-cycling the device.

The name of the firmware update archive ($*$ in the figure above) is:

- **mvBlueCOUGAR-X_Update.mvu**

Only during a firmware update the firmware file that has been selected from the file system of the host system will be downloaded permanently into the device's non-volatile memory.

Attention

"Wrong firmware"

Each firmware archive might contain multiple firmware files (e.g. for different device types or device hardware revisions). Installing an unsuitable firmware version can damage the device.

→ In order to make sure suitable firmware versions for a specific device are installed appropriate tools such as **mvDeviceConfigure** (p. 139) should be used.

So assume a device with a certain firmware version is connected to a host system.

During an explicit firmware update, the firmware file will be downloaded onto the device. In order to become active the device must be power-cycled:

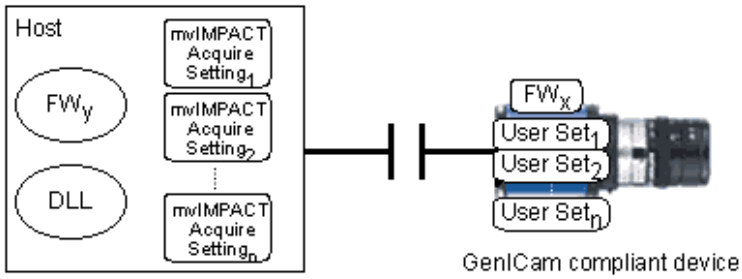


Figure 9: Firmware file will be downloaded during an firmware update...

This can either be done by unplugging the device and then by plugging it back in or (for devices supporting this feature) by resetting/rebooting the device by a certain software command (DeviceControl/DeviceReset). When using **mvDeviceConfigure** (p. 139) to update devices the latter mechanism will be used by the tool thus it is NOT necessary to unplug the device.

When the device has completed rebooting the new firmware version will become active:

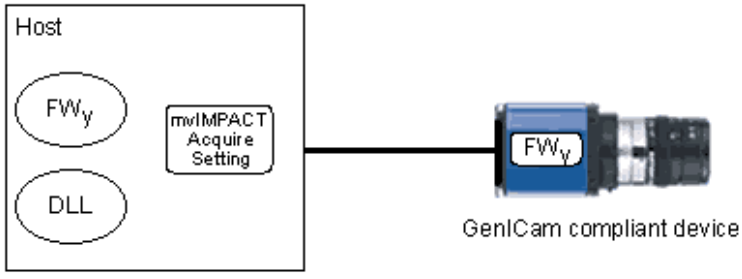


Figure 10: ... after re-powering the device it will be active

- The current firmware version of the device can be obtained either by using one of the applications which are part of the SDK such as **mvDeviceConfigure** (p. 139) or by reading the value of the property `Device/↩FirmwareVersion` or `DeviceControl/DeviceFirmwareVersion` using the API
- The current FPGA file version used by the device can be obtained by reading the value of the property `DeviceControl/mvDeviceFPGAVersion`

Note

The FPGA file is a part of the firmware and cannot be updated independently thus reading it's version just provides some additional information.

Using **wxPropView** (p. 139) the same information is available as indicated by the following figure:

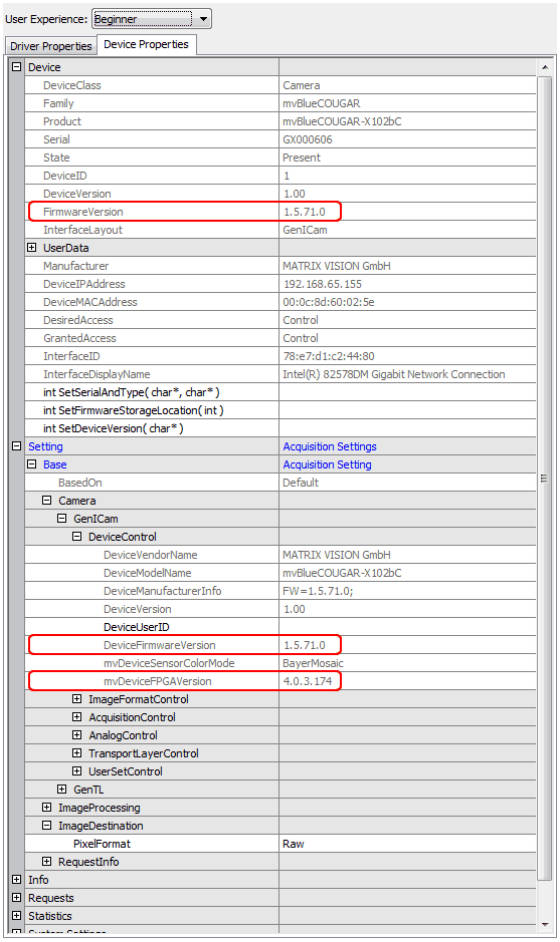


Figure 11: wxPropView - FPGA and Firmware version numbers

Apart from the device driver and firmware relationship there are certain places where a device configuration can be stored when dealing with **GenICam** (p. 213) compliant devices:

- There may be User Sets which are stored in the device's non-volatile memory. User Sets contain all the features, which affect the device's behaviour such as transfer pixel format, exposure time etc. User Sets are bound to major **GenICam** (p. 213) XML file releases, thus these settings will be lost whenever a firmware contains a different major version of a devices **GenICam** (p. 213) XML file
- mvIMPACT Acquire settings which contain the state of all the features also stored in a User Set as well as other features added by the device driver. These settings will be stored on the host system either as a XML file or (on Windows only) in the Registry

Both methods can be used to pre-configure a device. Using the first method, the state of the features will travel with the physical device, using the mvIMPACT Acquire settings, feature states can be copied from host to host as a file.

1.9.6 Optimizing Network Performance

Note

This section is only relevant for applications working with GigE Vision™ or other (MATRIX VISION) network devices!

Besides the host system requirements, to achieve the desired data rates fine-tuning the network environment might be required.

There are two locations where you can adapt the parameters concerning the network environment:

1. on the camera's side using **wxPropView** (p. 139) or the API
2. on the **network interface card's** (p. 219) side using the operating system's possibilities to adjust driver parameters

Note

Only network experts should change settings in the following section. However it is highly recommend to optimize the network settings so this should be done! Another important aspect when dealing with GigE Vision™ devices is described **here** (p. 343).

The following sections will describe the most important settings which are necessary to configure the network interface card for a flawless image transmission.

- **Checklist for Windows** (p. 46)
- **Checklist for Linux** (p. 57)
- **Multi-Core Acquisition Optimizer** (p. 64)

1.9.6.1 Checklist for Windows

Note

The screenshots below are examples only and show how a dialog for a feature described here might look like. Not every NIC/NIC driver will support every feature discussed here and different drivers might use different names for the features presented. The hints are just meant to give an idea for what to look for.

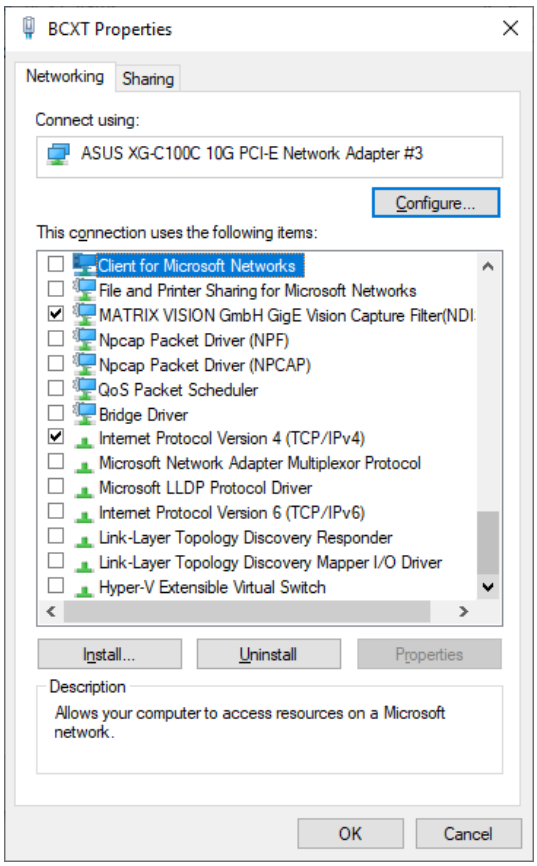
1.9.6.1.1 1. Network Interface Card Using the correct **network interface card** (p. 219) is crucial. In case of mvBlueCOUGAR-X(D) this would be a 1 Gigabit Ethernet controller which is capable of full duplex transmission. In case an application shall use both links of an mvBlueCOUGAR-XD then at least a dual-port 1 Gigabit **NIC** (p. 219) is needed.

In the properties of the **network interface card** (p. 219) settings, please check, if the **network interface card** (p. 219) is operating as a **"1000MBit Full Duplex Controller"** and if this mode is activated.

1.9.6.1.2 2. Network Interface Card Driver Also the **network interface card** (p. 219) manufacturers provide driver updates for their cards every now and then. Using the latest NIC drivers is always recommended and might improve the overall performance of the system dramatically!

1.9.6.1.3 3. GigE Vision Capture Driver Please check, if the GigE Vision™ capture filter driver has been installed correctly by **mvGigEConfigure** (p. 139).

1.9.6.1.4 4. Network Protocol Drivers All unused protocol drivers should be disabled in order to improve the overall performance and stability of the system! In the following screen-shot the minimum set of recommended protocol drivers is enabled only. If others are needed they can be switched on but it's best to reduce the enabled drivers to a bare minimum.



Unused protocol drivers should be disabled

Note

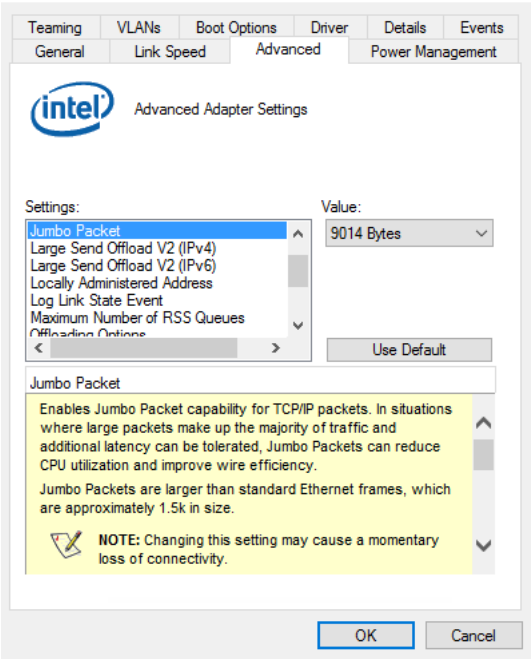
Especially drivers written against an older version of the NDIS specification should be disabled or even removed from the system since a single NDIS 5 (or smaller) driver in the driver stack can degrade the overall performance significantly. This is especially true for older versions of Wireshark coming with an old version of WinPcap! These should be upgraded to newer Wireshark releases using Npcap if possible and WinPcap drivers should be removed from the system! Using Wireshark at 10G is only recommended for debugging purposes. Shipped systems should NOT use Wireshark in the background since this comes with a huge performance overhead.

1.9.6.1.5 5. Maximum transmission unit (MTU) / Jumbo Frames The camera sets the **MTU** (p. 219) to the maximum value automatically given by the **NIC** (p. 219) or switch and supports a maximum **MTU** (p. 219) of 8K. You can manually change the network packet size the camera uses for transmitting data using this property: **"Setting -> Base -> Camera -> GenICam -> Transport Layer Control -> Gev Stream Channel Selector -> Gev SCPS Packet Size"**:

As a general rule of thumb it can be said, that the higher the **MTU** (p. 219), the better the overall performance, as less network packets are needed to transmit a full image, which results in less overhead that arises from the handling of each arriving network packet in the device driver etc. However every component involved in the data transmission (that includes every switch, router and other network component, that is installed in between the device and the system receiving the packets) must support the **MTU** (p. 219) or packet size, as otherwise the first component not

supporting the packet size will silently discard all packets, that are larger than the component can handle. Thus the `weakest link` here determines to overall performance of the full system!

On the **network interface card's** (p. 219) side, this might look like this:



Setting **"Jumbo Packet"** using an Intel Ethernet Server Adapter I350-T2 network interface

Note

There is no need to set the transfer packet size manually. By default the driver will determine the maximum possible packet size for the current network settings automatically when the device is initialised. However again the *weakest link* determines this packet size thus NICs should be chosen and configured appropriately and the same goes for switches!

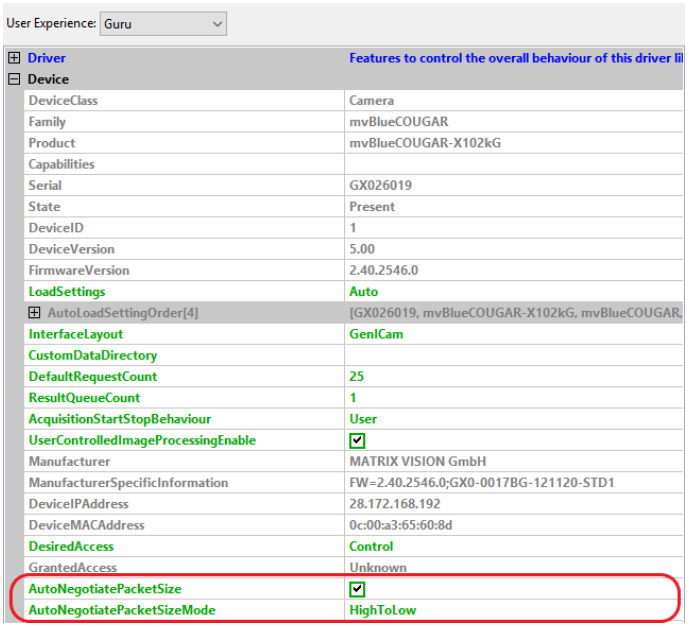
The behavior of the auto negotiation algorithm can be configured manually or to disable it completely if needed. The **AutoNegotiatePacketSize** property determines if the driver should try to find optimal settings at all and the way this is done can be influenced by the value of the **AutoNegotiatePacketSizeMode** property. The following modes for the are available:

Note

All components belonging to the network path (device, network interface card, switches, ...) will affect the MTU negotiation as every device will support the negotiated value.

Value	Description
HighToLow	The MTU is automatically negotiated starting from the NICs current MTU down to a value supported
LowToHigh	The negotiation starts with a small value and then tries larger values with each iteration until the optimal value has been found

To disable the MTU auto negotiation just set the **AutoNegotiatePacketSize** property to **"No"**.



wxPropView - MTU Auto negotiation

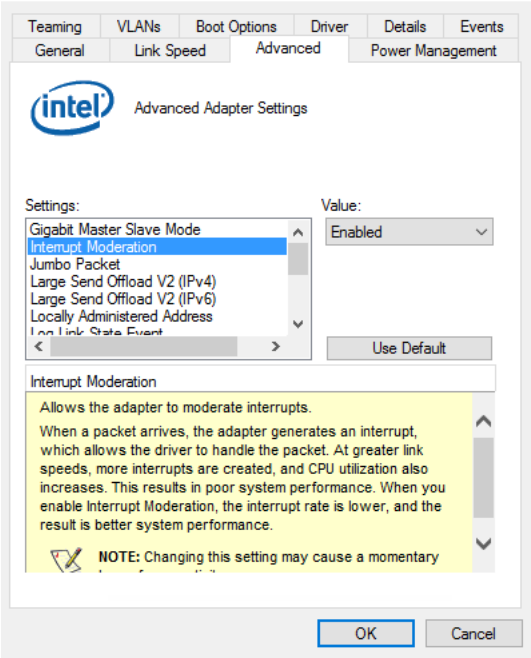
Note

Usually manually setting the packet size should only be done if

- auto-negotiation for some reasons doesn't work properly
- there are very good (e.g. application specific) reasons for doing so

1.9.6.1.6 6. Interrupt Moderation In the properties of the **network interface card** (p. 219) settings, please check, if the **"Interrupt Moderation"** is activated.

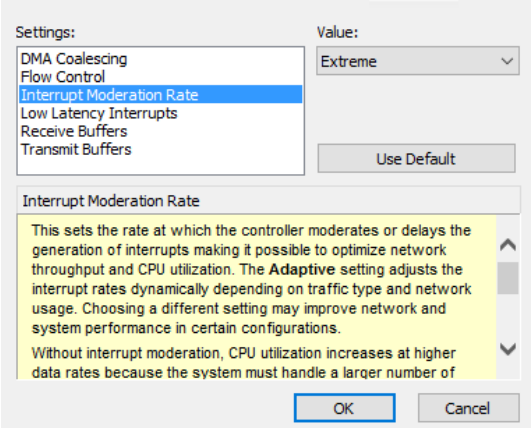
An Interrupt causes the CPU to - well - interrupt what it is currently doing. This of course consumes time and other resources so usually a system should process interrupts as little as possible. In cases of high data rates the number of interrupts might affect the overall performance of the system. The "Interrupt Moderation"-setting allows to combine interrupts of multiple packets to just a single interrupt. The downside is that this might slightly increase the latency of an individual network packet but that should usually be negligible.



Setting **"Interrupt Moderation"** using an Intel Ethernet Server Adapter I350-T2 network interface

Once "Interrupt moderation" is enabled there even might be the possibility to configure the interrupt moderation rate of the network interface.

Usually there are multiple possible options which will reflect a different tradeoff between latency and CPU usage. Depending on the expected data rate different values might be suitable here. Usually the NIC knowns best so setting the value to *Adaptive* is mostly the best option. Other values should be used after careful consideration.



Setting **"Interrupt Moderation Rate"** using an Intel Ethernet Server Adapter I350-T2 network interface

In case of high frame rates the best option sometimes might be **"Extreme"** when individual network packets are not needed to be processed right away. In that case however one needs to be aware of the consequences: If an application receives an incomplete image every now and then, then **"Extreme"** might not be the right choice since it basically means *"wait until the last moment before the NIC receive buffer overflows, then generate an interrupt"*. While this is good in terms of minimal CPU load this of course is bad if this interrupt then is not served at once because of other interrupts as then packet data will be lost. It really depends on the whole system and sometimes needs some trail and error work. The amount of buffer reserved for receiving network data is configured by the

1. **Receive/Transmit Descriptors**

(p. 52) and combined with the

1. **Maximum transmission unit (MTU) / Jumbo Frames**

(p. 47) parameter these 3 values are the most important onces to consider!

1.9.6.1.7 7. RSS (Receive Side Scaling) Queues Some **NICs** (p. 219) might also offer to configure the number of **"RSS (Receive Side Scaling) Queues"**. In certain cases this technology might help to improve the performance of the system but in some cases it might even reduce the performance.

The feature in general offers the possibility to distribute the CPU load caused by network traffic to different CPU cores instead of handling the full load just on one CPU core.

1.9.6.1.7.1 Where To Use It

- When work with cameras supporting the MATRIX VISION **Multi-Core Acquisition Optimizer** (p. 64) technology.
- In systems which work with multiple GigE-Vision cameras over one link. This would be the case if multiple Gigabit Ethernet cameras are connected to a switch which is connected to the host via e.g. a 10 Gigabit Ethernet up link port.
- If a device offers more than one network stream.

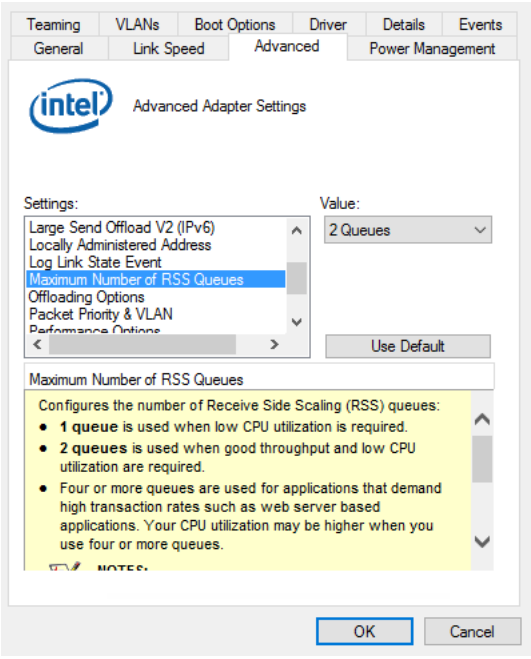
1.9.6.1.7.2 Where Not To Use It

- If multiple GigE-Vision cameras are connected via multiple NICs directly to the host.
- If a network device does just offer one network stream.

1.9.6.1.7.3 What It Does Usually a single network stream (source and destination port and IP address) will always be processed on a single CPU in order to benefit from cache lines etc.. Switching on RSS will not change this but will try to distribute which network stream is processed on which CPU more evenly and combined NIC properties like the base CPU etc. it is even possible to configure which NIC shall use which CPU(s) in the host system.

Note

Configuring more RSS queues than physical cores available in the system might have even a negative impact on the overall performance! However the operating system usually knows what to do, so if your system has 6 physical CPUs and your **NIC** (p. 219) allows either 4 or 8 RSS queues then 8 is the option to use to get access to **all** CPU cores. Only if any kind of strange behaviour is encountered then either go with 4 queues or get in touch with our support team.



Setting **"RSS Queues"** using an Intel Ethernet Adapter I350-T2

1.9.6.1.8 8. Receive/Transmit Descriptors Please check, if the number of **"Receive Descriptors"** (**RxDesc**) and **"Transmit Descriptors"** (**TxDesc**) of the **NIC** (p. 219) is set to the maximum value!

"Receive Descriptors" are data segments either describing a segment of memory on the NIC itself or in the host systems RAM for **incoming** network packets. Each incoming packet needs either one or more of these descriptors in order to transfer the packets data into the host systems memory. Once the number of free receive descriptors is insufficient packets will be dropped thus leads to data losses. So for demanding applications usually the bigger this value is configured the better the overall stability.

"Transmit Descriptors" are data segments either describing a segment of memory on the NIC itself or in the host systems RAM for **outgoing** network packets. Each outgoing packet needs either one or more of these descriptors in order to transfer the packets data out into the network. Once the number of free transmit descriptors is insufficient packets will be dropped thus leads to data losses. So for demanding applications usually the bigger this value is configured the better the overall stability.

These values have a close relationship to the

1. **Maximum transmission unit (MTU) / Jumbo Frames**

(p. 47) as well. You can get a feeling about values with following formula:

Note

The formula is per camera and per network port.

$$\text{network packets needed} \geq 1.1 * \frac{\text{PixelPerImg} * \text{BytesPerPixel}}{\text{MTU}}$$

"Example 1: 1500 at 1.3MPixel"

$$\text{network packets needed} \geq 1.1 * \frac{1.3\text{M} * 1}{1500} \geq 950$$

"Example 2: 8k at 1.3MPixel"

$$\text{network packets needed} \geq 1.1 * \frac{1.3\text{M} * 1}{8192} \geq 175$$

Both examples are showing the required network packets per image which **are NOT** to be confused with the number of available receive descriptors! A *descriptor* (receive or transmit) usually describes a fixed size piece of memory (e.g. 2048 bytes). Now with increasing values of images per second the number of **"Receive Descriptors"** might be consumed very soon given that the NIC is not always served directly by the operating system. Usually, the default values reserved by the NIC driver vary between 64 and 256 and usually for acquiring uncompressed image data at average speed this is not enough. Also processing a network packet consumes some time which is independent of the size (e.g. generating an interrupt, calling upper software layers, etc.), which is why larger packets usually result in a better overall performance.

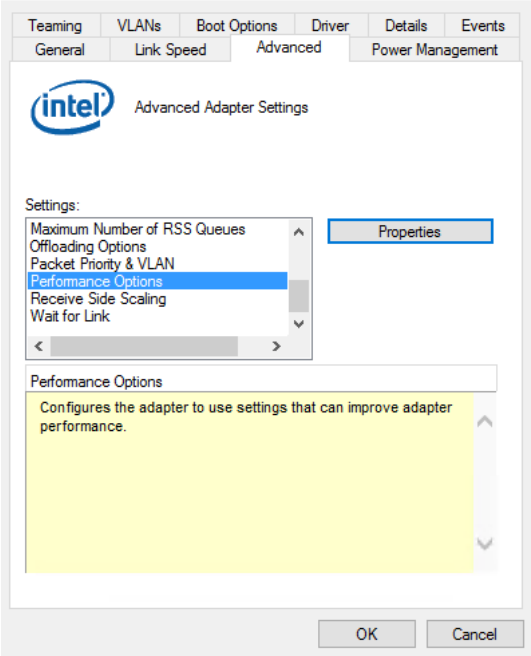
Note

If you are forced to use a small **MTU** (p. 47) values and/or if the NIC only supports a small value for **"Receive Descriptors"** or **"Transmit Descriptors"**, the transmission might not work reliable and you might need to consider using a different NIC. You may also consider to enable

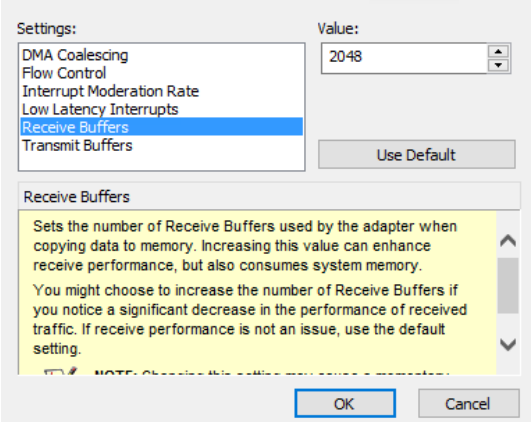
1. **Interrupt Moderation**

(p. 49) allowing the **NIC** (p. 219) to signal multiple network packets using a single interrupt to reduce the overall amount of interrupts. The downside of this of course is that some time sensitive network packets (e.g. a GigE Vision™ event sometimes might arrive in the application a little later. Usually this is acceptable but sometimes it may be not.

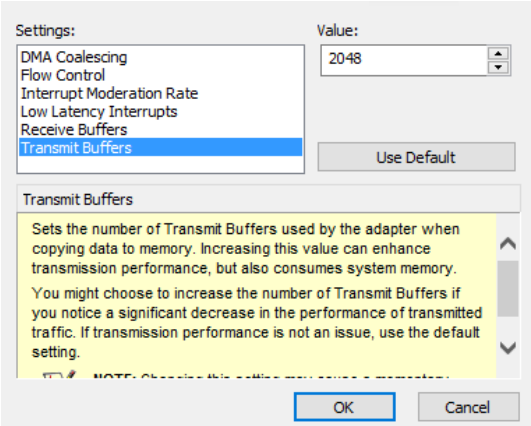
The **"Receive Descriptors"** a.k.a. **"Receive Buffers"** and **"Transmit Descriptors"** a.k.a. **"Transmit Buffers"** (Intel Ethernet Server Adapter I350-T2 network interface) can usually be found under *"Performance Options"*:



"Performance Options"



Setting **"Receive Buffers"**



Setting **"Transmit Buffers"**

Note

Depending on the used network card and driver these options might have slightly different locations and/or names.

1.9.6.1.9 9. Link Aggregation, Teaming Or Bonding (GigE Vision Devices With Multiple NICs (mvBlue↩ COUGAR-XD)) While it is possible to operate mvBlueCOUGAR-XD Dual-GigE devices with a single network cable (which requires to use the **connector LAN 1** (p. 99) only), the usage of two network cables operated in static **LAG** (**Link Aggregation**) is recommended when the devices should run at full speed. This **Link Aggregation** or **SLA Static Link Aggregation** feature is a technology invented to increase the performance between two network devices by combining two single network interfaces to one virtual link which provides the bandwidth of both single links (e.g. two 1 GBit links would result in a virtual single 2 GBit link). Swichting the modes (single or double link) requires a reboot of the camera.

Note

Not relevant for building a LAG team but still important to know: Both links are used in a round robin scheme thus packages will leave the device *in order* they still might arrive in the driver stack in a different order. The reason for this is that both NICs never generate an interrupt in an absolutely predictive way nor will they be served by the operating system always in a predictable manner. So assuming one NIC of the camera transmits all the odd packets and the other all the even ones (which is more or less how the mvBlueCOUGAR-XD will do it) it is still very likely or at least not uncommon, that a lot of either even or odd packets are received before a bunch of odd or even ones is processed. This depends on various factors and a driver should simply be able to deal with it. Interrupt moderation as well as thread scheduling done by the operating system and other things are at work here. As a result this means a packet with a lower block or packet ID might be received after one with a higher one. It would be fair to assume a data loss then but in case of LAG it is a little more complicated.

If two network cables should be used either a network interface card with two network interfaces or two identical network interface cards allowing to be teamed are needed. Using a single card is recommended. In addition to this requirement the network interface card has to support so-called **Link Aggregation**, **teaming**, or **bonding**. Normally, Windows installs a standard driver which does not support link aggregation. For this, you have to install and download a driver from the manufacturer's website. When e.g. using a **"Intel Ethernet Server Adapter I350-T2"** network controller this will be

- https://downloadcenter.intel.com/download/18713/Intel-Network-Adapter-↩ Driver-for-Windows-7 (PROWinLegacy.exe for Windows 7)
- https://downloadcenter.intel.com/download/25016/Ethernet-Intel-Network-↩ Adapter-Driver-f-r-Windows-10 (PROWin.exe for Windows 10).

See also

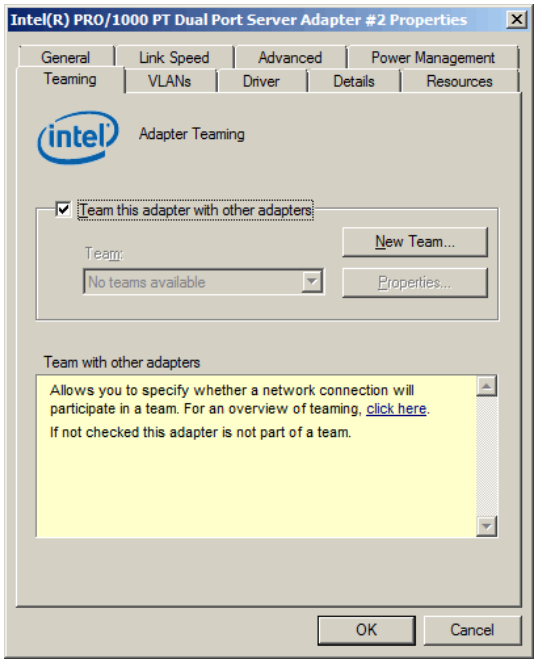
List of Intel network controllers supporting teaming with Windows 10:

- https://www.intel.com/content/www/us/en/support/articles/000020905/network-and-i html

After installation you have to combine the two interfaces which are used by the mvBlueCOUGAR-XD. For this, open the driver settings of the network controller via the **"Device Manager"**:

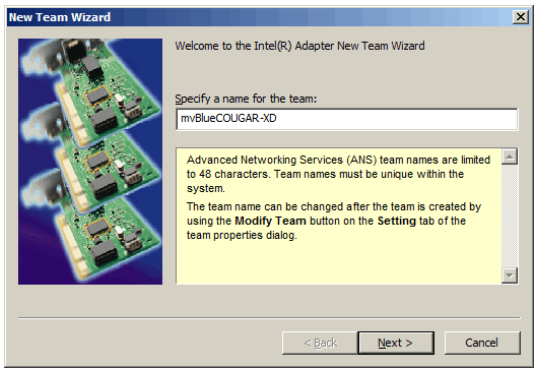
1.9.6.1.9.1 Windows 7

1. Open the tab **"Teaming"** and
2. create a new team.



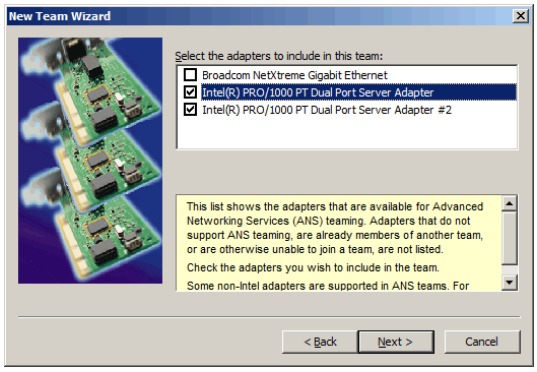
Create an aggregation group

3. Then enter a name for this group e.g. "mvBlueCOUGAR-XD" and
4. click on **"Next"**.



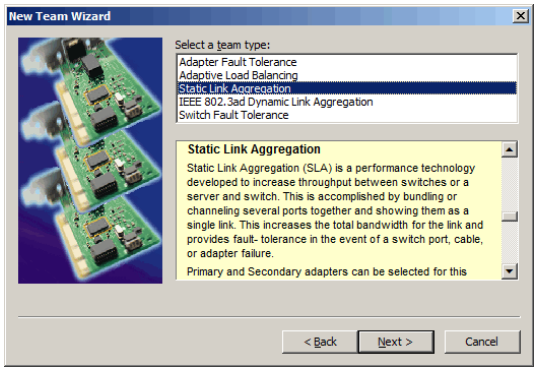
Enter an aggregation group name

5. Select the two interfaces which where used for the mvBlueCOUGAR-XD.



Select the used interfaces

6. Finally, select the connection type **"Static Link Aggregation"**.



Select the connection type

After closing the group assistant, you can connect the camera using the two network interfaces. There is no special connection necessary between camera and network controller interfaces.

It is also possible to use the camera with one network interface. This of course will result in a reduced data rate. For this use the right (primary) network interface of the camera only (the label on the back can be read normally).

1.9.6.1.9.2 Windows 10

1. Click Start (Windows Icon in the taskbar) and type in **Powershell**.
2. Right-click the Powershell icon and choose **"Run as Administrator"**.
3. If you are prompted to allow the action by User Account Control, click **Yes** .
4. Enter the command (e.g. for Intel NIC)

```
Import-Module -Name "C:\Program Files\Intel\Wired Networking\IntelNetCmdlets\IntelNetCmdlets"
```

5. Afterwards, execute the command:

```
New-IntelNetTeam -TeamName "MyLAGTeam" -TeamMemberNames "Intel(R) Ethernet Server Adapter I350-T2","Intel
-TeamMode StaticLinkAggregation
```

Replace *MyLAGTeam* with your preferred teaming name and also *"Intel(R) Ethernet Server Adapter I350-T2"* with the name of your used NIC.

1.9.6.1.9.3 Windows 11

Note

At the moment (March 2022) there seems to be no support for **Link Aggregation** or **SLA Static Link Aggregation** using Intel network interface adapters. As Intel discontinued the support for Intel Intel® PROSet and Intel® Advanced Network Services (Intel® ANS), it is unclear if the required functionality will be availble for Windows 11.

See also

More details regarding the support of Intel® network adapters can be found here:

- https://www.intel.co.uk/content/www/uk/en/support/articles/000087483/ethernet-p
  html
- https://www.intel.co.uk/content/www/uk/en/support/articles/000025890/ethernet-p
  html

1.9.6.2 Checklist for Linux

Note

In Ubuntu releases newer than Ubuntu 17.04 (Desktop and Server) network configuration is managed by netplan.io instead of /etc/network/interfaces. Netplan is a powerful tool which uses the information provided in yaml (a human-readable data-serialization language) files for netplan to create configuration files for NetworkManager or system-networkd within their own syntax. A detailed description how to setup the **NIC** (p. 219) will be available at each step of the following list if the settings could be applied using netplan.

See also

More details regarding netplan can be found here:

- https://netplan.io/
- https://netplan.io/examples/
- https://en.wikipedia.org/wiki/YAML

1.9.6.2.1 1. Receive/Transmit Buffers A sufficient amount of kernel socket buffer for receiving and transmitting network packets is crucial for a flawless operation!

The following table explains the important settings and recommended values.

Setting	Value	Description
net.core.wmem_max	16777216	Maximum memory size of a socket buffer for sending in bytes
net.core.rmem_max	16777216	Maximum memory size of a socket buffer for receiving in bytes
net.core.udp_mem	10240, 87380, 16777216	Minimum, Default and Maximum memory size of a UDP socket buffer for receiving and sending data in bytes
net.core.netdev_max_backlog	10000	Maximum number of packets which can be buffered if the kernel does not manage to process them as fast as they are received

Usually when receiving TCP traffic on a bad connection the transmission speed simply goes down. For GigE Vision™ UDP streaming the transmission will always run at a dedicated, guaranteed speed and that speed is usually higher than any other network streams speed in a given system. As the operating system not always has a free timeslot available for reporting a network packet to the upper software layer it might have to store a certain amount of packets for a certain period of time. This will be done in the memory block described by the above 3 parameters for sending and receiving data.

To set those values, you have to

1. create the file /etc/sysctl.d/62-buffers-performance.conf and
2. add the following lines to this file:

```
net.core.wmem_max=16777216
net.core.rmem_max=16777216
net.core.udp_mem=10240 87380 16777216
net.core.netdev_max_backlog=10000
```

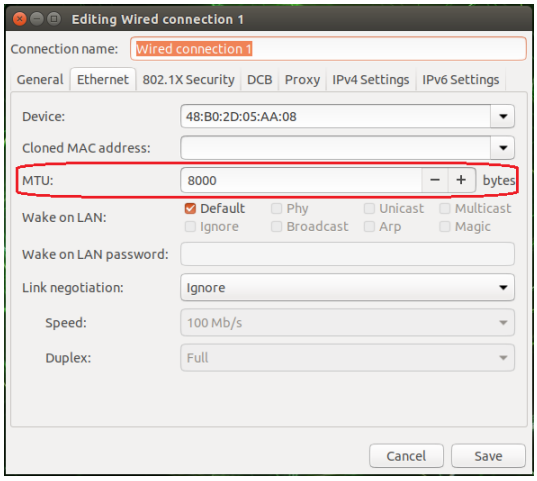
3. Afterwards, reboot the system or enter "sudo sysctl -p".

See also

For more information about Linux network tuning, please have a look at
http://www.cyberciti.biz/faq/linux-tcp-tuning/.

1.9.6.2.2 2. Maximum Transmission Unit (MTU) / Jumbo Frames

1. By running **ifconfig** you can check the current MTU of the network interface.
2. Usually it is best to increase the MTU to the highest value supported by all network components which are within the transmission chain for a better performance. You can set this value permanently in the network configuration GUI (see the image below).



ifconfig - Ethernet

3. Reboot the system. Afterwards, you can verify the configured MTU by running **ifconfig** again.

The camera sets the **MTU** (p. 219) to the maximum value automatically given by the **NIC** (p. 219) or switch and supports a maximum **MTU** (p. 219) of 8K. You can manually change the network packet size the camera uses for transmitting data using this property: **"Setting ->** **Base ->** **Camera ->** **GenICam ->** **Transport Layer Control ->** **Gev Stream Channel Selector ->** **Gev SCPS Packet Size"**:

As a general rule of thumb it can be said, that the higher the **MTU** (p. 219), the better the overall performance, as less network packets are needed to transmit a full image, which results in less overhead that arises from the handling of each arriving network packet in the device driver etc. However every component involved in the data transmission (that includes every switch, router and other network component, that is installed in between the device and the system receiving the packets) must support the **MTU** (p. 219) or packet size, as otherwise the first component not supporting the packet size will silently discard all packets, that are larger than the component can handle. Thus the `weakest link` here determines to overall performance of the full system!

> **Note**
>
> There is no need to set the transfer packet size manually. By default the driver will determine the maximum possible packet size for the current network settings automatically when the device is initialised. However again the *weakest link* determines this packet size thus NICs should be chosen and configured appropriately and the same goes for switches!

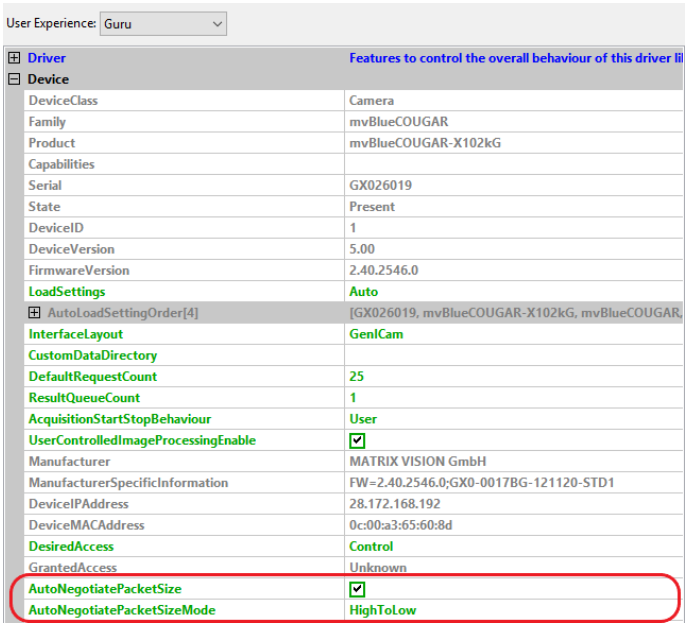
The behavior of the auto negotiation algorithm can be configured manually or to disable it completely if needed. The **AutoNegotiatePacketSize** property determines if the driver should try to find optimal settings at all and the way this is done can be influenced by the value of the **AutoNegotiatePacketSizeMode** property. The following modes for the are available:

Note

All components belonging to the network path (device, network interface card, switches, ...) will affect the MTU negotiation as every device will support the negotiated value.

Value	Description
HighToLow	The MTU is automatically negotiated starting from the NICs current MTU down to a value supported
LowToHigh	The negotiation starts with a small value and then tries larger values with each iteration until the optimal value has been found

To disable the MTU auto negotiation just set the **AutoNegotiatePacketSize** property to **"No"**.



wxPropView - MTU Auto negotiation

Note

Usually manually setting the packet size should only be done if

- auto-negotiation for some reasons doesn't work properly
- there are very good (e.g. application specific) reasons for doing so

1.9.6.2.2.1 2.1 Maximum transmission unit (MTU) / Jumbo Frames using netplan

Note

This chapter applies to Ubuntu version 17.10 and newer only.

To configure the **MTU** (p. 219) without using the NetworkManagerGUI there is as well the possibility to use netplan for configuration. The following sample configuration for netplan will illustrate how the MTU is configured to a reasonable value.

Note

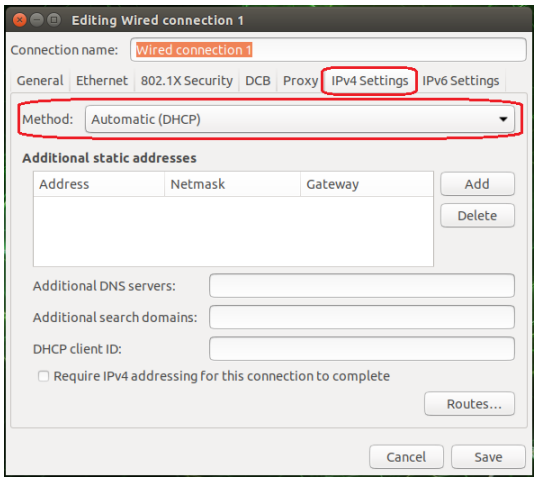
The following example assumes that the system has a network interface 'enp0s8' which needs to be configured and the renderer NetworkManager is used on the system!

```
network:
    version: 2
    renderer: NetworkManager
    ethernets:
        enp0s8:
            mtu: 8020
```

After applying the configuration using the **"netplan apply"** command, the configuration will be used.

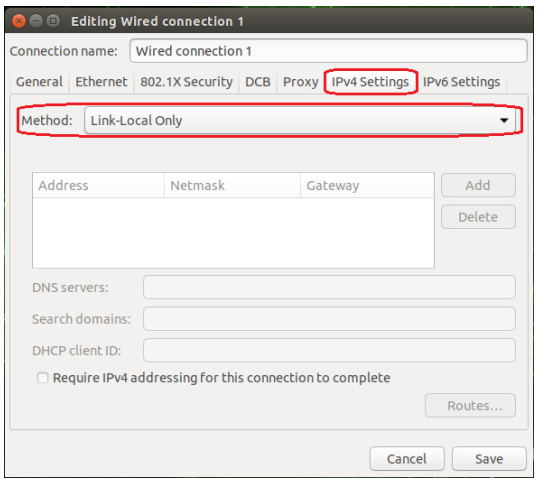
1.9.6.2.3 3. IPv4 addressing "DHCP"

Connecting the camera via DHCP (e.g. when connecting the camera through a network switch): set the IPv4 addressing method of the host's network controller as DHCP in the IPv4 configuration. Reboot the system afterwards.



ifconfig - IPv4 Settings - DHCP

"LLA (Link-Local Address)"



ifconfig - IPv4 Settings - LLA

1.9.6.2.3.1 IPv4 addressing using netplan

Note

This chapter applies to Ubuntu version 17.10 and newer only.

To configure the network interface for the correct IPv4 ranges the following samples will show different options. Depending on the existing configuration the shown samples might need to be integrated into existing configuration files.

Note

The following example assumes that the system has a network interface 'enp0s8' which needs to be configured and the renderer NetworkManager is used on the system!

The following sample shows how the NIC is configured to negotiate its IPv4 address using link-local addressing.

Configuring NIC for link-local addressing

```
network:
    version: 2
    renderer: NetworkManager
    ethernets:
        enp0s8:
            link-local: [ ipv4 ]
```

See also

https://en.wikipedia.org/wiki/Link-local_address

The following sample shows how the NIC is configured to obtain its IPv4 address from a DHCP server.

Configuring NIC for DHCP

```
network:
    version: 2
    renderer: NetworkManager
    ethernets:
        enp0s8:
            dhcp4: true
```

The following sample shows how the NIC is configured to use a static IPv4 address.

Configuring NIC with a static IP address

```
network:
    version: 2
    renderer: NetworkManager
    ethernets:
        enp0s8:
            addresses:
                - 192.168.0.2/24
```

After applying the configuration using the **"netplan apply"** command, the configuration will be used.

1.9.6.2.4 4. Link Aggregation, Teaming Or Bonding (GigE Vision Devices With Multiple NICs (mvBlue↩ COUGAR-XD)) The mvBlueCOUGAR-XD Dual-GigE camera needs a network interface card with two network interfaces which support so-called **"Link Aggregation"** or **"bonding"**.

To bond the network interface, the following description assumes that your system has three interfaces `eth0`, `eth1`, and `eth2`,

- `eth0` is the standard system interface (e.g. connected to the Internet or intranet) and
- the mvBlueCOUGAR-XD is connected to `eth1` and `eth2` (these interfaces will be bonded).

Furthermore, it is assumed that the

- *Class C* `subnet 192.168.10.1` is used for bonding, that the
- **receive buffers value was raised** (p. 58), and
- Intel network interface controller (*NIC*) is used and the Intel e1000e Kernel module is available.

1.9.6.2.5 4.1 Link Aggregation, Teaming Or Bonding Using /etc/network/interfaces

Note

If your system uses an active network manager, which assigns IP automatically, you should stop the network manager as follows:

```
sudo service network-manager stop
```

To bond the network interfaces, please follow these steps:

1. Install the package ifenslave:

   ```
   sudo apt-get install ifenslave
   ```

2. Turn the network interfaces down, which you want to bond:

   ```
   sudo ifconfig eth1 down
   sudo ifconfig eth2 down
   ```

3. Afterwards, load the bonding Kernel module:

   ```
   sudo modprobe bonding mode=0 miimon=100
   ```

   "`mode=0`" means **"round robin"** mode which corresponds to the **"static link aggregation"** on Windows systems
   "`miimon=100`" means that it will be checked every 100 ms if there are link failures in the bonding. `100` is the recommended value.

4. Now you have to define the parameters of the bonding:

   ```
   sudo ifconfig bond0 192.168.10.1 netmask 255.255.255.0 mtu 8000
   ```

5. You have to link the physical network interface controllers to the bonding and turn them on:

   ```
   sudo ifenslave bond0 eth1 eth2
   sudo ifconfig eth1 up
   sudo ifconfig eth2 up
   ```

6. Finally, you have to restart the Linux networking services:

   ```
   sudo /etc/init.d/networking restart
   ```

Now, you can check if you can ping the mvBlueCOUGAR-XD via the bonding.

1.9.6.2.6 4.2 Link Aggregation, Teaming Or Bonding Using netplan

Note

This chapter applies to Ubuntu version 17.10 and newer only.

Configuring network bonds is much more convenient using netplan. The following sample illustrates how two network interfaces can be combined to a bond.

It is assumed that the system has two network interfaces 'enp1s0f0' and 'enp1s0f1' which need to be configured to the bond and the renderer NetworkManager is used on the system. Both interfaces should not use a DHCP server to obtain their IP addresses.

Note

The interface names or renderer might be differ depending on the target systems configuration!

The configuration of the bond includes three parameters which define the following functions:

Parameter	Description
mode	Configures the bond mode to round robin balancing mode. Packets are sent or received in a sequential order from the interfaces listed.
mii-monitor-interval	Specifies the interval in which the connection state of the interface is verified. Without time suffix it will be interpreted as milliseconds.
min-links	Specifies the minimum number of links which will be necessary to consider the interface to be up.

```
network:
    version: 2
    renderer: NetworkManager
    ethernets:
        enp1s0f0:
            dhcp4: no
        enp1s0f1:
            dhcp4: no
    bonds:
        bond0:
            mtu: 8000
            link-local: [ ipv4 ]
            interfaces: [enp1s0f0, enp1s0f1]
            parameters:
                mode: balance-rr
                mii-monitor-interval: 1
                min-links: 2
```

After applying the configuration using the **"netplan apply"** command, the configuration will be used.

1.9.6.3 Multi-Core Acquisition Optimizer

The *MATRIX VISION Multi-Core Acquisition Optimizer* is a combination of device firmware and host driver features that allow to explicitly define which and how many CPU cores are used for processing the network data that is received by the host system coming from a GigE Vision™ device. It enables applications to share the load in a way the host system can deal with it best and as a side effect it allows to decouple the network data processing from the rest of the application.

The following table contains the minimum mvIMPACT Acquire driver versions needed to use the *MATRIX VISION Multi-Core Acquisition Optimizer*:

	Windows 32-bit	Windows 64-bit	Linux x86_64	Linux ARMhf	Linux ARM64
Supported	2.44.0	2.44.0	2.46.0	2.46.0	2.46.0

The following table contains the minimum firmware version for product families needed to use the *MATRIX VISION Multi-Core Acquisition Optimizer*:

Family	Minimum Firmware
mvBlueCOUGAR-X	2.44.0
mvBlueCOUGAR-XD	2.44.0
mvBlueCOUGAR-XT	2.43.0

- **Abstract** (p. 65)
- **Usage** (p. 67)
- **Use Cases** (p. 76)
- **Troubleshooting** (p. 77)

1.9.6.3.1 Abstract

1.9.6.3.1.1 Initial Situation Nowadays computer architectures usually incorporate multiple CPU cores inside a single chip or even multiple physical CPUs on a motherboard. This is why modern operating systems, SDKs used for building applications and applications themselves often use multiple threads or processes to make good use of this hardware. This is done to spread the CPU load caused by the applications equally across all the available processors to achieve optimal performance. *Optimal performance* can mean a couple of things but usually it boils down to get certain results faster than these could be generated by a single thread or to keep a graphical user interface responsive while certain tasks are being performed in the background.

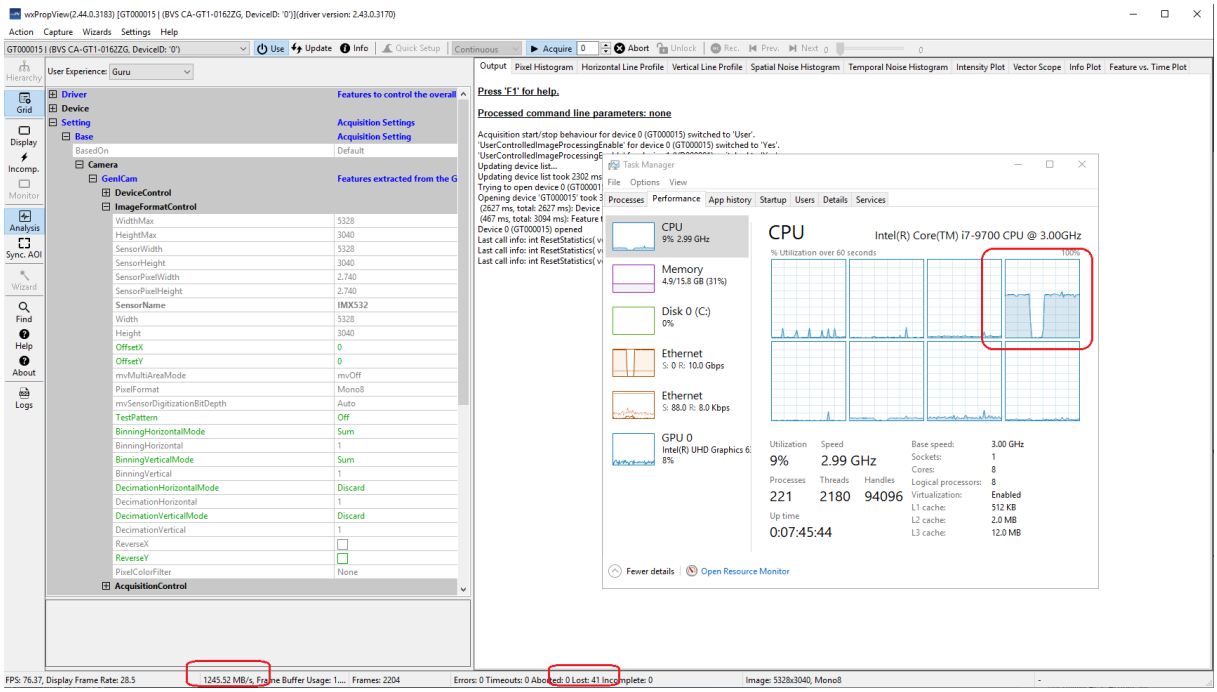
With GigE Vision™ devices streaming at bandwidths of around 5 GBit/s or higher this sometimes becomes a problem! Especially maintaining a stable data reception without losing individual images or parts of images while performing extensive number crunching in parallel might not always work as reliable as expected.

This comes due to the fact that usually a network connection is always processed on a certain CPU core. Assuming that there are a lot of parallel network connections and all these network connections have been assigned to a certain CPU and also that each individual network connection does not add significantly to the overall bandwidth consumed this makes sense: Usually an incoming network packet will generate an interrupt sooner or later (see **interrupt moderation** (p. 49) about that *sooner or later*). When this interrupt is always generated on the same CPU core this saves some time since register content etc. does not need to be transferred to a different core without generating any value while doing so and maybe more important the number of cache misses when accessing RAM is reduced. So modern operating system and network card driver developers have spent quite some development effort **NOT** to switch CPUs unless necessary.

This works well and improves system performance when the previous assumption is true, i.e. when dealing with a lot or at least multiple parallel connections that all carry more or less a similar amount of data per time slot. Now with GigE Vision™ devices this is usually not the case. Quite the opposite is true: Usually a single network stream

carries **all**, or at least a significant share, of the data that is received by a **NIC** (p. 219) in the system. This results in some CPU cores being burdened with more work than others.

For one 1 GBit/s connections in modern hardware this usually is negligible since the optimized NIC drivers and GigE Vision™ filter drivers combined with a powerful CPU are able to cope with the incoming data easily. Also multiple 1 GBit/s streams usually can be dealt with without running into trouble since the NIC driver will try to bind each connection/stream to a different CPU core at least in a pseudo-random fashion so that chances are good that all the work will be equally distributed. Sometimes (when complete control is needed/wanted) this *randomness* is not desirable and then continue to read this section makes sense for users of 1-2 GBit/s cameras. In any case 5 GBit/s or higher bandwidths coming from a single device via a single connection are much more challenging even for current CPU architectures:



wxPropView - Receiving Data From A 10 GBit/s Device

In the above image it can be seen that the CPU load caused by a 10 GigE device can no longer be ignored! The important thing to keep in mind is also, that the CPU actually processing the data cannot be known upfront. It is quite possible that it is a different one each time the acquisition is started! This depends on various things like

- the operating system
- the NIC driver and it's configuration
- the way the GigE Vision™ stream is constructed by the host driver

Now when adding a time consuming parallel algorithm to the system it usually can be observed that the operating system will assign equal chunks of work to **EVERY** CPU core in the system. Even to those processing the incoming network data. This can overload the CPU core which is quite busy already dealing with image data and can result in lost data. Even if the core is not constantly overloaded temporary problems may arise simply from the fact that every now and then the processor must perform a short time-slice of work that is **not** GigE Vision™ specific. Since data is arriving in the NIC very fast and temporary buffers available to the NIC driver are usually small (see **Optimizing Network Performance** (p. 46) for fine-tuning) switching to another task might consume just enough time to let internal buffers overflow.

1.9.6.3.1.2 Possible Optimization With the *MATRIX VISION Multi-Core Acquisition Optimizer* two things can be achieved that can result in an improved overall system stability:

- Instead of a network stream being processed by a random CPU core, the core receiving the network data can be selected explicitly allowing to use that knowledge to remove all application specific work from that particular processor. This can also be very useful for single or dual Gigabit Ethernet devices!
- The incoming network traffic from a device running a *MATRIX VISION Multi-Core Acquisition Optimizer* aware firmware can be distributed equally across a user selectable number of dedicated processors. This is usually only available for cameras delivering 5 or more GBit/s of data!

1.9.6.3.1.3 How It Works The *MATRIX VISION Multi-Core Acquisition Optimizer* makes use of modern NICs RSS (Receive Side Scaling) features and therefore requires a RSS capable network interface card to operate. Most of today's NICs have this feature however the quality of the implementation and the amount of features that can be controlled may vary across manufacturers and models.

So far good results have been achieved with the following products:

- Mellanox ConnectX series
- ASUS XG-C100C or similar products based on the former Acquantia now Marvel chipsets
- Various IOI 10 GigE NICs (some again using Acquantia, now Marvel chips)
- Intel I350 (1GBit/s)

Acceptable results have been achieved with:

- Intel X540-T2
- Intel X550-T2

1.9.6.3.2 Usage Everything regarding the RSS technology that is not covered here is beyond the scope of this documentation! More information can be found on the Internet if needed.

1.9.6.3.2.1 Network Interface Card Settings In order to use the *MATRIX VISION Multi-Core Acquisition Optimizer* first the **NIC** (p. 219) must be configured accordingly.

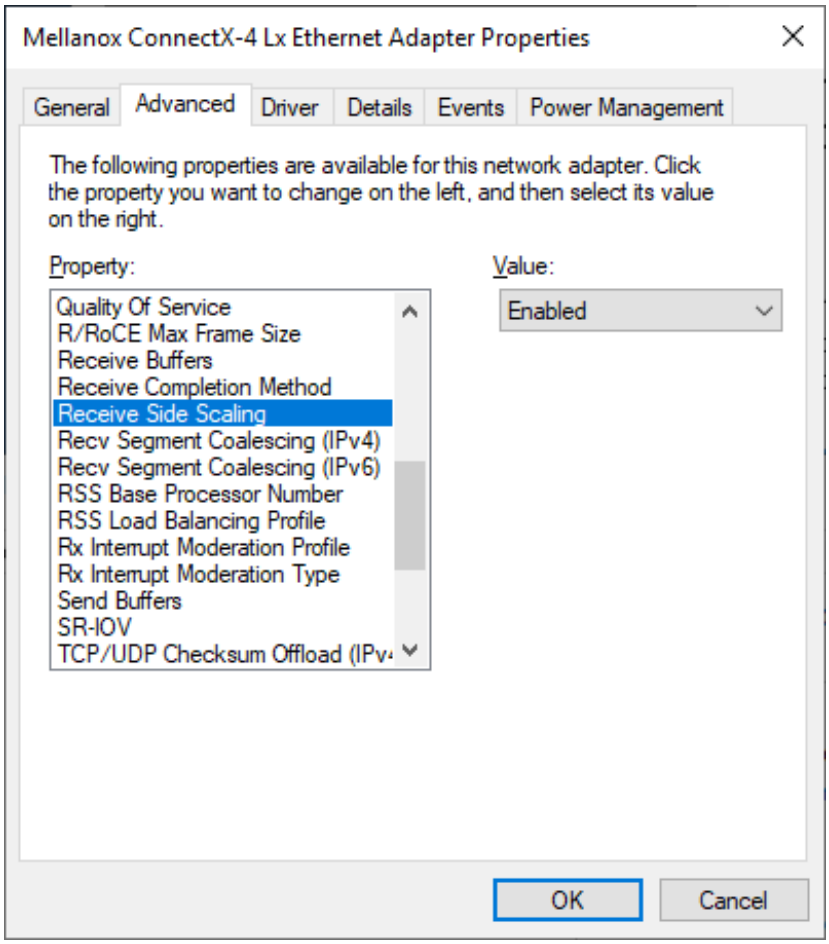
Note

- This section assumes, that all the general settings described in **Optimizing Network Performance** (p. 46) have already been applied!
- Screenshots in this section have been made on a Windows 10 system with a Mellanox ConnectX NIC. Different systems or device combinations might use slightly different terms or a slightly different (possibly reduced) feature set.
- If several NICs are involved of course each NIC needs to be configured accordingly.
- **Windows only:** If certain parameters cannot be configured by the NIC drivers property dialog using the Powershell cmdlets `Set-NetAdapterRss` and `Get-NetAdapterRss` might be used as an alternative!

1.9.6.3.2.2 Enabling RSS First of all RSS must be switched on for the NIC to use.

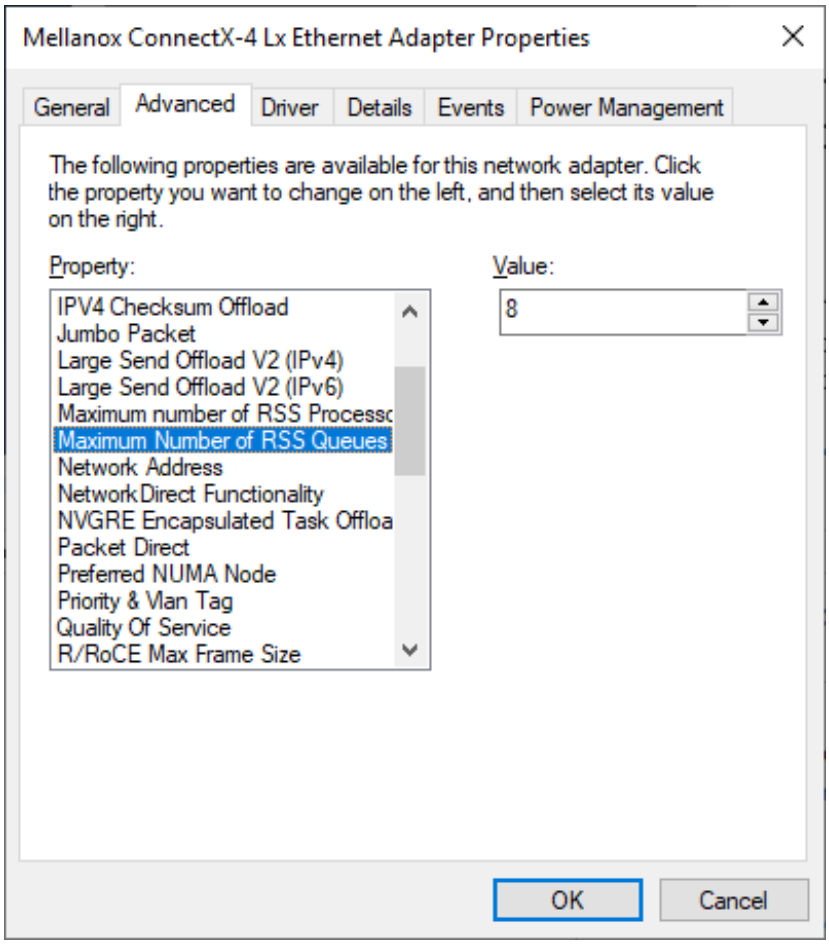
Attention

Without RSS everything described here will either not be available or will not result in any performance improvement.



NIC - RSS Enabled

1.9.6.3.2.3 RSS Queue Count The next thing that must be configured is the RSS queue count. This parameter determines the number of CPU cores that can be used to process data received **by this particular NIC**.

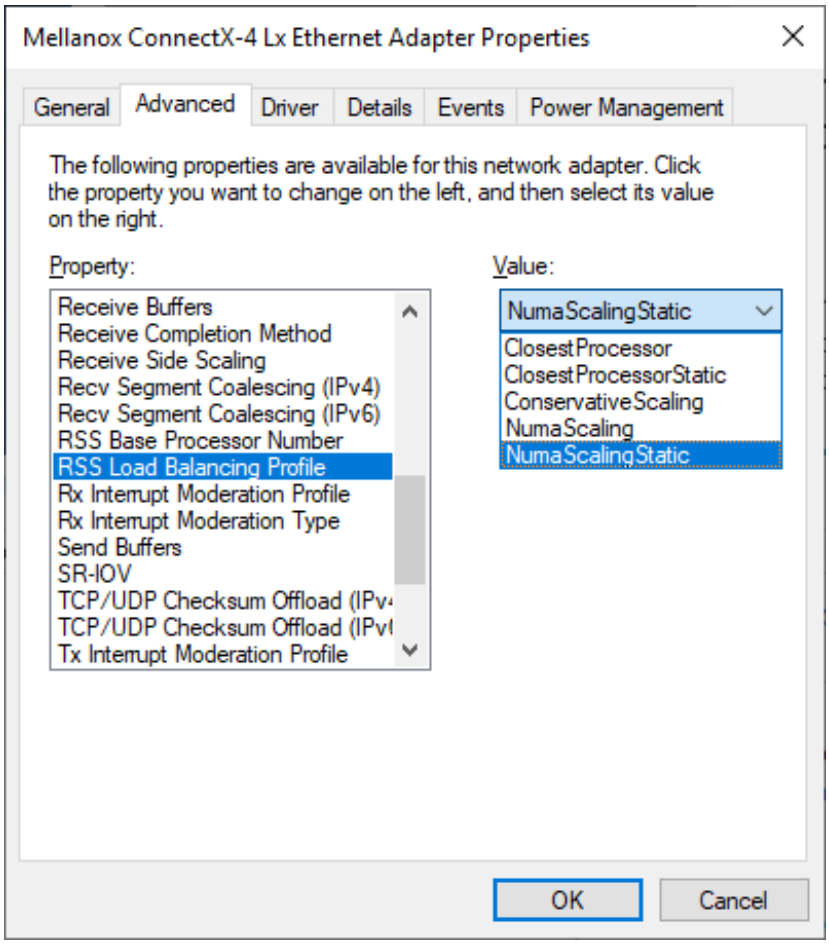


NIC - RSS Queue Count

Note

The number of RSS queues should not exceed the number of **physical** CPU cores in the system! However the operating system will usually not use more queues than processors available. So in case e.g. a system has 6 physical cores but the NIC only supports the selection of either 4 or 8 RSS queues, then 8 has to be selected to be able to access **all** physical cores in the system.

1.9.6.3.2.4 RSS Profile In order to have a deterministic system response it is recommended to select a **static** RSS profile.

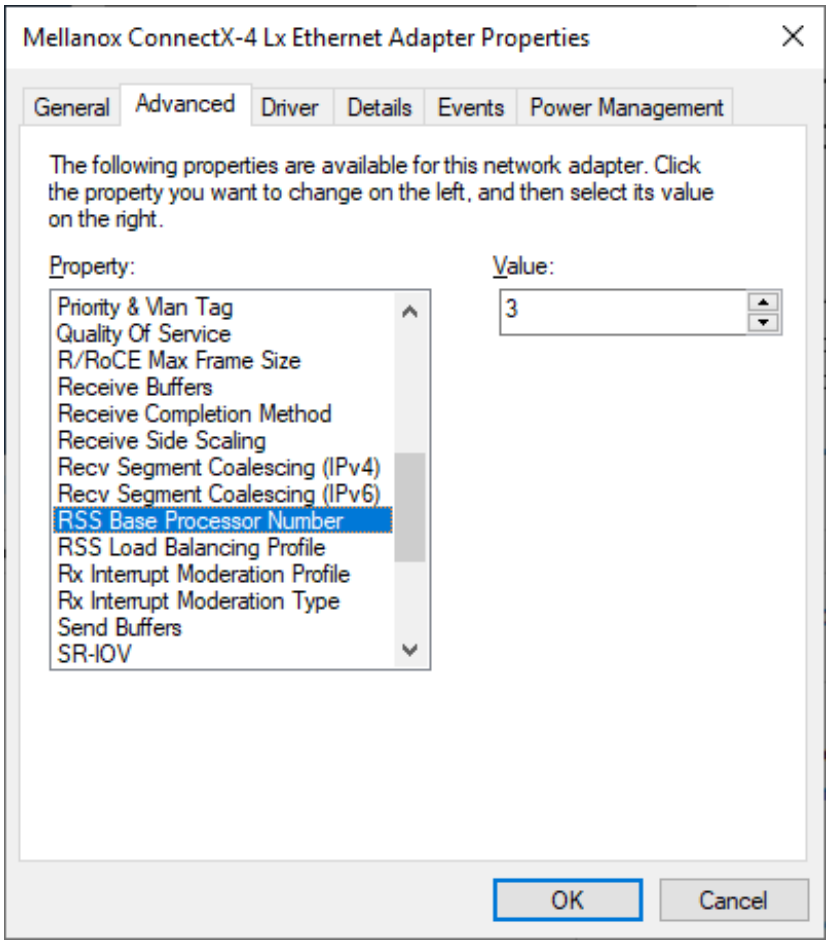


NIC - RSS Profile

Note

More on RSS profiles can be found on various locations in the Internet and is beyond the scope of this documentation. Selecting a `dynamic` profile here will allow the operating system to change certain settings at runtime thus to bypass the fixed assignment of CPU cores and thus eventually to jeopardize the complete setup!

1.9.6.3.2.5 RSS Base Processor (Optional) If supported by the NIC driver, the RSS base processor value can be configured to exclude every physical CPU core with an index smaller than this properties value from processing network data at all.



NIC - RSS Base Processor

Usually this does not result in a huge advantage since the same thing can be achieved by later configuring the mvIMPACT Acquire driver instance accordingly. But setting the RSS base processor does a little more: Since then **ALL** network data processing is removed from some of the CPU cores this affects not only GigE Vision™ data but **ALL** network data. Sometimes this might make a difference.

Attention

When combining this value with the **RSS Queue Count** (p. 68), the sum of the number of queues and the base processor cannot exceed the number of physical CPU cores in the system for the same reasons as mentioned previously!

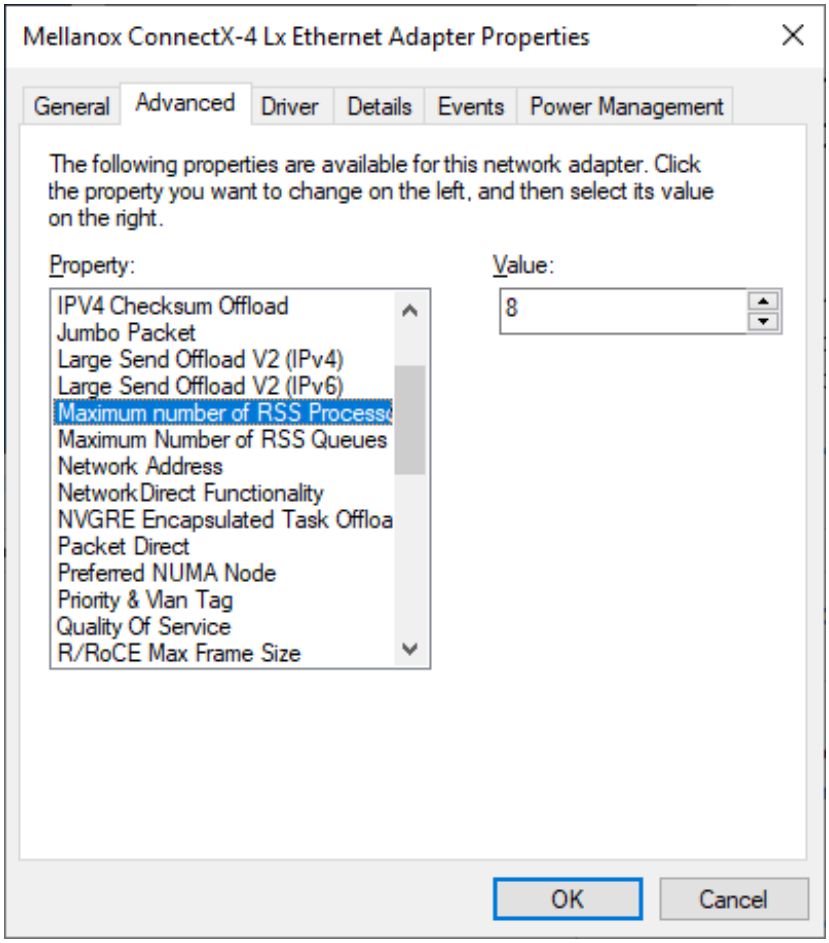
Note

An interesting use case for this parameter are setups with multiple NICs as then e.g. when working with 2 NICs in an 8 core system the following setup will decouple the CPU core usage of the NICs:

1. Enable RSS for both NICs
2. Set the RSS Queue count for both NICs to 4
3. Set the RSS Base Processor for **one** of the NICs to 4

Now one NIC will operate on CPUs 0-3 while the other one will operate on CPUs 4-7.

1.9.6.3.2.6 Maximum RSS Processor Count (Optional)



NIC - Maximum RSS Processor Count

Note

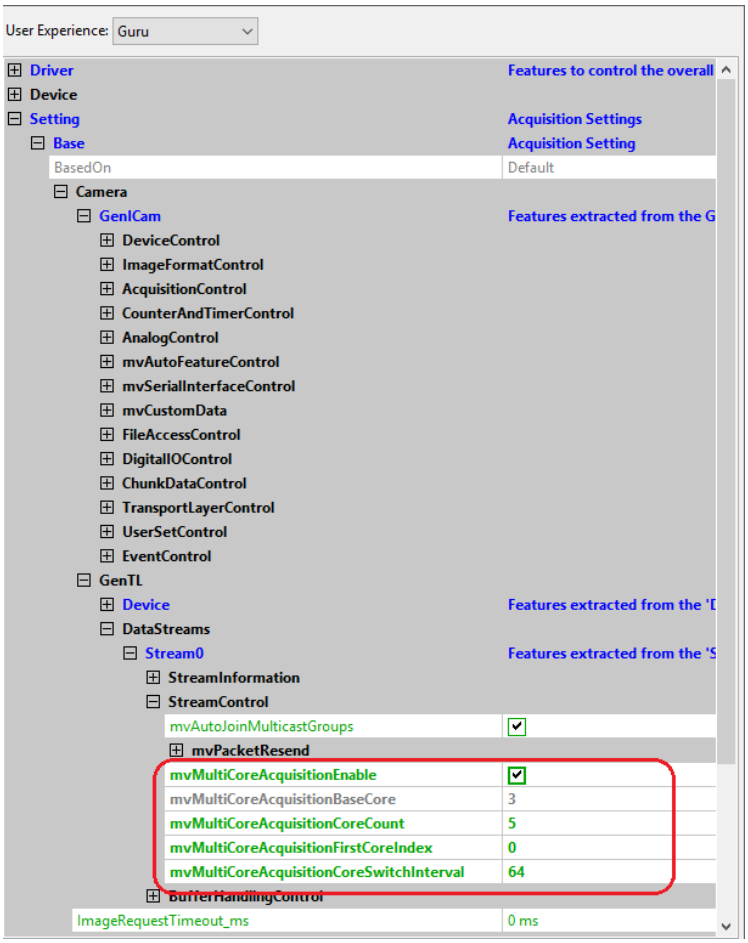
Changing this value usually requires a reboot of the system!

Attention

This value must not be higher than the number of physical CPU cores that can process network data thus the value of the physical processors minus the RSS base processor. Otherwise the system will perform in a nondeterministic way!

1.9.6.3.2.7 BIOS Settings (Optional) Since RSS works on physical processors only, it might be beneficial for the overall system performance to turn off Hyper-Threading in the systems BIOS or somewhere in the operating systems settings, if possible. This is an optional setting that should be considered carefully as, while potentially improving the network data reception, it might have negative impact on the systems capability to perform extensive calculations!

1.9.6.3.2.8 mvIMPACT Acquire Settings Once the NIC and the host system have been configured accordingly, the mvIMPACT Acquire related features can be found in the DataStream module related feature tree:



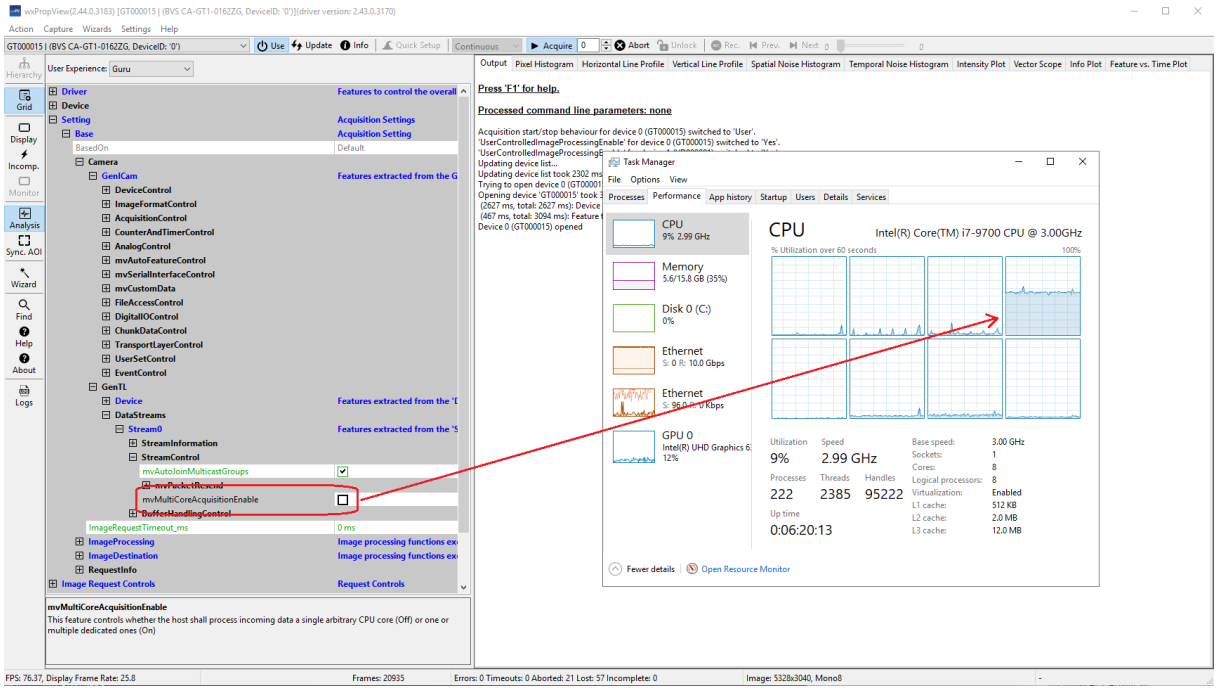
wxPropView - Multi-Core Acquisition Optimizer - Features

More information about the concept of the `DataStreamModule` can be found in the GenICam GenTL specification and the mvIMPACT Acquire API manual of the programming language of your choice.

If the **mvMultiCoreAcquisitionEnable** feature is not available either the device doesn't support the minimal feature set for running the *MATRIX VISION Multi-Core Acquisition Optimizer* or the NIC is not set up correctly.

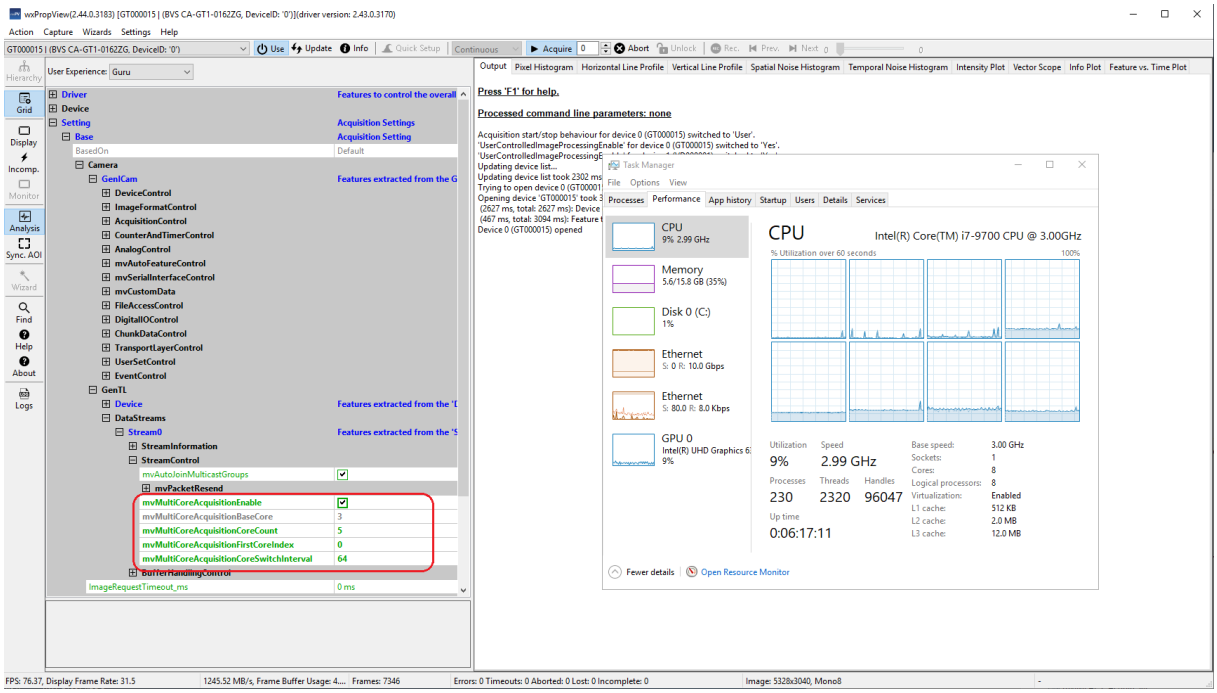
- See **here** (p. 64) for supported products and versions.
- See **here** (p. 77) for potential solutions if your device should support all features needed.

When **mvMultiCoreAcquisitionEnable** is switched off everything will behave as if the feature is not available at all. The incoming data will be processed exclusively on a random CPU potentially changing with every start of the acquisition:



wxPropView - Multi-Core Acquisition Optimizer - Off

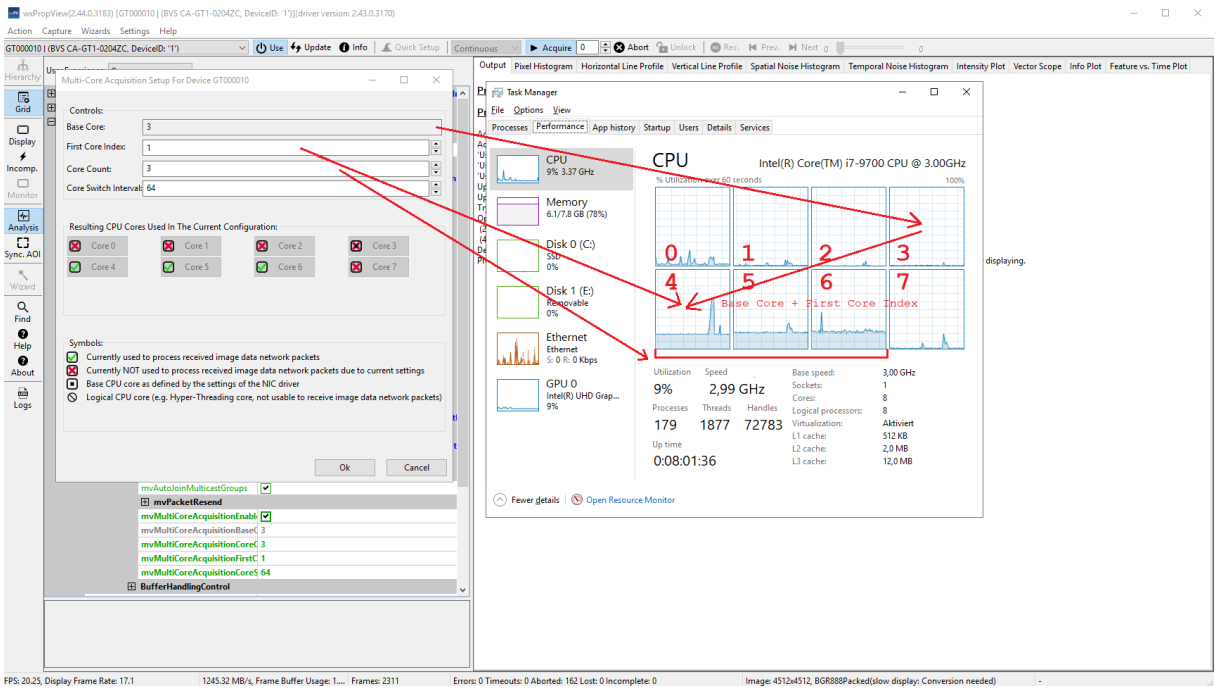
Switching **mvMultiCoreAcquisitionEnable** On will enable various other properties allowing the configuration of the CPU cores to use:



wxPropView - Multi-Core Acquisition Optimizer - On

As it can be seen in this example 5 CPU cores are now used for processing the network data. Each of them gets an equal share of the load while the overall system utilization stays the same. The first 3 CPU cores do not get any work due to the RSS base processor being set to 3 in the NICs properties (see **RSS Base Processor** (p. 70)).

wxPropView also includes a wizard that allows an easier setup of all the features involved:



wxPropView - Multi-Core Acquisition Optimizer - Wizard

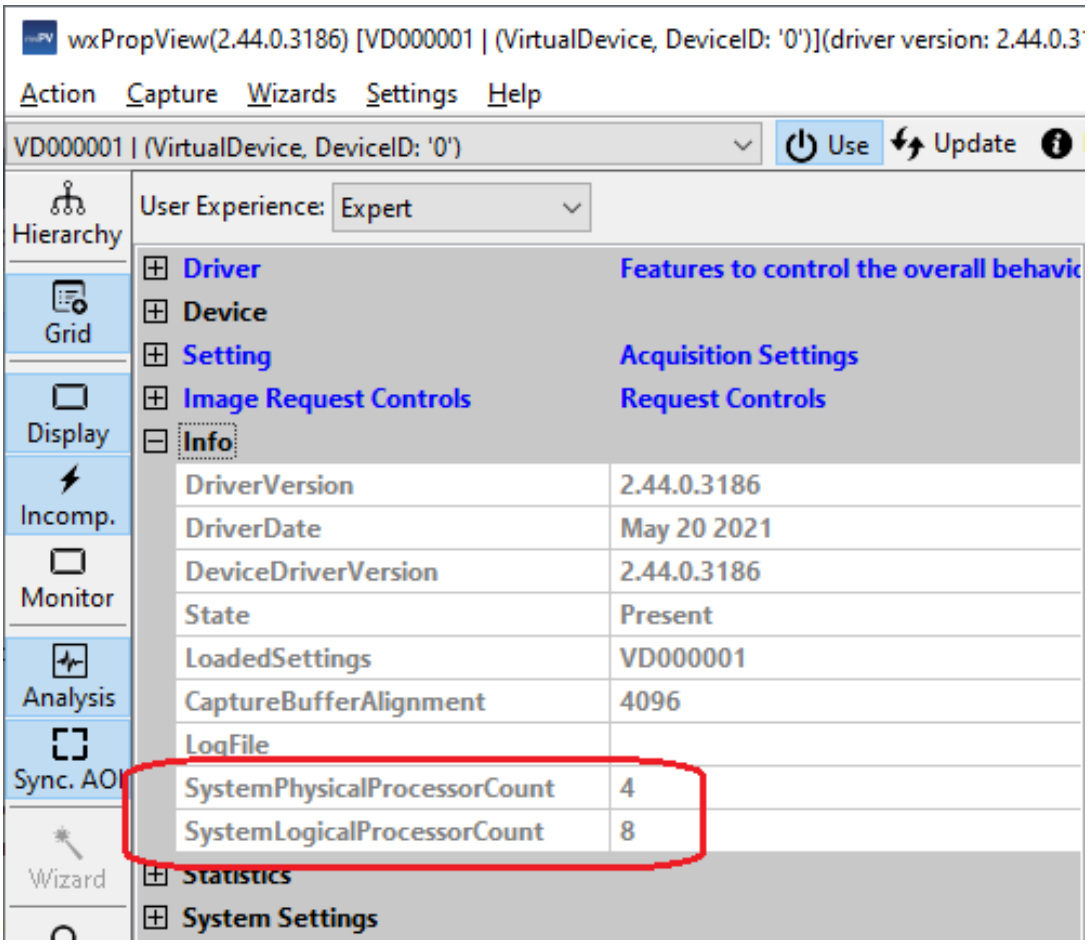
1.9.6.3.2.9 API In total the mvIMPACT Acquire driver stack exposes 5 properties that in combination with the settings described in **Network Interface Card Settings** (p. 67) can be used to configure the desired behaviour:

- **mvMultiCoreAcquisitionEnable:** Switch the *MATRIX VISION Multi-Core Acquisition Optimizer* on and off
- **mvMultiCoreAcquisitionBaseCore:** This is a read-only feature basically just reflecting the **RSS Base Processor** (p. 70) value configured for the NIC in case this is either not exposed by the NIC driver or shall be used by an application. This is purely meant for informational purposes.
- **mvMultiCoreAcquisitionCoreCount:** Defines the number of CPU cores to use for processing this data stream. Please note that single or dual Gigabit Ethernet cameras usually just allow to explicitly select the receiving CPU but not to split up the work onto multiple CPUs, thus the maximum value for this feature will be 1 then!
- **mvMultiCoreAcquisitionFirstCoreIndex:** Defines which CPU core is the first one to use for processing this data stream relative to **mvMultiCoreAcquisitionBaseCore**.
- **mvMultiCoreAcquisitionCoreSwitchInterval:** Defines the number of network packets to process on one CPU core before switching to the next in a round-robin fashion (only available when multiple CPU cores are used for processing this data stream thus when **mvMultiCoreAcquisitionCoreCount** is larger than 1)

Attention

Modifying these properties while streaming is active will not affect the current behavior until streaming has been stopped and restarted!

In addition to that mvIMPACT Acquire also allows to get information about the CPU cores available in the system via the **Info** category:



wxPropView - Available CPU cores

These features are available within an application either by creating an instance of the `Info` class or (when using the C-API) by obtaining handles to the properties in question.

1.9.6.3.3 Use Cases

Note

The best effect with any of the use cases described in this section will be achieved when removing all work caused by the application from the CPU cores used for processing network data. This can be done using various platform specific functions. As a hint for further reading please refer to the function `SetProcessAffinityMask` for Windows or `sched_setaffinity` for Linux based systems. Unfortunately there is no platform agnostic way to achieve this. The driver specific setup for this use case would be **Bind Network Data Processing Of A Single Camera To A Dedicated Core** (p. 77).

1.9.6.3.3.1 Distribute Network Data Processing Of A Single Camera To Multiple CPU Cores This probably is the easiest way to make use of the *MATRIX VISION Multi-Core Acquisition Optimizer*: For example assume a device that transfers Bayer raw data at a high data rate. Without the *MATRIX VISION Multi-Core Acquisition Optimizer* the raw network data will be processed by a single, arbitrary CPU core on the host system. If a de-Bayer algorithm runs in parallel on every core in the same system to generate color images out of that data stream, then data might get lost as soon as the core doing de-Bayering AND network data processing gets overloaded, even if the other CPU cores still have resources available.

Setting up the *MATRIX VISION Multi-Core Acquisition Optimizer* in a way that distributes the load caused by the network traffic processing equally among multiple CPU cores can then improve the stability as well as the overall data throughput of the application.

To achieve this:

1. Enable **mvMultiCoreAcquisitionEnable**
2. Set **mvMultiCoreAcquisitionCoreCount** to the desired number of CPU cores
3. Optional: Set **mvMultiCoreAcquisitionFirstCoreIndex** to the desired value
4. Optional: Set **mvMultiCoreAcquisitionCoreSwitchInterval** to a different, more suitable value if necessary

If possible **Bind Network Data Processing Of A Single Camera To A Dedicated Core** (p. 77) should be used instead of this operation mode since it runs more reliable and with less overhead.

1.9.6.3.3.2 Bind Network Data Processing Of A Single Camera To A Dedicated Core This is an optimized version of **Distribute Network Data Processing Of A Single Camera To Multiple CPU Cores** (p. 76). It should be used in favor of using **Distribute Network Data Processing Of A Single Camera To Multiple CPU Cores** (p. 76) but requires that the process later doing the de-Bayering or any other number crunching can be configured accordingly by setting the processor affinity as described earlier.

This allows to fully decouple data reception and processing and in addition to that the CPU core switches while processing the incoming data are not needed thus the overhead otherwise introduced can be avoided while keeping the benefits of decoupling application and data reception and improved stability.

To achieve this:

1. Enable **mvMultiCoreAcquisitionEnable**
2. Set **mvMultiCoreAcquisitionCoreCount** to 1
3. Set **mvMultiCoreAcquisitionFirstCoreIndex** to a value that results in the desired CPU core being utilized. Keep the potential effect of the **RSS Base Processor** (p. 70) in mind.

1.9.6.3.3.3 Bind Network Data Processing Of Multiple Cameras To Dedicated Cores This is an advanced version of any of the 2 scenarios described above! In any case the CPU core assignment should be done in a way that each camera gets exclusive access to a certain core, so 2 cameras should never operate on the same core! If the applications processor affinity can **not** be modified then e.g. when working with 2 cameras the lower half of cores should be assigned to one camera and the upper half of cores to the other camera.

Better overall performance again can be achieved when exclusively reserving as many cores per camera as needed for reliable data capture and by assigning the remaining cores to the application. For 1 GBit/s devices it may even turn out to be beneficial to bind ALL cameras to a single CPU while running the application on the remaining CPUs (if all cameras are connected to the same NIC, probably a 10 GBit/s one).

The setup is the same as for this use case is obviously the same as for **Bind Network Data Processing Of A Single Camera To A Dedicated Core** (p. 77) (for the 2nd use case) but with the **mvMultiCoreAcquisitionFirstCoreIndex** value being different for each camera.

1.9.6.3.4 Troubleshooting

- **None Of The Multi-Core Acquisition Optimizer Properties Are Available** (p. 77)
- **Data Is Lost / Images Are Returned Incompletely** (p. 78)

1.9.6.3.4.1 None Of The Multi-Core Acquisition Optimizer Properties Are Available

1.9.6.3.4.2 Symptoms None of the properties described **here** (p. 75) are available for a GigE Vision™ camera the application shall work with.

1.9.6.3.4.3 Cause Potential reasons for this include:

1. The camera doesn't support certain special features needed for the *MATRIX VISION Multi-Core Acquisition Optimizer* to operate or the mvIMPACT Acquire driver doesn't support the feature. See **here** (p. 64) for supported products and versions.
2. The NIC doesn't support RSS.
3. RSS features on the NIC have not been configured properly.

1.9.6.3.4.4 Resolution

- Make sure the camera supports the required features and if needed update the firmware of the device to a suitable version.
- Update mvIMPACT Acquire to a suitable version.
- Check for updated NIC drivers.
- Refer to **Network Interface Card Settings** (p. 67).
- Check the log-files or provide the combined with information about what has been configured on the NIC as well as detailed information about the NIC and the used NIC driver and everything else that might help to the MATRIX VISION support team using our Service Desk at `https://servicedesk.matrix-vision.com`.

1.9.6.3.4.5 Data Is Lost / Images Are Returned Incompletely

1.9.6.3.4.6 Symptoms While acquiring data from the device image data is lost e.g. the counter for incomplete or lost images is increasing or request objects inside an application are returned with errors.

1.9.6.3.4.7 Cause There are **A LOT** of potential causes for this so a general cause as well as a general resolution cannot be given:

- Switches, routers or cables in between the device and the NIC of the host system might simply not be able to cope with the amount of data coming from the device.
- The NIC might not be able to get rid of the incoming data fast enough.
- The underlying driver might not be fast enough to send the captured data up to the application.
- The application might not process the received data fast enough eventually blocking all the memory that could be used for capturing data into thus the driver must drop incoming data.

1.9.6.3.4.8 Resolution

1.9.6.3.4.9 General General information about potential resolutions for network related transmission problems that are not specific to the *MATRIX VISION Multi-Core Acquisition Optimizer* can be found **here** (p. 199).

1.9.6.3.4.10 Multi-Core Acquisition Optimizer Specific The *MATRIX VISION Multi-Core Acquisition Optimizer* makes heavy use of the RSS features offered by NICs. Especially the combination of the **mvMultiCore↩ AcquisitionCoreCount** and the **mvMultiCoreAcquisitionCoreSwitchInterval** needs to be mentioned here. Internally these 2 properties will configure a connected device in a way that every *mvMultiCoreAcquisitionCoreSwitch↩ Interval network packets* sent out by the device, the packets will be modified in a way that, when received, they will be processed by another CPU. Only **mvMultiCoreAcquisitionCoreCount** different CPU cores will be used by the algorithm.

Note

This applies to the NIC configuration described **here** (p. 67): If possible do not configure more **RSS queues** (p. 68) than physical processors available! While this *should* work in case something doesn't behave as expected go with a smaller value at least to rule out this potential source of trouble. Keep in mind the potential impact of the **RSS Base Processor** (p. 70) as well!

Be aware of the impact of using multiple CPUs for receiving data! If possible always favor using a single, dedicated CPU core for processing the data stream of a single camera while moving the load of the application away from this core. If dealing with multiple cameras try to configure the system in a way that each camera uses its own, dedicated core to transmit network data to while the application uses the remaining cores. This will result in the best performance and in that case the parameter **mvMultiCoreAcquisitionCoreSwitchInterval** is not important since CPU switching for an individual network stream is not necessary.

If this is not possible for whatever reason try to find a good balance between CPU cores used and CPU core switch interval! As described **here** (p. 64) a network stream is usually bound to a specific CPU anyway! This is done for performance reasons, as switching from one core to another consumes additional CPU cycles and thus is avoided by the operating system. So regarding this troubleshooting section be aware of the fact that the smaller the value of the property **mvMultiCoreAcquisitionCoreSwitchInterval** is selected the higher the additional overhead will become. Extensive testing did show that values around 64 - 128 result in a good compromise between improved reliability and introduced overhead which becomes almost undetectable then.

Switching the CPU core every now and then in combination with parallel processors however introduces another aspect to the incoming traffic usually only encountered with LAG (Link Aggregation) configurations: It is no longer guaranteed that the driver receives the network packets in order! So a driver needs to be aware of this!

Also selecting a lot of CPU cores for processing with a large number (256 or greater) for the switch interval will reduce the overhead to a minimum but might cause the packets to arrive way out of order since several RSS queues might raise interrupts and the order these are served is not defined.

When using a very small value for **mvMultiCoreAcquisitionCoreSwitchInterval** in combination with an **interrupt moderation** (p. 49) scheme that results in a very low number of interrupts might also have a negative effect as then all processors will wait until their queues are almost full until signaling an interrupt. With a small switch interval this will likely happen when all queues are almost full and then it might happen that certain queues do overrun since every core wants to process data at the same time. Another aspect of this is, that the number of RSS queues divided by the number of receive descriptors allocated by the NIC should always be smaller than the value for **mv↩ MultiCoreAcquisitionCoreSwitchInterval** since otherwise the card might actually run out of receive descriptors before getting rid of its data.

Note

A receive descriptor typically carries 2k bytes while the actual network packets might be larger thus a network packet might consume more than one receive descriptor! This again is another argument for **mvMultiCoreAcquisitionCoreSwitchInterval** not being selected too large.

1.9.7 Using GigE Vision™ Devices In A Docker Container

When developing machine vision applications using Docker containers, chances are that you would like to access GigE Vision™ devices inside the container. With the mvIMPACT Acquire driver stack this can be achieved fairly easy and this chapter will show you how to build a basic Docker container where you can use GigE Vision™ devices. The current sample Docker container runs on a native Linux machine.

Note

The following chapter is documented only for a native Linux host system.

1.9.7.1 Host Preparation

Note

For this demo Docker container the operating system of the host machine is Linux.

The host **network interface card** (p. 219) has to be first configured for an optimized GigE Vision™ transmission: **Checklist for Linux** (p. 57)

1.9.7.2 Building A Docker Image

The following demo Dockerfile builds a basic Docker image based on a slim version of Debian, where the mv↩ IMPACT Acquire GenTL driver package and its sample programs are installed. This Dockerfile can be used in many ways:

- Use it directly to test your device in a Docker container.
- Use it as a base image for your device applications.
- Use it as an inspiration for building your own Dockerfile.

Before building the Dockerfile, please download the mvIMPACT Acquire GenTL driver installation files from MATRIX VISION GmbH website (`https://www.matrix-vision.com/treiber-software.html`) (user login is required):

- The installation script: *install_mvGenTL_Acquire.sh*
- The installation package: *mvGenTL_Acquire-x86_64_ABI2-*.tgz* (* should be replaced by the version number)

Create a directory called *mvIMPACT_Acquire* (as used in this demo Dockerfile) and move both installation files into this directory. In this example, both files are downloaded into the *Downloads* directory and the *mvIMPACT_Acquire* directory is created inside the *Downloads* directory:

```
$ cd ~/Downloads
$ mkdir mvIMPACT_Acquire
$ mv install_mvGenTL_Acquire.sh mvGenTL_Acquire-x86_64_ABI2-*.tgz mvIMPACT_Acquire/
```

Make the installation script *install_mvGenTL_Acquire.sh* executable:

```
$ cd mvIMPACT_Acquire
$ chmod a+x install_mvGenTL_Acquire.sh
```

Navigate back into the directory where *mvIMPACT_Acquire* resides (e.g. *Downloads*) and create your Dockerfile:

```
$ cd ~/Downloads
$ touch Dockerfile
```

Create the content of your Dockerfile. Our demo Dockerfile (for Linux x86_64) looks as follows:

```
# start with slim version of actual Debian
FROM debian:9-slim

ENV LC_ALL C
ENV DEBIAN_FRONTEND noninteractive

# entrypoint of Docker
CMD ["/bin/bash"]

# set environment variables
ENV TERM linux
ENV MVIMPACT_ACQUIRE_DIR /opt/mvIMPACT_Acquire
ENV MVIMPACT_ACQUIRE_DATA_DIR /opt/mvIMPACT_Acquire/data
ENV GENICAM_GENTL64_PATH /opt/mvIMPACT_Acquire/lib/x86_64
ENV GENICAM_ROOT /opt/mvIMPACT_Acquire/runtime
ENV container docker

# update packets and install minimal requirements
# after installation it will clean apt packet cache
RUN apt-get update && apt-get -y install build-essential && \
    iproute2 && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*

# move the directory mvIMPACT_Acquire with *.tgz and *.sh files to the container
COPY mvIMPACT_Acquire /var/lib/mvIMPACT_Acquire

# execute the setup script in an unattended mode
RUN cd /var/lib/mvIMPACT_Acquire && \
    ./install_mvGenTL_Acquire.sh -u && \
    rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
```

Note

In case of ARM architectures, all occurrences of "x86_64" in this demo dockerfile have to be replaced by the correct platform e.g. "arm64" and the install script to use will be *install_mvGenTL_Acquire_ARM.sh* then.

Finally, build a Docker image using this Dockerfile:

```
$ sudo docker build -t [image_name] .
```

Note

Please make sure to call *docker build* from within the directory where the Dockerfile resides. Note that Internet access is required for the *docker build*.

If built successfully, you will be able to see [image_name] being listed when calling:

```
$ sudo docker images
```

1.9.7.3 Starting The Docker Container

Though the Docker container is isolated from the host system, we can connect the container to the host network by using *–net=host* flag when starting the container:

```
$ sudo docker run -ti --net=host [image_name] /bin/bash
```

1.9.7.4 Validation

After starting the container the correct operation of GigE Vision™ devices can be validated by running one of the sample programs provided by the mvIMPACT Acquire (e.g. *SingleCapture*):

```
$ cd /opt/mvIMPACT_Acquire/apps/SingleCapture/x86_64
$ ./SingleCapture
```

If the attached GigE Vision™ device appears in the device list of the program's output, congratulations, you've managed to access GigE Vision™ devices in the container by using the mvIMPACT Acquire. Now you can use them inside the Docker container for your machine vision applications.

1.10 Technical Data

1.10.1 mvBlueCOUGAR-X / -POE

1.10.1.1 Dimensions

1.10.1.1.1 Standard model (-xx1x)

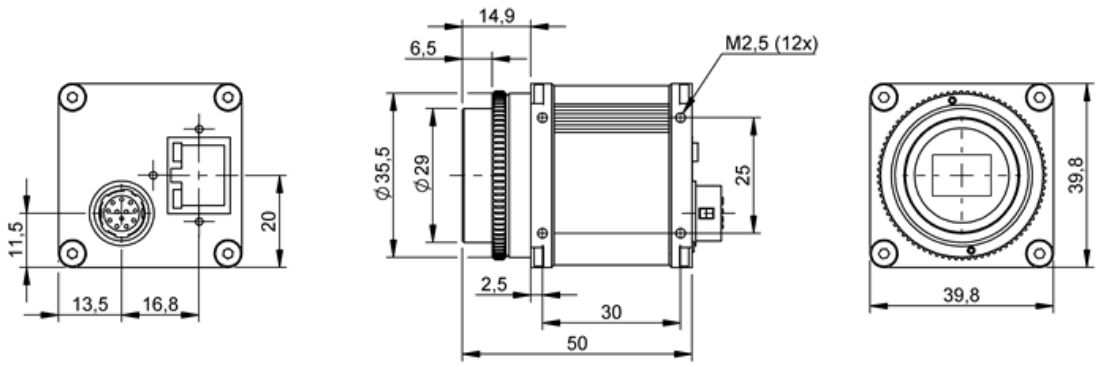


Figure 1: Connectors mvBlueCOUGAR-X

mvBlueCOUGAR-X	
Size of body (w x h x l)	39.8 x 39.8 x X mm

	C-Mount	CS-Mount
X	50	45

Lens protrusion	C-Mount	CS-Mount
	8 mm (9.5 mm with max. Ø 20 mm)	6 mm

1.10.1.1.2 Model without housing (-xx9x) (on demand)

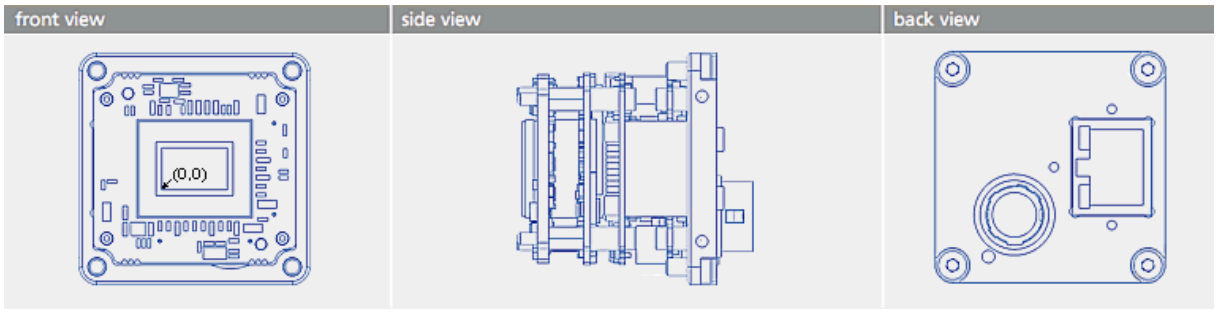


Figure 2: mvBlueCOUGAR-X without housing

1.10.1.1.3 Tripod adapter

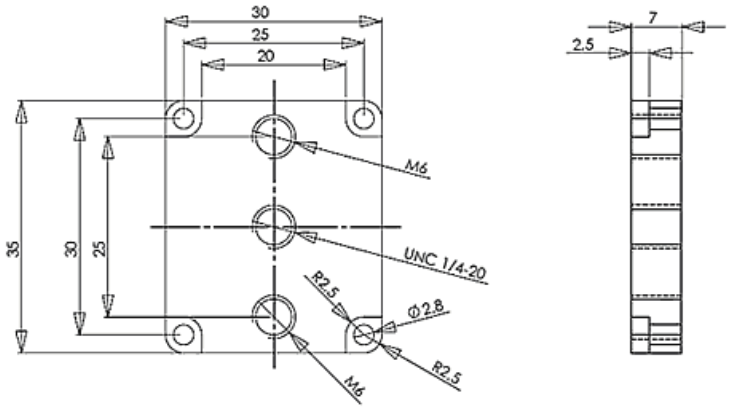


Figure 3: Dimensional drawing of tripod adapter

1.10.1.2 Connectors

1.10.1.2.1 Circular connector male

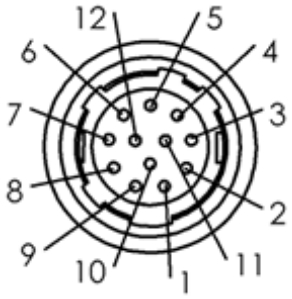


Figure 4: 12-pin (male; top view), digital I/O, power; orientation mvBlueCOUGAR-X

Pin.	Signal (Standard version)	Signal (**-POE** (p. 220) option, not available for all products)	Digital I/O color code of power supply MV-DC1201 BCSX IO; Rev. 2
1	GND (for PWR_IN)	GND (for PWR_IN)[1]	gray-pink (not available with Rev. smaller than 2)
2	**PWR_IN** (p. 114)	**PWR_IN** (p. 114)[1]	
3	DigOut3 (**wxPropView** (p. 139) numbering: line3)	Reserved (do not use)	white
4	Opto DigIn0 (line4)	Opto DigIn0 (line4)	brown
5	DigOut2 (line2)	N.C.	green
6	DigOut0 (line0)	Opto DigOut0 (line0)	yellow
7	Opto DigIn_GND	Opto DigIn_GND	gray
8	RS232_RX	RS232_RX	pink
9	RS232_TX	RS232_TX	blue
10	DigOut_PWR_IN (12 to 24 V supply for the outputs)	Opto DigOut_PWR_IN (12 to 24 V supply for the outputs)	red (using MV-DC1201 BCSX IO: you have to add supply for the outputs)
11	Opto DigIn1 (line5)	Opto DigIn1 (line5)	black
12	DigOut1 (line1)	Opto DigOut1 (line1)	violet

[1] Using POE and an external power supply at the same time, the external power supply will be treated prioritized.

Connector (camera side): SAMWOO SNH-10-12 (RPCB) or equivalent
Plug (matching cable plug): Hirose HR10A-10P-12S (01) or equivalent

Pinning of cable KS-BCX-HR12

Pin.	CON 1 mvBlueCOUGAR-X / -XD | Signal	CON 2 open ended cable | Color
1	GND (for PWR_IN)	black
2	**PWR_IN** (p. 114)	brown
3	DigOut3 12 V to 24 V (**wxPropView** (p. 139) numbering: line3)	red
4	Opto DigIn0 3.3 V to 24 V (line4)	orange
5	DigOut2 (line2)	yellow
6	DigOut0 (line0)	green
7	Opto DigIn_GND	blue
8	RS232_RX	violet
9	RS232_TX	gray
10	DigOut_PWR_IN (12 to 24 V supply for the outputs)	white
11	Opto DigIn1 3.3 V to 24 V (line5)	white-black
12	Opto DigOut1 12 V to 24 V (line1)	white-brown
	Main connector shield	main shield

Color assignment following international code for UL wiring.

Characteristics of the digital inputs

Delay

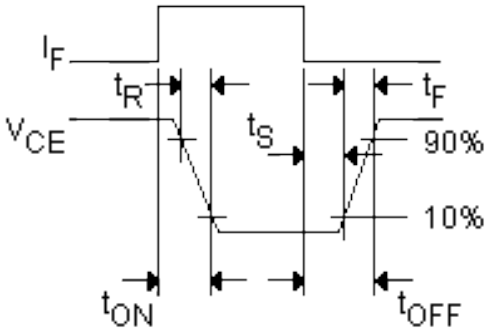


Figure 5: Input switching times

Characteristics	Symbol	Test conditions	Typ.	Unit
Minimum trigger pulse width			5	
Turn-On time	t_{ON}		10	
Storage time	t_S	R = 2 kOhm (Figure 7), internal output voltage 5 V, I_F = 16.25 mA	25	us
Turn-Off time	t_{OFF}		40	

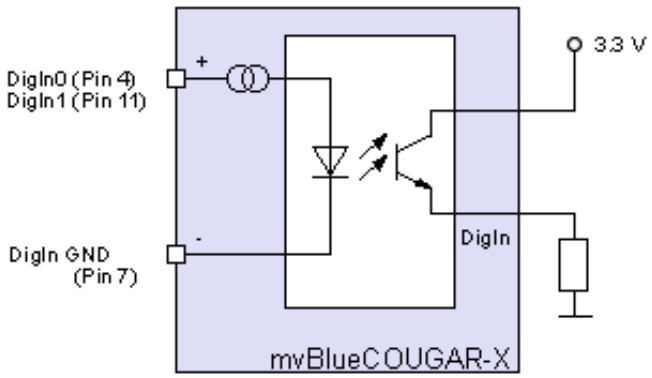
	Standard and **-POE** (p. 220) option
Notes	
High level	+3 to +24 V
Low level	0 to +0.3 V
Threshold (Low --> High / High --> Low)	2.5 V +- 0.5 V
I_{max}	5 mA
	

Figure 6: DigIn mvBlueCOUGAR-X (standard)

	PLC option (inputs with PLC levels only)

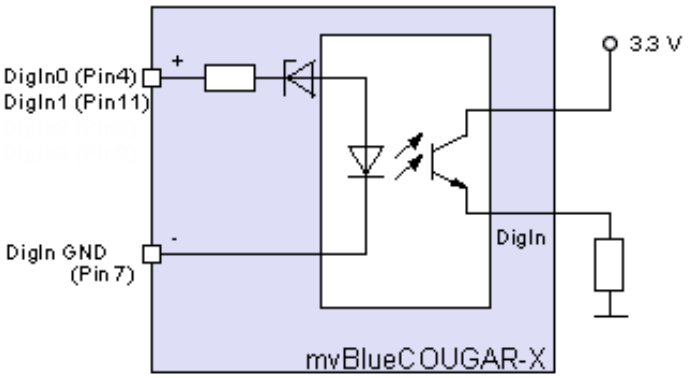
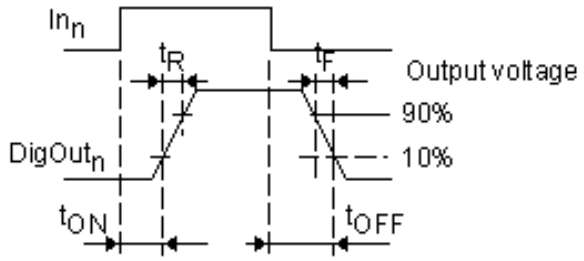
Notes	Not available in combination with **-POE** (p. 220) option. Inputs with highest tolerance against interferences.
High level	+11 to +24 V
Low level	0 to +8 V
Threshold (Low --> High / High --> Low)	10 V +- 1 V
I_{max}	16 mA
	

Figure 7: DigIn mvBlueCOUGAR-X (PLC option)

Characteristics of the digital outputs

	Standard version
Delay	
	Figure 8: Output switching times (standard)
Characteristics	

	Symbol	Typ.	Unit
Turn-On time	t_{ON}	20	us
Rise time	t_R	10	
Fall time	t_F	8	
Turn-Off time	t_{OFF}	30	

High level		+12.0 V to +24 V
Low level		max. +1 V
	I_{out}	0.7 A static load 1.3 A dynamic load

Notes	Each output has a short circuit protection between 1 A and 1.7 A (generally 1.3 A). So if you combine two outputs with one load, the short circuit protection can have an effect.

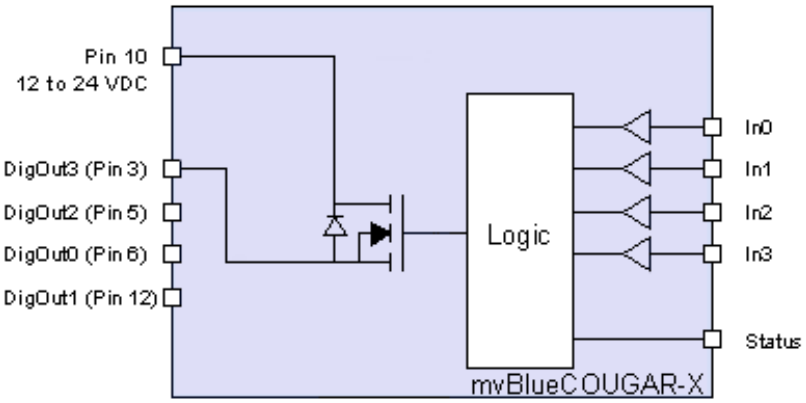


Figure 9: DigOut mvBlueCOUGAR-X

Features	- undervoltage detection - current limitation - overtemperature protection - short circuit protection

	-POE (p. 220) option
Delay	

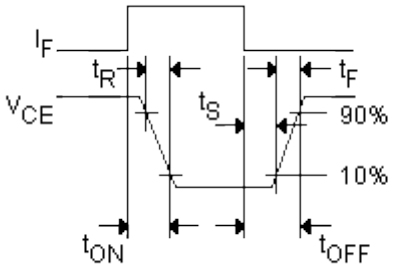


Figure 10: Output switching times (**-POE** (p. 220) option)

Characteristics

Characteristics	Symbol	Test conditions	Typ.	Unit
Turn-On time	t_{ON}	R_L = 100 Ohm, V_{CE} = 10 V, I_C = 2 mA	10	us
Storage time	t_S		3	
Turn-Off time	t_{OFF}		3	
Turn-On time	t_{ON}	R_L = 1.9 kOhm, V_{CE} = 5 V, I_C = 16 mA	2	us
Storage time	t_S		25	
Turn-Off time	t_{OFF}		40	

	Comment	Min.	Typ.	Max.	Unit
I_C	load current			15	mA
$V_{CE(sat)}$	@ I_C = 7 mA		0.4		V
V_{OUT}	Output Voltage			30	V

Notes	

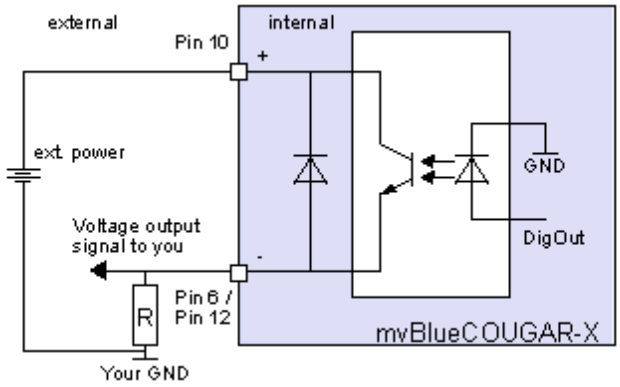


Figure 11: DigOut mvBlueCOUGAR-X with example circuit (**-POE** (p. 220) option)

Features	

Measurement conditions : V_{CC} = 24 V; -25 °C < T_J < 125 °C, R_L = 48 Ohm

Example circuit 1: High-side switch uses power supply of the camera

In this case you have to bridge pin 2 (external power supply) with pin 10 (the supply of the outputs):

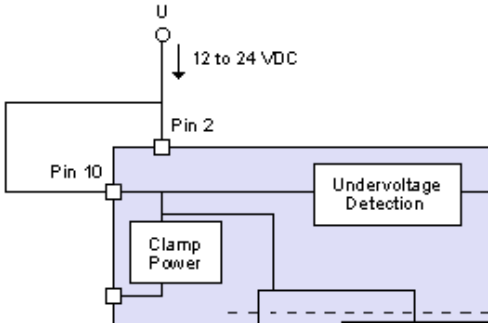


Figure 12: Bridge circuit between pin 2 and pin 10

Example circuit 2: High-side switch uses external power supply

Following figure shows, how to connect, for example, high power flash LEDs in series at the direct drive output at pin 3:

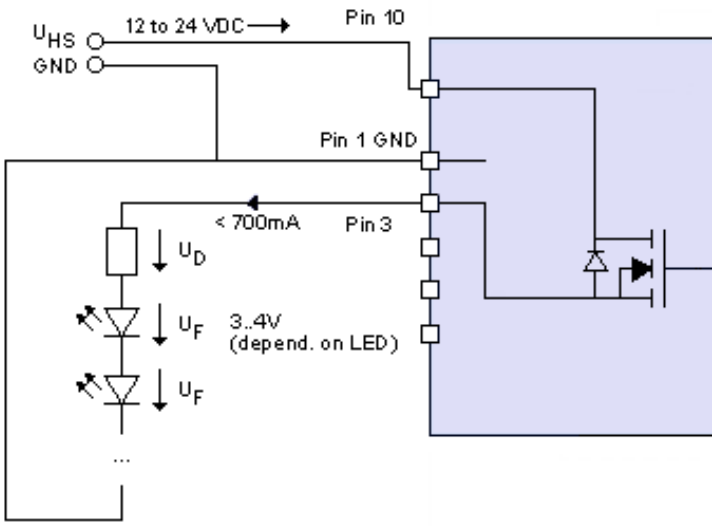


Figure 13: LEDs connected in series to pin 3 powered externally via pin 10

A 24V power supply is connected at pin 10. To protect the LEDs a series resistor is needed, which can be calculated in this way:

Voltage:

```
U_HS - (n * U_F) = U_D
```

Series resistor:

```
      U_D
R = --------
     0.7A
```

Power:

```
P = duty cycle * U_D * I
```

The duty cycle is a coefficient < 1, which defines the power of the resistor. It is the ratio of the time period while current flows compared to the total time. If the current always flows, "`P = U * I`" will be the determining factor. If the current only flows a hundredth of time (duty cycle 1%), a resistor with a hundredth of power will be enough.

Note

For the connected LEDs and the ground of the external power supply, you have to use the ground of the camera at Pin 1.

Attention

"Surge"

The series resistor has to handle the power, which depends on the duty cycle and the frequency. Otherwise surge is the result and can damage the used LEDs.

→ Calculate the suitable size of the series resistor using the above mentioned formulas.

Example circuit 3: Control motorized lens with mvBlueCOUGAR-X

It is possible to control a motorized lens using the mvBlueCOUGAR-X. However, an external micro relay is necessary which changes the polarity of the lens slider when triggered by the camera's digital output. The following figure shows an example circuit:

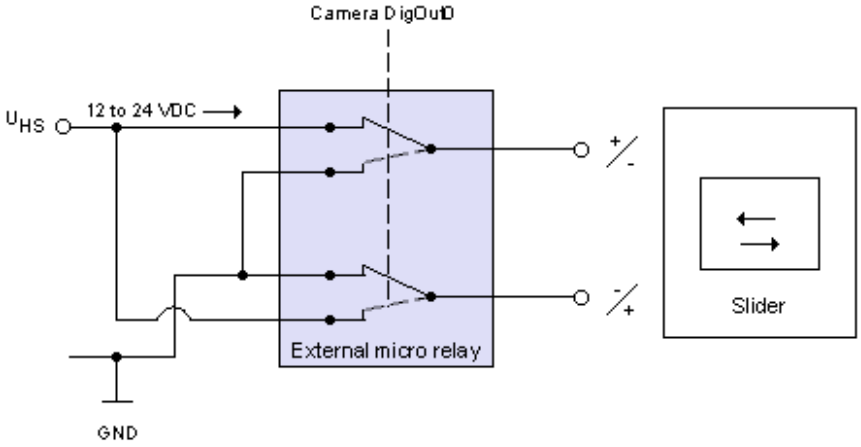


Figure 14: External micro delay

RJ45 100/1000 MBit Ethernet

With the -POE option, the camera is a class 2 compliant PoE device.

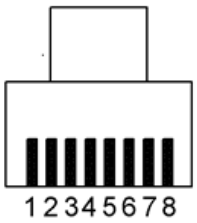


Figure 15: RJ45 pin numbering (female)

Pin.	Signal	I/O	Comment
1	BI_DA+	bi	data positive
2	BI_DA-	bi	data negative
3	BI_DB+	bi	data positive
4	BI_DC+	bi	data positive
5	BI_DC-	bi	data negative
6	BI_DB-	bi	data negative
7	BI_DD+	bi	data positive
8	BI_DD-	bi	data negative

Note

The Ethernet signals are galvanically isolated from the camera electronics and camera housing.

RJ45 LED states

States	LED1	LED2
1Gb transmission	Green light on	Yellow blink on
100Mb transmission	Green light off[1]	Yellow blink on

[1] Hardware Rev 5.xx - Green light on.

1.10.1.3 Signal LED

The mvBlueCOUGAR-X features a RGB LED. There are following states:

Typical start sequence (LLA)

See also

LLA (p. 35)

State	LED color
FPGA loaded	White on
Self-test running	Green on
Waiting for Ethernet connection	White blink
LLA (auto IP)	Green blink
LLA got	Green on
Waiting for client	Blue on

Typical start sequence (DHCP)

See also

DHCP (p. 35)

State	LED color
FPGA loaded	White on
Self-test running	Green on
Waiting for Ethernet connection	White blink
DHCP request	Blue blink
DHCP got	Bright Blue
Waiting for client	Blue on

Typical start sequence (Fixed IP)

See also

Fixed IP (p. 35)

State	LED color
FPGA loaded	White on
Self-test running	Green on
Waiting for Ethernet connection	White blink
Fixed IP	Bright Green
Waiting for client	Blue on

General behavior

State	LED color
Starting **wxPropView** (p. 139)	
Connected, streaming off	Yellow on
Streaming on	Green on
Error	
	Red on

1.10.2 mvBlueCOUGAR-X-POE-I / -IP67

1.10.2.1 Dimensions

1.10.2.1.1 Power over Ethernet model (-POE-I)

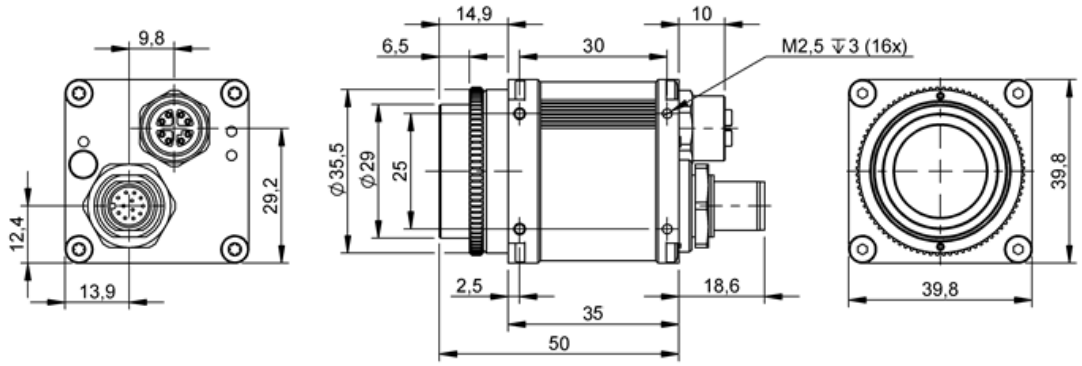


Figure 16: mvBlueCOUGAR-X-POE-I

1.10.2.1.2 Compact IP67 version (-IP67C)

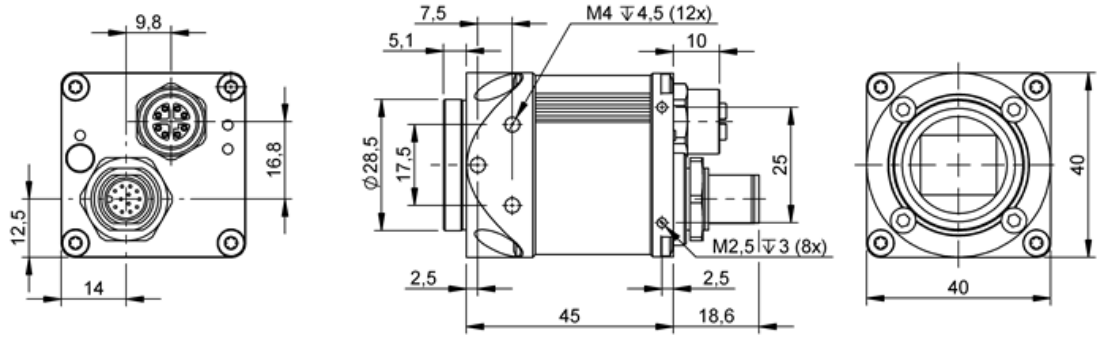


Figure 17: mvBlueCOUGAR-X-IP67C

1.10.2.2 Connectors

1.10.2.2.1 Circular connector male (POE-I Rev. 2.00)

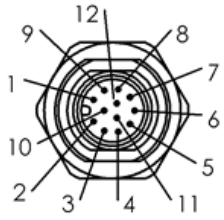


Figure 18: M12 12-pin (male; top view), digital I/O, power;

Note

The power pinning is different to the **circular connector male** (p. 84) pinning due to the different cable color scheme.

Pin.	Signal	Description	Pin color cable BCC M41C-0000-1↩ A-169-PS0C25-xxx-C0009 / KS-M12-PWR-IO xx.x
1	**PWR_IN** (p. 114) [1]		white
2	GND (for PWR_IN) [1]		brown
3	DigOut3	**wxPropView** (p. 139) numbering: line3, high side solid state relays	green
4	Opto DigIn0	line4, opto-coupler	yellow
5	DigOut2	line2, high side solid state relays	grey
6	DigOut0	line0, high side solid state relays	pink
7	Opto DigIn_GND	Ground for opto-coupler (In)	blue
8	Opto DigIn2	line6, opto-coupler	red
9	Opto DigIn3	line7, opto-coupler	black
10	DigOut_PWR_IN	10 to 28 V supply for the high side outputs, ground via pin 2 "PWR_IN-/GND"	purple
11	Opto DigIn1	line5, opto-coupler	grey / pink
12	DigOut1	line1, high side solid state relays	red / blue

[1] Using POE and an external power supply at the same time, the external power supply will be treated prioritized.

Camera connector (male): Binder M12/12pol 09-3491-90-12 or equivalent

Matching cable and connector (female):

- Binder 79-3490-32-12 | M 12 | A-coded series 763, length 2m open end
- Binder 79-3490-35-12 | M 12 | A-coded series 763, length 5m open end

Note

- All inputs (DigIn1..DigIn4 and GND Opto-coupler) are galvanically isolated from the camera electronics.
  - The digital inputs have the same characteristics as the inputs of mvBlueCOUGAR-X with PLC option.
  - The digital outputs have the same characteristics as the outputs of the standard mvBlueCOUGAR-

X.

Characteristics of the digital inputs

Delay

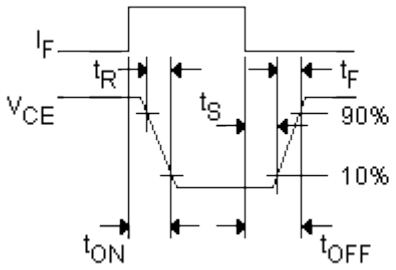


Figure 19: Input switching times

Characteristics	Symbol	Test conditions	Typ.	Unit
Minimum trigger pulse width			5	us
Turn-On time	t_{ON}		2	
Storage time	t_S	R = 2 kOhm (Figure 21), internal output voltage 5 V, I_F = 16 mA	44	
Turn-Off time	t_{OFF}		95	

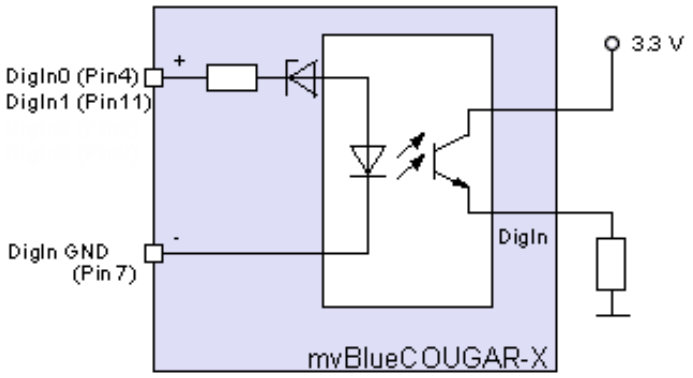
Notes	Inputs with highest tolerance against interferences.
High level	+11 to +24 V
Low level	0 to +8 V
Threshold (Low --> High / High --> Low)	10 V +- 1 V
I_{max}	16 mA
	

Figure 20: DigIn mvBlueCOUGAR-X-POE-I

Characteristics of the digital outputs

Delay

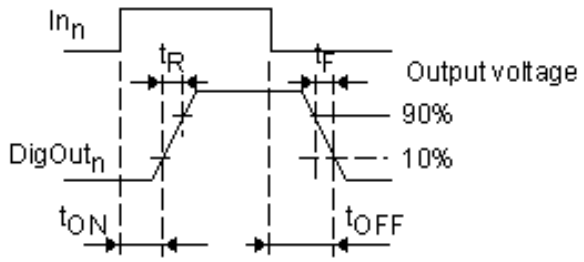


Figure 21: Output switching times (standard)

Characteristics

	Symbol	Typ.	Unit
Turn-On time	t_{ON}	20	us
Rise time	t_R	10	
Fall time	t_F	8	
Turn-Off time	t_{OFF}	30	

High level		+12.0 V to +24 V
Low level		max. +1 V
	I_{out}	0.7 A static load 1.3 A dynamic load

Notes

Each output has a short circuit protection between 1 A and 1.7 A (generally 1.3 A). So if you combine two outputs with one load, the short circuit protection can have an effect.

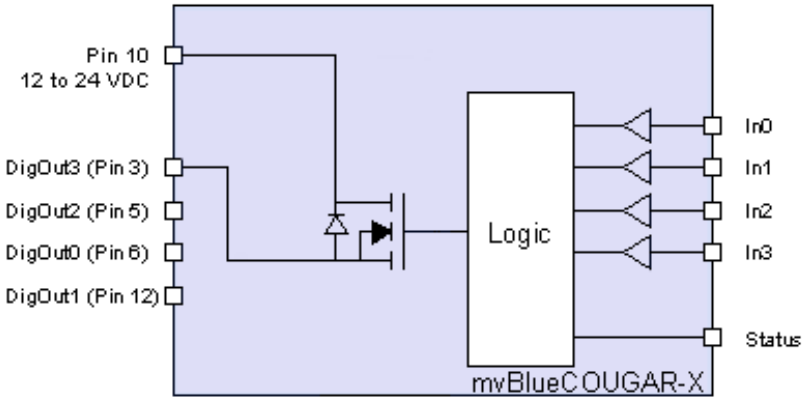


Figure 22: DigOut mvBlueCOUGAR-X POE-I

Features

- undervoltage detection
- current limitation
- overtemperature protection
- short circuit protection

Measurement conditions : V_{CC} = 24 V; -25 °C < T_J < 125 °C, R_L = 48 Ohm

1.10.2.2.2 M12 X-coded 100/1000 MBit Ethernet (option POE-I Rev. 2.00)

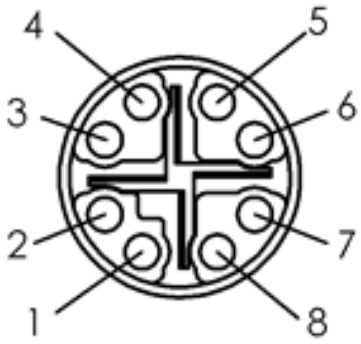


Figure 23: M12 x-coded 8-pin Ethernet connector pin numbering (female)

Pin.	Signal
1	BI_DA+
2	BI_DA-
3	BI_DB+
4	BI_DB-
5	BI_DD+
6	BI_DD-
7	BI_DC-
8	BI_DC+

Note

The Ethernet signals are galvanically isolated from the camera electronics.

1.10.3 mvBlueCOUGAR-XD

1.10.3.1 Dimensions

1.10.3.1.1 Standard model with lens holder type 1 (-1x1x)

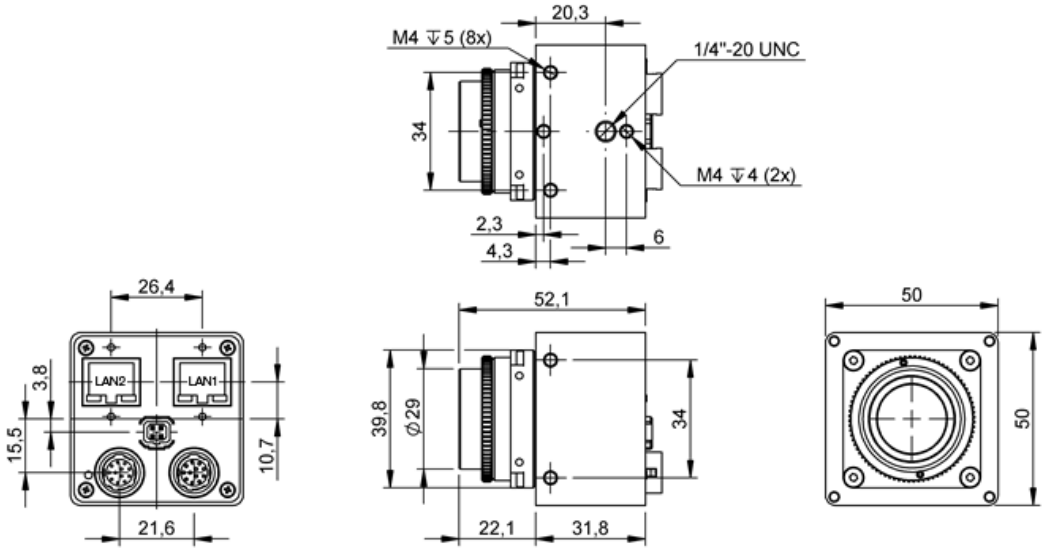


Figure 24: Connectors mvBlueCOUGAR-XD

mvBlueCOUGAR-XD	
Size of body (w x h x l)	50 x 50 x 56.8 mm

1.10.3.1.2 Standard model with lens holder type 2 (-6x1x)

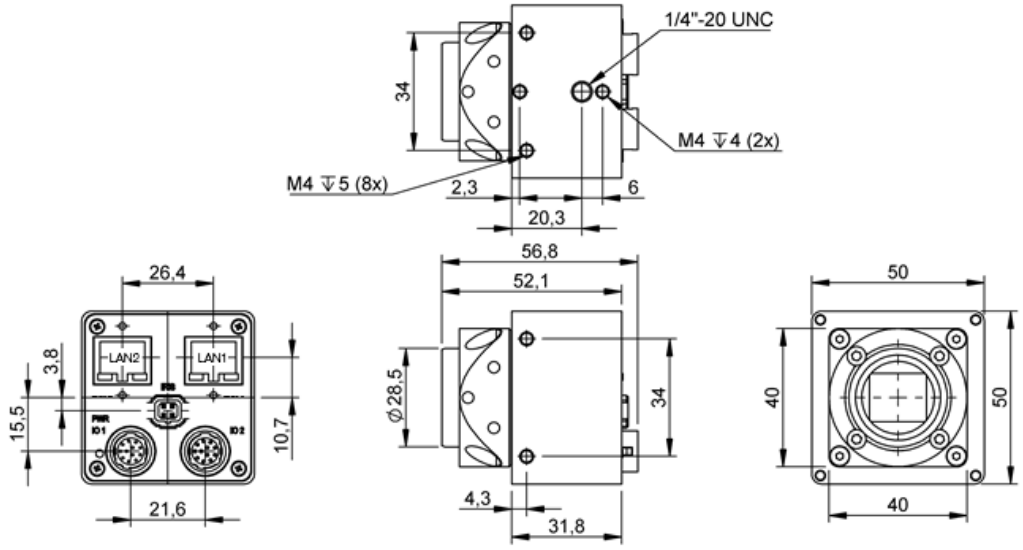


Figure 25: Connectors mvBlueCOUGAR-XD

mvBlueCOUGAR-XD	
Size of body (w x h x l)	50 x 50 x 56.8 mm

1.10.3.1.3 Hi-res model with M42 mount (-Nx1x)

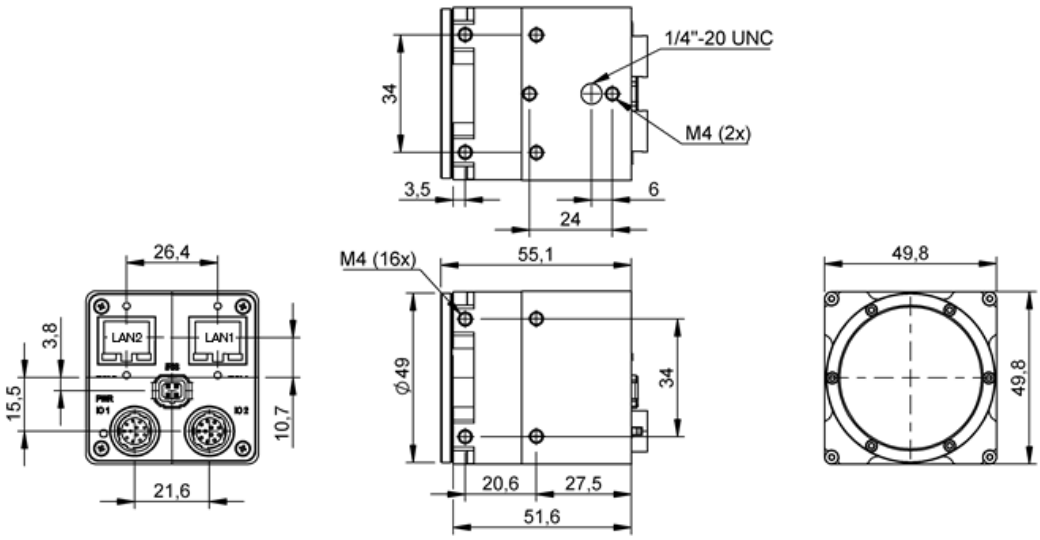


Figure 26: Connectors mvBlueCOUGAR-XD1031 with M42 mount

mvBlueCOUGAR-XD	
Size of body (w x h x l)	50 x 50 x 55.5 mm

Available M42 adapter:

Part code	Description
BAM AD-VS-020-M42/FM-1	Adapter from M42 at F-mount
BAM AD-VS-020-M42/M35X0,75-1	Adapter from M42 to M35 distance 5.5mm for TFL lenses

1.10.3.2 Connectors

1.10.3.2.1 Circular connector male

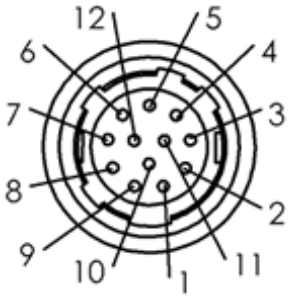


Figure 27: 12-pin (male; top view), digital I/O, power; orientation mvBlueCOUGAR-XD

Pin.	Signal	Digital I/O color code of power supply MV-DC1201 BCSX IO; Rev. smaller than 2	Digital I/O color code of power supply MV-DC1201 BCSX IO; Rev. 2
1	GND (for PWR_IN)	GND	-
2	**PWR_IN** (p. 114)		
3	DigOut3 (**wxPropView** (p. 139) numbering: line3)	white	white
4	Opto DigIn0 (line4)	brown	brown
5	DigOut2 (line2)	green	green
6	DigOut0 (line0)	yellow	yellow
7	Opto DigIn_GND	gray	gray
8	RS232_RX	pink	pink
9	RS232_TX	blue	blue
10	DigOut_PWR_IN (12 to 24 V supply for the outputs)	red	red
11	Opto DigIn1 (line5)	black	black
12	DigOut1 (line1)	violet	violet

Connector (camera side): SAMWOO SNH-10-12 (RPCB) or equivalent
Plug (matching cable plug): Hirose HR10A-10P-12S (01) or equivalent

We recommend to use the connection cable **KS-BCX-HR12** .

1.10.3.2.2 Circular connector female

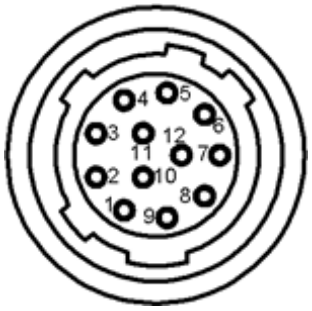


Figure 28: 12-pin (female; top view), digital I/O

Pin.	Signal	Description	Color of open end of cable KS-BCXD-HR12
1	Opto DigIn2 (line6)	Additional input with the same characteristics as the inputs of the male connector	blue
2	Opto DigIn_GND	Ground of DigIn2 and DigIn3	red
3	Opto DigIn3 (line7)	Additional inputs with the same characteristics as the inputs of the male connector	pink
4	Focus+	Focus control for motorized lenses (12V PWM)	gray
5	Focus-		yellow
6	Zoom+	Zoom control for motorized lenses	green
7	Zoom-		brown
8	Iris+	Lens aperture control for lenses with three motors	white
9	Iris-		black
10	Channel4+	Optional fourth channel	light blue
11	Channel4-		orange
12	GND	Ground	violet

Connector (camera side): SAMWOO SN-10-12 (RPCB) or equivalent
Plug (matching cable plug): Hirose HR10A-10P-12P (01) or equivalent

Characteristics of the digital inputs

Delay

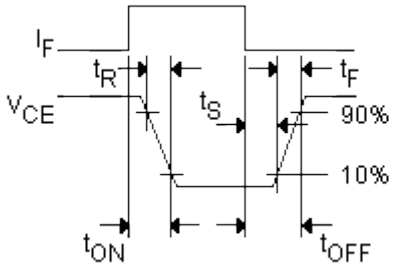
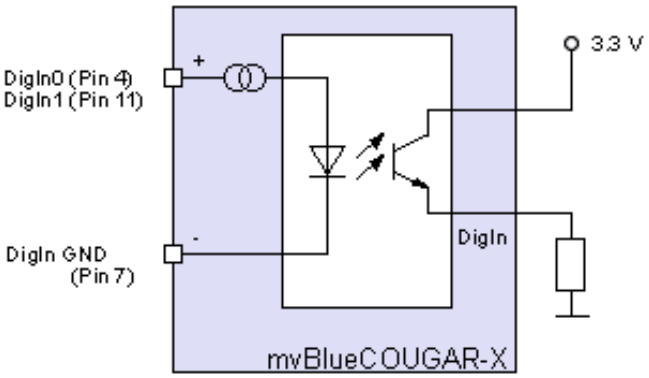


Figure 29: Input switching times

Characteristics	Symbol	Test conditions	Typ.	Unit
Minimum trigger pulse width			5	us
Turn-On time	t_{ON}		10	
Storage time	t_S	R = 2 kOhm (Figure 34), internal output voltage 5 V, I_F = 16 mA	25	
Turn-Off time	t_{OFF}		40	

	Standard
Notes	
High level	+3 to +24 V
Low level	0 to +0.3 V
Threshold (Low --> High / High --> Low)	2.5 V +- 0.5 V
I_{max}	5 mA
	 Figure 30: DigIn mvBlueCOUGAR-XD

Characteristics of the digital outputs

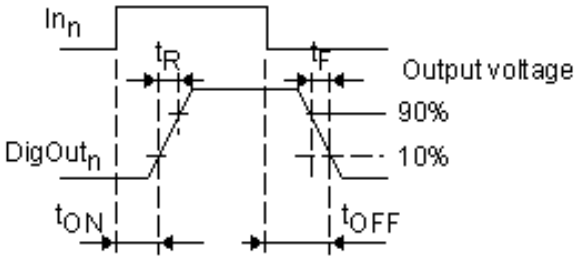
<table>
<tr><td></td><td></td></tr>
<tr><td>Delay</td><td> 

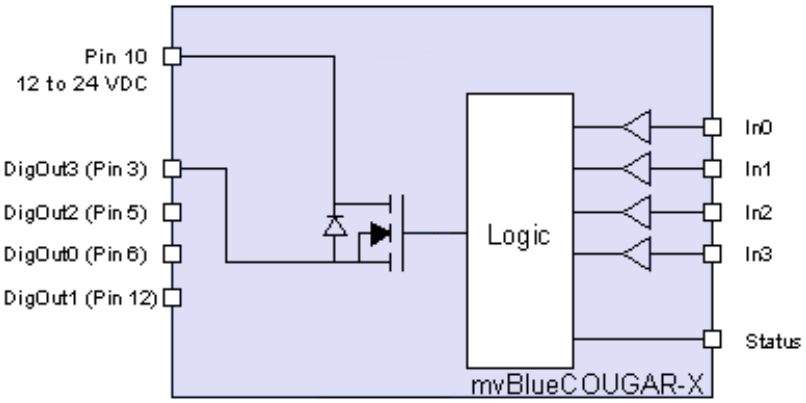
Figure 31: Output switching times (standard)</td></tr>
<tr><td>Characteristics</td><td>
<table>
<tr><td></td><td>Symbol</td><td>Typ.</td><td>Unit</td></tr>
<tr><td>Turn-On time</td><td>t_{ON}</td><td>20</td><td rowspan="4">us</td></tr>
<tr><td>Rise time</td><td>t_R</td><td>10</td></tr>
<tr><td>Fall time</td><td>t_F</td><td>8</td></tr>
<tr><td>Turn-Off time</td><td>t_{OFF}</td><td>30</td></tr>
</table>
<table>
<tr><td>High level</td><td></td><td>+12.0 V to +24 V</td></tr>
<tr><td>Low level</td><td></td><td>max. +1 V</td></tr>
<tr><td></td><td>I_{out}</td><td>0.7 A static load
1.3 A dynamic load</td></tr>
</table>
</td></tr>
<tr><td>Notes</td><td>Each output has a short circuit protection between 1 A and 1.7 A (generally 1.3 A). So if you combine two outputs with one load, the short circuit protection can have an effect.</td></tr>
<tr><td></td><td> 

Figure 32: DigOut mvBlueCOUGAR-XD (same as mvBlueCOUGAR-X)</td></tr>
<tr><td>Features</td><td>- undervoltage detection
- current limitation
- overtemperature protection
- short circuit protection</td></tr>
</table>

Measurement conditions : V_{CC} = 24 V; -25 °C < T_J < 125 °C, R_L = 48 Ohm

Example circuit 1: High-side switch uses power supply of the camera

In this case you have to bridge pin 2 (external power supply) with pin 10 (the supply of the outputs):

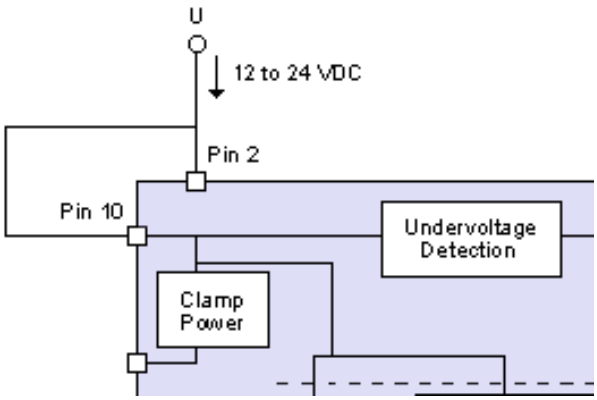


Figure 33: Bridge circuit between Pin 2 and Pin 10

Example circuit 2: High-side switch uses external (higher) power supply

Following figure shows, how to connect, for example, high power flash LEDs in series at the direct drive output at Pin 3:

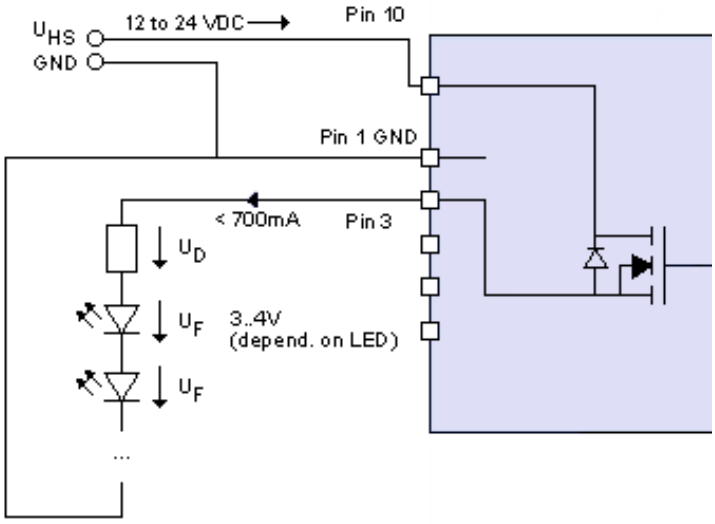


Figure 34: LEDs connected in series to Pin 3 powered externally via Pin 10

A 24V power supply is connected at Pin 10. To protect the LEDs a series resistor is needed, which can be calculated in this way:

Voltage:

```
U_HS - (n * U_F) = U_D
```

Series resistor:

```
      U_D
R = --------
     0.7A
```

Power:

```
P = dutycycle * U_D * I
```

The duty cycle is a coefficient < 1, which defines the power of the resistor. It is the ratio of the time period while current flows compared to the total time. If the current always flows, "`P = U * I`" will be the determining factor. If the current only flows a hundredth of time (duty cycle 1%), a resistor with a hundredth of power will be enough.

Note

For the connected LEDs and the ground of the external power supply, you have to use the ground of the camera at Pin 1.

Attention

The series resistor has to handle the power, which depends on the duty cycle and the frequency.

1.10.3.2.3 RJ45 100/1000 MBit Ethernet

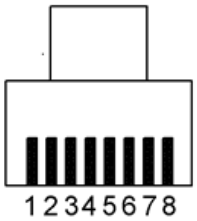


Figure 35: RJ45 pin numbering (female)

Pin.	Signal	I/O	Comment
1	BI_DA+	bi	data positive
2	BI_DA-	bi	data negative
3	BI_DB+	bi	data positive
4	BI_DC+	bi	data positive
5	BI_DC-	bi	data negative
6	BI_DB-	bi	data negative
7	BI_DD+	bi	data positive
8	BI_DD-	bi	data negative

Note

The Ethernet signals are galvanically isolated from the camera electronics and camera housing.

RJ45 LED states

States	LED1	LED2
1Gb transmission	Green light on	Yellow blink on
100Mb transmission	Green light off[1]	Yellow blink on

[1] Hardware Rev 5.xx - Green light on.

1.10.3.2.4 Video iris

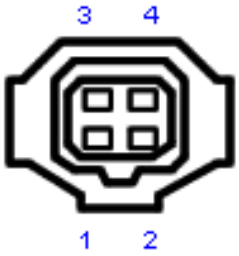


Figure 36: Video iris pin numbering

This pin configuration is compliant to the JEITA standard:

Pin	Signal
1	IRIS_PWR_IN (12 V)
2	NC
3	IRIS_CTRL
4	Ground

Attention

"Wrong polarity"

Please check the wiring of your video iris lenses, because there are brands which uses the standard 4-pin connector with non-standard wiring. Otherwise you can damage the lens.

1.10.3.3 Signal LED

The mvBlueCOUGAR-XD features a RGB LED. There are following states:

1.10.3.3.1 Typical start sequence (LLA)

See also

LLA (p. 35)

State	LED color
FPGA loaded	White on
Self-test running	Green on
Waiting for Ethernet connection	White blink
LLA (auto IP)	Green blink
LLA got	Green on
Waiting for client	Blue on

1.10.3.3.2 Typical start sequence (DHCP)

See also

DHCP (p. 35)

State	LED color
FPGA loaded	White on
Self-test running	Green on
Waiting for Ethernet connection	White blink
DHCP request	Blue blink
DHCP got	Bright Blue on
Waiting for client	Blue on

1.10.3.3.3 Typical start sequence (Fixed IP)

See also

Fixed IP (p. 35)

State	LED color
FPGA loaded	White on
Self-test running	Green on
Waiting for Ethernet connection	White blink
Fixed IP	Bright Green on
Waiting for client	Blue on

General behavior

State	LED color
Starting **wxPropView** (p. 139)	
Connected, streaming off	Yellow on
Streaming on	Green on
Error	
	Red on

1.10.4 mvBlueCOUGAR-3X

1.10.4.1 Dimensions

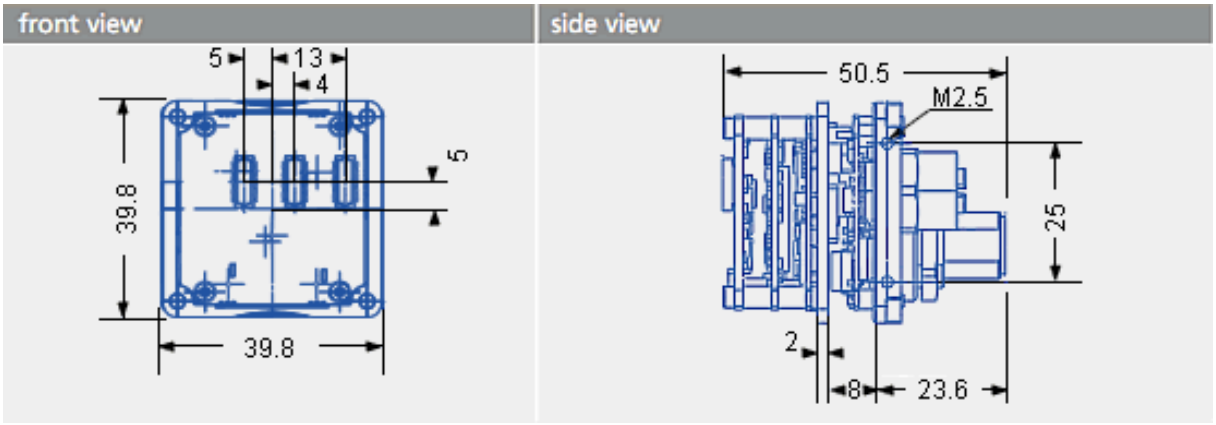


Figure 37: Connectors mvBlueCOUGAR-3X

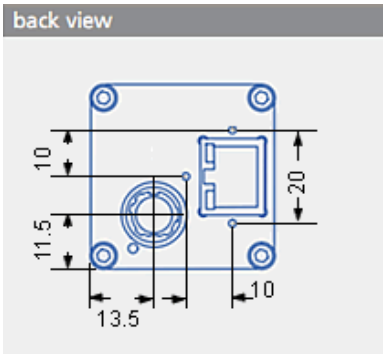
Size of body (w x h x l)	39.8 x 39.8 x 50.5 mm

Connectors

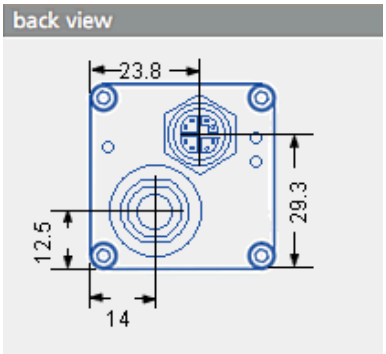
Please have a look at chapter **connectors of the standard model** (p. 84) for the characteristics of the standard connectors or at chapter **connectors of the POE-I model** (p. 95) for the characteristics of the POE-I connectors.

The 3-head-model uses the same back panel as the standard mvBlueCOUGAR-X. Two version are available for the 3-head-model:

Standard



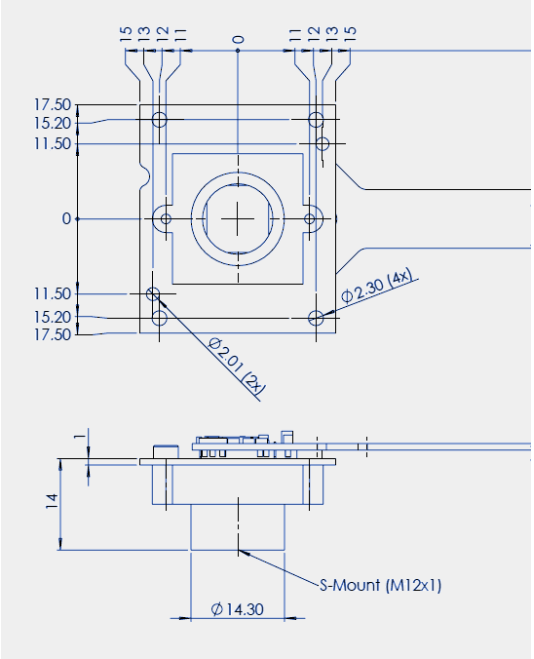
POE-I



Sensor heads

The three sensor heads of the mvBlueCOUGAR-3X are connected via a starrflex connection of 200 mm length. The sensor heads feature a S-mount lensholder. For both available sensors there are different sensor heads:

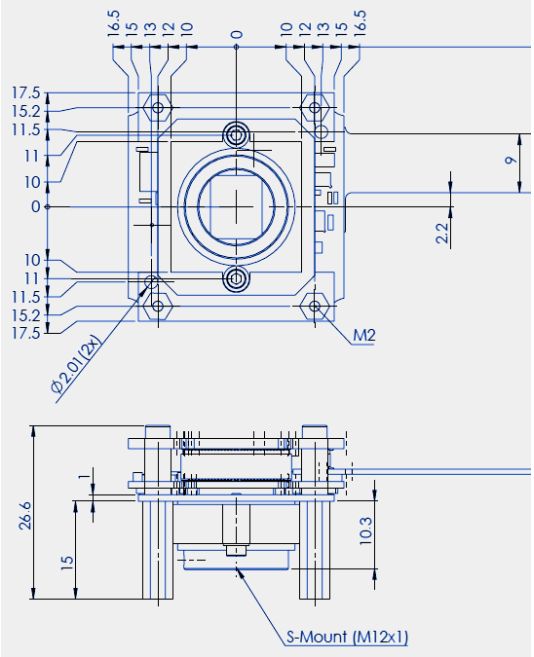
-120a



See also

-120a (p. 593)

-123



See also

-123 (p. 617)

1.10.4.2 Model specific features

In contrast to the standard mvBlueCOUGAR-X model:

Features		mvBlueCOUGAR-3Xxxx -G/-C
Environment		
	Ambient Temperature	
	Operation	0 to 40 deg C / 30 to 80% RH
Power supply		
	P_{max}	3 to 6 W according to the used sensor

1.10.5 Summary of components

Features		mvBlueCOUGAR-Xxxx -G/-C					mvB COU XDx
		Standard version	"-PLC" option (inputs with PLC levels only)	**Power over Ethernet** (p. 220)	**Power over Ethernet Industrial** (p. 220)		
					IP40	IP67	
Image Memory		64 MByte (128 MB option available: mvBlueCOUGAR-X2xx)					256
Proc. Memory		64 MByte					
ADC resolution		CCD: 14 bits (14/12/8 bit transfer)CMOS: ADC on chip					
Inputs		2	2 (PLC levels only)	2	4		4
	Type	opto-isolated with current limiters					
Outputs		4		2	4		4
	Type	high-side solid state relay		opto-isolated	high-side solid state relay		high- solid relay
Summary of available Digital I/O's		http://www.matrix-vision.com/faq-mvbluecougar-en.html?show=760					
Ethernet		100/1000 Mbit on RJ45			100/1000 Mbit on M12 X-coded		2x 1 Mbit RJ45
Optics							
	Lens Mount (Focal Distance)	C-mount (17.526 mm in air), CS-mount (12.5 mm in air), optional S-mount					
	Lens Protrusion	6 .. 8 mm					
Environment							
	Ambient Temperature						
	Operation	0..45 deg C / 30 to 80% RH			0..45 deg C / 0 to 100% RH		0..45 / 30 RH
	Storage	-20..60 deg C / 20 to 90% RH			-20..60 deg C / 0 to 100% RH		-20.. C / 2 RH
	Protection class [1]	IP40				IP67	IP30
Additional features		temperature sensor					temp sens vide lens (zoo iris)

Weight without lens		approx. 110 g	approx. 127 g	appr g, n COU XD1 appr g
Power supply (PWR_IN)				
	DC	10 to 28 V		
	P_{max}	6.5 W		9 W
	Max. current consumption	1.0 A @ 10 V		
	Reverse polarity protection	up to 28 V		

[1] not evaluated by UL

1.10.6 Summary of advanced features

The following table shows an excerpt of the advanced features.

Advanced features		mvBlueCOUGAR-X1xx	mvBlueCOUGAR-X2xx	mvBlueCOUGAR-XDxxx
MultiFrame / SingleFrame / Continuous (p. 144)		X	X	X
Trigger Overlap (p. 144)		X	X	X
Temperature settings (p. 404)	upper limit (<= 255 deg C) lower limit (>= 0 deg C)	X	X	X
User Sets (p. 163)	5 config sets storable	X	X	X
User Data (p. 394)	512 byte of EEPROM data	X	X	X
FPGA based look-up table (LUT) (p. 170)	on the fly on the camera	X	X	X
Timestamp (latch/reset) (p. 162)	microsecond precise camera individual 64 bit counter	X	X	X
Flat Field Correction (FFC) (p. 289)	full AOI, 14 bit to 14 bit (12 bit coeff.), pixel to pixel	X	-	-
Enhanced I/O functionality acc. to SFNC (counter, timer) (p. 148)	timers can be used for pulse width modulation	4 counters, 2 timers	4 counters, 2 timers	8 counters, 2 timers

Auto Exposure Control (AEC) (p. 144)		X	X	X
Auto Gain Control (AGC) (p. 144)		X	X	X
White balance		X	X	X
Auto white balance		X	X	X
RGB-to-YUV conversion		X	X	X
Color correction matrix (p. 152)		X	X	X
Timestamp for inter-camera synchronization		X	X	X
Image recorder with pretrigger option (p. 225)		X	X	X
Linescan mode (p. 225)	**mvBlueCOUGAR-X102e (1.3 Mpix [1280 x 1024])** (p. 554) **mvBlueCOUGAR-X102eGE (1.3 Mpix [1280 x 1024])** (p. 560) **mvBlueCOUGAR-X104e (1.9 Mpix [1600 x 1200])** (p. 567)	X	X	X
Logic gates (p. 165)		X	X	X
Chunk Data (p. 154)		X	X	X
Introducing multicasting (p. 276)		X	X	X
Primary application switchover (p. 430)		X	X	X
Frame average (p. 312)		-	X	-
Horizontal mirror (CCD)		-	X	X

1.11 Sensor Overview

1.11.1 Image data flow

The following block diagrams show the data flow of the image data after being read from the sensor chip in the camera. The transfer latency (stream controller + frame buffer + resend buffer + packet generator) of the data is less than 100 us.

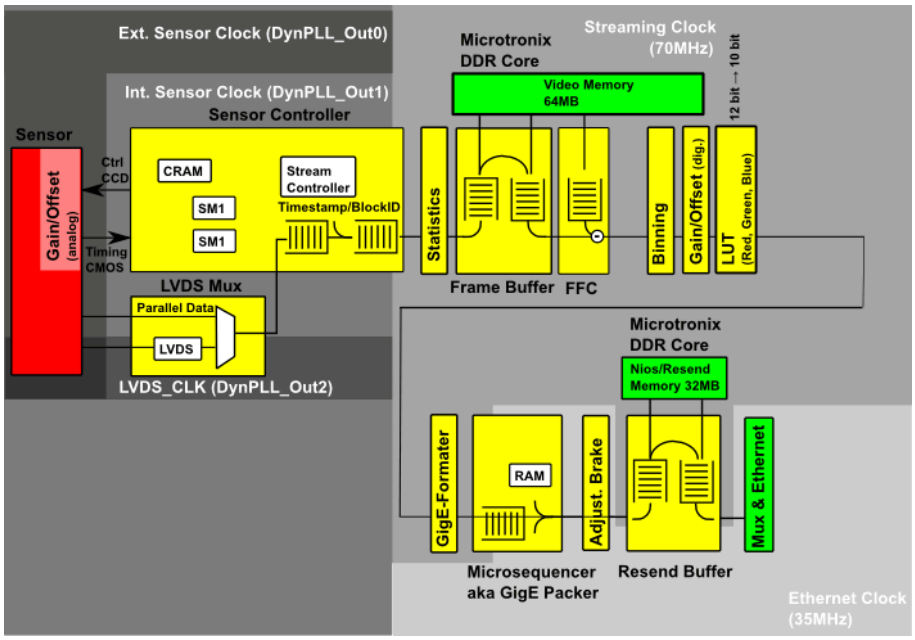


Figure 1: Block diagram gray scale sensors

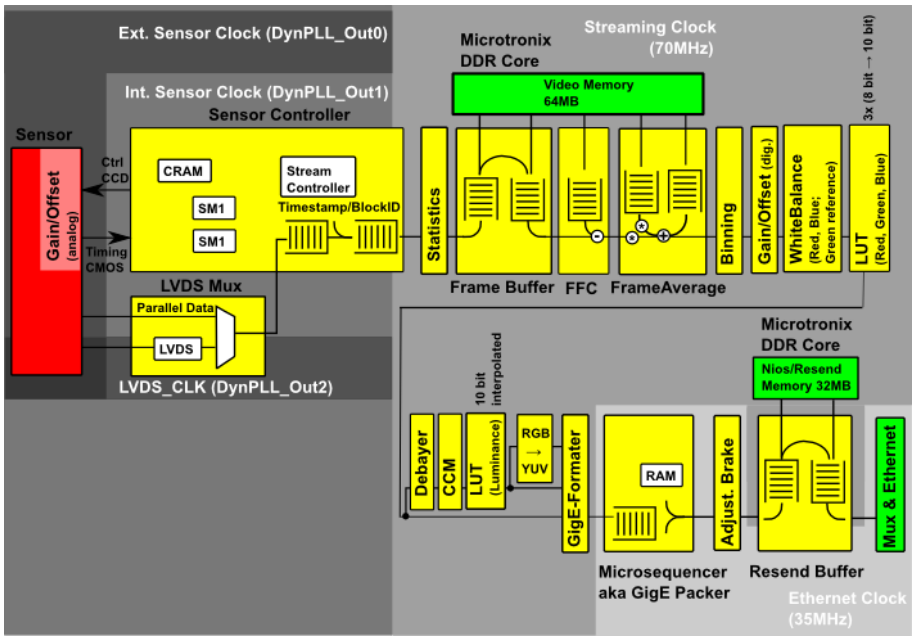


Figure 2: Block diagram color sensors

1.11.2 mvBlueCOUGAR-X specific CMOS sensors

The CMOS sensor modules incorporate the following features:

1.11.2.1 Sony Pregius S

Sensors:	5.1 Mpix (-x05d)	8.1 Mpix (-108a)	8.1 Mpix (-108u)	12.4 Mpix (-1012d)
Sensor supplier	Sony	Sony	Sony	Sony
Sensor name	IMX547	IMX546	IMX487	IMX545
Resolution	2472 x 2064 gray scale or RGB	2856 x 2848 gray scale or RGB	2848 x 2848 gray scale or RGB	4128 x 3008 gray scale or RGB
Sensor size	1/1.8"	2/3"	2/3"	1/1.1"
Max. FPS (in free-running full frame mode)	39.0 / 74.0 (-XD) [**burst mode** (p. 245) required] | 23.2 / 46.5 (-XD) [streaming]	25.0 / 48.0 (-XD) [**burst mode** (p. 245) required] | 14.5 / 29.1 (-XD) [streaming]	25.0 [**burst mode** (p. 245) required] | 14.5 [streaming]	17.0 / 33.0 (-XD) [**burst mode** (p. 245) required] | 9.5 / 19.1 (-XD) [streaming]
Frame rate exactness (p. 145)	-	-	-	-
ADC resolution / Out [bit]	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8
SNR_{max} [dB][1]	39.7	39.7	39.2	39.7
DR (normal / **HDR** (p. 378)) [dB][1]	69.9	69.9	69	70
Progressive scan sensor (no interlaced problems!)	X	X	X	X
Rolling shutter	-	-	-	-
Global shutter	X	X	X	X
Trigger (HW / SW)	X / X	X / X	X / X	X / X
Pipelined global shutter in trigger mode (p. 220)	X	X	X	X
Linescan mode	-	-	-	-
High color reproductivity (for color version)	X	X	X	X
Programmable readout timing with free capture windows and partial scan	X	X	X	X
Many trigger modes (free-running, software-triggered, hardware-triggered)	X	X	X	X
Flash control output, synchronous to integration period	X	X	X	X
Power consumption @ 24 V [W]	approx. 3.1	approx. 3.1	approx. tbd	approx. 3.1

More specific data	**mvBlueCOUGAR-X[D]105d (5.1 Mpix [2472 x 2064])** (p. 442)	**mvBlueCOUGAR-X[D]108a (8.1 Mpix [2856 x 2848])** (p. 446)	**mvBlueCOUGAR-X108u (8.1 Mpix [2848 x 2848])** (p. 449)	**mvBlueCOUGAR-X[D]1012d (12.4 Mpix [4128 x 3008])** (p. 452)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.11.2.2 Sony Pregius

< 3 Mpix

Sensors:	0.4 Mpix (-x00f)	1.6 Mpix (-x02f)	1.7 Mpix (-102k)	1.7 Mpix (-102m)	2 Mpix (-102n)	2.4 Mpix (-x04f)
Sensor supplier	Sony	Sony	Sony	Sony	Sony	Sony
Sensor name	IMX287	IMX273	IMX432	IMX425	IMX430	IMX249
Resolution	728 x 544 gray scale or RGB	1456 x 1088 gray scale or RGB	1600 x 1104 gray scale or RGB	1600 x 1104 gray scale or RGB	1632 x 1248 gray scale or RGB	1936 x 1216 gray scale or RGB
Sensor size	1/2.9"	1/2.9"	1.1"	1.1"	1/1.7"	1/1.2"
Max. FPS (in free-running full frame mode)	436 [**burst mode** (p. 245) required] / 299.8 [streaming]	126.3 / 226.5 (-XD) [**burst mode** (p. 245) required] | 74.9 / 149.9 (-XD) [streaming]	98.3 [**burst mode** (p. 245) required] / 67.2 [streaming]	115.6 [**burst mode** (p. 245) required] / 67.2 [streaming]	87.8 [**burst mode** (p. 245) required] / 58.3 [streaming]	46.9
Frame rate exactness (p. 145)	-	-	-	-	-	-
ADC resolution / Out [bit]	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8	12 / 12	12 / 16, 12, 8
SNR_{max} [dB][1]	43.3	40.2	49.5	49.9	43.9	45
DR (normal / **HDR** (p. 378)) [dB][1]	74	71.2	72	72.3	71.8	72.9 /
Progressive scan sensor (no interlaced problems!)	X	X	X	X	X	X
Rolling shutter	-	-	-	-	-	-
Global shutter	X	X	X	X	X	X
Trigger (HW / SW)	X / X	X / X	X / X	X / X	X / X	X / X

Pipelined global shutter in trigger mode (p. 220)	X	X	X	X	X	X
Linescan mode	-	-	-	-	-	-
High color reproductivity (for color version)	X	X	X	X	X	X
Programmable readout timing with free capture windows and partial scan	X	X	X	X	X	X
Many trigger modes (free-running, software-triggered, hardware-triggered)	X	X	X	X	X	X
Flash control output, synchronous to integration period	X	X	X	X	X	X
Power consumption @ 24 V [W]	approx. 3.6	approx. 3.6	approx. 3.8	approx. 3.7	approx. 3.7	approx. 3.4
More specific data	**mvBlue↩ COUGAR-↩ X100f (0.4 Mpix [728 x 544])** (p. 468)	**mvBlue↩ COUGAR-X[D]102f (1.6 Mpix [1456 x 1088])** (p. 471)	**mvBlue↩ COUGAR-↩ X102k (1.8 Mpix [1600 x 1104])** (p. 475)	**mvBlue↩ COUGAR-↩ X102m (1.8 Mpix [1600 x 1104])** (p. 478)	**mvBlue↩ COUGAR-↩ X102n (2 Mpix [1632 x 1248])** (p. 482)	**mvBlue↩ COUGAR-↩ X104f (2.4 Mpix [1936 x 1216])** (p. 490)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

> 3 Mpix < 15 Mpix

Sensors:	3.2 Mpix (-x04i)	5.1 Mpix (-x05b)	7.1 Mpix (-107b)	8.9 Mpix (-109b)	12.4 Mpix (-1012b)
Sensor supplier	Sony	Sony	Sony	Sony	Sony
Sensor name	IMX265	IMX264	IMX428	IMX267	IMX304
Resolution	2064 x 1544 gray scale or RGB	2464 x 2056 gray scale or RGB	3216 x 2208 gray scale or RGB	4112 x 2176 gray scale or RGB	4112 x 3008 gray scale or RGB
Sensor size	1/1.8"	2/3"	1.1"	1"	1.1"

Max. FPS (in free-running full frame mode)	55 [**burst mode** (p. 245) required] / 37.2 [streaming]	35 [**burst mode** (p. 245) required] / 23.4 [streaming]	20.6 / 50.5 (-XD) [**burst mode** (p. 245) required] | 16.7 / 33.4 (-XD) [streaming]	32 [**burst mode** (p. 245) required] / 13.2 [streaming]	15 [**burst mode** (p. 245) required] / 9.6 [streaming]
Frame rate exactness (p. 145)	-	-	-	-	-
ADC resolution / Out [bit]	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8	12 / 12, 10, 8	12 / 12, 10, 8
SNR_{max} [dB][1]	40.3	40.2	43.9	40.2	40.2
DR (normal / **HDR** (p. 378)) [dB][1]	71.4 /	71.3 /	71.8	70.7 /	70.8 /
Progressive scan sensor (no interlaced problems!)	X	X	X	X	X
Rolling shutter	-	-	-	-	-
Global shutter	X	X	X	X	X
Trigger (HW / SW)	X / X	X / X	X / X	X / X	X / X
Pipelined global shutter in trigger mode (p. 220)	X	X	X	X	X
Linescan mode	-	-	-	-	-
High color reproductivity (for color version)	X	X	X	X	X
Programmable readout timing with free capture windows and partial scan	X	X	X	X	X
Many trigger modes (free-running, software-triggered, hardware-triggered)	X	X	X	X	X
Flash control output, synchronous to integration period	X	X	X	X	X
Power consumption @ 24 V [W]	approx. 3.8	approx. 3.7	approx. 3.9	approx. 3.9	approx. 3.9

More specific data	**mvBlueCOUGAR-X104i (3.2 Mpix [2064 x 1544])** (p. 496)	**mvBlueCOUGAR-X105b (5.1 Mpix [2464 x 2056])** (p. 503)	**mvBlueCOUGAR-X[D]107b (7.1 Mpix [3216 x 2208])** (p. 511)	**mvBlueCOUGAR-X[D]109b (8.9 Mpix [4112 x 2176])** (p. 515)	**mvBlueCOUGAR-X[D]1012b (12.4 Mpix [4112 x 3008])** (p. 519)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.11.2.3 Sony Starvis

Sensors	6.4 Mpix (-106)	12.4 Mpix (-1012r)	20.5 Mpix (-1020)
Sensor supplier	Sony	Sony	Sony
Sensor name	IMX178	IMX226	IMX183
Res.	3096 x 2080 gray scale / RGB	4072 x 3044 gray scale / RGB	5544 x 3692 gray scale / RGB
Sensor size	1/1.8"	1/1.7"	1"
Max. FPS (in free-running full frame mode)	34.97 [**burst mode** (p. 245) required] / 18.4 [streaming]	15.9 [**burst mode** (p. 245) required] / 9.6 [streaming]	8.82 [**burst mode** (p. 245) required] / 5.8 [streaming]
Frame rate exactness (p. 145)	-	-	-
ADC resolution / Out	12 / 12, 10, 8	12 / 12, 10, 8	12 / 12, 10, 8
SNR_{max} [dB][1/]	tbd	40.5	41.6
DR (normal / **HDR** (p. 378)) [dB][1/]	tbd	69.6	71.4
Rolling shutter	X	X	X
Global shutter	-	-	-
Global Reset	-	-	-
Trigger (HW / SW)	- / X	- / X	- / X
Pipelined global shutter in trigger mode (p. 220)	-	-	-
Linescan mode	-	-	-
High color reproductivity (for color version)	X	X	X
Power consumption (since FW 2.5.146) [W]	approx. tbd	approx. 3	approx. 2.9
More specific data	**mvBlueCOUGAR-X106 (6.4 Mpix [3096 x 2080])** (p. 526)	**mvBlueCOUGAR-X1012r (12.4 Mpix [4072 x 3044])** (p. 530)	**mvBlueCOUGAR-X1020 (20.5 Mpix [5544 x 3692])** (p. 533)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.11.2.4 Sony Polarsens

Sensors	5.1 Mpix (-105p)
Sensor supplier	Sony

Sensor name	IMX250_POL
Res.	2464 x 2056 gray scale / RGB
Sensor size	2/3"
Max. FPS (in free-running full frame mode)	tbd [**burst mode** (p. 245) required] / tbd [streaming]
Frame rate exactness (p. 145)	-
ADC resolution / Out	12 / 12, 10, 8
SNR_{max} [dB][1/]	40.2
DR (normal / **HDR** (p. 378)) [dB][1/]	71.1
Rolling shutter	-
Global shutter	X
Global Reset	-
Trigger (HW / SW)	X / X
Pipelined global shutter in trigger mode (p. 220)	X
Linescan mode	-
High color reproductivity (for color version)	X
Power consumption (since FW 2.5.146) [W]	approx. 3.7
More specific data	**mvBlueCOUGAR-X105p (5.1 Mpix [2464 x 2056])** (p. 537)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.11.2.5 Aptina, CMOSIS, e2v

< 2 Mpix

Sensors:	0.4 Mpix (-x00w)	1.2 Mpix (-x02b)[1]	1.2 Mpix (-x02d)	1.3 Mpix (-x02e)	1.3 Mpix (-x02eGE)[2]
Sensor supplier	Aptina	Aptina	Aptina	e2v	e2v
Sensor name	MT9V034	MT9M021	MT9M034	EV76C560	EV76C661
Resolution	752 x 480 gray scale or RGB	1280 x 960 gray scale or RGB	1280 x 960 gray scale or RGB	1280 x 1024 gray scale or RGB	1280 x 1024 gray scale
Sensor size	1/3"	1/3"	1/3"	1/1.8"	1/1.8"
Pixel clock [MHz]	27 / 40 / 50	40 / 66 / 74.25	40 / 66 / 74.25	85	85
Max. FPS (in free-running full frame mode)	117	45.6	45.6	60	60
Frame rate exactness (p. 145)	-	-	-	-	-
ADC resolution / Out [bit]	10 / 12, 10, 8	10 / 12, 10, 8	10 / 12, 10, 8	10 (10-8 companding) / 10, 8	10 (10-8 companding) / 10, 8
SNR_{max} [dB][3]	42.7	37.3	37.7	39.8	38.9
DR (normal / **HDR** (p. 378)) [dB][3]	56.2 / > 110	55.4 /	63.8 / > 115	52.8 / > 100	50.3 / > 100
Progressive scan sensor (no interlaced problems!)	X	X	X	X	X

Rolling shutter	-	-	X	-	-
Global shutter	X	X	-	X	X
Trigger (HW / SW)	X / X	X / X	X / X	X / X	X / X
Pipelined global shutter in trigger mode (p. 220)	-	-	-	-	-
Linescan mode	-	-	-	**available** (p. 556)	**available** (p. 563)
High color reproductivity (for color version)	X	X	X	X	X
Programmable readout timing with free capture windows and partial scan	X	X	X	X	X
Many trigger modes (free-running, software-triggered, hardware-triggered)	X	X	X	X	X
Flash control output, synchronous to integration period	X	X	X	X	X
Power consumption @ 24 V [W]	approx. 2.5 (-100w)	approx. 2.8 (-102b)	approx. 3.2 (-102d)	approx. 2.8 (-102e)	approx. 2.8 (-102eGE)
More specific data	**mvBlue↩ COUGAR-↩ X100w (0.4 Mpix [752 x 480])** (p. 542)	**mvBlue↩ COUGAR-↩ X102b (1.2 Mpix [1280 x 960])** (p. 546)	**mvBlue↩ COUGAR-↩ X102d (1.2 Mpix [1280 x 960])** (p. 549)	**mvBlue↩ COUGAR-↩ X102e (1.3 Mpix [1280 x 1024])** (p. 554)	**mvBlue↩ COUGAR-↩ X102eGE (1.3 Mpix [1280 x 1024])** (p. 560)

> 2 Mpix < 15 Mpix

Sensors:	2.2 Mpix (-x04)	4.2 Mpix (-x04b)	1.9 Mpix (-x04e)	5 Mpix (-x05)	10 Mpix (-x010)
Sensor supplier	CMOSIS	CMOSIS	e2v	Aptina	Aptina
Sensor name	CMV2000	CMV4000	EV76C570	MT9P031	MT9J003
Resolution	2048 x 1088 gray scale or RGB	2048 x 2048 gray scale or RGB	1600 x 1200 gray scale or RGB	2952 x 1944 gray scale or RGB	3856 x 2764 gray scale or RGB
Sensor size	2/3"	1"	1/1.8"	1/2.5"	1/2.3"
Pixel clock [MHz]	40	40	85	40 / 98	81.25

Max. FPS (in free-running full frame mode)	270 [**burst mode** (p. 245) required] / 106.5 [streaming] (-XD) | 34.8 (-X)	149 [**burst mode** (p. 245) required] / 56.6 [streaming] (-XD) | 18 (-X)	51	14	7
Frame rate exactness (p. 145)	X	X	-	-	-
ADC resolution / Out [bit]	10 / 16, 12, 8 (-XD) | 10 / 10, 8 (-X)	10 / 16, 12, 8 (-XD) | 10 / 10, 8 (-X)	10 (10-8 companding) / 10, 8	10 / 12, 10, 8	10 / 12, 10, 8
SNR_{max} [dB][3]	38.2	39.1	38.4	38.2	37.2
DR (normal / **HDR** (p. 378)) [dB][3]	59.6 /	56.1 /	50.2 / > 100	59.6 /	56.3 /
Progressive scan sensor (no interlaced problems!)	X	X	X	X	X
Rolling shutter	-	-	-	X	X
Global shutter	X	X	X	X	X
Trigger (HW / SW)	X / X	X / X	X / X	X / X	X / X
Pipelined global shutter in trigger mode (p. 220)	X	X	-	-	-
Linescan mode	-	-	**available** (p. 569)	-	-
High color reproductivity (for color version)	X	X	X	X	X
Programmable readout timing with free capture windows and partial scan	X	X	X	X	X
Many trigger modes (free-running, software-triggered, hardware-triggered)	X	X	X	X	X
Flash control output, synchronous to integration period	X	X	X	X	X
Power consumption @ 24 V [W]	approx. 3.5 (-104)	approx. 3.5 (-104b)	approx. 3 (-104e)	approx. 3 (-105)	approx. 3.3 (-1010)

More specific data	**mvBlue↩ COUGAR-X[D]x04 (2.2 Mpix [2048 x 1088])** (p. 573)	**mvBlue↩ COUGAR-X[D]x04b (4.2 Mpix [2048 x 2048])** (p. 577)	**mvBlue↩ COUGAR-↩ X104e (1.9 Mpix [1600 x 1200])** (p. 567)	**mvBlue↩ COUGAR-↩ X105 (5 Mpix [2592 x 1944])** (p. 581)	**mvBlue↩ COUGAR-↩ X1010 (10 Mpix [3856 x 2764])** (p. 586)

[1] No AEC with external trigger
[2] Infrared enhanced
[3] Measured accord. to EMVA1288 with gray scale version of the camera

Note

For further information about rolling shutter, please have a look at the practical report about rolling shutter on our website: `https://www.matrix-vision.com/files/mv11/Glossary/art_↩ rolling_shutter_en.pdf`

For further information about image errors of image sensors, please have a look at **Correcting image errors of a sensor** (p. 284).

1.11.2.6 mvBlueCOUGAR-XD specific CMOS sensors

The CMOS sensor modules incorporate the following features:

< 5 Mpix

Sensors:	2.2 Mpix (-XD104) / (-XD204) / (-XD104a12)	4.2 Mpix (-XDx04b) / (-XD204b)	2.4 Mpix (-XD104d)	3.2 Mpix (-XD104h)
Sensor supplier	CMOSIS	CMOSIS	Sony	Sony
Sensor name	CMV2000	CMV4000	IMX174	IMX252
Resolution	2048 x 1088 gray scale or RGB	2048 x 2048 gray scale or RGB	1936 x 1216 gray scale or RGB	2064 x 1544 gray scale or RGB
Sensor size	2/3"	1"	1/1.2"	1/1.8"
Pixel clock [MHz]	30	30		
Max. FPS (in free-running full frame mode)	270 [**burst mode** (p. 245) required] / 106.5 [streaming]	27	164 [**burst mode** (p. 245) required] / 100.8 [streaming]	123 [**burst mode** (p. 245) required] / 74.5 [streaming]
Frame rate exactness (p. 145)	-	-	-	-
ADC resolution / Out [bit]	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8
SNR_{max} [dB][1]	39.2 | -XD104a12: 38.6	38.4 | -XD204b↩ : 40.2	45.1	40.3
DR (normal / **HDR** (p. 378)) [dB][1]	56.1 / | -XD104a12: 54.6 /	54.8 / | -XD204b↩ : 57.1 /	73.7 /	71.3 /
Progressive scan sensor (no interlaced problems!)	X	X	X	X
Rolling shutter	-	-	-	-
Global shutter	X	X	X	X
Trigger (HW / SW)	X / X	X / X	X / X	X / X

Pipelined global shutter in trigger mode (p. 220)	X	X	X	X
Linescan mode	-	-	-	-
High color reproductivity (for color version)	X	X	X	X
Programmable readout timing with free capture windows and partial scan	X	X	X	X
Many trigger modes (free-running, software-triggered, hardware-triggered)	X	X	X	X
Flash control output, synchronous to integration period	X	X	X	X
Power consumption @ 24 V [W]	approx. 5.3	approx. 5.3	approx. 5.0	approx. 5.5
More specific data	**mvBlueCOUGAR-X[D]x04 (2.2 Mpix [2048 x 1088])** (p. 573)	**mvBlueCOUGAR-X[D]x04b (4.2 Mpix [2048 x 2048])** (p. 577)	**mvBlueCOUGAR-XD104d (2.4 Mpix [1936 x 1216])** (p. 485)	**mvBlueCOUGAR-XD104h (3.2 Mpix [2064 x 1544])** (p. 493)

< 5 Mpix < 15 Mpix

Sensors:	5.1 Mpix (-XD105a)	7.1 Mpix (-XD107)	8.9 Mpix (-XD109b)	12.4 Mpix (-XD1012b)
Sensor supplier	Sony	Sony	Sony	Sony
Sensor name	IMX250	IMX420	IMX267	IMX304
Resolution	2464 x 2056 gray scale or RGB	3216 x 2208 gray scale or RGB	4112 x 2176 gray scale or RGB	4112 x 3008 gray scale or RGB
Sensor size	2/3"	1.1"	1"	1.1"
Pixel clock [MHz]				
Max. FPS (in free-running full frame mode)	80 [**burst mode** (p. 245) required] / 46.8 [streaming]	60.6 [**burst mode** (p. 245) required] / 33.4 [streaming]	32 [**burst mode** (p. 245) required] / 26.5 [streaming]	23 [**burst mode** (p. 245) required] / 19.2 [streaming]
Frame rate exactness (p. 145)	-	-	-	-
ADC resolution / Out [bit]	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8
SNR_{max} [dB][1]	40.3	43.9 [DualADC]	40.2	40.2
DR (normal / **HDR** (p. 378)) [dB][1]	71.4 /	74.1 [DualADC]	70.7 /	70.5 /
Progressive scan sensor (no interlaced problems!)	X	X	X	X
Rolling shutter	-	-	-	-
Global shutter	X	X	X	X
Trigger (HW / SW)	X / X	X / X	X / X	X / X

Pipelined global shutter in trigger mode (p. 220)	X	X	X	X
Linescan mode	-	-	-	-
High color reproductivity (for color version)	X	X	X	X
Programmable readout timing with free capture windows and partial scan	X	X	X	X
Many trigger modes (free-running, software-triggered, hardware-triggered)	X	X	X	X
Flash control output, synchronous to integration period	X	X	X	X
Power consumption @ 24 V [W]	approx. 5.5	approx. 6.3	approx. 5.8	approx. 5.8
More specific data	**mvBlueCOUGAR-XD105a (5.1 Mpix [2464 x 2056])** (p. 500)	**mvBlueCOUGAR-XD107 (7.1 Mpix [3216 x 2208])** (p. 507)	**mvBlueCOUGAR-X[D]109b (8.9 Mpix [4112 x 2176])** (p. 515)	**mvBlueCOUGAR-X[D]1012b (12.4 Mpix [4112 x 3008])** (p. 519)

> 15 Mpix

Sensors:	16.2 Mpix (-XD1016)	20.4 Mpix (-XD1020a)	24.6 Mpix (-XD1025)	31.5 Mpix (-XD1031)
Sensor supplier	Sony	Sony	Sony	Sony
Sensor name	IMX542	IMX541	IMX540	IMX342
Resolution	5328 x 3040 gray scale or RGB	4512 x 4512 gray scale or RGB	5328 x 4608 gray scale or RGB	6480 x 4856 gray scale or RGB
Sensor size	1.1"	1.1"	1.2"	APS-C
Pixel clock [MHz]				
Max. FPS (in free-running full frame mode)	26.2 [**burst mode** (p. 245) required] / 14.7 [streaming]	20.9 [**burst mode** (p. 245) required] / 11.7 [streaming]	27.6 [**burst mode** (p. 245) required] / 9.7 [streaming]	14.4 [**burst mode** (p. 245) required] / 7.5 [streaming]
Frame rate exactness (p. 145)	-	-	-	-
ADC resolution / Out [bit]	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8	12 / 16, 12, 8
SNR_{max} [dB][1]	39.8	39.8	39.6	39.6
DR (normal / **HDR** (p. 378)) [dB][1]	70.6 /	70.7	70.1	69.4
Progressive scan sensor (no interlaced problems!)	X	X	X	X
Rolling shutter	-	-	-	-
Global shutter	X	X	X	X
Trigger (HW / SW)	X / X	X / X	X / X	X / X

Pipelined global shutter in trigger mode (p. 220)	X	X	X	X
Linescan mode	-	-	-	-
High color reproductivity (for color version)	X	X	X	X
Programmable readout timing with free capture windows and partial scan	X	X	X	X
Many trigger modes (free-running, software-triggered, hardware-triggered)	X	X	X	X
Flash control output, synchronous to integration period	X	X	X	X
Power consumption @ 24 V [W]	approx. 4.7	approx. 4.7	approx. 5.3	approx. 6.8
More specific data	**mvBlueCOUGAR-XD1016 (16.2 Mpix [5328 x 3040])** (p. 456)	**mvBlueCOUGAR-XD1020a (20.4 Mpix [4512 x 4512])** (p. 460)	**mvBlueCOUGAR-XD1025 (24.6 Mpix [5328 x 4608])** (p. 463)	**mvBlueCOUGAR-XD1031 (31.5 Mpix [6480 x 4856])** (p. 523)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

Note

For further information about rolling shutter, please have a look at the practical report about rolling shutter on our website: `https://www.matrix-vision.com/files/mv11/Glossary/art_rolling_shutter_en.pdf`

For further information about image errors of image sensors, please have a look at the practical report on our website: `https://www.matrix-vision.com/files/mv11/Glossary/art_image_errors_sensors_en.pdf`

1.11.3 CCD sensors

The CCD sensors are highly programmable imaging modules which will, for example, enable the following type of applications

< 1.3 Mpix

Sensors	0.3 Mpix (-x20a)	0.3 Mpix (-x20b)	0.5 Mpix (-x20d)	1.2 Mpix (-x22)
Sensor supplier	Sony	Sony	Sony	Sony
Sensor name	ICX424 AL/AQ	ICX414 AL/AQ	ICX415 AL/AQ	ICX445 ALA/AQA
Resolution	640 x 480 gray scale or RGB	640 x 480 gray scale or RGB	776 x 580 gray scale or RGB	1280 x 960 gray scale or RGB
Pixel clock [MHz]	24.54 / 32 / 40	24.54 / 32 / 40	29.5 / 48	36 / 50

Max. FPS	104 | **mvBlueCOUGAR-3X** (p. 110): 64 (@ 8 bit per pixel)	104	87	31
Frame rate exactness (p. 145)	-			
ADC (on sensor board) resolution [bit]	14	14	14	14
Programmable analog gain and offset	X			
Frame integrating progressive scan sensor (no interlaced problems!)	X			
High resolution	X			
High color reproductivity (for color version)	X			
Low dark current	X			
Continuous variable-speed shutter	X			
Overlapped mode (p. 220)	X			
Low smear	X			
Excellent antiblooming characteristics	X			
Programmable integration time from usec to sec.	X			
Trigger (HW / SW)	X / X			
Flash control output, synchronous to integration period	X			
Windowing and partial scan	X			
Power consumption @ 24 V [W]	approx. 3.2 (-120a)	approx. 3.2 (-120b)	approx. 3.2 (-120d)	approx. 3.3 (-122)
More specific data	**mvBlueCOUGAR-X120a (0.3 Mpix [640 x 480])** (p. 593)	**mvBlueCOUGAR-X120b (0.3 Mpix [640 x 480])** (p. 599)	**mvBlueCOUGAR-X120d (0.5 Mpix [776 x 580])** (p. 605)	**mvBlueCOUGAR-X122 (1.2 Mpix [1280 x 960])** (p. 611)

> 1.3 Mpix

Sensors	1.4 Mpix (-x23)	1.9 Mpix (-x24)	5.1 Mpix (-x25a)	5.1 Mpix (-225)
Sensor supplier	Sony	Sony	Sony	Sony
Sensor name	ICX267 AL/AQ	ICX274 AL/AQ	ICX655 ALA/AQA	ICX625 ALA/AQA
Resolution	1360 x 1024 gray scale or RGB	1600 x 1200 gray scale or RGB	2448 x 2050 gray scale or RGB	2448 x 2050 gray scale or RGB
Pixel clock [MHz]	28 / 56	36 / 56 / 65	60 / 66	60 / 66

Max. FPS	30 | **mvBlue↩ COUGAR-3X** (p. 110): 30 (@ 8 bit per pixel)	28	10	16
Frame rate exactness (p. 145)	-			
ADC (on sensor board) resolution [bit]	14	14	14	14
Programmable analog gain and offset	X			
Frame integrating progressive scan sensor (no interlaced problems!)	X			
High resolution	X			
High color reproductivity (for color version)	X			
Low dark current	X			
Continuous variable-speed shutter	X			
Overlapped mode (p. 220)	X			
Low smear	X			
Excellent antiblooming characteristics	X			
Programmable integration time from usec to sec.	X			
Trigger (HW / SW)	X / X			
Flash control output, synchronous to integration period	X			
Windowing and partial scan	X			
Power consumption @ 24 V [W]	approx. 3.7 (-123)	approx. 4.1 (-124)	approx. 4.2 (-125a)	approx. 5.3 (-225)
More specific data	**mvBlueCOUGAR-X123 (1.4 Mpix [1360 x 1024])** (p. 617)	**mvBlueCOUGAR-X124 (1.9 Mpix [1600 x 1200])** (p. 624)	**mvBlueCOUGAR-X125a (5.1 Mpix [2448 x 2050])** (p. 642)	**mvBlueCOUGAR-X225 (5.1 Mpix [2448 x 2050])** (p. 637)

1.11.3.1 mvBlueCOUGAR-XD specific CCD sensors

The CCD sensors are highly programmable imaging modules which will, for example, enable the following type of applications

< 7 Mpix

Sensors	3 Mpix (-XD124a)	6 Mpix (-XD126)	6 Mpix (-XD126a)
Sensor supplier	Sony	Sony	Sony
Sensor name	ICX674 ALG/AQG	ICX694 ALG/AQG	ICX695 ALG/AQG
Resolution	1936 x 1460 gray scale or RGB	2752 x 2208 gray scale or RGB	2752 x 2208 gray scale or RGB
Pixel clock [MHz]	56 / 66	56 / 66	56
Max. FPS	64	33	14
Frame rate exactness (p. 145)	-		
ADC (on sensor board) resolution / Out [bit]	14 / 8 .. 16	14 / 8 .. 16	14 / 8 .. 16
Programmable analog gain and offset	X		
Frame integrating progressive scan sensor (no interlaced problems!)	X		
High resolution	X		
High color reproductivity (for color version)	X		
Low dark current	X		
Continuous variable-speed shutter	X		
Overlapped mode (p. 220)	X		
Low smear	X		
Excellent antiblooming characteristics	X		
Programmable integration time from usec to sec.	X		
Trigger (HW / SW)	X / X		
Flash control output, synchronous to integration period	X		
Windowing and partial scan	X		
Power consumption @ 24 V with 2x GigE lanes [W] Using one GigE lane reduces the consumption by about 1 W.	approx. 6.9 (2 taps, 56 MHz)	approx. 7.7 (4 taps, 56 MHz)	approx. 7.7 (2 taps, 56 MHz)
More specific data	**mvBlueCOUGAR-↩ XD124a (3 Mpix [1936 x 1460])** (p. 630)	**mvBlueCOUGAR-↩ XD126 (6 Mpix [2752 x 2208])** (p. 649)	**mvBlueCOUGAR-↩ XD126a (6 Mpix [2752 x 2208])** (p. 655)

> 7 Mpix

Sensors	9.2 Mpix (-XD129)	9.2 Mpix (-XD129a)	12.1 Mpix (-XD1212a)
Sensor supplier	Sony	Sony	Sony
Sensor name	ICX814 ALG/AQG	ICX815 ALG/AQG	ICX834 ALG/AQG
Resolution	3384 x 2712 gray scale or RGB	3384 x 2712 gray scale or RGB	4248 x 2836 gray scale or RGB
Pixel clock [MHz]	56 / 66	56	36 / 56

Max. FPS	22	9.5	7.5
Frame rate exactness (p. 145)	-		
ADC (on sensor board) resolution / Out [bit]	14 / 8 .. 16	14 / 8 .. 16	14 / 8 .. 16
Programmable analog gain and offset	X		
Frame integrating progressive scan sensor (no interlaced problems!)	X		
High resolution	X		
High color reproductivity (for color version)	X		
Low dark current	X		
Continuous variable-speed shutter	X		
Overlapped mode (p. 220)	X		
Low smear	X		
Excellent antiblooming characteristics	X		
Programmable integration time from usec to sec.	X		
Trigger (HW / SW)	X / X		
Flash control output, synchronous to integration period	X		
Windowing and partial scan	X		
Power consumption @ 24 V with 2x GigE lanes [W] Using one GigE lane reduces the consumption by about 1 W.	approx. 8.6 (4 taps, 66 MHz)	approx. 5.8 (2 taps, 66 MHz)	approx. 8.14 (4 taps, 56 MHz)
More specific data	**mvBlueCOUGAR-XD129 (9.2 Mpix [3384 x 2712])** (p. 661)	**mvBlueCOUGAR-XD129a (9.2 Mpix [3384 x 2712])** (p. 667)	**mvBlueCOUGAR-XD1212a (12.1 Mpix [4248 x 2836])** (p. 673)

1.11.3.2 Negative gain

The mvBlueCOUGAR-X cameras use a minimum needed basic gain for typical sensor output signals to get near the maximum **ADC** (p. 213) (analog-to-digital converter) saturation limit (Figure 3). This guarantees a higher sensitivity. However, to take advantage of the whole **ADC** (p. 213) range of the sensor, you can use **"negative gain"**. Together with enough illumination, you can get a better signal-to-noise ratio given that this setting will affect the analog signal before it is processed by the **ADC** (p. 213).

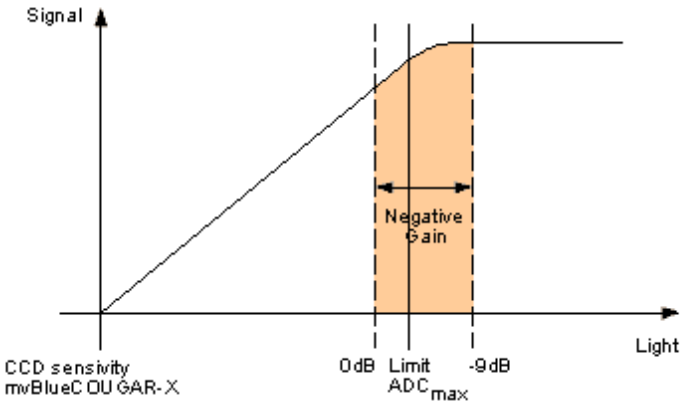


Figure 3: Negative gain diagram

Note

Depending on the value of the adjusted negative gain, it can be possible that single pixels pass over their signal limits. In the live image, you will have the effect that some pixels do not change their behavior and the whole image seems to be "frozen".

1.11.4 Output sequence of color sensors (RGB Bayer)

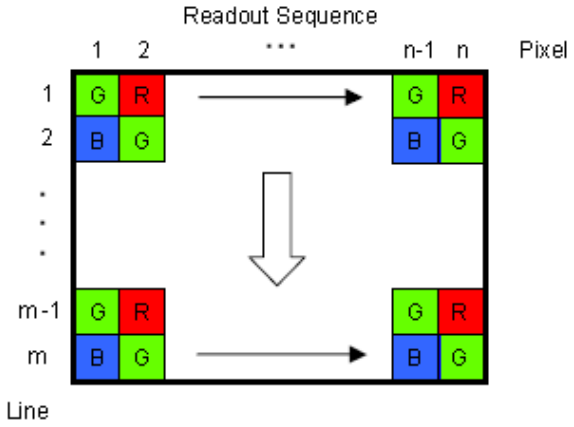


Figure 4: Output sequence of RAW data

1.11.5 Bilinear interpolation of color sensors (RGB Bayer)

For Bayer demosaicing in the camera, we use bilinear interpolation:

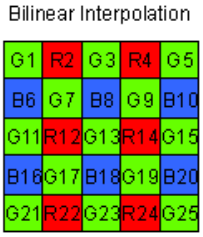


Figure 5: Bilinear interpolation

1. Interpolation of green pixels: the average of the upper, lower, left and right pixel values is assigned as the G value of the interpolated pixel.
   For example:

$$G8 = \frac{(G3+G7+G9+G13)}{4}$$

   For G7:

$$G7_new = 0.5 * G7 + 0.5 * \frac{(G1+G3+G11+G13)}{4}$$

2. Interpolation of red/blue pixels:
   Interpolation of a red/blue pixel at a green position: the average of two adjacent pixel values in corresponding color is assigned to the interpolated pixel.
   For example:

$$B7 = \frac{(B6+B8)}{2} \quad ; \quad R7 = \frac{(R2+R12)}{2}$$

   Interpolation of a red/blue pixel at a blue/red position: the average of four adjacent diagonal pixel values is assigned to the interpolated pixel.
   For example:

$$R8 = \frac{(R2+R4+R12+R14)}{4} \quad ; \quad B12 = \frac{(B6+B8+B16+B18)}{4}$$

Any colored edge which might appear is due to Bayer false color artifacts.

Note

There are more advanced and adaptive methods (like edge sensitive ones) available if the host is doing this debayering.

1.11.6 Supported image formats

It depends on the sensor model which image formats are actually supported.

The parameter **PixelFormat** in the **ImageFormatControl** (p. 143) lists all the supported image formats of the specific camera model. In **wxPropView** (p. 139) you can find the **PixelFormat** parameter in *"Setting -> Base -> Camera -> GenICam -> Image Format Control -> Pixel Format"*.

Gray scale version	Color version	Color version (**3 head model** (p. 110)) Each color channel represents one sensor
Mono8		
Mono10		
Mono12		
Mono14		
Mono16		
Mono12Packed		
	BayerGR8 (GR, RG, GB, BG depends on camera type)	
	BayerGR10 (GR, RG, GB, BG depends on camera type)	

	BayerGR12 (GR, RG, GB, BG depends on camera type)	
	BayerGR16 (GR, RG, GB, BG depends on camera type)	
	BayerGR12Packed (GR, RG, GB, BG depends on camera type)	
	BayerGR12Packed (GR, RG, GB, BG depends on camera type)	
	RGB8Packed	
	BGR8Packed	
	BGRA8Packed	
	BGR10V2Packed	
		RGB8
		BGR8
		RGB10p32 (10 bit packed in 32 bit pixel)
	YUV422Packed	
	YUV422_YUYVPacked	
	YUV444Packed	

See also

For more details about the image formats, please have a look at the enums `TImageDestination↩PixelFormat (C++)` and `TImageBufferPixelFormat (C++)` in the `mvIMPACT Acquire API manuals`.
An example application about the pixel formats called **ContinuousAllFormats** is also available.

1.12 Filters

MATRIX VISION offers several specific filters for cameras. The **hot mirror filter** (p. 136) is part of the standard delivery condition.

1.12.1 Hot Mirror Filter

The hot mirror filter has great transmission in the visible spectrum and blocks out a significant portion of the IR energy.

Technical data	
Diameter	Varies depending on the camera
Thickness	1.0 mm
Material	Borofloat
Characteristics	T = 50% @ 650 +/- 10 nm
	T $>$ 92% 390-620 nm
	R_{avg} $>$ 95% 700-1150 nm
	AOI = 0 degrees
Surface quality	Polished on both sides P4
Surface irregularity	5/3x0.06 on both sides

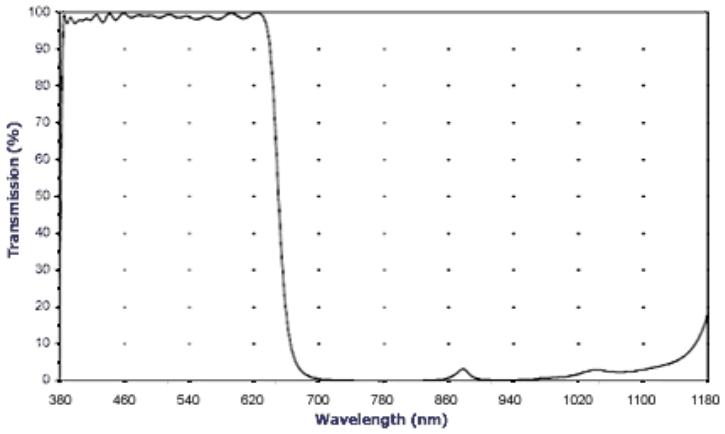


Figure 1: IR-CUT wavelengths and transmission diagram

1.12.2 Cold mirror filter

The high-quality daylight cut filter has optically polished surfaces. The polished surface allows the use of the filter directly in the path of rays in image processing applications. The filter is protected against scratches during the transport by a protection film that has to be removed before the installing the filter.

Technical data	
Diameter	Varies depending on the camera
Thickness	1.0 mm
Material	Solaris S 306
Characteristics	$T_{avg} > 80\% > 780$ nm
	AOI = 0 degrees
	Protective foil on both sides
	Without antireflexion
	Without bezel

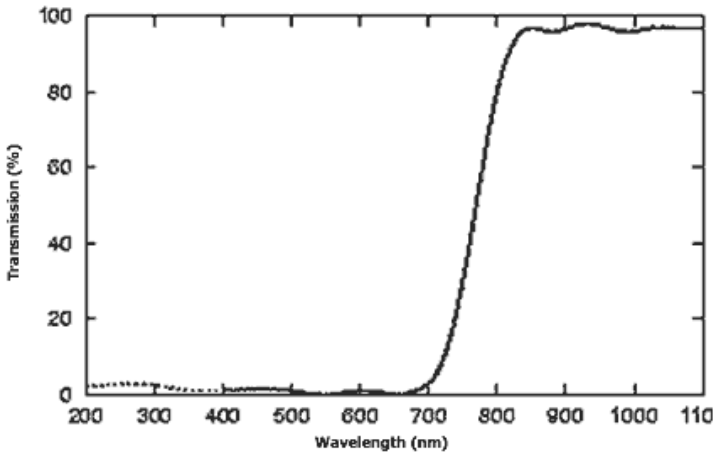


Figure 2: DL-CUT wavelengths and transmission diagram

1.12.3 Glass filter

It is also possible to choose a glass filter with following characteristics:

Technical data	
Glass thickness	1.0 mm
Material	Borofloat without coating
	ground with protection chamfer
Surface quality	polished on both sides P4
Surface irregularity	5/3x0.06 on both sides

1.13 GUI tools

1.13.1 Introduction

MATRIX VISION provides several convenient tools with graphical user interface to set up and work with their devices. Please find a short list and description below:

1.13.2 wxPropView

With wxPropView it is possible

- to acquire images
- to configure the device, and
- to display and modify the device properties.

1.13.3 mvDeviceConfigure

mvDeviceConfigure can be used to

- to set the device ID
- to update firmware- to upload a **GenICam** (p. 213) XML file (mvBlueCOUGAR-S devices only) - to disable CPU sleep states(some version of Windows only)

1.13.4 mvIPConfigure

With mvIPConfigure it is possible

- to configure the network behavior of GigE Vision™ devices
- to determine and solve network issues.

1.13.5 mvGigEConfigure

With mvGigEConfigure it is possible

- to install, remove or configure the MATRIX VISION GigE Vision™ capture filter driver.

See also

For further information about the tools, please follow the link to the separate manual describing the GUI tools in great detail on our website: `https://www.matrix-vision.com/manuals/`

1.14 GenICam and advanced features

1.14.1 Introduction

For new applications or to set the device via **wxPropView** (p. 139) we recommend to use the **GenICam** (p. 213) interface layout as it allows the most flexible access to the device features.

After you've set the interface layout to **GenICam** (p. 213) (either programmed or using **wxPropView** (p. 139)), all **GenICam** (p. 213) controls of the device are available.

> **Note**
>
> It depends on the device, which controls are supported.

In **wxPropView** (p. 139), you can see them in *"Setting -> Base -> Camera -> GenICam"*:

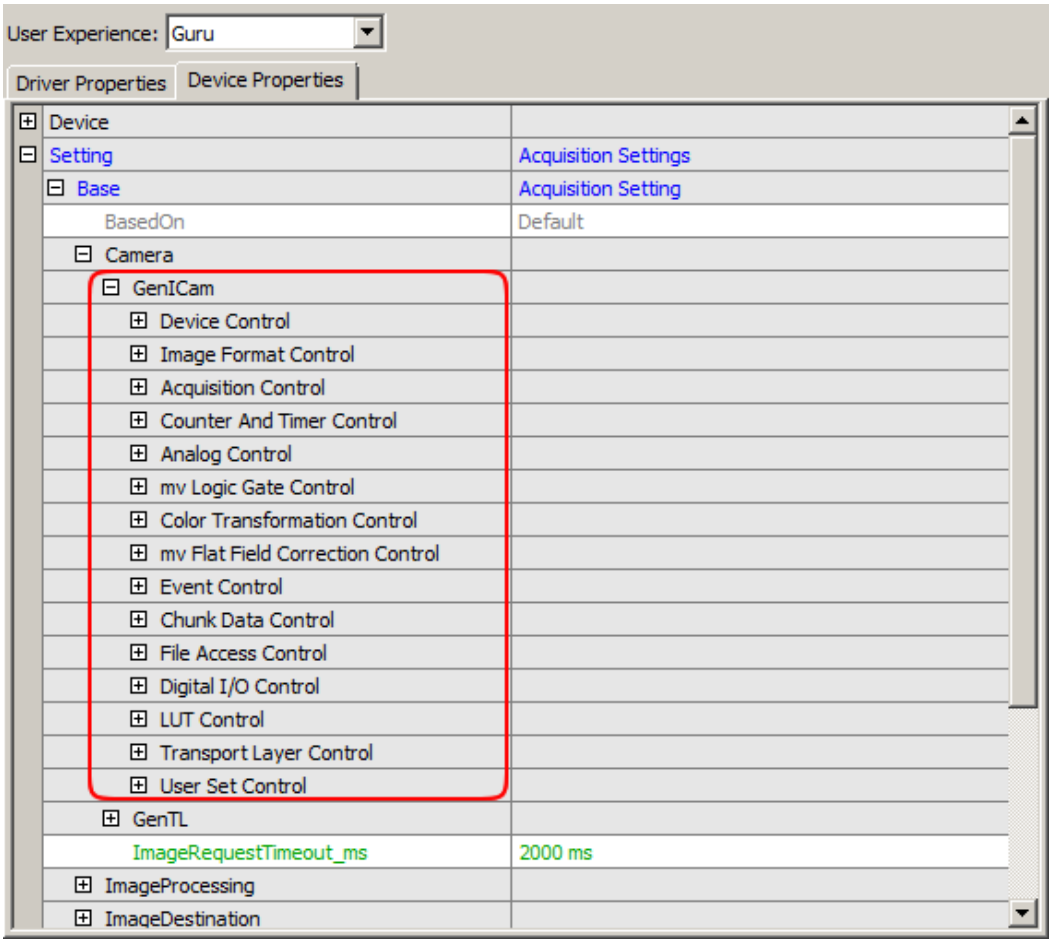


Figure 1: wxPropView - GenICam controls (depends on device and FW version)

As you can see, there are some controls with and without the prefix **"mv"**.

- **"mv"** prefix features are unique non-standard features developed by MATRIX VISION.
- Without **"mv"** are standard features as known from the **Standard Feature Naming Convention of GenICam properties** (p. 221) (SFNC).

All those features are **"camera based** / **device based"** features which can also be accessed using the camera with other **GenICam** (p. 213) / **GigE Vision** (p. 215) compliant third-party software.

Note

In GigE Vision timestamps are denoted in "device ticks" but for MATRIX VISION devices this equals microseconds.

Do not mix up the camera based / device based features with the features available in *"Setting -> Base -> Image Processing"*. Theses features are driver based features which are processed by the software and therefore need CPU load.

1.14.2 Device Control

The **"Device Control"** contains the features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
DeviceType	*deviceType*	Returns the device type.
DeviceScanType	*deviceScanType*	Scan type of the sensor of the device.
DeviceVendorName	*deviceVendorName*	Name of the manufacturer of the device.
DeviceModelName	*deviceModelName*	Name of the device model.
DeviceManufacturerInfo	*deviceManufacturerInfo*	Manufacturer information about the device.
DeviceVersion	*deviceVersion*	Version of the device.
DeviceFirmwareVersion	*deviceFirmwareVersion*	Firmware version of the device.
DeviceSerialNumber	*deviceSerialNumber*	Serial number of the device.
DeviceUserID	*deviceUserID*	User-programmable device identifier.
DeviceTLVersionMajor	*deviceTLVersionMajor*	Major version of the transport layer of the device.
DeviceTLVersionMinor	*deviceTLVersionMinor*	Minor version of the transport layer of the device.
DeviceLinkSpeed	*deviceLinkSpeed*	Indicates the speed of transmission negotiated on the specified Link.
DeviceTemperature	*deviceTemperature*	Device temperature.
etc.		

related to the device and its sensor.

This control provides a **bandwidth control feature** (p. 343). You have to select **DeviceLinkThroughputLimit**. Here you can set the maximum bandwidth in KBps.

Additionally, MATRIX VISION offers two temperature sensors and for this special features

- device temperature upper limit (**mvDeviceTemperatureUpperLimit**)
- device temperature lower limit (**mvDeviceTemperatureLowerLimit**)
- device temperature limit hysteresis (**mvDeviceTemperatureLimitHysteresis**)

The use case **Working with the temperature sensors** (p. 404) shows how you can work with the temperature sensors.

MATRIX VISION offers also some information properties about the

- FPGA
  - **mvDeviceFPGAVersion**
- and the image sensor
  - **mvDeviceSensorColorMode**

A further use case related to the **"Device Control"** is:

- **Disabling the heartbeat** (p. 406)

1.14.3 Image Format Control

The **"Image Format Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
SensorWidth	*sensorWidth*	Effective width of the sensor in pixels.
SensorHeight	*sensorHeight*	Effective height of the sensor in pixels.
SensorName	*sensorName*	Name of the sensor.
Width	*width*	Width of the image provided by the device (in pixels).
Height	*height*	Height of the image provided by the device (in pixels).
BinningHorizontal, BinningVertical	*binningHorizontal*, *binningVertical*	Number of horizontal/vertical photo-sensitive cells to combine together.
DecimationHorizontal, DecimationVertical	*decimationHorizontal*, *decimationVertical*	Sub-sampling of the image. This reduces the resolution (width) of the image by the specified decimation factor.
TestPattern	*testPattern*	Selects the type of test image that is sent by the device.
etc.		

related to the format of the transmitted image.

With **TestPattern**, for example, you can select the type of test image that is send by the device. Here, MATRIX VISION offers two special types:

- **mvBayerRaw** (the Bayer mosaic raw image)
- **mvFFCImage** (the **flat-field correction image** (p. 289))

Additionally, MATRIX VISION offers numerous additional features like:

- **mvMultiAreaMode**
  which can be used to define multiple AOIs (Areas of Interests) in one image.

  See also

  The use case **Working with multiple AOIs (mv Multi Area Mode)** (p. 241) shows how this feature works.

1.14.4 Acquisition Control

The **"Acquisition Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
AcquisitionMode	*acquisitionMode*	Sets the acquisition mode of the device. The different modes configures a device to send • exactly **one** image (**"SingleFrame"**), • exactly the **set number** of frames (**"MultiFrame"**) or • images constantly until an application explicitly stops the acquisition again (**"Continuous"**). and can be used for asynchronously grabbing and sending image(s). It works with internal and external hardware trigger where the edge is selectable. The external trigger uses ImageRequestTimeout (ms) to time out. • The chapter **"How To See The First Image"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual shows how to acquire images with **wxPropView** (p. 139). • The use case **Acquiring a number of images** (p. 223) shows how to acquire a number of images; also triggered externally.
AcquisitionStart	*acquisitionStart*	Starts the acquisition of the device.
AcquisitionStop	*acquisitionStop*	Stops the acquisition of the device at the end of the current Frame.
AcquisitionAbort	*acquisitionAbort*	Aborts the acquisition immediately.

AcquisitionFrameRate	*acquisitionFrameRate*	Controls the acquisition rate (in Hertz) at which the frames are captured. Some cameras support a special internal trigger mode that allows more exact frame rates. This feature keeps the frame rate constant to an accuracy of +/-0.005 fps at 200 fps. This is achieved using frames with a length difference of up to 1 us. Please check in the **sensor summary** (p. 116) if this feature exists for the requested sensor.
TriggerSelector	*triggerSelector*	Selects the type of trigger to configure. A possible option is **mv↩ TimestampReset**. The use case about **mvTimestampReset is available** (p. 407).
TriggerOverlap[TriggerSelector]	*triggerOverlap*	Specifies the type trigger overlap permitted with the previous frame. *TriggerOverlap* is only intended for external trigger (which is usually non-overlapped: i.e. exposure and readout are sequentially). This leads to minimal latency / jitter between trigger and exposure. However, the camera accepts a new trigger (the exposure time earlier) before the end of the transmission of the current image. `Maximum frame rate in triggered mode = frame rate of continuous mode` This however leads to higher latency / jitter between trigger and exposure. A trigger will be **not** latched if it occurs before this moment (trigger is accurate in time). **See also** **Principles of overlapped and pipelined trigger** (p. 220).
ExposureMode	*exposureMode*	Sets the operation mode of the exposure (or shutter).
ExposureTime	*exposureTime*	Sets the exposure time (in microseconds) when *ExposureMode* is **Timed** and *ExposureAuto* is **Off**.
ExposureAuto	*exposureAuto*	Sets the automatic exposure mode when *ExposureMode* is **Timed**.
etc.		

related to the image acquisition, including the triggering mode.

Additionally, MATRIX VISION offers numerous additional features like:

- **mvShutterMode**
  which selects the shutter mode of the CMOS sensors like rolling shutter or global shutter.
- **mvDefectivePixelEnable**
  which activates the sensor's defective pixel correction.

  See also

  https://www.matrix-vision.com/files/mv11/Glossary/art_image_errors_sensors_en.pdf
- **mvExposureAutoSpeed**
  which determines the increment or decrement size of exposure value from frame to frame.
- **mvExposureAutoDelayImages**
  the number of frames that the AEC must skip before updating the exposure register.
- **mvExposureAutoAverageGrey**
  common desired average grey value (in percent) used for Auto Gain Control (AGC) and Auto Exposure Control (AEC).
- **mvExposureAutoAOIMode**
  common AutoControl AOI used for Auto Gain Control (AGC), Auto Exposure Control (AEC) and Auto White Balance (AWB).
- **mvAcquisitionMemoryMode**
  MATRIX VISION offers three additional acquisition modes which use the internal memory of the camera:
  - **mvRecord** (p. 224)
    which is used to store frames in memory.
  - **mvPlayback** (p. 224)
    which transfers stored frames.
    - **mvPretrigger** (p. 225)
      which stores frames in memory to be transferred after trigger.
      To define the number of frames to acquire before the occurrence of an AcquisitionStart or AcquisitionActive trigger, you can use **mvPretriggerFrameCount**.

      See also

      The use case **Recording sequences with pre-trigger** (p. 225) shows how this feature works.
- **mvAcquisitionMemoryMaxFrameCount**
  which shows the maximum of frames the internal memory can save.

  See also

  The use case **Working with burst mode buffer** (p. 245) lists some maximum frame counts of some camera models.
- **mvSmearReduction**
  smear reduction in triggered and non-overlapped mode.
- **mvSmartFrameRecallEnable**
  which configures the internal memory to store each frame (thats gets transmitted to the host) in full resolution.

  See also

  The use case **SmartFrameRecall** (p. 249) shows how this feature works.

Since

FW Revision 2.40.2546.0

Common properties for *AutoExposureControl*, *AutoGainControl* , and *AutoWhiteBalance* are available via **mv Auto Feature Control** (p. 169).

For **"Exposure Auto Mode"**, in **wxPropView** (p. 139) just select in *"Exposure Auto"* **"Continuous"**. Afterwards, you have the possibility to set lower and upper limit, average gray combined with AOI setting:

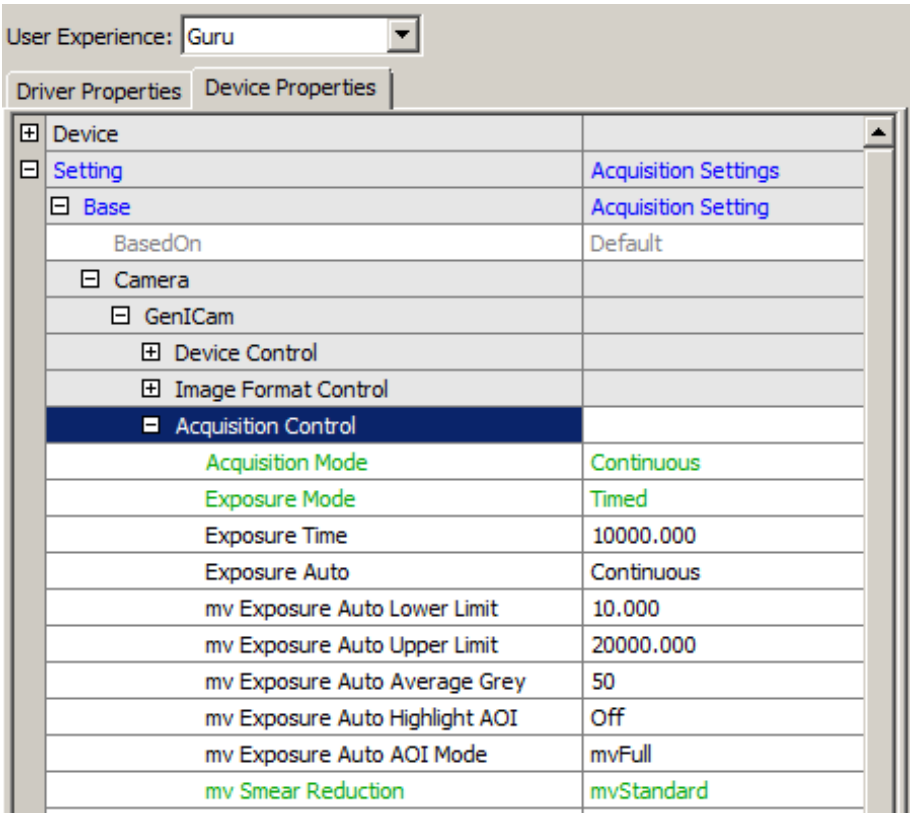


Figure 2: Acquire Control -> Exposure Auto

- **mvSmartFrameRecallFrameSkipRatio**
  When set to a value != 0, the smaller frames get thinned out. AOI requests can still be done for all frames.
- **mvSmartFrameRecallTimestampLookupAccuracy**
  is needed for the SkipRatio feature since you **don't know** the timestamps of the **internal** frames. This value defines the strictness of the timestamp-check for the recalled image (given in us).

1.14.5 Counter And Timer Control

The **"Counter And Timer Control"** is a powerful feature which MATRIX VISION customers already know under the name Hardware Real-Time Controller (HRTC). MATRIX VISION cameras provide:

- 4 counters for counting events or external signals (compare number of triggers vs. number of frames; over-trigger) and
- 2 timers.

Counter and Timers can be used, for example,

- for **pulse width modulation (PWM)** (p. 351) and
- to generate output signals of variable length, depending on conditions in camera.

This achieves complete HRTC functionality which supports following applications:

- frame rate by timer
- exposure time by timer
- pulse width at input

The **"Counter And Timer Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
CounterSelector	*counterSelector*	Selects which counter to configure.
CounterEventSource[Counter↩ Selector]	*counterEventSource*	Selects the events that will be the source to increment the counter.
CounterEventActivation[Counter↩ Selector]	*counterEventActivation*	Selects the activation mode event source signal.
etc.		
TimerSelector	*timerSelector*	Selects which timer to configure.
TimerDuration[TimerSelector]	*timerDuration*	Sets the duration (in microseconds) of the timer pulse.
TimerDelay[TimerSelector]	*timerDelay*	Sets the duration (in microseconds) of the delay.
etc.		

related to the usage of programmable counters and timers.

Because there are many ways to use this feature, the list of use cases is long and not finished yet:

- **Processing triggers from an incremental encoder** (p. 348)
- **Creating different exposure times for consecutive images** (p. 355)
- **Creating synchronized acquisitions using timers** (p. 424)
- **Generating a pulse width modulation (PWM)** (p. 351)

- **Outputting a pulse at every other external trigger** (p. 354)
- **Generating very long exposure times** (p. 239)

1.14.6 Analog Control

The **"Analog Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
GainSelector	*gainSelector*	Selects which gain is controlled by the various gain features.
Gain[GainSelector]	*gain*	Controls the selected gain as an absolute physical value [in dB].
GainAuto[GainSelector]	*gainAuto*	Sets the automatic gain control (AGC) mode.
GainAutoBalance	*gainAutoBalance*	Sets the mode for automatic gain balancing between the sensor color channels or taps.
BlackLevelSelector	*blackLevelSelector*	Selects which black level is controlled by the various BlackLevel features.
BlackLevel[BlackLevelSelector]	*blackLevel*	Controls the selected BlackLevel as an absolute physical value.
BalanceWhiteAuto	*balanceWhiteAuto*	Controls the mode for automatic white balancing between the color channels.
Gamma	*gamma*	Controls the gamma correction of pixel intensity.
etc.		

related to the video signal conditioning in the analog domain.

Additionally, MATRIX VISION offers:

- **mvBalanceWhiteAuto** functions and
- **mvGainAuto** functions.

In **wxPropView** (p. 139) just select in *"Gain Auto"* (AGC) **"Continuous"**. Afterwards, you have the possibility to set minimum and maximum limit combined with AOI setting:

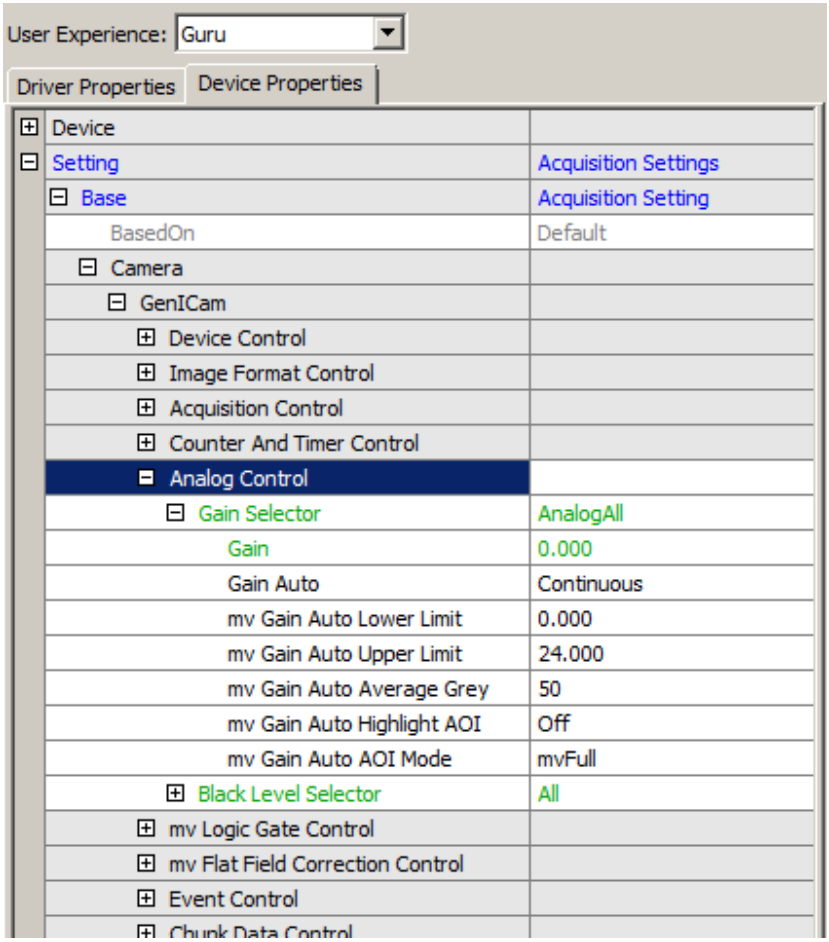


Figure 3: Analog Control -> Gain Auto

See also

Optimizing the color/luminance fidelity of the camera (p. 297)

1.14.7 Color Transformation Control

The **"Color Transformation Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
ColorTransformationSelector	*colorTransformationEnable*	Activates the selected color transformation module.
ColorTransformationSelector	*colorTransformationSelector*	Selects which color transformation module is controlled by the various color transformation features.
ColorTransformationValue	*colorTransformationValue*	Represents the value of the selected Gain factor or Offset inside the transformation matrix.
ColorTransformationValueSelector	*colorTransformationValueSelector*	Selects the gain factor or Offset of the Transformation matrix to access in the selected color transformation module.

related to the control of the color transformation.

This control offers an enhanced color processing for optimum color fidelity using a **color correction matrix** (CCM) and enables

- 9 coefficients values ($Gain_{00}$ .. $Gain_{22}$) and
- 3 offset values ($Offset_0$ .. $Offset_2$)

to be entered for RGB_{IN} -> RGB_{OUT} transformation. This can be used to optimize specific colors or specific color temperatures.

$$\begin{pmatrix} R\\ G\\ B \end{pmatrix}_{out} = \begin{pmatrix} Gain00 & Gain01 & Gain02\\ Gain10 & Gain11 & Gain12\\ Gain20 & Gain21 & Gain22 \end{pmatrix} \begin{pmatrix} C0\\ C1\\ C2 \end{pmatrix}_{in} + \begin{pmatrix} Offset0\\ Offset1\\ Offset2 \end{pmatrix}$$

$$\text{Equivalent: } \begin{pmatrix} R\\ G\\ B \end{pmatrix}_{out} = \begin{pmatrix} RR & RG & RB\\ GR & GG & GB\\ BR & BG & BB \end{pmatrix} \begin{pmatrix} R\\ G\\ B \end{pmatrix}_{in} + \begin{pmatrix} R\\ G\\ B \end{pmatrix}_{offset}$$

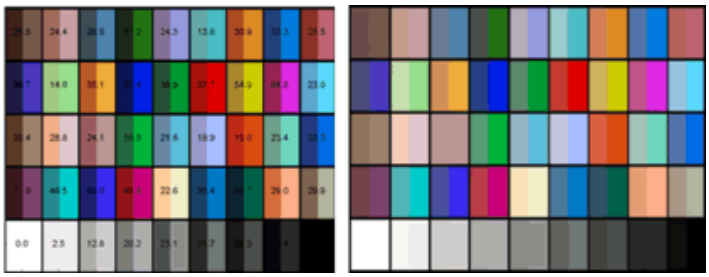


Figure 4: Color correction sample

Coefficients will be made available for sensor models and special requirements on demand.

See also

Optimizing the color/luminance fidelity of the camera (p. 297)

1.14.8 Event Control

The **"Event Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
EventSelector	*eventSelector*	Selects which Event to signal to the host application.
EventNotification[EventSelector]	*eventNotification*	Activate or deactivate the notification to the host application of the occurrence of the selected Event.
EventFrameTriggerData	*eventFrameTriggerData*	Category that contains all the data features related to the *FrameTrigger* event.
EventFrameTrigger	*eventFrameTrigger*	Returns the unique identifier of the *FrameTrigger* type of event.
EventFrameTriggerTimestamp	*eventFrameTriggerTimestamp*	Returns the timestamp of the *AcquisitionTrigger* event.
EventFrameTriggerFrameID	*eventFrameTriggerFrameID*	Returns the unique identifier of the frame (or image) that generated the *FrameTrigger* event.
EventExposureEndData	*eventExposureEndData*	Category that contains all the data features related to the *ExposureEnd* event.
EventExposureEnd	*eventExposureEnd*	Returns the unique identifier of the *ExposureEnd* type of event.
EventExposureEndTimestamp	*eventExposureEndTimestamp*	Returns the timestamp of the *ExposureEnd* Event.
EventExposureEndFrameID	*eventExposureEndFrameID*	Returns the unique identifier of the frame (or image) that generated the *ExposureEnd* event.
EventErrorData	*eventErrorData*	Category that contains all the data features related to the error event.
EventError	*eventError*	Returns the unique identifier of the error type of event.
EventErrorTimestamp	*eventErrorTimestamp*	Returns the timestamp of the error event.
EventErrorFrameID	*eventErrorFrameID*	If applicable, returns the unique identifier of the frame (or image) that generated the error event.
EventErrorCode	*eventErrorCode*	Returns an error code for the error(s) that happened.
etc.		

related to the generation of Event notifications by the device.

The use case **Working with Event Control** (p. 258) shows how this control can be used.

1.14.9 Chunk Data Control

The **"Chunk Data Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
ChunkModeActive	*chunkModeActive*	Activates the inclusion of chunk data in the payload of the image.
ChunkSelector	*chunkSelector*	Selects which chunk to enable or control.
ChunkEnable[ChunkSelector]	*chunkEnable*	Enables the inclusion of the selected chunk data in the payload of the image.
ChunkImage	*chunkImage*	Returns the entire image data included in the payload.
ChunkOffsetX	*ChunkOffsetX*	Returns the offset x of the image included in the payload.
ChunkOffsetY	*chunkOffsetY*	Returns the offset y of the image included in the payload.
ChunkWidth	*chunkWidth*	Returns the width of the image included in the payload.
ChunkHeight	*chunkHeight*	Returns the height of the image included in the payload.
ChunkPixelFormat	*chunkPixelFormat*	Returns the pixel format of the image included in the payload.
ChunkTimestamp	*chunkTimestamp*	Returns the timestamp of the image included in the payload at the time of the *FrameStart* internal event.
etc.		

related to the Chunk Data Control.

A description can be found in the image acquisition section of the **"mvIMPACT Acquire API"** manuals.

1.14.10 File Access Control

The **"File Access Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
FileSelector	*fileSelector*	Selects the target file in the device.
FileOperationSelector[File↩ Selector]	*fileOperationSelector*	Selects the target operation for the selected file in the device.
FileOperationExecute[File↩ Selector][FileOperationSelector]	*fileOperationExecute*	Executes the operation selected by *FileOperationSelector* on the selected file.
FileOpenMode[FileSelector]	*fileOpenMode*	Selects the access mode in which a file is opened in the device.
FileAccessBuffer	*fileAccessBuffer*	Defines the intermediate access buffer that allows the exchange.
etc.		

related to the File Access Control that provides all the services necessary for generic file access of a device.

The use case **Working with the UserFile section (Flash memory)** (p. 400) shows how this control can be used.

1.14.11 Digital I/O Control

The **"Digital I/O Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
LineSelector	*lineSelector*	Selects the physical line (or pin) of the external device connector to configure.
LineMode[LineSelector]	*lineMode*	Controls if the physical Line is used to Input or Output a signal.
UserOutputSelector	*userOutputSelector*	Selects which bit of the user output register will be set by *UserOutput↩ Value*.
UserOutputValue[UserOutput↩ Selector]	*userOutputValue*	Sets the value of the bit selected by *UserOutputSelector*.
etc.		

related to the control of the general input and output pins of the device.

Additionally, MATRIX VISION offers:

- **mvLineDebounceTimeRisingEdge** and
- **mvLineDebounceTimeFallingEdge** functionality.

A description of these functions can be found in the use case **Creating a debouncing filter at the inputs** (p. 370).

How you can test the digital inputs and outputs is described in **"Testing The Digital Inputs"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual. The use case **Creating synchronized acquisitions using timers** (p. 424) is a further example which shows you how to work with digital inputs and outputs.

1.14.12 Encoder Control

The **"Encoder Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
EncoderSourceA	*encoderSourceA*	Selection of the A input line.
EncoderSourceB	*encoderSourceB*	Selection of the B input line.
EncoderMode [FourPhase]	*encoderMode*	The counter increments or decrements 1 for every full quadrature cycle.
EncoderDivider	*encoderDivider*	Sets how many Encoder increment/decrements that are needed generate an encoder output signal.
EncoderOutputMode	*encoderOutputMode*	Output signals are generated at all new positions in one direction. If the encoder reverses no output pulse are generated until it has again passed the position where the reversal started.
EncoderValue	*encoderValue*	Reads or writes the current value of the position counter of the selected Encoder. Writing to EncoderValue is typically used to set the start value of the position counter.

related to the usage if quadrature encoders.

The following figure explains the different **EncoderOutputModes** :

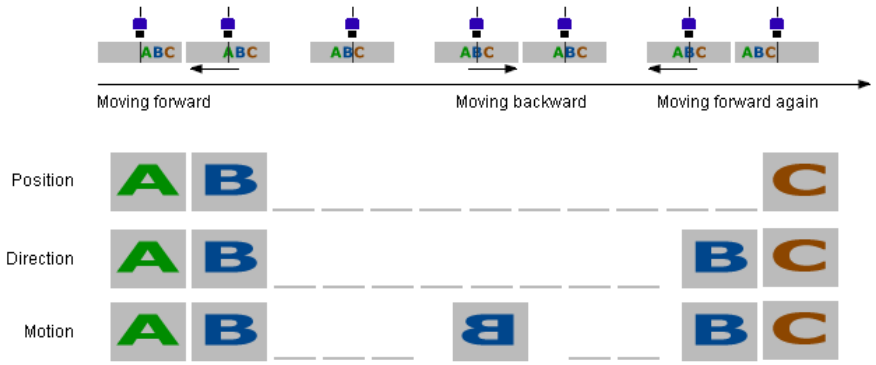


Figure 5: EncoderOutputModes

Additionally, the **Encoder** is also available as **TriggerSource** and as an **EventSource**.

A description of incremental encoder's principle can be found in the use case **Processing triggers from an incremental encoder** (p. 348).

1.14.13 Sequencer Control

1.14.13.1 Sequencer overview

The purpose of a sequencer is to allow the user of a camera to define a series of feature sets for image acquisition which can consecutively be activated during the acquisition by the camera. Accordingly, the proposed sequence is configured by a list of parameter sets. Each of these sequencer sets contains the settings for a number of camera features. Similar to user sets, the actual settings of the camera are overwritten when one of these sequencer sets is loaded. The order in which the features are applied to the camera depends on the design of the vendor. It is recommended to apply all the image related settings to the camera, before the first frame of this sequence is captured. The sequencer sets can be loaded and saved by selecting them using SequencerSetSelector. The Execution of the sequencer is completely controlled by the device.

(quoted from the `GenICam SFNC 2.3`)

1.14.13.2 Configuration of a sequencer set

The index of the adjustable sequencer set is given by the SequencerSetSelector. The number of available sequencer sets is directly given by the range of this feature. The features which are actually part of a sequencer set are defined by the camera manufacturer. These features can be read by **SequencerFeatureSelector** and activated by **SequencerFeatureEnable**[SequencerFeatureSelector]. This configuration is the same for all Sequencer Sets. To configure a sequencer set the camera has to be switched into configuration mode by SequencerConfigurationMode. Then the user has to select the desired sequencer set he wants to modify with the SequencerSetSelector. After the user has changed all the needed camera settings it is possible to store all these settings within a selected sequencer set by executing **SequencerSetSave**[SequencerSetSelector]. The user can also read back these settings by executing **SequencerSetLoad**[SequencerSetSelector]. To permit a flexible usage, more than one possibility to go from one sequencer set to another can exist. Such a path is selected by **SequencerPathSelector**[SequencerSetSelector]. Each path and therefore the transition between different sequencer sets is based on a defined trigger and an aimed next sequencer set which is selectable by **SequencerSetNext**[SequencerSetSelector][SequencerPathSelector]. After the trigger occurs the settings of the next set are active. The trigger is defined by the features **SequencerTriggerSource**[SequencerSetSelector][SequencerPathSelector] and **SequencerTriggerActivation**[SequencerSetSelector][SequencerPathSelector]. The functions of these features are the same as **TriggerSource** and **TriggerActivation**. For a flexible sequencer implementation, the **SequencerPathSelector**[SequencerSetSelector] should be part of the sequencer sets.

(quoted from the `GenICam SFNC 2.3`)

The **"Sequencer Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
SequencerMode	*sequencerMode*	Controls if the sequencer mechanism is active. Possible values are: • Off: Disables the sequencer. • On: Enables the sequencer.

SequencerConfigurationMode	*sequencerConfigurationMode*	Controls if the sequencer configuration mode is active. Possible values are: • Off: Disables the sequencer configuration mode. • On: Enables the sequencer configuration mode.
SequencerFeatureSelector	*sequencerFeatureSelector*	Selects which sequencer features to control. The feature lists all the features that can be part of a device sequencer set. All the device's sequencer sets have the same features. Note that the name used in the enumeration must match exactly the device's feature name.
SequencerFeature↩ Enable[SequencerFeature↩ Selector]	*sequencerFeatureEnable*	Enables the selected feature and make it active in all the sequencer sets.
SequencerSetSelector	*sequencerSetSelector*	Selects the sequencer set to which further settings applies.
SequencerSetSave	*sequencerSetSave*	Saves the current device state to the selected sequencer set selected by *SequencerSetSelector*.
SequencerSetLoad	*sequencerSetLoad*	Loads the sequencer set selected by *SequencerSetSelector* in the device. Even if *SequencerMode* is **Off**, this will change the device state to the configuration of the selected set.
SequencerSetActive	*sequencerSetActive*	Contains the currently active sequencer set.
SequencerSetStart	*sequencerSetStart*	Sets the initial/start sequencer set, which is the first set used within a sequencer.
SequencerPath↩ Selector[SequencerSetSelector]	*sequencerPathSelector*	Selects to which branching path further path settings apply.
SequencerSetNext	*sequencerSetNext*	Select the next sequencer set.

SequencerTriggerSource	*sequencerTriggerSource*	Specifies the internal signal or physical input line to use as the sequencer trigger source. Values supported by MATRIX VISION devices are: • Off: Disables the sequencer trigger. • ExposureEnd: Starts with the reception of the *ExposureEnd* event. • Counter1End: Starts with the reception of the *Counter1End* event. • UserOutput0: Specifies *UserOutput0* bit signal to use as internal source for the trigger. Other possible values that might be supported by third party devices are: • AcquisitionTrigger: Starts with the reception of the Acquisition Trigger. • AcquisitionTriggerMissed: Starts with the reception of the missed Acquisition Trigger. • AcquisitionStart: Starts with the reception of the Acquisition Start. • AcquisitionEnd: Starts with the reception of the Acquisition End. • FrameTrigger: Starts with the reception of the Frame Start Trigger. • FrameTriggerMissed: Starts with the reception of the missed Frame Trigger.
		• FrameStart: Starts with the reception of the Frame Start.

SequencerTriggerActivation	*sequencerTriggerActivation*	Specifies the activation mode of the sequencer trigger. Supported values for UserOutput0 are: • RisingEdge: Specifies that the trigger is considered valid on the rising edge of the source signal. • FallingEdge: Specifies that the trigger is considered valid on the falling edge of the source signal. • AnyEdge: Specifies that the trigger is considered valid on the falling or rising edge of the source signal. • LevelHigh: Specifies that the trigger is considered valid as long as the level of the source signal is high. • LevelLow: Specifies that the trigger is considered valid as long as the level of the source signal is low.

The sequencer mode can be used to set a series of feature sets for image acquisition. The sets can consecutively be activated during the acquisition by the camera. The sequence is configured by a list of parameters sets.

Note

At the moment, the Sequencer Mode is only available for MATRIX VISION cameras with CCD sensors and Sony's CMOS sensors. Please consult the **"Device Feature and Property List"s** to get a summary of the actually supported features of each sensor.

The following features are currently available for using them inside the sequencer control:

Feature	Note	Changeable during runtime
BinningHorizontal		-
BinningVertical		-
CounterDuration	Can be used to configure a certain set of sequencer parameters to be applied for the next `CounterDuration` frames.	since Firmware version 2.15
DecimationHorizontal		-
DecimationVertical		-
ExposureTime		since Firmware version 2.15
Gain		since Firmware version 2.15

Height		since Firmware version 2.36
OffsetX		since Firmware version 2.35
OffsetY		since Firmware version 2.35
Width		since Firmware version 2.36
mvUserOutput		-
UserOutputValueAll		-
UserOutputValueAllMask		-
Multiple conditional sequencer paths		-

Note

Configured sequencer programs are stored as part of the **User Sets** (p. 396) like any other feature.

Actual settings of the camera are overwritten when a sequencer set is loaded.

See also

- **Define multiple exposure times using the Sequencer Control** (p. 226)
- There are 3 C++ examples called **GenICamSequencerUsage**, **GenICamSequencerUsageWithPaths** and **GenICamSequencerParameterChangeAtRuntime** that show how to control the sequencer from an application. They can be found in the `Examples` section of the `mvIMPACT Acquire C++ API`

1.14.14 Transport Layer Control

The **"Transport Layer Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
PayloadSize	*payloadSize*	Provides the number of bytes transferred for each image or chunk on the stream channel.
GevInterfaceSelector	*gevInterfaceSelector*	Selects which physical network interface to control.
GevMACAddress[GevInterface↩ Selector]	*gevMACAddress*	MAC address of the network interface.
GevStreamChannelSelector	*gevStreamChannelSelector*	Selects the stream channel to control.
PtpControl	*ptpEnable*	Enables the Precision Time Protocol (PTP).
etc.		

related to the Transport Layer Control.

Use cases related to the **"Transport Layer Control"** are:

- **Using the primary application switchover functionality** (p. 430)
- **Synchronizing camera timestamps** (p. 408)

1.14.15 User Set Control

The **"User Set Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
UserSetSelector	*userSetSelector*	Selects the feature user set to load, save or configure.
UserSetLoad[UserSetSelector]	*userSetLoad*	Loads the user set specified by *UserSetSelector* to the device and makes it active.
UserSetSave[UserSetSelector]	*userSetSave*	Endianess of the device registers.
UserSetDefault	*userSetDefault*	Selects the feature user set to load and make active when the device is reset.

related to the User Set Control to save and load the user device settings.

The camera allows the storage of up to four configuration sets in the camera. This feature is similar to storing settings in the registry but this way in the camera. It is possible to store

- exposure,
- gain,
- AOI,
- frame rate,
- **LUT** (p. 170),
- one **Flat-Field Correction** (p. 289)
- etc.

permanently. You can select, which user set comes up after hard reset.

Use case related to the **"User Set Control"** is:

- **Creating user set entries** (p. 396)

Another way to create user data is described here: **Creating user data entries** (p. 394)

1.14.16 Action Control

The **"Action Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
ActionDeviceKey	*actionDeviceKey*	Provides the device key that allows the device to check the validity of action commands. The device internal assertion of an action signal is only authorized if the ActionDeviceKey and the action device key value in the protocol message are equal.
ActionSelector	*actionSelector*	Selects to which action signal further action settings apply.
ActionGroupKey	*actionGroupKey*	Provides the key that the device will use to validate the action on reception of the action protocol message.
ActionGroupMask	*actionGroupMask*	Provides the mask that the device will use to validate the action on reception of the action protocol message.

related to the Action control features.

The use case **Using Action Commands** (p. 418) shows in detail how this feature works.

1.14.17 mv Logic Gate Control

The **"mv Logic Gate Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
	mvLogicGateANDSelector	Selects the AND gate to configure.
	mvLogicGateANDSource1	Selects the first input signal of the AND gate selected by *mvLogicGateANDSelector*.
	mvLogicGateANDSource2	Selects the second input signal of the AND gate selected by *mvLogicGateANDSelector*.
	mvLogicGateORSelector	Selects the OR gate to configure.
	mvLogicGateORSource1	Selects the first input signal of the OR gate selected by *mvLogicGateORSelector*.
	mvLogicGateORSource2	Selects the second input signal of the OR gate selected by *mvLogicGateORSelector*.
	mvLogicGateORSource3	Selects the third input signal of the OR gate selected by *mvLogicGateORSelector*.
	mvLogicGateORSource4	Selects the fourth input signal of the OR gate selected by *mvLogicGateORSelector*.

related to control the devices Logic Gate Control parameters. It performs a logical operation on one or more logic inputs and produces a single logic output.

The use case **Creating different exposure times for consecutive images** (p. 355) shows how you can create different exposure times with timers, counters and the logic gate functionality.

See also

Triggering of an indefinite sequence with precise starting time (p. 362)

1.14.18 mv Flat Field Correction Control

The **"mv Flat Field Correction Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
	mvFFCEnable	Enables the flat field correction.
	mvFFCCalibrationImageCount	The number of images to use for the calculation of the correction image.
	mvFFCCalibrate	Starts the calibration of the flat field correction.

related to control the devices Flat Field Correction parameters.

The use case **Flat-Field Correction** (p. 289) shows how this control can be used.

1.14.19 mv Serial Interface Control

The **"mv Serial Interface Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
	mvSerialInterfaceMode	States the interface mode of the serial interface.
	mvSerialInterfaceEnable	Controls whether the serial interface is enabled or not.
	mvSerialInterfaceBaudRate	Serial interface clock frequency.
	mvSerialInterfaceASCIIBuffer	Buffer for exchanging ASCII data over serial interface.
	mvSerialInterfaceWrite	Command to write data from the serial interface.
	mvSerialInterfaceRead	Command to read data from the serial interface.
etc.		

related to control the devices Serial Interface Control parameters. It enables the camera to be controlled via serial interface.

The use case **Working With The Serial Interface (mv Serial Interface Control)** (p. 414) shows how you can work with the serial interface control.

See also

Working With The Serial Interface (mv Serial Interface Control) (p. 414)

1.14.20 mv Defective Pixel Correction Control

The **"mv Defective Pixel Correction Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
	mvDefectivePixelCount	Contains the number of valid defective pixels.
	mvDefectivePixelSelector	Controls the index of the defective pixel to access.
	mvDefectivePixelDataLoad	Loads the defective pixels from the device non volatile memory.
	mvDefectivePixelDataSave	Saves the defective pixels to the device non volatile memory.

related to control the devices Defective Pixel data.

See also

Correcting image errors of a sensor (p. 284)

1.14.21 mv Frame Average Control (only with specific models)

The **"mv Frame Average Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
	mvFrameAverageEnable	Enables the frame averaging engine.
	mvFrameAverageSlope	The slope in full range of register.

related to the frame averaging engine.

The use case **Reducing noise by frame averaging** (p. 312) shows in detail how this feature works.

1.14.22 mv Auto Feature Control

The **"mv Auto Feature Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
	mvAutoFeatureAOIMode	Common AutoControl AOI used for Auto Gain Control(AGC), Auto Exposure Control(AEC) and Auto White Balancing.
	mvAutoFeatureSensitivity	The controllers sensitivity of brightness deviations. This parameter influences the gain as well as the exposure controller.
	mvAutoFeatureCharacteristic	Selects the prioritization between Auto Exposure Control(AEC) and Auto Gain Control(AGC) controller.
	mvAutoFeatureBrightness↩ Tolerance	The error input hysteresis width of the controller. If the brightness error exceeds the half of the value in positive or negative direction, the controller restarts to control the brightness.

related to the automatic control of exposure, gain, and white balance.

With this control you can influence the characteristic of the controller depending on certain light situations. AEC/AGC can be controlled with the new additional properties **mvAutoFeatureSensitivity**, **mvAutoFeatureCharacteristic** and **mvAutoFeatureBrightnessTolerance**.

1.14.23 mv High Dynamic Range Control (only with specific sensor models)

The **"mv High Dynamic Range Control"** contains features like

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
	mvHDREnable	Enables the high dynamic range feature.
	mvHDRPreset	Selects the HDR parameter set.

	mvHDRSelector	Selects the HDR parameter set to configure.
	mvHDRVoltage1	First HDR voltage in mV.
	mvHDRVoltage2	Second HDR voltage in mV.
	mvHDRExposure1	First HDR exposure in ppm.
	mvHDRExposure2	Second HDR exposure in ppm.

related to the control of the device High Dynamic Range parameters.

The use cases **Adjusting sensor of camera models -x00w** (p. 378) and **Adjusting sensor of camera models -x02d (-1012d)** (p. 381) show the principle of the HDR.

1.14.24 LUT Control

The **"LUT Control"** contains features like

Since

mvIMPACT Acquire 2.35.2054.0

A 12 to 9 RawLUT for color cameras was added. The RawLUT works identically for all colors but with a higher resolution. This is useful if higher dynamic is needed.

Feature name (acc. to **SFNC** (p. 221))	Property name (acc. to **mvIMPACT Acquire** (p. 219))	Description
LUTSelector	*LUTSelector*	Selects which LUT to control.
LUTEnable[LUTSelector]	*LUTEnable*	Activates the selected LUT.
LUTIndex[LUTSelector]	*LUTIndex*	Controls the index (offset) of the coefficient to access in the selected LUT.
LUTValue[LUTSelector][LUTIndex]	*LUTValue*	Returns the value at entry *LUTIndex* of the LUT selected by *LUTSelector*.
LUTValueAll[LUTSelector]	*LUTValueAll*	Allows access to all the LUT coefficients with a single read/write operation.
	mvLUTType	Describes which type of LUT is used for the current *LUTSelector*
	mvLUTInputData	Describes the data the LUT is applied to (e.g bayer, RGB, or gray data)
	mvLUTMapping	Describes the LUT mapping (e.g. 10 bit -> 12 bit)

related to the look-up table (LUT) control.

The look-up table (LUT) is a part of the signal path in the camera and maps data of the ADC into signal values. The LUT can be used e.g. for:

- High precision gamma

- Non linear enhancement (e.g. S-Shaped)
- Inversion (default)
- Negative offset
- Threshold
- Level windows
- Binarization

This saves (approx. 5%) CPU load, works on the fly in the FPGA of camera, is less noisy and there are no missing codes after Gamma stretching.

See also

Working with LUTValueAll (p. 390)
Implementing a hardware-based binarization (p. 392)
Optimizing the color/luminance fidelity of the camera (p. 297)

Three read-only registers describe the LUT that is selected using the LUTSelector register:

1.14.24.1 mvLUTType

There are two different types of LUTs available in MATRIX VISION cameras:

- Direct LUTs define a mapping for each possible input value, for example a 12 -> 10 bit direct LUT has 2^12 entries and each entry has 10 bit.
- Interpolated LUTs do not define a mapping for every possible input value, instead the user defines an output value for equidistant nodes. In between the nodes linear interpolation is used to calculate the correct output value.
  Considering a 10 -> 10 bit interpolated LUT with 256 nodes (as usually used in MATRIX VISION cameras), the user defines a 10 bit output value for 256 equidistant nodes beginning at input value 0, 4, 8, 12, 16 and so on. For input values in between the nodes linear interpolation is used.

1.14.24.2 mvLUTInputData

This register describes on which data the LUT is applied to:

- Bayer means that the LUT is applied to raw Bayer data, thus (depending on the de-Bayer algorithm) a manipulation of one pixel may also affect other pixels in its neighborhood.
- Gray means that the LUT is applied to gray data.
- RGB means that the LUT is applied to RGB data (i.e. after de-Bayering). Normally this is used to change the luminance on an RGB image and the LUT is applied to all three channels.

1.14.24.3 mvLUTMapping

This register describes the mapping of the currently selected LUT, e.g "map_10To10" means that a 10 bit input value is mapped to a 10 bit output values whereas "map_12To10" means that a 12 bit input value is mapped to a 10 bit output value.

1.14.24.4 LUT support in MATRIX VISION cameras

mvBlueCOUGAR	LUTSelector	LUT type	LUT mapping	LUT input data
-X100wG -X100wG-POE -X102bG -X102bG-POE -X102bG-POEI -X102dG -X102dG-POE -X102dG-POEI -X102eG -X102eGE -X104eG -X104eG-POE -X104eG-PLC -X102eG-POE -X102eGE-POE -X102eG-POEI -X102eGE-POEI -X105G -X120aG -X120aG-POE -X120bG -X120bG-POE -X120bG-PLC -X120dG -X120dG-POE -X120dG-POEI -X122G -X122G-POE -X122G-PLC -X122G-POEI -X123G -X123G-PLC -X123G-POE -X123G-POEI -X124G -X124G-POE -X1010G -X1010G-POE -X225G-POEI -X225G -X225G-PLC -X225G-1211-ET	Luminance	Direct	map_10To10	Gray

-X100wC -X100wC-POE -X102bC -X102bC-POE -X102bC-POEI -X102dC -X102dC-POE -X102dC-POEI -X102eC -X104eC -X104eC-POE -X104eC-PLC -X102eC-POE -X102eC-POEI -X105C -X120aC -X120aC-POE -X120bC -X120bC-POE -X120bC-PLC -X120dC -X120dC-POE -X120dC-POEI -X122C -X122C-POE -X122C-PLC -X123C -X123C-PLC -X123C-POE -X123C-POEI -X124C -X124C-POE -X1010C -X1010C-POE -X225C-POEI -X225C	Luminance Red Green Blue	Interpolated Direct Direct Direct	map_10To10 map_8To10 map_8To10 map_8To10	RGB Bayer Bayer Bayer
-X104G -X104G-POEI -X104aG -X104a12G -X104bG -X104bG-POE -X125aG -X125aG-POE -X125aG-POEI	Luminance	Direct	map_12To10	Gray
-X104C -X104C-POE -X104aC -X104a12C -X104bC -X104bC-POE -X125aC -X125aC-POE -X125aC-POEI	Red Green Blue	Direct Direct Direct	map_10To10 map_10To10 map_10To10	Bayer Bayer Bayer

-X100fG -X100fG-POE -X100fG-POEI -X102fG -X102fG-POE -X102fG-POEI -X102kG -X102kG-POE -X102kG-POEI -X102mG -X102mG-POE -X102mG-POEI -X102nG -X102nG-POE -X102nG-POEI -X104fG -X104fG-POE -X104fG-POEI -X104iG -X104iG-POE -X104iG-POEI -X105bG -X105bG-POE -X105bG-POEI -X105dG -X105dG-POE -X105dG-POEI -X105pG -X105pG-POE -X105pG-POEI -X107bG -X107bG-POE -X107bG-POEI -X108aG -X108aG-POE -X108aG-POEI -X109bG -X109bG-POE -X109bG-POEI -X1012bG -X1012bG-POE -X1012bG-POEI -X1012dG -X1012dG-POE -X1012dG-POEI -X1020G -X1020G-POE -X1020G-POEI	Luminance	Direct	map_12To9	Gray

-X100fC -X100fC-POE -X100fC-POEI -X102fC -X102fC-POE -X102fC-POEI -X102kC -X102kC-POE -X102kC-POEI -X102mC -X102mC-POE -X102mC-POEI -X102nC -X102nC-POE -X102nC-POEI -X104fC -X104fC-POE -X104fC-POEI -X104iC -X104iC-POE -X104iC-POEI -X105bC -X105bC-POE -X105bC-POEI -X105dC -X105dC-POE -X105dC-POEI -X105pC -X105pC-POE -X105pC-POEI -X107bC -X107bC-POE -X107bC-POEI -X108aC -X108aC-POE -X108aC-POEI -X109bC -X109bC-POE -X109bC-POEI -X1012bC -X1012bC-POE -X1012bC-POEI -X1012dC -X1012dC-POE -X1012dC-POEI -X1020C -X1020C-POE -X1020C-POEI	Red Green Blue mvRaw	Direct Direct Direct Direct	map_10To9 map_10To9 map_10To9 map_12To9	Bayer Bayer Bayer Bayer
-X2012bG -X2012bG-POE -X2012bG-POEI	Luminance	Direct	map_12To9	Gray
-X2012bC -X2012bC-POE -X2012bC-POEI	Red Green Blue mvRaw	Direct Direct Direct Direct	map_10To9 map_10To9 map_10To9 map_12To9	Bayer Bayer Bayer Bayer
-XD124aG -XD126G -XD129G -XD124bG -XD126aG -XD129aG -XD204G -XD204aG -XD204bG -XD204baG -XD104G -XD104aG -XD104a12G -XD104bG -XD104baG -XD1212aG	Luminance	Direct	map_10To12	Gray

-XD124aC -XD126C -XD129C -XD124bC -XD126aC -XD129aC -XD204C -XD204aC -XD204bC -XD204baC -XD104C -XD104aC -XD104bC -XD104baC -XD1212aC	Luminance Red Green Blue	Interpolated Direct Direct Direct	map_10To10 map_8To12 map_8To12 map_8To12	RGB Bayer Bayer Bayer
-XD102mG -XD104dG -XD105aG -XD105dG -XD104hG -XD107G -XD107bG -XD108aG -XD109bG -XD1012bG -XD1012dG -XD1016G -XD1020aG -XD1025G -XD1031G	Luminance	Direct	map_12To9	Gray
-XD104dC -XD105aC -XD104hC -XD105dC -XD107C -XD107bC -XD108aC -XD109bC -XD1012bC -XD1012dC -XD1016C -XD1020aC -XD1025C -XD1031C	Luminance Red Green Blue mvRaw	Interpolated Direct Direct Direct Direct	map_10To10 map_10To9 map_10To9 map_10To9 map_12To9	RGB Bayer Bayer Bayer Bayer
mvBlueCOUGAR-3X120aG mvBlueCOUGAR-3X120aG-POEI	-	-	-	-
mvBlueCOUGAR-3X123G	Red(1st head) Green(2nd head) Blue(3rd head)	Interpolated Interpolated Interpolated	map_12To12 map_12To12 map_12To12	Gray Gray Gray
mvBlueCOUGAR-3X123C	Red(1st head) Green(2nd head) Blue(3rd head)	Interpolated Interpolated Interpolated	map_12To12 map_12To12 map_12To12	Bayer Bayer Bayer

1.15 Developing applications using the mvIMPACT Acquire SDK

The `mvIMPACT Acquire SDK` is a comprehensive software library that can be used to develop applications using the devices described in this manual. A wide variety of programming languages is supported.

For C, C++, .NET, Python or Java developers separate API descriptions can be found on the MATRIX VISION website:

- `mvIMPACT Acquire C API`
- `mvIMPACT Acquire C++ API`
- `mvIMPACT Acquire Java API`
- `mvIMPACT Acquire .NET API`
- `mvIMPACT Acquire Python API`

Compiled versions (CHM format) might already be installed on your system. These manuals contain chapters on

- how to link and build applications using mvIMPACT Acquire
- how the log output for **"mvIMPACT Acquire"** devices is configured and how it works in general
- how to create your own installation packages for Windows and Linux
- a detailed API documentation
- etc.

1.16 DirectShow interface

Note

DirectShow can only be used in combination with the Microsoft Windows operating system.

Since Windows Vista, **Movie Maker** does not support capturing from a device registered for DirectShow anymore.

This is the documentation of the MATRIX VISION DirectShow_acquire interface. A MATRIX VISION specific property interface based on the IKsPropertySet has been added. All other features are related to standard DirectShow programming.

- **Supported interfaces** (p. 178)
- **Logging** (p. 178)
- **Setting up devices for DirectShow usage** (p. 178)

1.16.1 Supported interfaces

1.16.1.1 IAMCameraControl

1.16.1.2 IAMDroppedFrames

1.16.1.3 IAMStreamConfig

1.16.1.4 IAMVideoProcAmp

1.16.1.5 IKsPropertySet

The DirectShow_acquire supports the IKsPropertySet Interface. For further information please refer to the Microsoft DirectX 9.0 Programmer's Reference.

Supported property set GUID's:

- AMPROPERTY_PIN_CATEGORY
- DIRECT_SHOW_ACQUIRE_PROPERTYSET

1.16.1.6 ISpecifyPropertyPages

1.16.2 Logging

The DirectShow_acquire logging procedure is equal to the logging of the MATRIX VISION products which uses mvIMPACT Acquire. The log output itself is based on **XML**.

If you want more information about the logging please have a look at the **Logging** chapter of the respective **"mvIMPACT Acquire API"** manual or read on how to configure the log-output using **mvDeviceConfigure** in the **"mvIMPACT Acquire GUI Applications"** manual.

1.16.3 Setting up devices for DirectShow usage

In order to be able to access a device through the mvIMPACT Acquire driver stack from an application through DirectShow a registration procedure is needed. This can either be done using mvDeviceConfigure or by a command line tool that is part of the Windows operating system.

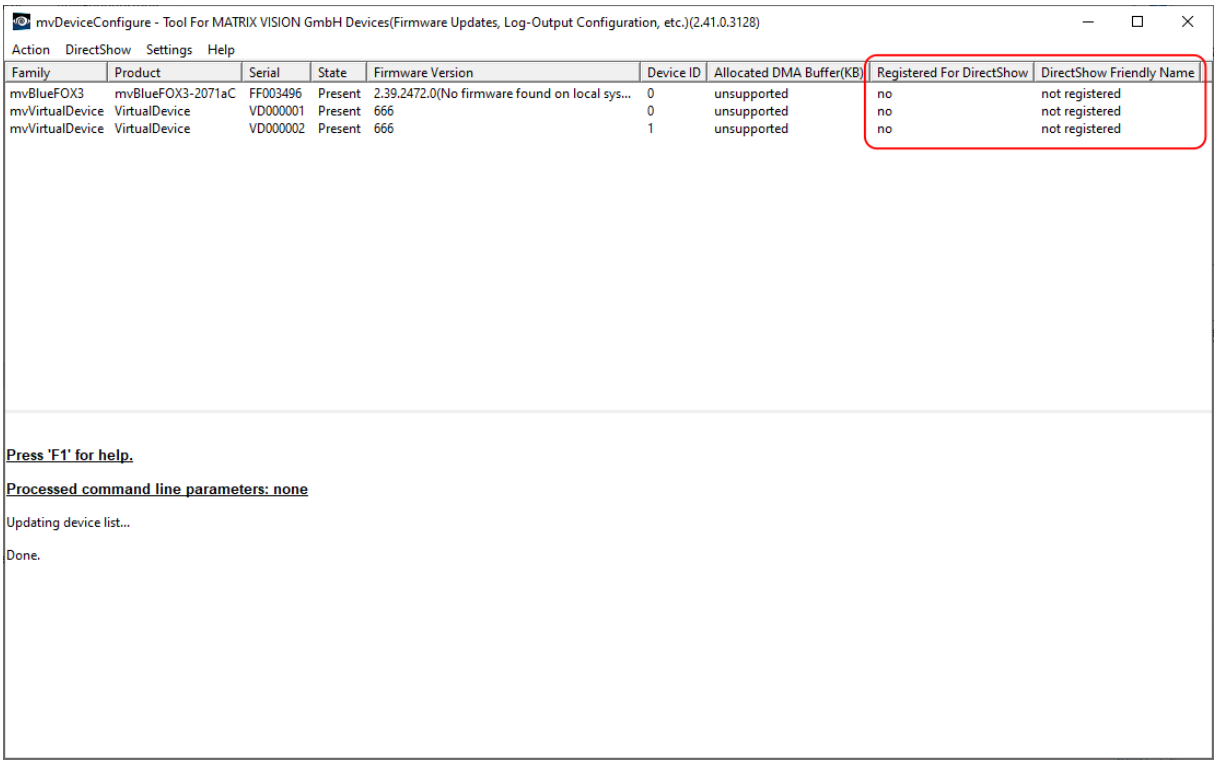
Note

Please be sure to register the MV device for DirectShow with a matching version of **mvDeviceConfigure**. I.e. if you have installed the 32-bit version of the VLC Media Player, Virtual Dub, etc., you have to register devices with the 32-bit version of mvDeviceConfigure (*"C:\Program Files\MATRIX VISION\mvIMPACT Acquire\bin"*, the 64-bit version resides in *"C:\Program Files\MATRIX VISION\mvIMPACT Acquire\bin\x64"*)!

1.16.3.1 Registering devices

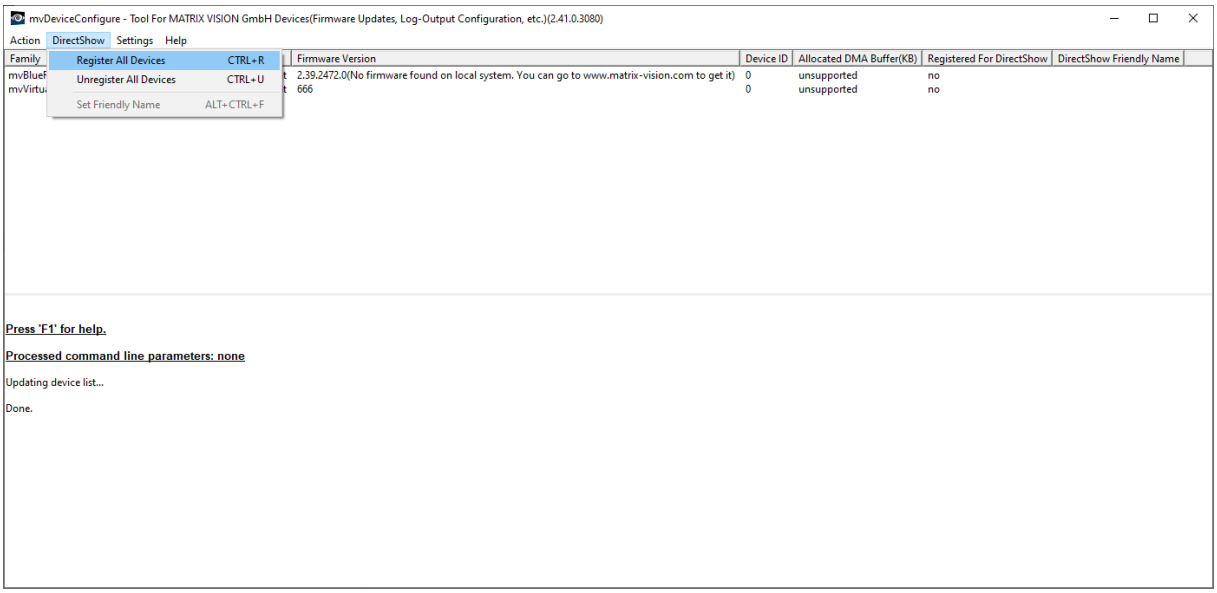
To register all devices currently recognized by the mvIMPACT Acquire driver stack for access with DirectShow the following registration procedure is needed:

1. >mvDeviceConfigure needs to be started(with elevated rights).
   If no device has been registered the application will more or less (depending on the installed devices) look like this.



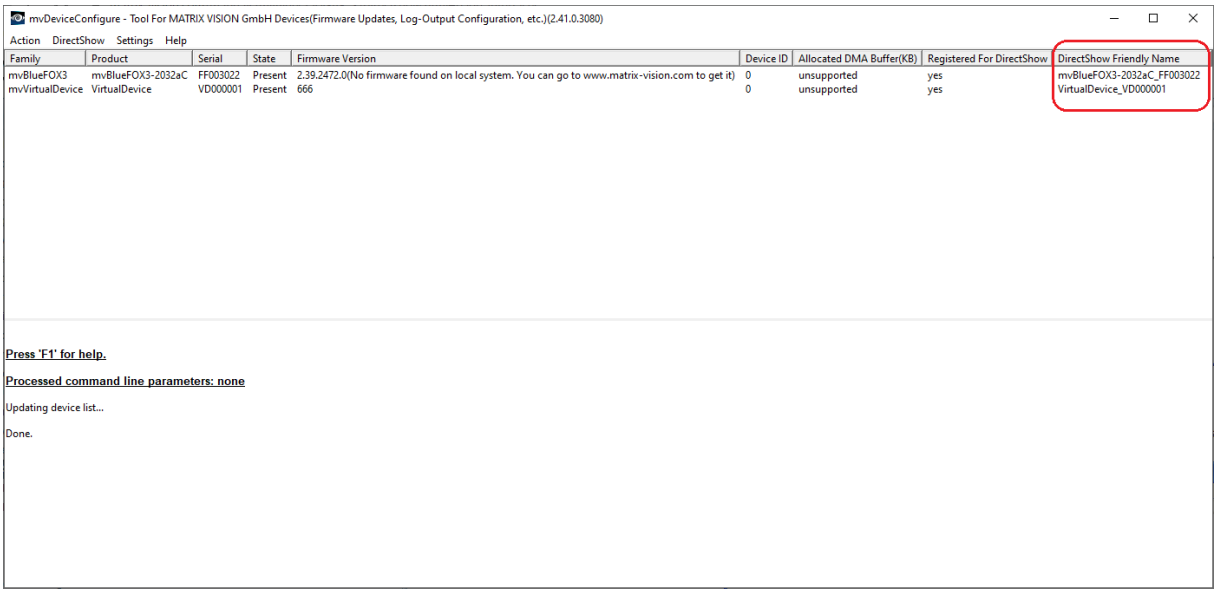
mvDeviceConfigure - After Start

2. To register every installed device for DirectShow access click on the menu item **"DirectShow"** → **"Register All Devices"**.



mvDeviceConfigure - Register All Devices

3. After a successful registration the column "Registered For DirectShow" will display **"yes"** for every device and the devices will be registered with a default DirectShow friendly name which is displayed in the **"DirectShow Friendly Name"** column.

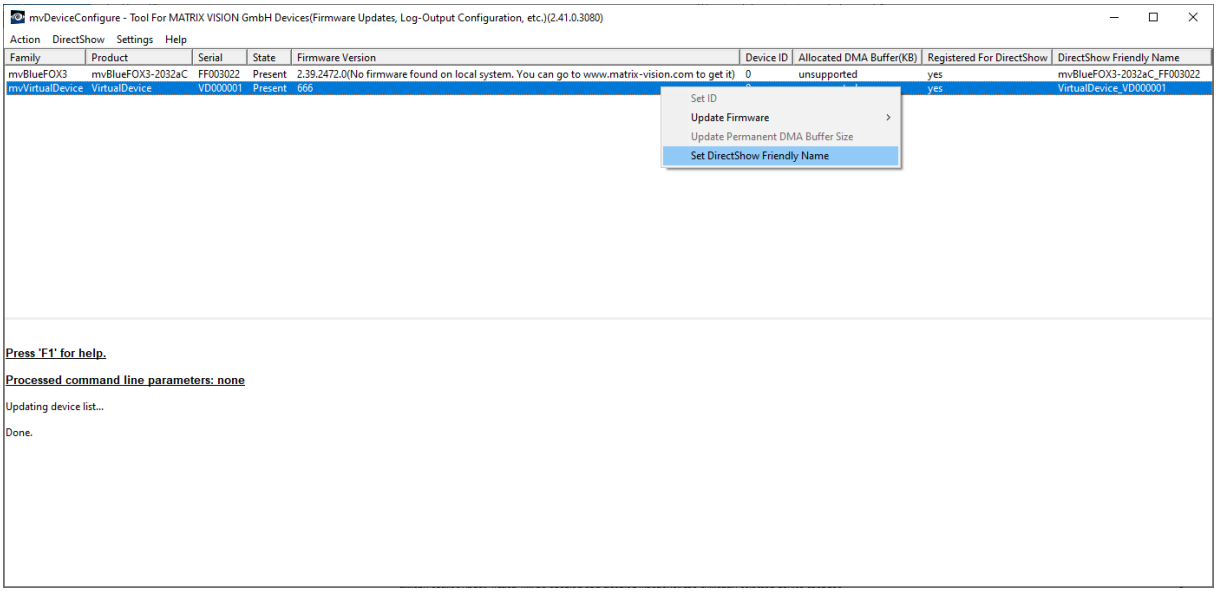


mvDeviceConfigure - All Devices Registered For DirectShow Access

1.16.3.2 Renaming devices

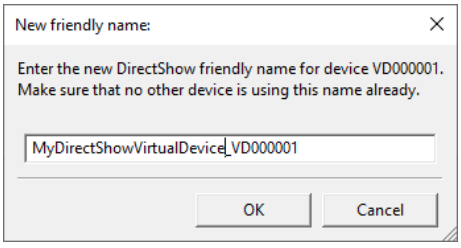
To modify the DirectShow friendly name of a device:

1. mvDeviceConfigure needs to be started(with elevated rights).
2. right-click on the device to rename and select **"Set DirectShow Friendly Name"**:



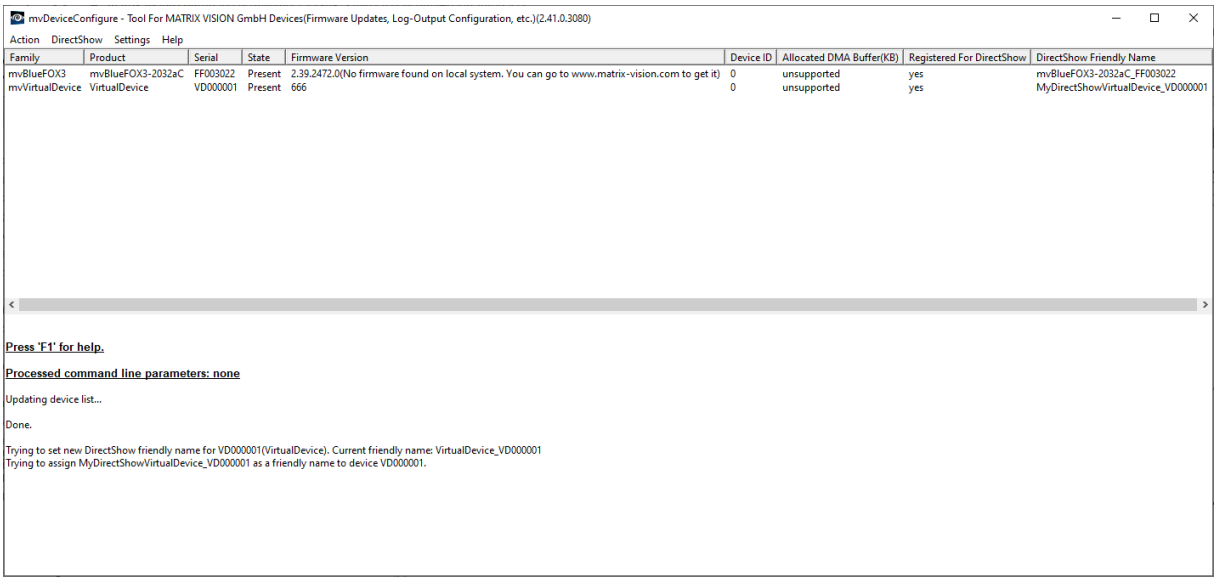
mvDeviceConfigure - Set DirectShow Friendly Name

3. Then, a dialog will appear. Please enter the new name and confirm it with **"OK"**.



mvDeviceConfigure - Dialog For New Name

4. Afterwards the column "DirectShow friendly name" will display the newly assigned friendly name.



mvDeviceConfigure - Renamed Device

Note

Please do not select the same friendly name for two different devices. In theory this is possible, however the mvDeviceConfigure GUI will not allow this to avoid confusion.

1.16.3.3 Using regsvr32

To register all devices currently recognized by the mvIMPACT Acquire driver stack with auto-assigned names, the Windows tool "`regsvr32`" can be used from an elevated command shell.

The following command line options are available and can be passed during the silent registration:

EXAMPLES:

Register ALL devices that are recognized by mvIMPACT Acquire (this will only register devices which have drivers installed) without any user interaction:

```
regsvr32 <path>\DirectShow_acquire.ax /s
```

Unregister ALL devices that have been registered before without any user interaction:

```
regsvr32 <path>\DirectShow_acquire.ax /u /s
```

1.17 Troubleshooting

- **Error code list** (p. 183)
- **Accessing log files** (p. 196)
- **General Issues** (p. 198)
- **Windows** (p. 210)
- **Linux** (p. 211)

1.17.1 Error code list

Numerical Value	String Representation	Brief Description	Detailed Description
-2000	PROPHANDLING_NOT_↩ A_LIST	This component is not a list.	A list operation for this component has been called but this component does not reference a list. [-2000]
-2001	PROPHANDLING_NOT_↩ A_PROPERTY	This component is not a property.	A property operation for this component has been called but this component does not reference a property. [-2001]
-2002	PROPHANDLING_NOT_↩ A_METHOD	This component is not a method.	A method operation for this component has been called but this component does not reference a method. [-2002]
-2003	PROPHANDLING_NO_↩ READ_RIGHTS	The caller has no read rights for this component.	It has been tried to read data from this component, but the caller has no read rights for this component. [-2003]
-2004	PROPHANDLING_NO_↩ WRITE_RIGHTS	The caller has no write rights for this component.	It has been tried to modify data of this component, but the caller has no write rights for this component. [-2004]

-2005	PROPHANDLING_NO_↩ MODIFY_SIZE_RIGHTS	The caller can't modify the size of this component.	It has been tried to modify the size of this list or the number of values stored by a property, but the caller doesn't have the required right to do this. This error will also be reported if the user tried to increase the number of values handled by a property above the maximum number of values it can handle. Therefore before resizing a property check if the new size might exceeds this maximum value by calling the appropriate function. [-2005]
-2006	PROPHANDLING↩ _INCOMPATIBLE_↩ COMPONENTS	The two involved components are not compatible.	An operation requiring two compatible components has been called with two components, which are not compatible. [-2006]
-2008	PROPHANDLING↩ _UNSUPPORTED_↩ PARAMETER	One or more of the specified parameters are not supported by the function.	This error might also be generated if a certain feature is not available on the current platform. [-2008]
-2009	PROPHANDLING_SIZE_↩ MISMATCH	Different sized value buffers have been passed.	While trying to read value pairs the caller passed two different sized value buffers to a function while one is too small to hold all the information. [-2009]
-2010	PROPHANDLING_↩ IMPLEMENTATION_↩ MISSING	A feature that is not implemented so far has been requested.	The caller requested a feature, that hasn't been implemented so far. This error code is only provided for compatibility and will be set in very rare cases only. [-2010]
-2011	PROPHANDLING↩ _ACCESSTOKEN_↩ CREATION_FAILED	An access token object couldn't be created.	This can either happen, because the caller has not the rights required to create an access token or because the system runs very low on memory. DeprecatedThis error code currently is not used anywhere within this framework. It might be removed in a future version. [-2011]

-2012	PROPHANDLING_↩ INVALID_PROP_VALUE	It has been tried to assign an invalid value to a property.	This can either happen if the value lies above or below the min. or max. value for a property or when it has been tried to write a value to a property, which is not in the properties translation dictionary (if it defines one). To find out, which values are allowed for the property in question the user should Check if the property defines a translation dictionary. Check the allowed values within a translation dictionary if one is defined. Check the min and max value for properties, that define limits. [-2012]
-2013	PROPHANDLING_PROP↩ _TRANSLATION_TABLE↩ _CORRUPTED	The properties translation table has been corrupted.	The properties translation table has been corrupted for an unknown reason and can't be used anymore. [-2013]
-2014	PROPHANDLING_PROP↩ _VAL_ID_OUT_OF_↩ BOUNDS	Invalid value index.	The caller tried to read a value from an invalid index from a property. Most properties store one value only, thus the only valid positive value index will be 0 (some negative index values are reserved for special values like e.g. the min/max value of a property). However some properties might store more than one value, thus the max. allowed index might be higher. The highest index allowed will always be the value count of a property minus one for properties with the cfFixedSize flag set. Other properties will automatically adjust the size once the user writes to an index out of bounds. [-2014]
-2015	PROPHANDLING_PROP↩ _TRANSLATION_TABLE↩ _NOT_DEFINED	This property doesn't define a translation table.	The caller tried to modify a translation table, that hasn't been defined for this property. [-2015]

-2016	PROPHANDLING_↩ INVALID_PROP_VALUE_↩ TYPE	An invalid value has been passed to the property.	Although properties are quite tolerant regarding the allowed assignment for them some value types can't be used to write all properties. As an example assigning a float value to an integer property would result in this error. Another reason for this error might be when a user tried to access e.g. a float property with functions meant to be used for int properties. [-2016]
-2017	PROPHANDLING_PROP↩ _VAL_TOO_LARGE	A too large value has been passed.	One or more of the values the caller tried to write to the property are larger than the max. allowed value for this property. [-2017]
-2018	PROPHANDLING_PROP↩ _VAL_TOO_SMALL	A too small value has been passed.	One or more of the values the caller tried to write to the property are smaller than the min. allowed value for this property. [-2018]
-2019	PROPHANDLING_↩ COMPONENT_NOT_↩ FOUND	The specified component could not be found.	[-2019]
-2020	PROPHANDLING_LIST_↩ ID_INVALID	An invalid list has been referenced.	[-2020]
-2021	PROPHANDLING_↩ COMPONENT_ID_INVALID	An invalid component within a list has been referenced.	[-2021]
-2022	PROPHANDLING_LIST_↩ ENTRY_OCCUPIED	The specified list index is occupied.	During the creation of a new component the caller tried the insert the newly created component into a list at a position already used to store another component. [-2022]
-2023	PROPHANDLING_↩ COMPONENT_HAS_↩ OWNER_ALREADY	The specified component already has an owner.	The caller tried to assign an owner to a component that already has an owner. An owner once defined can't be modified anymore. [-2023]
-2024	PROPHANDLING↩ _COMPONENT_↩ ALREADY_REGISTERED	It has been tried to register the same component at twice in the same list.	[-2024]

-2025	PROPHANDLING_LIST_↩ CANT_ACCESS_DATA	The desired data can't be accessed or found.	During loading or saving data this error can occur e.g. if it has been tried to import a setting from a location where the desired setting couldn't be found. Another reason for this error might be that the current user is not allowed to perform a certain operation on the desired data (e.g. a user tries to delete a setting that is stored with global scope but does not have elevated access rights). [-2025]
-2026	PROPHANDLING_↩ METHOD_PTR_INVALID	The function pointer of the referenced method object is invalid.	[-2026]
-2027	PROPHANDLING_↩ METHOD_INVALID_↩ PARAM_LIST	A method object has an invalid parameter list.	[-2027]
-2028	PROPHANDLING_SWIG↩ _ERROR	This indicates an internal error occurred within the SWIG generated wrapper code, when working under Python.	[-2028]
-2029	PROPHANDLING↩ _INVALID_INPUT_↩ PARAMETER	A invalid input parameter has been passed to a function of this module.	In most cases this might be a unassigned pointer, where a valid pointer to a user defined storage location was expected. [-2029]
-2030	PROPHANDLING_↩ COMPONENT_NO_↩ CALLBACK_REGISTERED	The user tried to modify a registered callback, but no callback has been registered for this component.	[-2030]
-2031	PROPHANDLING_INPUT↩ _BUFFER_TOO_SMALL	The user tried to read data into a user supplied storage location, but the buffer was too small to accommodate the result.	[-2031]
-2032	PROPHANDLING_↩ WRONG_PARAM_COUNT	The number of parameters is incorrect.	This error might occur if the user called a function with a variable number of input or output parameters and the number of parameters passed to the function does not match the number of required parameters. [-2032]
-2033	PROPHANDLING↩ _UNSUPPORTED_↩ OPERATION	The user tried to execute an operation, which is not supported by the component he is referring to.	[-2033]

-2034	PROPHANDLING_CANT↩ _SERIALIZE_DATA	The user tried to save(serialize) a property list without having the right to do this.	[-2034]
-2035	PROPHANDLING_↩ INVALID_FILE_CONTENT	The user tried to use a file to update or create a component list, that does not contain valid data for this operation.	This e.g. might happen, if the file does not contain valid XML data or XML data that is not well formed. [-2035]
-2036	PROPHANDLING_CANT↩ _ALLOCATE_LIST	This error will occur when the modules internal representation of the tree structure does not allow the allocation of a new list.	In this case either new list can't be allocated. The only way to solve this problem is to delete another list. [-2036]
-2037	PROPHANDLING_↩ CANT_REGISTER_↩ COMPONENT	The referenced list has no space left to register this component at the desired position.	There might however be an empty space within the list where this element could be registered, but no more components can be registered at the end of this list. [-2037]
-2038	PROPHANDLING_PROP↩ _VALIDATION_FAILED	The user tried to assign a value to a property, that is invalid.	This will result in a detailed error message in the logfile. This error might arise e.g. when a string property doesn't allow the string to contain numbers. In this case trying to set the properties value to 'blabla7bla' would cause this error. [-2038]
-2099	PROPHANDLING_LAST↩ _VALID_ERROR_CODE	Defines the last valid error code value for the property module.	[-2099]
-2100	DMR_DEV_NOT_FOUND	The specified device can't be found.	This error occurs either if an invalid device ID has been passed to the device manager or if the caller tried to close a device which currently isn't initialized. [-2100]
-2101	DMR_INIT_FAILED	The device manager couldn't be initialized.	This is an internal error. [-2101]
-2102	DMR_DRV_ALREADY_IN↩ _USE	The device is already in use.	This error e.g. will occur if this or another process has initialized this device already and an application tries to open the device once more or if a certain resource is available only once but shall be used twice. [-2102]

-2103	DMR_DEV_CANNOT_↩ OPEN	The specified device couldn't be initialized.	[-2103]
-2104	DMR_NOT_INITIALIZED	The device manager or another module hasn't been initialized properly.	This error occurs if the user tries e.g. to close the device manager without having initialized it before or if a library used internally or a module or device associated with that library has not been initialized properly or if [-2104]
-2105	DMR_DRV_CANNOT_↩ OPEN	A device could not be initialized.	In this case the log-file will contain detailed information about the source of the problem. [-2105]
-2106	DMR_DEV_REQUEST_↩ QUEUE_EMPTY	The devices request queue is empty.	This error e.g. occurs if the user waits for an image request to become available at a result queue without having send an image request to the device before. It might also arise when trying to trigger an image with a software trigger mechanism before the acquisition engine has been completely started. In this case a small delay and then again calling the software trigger function will succeed. [-2106]
-2107	DMR_DEV_REQUEST_↩ CREATION_FAILED	A request object couldn't be created.	The creation of a request object failed. This might e.g. happen, if the system runs extremely low on memory. [-2107]
-2108	DMR_INVALID_↩ PARAMETER	An invalid parameter has been passed to a function.	This might e.g. happen if a function requiring a pointer to a structure has been passed an unassigned pointer or if a value has been passed, that is either too large or too small in that context. [-2108]

-2109	DMR_EXPORTED_↩ SYMBOL_NOT_FOUND	One or more symbols needed in a detected driver library couldn't be resolved.	In most cases this is an error handled internally. So the user will not receive this error code as a result of a call to an API function. However when the user tries to get access to an IMPACT buffer type while the needed IMPACT Base libraries are not installed on the target system this error code also might be returned to the user. [-2109]
-2110	DEV_UNKNOWN_ERROR	An unknown error occurred while processing a user called driver function.	[-2110]
-2111	DEV_HANDLE_INVALID	A driver function has been called with an invalid device handle.	[-2111]
-2112	DEV_INPUT_PARAM_↩ INVALID	A driver function has been called but one or more of the input parameters are invalid.	There are several possible reasons for this error↩ : an unassigned pointer has been passed to a function, that requires a valid pointer one or more of the passed parameters are of an incorrect type one or more parameters contain an invalid value (e.g. a filename that points to a file that can't be found, a value, that is larger or smaller than the allowed values. within the current setup one or more parameters impose restrictions on the requested operation that don't allow its execution. [-2112]
-2113	DEV_WRONG_INPUT_↩ PARAM_COUNT	A function has been called with an invalid number of input parameters.	[-2113]
-2114	DEV_CREATE_SETTING↩ _FAILED	The creation of a setting failed.	This can either happen, when a setting with the same name as the one the user tried to create already exists or if the system can't allocate memory for the new setting. [-2114]

-2115	DEV_REQUEST_CANT_↩ BE_UNLOCKED	The unlock for a Request object failed.	This might happen, if the Request is not locked at the time of calling the unlock function. It either has been unlocked by the user already or this request has never been locked as the request so far has not been used to capture image data into its buffer. Another reason for this error might be that the user tries to unlock a request that is currently processed by the device driver. [-2115]
-2116	DEV_INVALID_↩ REQUEST_NUMBER	The number for the Request object is invalid.	The max. number for a Request object is the value of the property RequestCount in the SystemSettings list - 1. [-2116]
-2117	DEV_LOCKED_↩ REQUEST_IN_QUEUE	A Request that hasn't been unlocked has been passed back to the driver.	This error might occur if the user requested an image from the driver but hasn't unlocked the Request that will be used for this new image. [-2117]
-2118	DEV_NO_FREE_↩ REQUEST_AVAILABLE	The user requested a new image, but no free Request object is available to process this request.	[-2118]
-2119	DEV_WAIT_FOR_↩ REQUEST_FAILED	The wait for a request failed.	This might have several reasons: The user waited for an image, but no image has been requested before. The user waited for a requested image, but the image is still not ready(e.g. because of a short timeout and a long exposure time). A triggered image has been requested but no trigger signal has been detected within the wait period. A plug and play device(e.↩ g. an USB device) has been unplugged and therefore can't deliver images anymore. In this case the 'state' property should be checked to find out if the device is still present or not. [-2119]
-2120	DEV_UNSUPPORTED_↩ PARAMETER	The user tried to get/set a parameter, which is not supported by this device.	[-2120]

-2121	DEV_INVALID_RTC_↩ NUMBER	The requested real time controller is not available for this device.	[-2121]
-2122	DMR_INTERNAL_ERROR	Some kind of internal error occurred.	More information can be found in the ∗.log-file or the debug output. [-2122]
-2123	DMR_INPUT_BUFFER_↩ TOO_SMALL	The user allocated input buffer is too small to accommodate the result.	[-2123]
-2124	DEV_INTERNAL_ERROR	Some kind of internal error occurred in the device driver.	More information can be found in the ∗.log-file or the debug output. [-2124]
-2125	DMR_LIBRARY_NOT_↩ FOUND	One or more needed libraries are not installed on the system.	[-2125]
-2126	DMR_FUNCTION_NOT_↩ IMPLEMENTED	A called function or accessed feature is not available for this device.	[-2126]
-2127	DMR_FEATURE_NOT_↩ AVAILABLE	The feature in question is (currently) not available for this device or driver.	This might be because another feature currently blocks the one in question from being accessible. More information can be found in the ∗.log-file or the debug output. [-2127]
-2128	DMR_EXECUTION_↩ PROHIBITED	The user is not permitted to perform the requested operation.	This e.g. might happen if the user tried to delete user data without specifying the required password. [-2128]
-2129	DMR_FILE_NOT_FOUND	The specified file can't be found.	This might e.g. happen if the current working directory doesn't contain the file specified. [-2129]
-2130	DMR_INVALID_LICENCE	The licence doesn't match the device it has been assigned to.	When e.g. upgrading a device feature each licence file is bound to a certain device. If the device this file has been assigned to has a different serial number then the one used to create the licence this error will occur. [-2130]
-2131	DEV_SENSOR_TYPE_↩ ERROR	There is no sensor found or the found sensor type is wrong or not supported.	[-2131]

-2132	DMR_CAMERA_↩ DESCRIPTION_INVALID	A function call was associated with a camera description, that is invalid.	One possible reason might be, that the camera description has been deleted(driver closed?). 1.5.0 [-2132]
-2133	DMR_NEWER_LIBRARY↩ _REQUIRED	A suitable driver library to work with the device manager has been detected, but it is too old to work with this version of the mvDevice↩ Manager library.	This might happen if two different drivers have been installed on the target system and one introduces a newer version of the device manager that is not compatible with the older driver installed on the system. In this case this error message will be written into the logfile together with the name of the library that is considered to be too old. The latest drivers will always be available online under www.↩ matrix-vision.de. There will always be an updated version of the library considered to be too old for download from here. 1.6.6 [-2133]
-2134	DMR_TIMEOUT	A general timeout occurred.	This is the typical result of functions that wait for some condition to be met with a timeout among their parameters. More information can be found in the ∗.log-file or the debug output. 1.7.2 [-2134]
-2135	DMR_WAIT_ABANDONED	A wait operation has been aborted.	This e.g. might occur if the user waited for some message to be returned by the driver and the device driver has been closed within another thread. In order to inform the user that this waiting operation terminated in an unusual wait, DMR_↩ WAIT_ABANDONED will be returned then. 1.7.2 [-2135]
-2136	DMR_EXECUTION_↩ FAILED	The execution of a method object or reading/writing to a feature failed.	More information can be found in the log-file. 1.9.0 [-2136]

-2137	DEV_REQUEST_↩ ALREADY_IN_USE	This request is currently used by the driver.	This error may occur if the user tries to send a certain request object to the driver by a call to the corresponding image request function. 1.10.31 [-2137]
-2138	DEV_REQUEST_↩ BUFFER_INVALID	A request has been configured to use a user supplied buffer, but the buffer pointer associated with the request is invalid.	1.10.31 [-2138]
-2139	DEV_REQUEST_↩ BUFFER_MISALIGNED	A request has been configured to use a user supplied buffer, but the buffer pointer associated with the request has an incorrect alignment.	Certain devices need aligned memory to perform efficiently thus when a user supplied buffer shall be used to capture data into this buffer must follow these alignment constraints 1.10.31 [-2139]
-2140	DEV_ACCESS_DENIED	The requested access to a device could not be granted.	There are multiple reasons for this error code. Detailed information can be found in the ∗.log-file. POSSIBLE CAUSES: an application tries to access a device exclusively that is already open in another process a network device has already been opened with control access from another system and the current system also tries to establish control access to the device an application tried to execute a function that is currently not available an application tries to write to a read-only location. 1.↩ 10.39 [-2140]

-2141	DMR_PRELOAD_↩ CHECK_FAILED	A pre-load condition for loading a device driver failed.	Certain device drivers may depend on certain changes applied to the system in order to operate correctly. E.g. a device driver might need a certain environment variable to exist. When the device manager tries to load a device driver it performs some basic checks to detect problems like this. When one of these checks fails the device manager will not try to load the device driver and an error message will be written to the selected log outputs. 1.10.52 [-2141]
-2142	DMR_CAMERA_↩ DESCRIPTION_INVALID↩ _PARAMETER	One or more of the camera descriptions parameters are invalid for the grabber it is used with.	There are multiple reasons for this error code. Detailed information can be found in the ∗.log-file. POSSIBLE CAUSES: The TapsXGeometry or Taps↩ YGeometry parameter of the selected camera description cannot be used with a user defined AOI. A scan standard has been selected, that is not supported by this device. An invalid scan rate has been selected. ... This error code will be returned by frame grabbers only. 1.10.57 [-2142]
-2143	DMR_FILE_ACCESS_↩ ERROR	A general error returned whenever there has been a problem with accessing a file.	There can be multiple reasons for this error and a detailed error message will be sent to the log-output whenever this error code is returned. POSSIBLE CAUSES: The driver tried to modify a file, for which it has no write access. The driver tried to read from a file, for which it has no read access. ... 1.10.87 [-2143]
-2144	DMR_INVALID_QUEUE_↩ SELECTION	An error returned when the user application attempts to operate on an invalid queue.	1.11.0 [-2144]

-2145	DMR_ACQUISITION_↩ ENGINE_BUSY	An error returned when the user application attempts to start the acquisition engine at a time, where it is already running.	2.5.3 [-2145]
-2146	DMR_BUSY	An error returned when the user application attempts to perform any operation that currently for any reason cannot be started because something else already running.	The log-output will provide additional information. 2.↩ 32.0 [-2146]
-2147	DMR_OUT_OF_MEMORY	An error returned when for any reason internal resources (memory, handles, ...) cannot be allocated.	The log-output will provide additional information. 2.↩ 32.0 [-2147]
-2199	DMR_LAST_VALID_↩ ERROR_CODE	Defines the last valid error code value for device and device manager related errors.	[-2199]

1.17.2 Accessing log files

If you need support using our products, you can shorten response times by sending us your log files. Accessing the log files is different in Windows and Linux:

1.17.2.1 Windows

Since

mvIMPACT Acquire 2.11.9

You can access the log files in Windows using **wxPropView** (p. 139). The way to do this is described in **"Accessing Log Files"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual.

1.17.2.2 Linux

Since

mvIMPACT Acquire 2.24.0

You can access the log files in Linux via `/opt/mvIMPACT_Acquire/data/logs`.

You can also extract the directory using the following command

```
env | grep MVIMPACT_ACQUIRE_DATA_DIR
```

or change the directory directly via

```
cd $MVIMPACT_ACQUIRE_DATA_DIR/logs
```

For older versions:

Like on Windows, log files will be generated, if the activation flag for logging called **mvDebugFlags.mvd** is available in the same folder as the application (however, using Windows log files will be generated automatically, because the applications are started from the same folder). By default, on Linux the **mvDebugFlags.mvd** will be installed in the installation's destination folder in the sub-folder "`apps`". For example, if the destination folder was "`/home/workspace`", you can locate the **mvDebugFlags.mvd** like the following way:

```
user@linux-desktop:~$
// <- Starting the console window, you will be in the home directory: /home/
user@linux-desktop:~$ cd workspace/apps/
// <- Change the directory
user@linux-desktop:/home/workspace/apps$ ls -l
// <- List the directory
insgesamt 144
drwxr-xr-x  9 user user  4096 Mai 21 15:08 Callback
drwxr-xr-x  8 user user  4096 Mai 21 15:08 Callback_C
drwxr-xr-x  9 user user  4096 Mai 21 15:08 CaptureToUserMemory_C
drwxr-xr-x  3 user user  4096 Mai 21 15:03 Common
drwxr-xr-x 11 user user  4096 Mai 21 15:09 ContinuousCapture
drwxr-xr-x  9 user user  4096 Mai 21 15:09 ContinuousCaptureAllDevices
drwxr-xr-x  6 user user  4096 Mai 21 15:09 ContinuousCaptureFLTK
drwxr-xr-x  9 user user  4096 Mai 21 15:09 ContinuousCapture_C
drwxr-xr-x 11 user user  4096 Mai 21 15:09 DigitalIOs
drwxr-xr-x  9 user user  4096 Mai 21 15:09 FirmwareUpgrade
drwxr-xr-x 11 user user  4096 Mai 21 15:09 GenericInterfaceLayout
drwxr-xr-x 11 user user  4096 Mai 21 15:09 GenICamInterfaceLayout
-rw-r--r--  1 user user   854 Mai 21 15:03 Makefile
-rw-r--r--  1 user user  7365 Mai 21 15:03 Makefile.samp.inc
-rw-r--r--  1 user user 20713 Mai 21 15:03 mvDebugFlags.mvd
// <- Log activation flag
drwxr-xr-x  7 user user  4096 Mai 21 15:09 mvDeviceConfigure
drwxr-xr-x  6 user user  4096 Mai 21 15:10 mvIPConfigure
drwxr-xr-x  6 user user  4096 Mai 21 15:11 mvPropView
drwxr-xr-x  9 user user  4096 Mai 21 15:11 SingleCapture
drwxr-xr-x  9 user user  4096 Mai 21 15:11 SingleCaptureStorage
```

For log file generation you have to execute your app from the folder where **mvDebugFlags.mvd** is located. E.g. if you want to start wxPropView:

```
user@linux-desktop:/home/workspace/apps$ ./mvPropView/x86/wxPropView
// <- Start the executable from the folder, where \b mvDebugFlags.mvd is located.
```

Another possibility would be, to copy the **mvDebugFlags.mvd** file to the folder of the executable:

```
user@linux-desktop:/home/workspace/apps$ cp mvDebugFlags.mvd ./mvPropView/x86/wxPropView
// <- Copy the log activation flag
user@linux-desktop:/home/workspace/apps$ cd ./mvPropView/x86/
// <- Change the directory
user@linux-desktop:/home/workspace/apps/mvPropView/x86/$ ./wxPropView
// <- Start the executable
```

Afterwards, several log files are generated which are listed in `files.mvloglist`. The log files have the file extension .mvlog. Please send these files to our support team.

1.17.3 General Issues

- **I can see the camera in mvIPConfigure, but I cannot access the camera** (p. 198)
- **Image Data Is Lost (Incomplete Frames)** (p. 199)
- **The error counter increases** (p. 200)
- **I get an exception when setting OffsetX or OffsetY** (p. 202)
- **Cognex VisionPro** (p. 203)
- **I cannot focus on distant objects** (p. 208)
- **mvGigEConfigure freezes when trying to Activate/Deactivate the filter driver a Link Aggregation Group(LAG)** (p. 208)
- **mvBlueCOUGAR-X105G with frame rates $<$ 5.75 Hz** (p. 208)
- **I get an oscillating frame rate** (p. 209)
- **Why does updating the device list take so long** (p. 209)

1.17.3.1 I can see the camera in mvIPConfigure, but I cannot access the camera

In some cases, it can be possible that you can see the camera in **mvIPConfigure** (p. 139), however,

- there are error messages in **mvIPConfigure** (p. 139) like *"Cannot Access Device"*,
- you cannot *Use* the camera in **wxPropView** (p. 139), or
- the camera is even not listed in **wxPropView** (p. 139).

There could be two reasons for this problem:

1. The communication is blocked by a firewall.
2. There are network configuration problems, which means that either the network is incorrectly configured or with regards to the network settings the camera itself.

To solve this issue, please check the firewall settings of your system. In the latter case, please have a look at the following chapters of the manual:

- **mvIPConfigure** (p. 139)
- **Optimizing Network Performance (Windows)** (p. 46)
- **Optimizing Network Performance (Linux)** (p. 57)
- **9. Link Aggregation, Teaming Or Bonding (GigE Vision Devices With Multiple NICs (mvBlueCOUGAR-XD))** (p. 54) (Windows)
- **4. Link Aggregation, Teaming Or Bonding (GigE Vision Devices With Multiple NICs (mvBlueCOUGAR-XD))** (p. 63) (Linux)

1.17.3.2 Image Data Is Lost (Incomplete Frames)

1.17.3.2.1 Symptoms While acquiring data from the device image data is lost e.g. the counter for incomplete or lost images is increasing or request objects inside an application are returned with errors.

1.17.3.2.2 Cause There are **A LOT** of potential causes for this so a general cause as well as a general resolution cannot be given:

- Switches, routers or cables in between the device and the NIC of the host system might simply not be able to cope with the amount of data coming from the device.
- The NIC might not be able to get rid of the incoming data fast enough.
- The underlying driver might not be fast enough to send the captured data up to the application.
- The application might not process the received data fast enough eventually blocking all the memory that could be used for capturing data into thus the driver must drop incoming data.

1.17.3.2.3 Resolution Since there are so many potential reasons for data loss, the following section will try to provide some guidance to track down the root cause on a specific system step by step.

1. Make sure that everything described here has already been set up in an optimal way: **Optimizing Network Performance** (p. 46).
2. Check your BIOS for stuff mentioned here: **The error counter increases** (p. 200).
3. Make sure you have understood the concepts for bandwidth management described **here** (p. 343) and that you have set up your system and application accordingly.

If the problem persists after everything described in the sections mentioned above has been configured correctly and it has been double checked that the bandwidth coming from the device(s) should be digestible by the NICs they are connected to:

First of all try to reproduce the issues using wxPropView or multiple instances of wxPropView with the same settings (or at least almost the same) as your application. Depending on that:

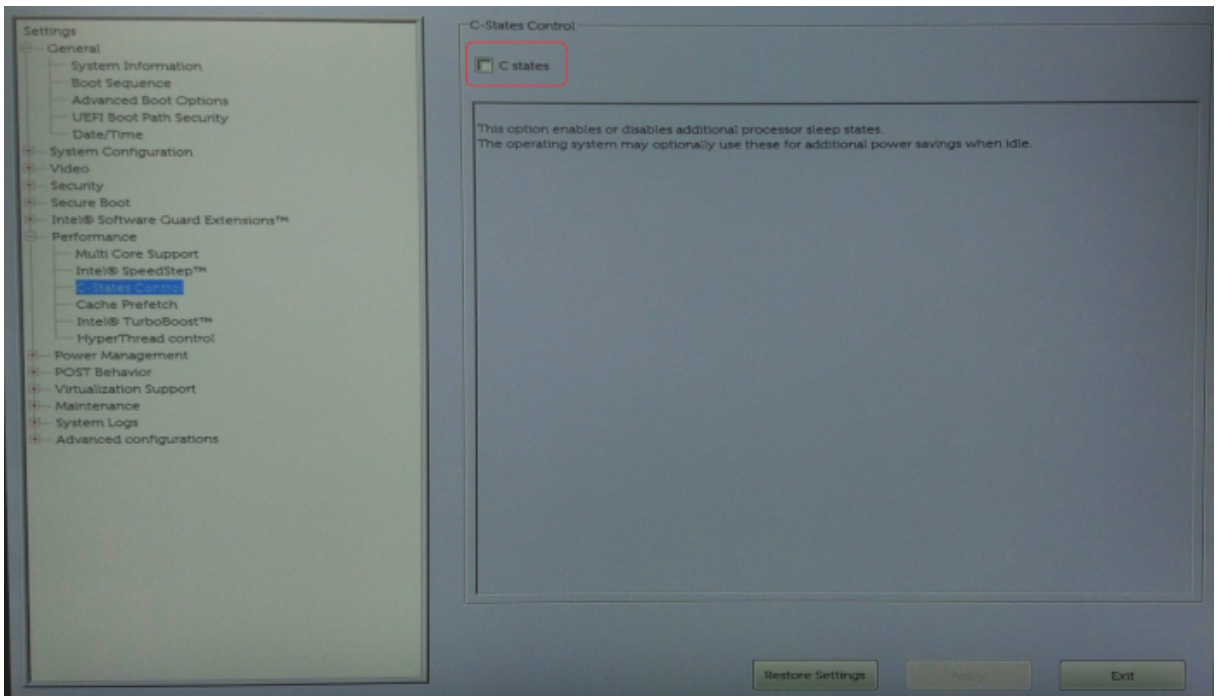
- If wxPropView is working fine but your application is not try to find out what is different:
  1. Check the amount of request objects used by wxPropView and your application.
  2. Check if your application sometimes blocks a lot of request objects because of extensive image processing so the driver has no more buffers to capture into.
  3. Check if queuing and retrieving buffers from the mvIMPACT Acquire driver in your application actually works as expected by checking for error codes returned from the various *imageRequestXYZ* functions.
- If wxPropView is showing the same behaviour:
  - When every frame is corrupted check the actual link speed reported by the device and NIC for link speed negotiation issues. Maybe one of the components for some reason is running at a reduced (100 MBit/s, ...) link speed.
  - Check if the MTU has actually been negotiated as expected, so look for the *GevSCPSPacketSize* property and check if the value is close to the MTU configured for the NIC. If it is not try to set it to a higher value (close to the MTU of the NIC) manually but be aware that a switch in between might not be capable of dealing with jumbo packets. If that is the case you will no longer receive **ANY** data! Think about exchanging components that cannot deal with jumbo frames with components that can.

- Check if reducing the image size, frame rate or the bandwidth (see **Limiting the bandwidth of the imaging device** (p. 343)) helps. **IMPORTANT:** Reducing the image size and reducing the bandwidth can result in the same amount of data being sent per second, but still the transmission on the wire is different! Smaller **ROIs** (p. 221) or a reduced frame rate will transmit the images at full link speed but with **LARGE GAPS** in between the **individual images** while using the bandwidth limit features the network data will be sent using a lot of **SMALL GAPS** in between all **individual network packets** (and one image typically consists of a lot of network packets). So reduced **ROIs** (p. 221) might actually have no effect on the amount of lost data while reducing the bandwidth might! If that is the case your network connection in combination with the NIC is most likely the cause for the problem. It is also important to understand that when 2 devices are transmitting their data simultaneously on the same link without reducing their bandwidth usage, the peak data might be beyond the transmission links capacity! E.g. 2 1 GBit/s devices sending a very small image at exactly the same time might generate 2 GBit/s peak bandwidth! As most switches have very small buffers this might result in data loss very quickly! The only downside of reducing the bandwidth usage is that the full image will arrive in the application slightly later, since the packets are sent with some delay, but for multi-camera scenarios this very often is the only feasible option.
- Try to remove/exchange certain components like switches.
- Look for updated NIC or mvIMPACT Acquire drivers.

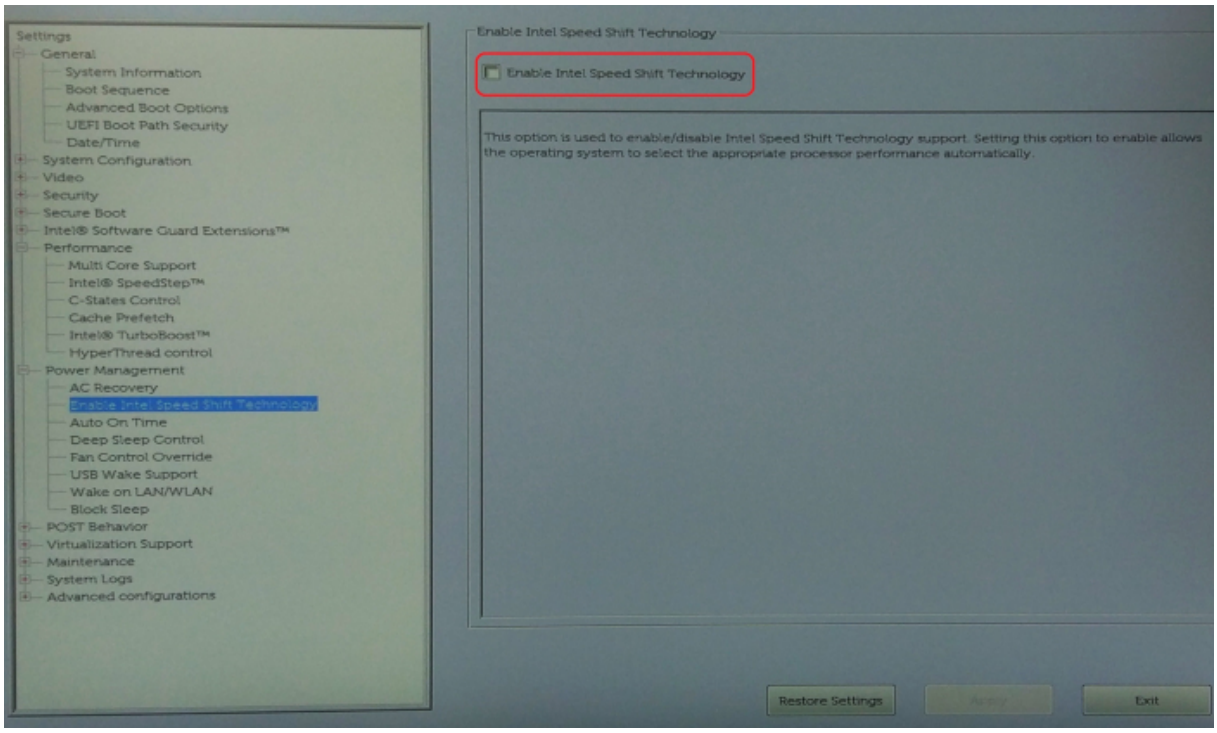
1.17.3.3 The error counter increases

Modern PC's, notebook's, etc. try to save energy by using a smart power management. If processor performance is not needed the processor will change to a power saving (sleep) state automatically and vice versa. Every state change will stop the processor for microseconds. This time is enough to cause image error counts!

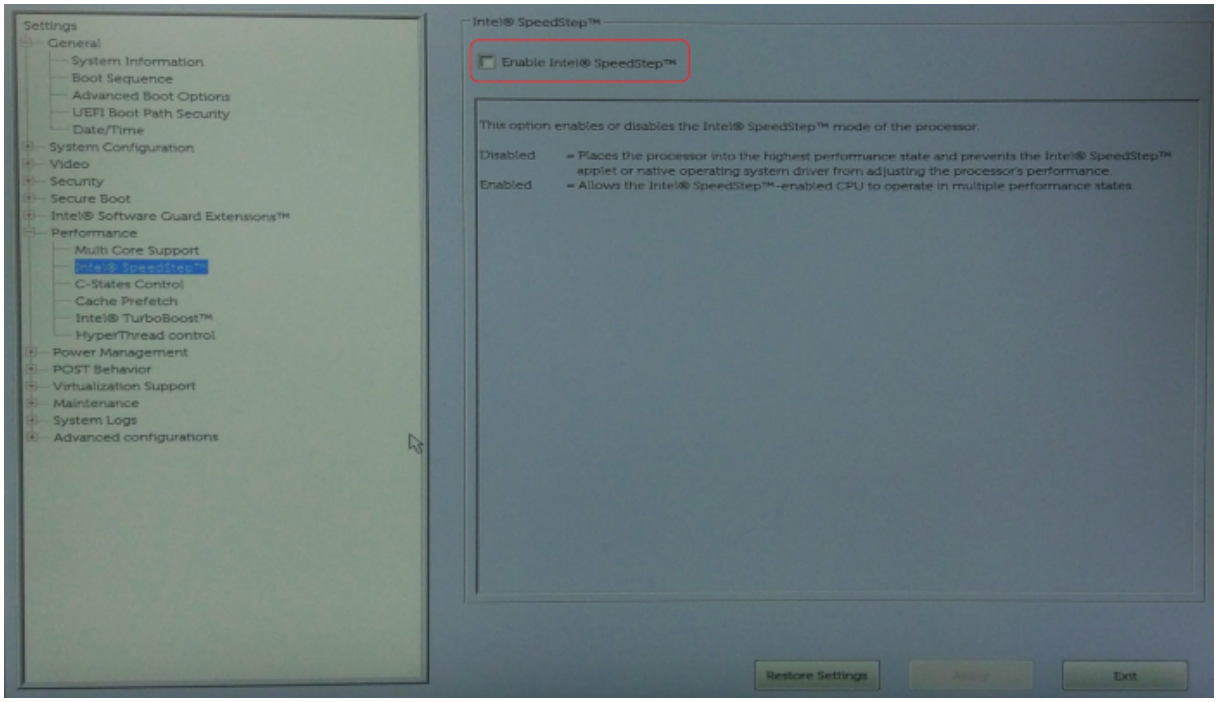
With mvDeviceConfigure running on Windows 7 or below you can disable the CPU sleep states on Windows systems. As the API needed for this has been removed in Windows 8 and above doing the same on Windows 8 or higher or when working with Linux system must be done by using BIOS options. Below are some exemplary screen-shots from BIOS settings. Your system might use different names or look different of course:



Swiching off the C states might improve the situation



Swiching off Intel SpeedShift options might improve the situation



Swiching off Intel SpeedStep options might improve the situation

Note

Another that is always worth considering is to look for an updated version of your systems BIOS! Whenever you encounter problems like described above and you are sure that you have set up the system as perfect as possible (e.g. by configuring network cards of USB interfaces like described in the corresponding sections of the mvBlueFOX3 or mvBlueCOUGAR series) a BIOS update might solve all remaining issues!

1.17.3.4 I get an exception when setting OffsetX or OffsetY

offsetX and **offsetY** are horizontal or vertical offsets from the origin to the region of interest (in pixels). The max. value of both **offsetX** and **offsetY** changes in reference to **width** or **height**. For this reason, if you want to change both values, **offsetX** / **offsetY** and **width** / **height**, you first have to change **width** / **height** and then **offsetX** / **offsetY**.

1.17.3.5 Cognex VisionPro

1.17.3.5.1 I cannot use mvBlueCOUGAR-XD devices By default, VisionPro 7.2 by Cognex does not support Dual-GigE cameras like mvBlueCOUGAR-XD. However, there is a workaround to include Dual-GigE cameras.

> **Note**
>
> Since VisionPro 8.0, you have to start the program as administrator. Afterwards, the products using the MATRIX VISION driver (`"Device: "`) are displayed.

Instead of choosing `"GigE Vision: MATRIX VISION..."` from the image acquisition devices dropdown list, select the item `"Device: mvBlueCOUGAR-XD ..."`. Afterwards, the MATRIX VISION driver will be used:

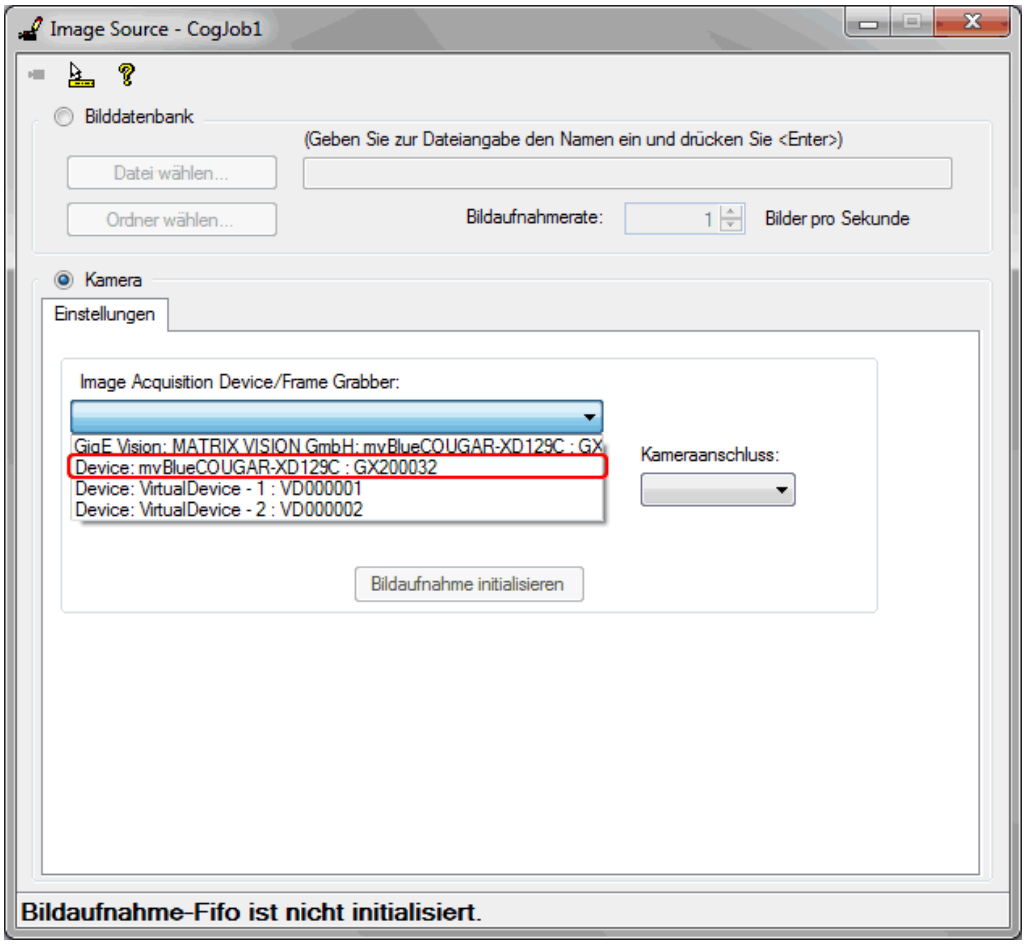


Figure 1: VisionPro - Choosing the Image Acquisition Device

1.17.3.5.2 I cannot initialize 4 tap sensor mvBlueCOUGAR-XDs VisionPro 7.2 by Cognex does not support 4 Tap sensors, like the CCD sensors by Sony available for the mvBlueCOUGAR-XD.

> **Note**
>
> Since VisionPro 8.0, you have to start the program as administrator. Afterwards, the products using the MATRIX VISION driver (`"Device: "`) are displayed.

If you use the **workaround to use Dual-GigE cameras** (p. 203) (1) and click on **"Initialize Image Acquisition"** (2), ...

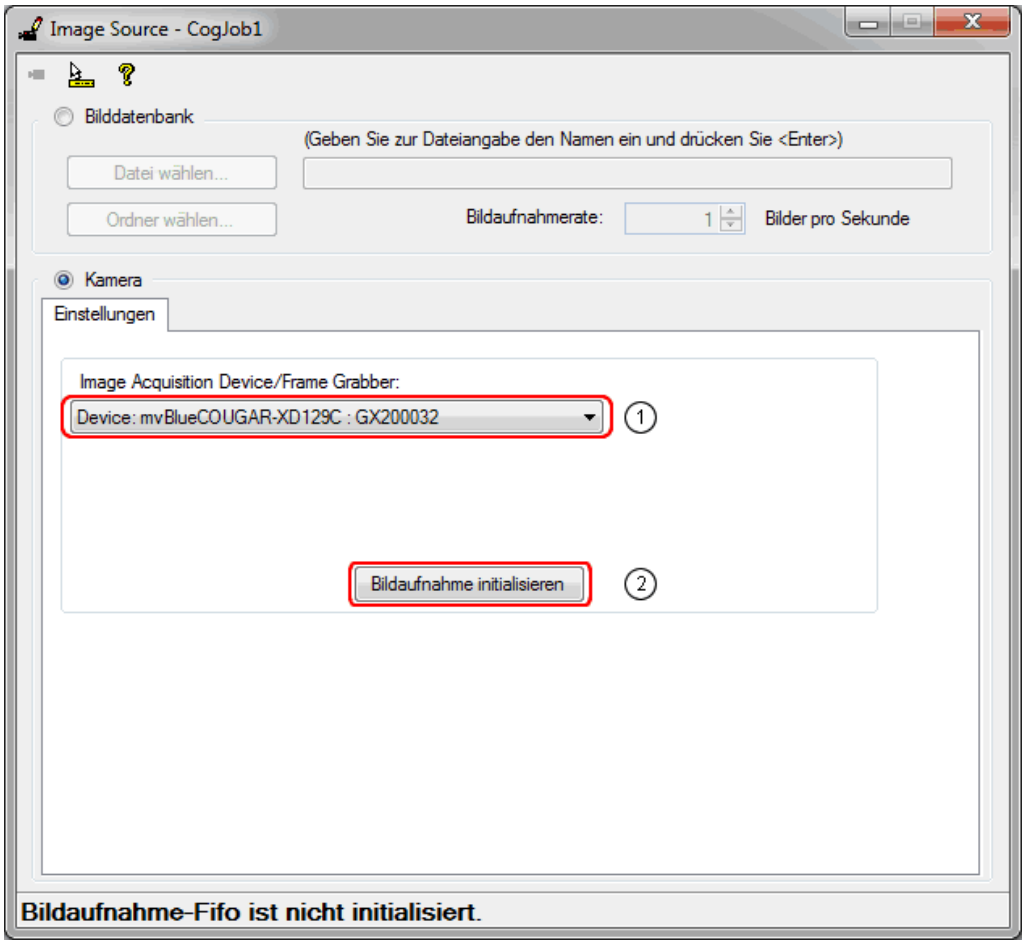


Figure 2: VisionPro - Initialize Image Acquisition

... you can get following error message:

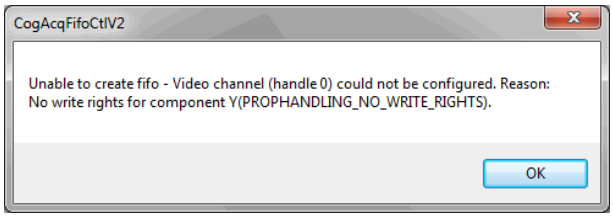


Figure 3: VisionPro - Error message

In this case, the 4 Tap mode of the mvBlueCOUGAR-XD is activated, which cannot be changed by VisionPro directly:

1. Close VisionPro,
2. open **wxPropView** (p. 139) and
3. set **SensorTaps** to **"Two"** in `"Setting -> Base -> Camera -> GenICam -> ImageFormatControl"`. The frame rate will be halved.

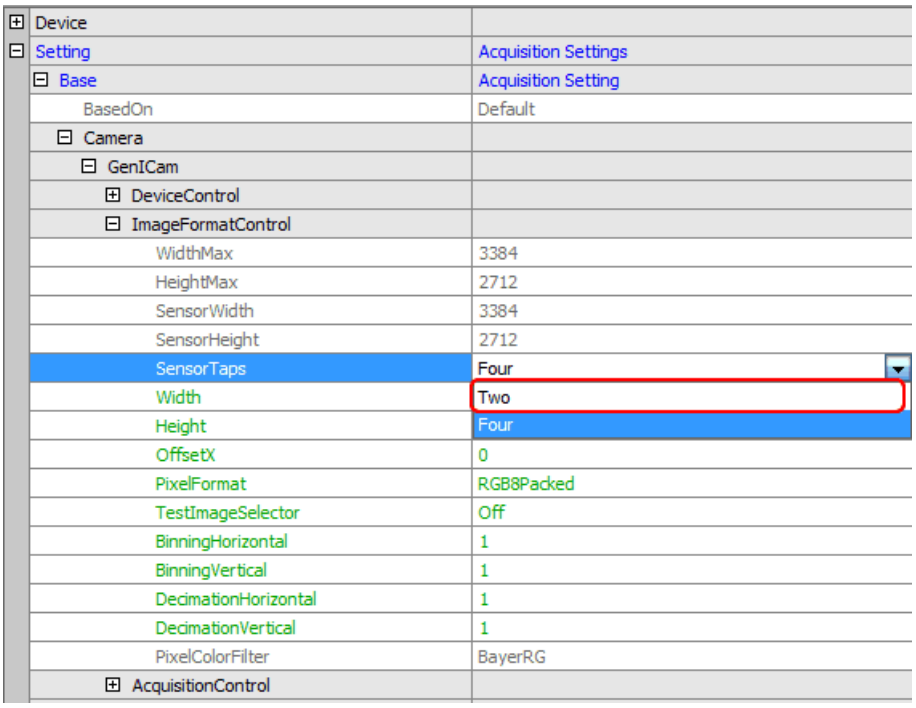


Figure 4: wxPropView - Setting SensorTaps

4. In order that these settings are using during the start, you can save them in "`Setting -> Base -> Camera -> GenICam -> UserSetControl`".
   Choose a **UserSet**, e.g. **"UserSet1"** (1),
5. save the **UserSet** by clicking on **"int UserSetSave()"** (2), and
6. select the **UserSet** in **"UserSetDefault"** (3).

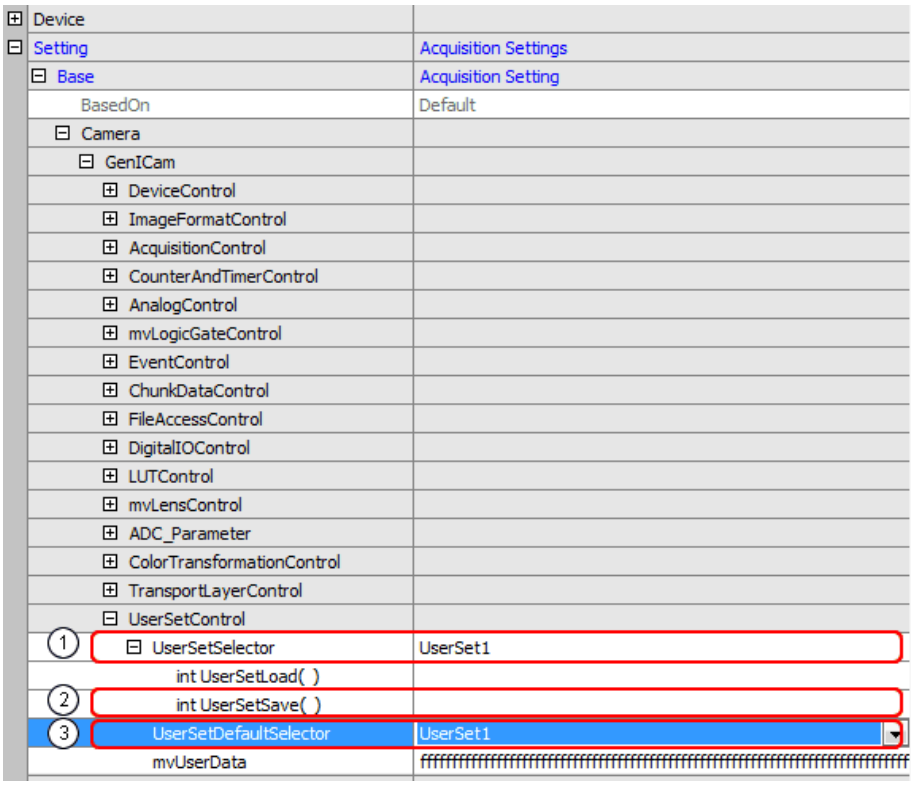


Figure 5: wxPropView - Setting UserSet

7. Afterwards it is possible to initialize the mvBlueCOUGAR-XD by clicking on **"Initialize Image Acquisition"**:

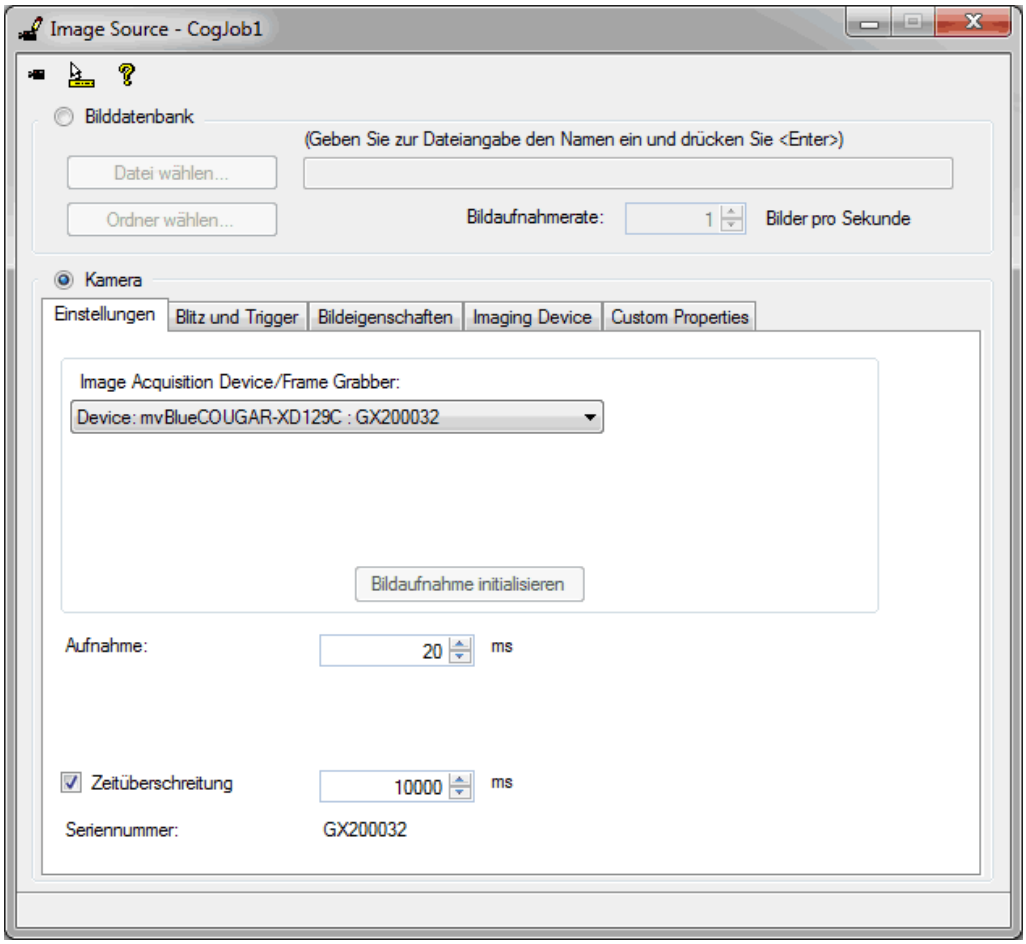


Figure 6: VisionPro - Initializing the mvBlueCOUGAR-XD

8. By clicking on the camera icon (top left), you should see a live image. Perhaps you have to adapt the acquisition parameters:

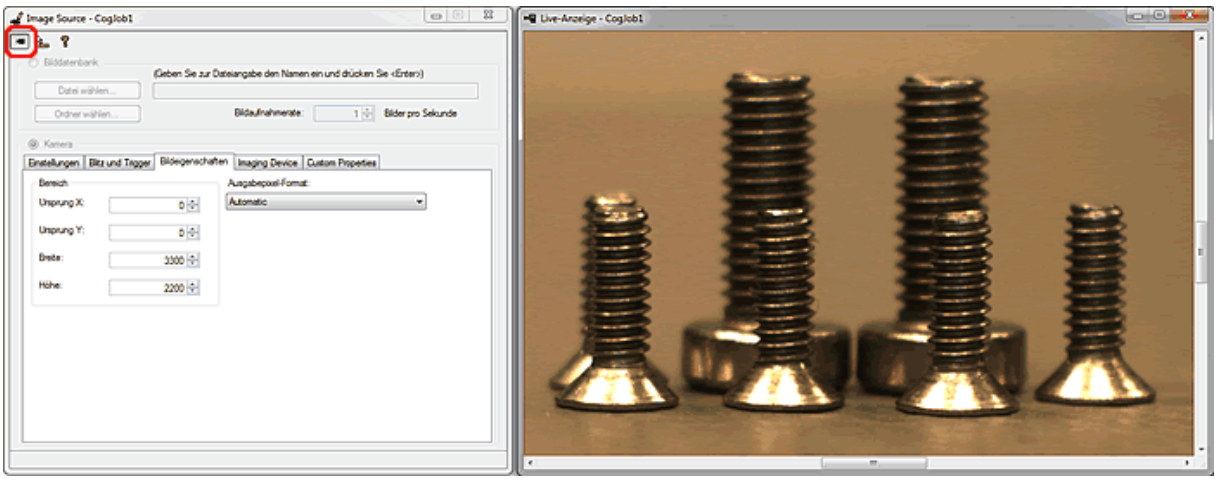


Figure 7: VisionPro - Live image

Note

By default, the resolution of VisionPro 7.2 is 640 x 480 pixels. For this reason you will see a part of the whole live image.

1.17.3.6 I cannot focus on distant objects

In most vision applications it is necessary to focus on close objects.
For this reason we adjust the lensholder to meet the lens' MOD (= minimum object distance).

If you need to focus on distant objects, you can adjust the C-mount lensholder as described in **Adjusting the C-mount** (p. 10).

1.17.3.7 mvGigEConfigure freezes when trying to Activate/Deactivate the filter driver a Link Aggregation Group(LAG)

1.17.3.7.1 Symptoms Activating or deactivating the filter driver on a Link Aggregation Group (LAG for short) using **mvGigEConfigure** (p. 139), the NDIS filter driver will freeze and the process cannot be killed even with task manager. After a restart, **mvGigEConfigure** (p. 139) will show the filter driver status correctly however. Also when connecting a debugger the situation as described here: `https://social.msdn.microsoft.com/↩Forums/windowsdesktop/en-US/0735c73e-6d2e-49ab-a947-e10fe8137ac3/ndis-driver-freezes-dur` can be observed.

1.17.3.7.2 Cause The exact cause for this issue is unknown but it is believed to be a bug in the network cards kernel module.

1.17.3.7.3 Resolution So far, to avoid this behavior disconnect both network cables just before activating or deactivating the filter driver. Afterwards reconnect the network cables again.

1.17.3.8 mvBlueCOUGAR-X105G with frame rates $<$ 5.75 Hz

1.17.3.8.1 Cause The frame rate can not be reduced below 5.75fps because of some technical limitation related to the free running mode of the sensor.

1.17.3.8.2 Resolution As soon as the sensor is used in triggered mode, the frame rate can be below 5.75 fps.

To get the camera working at lower frame rates, it is necessary to configure two timers which are used to trigger the sensor.

E.g., the camera is triggered at 4 fps.

- **Timer1** represents the duration of the "low" signal. It is configured to `240`,000 us. `TimerTriggerSource` is configured to `"Timer2End"`.
- **Timer2** represents the "high" signal. It is configured to 10,000us. `TimerTriggerSource` of Timer2 is `"Timer1End"`. The sum of both durations is 250,000 us which means the sequence runs with 4 cycles per second. (1s/250,000us=4)

Afterwards the `"TriggerSource"` has to be configured to `"Timer2Start"` and the `"TriggerMode"` has to be switched to `"On"`.

Once you start the acquisition by clicking the **"Acquire"** button, the camera should run at 4 fps.

To get the camera working at different frame rates, **Timer1** needs to be configured accordingly. E.g. 500,000 us for 2 fps.

1.17.3.9 I get an oscillating frame rate

If your camera supports **FrameRateExactness** it is possible that you may receive an oscillating frame rate. This is due to the fact, that frames of different lengths are used to achieve an overall stable and exact frame rate. We use 10 frames to achieve an more exact frame rate, i.e. if you capture 10 frames, your frame rate will be exact and stable.

Example:

1. Assume we want to achieve a frame rate of 150 fps.
2. This means we need to wait 6666,6667 us until the next frame start.
3. The possible step size is 1 us.
4. If we set 6666 us, the frame rate will be stable, but not exact enough.
5. => 6 frames of length 6666 and 4 frames of length 6667 make the frame rate more exact.

1.17.3.10 Why does updating the device list take so long

Since

mvIMPACT Acquire 2.24.0

The GenTL driver scans both the GigE Vision and the USB3 Vision interfaces for compliant devices. Given that the GigE Vision standard allows detection timeouts of 1000 ms, it could take long, especially for USB3 Vision users which even do not have GigE Vision in their application, until the device update is finished.

For this reason, we upgraded the **MATRIX VISION application tools** (p. 139) to excluded interfaces or single devices to be scanned or detected. The easiest way to do so is as follows:

1. Open **wxPropView** (p. 139) .
2. Open the device list drop down and
3. select **"Missing A Device? Click here..."**.

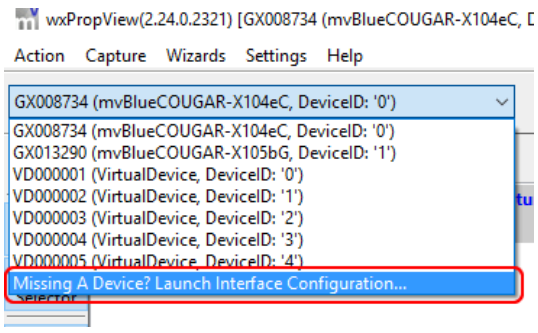


Figure 1: "Missing A Device? Click here..."

Now, you will see the interfaces and the detected device.

4. Finally, you can deselect the interfaces or devices which you do not want to scan for updating the device list. The following figure shows that the GigE Vision interface should be skipped:

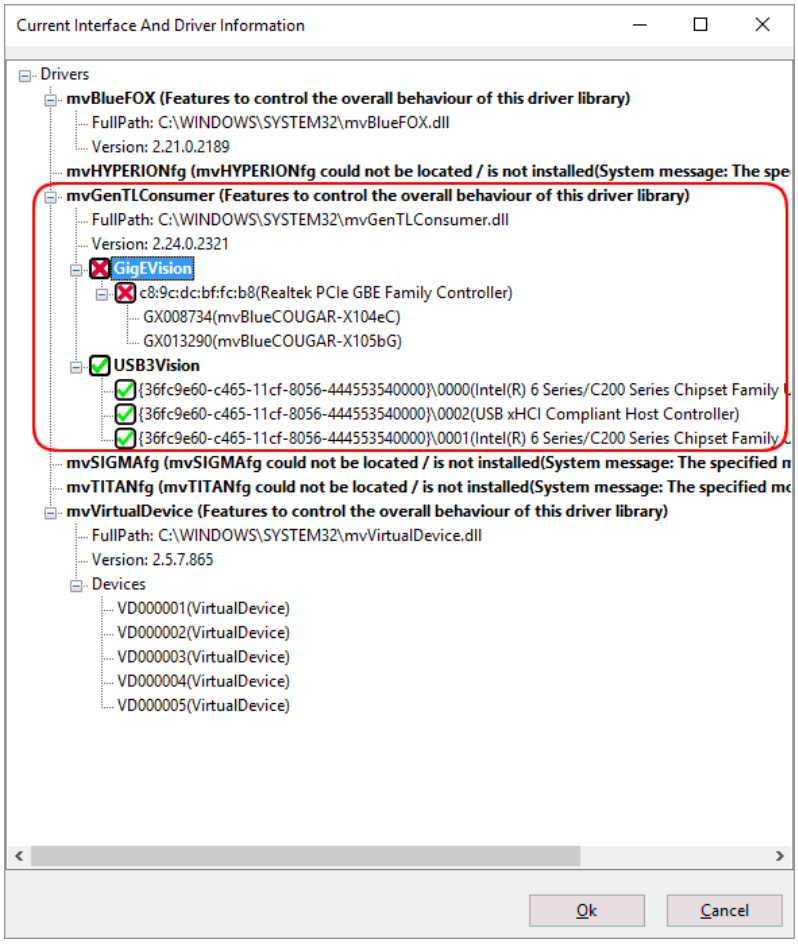


Figure 2: Skip the GigE Vision interface

1.17.4 Windows

- **The 'LostImagesCount' increases** (p. 210)

1.17.4.1 The 'LostImagesCount' increases

Symptoms

Especially when working with 10 GigE devices at high data rates on a Windows system the property `Lost↩ ImagesCount` increases sporadically while the system is more or less idle. This especially can be observed when

- One CPU's load is medium or high because of the 10G network traffic it processes
- All other CPUs are more or less idle

The effect is reduced or even disappears completely when additional load is introduced to the system (e.g. through image processing or different processes).

Cause

The exact cause for this so far is not known. It seems like when a single CPU in a multi-core Windows system is heavily loaded while all the others are not the operating system forces the process causing most of the load to interrupt for a longer period of time (100 - 300 ms have been observed on Windows 10). While the network traffic in kernel space (thus inside the NDIS filter driver) continues to get processed then the user space portion of the process starves thus not more complete images are reported to the application and as a consequence the filter driver runs out of buffers after a while.

Resolution

The most obvious resolution to overcome problems caused by this behavior is to increase the request count for the problematic data stream. Having enough request objects to survive about 500 ms have proven to always work reliable. So when running at 50 Hz when this problem is encountered increasing the request count to 25 buffers should resolve all problems.

Generating addition load on the system that gets processed on different CPUs also seems to improve the situation a lot. E.g. when displaying all the images or when operating another device in parallel usually makes this effect disappear.

1.17.5 Linux

- **No GenICam devices are detected on a Linux system** (p. 211)
- **No GigE Vision™ devices are detected on a Linux system** (p. 212)

1.17.5.1 No GenICam devices are detected on a Linux system

Symptoms

After starting any mvIMPACT Acquire based application no GenICam compliant devices are accessible or detected

Cause

The environment variables which are necessary to us the mvGenTL Acquire package are defined by two shell scripts which are set by profile.d. This causes the variables to be available within every login-shell run on the system. Once an application is started from a non-login shell which is using mvIMPACT Acquire libraries the variables from profile.d are not used. This results in a situation that the necessary runtime libraries can not be found which leads to missing interfaces and devices.

Resolution

To avoid this situation there are two possible solutions:

- Use a login-shell to run the application using the MATRIX VISION driver package.
- Insert the variables from: /etc/profile.d/genicam.sh and /etc/profile.d/acquire.sh to a shell configuration file which is read to configure environment variables.

1.17.5.2 No GigE Vision™ devices are detected on a Linux system

Symptoms

After starting any mvIMPACT Acquire based application no GigE Vision™ compliant devices are accessible or detected

Cause

At it is known from surfing the internet, normally the Ethernet communication channel is protect using a firewall. GigE Vision compliant devices are communicating via Ethernet, too, so perhaps the firewall prevents that the mv↩ BlueCOUGAR cameras are detected.

Resolution

As a test, you can switch off the firewall. If the GigE Vision™ devices is now detected, you have to adapt the firewall settings. The default installation of Ubuntu contains **ufw** - the **u** ncomplicated **f** ire **w** all, which is command line based front end for iptables.

At first it might be interesting to figure out if the firewall is actually enabled. This can be done by calling:

```
sudo ufw status
```

If the firewall is enabled, the output on Ubuntu 20.04 will be:

```
Status: active
```

To turn it off, just enter the following command in the command line:

```
sudo ufw disable
```

Checking the status of the firewall again using:

```
sudo ufw status
```

Should return:

```
Status: inactive
```

Once the firewall is disabled the device should be discovered flawlessly by any of the mvIMPACT Acquire applications.

Note

If it is necessary to have an enabled firewall and to use GigE Vision™ devices, it will be necessary to define rules for the firewall. The following UDP ports need to be open: **3956** and from **49152 to 65535** to make sure GigE Vision™ devices can be used without any interference from the firewall.

1.18 Glossary

ADC	Analog-to-digital converter (A/D converter).
API	Application programming interface (API). The standard API for MATRIX VISION products is called **mvIMPACT_Acquire** (p. 219).
BusyBox	Configurable monolithic application including shell and other useful command line tools - often called the "swiss army knife" for embedded systems. Even desktop distributions are sometimes relying on BusyBox due to its robustness. Please see `http://www.busybox.net` for details.
CIFS	Common Internet file system (CIFS) replaced Samba in 2006. It gets rid of NetBIOS packets an introduced Unix features like soft/hard links and allows larger files.
CPU	Central processing unit aka processor.
DAC	Digital to analog converter (D/A converter).
Defaults	Standard system settings.
DHCP	Dynamic Host Configuration Protocol (DHCP). DHCP is a protocol used by networked devices (clients) to obtain various parameters necessary for the clients to operate in an Internet Protocol (IP) network.
Digital I/O	Digital inputs and outputs.
GDB	GDB, the GNU Project debugger.

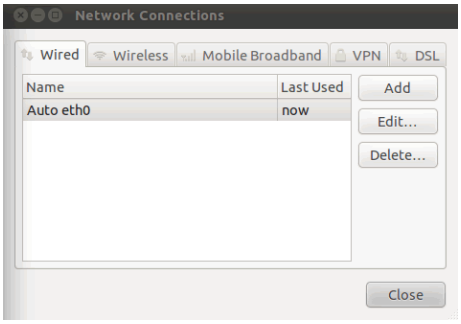
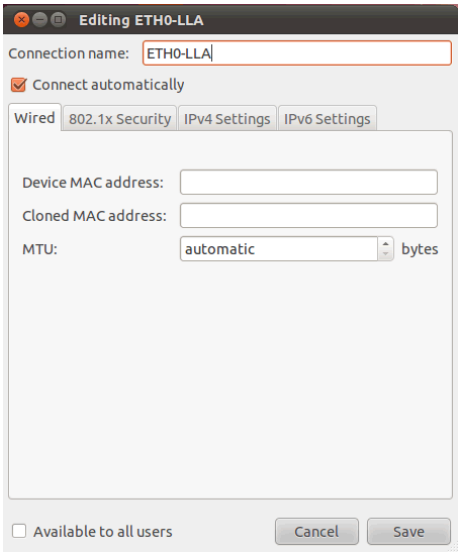
GenICam	GenICam stands for **GEN**eric programming **I**nterface for **CAM**eras. It's a generic way to access and modify device parameters with a unified interface. A GenICam compliant device either directly provides a GenICam compliant description file (in internal memory) or the description file can be obtained from a local (hard disk etc.) or web location. A GenICam description file is something like a machine readable device manual. It provides a user readable name and value range for **parameters** (p. 221) that are offered by the device for reading and/or writing and instructions on what command must be sent to a device when the user modifies or reads a certain parameter from the device. These description files are written in XML. An excerpt from such a file can be seen in the figure below:
	
	`<Float Name="GainAbs" NameSpace="Standard">`
	`  <ToolTip>Sets the camera's gain in dB</ToolTip>`
	`  <Description>This float value sets the camera's gain in dB.</Description>`
	`  <DisplayName>Gain (Abs)</DisplayName>`
	`  <pIsLocked>TLParamsLocked</pIsLocked>`
	`  <pValue>GainAbs_CtrlValueFao</pValue>`
	`  <Unit>dB</Unit>`
	`  <Representation>Linear</Representation>`
	`</Float>`
	`<FloatReg Name="GainAbs_CtrlValueFao" NameSpace="Custom">`
	`  <Address>0x00000004</Address>`
	`  <pAddress>CamCtrl_AnalogCtrlBaseAddress</pAddress>`
	`  <Length>8</Length>`
	`  <AccessMode>RW</AccessMode>`
	`  <pPort>Device</pPort>`
	`  <Endianess>BigEndian</Endianess>`
	`</FloatReg>`
	
	Excerpt of a GenICam description file (in XML)
	For further information on this topic please have a look at `https://en.wikipedia.org/wiki/GenICam`.

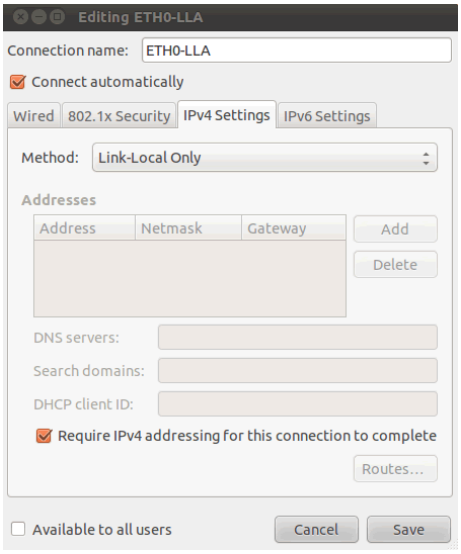
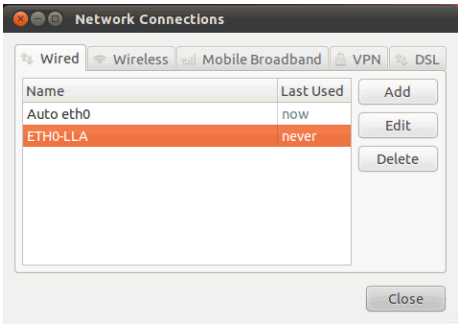
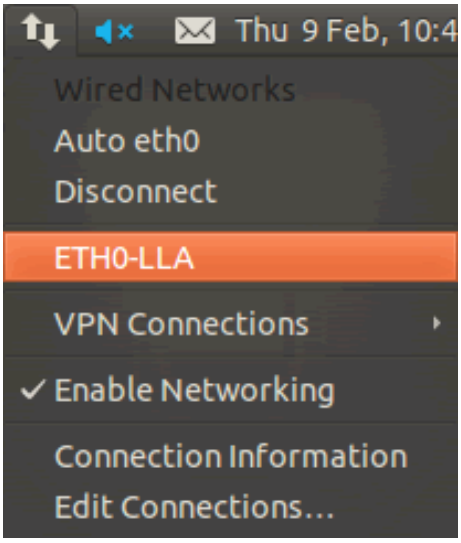
GenTL	GenTL is the transport layer interface for cameras, acquiring images from the camera, and moving them to the user application.

Gigabit Ethernet (GigE)	The term Gigabit Ethernet (defined by the IEEE 802.3-2008 standard) represents various technologies for transmitting Ethernet frames at a rate of a gigabit per second (1,000,000,000 bits per second).

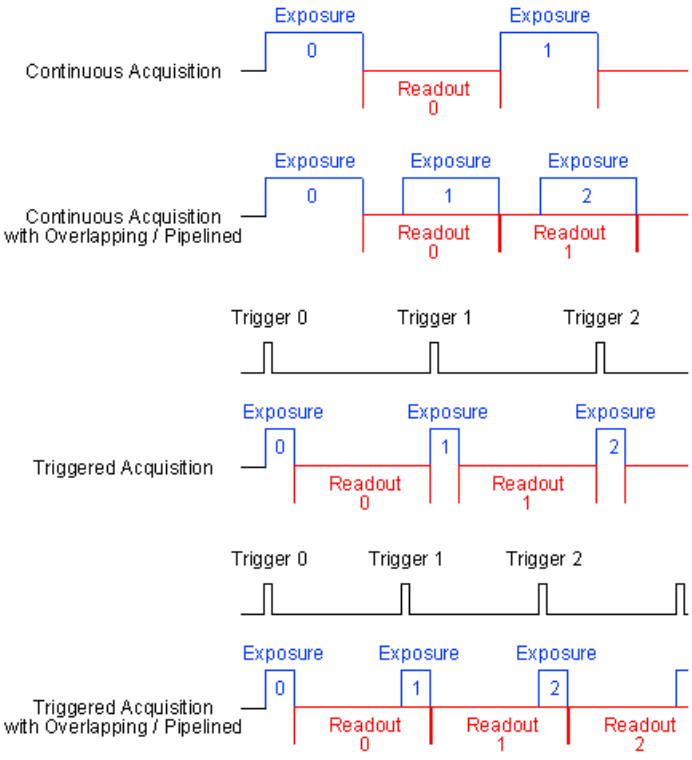
GigE Vision	GigE Vision is a network protocol designed for the communication between an imaging device and an application. This protocol completely describes: • device discovery • data transmission – image data – additional data • read/write of parameters. GigE Vision uses UDP for data transmission to reduce overhead introduced by TCP. **Note** UDP does not guarantee the order in which packets reach the client nor does it guarantee that packets arrive at the client at all. However, GigE Vision defines mechanisms that can detect lost packets. This allows capture driver manufacturers to implement algorithms that can reconstruct images and other data by requesting the device to resend lost data packets until the complete buffer has been assembled. For further information please have a look at `https://en.wikipedia.org/wiki/GigE_Vision` The MATRIX VISION GigE Vision capture filter driver as well as the socket based acquisition driver and all MATRIX VISION GigE Vision compliant devices support resending thus lost data can be detected and in most cases reconstructed. This of course can not enhance the max. bandwidth of the transmission line thus if e.g. parts of the transmission line are overloaded for a longer period of time data will be lost anyway. Both capture drivers will allow to fine tune the resend algorithm used internally and both drivers will also provide information about the amount of data lost and the amount of data that was re-requested. This information/configuration will be part of the drivers SDK. More information about it can be found in the corresponding interface description. **Note** On Windows 2000 the filter driver does not support the **"Resend"** mechanism.

High Dynamic Range (HDR)	The HDR (High Dynamic Range) mode increases the usable contrast range. This is achieved by dividing the integration time in two or three phases. The exposure time proportion of the three phases can be set independently. Furthermore, it can be set, how many signal of each phase is charged.
HRTC	With a *Hardware Real-Time Controller* (HRTC) built inside the FPGA users can define a PLC like sequence of operating steps to control basic time critical functions like exposure time, image trigger and I/O ports. Timing is hard real-time with sub microsecond high resolution.
IDE	a software application that provides comprehensive facilities to computer programmers for software development. An IDE normally consists of a source code editor, a compiler and/or interpreter, build automation tools, and (usually) a debugger.
IPKG	Itsy package management system originally designed for embedded systems. Please have a look at `https://en.wikipedia.org/wiki/Ipkg` or a more sophisticated documentation at `http://buffalo.nas-central.org/index.php` "Overview_of_the_ipkg_package_management_system" MATRIX VISION distributes all non-firmware, i.e. optional software as ipk packages.
JFFS2	JFFS2 is a file system which supports wear leveling. **See also** Sources about the JFFS file system: • `http://sources.redhat.com/jffs2/` • `http://www.linux-mtd.infradead.org/faq/jffs2.html`
Link Aggregation (LAG)	Dual-GigE cameras need a network interface card with two network interfaces. However, both network interfaces have to work as a unit. Link aggregation (LAG) or bonding is the name of the game and has to be supported by the network interface card's driver. With it you can bond the two network interfaces so that the work as one interface.

LLA	Logical link address (LLA) is a type of mechanism to obtain a valid IP address without a DHCP server being present. Whether an IP address is available or not is resolved using address resolution protocol (ARP) packets. If no ARP response is received on a given address it is considered unused and will be assigned to the interface. LLA space is 169.254.x.y, i.e. 16bit netmask yielding 64K possible device addresses. With Linux you have to add LLA as an additional interface. By default, you can find one interface in *Connections:* (This description uses *"Gnome Network Manager"*, however using KDE should be similar)  In *Wired*, you can add interfaces via **Add:** 

	In the tab *"IPv4 Setting"* you have to set **"Link-Local Only"**:  After saving, you will find both connections in the summary:  Now, you can select the wished connection using the left mouse button in the *"Network Manager"* menu. In the LLA case it is just the new created connection: 
MAC address	Media Access Control address (MAC address) is a quasi-unique identifier attached to most network adapters (NICs) in computer networking.

MTU	Maximum transmission unit (MTU) refers to the size (in bytes) of the largest packet that a given layer of a communications protocol can pass onwards. The default MTU for Ethernet is 1500. The optimum for Gigabit Ethernet is 8000 - 12000. Different MTU settings in the same subnet can cause package losses, i.e. **never ever** change the MTU unless you know what you're doing. Changing the MTU to other values than 1500 when using file or web services from other computers are almost always a bug. It's save to increase MTU when working in peer-to-peer mode with both devices sharing the same MTU. Please not that few network cards support 16K, most Gigabit Ethernet cards are limited to 9k, some don't support Jumbo Frames (MTU $>$ 1500) at all.
mvIMPACT Acquire	This driver supplied with MATRIX VISION products represents the port between the programmer and the hardware. The driver concept of MATRIX VISION provides a standardized programming interface to all image processing products made by MATRIX VISION GmbH. The advantage of this concept for the programmer is that a developed application runs without the need for any major modifications to the various image processing products made by MATRIX VISION GmbH. You can also incorporate new driver versions, which are available for download free of charge on our website: `https://www.matrix-vision.com`. The developer interface description of the mvIMPACT Acquire is called • mvIMPACT_Acquire_API_CPP_manual.chm • mvIMPACT_Acquire_API_C_manual.chm • mvIMPACT_Acquire_API_NET_manual.chm and can be downloaded from our website.
Netboot	With netboot you can boot mvBlueCOUGAR over network. This is especially useful when several devices share the same pieces of software, i.e. same root file system, which might be subject to change frequently.
NFS	Network File System (NFS) is a network file system protocol, allowing clients to access files over LAN. Given that you need a NFS server are uncommon on Windows, this protocol best fits for Linux-Linux connections.
NIC	Network interface card - synonym for network controller.

Overlapped / pipelined transfer	By default, the steps exposure and readout out of an image sensor are done one after the other. • By design, CCD sensors support overlap capabilities also combined with trigger (see figure). • In contrast, so-called pipelined CMOS sensors only support the overlapped mode. Even less CMOS sensors support the overlapped mode combined with trigger. Please check the **sensor summary** (p. 116). In overlapping mode, the exposure starts the exposure time earlier during readout.  **Note** In overlapped trigger mode, you have to keep in mind the following formula `interval between two trigger events >= (readout time - exposure time)`
PoE	Power over Ethernet. You can power the mvBlueCOUGAR over network cable (**IEEE** 802.3af (IEEE 802.3at Type 1) **See also** `https://en.wikipedia.org/wiki/Power_over_↩Ethernet`). The IEEE specification allows the power to be injected onto the Cat 5E cable in a number of ways. We support power supplied over the data pins as shown in the **"RJ45 table"** in the chapter *"Technical data"*. This alternative is most common.

PSE	Power sourcing equipment. The network PoE element that inserts power onto an Ethernet cable.
ROI/AOI	Region/Area of interest.
SDK	Sofware development kit (SDK). The standard image processing software library from MARTIX VISION is **mvIMPACT**.
SFNC	**S**tandard **F**eature **N**aming **C**onvention of **GenICam** (p. 213). **See also** The latest GenICam properties list can be found here: **http://www.emva.org/standards-technology/genicam/genicam-downloads/** The file is called "GenICam Standard Features Naming Convention (PDF)"
Shell	In computing, a shell is a piece of software that provides an interface for users. Command-line shells provide a command-line interface (CLI) to the operating system. The primary purpose of the shell is to invoke or "launch" another program; however, shells frequently have additional capabilities such as viewing the contents of directories.
USB3 Vision	A closed source framework, defined and administered by the Automated Imaging Association (AIA), for transmitting video and related control data over USB 3. Sometimes U3V is used as an acronym.
UDP	The User Datagram Protocol (UDP) is an Internet protocol. It is used by applications to send messages to other hosts on an Internet Protocol (IP) network.
Virtual Network Computing (VNC)	Virtual Network Computing (VNC) is a graphical desktop sharing system that uses the RFB protocol to remotely control another computer. Over a network, it transmits the mouse and keyboard events from one computer to another, relaying the graphical screen updates back in the other direction. To access the camera's desktop from a PC via VNC, • you have to know the IP address of the remote system. • Start a VNC viewer and • point it to the remote system. You won't need a password. Of course, you won't get a very fast live image display via the network with VNC but you should be able to start **wxPropView** (p. 139) and capture images.
wxWidgets	wxWidgets is a cross-platform GUI library. It can be used from languages such as C++, Python, Perl, and C#/.NET. **See also** http://www.wxwidgets.org

1.19 Use Cases

- **Introducing acquisition / recording possibilities** (p. 223)
- **Improving the acquisition / image quality** (p. 284)
- **Improving the communication** (p. 343)
- **Working with triggers** (p. 347)
- **Working with I/Os** (p. 366)
- **Working with HDR (High Dynamic Range Control)** (p. 378)
- **Working with LUTs** (p. 386)
- **Saving data on the device** (p. 394)
- **Working with device features** (p. 404)
- **Working with several cameras simultaneously** (p. 418)
- **Working with 3rd party tools** (p. 434)

1.19.1 Introducing acquisition / recording possibilities

There are several use cases concerning the acquisition / recording possibilities of the camera:

- **Acquiring a number of images** (p. 223)
- **Recording sequences in the camera** (p. 224)
- **Recording sequences with pre-trigger** (p. 225)
- **Creating acquisition sequences (Sequencer Control)** (p. 226)
- **Generating very long exposure times** (p. 239)
- **Working with multiple AOIs (mv Multi Area Mode)** (p. 241)
- **Working with burst mode buffer** (p. 245)
- **Using the SmartFrameRecall feature** (p. 249)
- **Using The Linescan mode** (p. 252)
- **Using the mvBlockscan feature** (p. 256)
- **Working with Event Control** (p. 258)
- **Polarized Data Extraction** (p. 261)
- **Using Video Stream Recording** (p. 265)
- **Introducing multicasting** (p. 276)

1.19.1.1 Acquiring a number of images

As described in chapter **Acquisition Control** (p. 144), if you want to acquire a number of images, you can use as *"Setting -> Base -> Camera -> Acquisition Control -> Acquisition Mode"* **"MultiFrame"** and you have to set the **"Acquisition Frame Count"**.

Afterwards, if you start the acquisition via the button *"Acquire"*, the camera will deliver the number of images.

The **"MultiFrame"** functionality can be combined with an external signal to start the acquisition.

There are several ways and combinations possible, e.g.:

- A trigger starts the acquisition (Figure 1).
- A trigger starts the acquisition start event and a second trigger starts the grabbing itself (Figure 2).

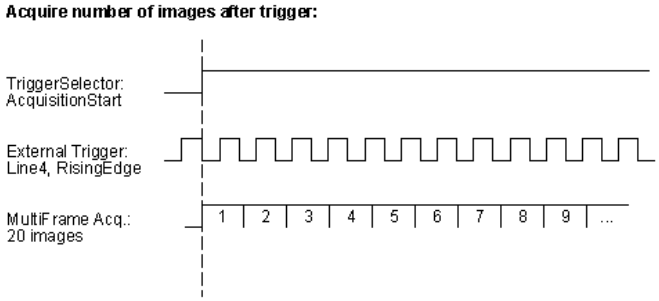


Figure 1: Starting an acquisition after one trigger event

For this scenario, you have to use the *"Setting -> Base -> Camera -> Acquisition Control -> Trigger Selector"* **"AcquisitionStart"**.

The following figure shows, how to set the scenario shown in Figure 1 with **wxPropView** (p. 139)

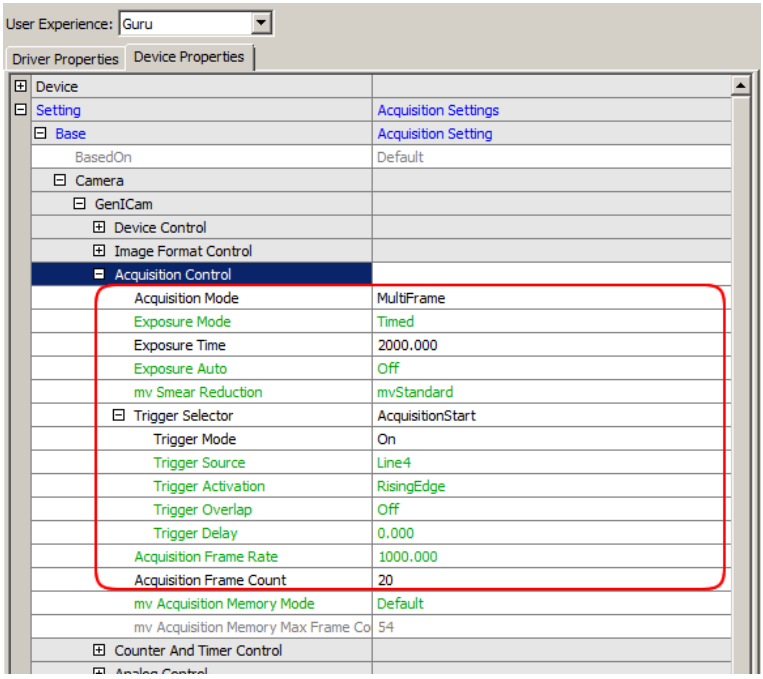


Figure 2: wxPropView - Setting acquisition of a number of images started by an external signal

A rising edge at line 4 will start the acquisition of 20 images.

1.19.1.2 Recording sequences in the camera

Besides the **mvPretrigger** (p. 225), there are two further **mv Acquisition Memory Mode** (p. 144) available:

1. **mvRecord** (p. 144)
   which uses frame rate and trigger settings.
2. **mvPlayback** (p. 144)
   which disregards the frame rate and trigger settings and outputs the images as fast as possible (throttling by **bandwidth setting** (p. 343)).

To use **mvRecord** (p. 144), just

1. set **mv Acquisition Memory Mode** (p. 144) to **mvRecord.** (p. 144)
2. Start the camera with **"Use"** and **"Acquire"**.

The camera will record **mv Acquisition Memory Max Frame Count** (p. 144) images into the memory.

To use **mvPlayback** (p. 144), just

1. set **mv Acquisition Memory Mode** (p. 144) to **mvPlayback.** (p. 144)
2. Start the camera with **"Use"** and **"Acquire"**.

The camera will playback (transfer) **mv Acquisition Memory Max Frame Count** (p. 144) images into the PC memory.

Note

mv Acquisition Memory Max Frame Count (p. 144) can be increased by reducing the image height.

1.19.1.3 Recording sequences with pre-trigger

1.19.1.3.1 What is pre-trigger?

With pre-trigger it is possible to record frames before and after a trigger event.

1.19.1.3.2 How it works

To use this functionality you have to set the **mv Acquisition Memory Mode** (p. 144) to **"mvPretrigger"**. This can be used to define the number of frames which will be recorded before the trigger event occurs:

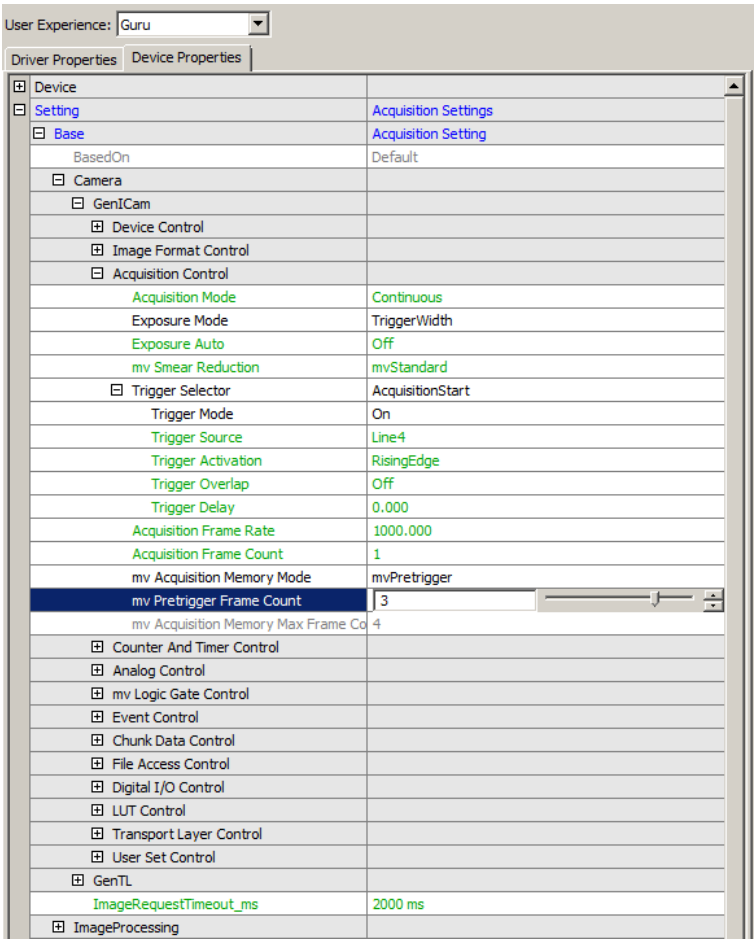


Figure 1: wxPropView - setting pre-trigger

Afterwards, you have to define the **"AcquisitionStart"** or **"AcquisitionActive"** event. In figure 1 this is **Line4** as trigger event, which starts the regular camera streaming.

Now, start the camera by pressing **"Live"** and generate the acquisition event.

The camera will output the number of pre-trigger frames as fast as possible followed by the frames in live mode as fast as possible until the frame rate is in sync:

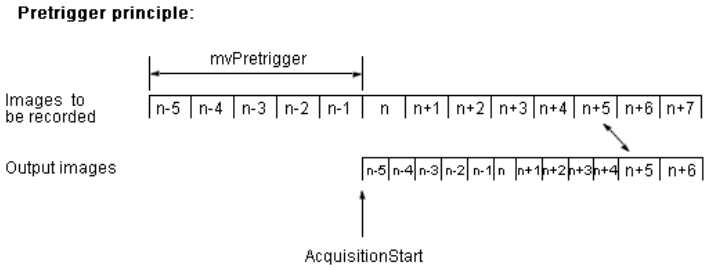


Figure 2: wxPropView - recorded / output images

1.19.1.4 Creating acquisition sequences (Sequencer Control)

1.19.1.4.1 Introduction

As mentioned in **GenICam And Advanced Features** section of this manual, the **Sequencer Mode** (p. 157) is a feature to define feature sets with specific settings. The sets are activated by a user-defined trigger source and event.

Note

At the moment, the Sequencer Mode is only available for MATRIX VISION cameras with CCD sensors and Sony's CMOS sensors. Please consult the **"Device Feature and Property List"s** to get a summary of the actually supported features of each sensor.

The following features are currently available for using them inside the sequencer control:

Feature	Note	Changeable during runtime
BinningHorizontal		-
BinningVertical		-
CounterDuration	Can be used to configure a certain set of sequencer parameters to be applied for the next `Counter↩ Duration` frames.	since Firmware version 2.15
DecimationHorizontal		-
DecimationVertical		-
ExposureTime		since Firmware version 2.15
Gain		since Firmware version 2.15
Height		since Firmware version 2.36
OffsetX		since Firmware version 2.35
OffsetY		since Firmware version 2.35
Width		since Firmware version 2.36
mvUserOutput		-
UserOutputValueAll		-
UserOutputValueAllMask		-
Multiple conditional sequencer paths		-

The **Sequencer Control** (p. 157) uses **Counter And Timer Control** (p. 148) Counter1 to work. If you already preset **Counter1** and you save a new acquisition sequence, the settings of Counter1 will be overwritten.

Note

Configured sequencer programs are stored as part of the **User Sets** (p. 396) like any other feature.

1.19.1.4.2 Creating a sequence using the Sequencer Control in wxPropView

In this sample, we define an acquisition sequence with five different exposure times on the device whereby the last step should be repeated five times. We also activate the digital outputs 0..3 to the sets accordingly - this, for example, can be used as flash signals. All the configuration is done on the device itself so after finishing the configuration and starting the acquisition the device itself will apply the parameter changes when necessary. The host application only needs to acquire the images then. This results in a much faster overall frame rate compared to when applying these changes on a frame to frame basis from the host application.

- 1000 us
- 5000 us
- 10000 us
- 20000 us
- 50000 us (5x)

This will result in the following flow diagram:

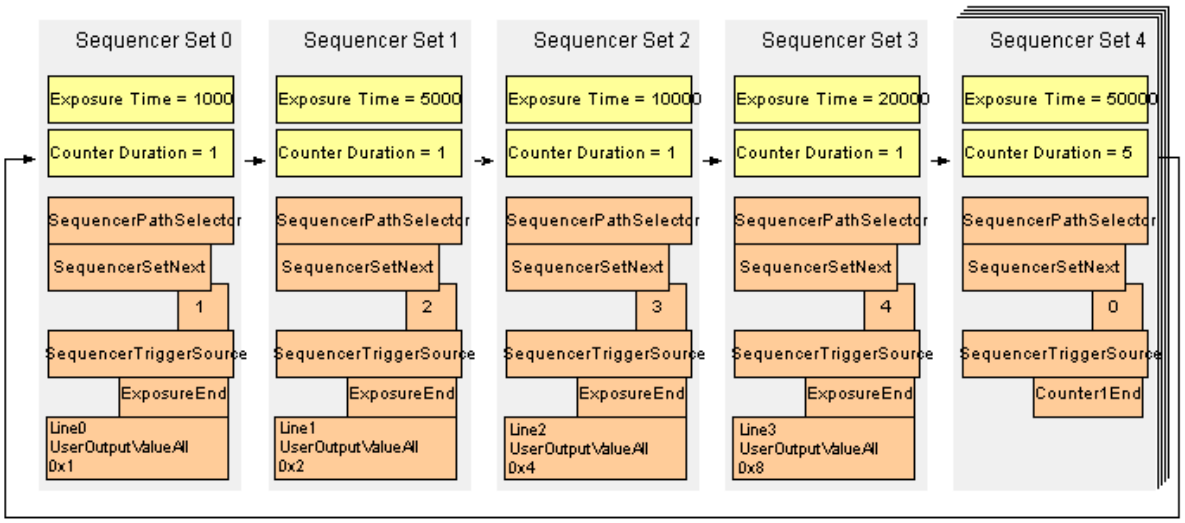


Figure 1: Working diagram of the sample

As a consequence the following exposure times will be used to expose images inside an endless loop once the sequencer has been started:

- Frame 1: 1000 us
- Frame 2: 5000 us
- Frame 3: 10000 us
- Frame 4: 20000 us

- Frame 5: 50000 us
- Frame 6: 50000 us
- Frame 7: 50000 us
- Frame 8: 50000 us
- Frame 9: 50000 us
- Frame 10: 1000 us
- Frame 11: 5000 us
- Frame 12: 10000 us
- ...

So the actual sequence that will be executed on the device later on will be like this

```
while( sequencerMode == On )
{
  take 1 image using set 0
  take 1 image using set 1
  take 1 image using set 2
  take 1 image using set 3
  take 5 images using set 4
}
```

- There are 2 C++ examples called **GenICamSequencerUsage** and **GenICamSequencerParameter↩ ChangeAtRuntime** that show how to control the sequencer from an application. They can be found in the `Examples` section of the `mvIMPACT Acquire C++ API`

The following steps are needed to configure the device as desired:

Note

This is the **SFNC** (p. 221) way how to create an acquisition sequence and consequently how you have to program it. However, **wxPropView offers a wizard** (p. 236) to define an acquisition sequence in a much easier way.

1. First, switch into the *"Configuration Mode"*: *"Sequencer Configuration Mode"* = **"On"**. Only then the sequencer on a device can be configured.

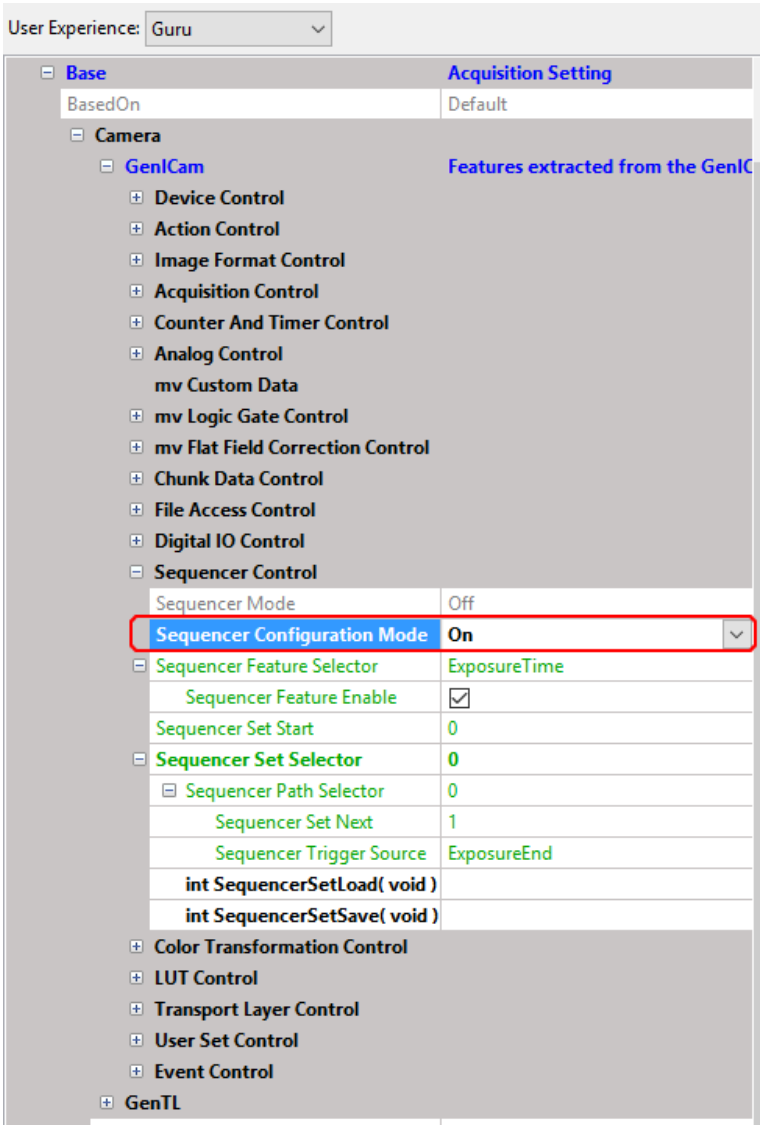


Figure 2: wxPropView - Sequencer Configuration Mode = On

2. Set the *"Sequencer Feature Selector"*, if it is not already active (pink box, figure 3): *"Sequencer Feature Selector"* = **"ExposureTime"** and *"Sequencer Feature Enable"* = **"1"**.

3. Set the *"Sequencer Feature Selector"* for the duration counter (pink box, figure 4): *"Sequencer Feature Selector"* = **"CounterDuration"** and *"Sequencer Feature Enable"* = **"1"**.

4. Set the *"Sequencer Feature Selector"* for using the UserOutputs: *"Sequencer Feature Selector"* = **"mvUser↩ Output"** and *"Sequencer Feature Enable"* = **"1"**.

5. Then, each sequencer set must be selected by the *"Sequencer Set Selector"* (orange box, figure 3): "↩ Sequencer Set Selector" = **"0"**.

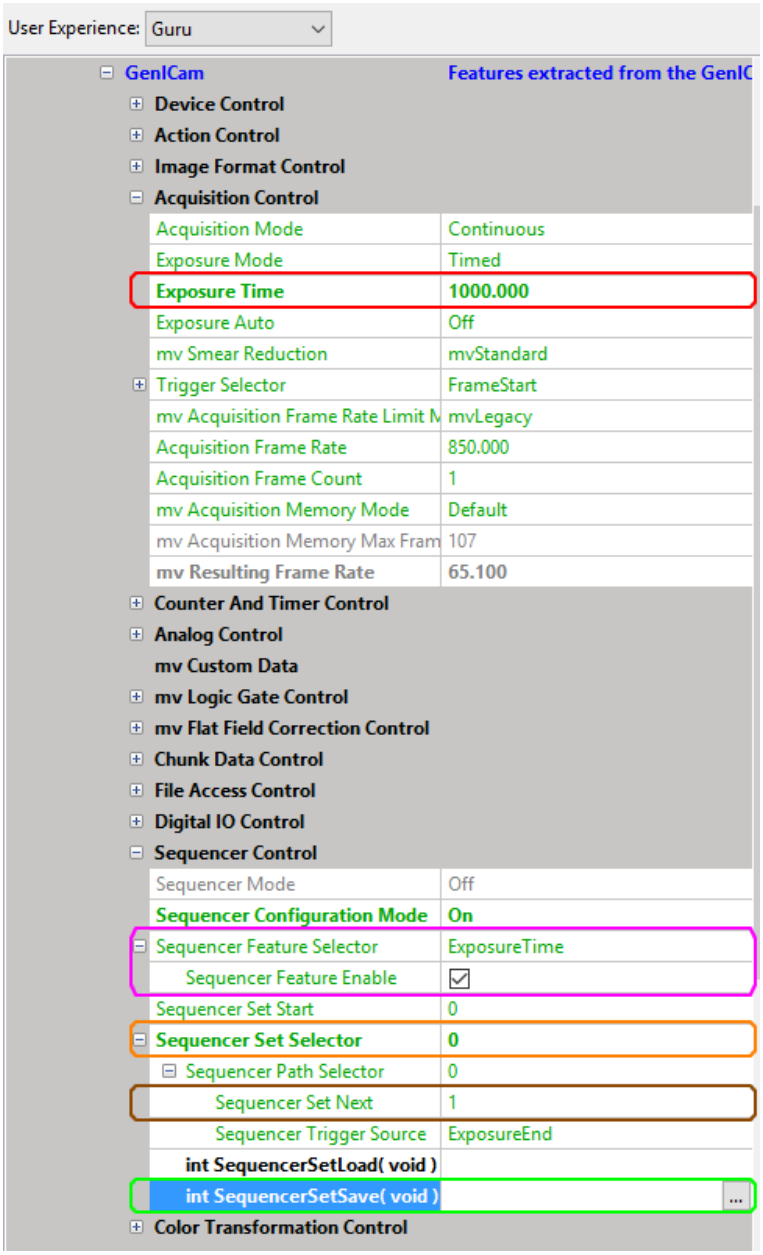


Figure 3: wxPropView - Sequencer set 0

6. Set the following sequencer set using *"Sequencer Set Next"* (brown box, figure 3): *"Sequencer Set Next"* = **"1"**.

7. Set the *"Exposure Time"* (red box, figure 3): *"Exposure Time"* = **"1000"**.

8. Finally, save the *"Sequencer Set"* (green box, figure 3): *"int SequencerSetSave()"*. This ends the configuration of this sequencer set and all the relevant parameters have been stored inside the devices RAM.

9. Set the *"UserOutputValueAllMask"* (purple box, figure 4) to a suitable value. In this case we want to use all UserOutputs, so we set it to **"0xf"**.

10. Set the *"UserOutputValueAll"* (red box, figure 4): *"UserOutputValueAll"* = **"0x1"**.

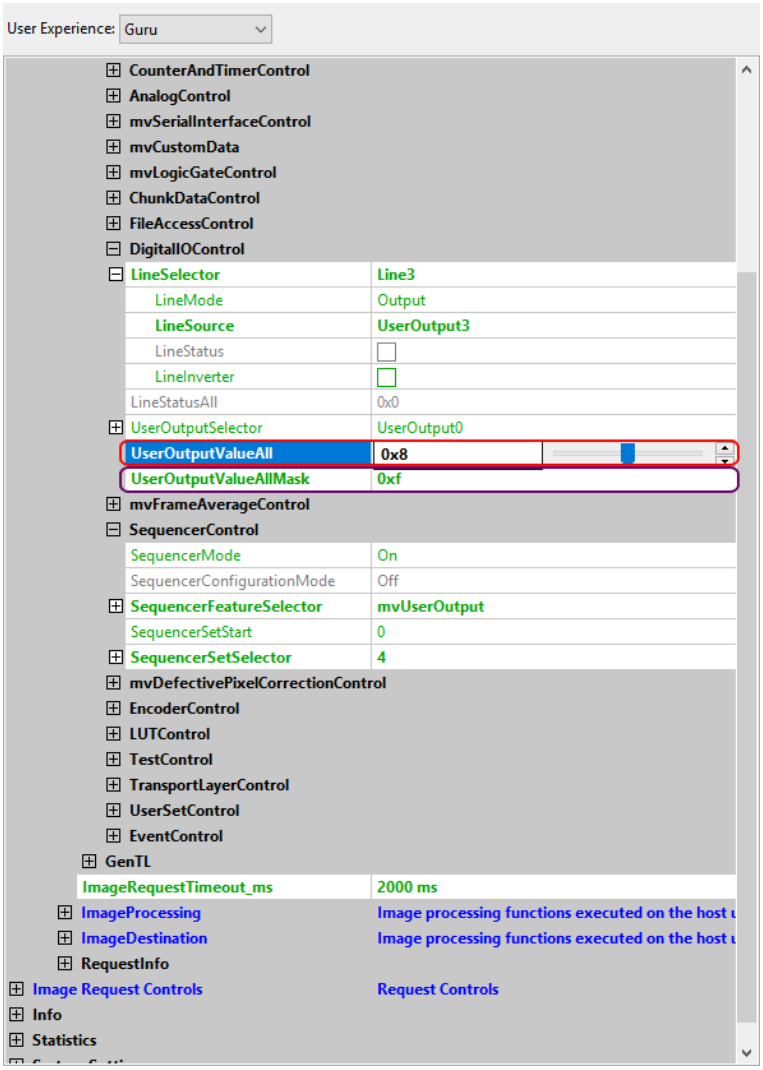


Figure 4: wxPropView - DigitalIOControl - Set UserOutputValueAll of Line3

11. Repeat these steps for the following 3 sequencer sets (Exposure Times 5000, 10000, 20000; UserOutput↩ ValueAll 0x2, 0x4, 0x8).

12. For the last sequencer set, set the desired sequencer set with *"Sequencer Set Selector"* (orange box, figure 5): *"Sequencer Set Selector"* = **"4"**.

13. Set the following sequencer set with *"Sequencer Set Next"* and trigger source with *"Sequencer Trigger Source"* (brown box, figure 5):
"Sequencer Set Next" = **"0"**. This will close the loop of sequencer sets by jumping from here back to the first one.
"Sequencer Trigger Source" = **"Counter1End"**.

14. Set the *"Exposure Time"* (red box, figure 5): *"Exposure Time"* = **"50000"**.

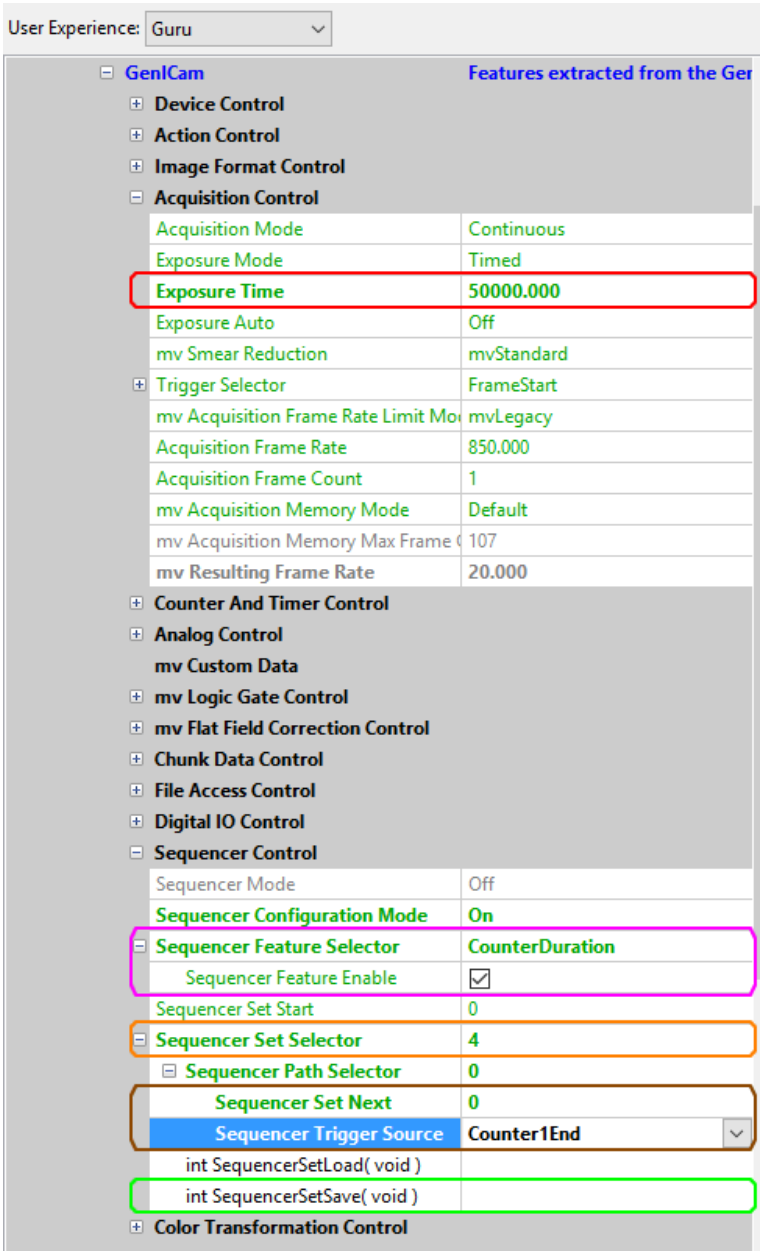


Figure 5: wxPropView - Sequencer set 4

15. Set the *"Counter Duration"* in *"Counter And Timer Control"* (red box, figure 6): *"Counter Duration"* = **"5"**.

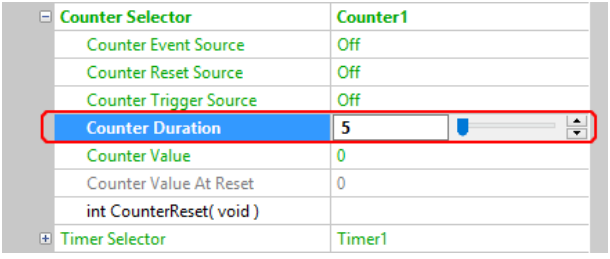


Figure 6: wxPropView - "Sequencer Mode" = "On"

16. As there are only four UserOutputs we decided not to show sequencer set "4" on the output lines.
17. Finally, save the *"Sequencer Set"* (green box, figure 3): *"int SequencerSetSave()"*.
18. Leave the *"Configuration Mode"* (red box, figure 4: *"Sequencer Configuration Mode"* = **"Off"**.
19. Activate the *"Sequencer Mode"* (red box, figure 4): *"Sequencer Mode"* = **"On"**.

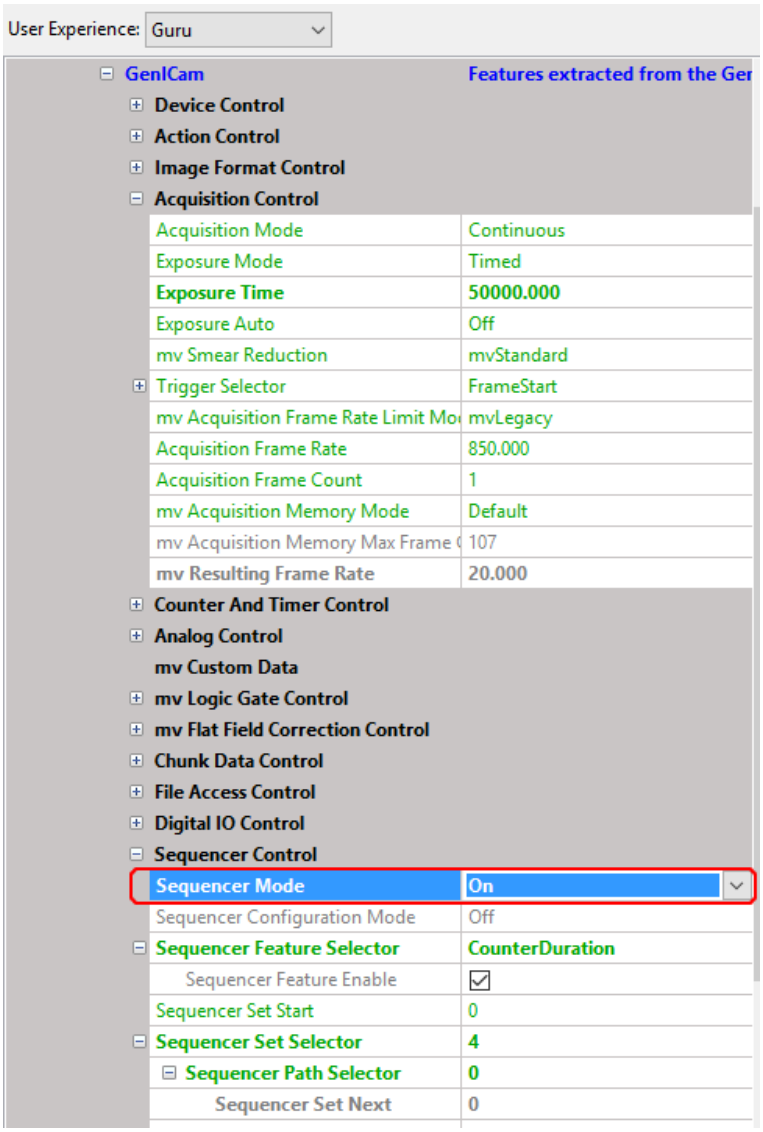


Figure 7: wxPropView - "Sequencer Mode" = "On"

Note

The **"Sequencer Mode"** will overwrite the current device settings.

You will now see that the sequencer sets are processed endlessly. Via the chunk data (activate **chunk data** (p. 154) via *"Setting -> Base -> Camera -> GenICam -> Chunk Data Control"* - activate **"Chunk Mode Active"**), the *"Info Plot"* of the analysis grid can be used to visualize the exposure times:

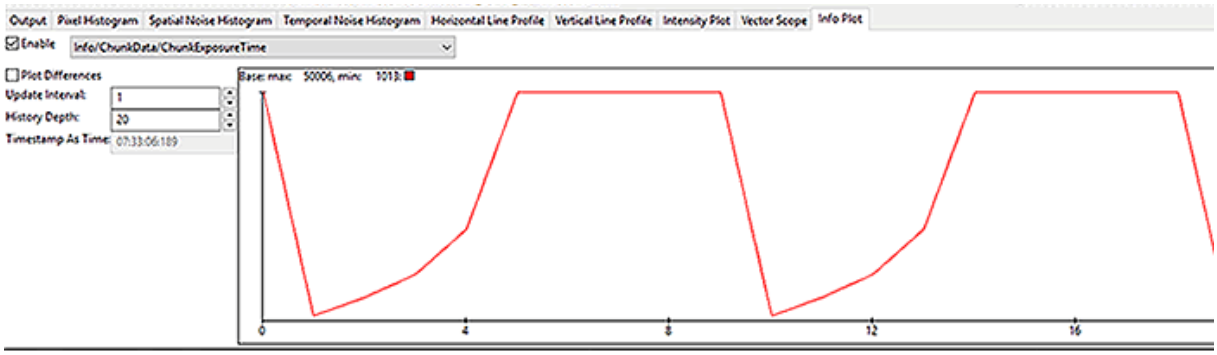


Figure 7: wxPropView - Info Plot shows the exposure times

1.19.1.4.2.1 Adapting the active time on the output lines via logic gates

If you do not want to see the whole active time of a given sequencer set but only the exposure time of a given sequencer set, you can combine your signal with the logic gates in **mvLogicGateControl** (p. 165). Figure 8 shows sample settings of Line3:

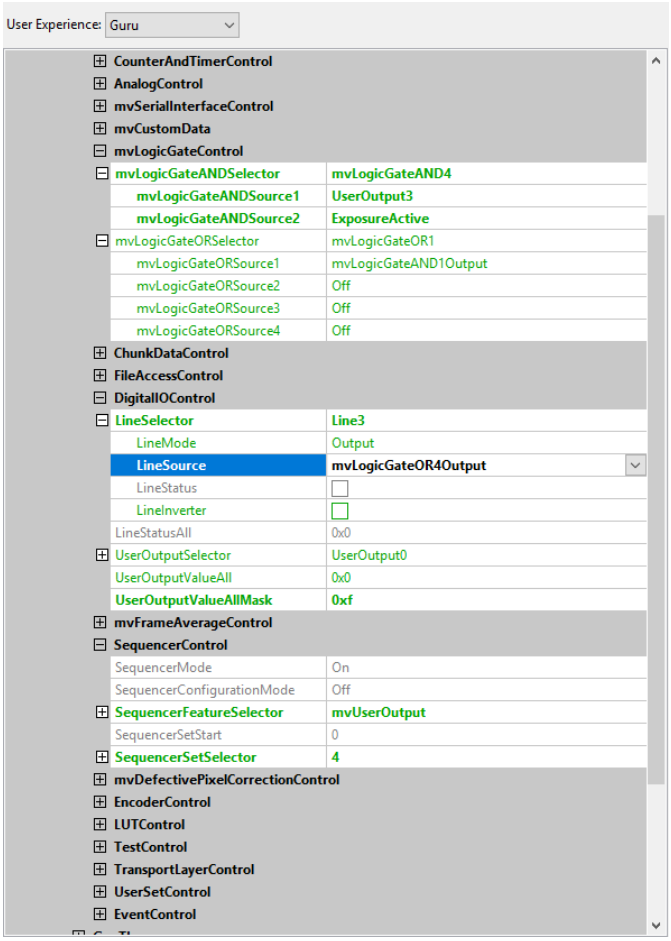


Figure 8: wxPropView - mvLogicGateControl

This produces the following output on the output lines:

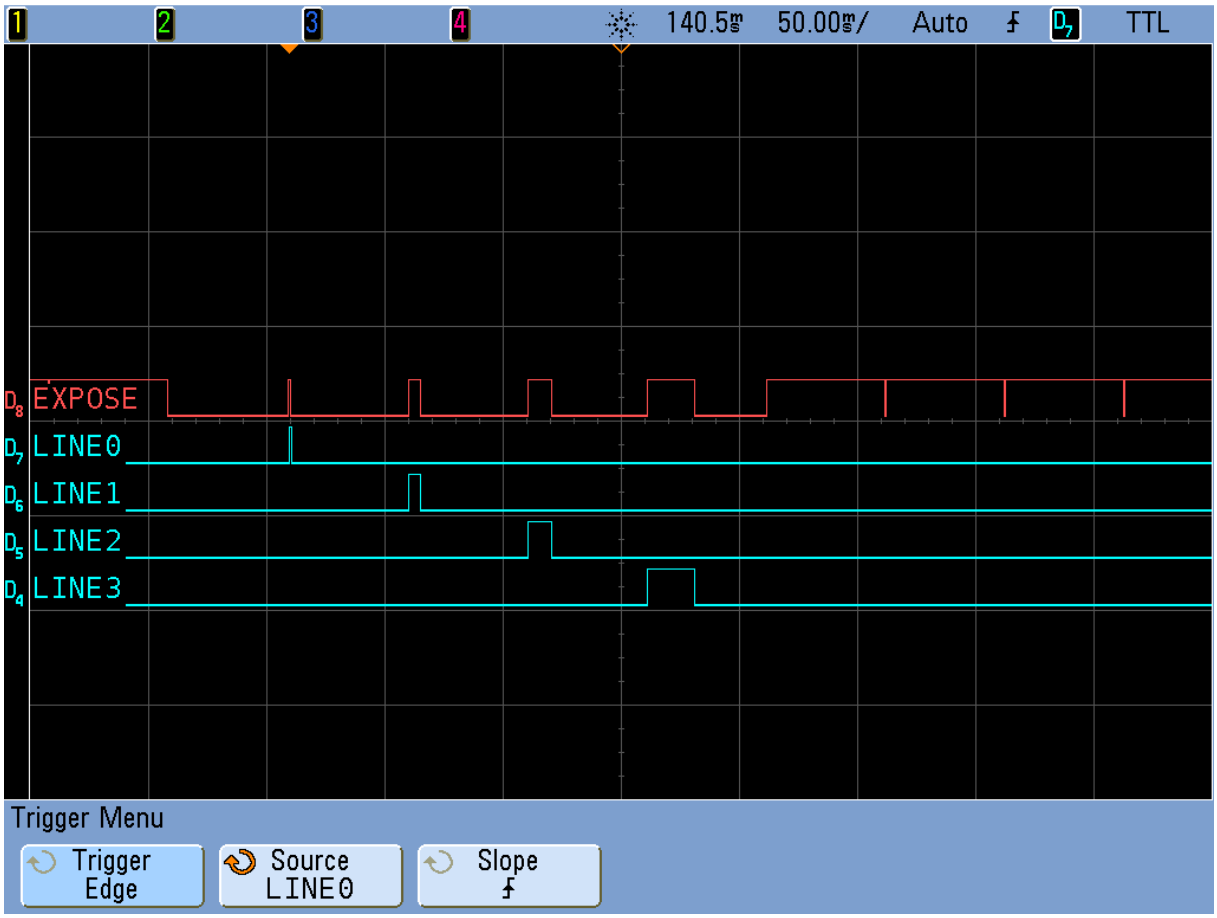


Figure 9: UserOutputs via mvLogicGateControl on output lines

These signals could be used as flash signals for separate sequencer sets.

You can program this as follows:

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

GenICam::DigitalIOControl dio = new GenICam::DigitalIOControl(pDev);
dio.userOutputValueAllMask.write( 0xF );
dio.userOutputValueAll.write( 0x1 ); // 0x2, 0x4, 0x8, 0x0

GenICam::mvLogicGateControl mvlgc = new GenICam::mvLogicGateControl(pDev);
mvlgc.mvLogicGateANDSelector.writeS("mvLogicGateAND1");
mvlgc.mvLogicGateANDSource1.writeS("UserOutput0");
mvlgc.mvLogicGateANDSource2.writeS("ExposureActive");
mvlgc.mvLogicGateORSelector.writeS("mvLogicGateOR1");
mvlgc.mvLogicGateORSource1.writeS("mvLogicGateAND1Output");
mvlgc.mvLogicGateORSource2.writeS("Off");
mvlgc.mvLogicGateORSource3.writeS("Off");
mvlgc.mvLogicGateORSource4.writeS("Off");

dio.lineSelector.writeS("Line0");
dio.lineSource.writeS("mvLogicGateOR1Output");

// To output the UserOutputs directly on the output lines would be like this:
// dio.lineSource.writeS("UserOutput0");
```

1.19.1.4.3 Using the Sequencer Control wizard

Since

mvIMPACT Acquire 2.18.0

wxPropView (p. 139) offers a wizard for the **Sequencer Control** (p. 157) usage:

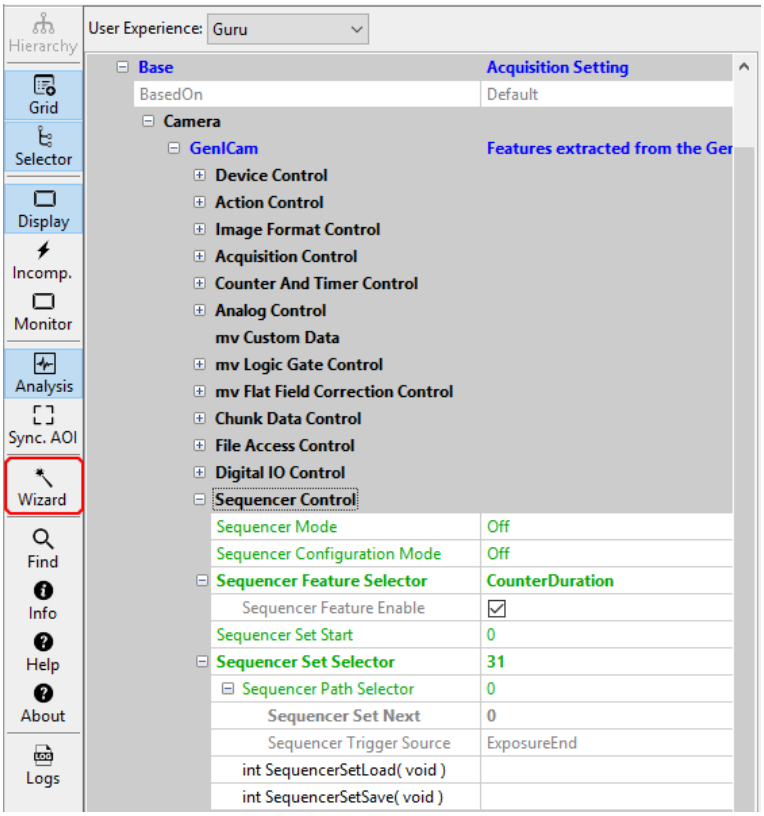


Figure 10: wxPropView - Wizard button

The wizard can be used to get a comfortable overview about the settings of the sequence sets and to create and set a sequence sets in a much easier way:

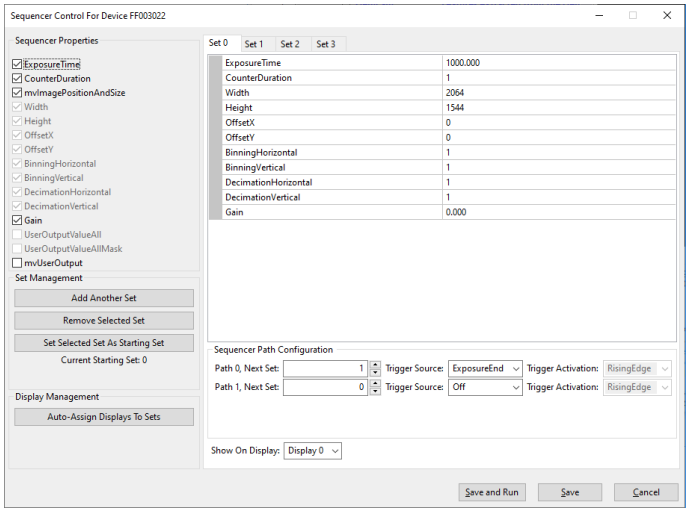


Figure 11: wxPropView - Sequencer Control wizard

Just

- select the desired properties,
- select the desired **Set** tabs,
- set the properties in the table directly, and
- **"Auto-Assign Displays To Sets"**, if you like to show each sequence set in a different display.

Do not forget to save the settings at the end.

1.19.1.4.4 Working with sequencer paths

Since

Firmware version 2.28.0

It is possible to define sets with a maximum of two active paths. The following diagram shows that two paths are defined in **"Set 0"**. **"Path 0"** (*"SequencePathSelector = 0"*) is the "standard" path that in this configuration loops and **"Path 1"** (*"SequencePathSelector = 1"*) will jump to **"Set 1"** after it is activated via a **"RisingEdge"** (*"SequencerTriggerActivation = RisingEdge"*) signal at **"UserOutput0"** (*"SequencerTriggerSource = 0"*). **"UserOutput0"** can be connected, for example, to one of the digital input lines of the camera:

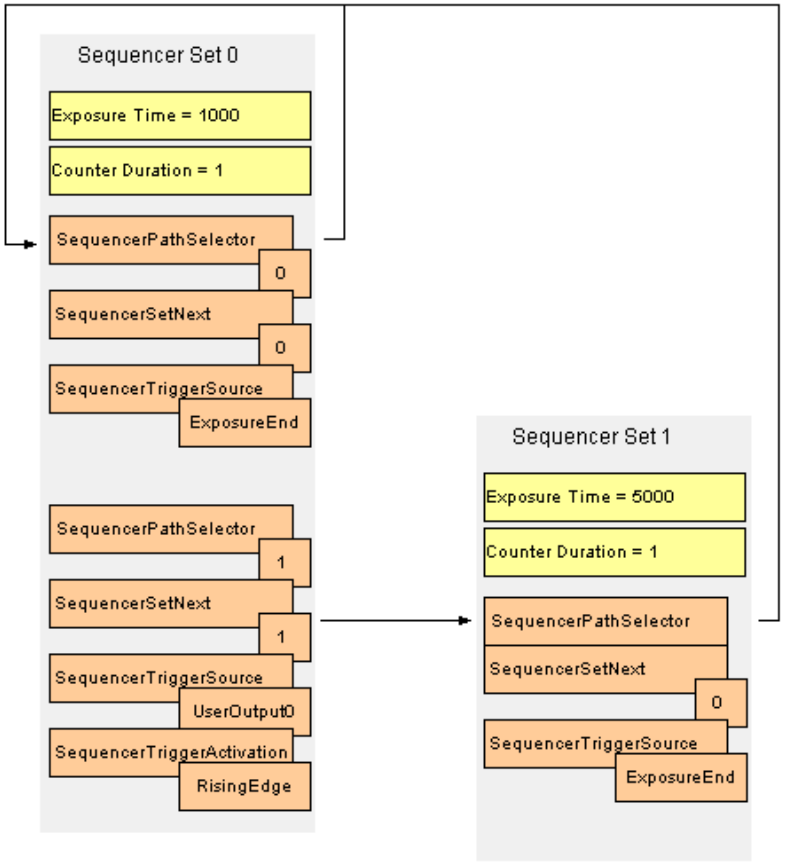


Figure 12: Working diagram of a sample with sequence paths

There are some specifications concerning the sequencer path feature:

- A path is inactive as soon as the **"SequencerTriggerSource"** is **Off**.
- If none of the paths are triggered or both parts are inactive, the sequencer will remain in the current set.
- If both paths were triggered, the path with the trigger that happened first, will be followed.
- As the next sequencer set parameters (like *ExposureTime*) need to be prepared beforehand, the set sequence might not seem straight forward at first glance. The sequencer will always need one frame to switch to the new set; this frame will be the already prepared set.

1.19.1.4.5 Programming a sequence with paths using the Sequencer Control

First the sequencer has to be configured.

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

GenICam::SequencerControl sc = new GenICam::SequencerControl( pDev );
GenICam::AcquisitionControl ac = new GenICam::AcquisitionControl( pDev );
TDMR_ERROR result = DMR_NO_ERROR;

// general sequencer settings
sc.sequencerMode.writeS( "Off" );
sc.sequencerConfigurationMode.writeS( "On" );
sc.sequencerFeatureSelector.writeS( "ExposureTime" );
sc.sequencerFeatureEnable.write( bTrue );
sc.sequencerFeatureSelector.writeS( "CounterDuration" );
sc.sequencerFeatureEnable.write( bFalse );
sc.sequencerFeatureSelector.writeS( "Gain" );
sc.sequencerFeatureEnable.write( bFalse );
sc.sequencerSetStart.write( 0 );

// set0
sc.sequencerSetSelector.write( 0LL );
ac.exposureTime.write( 1000 );
// set0 path0
sc.sequencerPathSelector.write( 0LL );
sc.sequencerTriggerSource.writeS( "ExposureEnd" );
sc.sequencerSetNext.write( 0LL );
// set0 path1
sc.sequencerPathSelector.write( 1LL );
sc.sequencerTriggerSource.writeS( "UserOutput0" );
sc.sequencerTriggerActivation.writeS( "RisingEdge" );
sc.sequencerSetNext.write( 1LL );
// save set
if( ( result = static_cast<TDMR_ERROR>( sc.sequencerSetSave.call() ) ) != DMR_NO_ERROR )
{
  std::cout << "An error was returned while calling function '" << sc.sequencerSetSave.displayName()
            << "' on device " << pDev->serial.read()
            << "(" << pDev->product.read() << "): "
            << ImpactAcquireException::getErrorCodeAsString( result ) << endl;
}

// set1
sc.sequencerSetSelector.write( 1LL );
ac.exposureTime.write( 5000 );
// set1 path0
sc.sequencerPathSelector.write( 0LL );
sc.sequencerTriggerSource.writeS( "ExposureEnd" );
sc.sequencerSetNext.write( 0LL );
// set1 path1
sc.sequencerPathSelector.write( 1LL );
sc.sequencerTriggerSource.writeS( "Off" );
// save set
if( ( result = static_cast<TDMR_ERROR>( sc.sequencerSetSave.call() ) ) != DMR_NO_ERROR )
{
  std::cout << "An error was returned while calling function '" << sc.sequencerSetSave.displayName()
            << "' on device " << pDev->serial.read()
```

```
            << "(" << pDev->product.read() << "): "
            << ImpactAcquireException::getErrorCodeAsString( result ) << endl;
}

// final general sequencer settings
sc.sequencerConfigurationMode.writeS( "Off" );
sc.sequencerMode.writeS( "On" );
```

Then it can later be triggered during runtime.

```
GenICam::DigitalIOControl dic = new GenICam::DigitalIOControl( pDev );
dic.userOutputSelector.write( 0 );
dic.userOutputValue.write( bTrue );
dic.userOutputValue.write( bFalse );
```

This will set an internal event that will cause the sequencer to use set0-path1 at the next possible time, i.e. the next time we are in set0.

There is a C++ example called **GenICamSequencerUsageWithPaths** that shows how to control the sequencer with paths from an application. It can be found in the `Examples` section of the **C++ interface documentation (mvIMPACT Acquire C++ API)** (p. 177)

1.19.1.5 Generating very long exposure times

1.19.1.5.1 Basics

At the moment the exposure time is limited to a maximum of 1 up to 20 seconds depending on certain internal sensor register restrictions. So each device might report a different maximum exposure time.

Since

Firmware version 2.28.0

Firmware version 2.28 did contain a major overhaul here so updating to at least this version can result in a much higher maximum exposure time. However, current sensor controllers can be configured to use even longer exposure times if needed using one of the devices timers to create an external exposure signal that can be fed back into the sensor. This use case will explain how this can be done.

This approach of setting up long exposure times requires the sensor of the camera to allow the configuration of an external signal to define the length of the exposure time, so only devices offering the `ExposureMode` `Trigger↩Width` can be used for this setup.

Note

The maximum exposure time in microseconds that can be achieved in this configuration is the maximum value offered by the timer used.

With **GenICam** (p. 213) compliant devices that support all the needed features the setup is roughly like this:

1. Select *"Setting -> Base -> Camera -> GenICam -> Counter And Timer Control -> Timer Selector -> Timer 1"* and .
2. set *"Timer Trigger Source"* = *"UserOutput0"*.
3. set *"Timer Trigger Activation"* = *"RisingEdge"*.
   I.e. a rising edge on **UserOutput0** will start **Timer1**.

4. Then set the *"Timer Duration"* property to the desired exposure time in *us*.
5. In *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"* set the *Trigger* Selector = *"FrameStart"*. I.e. the acquisition of one frame will start when
6. **Timer1** is active: *"Trigger Source"* = *"Timer1Active"*.
7. **Exposure** time will be the trigger width: *"Exposure Mode"* = *"TriggerWidth"*.

The following diagram illustrates all the signals involved in this configuration:

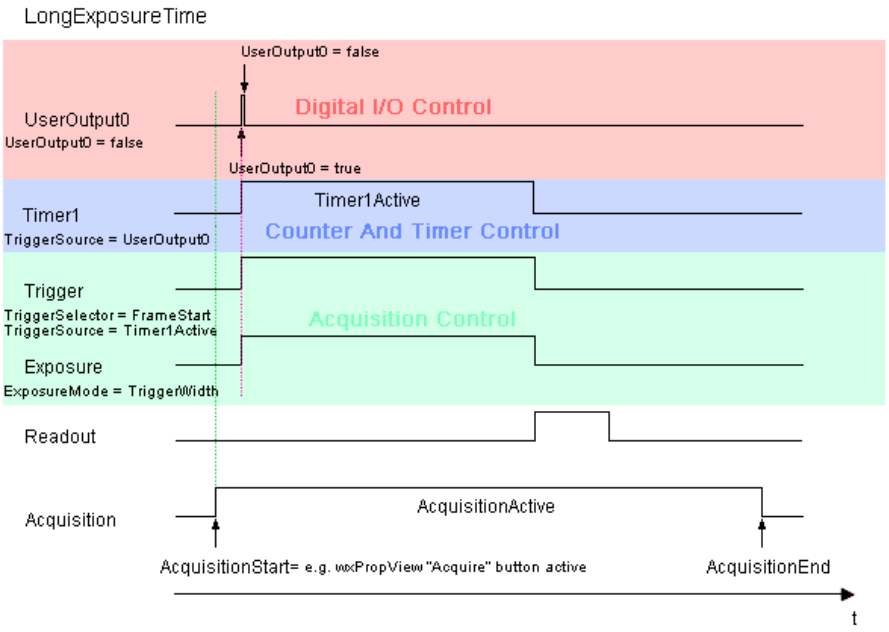


Figure 1: Long exposure times using GenICam

To start the acquisition of one frame a rising edge must be detected on **UserOutput0** in this example but other configurations are possible as well.

1.19.1.5.2 Setting up the device

The easiest way to define a long exposure time would be by using a single timer. The length of the timer active signal is then used as trigger signal and the sensor is configured to expose while the trigger signal is active. This allows to define exposure time with micro-second precision up the the maximum value of the timer register. With a 32 bit timer register this results in a maximum exposure time of roughly 4295 seconds (so roughly 71.5 minutes). When writing code e.g. in C# this could look like this:

```
private static void configureDevice(Device pDev, int exposureSec, GenICam.DigitalIOControl ioc)
{
  try
  {
    // establish access to the CounterAndTimerControl interface
    GenICam.CounterAndTimerControl ctc = new mv.impact.acquire.GenICam.CounterAndTimerControl(pDev);
    // set TimerSelector to Timer1 and TimerTriggerSource to UserOutput0
    ctc.timerSelector.writeS("Timer1");
    ctc.timerTriggerSource.writeS("UserOutput0");
    ctc.timerTriggerActivation.writeS("RisingEdge");

    // Set timer duration for Timer1 to value from user input
    ctc.timerDuration.write(exposureSec * 1000000);
```

```
    // set userOutputSelector to UserOutput0 and set UserOutput0 to inactive.
    // We will later generate a pulse here to initiate the exposure
    ioc.userOutputSelector.writeS("UserOutput0");
    ioc.userOutputValue.write(TBoolean.bFalse);

    // establish access to the AcquisitionCotrol interface
    GenICam.AcquisitionControl ac = new mv.impact.acquire.GenICam.AcquisitionControl(pDev);
    // set TriggerSelector to FrameStart and try to set ExposureMode to TriggerWidth
    ac.triggerSelector.writeS("FrameStart");
    // set TriggerSource for FrameStart to Timer1Active and activate TriggerMode
    ac.triggerSource.writeS("Timer1Active");
    ac.triggerMode.writeS("On");

    // expose as long as we have a high level from Timer1
    ac.exposureMode.writeS("TriggerWidth");
  }
  catch (Exception e)
  {
    Console.WriteLine("ERROR: Selected device does not support all features needed for
    this long time exposure approach: {0}, terminating...", e.Message);
    System.Environment.Exit(1);
  }
}
```

Note

Make sure that you adjust the `ImageRequestTimeout_ms` either to 0 (infinite)(this is the default value) or to a reasonable value that is larger than the actual exposure time in order not to end up with timeouts resulting from the buffer timeout being smaller than the actual time needed for exposing, transferring and capturing the image:

```
ImageRequestTimeout_ms = 0 # or reasonable value
```

See also

Counter And Timer Control (p. 148)

Digital I/O Control (p. 155)

Acquisition Control (p. 144)

1.19.1.6 Working with multiple AOIs (mv Multi Area Mode)

Since

mvIMPACT Acquire 2.18.3

1.19.1.6.1 Introduction

A special feature of Pregius sensors (a.k.a. IMX) from Sony is the possibility to define multiple AOIs (Areas of Interests - a.k.a. ROI - Regions of Interests) and to transfer them at the same time. Because many applications just need one or several specific parts in an image to be checked, this functionality can increase the frame rate.

Once activated, the *"mv Multi Area Mode"* allows you, depending on the sensor, to define up to eight AOIs (*mvArea0* to *mvArea7*) in one image. There are several parameters in combination with the AOIs which are illustrated in the following figure:

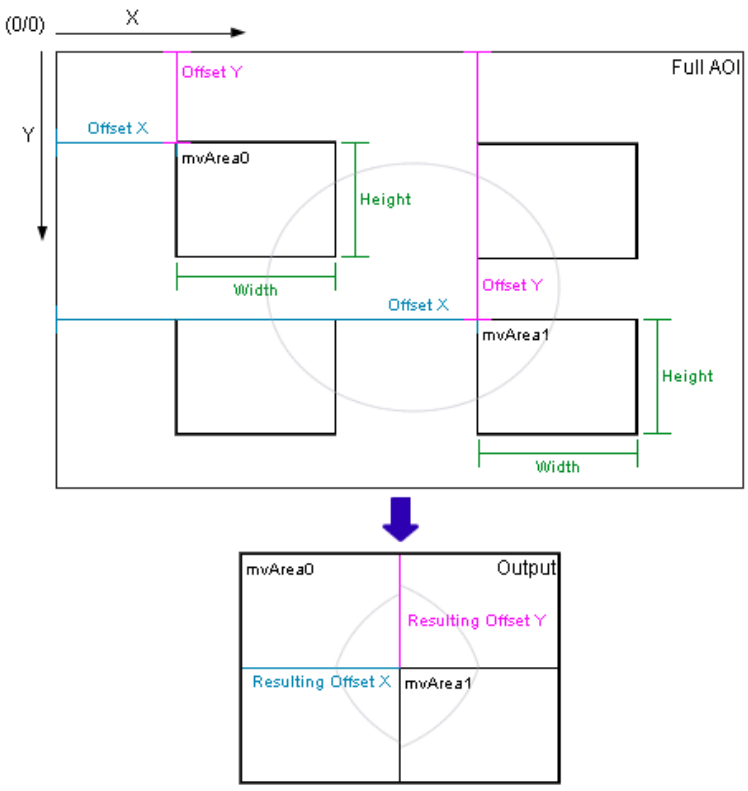


Figure 1: Multiple AOIs principle

The *"Resulting Offset X"* and *"Resulting Offset Y"* indicates the starting point of the specific AOI in the output image. To complete the rectangular output image, the "missing" areas are filled up with the image data horizontally and vertically. We recommend to use the wizard as a starting point - the wizard provides a live preview of the final merged output image.

1.19.1.6.2 Using wxPropView

To create multiple AOIs with **wxPropView** (p. 139), you have to do the following step:

1. Start **wxPropView** (p. 139) and
2. connect to the camera.
3. Then change in *"Setting -> Base -> Camera -> GenICam -> Image Format Control"* the **"mv Multi Area Mode"** to **"mvMultiAreasCombined"**. Afterwards, **"mv Area Selector"** is available.
4. Now, select the area a.k.a. AOI you want to create via *"mv Area Selector"*, e.g. **"mvArea3"** and
5. set the parameters **"mv Area Width"**, **"mv Area Height"**, **"mv Area Offset X"**, and **"mv Area Offset Y"** to your needs.
6. Activate the area a.k.a. AOI by checking the box of **"mv Area Enable"**.

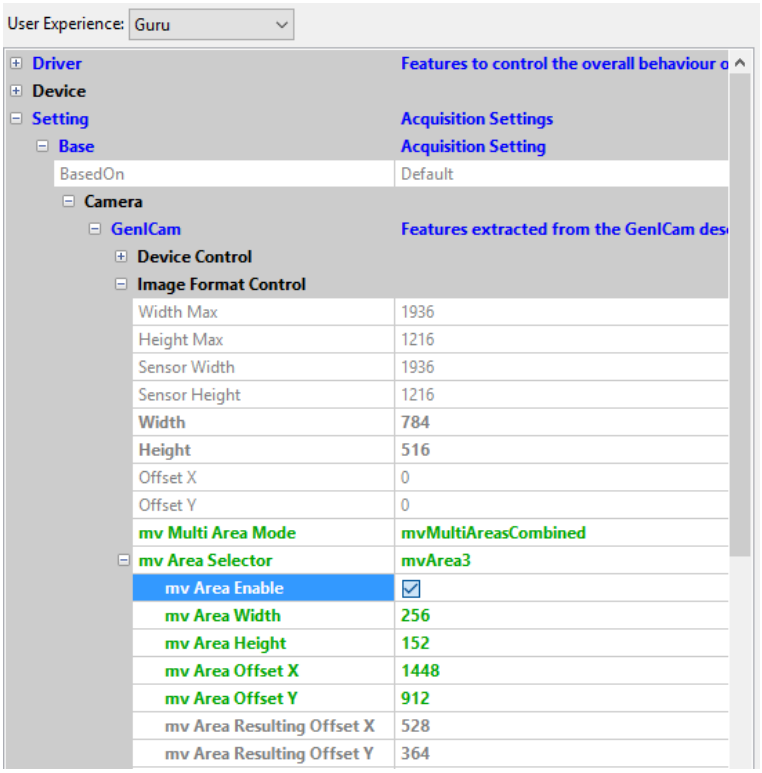


Figure 2: wxPropView - Multiple AOIs

7. Finally, start the acquisition by clicking the button *"Acquire"*.

1.19.1.6.2.1 Using the Multi AOI wizard

Since

mvIMPACT Acquire 2.19.0

wxPropView (p. 139) offers a wizard for the Multi AOI usage:

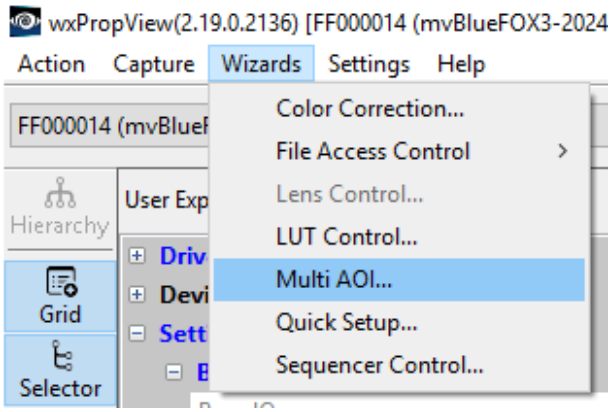


Figure 3: wxPropView - Wizard menu

The wizard can be used to get a comfortable overview about the settings of the AOIs and to create and set the AOIs in a much easier way:

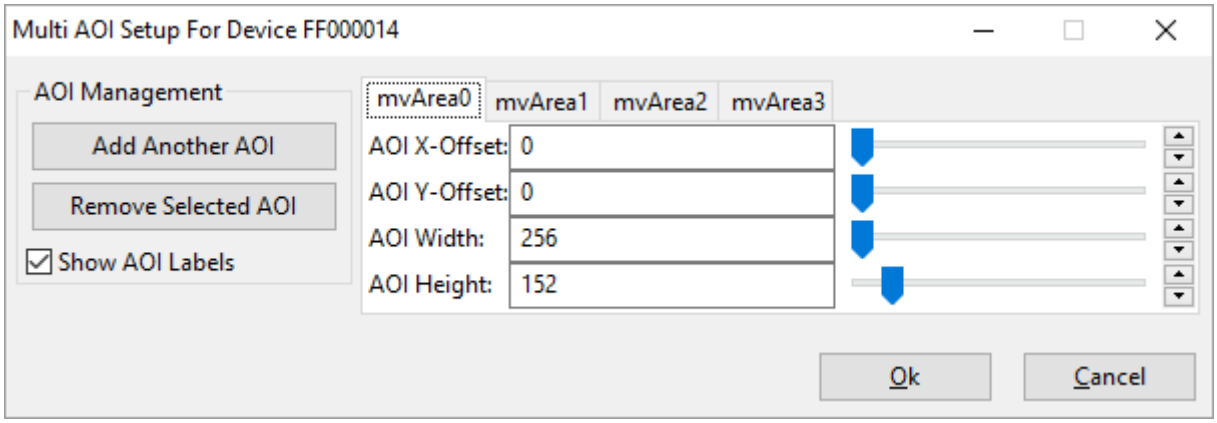


Figure 4: wxPropView - Multi AOI wizard

Just

- select the desired **mvArea** tabs,
- set the properties like offset, width, and height in the table directly, and
- confirm the changes at the end using the **Ok** button.

The live image shows the created AOIs and the merged or "missing" areas which are used to get the final rectangular output image.

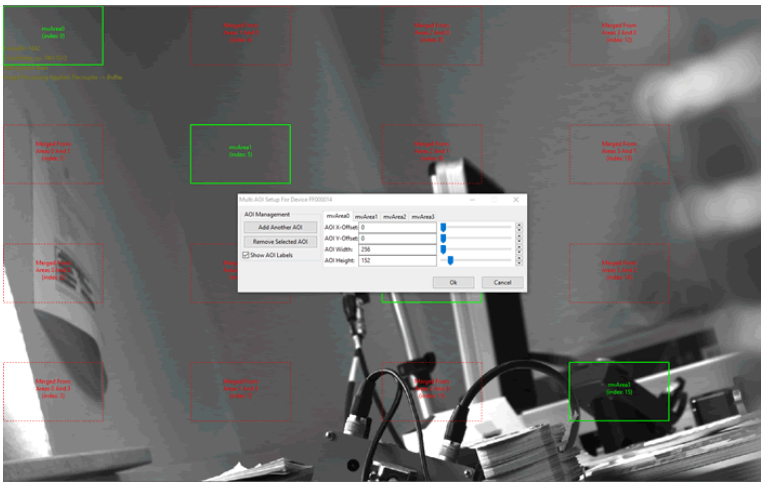


Figure 5: wxPropView - Multi AOI wizard - Live image

1.19.1.6.3 Programming multiple AOIs

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...
    GenICam::ImageFormatControl ifc( pDev );
    ifc.mvMultiAreaMode.writeS( "mvMultiAreasCombined" );
    ifc.mvAreaSelector.writeS( "mvArea0" );
    ifc.mvAreaEnable.write( bTrue );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 0 );
    ifc.mvAreaWidth.write( 256 );
    ifc.mvAreaHeight.write( 152 );
    ifc.mvAreaSelector.writeS( "mvArea1" );
    ifc.mvAreaEnable.write( bFalse );
    ifc.mvAreaSelector.writeS( "mvArea2" );
    ifc.mvAreaEnable.write( bFalse );
    ifc.mvAreaSelector.writeS( "mvArea3" );
    ifc.mvAreaEnable.write( bTrue );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 0 );
    ifc.mvAreaWidth.write( 256 );
    ifc.mvAreaHeight.write( 152 );
    ifc.mvAreaOffsetX.write( 1448 );
    ifc.mvAreaOffsetY.write( 912 );
...
```

1.19.1.7 Working with burst mode buffer

If you want to acquire a number of images at sensor's maximum frame rate while at the same time the image transfer should be at a lower frame rate, you can use the internal memory of the mvBlueCOUGAR-X .

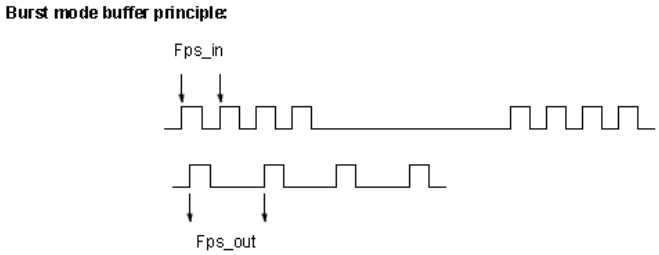


Figure 1: Principle of burst mode buffering of images

Note

The maximum buffer size can be found in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> mv Acquisition Memory Max Frame Count"*.

To create a burst mode buffering of images, please follow these steps:

1. Set image acquisition parameter (*"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> mv Acquisition Frame Rate Limit Mode"*) to **"mvDeviceMaxSensorThroughput"**.
2. Finally, set the acquisition parameter *"mv Acquisition Frame Rate Enable"* to **"Off"**.

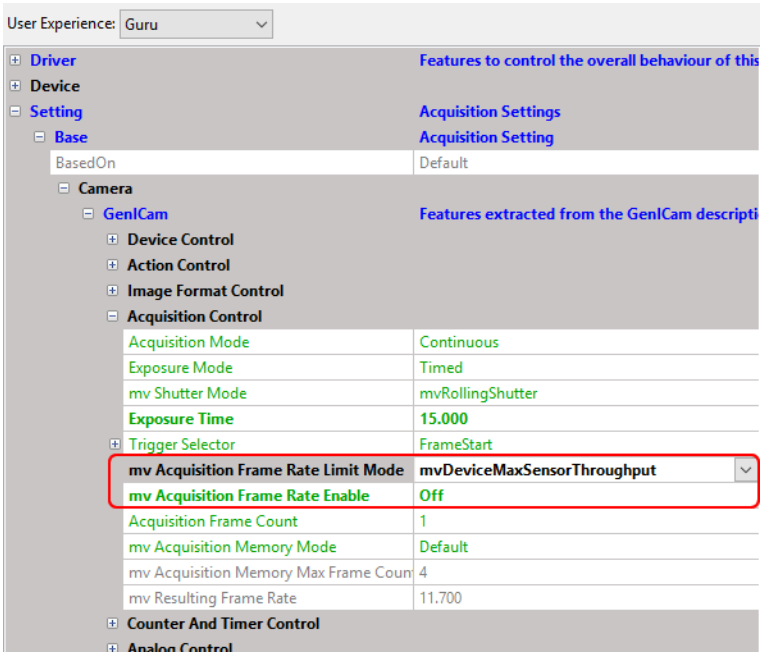


Figure 2: wxPropView - Setting the bandwidth using **"mv Acquisition Frame Rate Limit Mode"**

Alternatively, you can set the burst mode via the desired input frames and the desired output bandwidth:

1. Set image acquisition parameters to the desired input frames per second value (*"Setting -> Base -> Camera -> GenICam -> Acquisition Control"*).

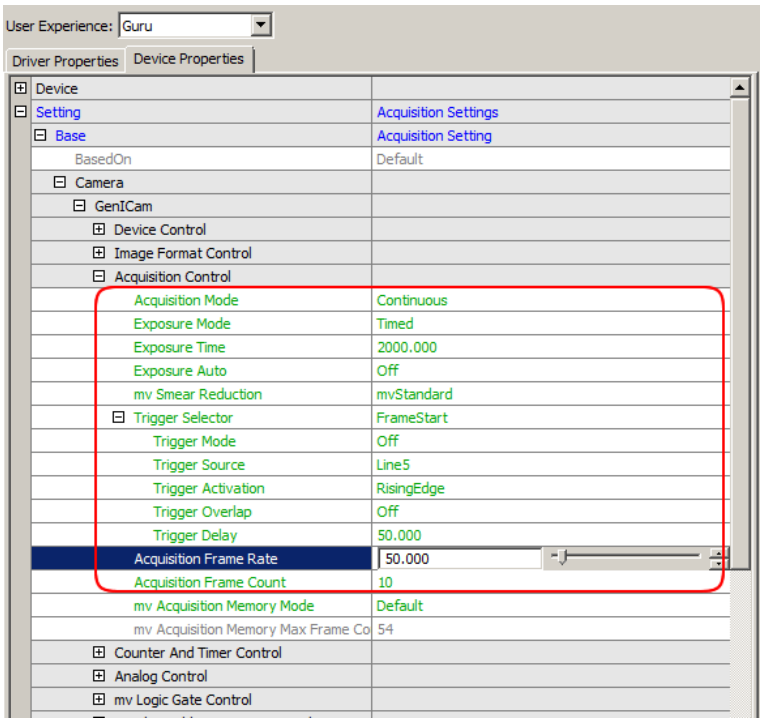


Figure 3: wxPropView - Setting the **"Acquisition Frame Rate"**

2. Set bandwidth control to the desired MByte/s out value in *"Setting -> Base -> Camera -> GenICam -> Device Control -> Device Link Selector -> Device Link Troughput Limit Mode"* to **"On"** and
3. set the desired **"Device Link Troughput Limit"** in Bits per second (Bps).

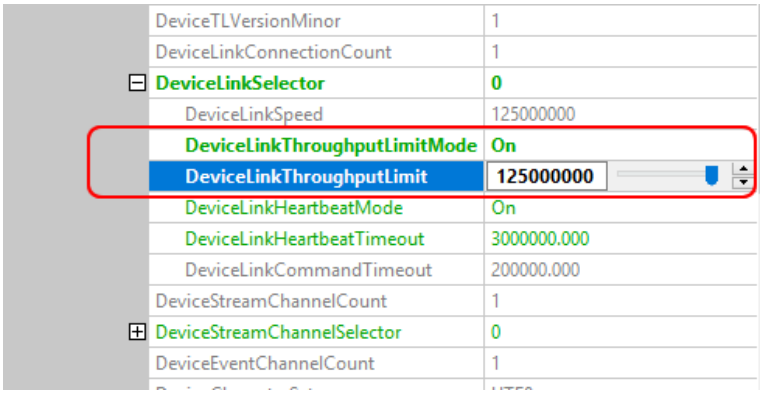


Figure 4: wxPropView - Setting the bandwidth using **"Device Link Troughput Limit Mode"**

Now, the camera will buffer burst number of images in internal memory and readout at frames per second out.

See also

Limiting the bandwidth of the imaging device (p. 343)

1.19.1.7.1 Triggered frame burst mode

With the **triggerSelector** "FrameBurstStart", you can also start a frame burst acquisition by a trigger. A defined number of images (**"AcquisitionBurstFrameCount"**) will be acquired directly one after the other. With the **"mv Acquisition Frame Rate Limit Mode"** set to **mvDeviceMaxSensorThroughput** , the there won't be hardly any gap between these images.

As shown in figure 5, **"FrameBurstStart"** can be trigger by a software trigger, too.

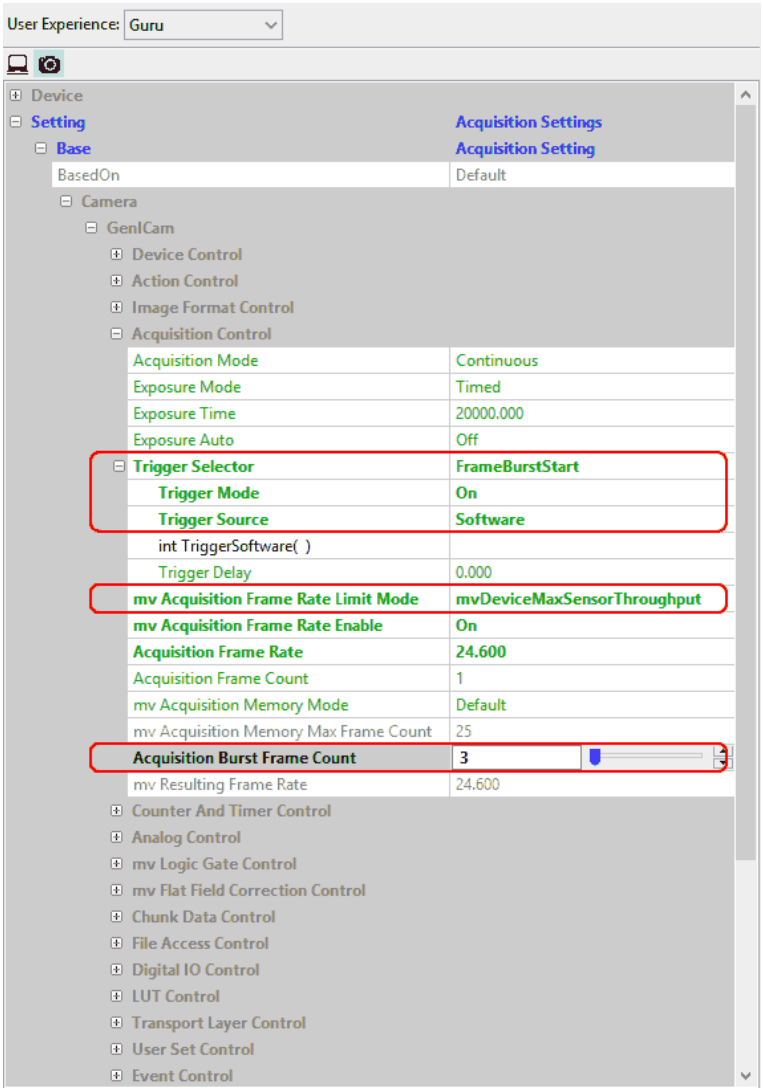


Figure 5: wxPropView - Setting the frame burst mode triggered by software

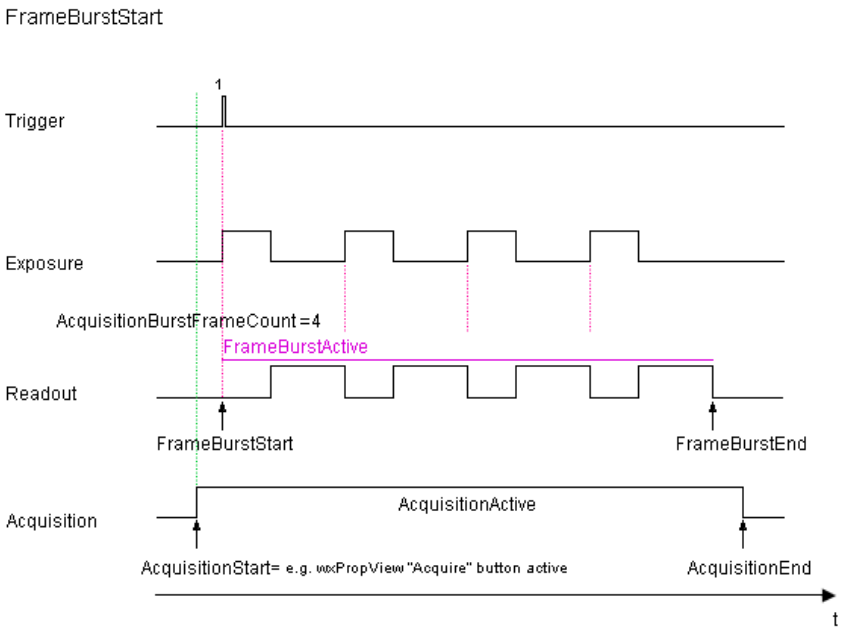


Figure 6: Principle of FrameBurstStart

1.19.1.8 Using the SmartFrameRecall feature

Since

mvIMPACT Acquire 2.18.0

1.19.1.8.1 Introduction

The **SmartFrameRecall** is a new, FPGA based smart feature which takes the data handling of industrial cameras to a new level.

So far, the entire data amount has to transferred to the host PC whereby the packetizer in the camera split the data packages and distributed them to the two Gigabit Ethernet lines. On the host PC, the data was merged again, shrank, and this AOI was possibly processed (Figure 1).

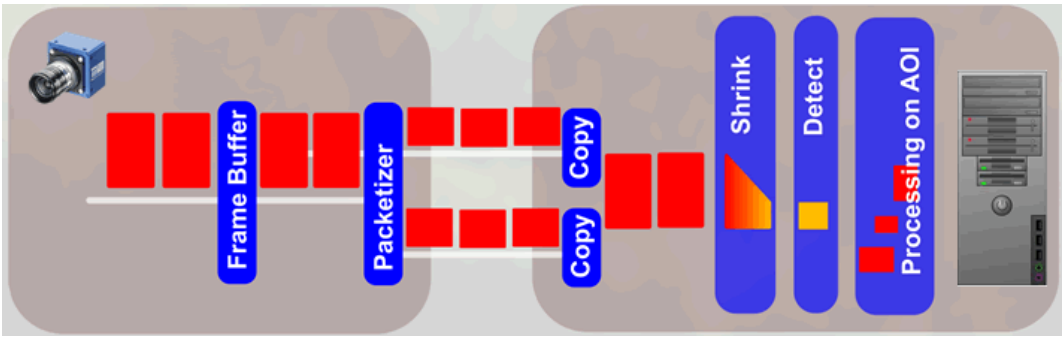


Figure 1: Data handling so far

This procedure has several disadvantages:

- Both data lines (in case of Dual GigE cameras for example) for each camera are required which
  - leads to high cabling efforts.
- A high end PC is needed to process the data which
  - leads to high power consumption.
- In USB 3 multi-camera solutions (depending on the resolution) each camera requires a separate connection line to the host PC which
  - limits the cabling possibilities, the possible distances, and makes an installation more complex (without the possibilities to use hubs, for example).
- Last but not least, the frame rates a limited to the bandwidth.

The **SmartFrameRecall** is a new data handling approach which buffers the hi-res images in the camera and only transfers thumbnails. You or your software decides on the host PC which AOI should be sent to the host PC (Figure 2).

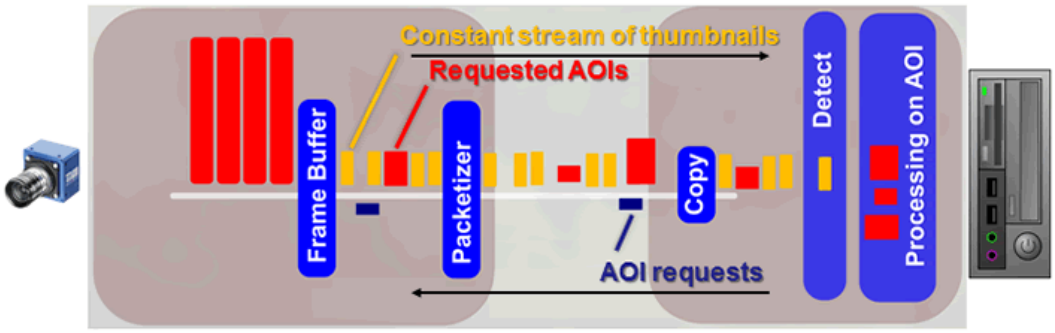


Figure 2: SmartFrameRecall working method

This approach allows

- higher bandwidths,
- less CPU load and power consumption,
- higher frame rates, and
- less complex cabling.

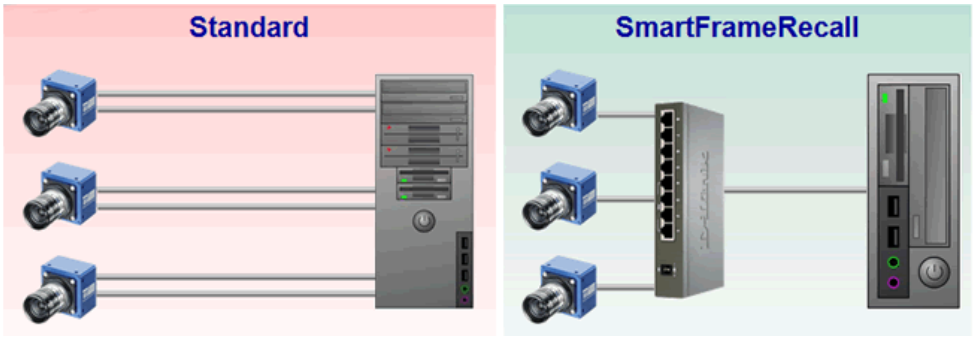


Figure 3: Connection advantages in combination with SmartFrameRecall

1.19.1.8.2 Implementing a SmartFrameRecall application

First of all, clarify if the **SmartFrameRecall** makes sense to be used in your application:

- Taking a short look on the thumbnails, could you specify which frames don't interest you at all or which portions of the frame suffice to you for further processing?

If you can, your application can do so too, and then **SmartFrameRecall** is the right framework for you. To use the **SmartFrameRecall** follow these steps:

- Activate the **Chunk Data Control:**

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...
GenICam::ChunkDataControl cdc( pDev );
cdc.chunkModeActive.write( bTrue );
cdc.chunkSelector.writeS( "Image" );
cdc.chunkEnable.write( bTrue );
cdc.chunkSelector.writeS( "mvCustomIdentifier" );
cdc.chunkEnable.write( bTrue );
...
```

It is necessary to activate the **chunk data** (p. 154) that your application can easily distinguish frames belonging to the normal stream from the ones requested by the host application.

- Reduce the size of the streamed images. This can reduce the size using **Decimation**, both horizontally and vertically. E.g. setting decimation to 16, a normal image will only consume 16∗16 thus 1/256th of the bandwidth:

```
...
GenICam::ImageFormatControl ifc( pDev );
ifc.decimationHorizontal.write( 16 );
ifc.decimationVertical.write( 16 );
...
```

- Make sure that the resulting image width is a multiple of 8! If this is not the case the SmartFrameRecall feature cannot be activated.
- Activate the **SmartFrameRecall** feature:

```
...

GenICam::AcquisitionControl ac( pDev );
ac.mvSmartFrameRecallEnable.write( bTrue );
...
```

This will configure the devices internal memory to store each frame (that gets transmitted to the host) in full resolution. These images can be requested by an application when needed. As soon as the memory is full, the oldest one will be removed from the memory whenever a new one becomes ready (FIFO).

- Analyze the images of the reduced data stream.
- If necessary, request the desired image in full resolution:

```
...
struct ThreadParameter
{
    Device* pDev;
    GenICam::CustomCommandGenerator ccg;
    ...
}
...
unsigned int DMR_CALL liveThread( void* pData )
{
    ThreadParameter* pThreadParameter = reinterpret_cast<ThreadParameter*>( pData );
    ...
    pThreadParameter->ccg.requestTransmission( pRequest, x, y, w, h, rtmFullResolution, cnt );
    ...
}
...
```

The last parameter of **requestTransmission** will be written into the **chunkmvCustomIdentifier** and your application can recognize the request.

- Finally do you analysis/processing with the requested image in full resolution.

We provide

- a basic **SmartFrameRecall** C++ sample called **GenICamSmartFrameRecallUsage.cpp** which will be installed together with **"mvGenTL Acquire"** driver.
  You find the sample's description in the online manual of the mvIMPACT Acquire API C++: https://www.matrix-vision.com/manuals/SDK_CPP/GenICamSmartFrameRecallUsage_8cpp-example.html
- a advanced **SmartFrameRecall** C# sample called **SmartFrameRecall.cs** which will be installed together with **"mvGenTL Acquire"** driver.
  You find the sample's description in the online manual of the mvIMPACT Acquire API .NET: https://www.matrix-vision.com/manuals/SDK_NET/SmartFrameRecall_8cs-example.html

1.19.1.9 Using The Linescan mode

Both CMOS sensors from e2v offer a line scan mode. One (gray scale sensor) or two lines (in terms of color sensor) can be selected to be read out of the full line height of 1024 or 1200 lines. This or these lines are grouped to a pseudo frame of selectable height in the internal buffer of the camera.

Complete instructions for using the line scan mode are provided here:

- **sensor description of line scan mode (-x02e)** (p. 556)
- **sensor description of line scan mode (-x02eGE)** (p. 563)
- **sensor description of line scan mode (-x04e)** (p. 569)

1.19.1.9.1 System requirements

- mvBlueCOUGAR-X
  - **"firmware version"** at least **"1.6.32.0"**
- mvBlueFOX3
  - **"firmware version"** at least **"1.6.139.0"**

1.19.1.9.2 Initial situation and settings

Generally, line scan cameras are suitable for inspections of moving, continuous materials. In order that the line scan camera acquires the line at the right time, an **incremental encoder** (p. 348), for example, at a conveyor belt triggers the line scan camera. Normally, an **incremental encoder** (p. 348) does this using a specific frequency like 1:1 which means that there is a signal for every line. However, during the adjustment of a line trigger application or while choosing a specific incremental encoder you have to keep the specific frequency in mind.

Note

Using **timers and counters** (p. 148) it is possible to skip trigger signals.

In line scan mode, the camera adds the single lines to one image of the height of max. 1024 or 1200 lines (according to the used sensor). The images are provided with no gap.

Note

While using the line scan mode with a gray scale sensor, one trigger signal will lead to one acquired image line. Using a color sensor, one trigger signal will lead to two acquired image lines.

Due to aberrations at the edges of the lens, you should set an offset in the direction of Y (**"Offset Y"**, see the following figure), generally around the half of the sensor's height (a.k.a. sensor's Y center). With Offset Y you can adjust the scan line in the direction of motion.

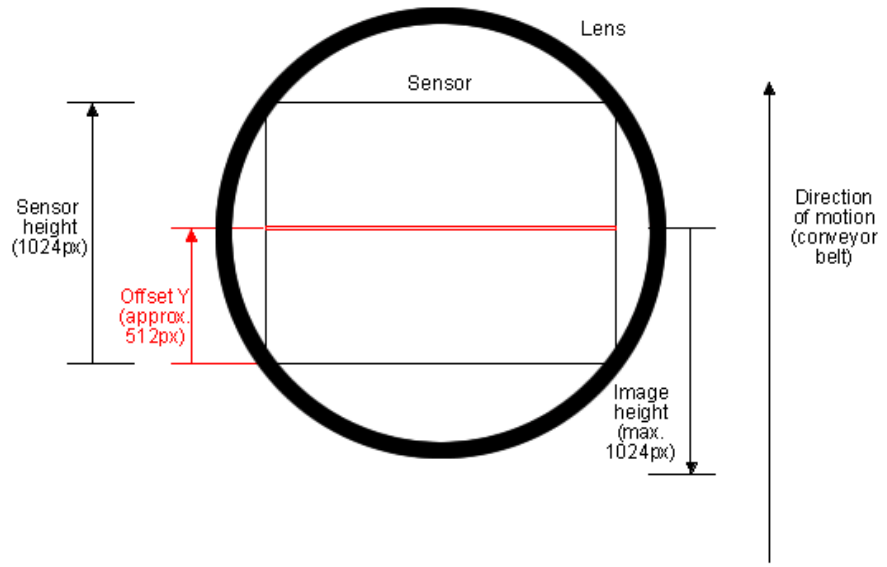


Figure 1: Sensor's view and settings with a sensor with max. height of 1024 pixels/lines (e.g. -x02e / -1013)

1.19.1.9.2.1 Scenarios

With regards to the external trigger signals provided by an **incremental encoder** (p. 348), there are two possible scenarios:

1. A conveyor belt runs continuously and so does the **incremental encoder** (p. 348), or - like in a reverse vending machine,
2. a single item is analyzed and the conveyor belt and so the **incremental encoder** (p. 348) stops after the inspection and restarts for the next item.

In the first scenario you can use the standard settings of the MATRIX VISION devices. Please have a look at the sample **Triggered line scan acquisition with exposure time of 250 us** (p. 253) which shows how you can set the line scan mode with continuous materials and signals from the encoder. However, it is absolutely required that the external trigger is always present. During a trigger interruption, controlling or communication to the camera is not possible.

In the second scenario, the external trigger stops. If there is a heartbeat functionality in the system (e.g. with GigE Vision), there can be a halt of the system. Please have a look at the sample **Triggered line scan acquisition with a specified number of image blocks and pausing trigger signals** (p. 253) which shows how you can handle pausing trigger signals.

1.19.1.9.3 Sample 1: Triggered linescan acquisition with exposure time of 250 us

This sample will show you how to use the line scan mode of the sensors **-x02e** (p. 556) and **-x02eGE** (p. 563) using an external trigger provided by an **incremental encoder** (p. 348) which sends a **"trigger for every line"** (1:1)

> **Note**
>
> You can also use the sensor **-x04e** (p. 567). However, the sensor is slower due to the higher number of pixels.

In this sample, we chose an exposure time of **"250 us"** and to ease the calculations we used **"1000 px image height"**.

Note

To get suitable image results, it might be necessary to increase the **gain** or the **illumination**.

These settings result in a max. **"frame rate"** of **"2.5 frames per second"**.

To adjust the opto-mechanics (focus, distance, illumination, etc.), you can use the area mode of the sensor. That's a main advantage of an area sensor with line scan mode compared to a line scan camera!

You will need the following pins of the mvBlueCOUGAR-X :

Pin.	Signal (Standard version)	Description
1	GND	Common ground
2	12V .. 24V	Power supply
4	Opto DigIn0 (line4)	The output signal A of the incremental encoder

1.19.1.9.3.1 Setting the application in wxPropView

Summary of our sample:

Property name wxPropView	Setting	**GenICam Control** (p. 140)	Comment
Device Scan Type	`line` scan	Device Control	
Height (in pixels)	`1000`	Image Format Control	
Offset Y (in pixels)	`500`	Image Format Control	
Exposure Time (in microseconds)	`250.000`	Acquisition Control	
Trigger Mode	`On`	Acquisition Control	
Trigger Source	`Line4`	Acquisition Control	
ImageRequestTimeout_ms (in milliseconds)	`0 ms`	-	This is necessary otherwise there will be error counts and no frames are created.

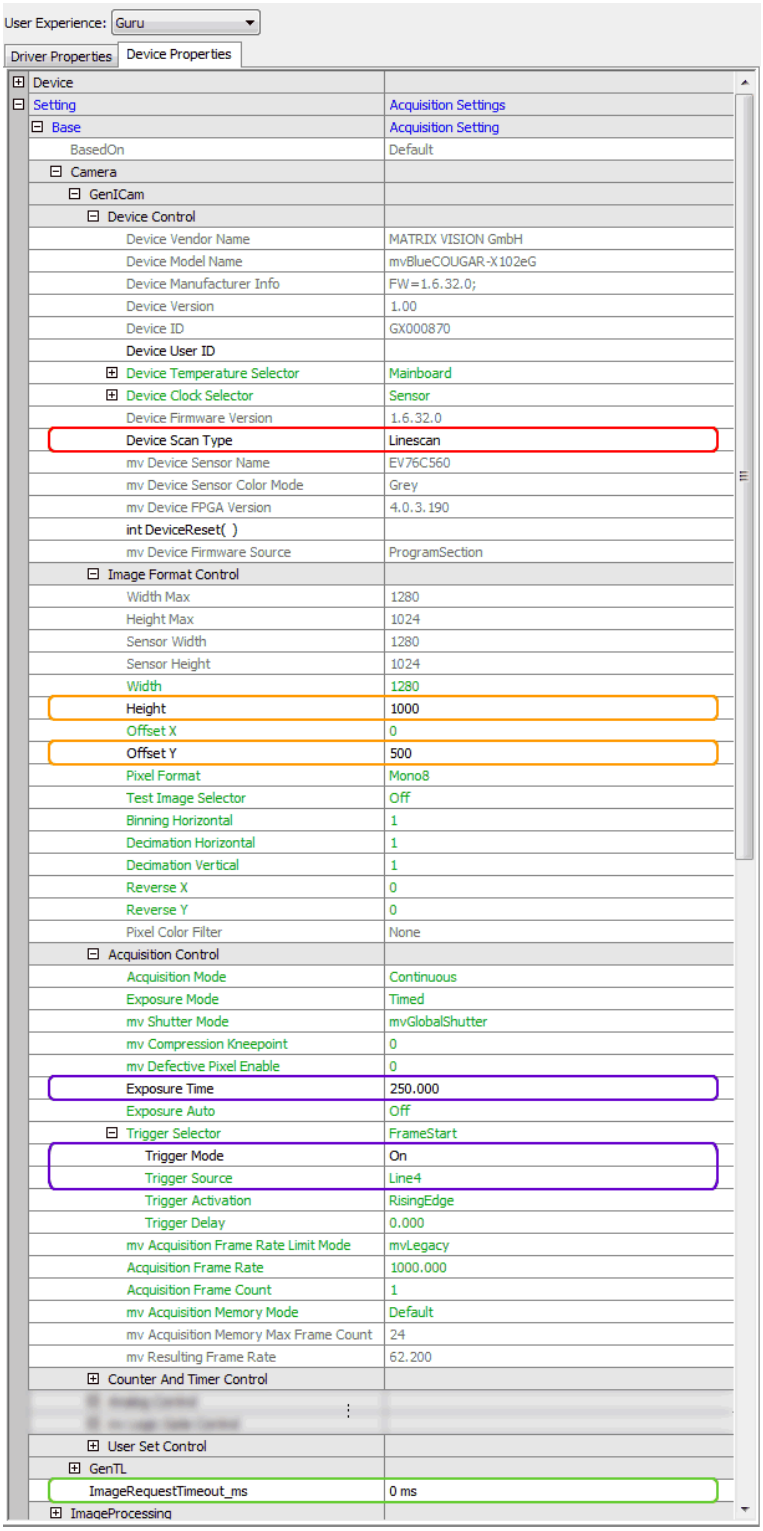


Figure 2: Settings in wxPropView

1.19.1.9.4 Sample 2: Triggered line scan acquisition with a specified number of image blocks and pausing trigger signals

This section will provide you with some information you have to keep in mind while working with pausing triggers and specified number of image blocks.

First of all, using mvBlueCOUGAR-X or mvBlueCOUGAR-XD it is necessary to disable the heartbeat of the GigE Vision control protocol (GVCP) (`"Device Link Heartbeat Mode = On"`) otherwise a paused trigger signal can be misinterpreted as a lost connection:

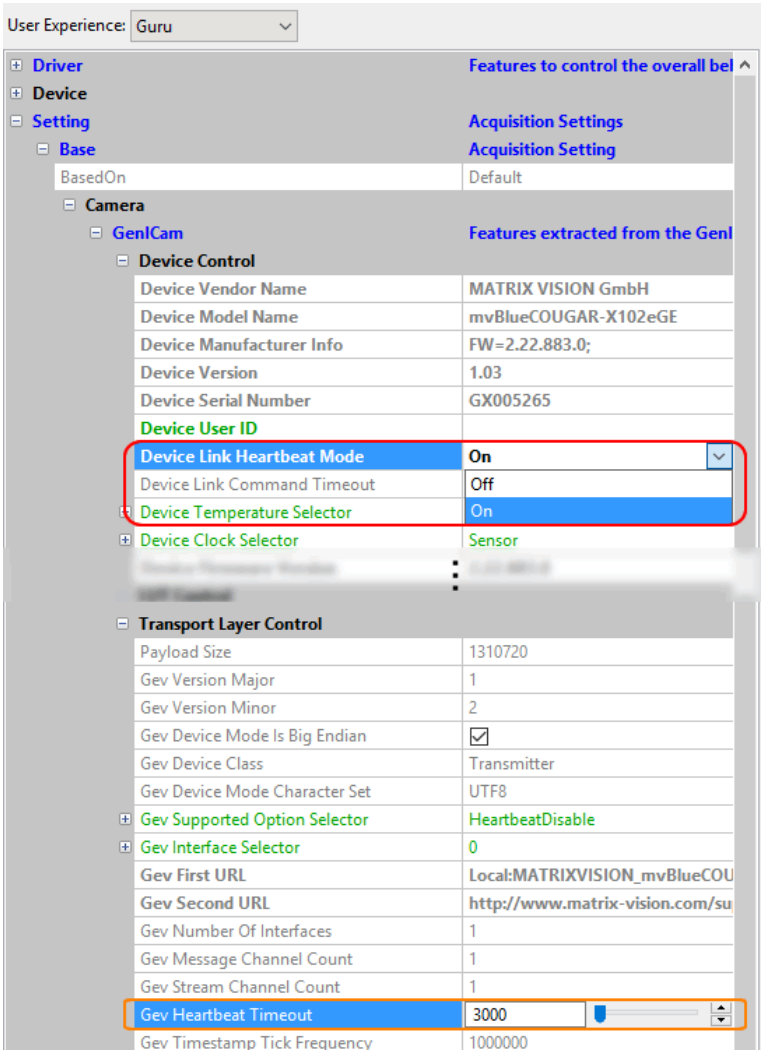


Figure 3: wxPropView - Disabling the heartbeat

Secondly, since the conveyor belt stops sometime, the trigger will do so, too. Be sure, that the trigger signal is available until the last image block was received.

Thirdly, if you know the number of image blocks, you can use the **"MultiFrame functionality"** (in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"* `set` "Acquisition Mode = MultiFrame" and "`Acquisition Frame Count`"). This will acquire the number of image blocks and will stop the acquisition afterwards.

1.19.1.10 Using the mvBlockscan feature

Since

Firmware version 2.35.0

1.19.1.10.1 Introduction

The **mvBlockscan** mode can be used for implementing a kind of linescan application. Instead of a single line, an Area of Interest (AOI) with a few lines (**LinesPerBlock**) will be captured. Then a selectable number of these small AOIs will be sent as a single image. This helps to minimize the overhead of a "high frequency" streaming of every single AOI when doing it in the conventional way. This mode is available for all mvBlueFOX3 cameras with Sony Global Shutter CMOS sensors.

The minimum size of **mvBlockscanLinesPerBlock** and the increment/decrement step size is similar the the limitations of the **Height** property for the specific camera model. Typically these are values in the range of 4 - 16 but it is always better to check these limits at runtime.

The advantages of the block scan mode over a line scan camera are:

- Standard interface: USB3 Vision instead of CoaXPress and CameraLink.
- Easier system setup: Since the camera can use area scan too, it is much easier to adjust the focus and get a sharp image than this would be with a line scan sensor.
- Less block loss due to the FPGA inside the camera.
- Less load on the host: Blocks are collected in the camera.
- Less pricy than a line scan camera (with the same line rate).

1.19.1.10.2 Using wxPropView

Let's say we have a rotating drum including an **incremental encoder** (p. 348) connected to the digital inputs (Encoder Source A → Camera Line 4; Encoder Source B → Camera Line 5) and also a light source connected to the digital output, the configuration of the mvBlockscan mode would look like this with wxPropView:

1. Set the **DeviceScanType** in *"Setting → Base → Camera → GenICam → DeviceControl → DeviceScanType"* to **mvBlockscan** .
2. Configure the **Initial situation and settings** (p. 252) **OffSetY**, the **mvBlockscanLinesPerBlock**(>= minimum value of **Height**) and the **mvBlockscanBlockCount**(>= 2) in *"Setting → Base → Camera → GenICam → ImageFormatControl"*.
3. Set the **PixelFormat** in *"Setting → Base → Camera → GenICam → ImageFormatControl"* to e.g. **BayerRG8** for color cameras .
4. Configure an **incremental encoder** (p. 348) in *"Setting → Base → Camera → GenICam → EncoderControl"*, e.g.

   (a) **EncoderSourceA:** Line4
   (b) **EncoderSourceB:** Line5
   (c) **EncoderDivider:** 1
   (d) **EncoderOutputMode:** PositionDown/DirectionDown
5. Configure a trigger in *"Setting → Base → Camera → GenICam → AcquisitionControl"*

   (a) **TriggerSelector:** FrameStart
   (b) **TriggerMode:** On
   (c) **TriggerSource:** Encoder0
6. Adjust the **ExposureTime** to your needs in *"Setting → Base → Camera → GenICam → AcquisitionControl"*.
7. If needed configure the settings for a light source in *"Setting → Base → Camera → GenICam → DigitalIOControl"*.

   (a) **LineSelector:** The output line physically connected with the light source
   (b) **LineSource:** ExposureActive

Optionally you can configure a debounce time for Line4 and Line5 in *"Setting → Base → Camera → GenICam → DigitalIOControl"* if needed.

After setting up the device like this an application will then receive image buffers with a height of *mvBlockscanLinesPerBlock* times `mvBlockscanBlockCount`. When the last of these images shall be terminated early thus with just a portion of the configured `mvBlockscanBlockCount` block in it *"Setting → Base → Camera → GenICam → AcquisitionControl → AcquisitionStop"* can be called. This will terminate the current acquisition and send out an image containing the number of blocks already captured.

1.19.1.11 Working with Event Control

MATRIX VISION devices can generate Event notifications. An Event is a message that is sent to the host application to notify it of the occurrence of an internal event. With *"Setting -> Base -> Camera -> GenICam -> Event Control"* you can handle these notifications.

At the moment, it is possible to handle

- Exposure End (= sensor's exposure end)
- Line 4 (= DigIn0) Rising Edge
- Line 5 (= DigIn1) Rising Edge
- Frame End (= the camera is ready for a new trigger)

1.19.1.11.1 Setting Event notifications using wxPropView

To activate the notifications, just

1. Select the Event via *"Setting -> Base -> Camera -> GenICam -> Event Control -> Event Selector"*, e.g. **ExposureEnd** .
2. Set the *"Event Notification"* to **"On"** .

Afterwards, it is possible to attach a custom callback that gets called whenever the property is modified. E.g. if you want to attach a callback to the Frame ID after the exposure was finished, you have to

1. select *"Setting -> Base -> Camera -> GenICam -> Event Control -> Event Exposure End Data -> Event Exposure End Frame ID"*,
2. right-click on the property, and
3. click on **"Attach Callback"**.

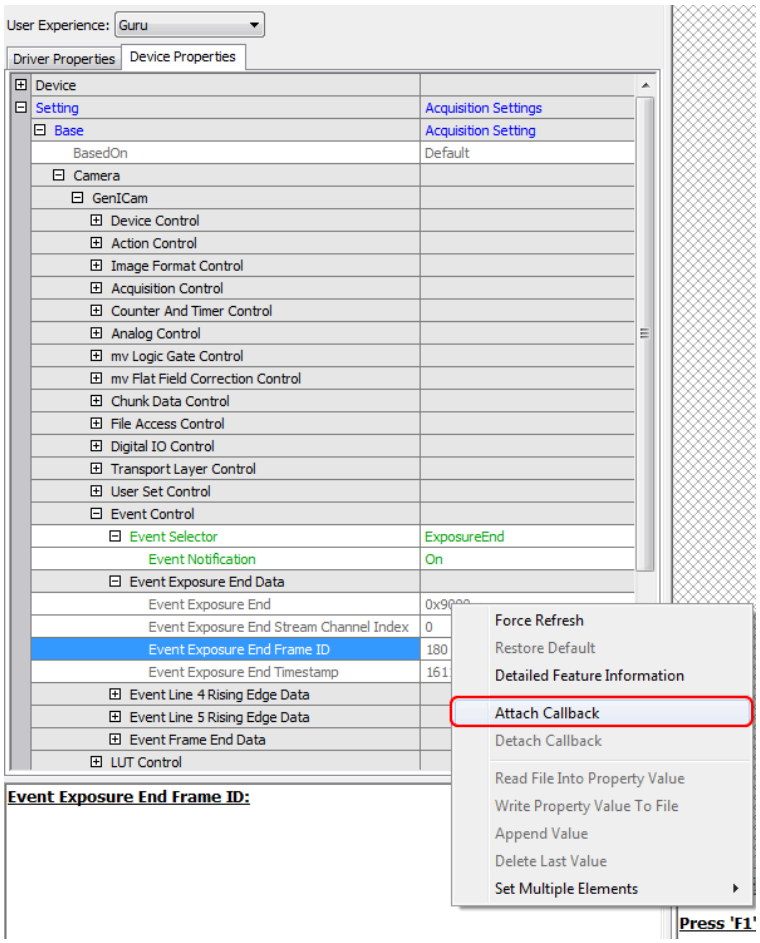


Figure 1: wxPropView - "Attach Callback" to Event Exposure End Frame ID

Now, you can track the property modifications in the output window:

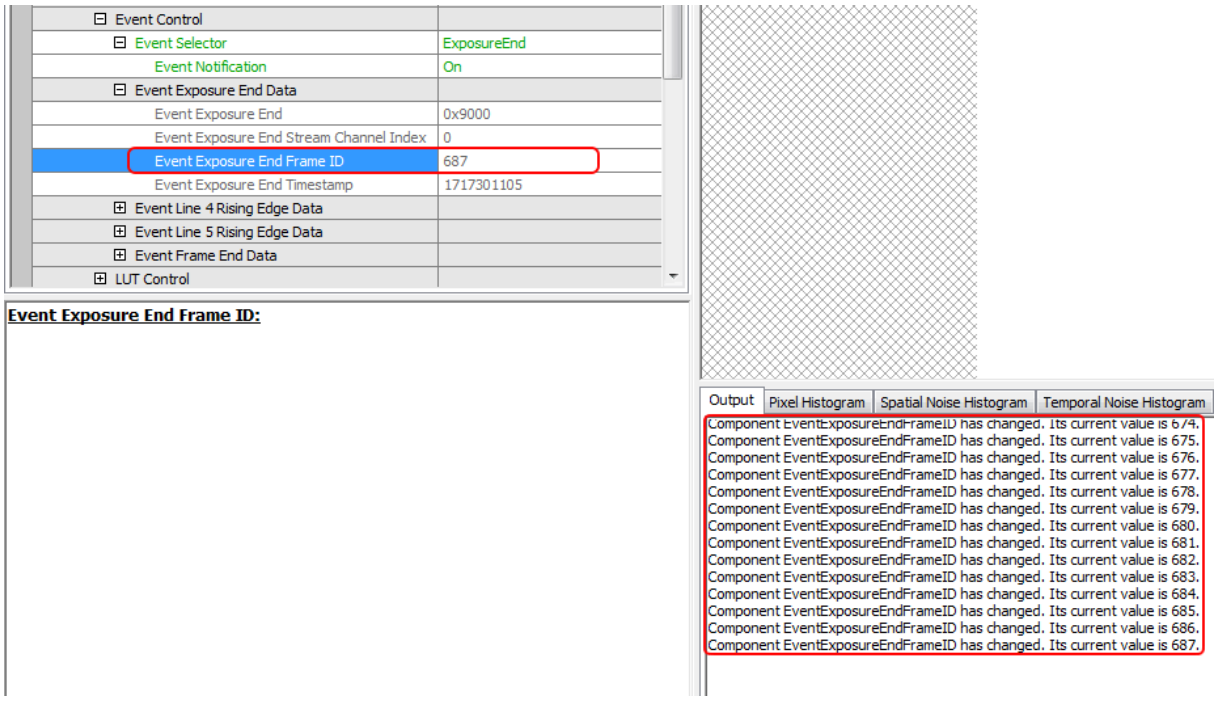


Figure 2: wxPropView - Output window with the Event notifications

You can find a detailed `Callback code example in the C++ API manual`.

1.19.1.12 Polarized Data Extraction

1.19.1.12.1 Introduction

The **mvBlueCOUGAR-X105p (5.1 Mpix [2464 x 2056])** (p. 537) sensor is equipped with a polarization filter. So every '2 ∗ 2' pixel area is sensitive for polarized light with the angles 0, 90, 135 and 45 degree. In previous versions, you could display the information from these 4 directions in 4 images, each image showing just the one pixel of the '2 ∗ 2' area with the same filtering direction.

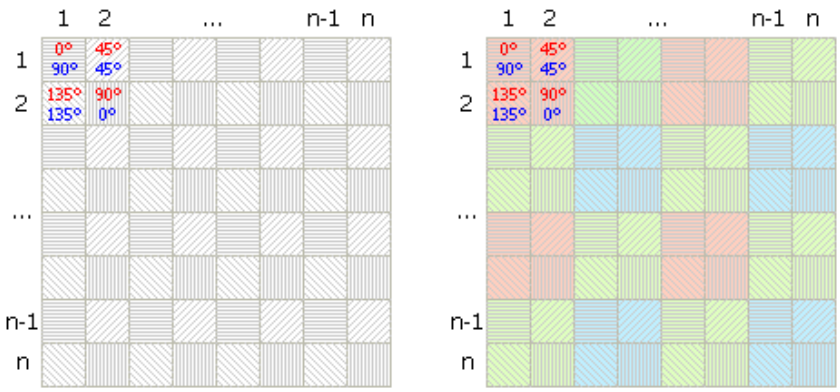


Figure 1: Pixel array mono and color

Since version 2.38.0 we support the calculation of the main polarization angle for each '2 ∗ 2' pixel area. Additionaly it is possible, to display also the degree of the polarized beam. The resulting image will have a width equal to 'input image width / 2' and height equal to 'input image height / 2'. From each 2 by 2 pixel area a single output value will be calculated.

1.19.1.12.2 Polarization value calculation

First we calculate the **Stokes parameters** S_0**,** S_1 **and** S_2 for each 2 by 2 pixel area. Since we just filter for linear polarized light, the circular parameter S_3 will not be used.

- S_0, the total intensity of the beam, is calculated with: $S_0 = P0 + P90$, with $P0$: pixel intensity with 0 degree polarization filter, $P90$: 90 degree polarization filter.
- S_1, the intensity with 0 (horizontal) and 90 (vertical) degree polarization angle. $S_1 = P0 - P90$
- S_2, the intensity with 45 and 135 degree polarization angle. $S_2 = P45 - P135$, with $P45$: pixel intensity with 45 degree polarization filter, $P135$: 135 degree polarization filter.

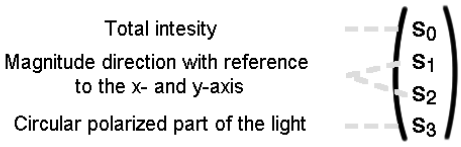


Figure 2: Strokes vector parts.

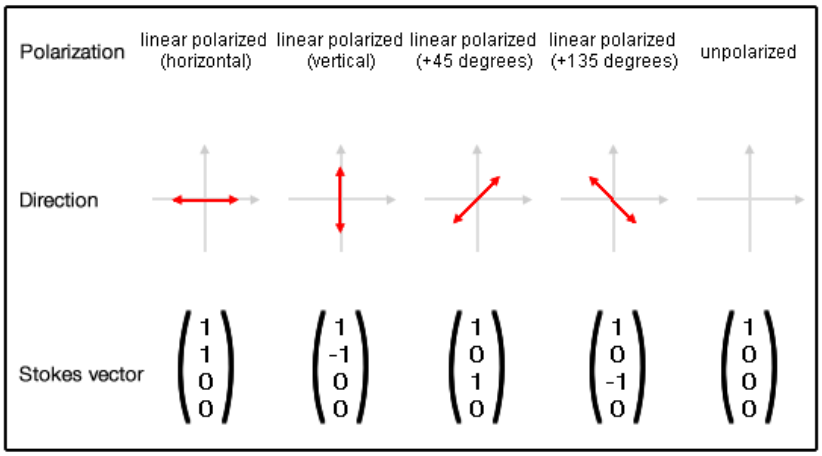


Figure 3: Angel of polarization with corresponding Stokes vector.

1.19.1.12.3 Degree of polarization

The **degree of polarization** Π describes the fraction of the ordered part of the wave. The higher this value, the more polarized is the wave in one single direction. Π is calculated one time for each 2 by 2 pixel area of an image.

$$\Pi = \frac{\sqrt{(P0 - P90)^2 + (P45 - P135)^2}}{(P0 + P90)}$$

1.19.1.12.4 Angle of polarization

The **angle of polarization** Θ describes the angle of the maximum polarization, it ranges from 0 to 180 degree. Like Π, Θ is also calculated one time for each 2 by 2 pixel area.

$$\Theta = \frac{1}{2} * atan\left((P45 - P135)\,;(P0 - P90)\right)$$

1.19.1.12.5 Image Representation

After the calculation of Θ and Π for each 2 by 2 pixel area, there are some ways on how these values can be displayed. The following images show all the same scene, first the raw pixel values and afterwards some pictures with computed values.



Figure 4: Unprocessed image as a reference.

The first possibility is, to display the angle of polarization. Therefore the resulting value for Θ will be mapped to the output pixel format, for example 180 degree would result in a maximum pixel value of 1023 for a 10-bit format.

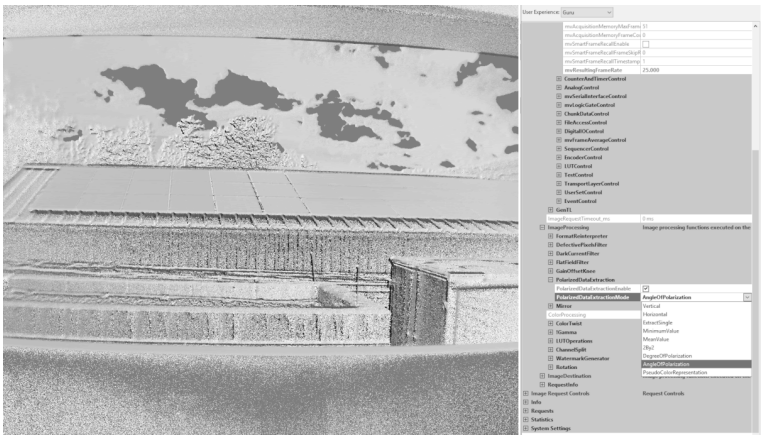


Figure 5: This image shows an example of the representation of the angel of polarization and the corresponding setting in wxPropView.

Another possibility is to display the degree of polarization. The value for Π will be mapped to the output pixel format, for example a maximum polarized beam with Π = 1 would result in a maximum pixel value of 255 for a 8-bit format.

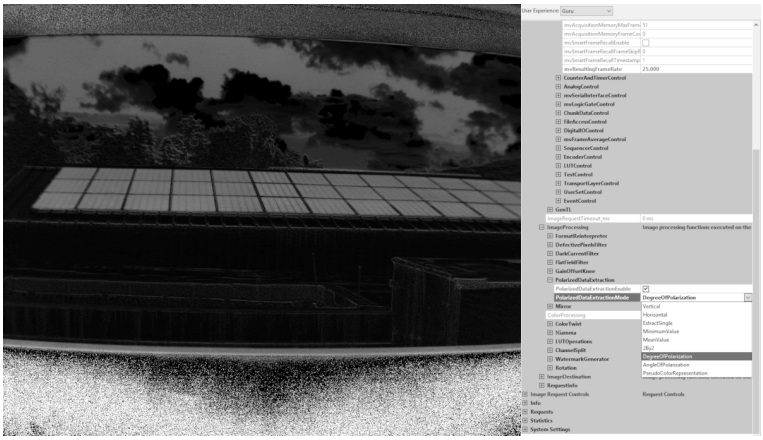


Figure 6: This image shows an example of the representation of the degree of polarization and the corresponding setting in wxPropView.

The third way is to display the angle and the degree of polarization in a single image as a pseudo color representation. Therefore both values will be mapped to a 8-bit HSL image. The angle value Θ represents the hue and the degree Π will be mapped to the saturation value of the image. Finally the HSL image is converted to RGB and displayed.

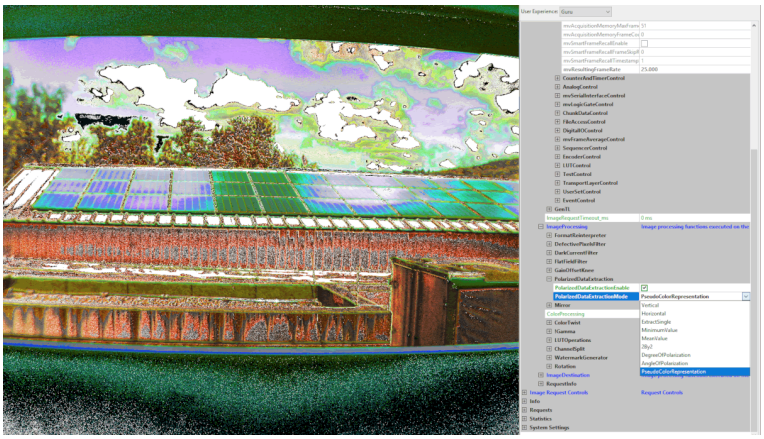


Figure 7: This image shows an example of the representation as pseudo color image and the corresponding setting in wxPropView.

1.19.1.13 Using Video Stream Recording

With the FFmpeg libraries it is possible to record an mvIMPACT Acquire image stream into a compressed video stream.

Since

2.39.0

1.19.1.13.1 Requirements Since the mvIMPACT Acquire API internally uses FFmpeg to record video streams, the FFmpeg libraries need to be present on the target system as well. They can either be built **OR** installed into the systems default library search path **OR** installed somewhere and afterwards an environment variable **MVIMPACT_ACQUIRE_FFMPEG_DIR** can be defined that points to the folder containing the libraries.

Note

Please make sure that you fully understand the license coming with FFmpeg! Have a look at the corresponding `legal section` inside any of the SDK manuals.

At least FFmpeg **4.x** is needed. Older versions of the FFmpeg API are NOT compatible!

1.19.1.13.1.1 Windows

1. Go to `https://ffmpeg.org/download.html` and download the dynamic libraries of FFmpeg (version >= 4.x) according to your operating system (*e.g. 'ffmpeg-20200809-6e951d0-win64-shared.zip'*)
2. Extract the ∗.zip file under *'${MVIMPACT_ACQUIRE_DIR}/Toolkits'*.
3. Rename the file to *'ffmpeg-4.2.2-win64-shared'*(64-bit)/*'ffmpeg-4.2.2-win32-shared'*(32-bit) **OR** set an environment variable *e.g. 'MVIMPACT_ACQUIRE_FFMPEG_DIR'* which points to the folder containing the libraries.

1.19.1.13.2 Recording in wxPropView In wxPropView, a video stream can be recorded by the *'Start'*, *'Pause'* and *'Stop'* buttons at the top right tool-bar. They are however inactive when the video stream recording mode is deactivated OR when the video stream is not correctly set up.



Figure 1: Video stream recording control buttons (inactive)

A video stream needs to be set up first to be able to get recorded. To do so:

1. Select the device to use and open it by clicking on the *'Use'* button.
2. Navigate to the *'Capture'* menu and click on the *'Video Stream Recording...'* option to start a setup dialog.

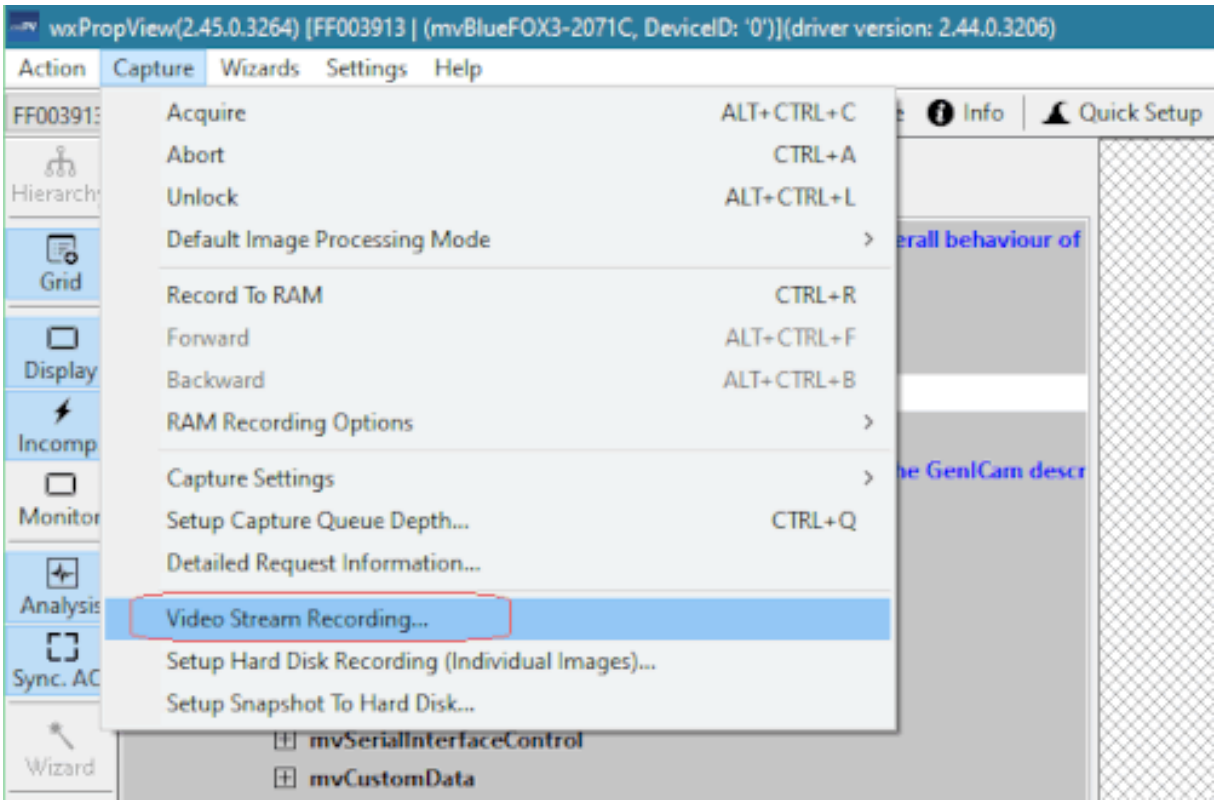


Figure 2: Click 'Video Stream Recording...'

3. A setup dialog will then be initialized as follows. Please read the setup hints in the text box for more information.

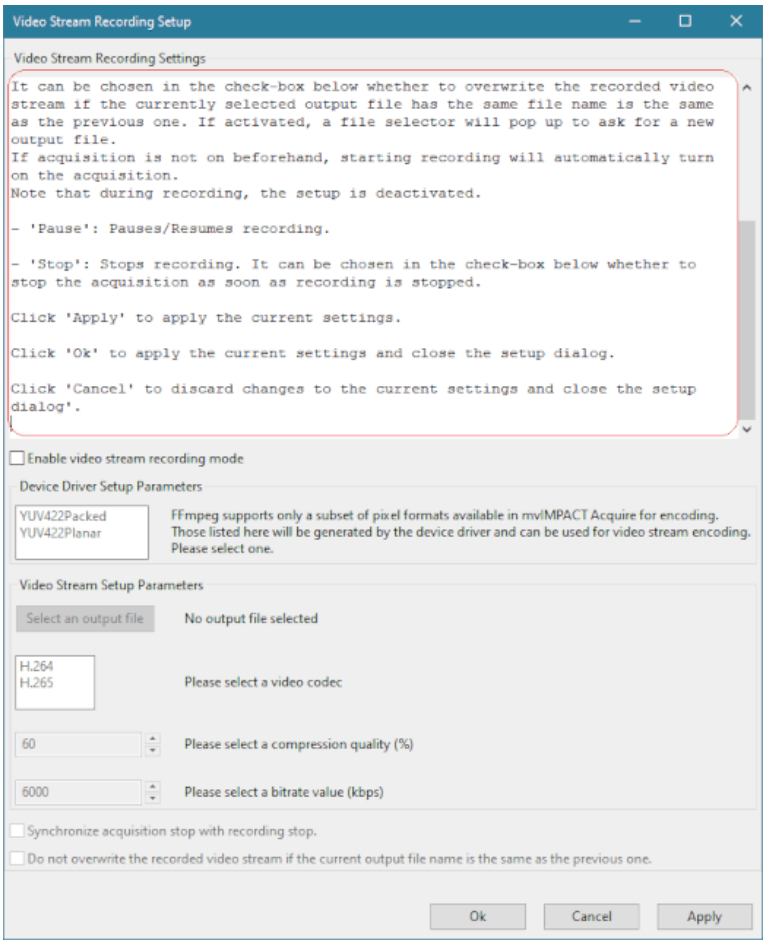


Figure 3: Video stream recording setup dialog

4. Enable the video stream recording mode. Choose a pixel format (e.g. *'YUV422Packed'* or *'YUV422Planar'*) that will be generated by the device driver and used by FFmpeg for video stream encoding. Then click on *'Select an output file'* to create/choose a file to hold the recorded video stream.

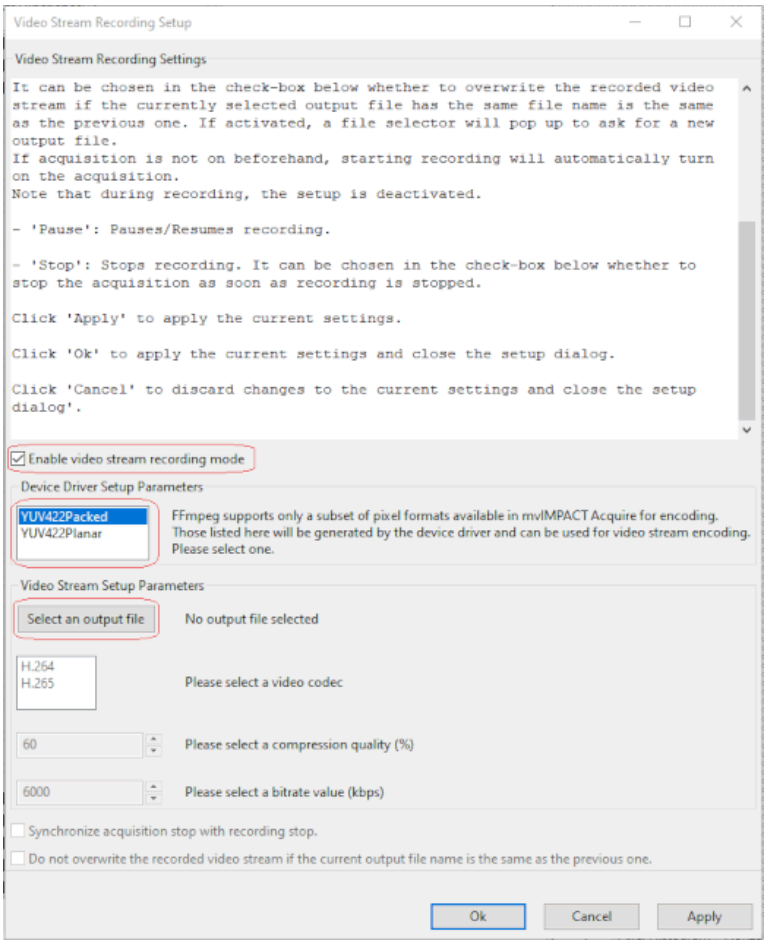


Figure 4: Enable the video stream recording mode and set up device driver related parameters

5. In the file selector, choose a file type (e.g. '∗.mp4' or '∗.m2v') and enter a file name.

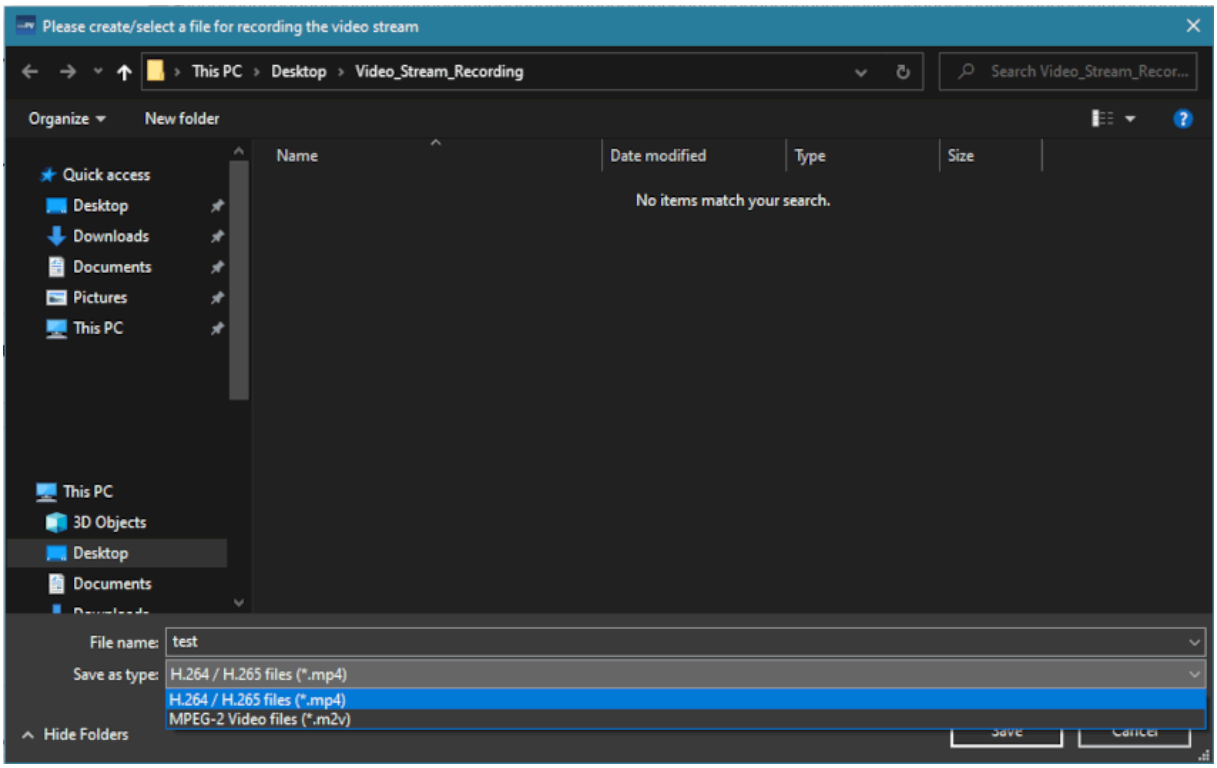


Figure 5: Select an output file

6. Set up video stream related parameters accordingly. In the check boxes below, users are allowed to choose whether to synchronize acquisition stop with recording stop and whether to overwrite the already recorded video stream if the currently selected output file has the same file name as the previous one.

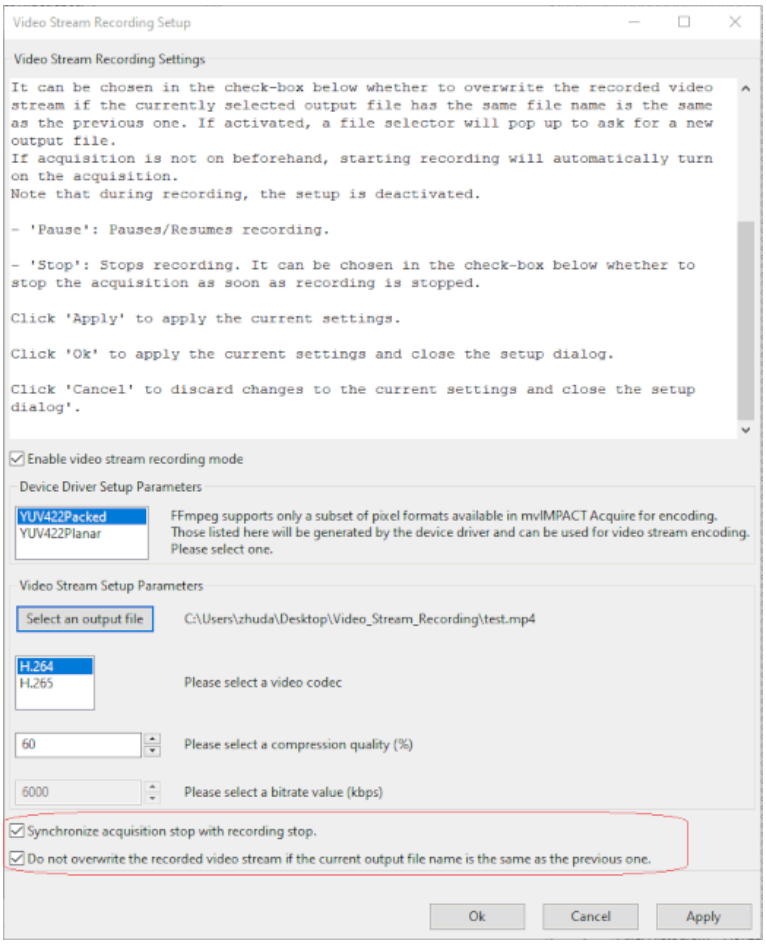


Figure 6: Set up video stream related parameters

7. Once the video stream recording has been set up, click *'Apply'* or *'Ok'* to apply the current settings. Afterwards, a log message in the analysis output will indicate whether the current settings have been applied successfully. If successful, the *'Start'* control button at the top right tool-bar will be enabled.

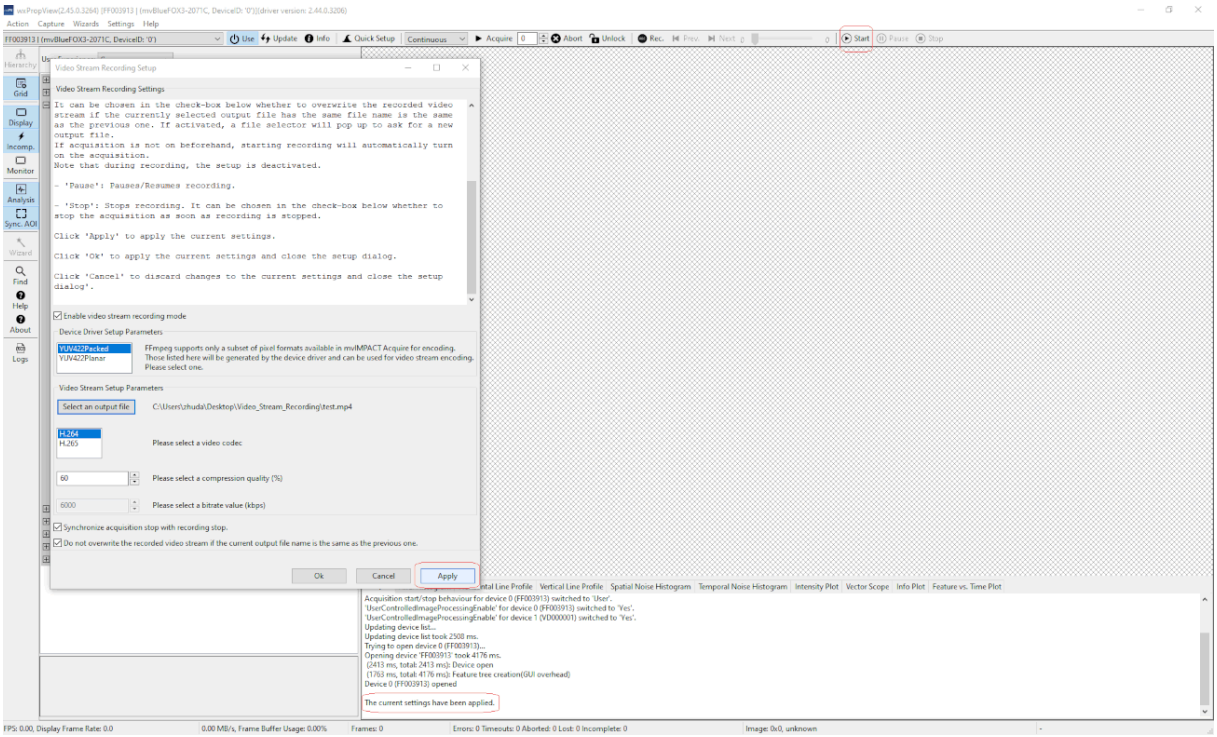


Figure 7: Apply the current settings

Note

When deactivating the video stream recording, uncheck the *'Enable video stream recording mode'* and then click *'Apply'* or *'Ok'* for the settings to take effect.

Once the settings have been applied, users can control the recording process via the *'Start'*, *'Pause'* and *'Stop'* buttons:

- **Start recording**: Click the *'Start'* control button to start recording the video stream. The current recording status and information will be displayed in the analysis output. During recording, the setup dialog as well as the *'Start'* button will be disabled. The *'Pause'* and *'Stop'* buttons will then be enabled.

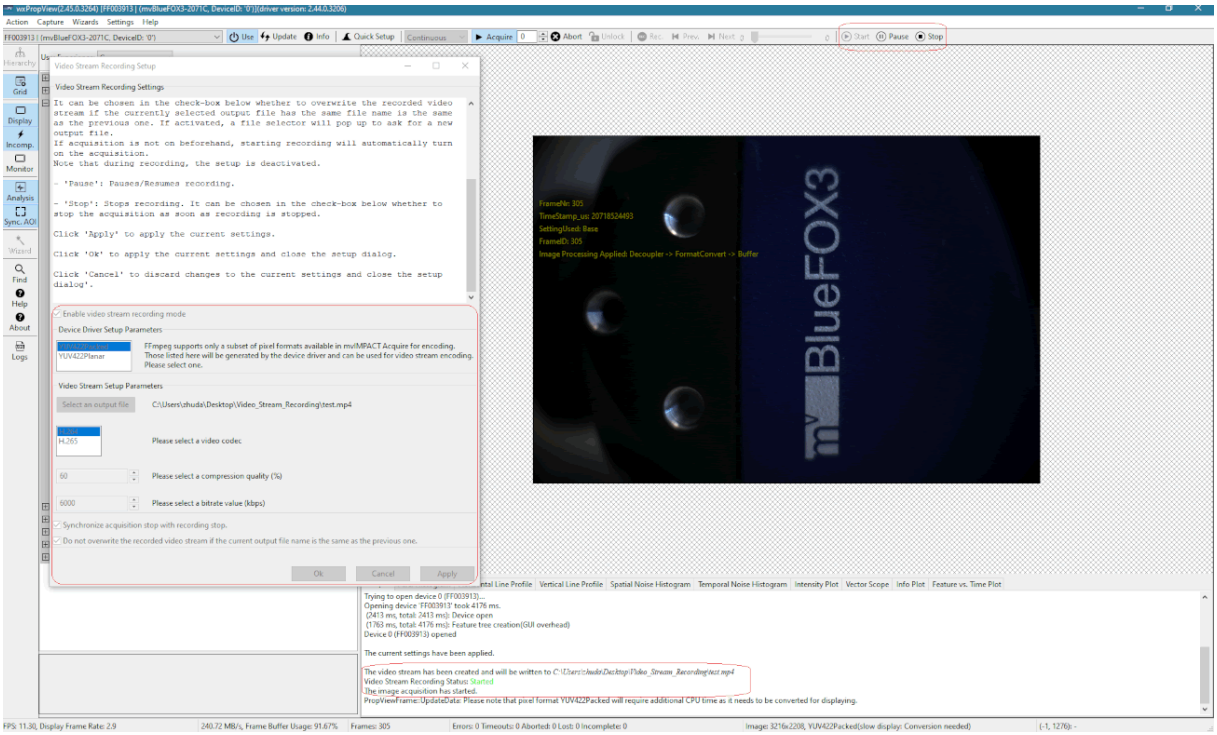


Figure 8: Start recording

- **Pause recording**: Click the *'Pause'* button to pause a running recording. The current recording status will be displayed in the analysis output.

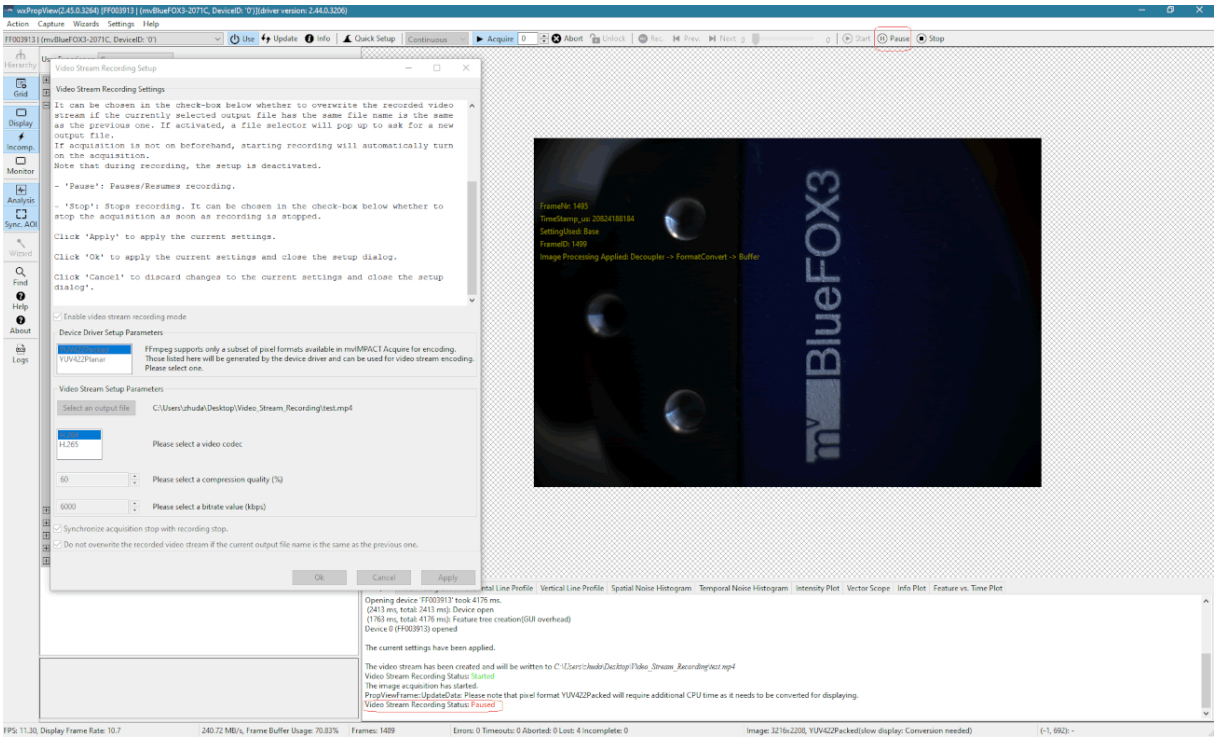


Figure 9: Pause recording

- **Resume recording**: Click the *'Pause'* button to resume a paused recording. The current recording status will be displayed in the analysis output.

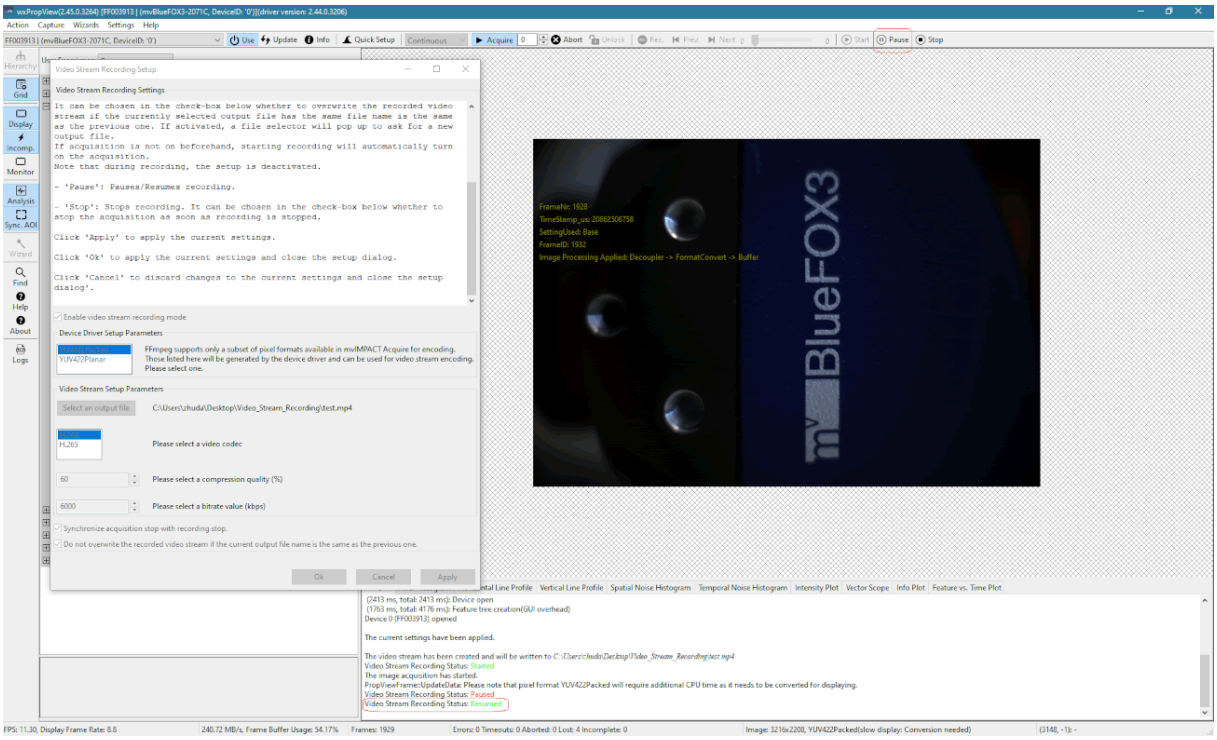


Figure 10: Resume recording

- **Stop recording**: Click the *'Stop'* button to stop recording the video stream. The current recording status and information will be displayed in the analysis output. Once the recording has been stopped, the setup dialog as well as the *'Start'* button will be enabled again. The *'Pause'* and *'Stop'* buttons will then be disabled.

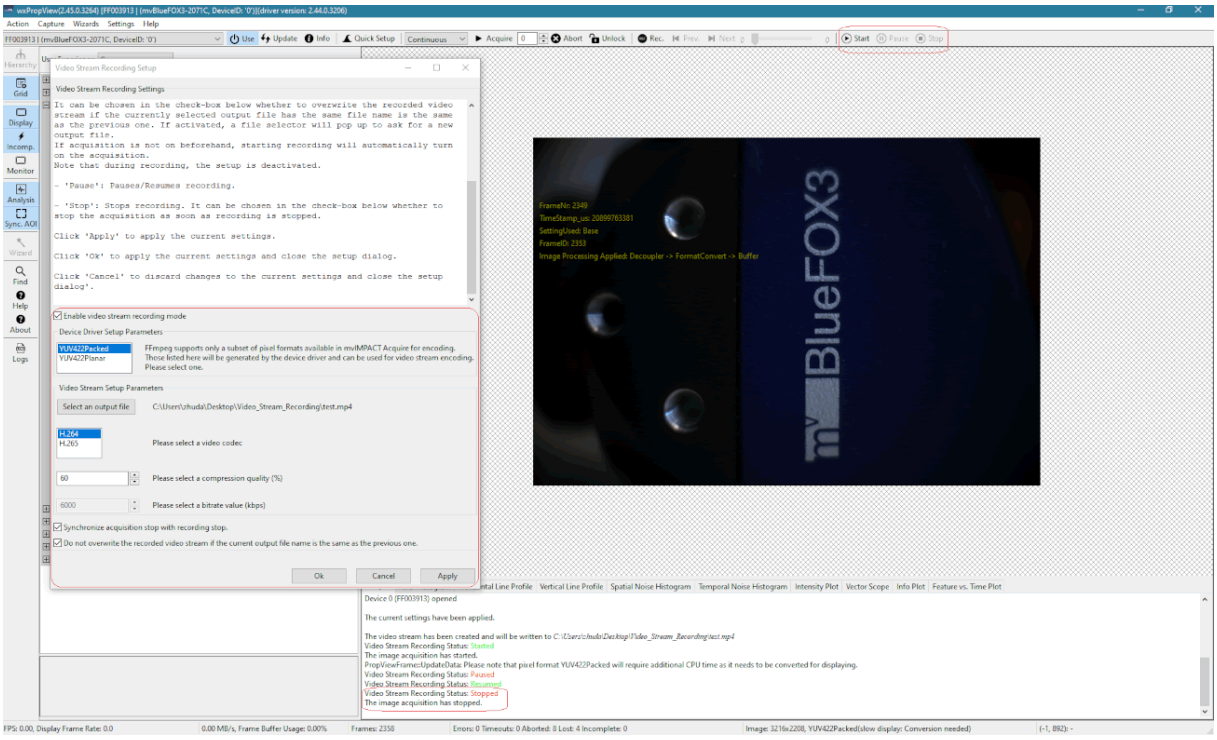


Figure 11: Stop recording

When recording to an output file which has the same file name as the previous one while overwriting the recorded content is not desired:

1. When clicking *'Start'*, a file selector will pop up to ask users to create a new output file with the same file type as the previous one. If a new set of parameters of the video stream recording is needed, please click *'Cancel'* in the file selector and re-configure parameters in the setup dialog.

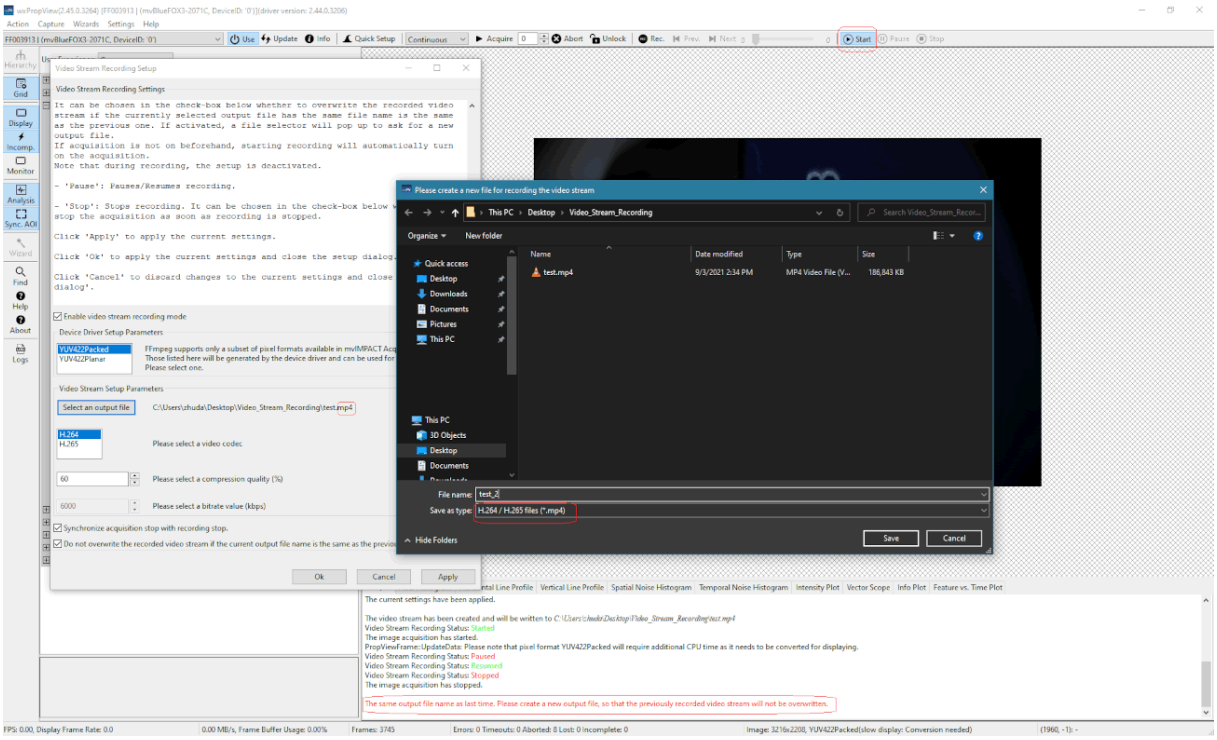


Figure 12: Select a new file when starting to record to an output file with the same file name as the previous one without overwriting

2. Once a new file has been created, the video stream will start to get recorded. The current recording status and information will be displayed in the analysis output. During recording, the setup dialog as well as the *'Start'* button will be disabled. The *'Pause'* and *'Stop'* buttons will then be enabled.

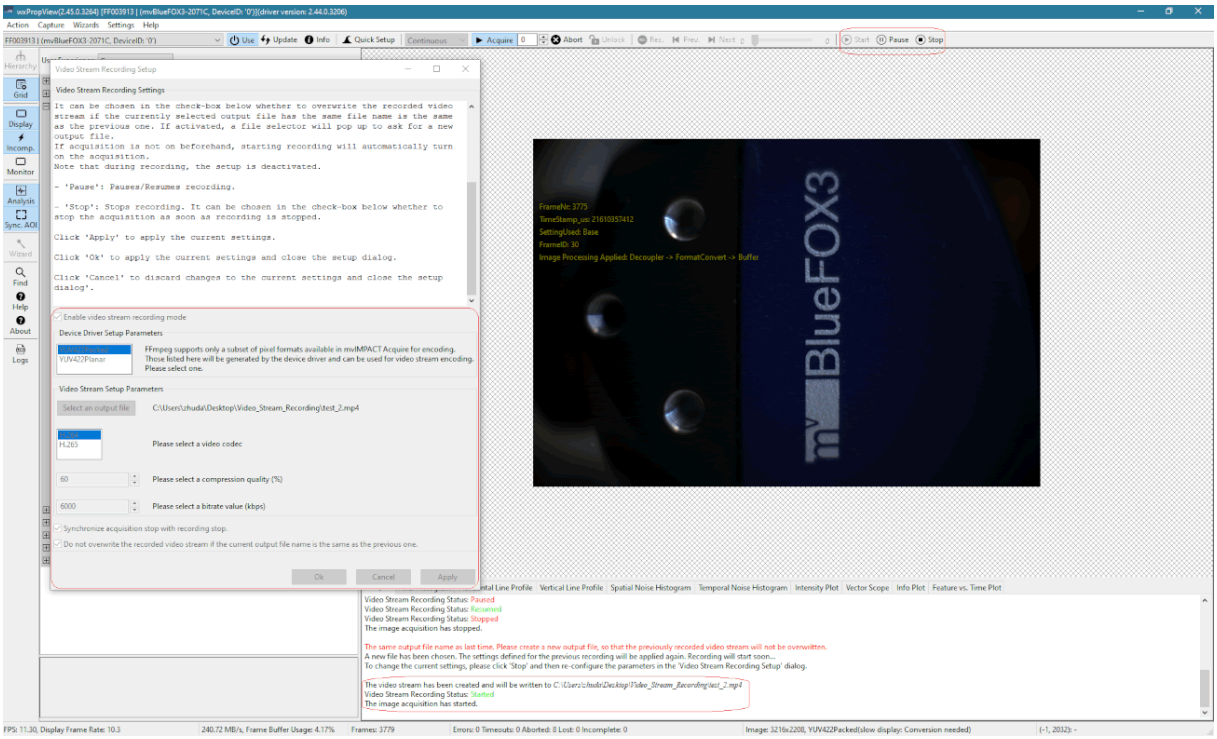


Figure 13: Start recording to an output file with the same file name as the previous one without overwriting

1.19.1.13.3 Recording Using The API

Please refer to the example on how to record a video stream using mvIMPACT Acquire C++ API: `Continuous↩CaptureFFmpeg.cpp` or have a look at the `VideoStream` class.

1.19.1.14 Introducing multicasting

GigE Vision (p. 215) supports to stream data over a network via Multicast to multiple destinations simultaneously. Multicast means, that one source streams data to a group of recipients in the same subnet. As long as the recipients are members of this Multicast group they will get the packages. Other members of the same local subnet will skip the packages (but the packages are sent and so they need bandwidth).

See also

`http://en.wikipedia.org/wiki/Multicast`

To set up a Multicast environment, **GigE Vision** (p. 215) introduced camera access types. The most important ones are

- **Control** and
- **Read**.

With **Control**, a primary application can used to set up the camera which will stream the data via Multicast. With **Read** you can set up secondary applications which will playback the data stream.

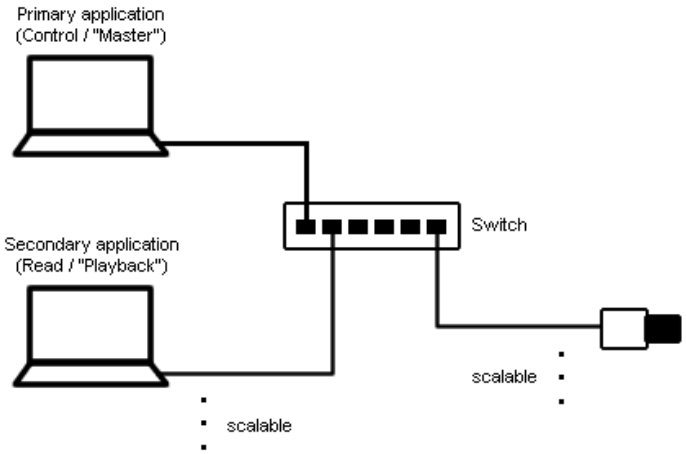


Figure 1: Multicast setup

Because the mvBlueCOUGAR-X cameras are **GigE Vision** (p. 215) compliant devices, you can set up a Multicast environment using **wxPropView** (p. 139). This use case will show how you can do this.

1.19.1.14.1 Sample

On (the primary) application you have to establish **"Control"** access.

For this,

1. please start **wxPropView** (p. 139) and select the camera.
2. Click on the **"Device"** section.
3. Click on **"DesiredAccess"** and choose **"Control"**.

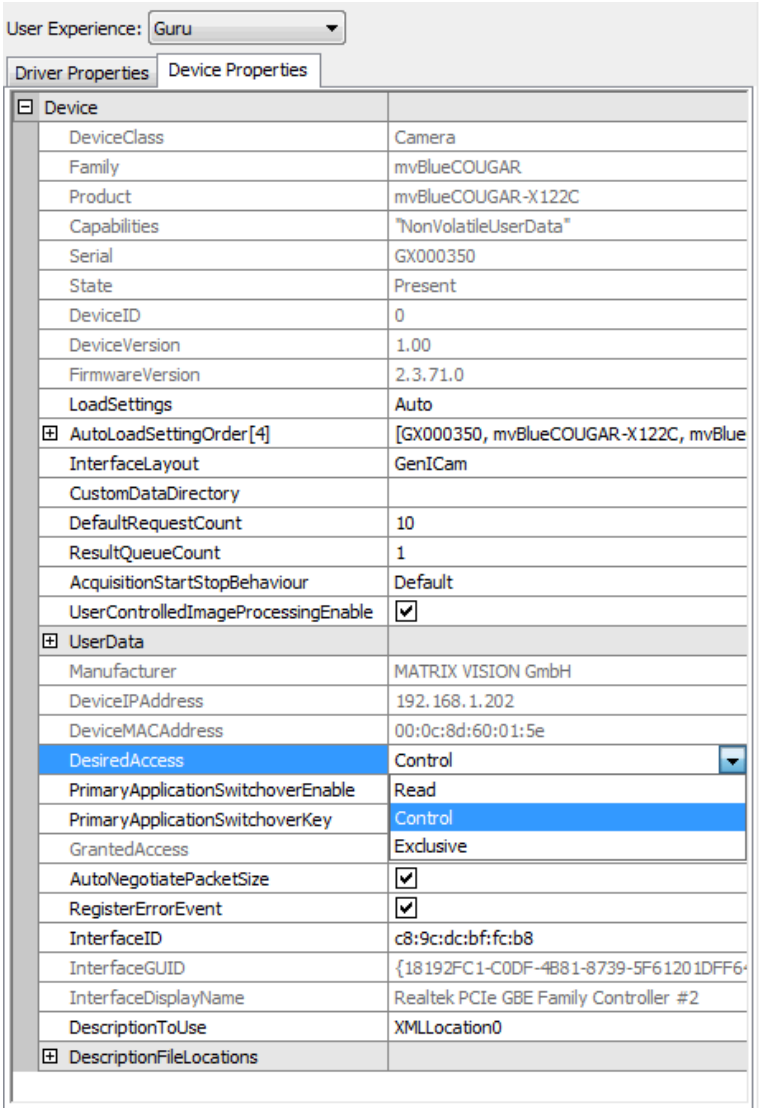


Figure 2: wxPropView - Primary application setting DesiredAccess to "Control"

See also

desiredAccess and grantedAccess in

- **mvIMPACT::acquire::Device** (in mvIMPACT_Acquire_API_CPP_manual.chm)
- **TDeviceAccessMode** (in mvIMPACT_Acquire_API_CPP_manual.chm)

4. Click on **"Use"**.
5. Now, select the *"Setting -> Base -> Camera -> GenICam"* section and open the *"Transport Layer Control"* subsection (using the device specific interface: *"System Settings"* and *"TransportLayer"*).
6. In **"GevSCDA"** enter a Multicast address like "`239.255.255.255`" .

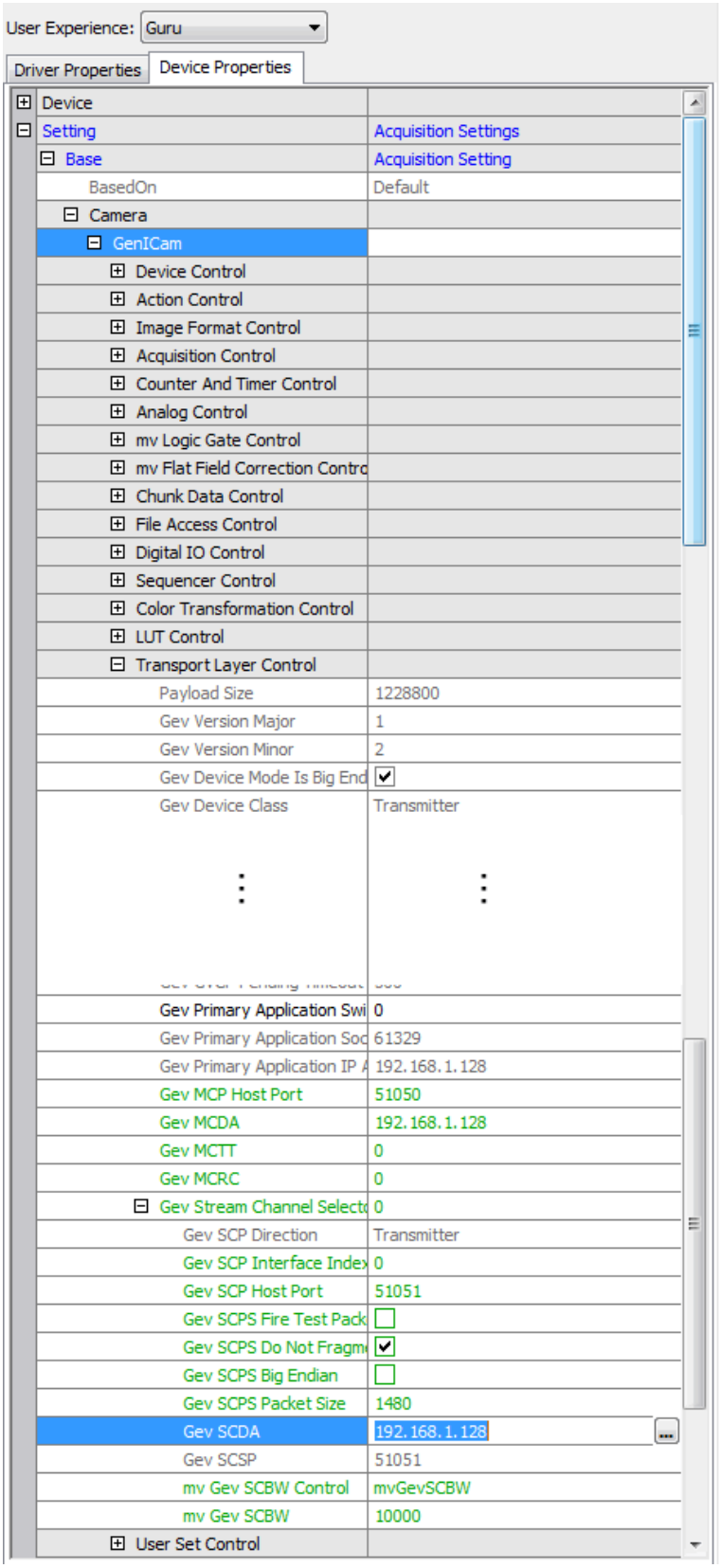


Figure 3: wxPropView - Primary application setting GevSCDA in "GevSCDA"

See also

`http://en.wikipedia.org/wiki/Multicast_address`

One or more applications running on different machines can then establish "read-only" access to the very same device.

Note

The machines of the secondary applications have to be connected to the same network as the primary application.

1. Please start **wxPropView** (p. 139) on the other machine and click on the **"Device"** section.

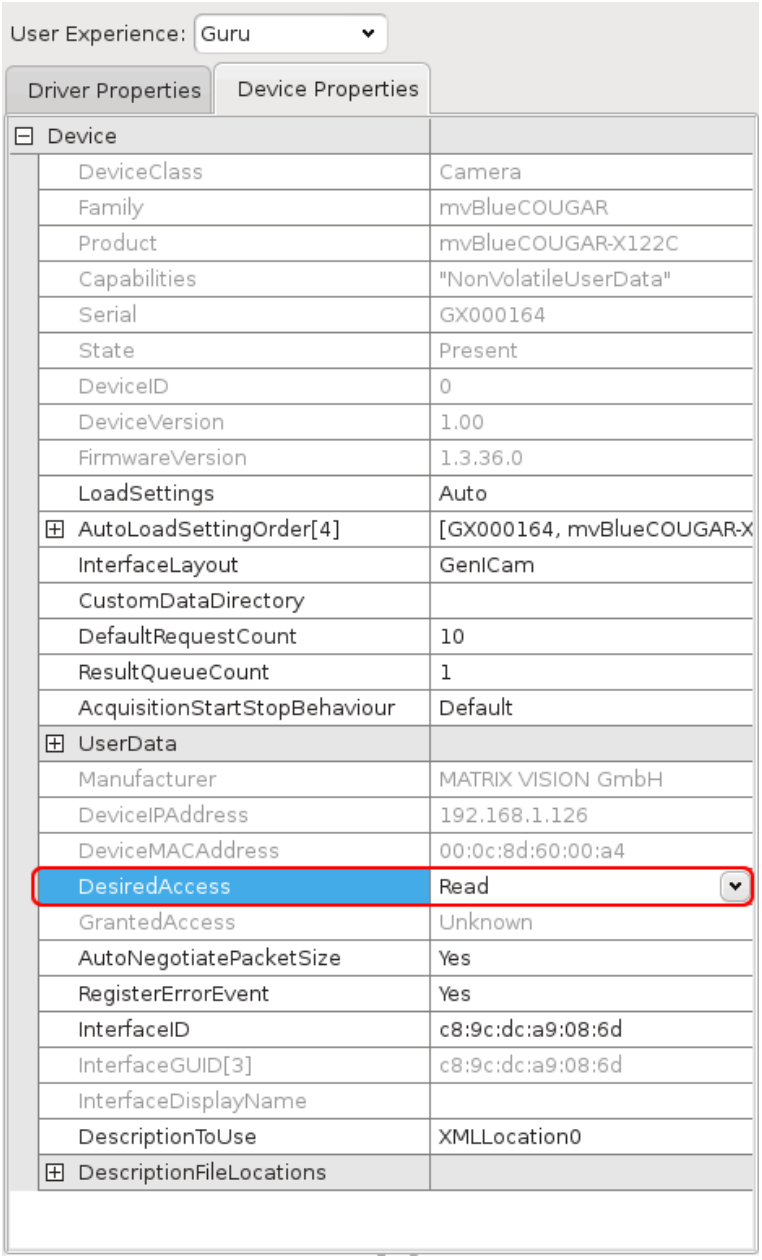


Figure 4: wxPropView - Secondary application setting DesiredAccess to "Read"

2. Features will not be writeable as you can see at the *"Transport Layer Control"* parameters in Figure 4.

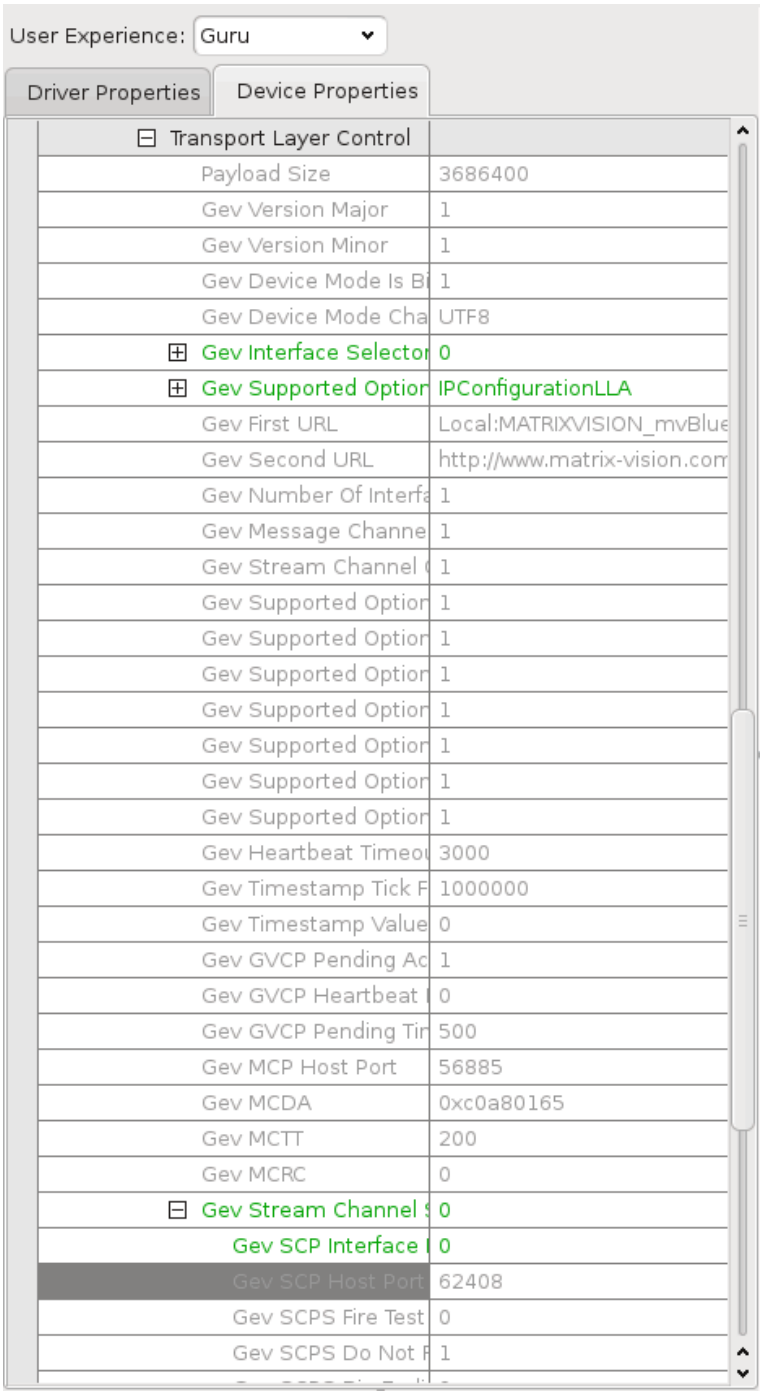


Figure 5: wxPropView - Secondary application read-only "Transport Layer Control" parameters

3. Once the primary application starts to request images, the secondary applications will be able to receive these images as well. Please click on **"Use"** and then **"Acquire"** (*"Acquisition Mode"* = *"Continuous"*).

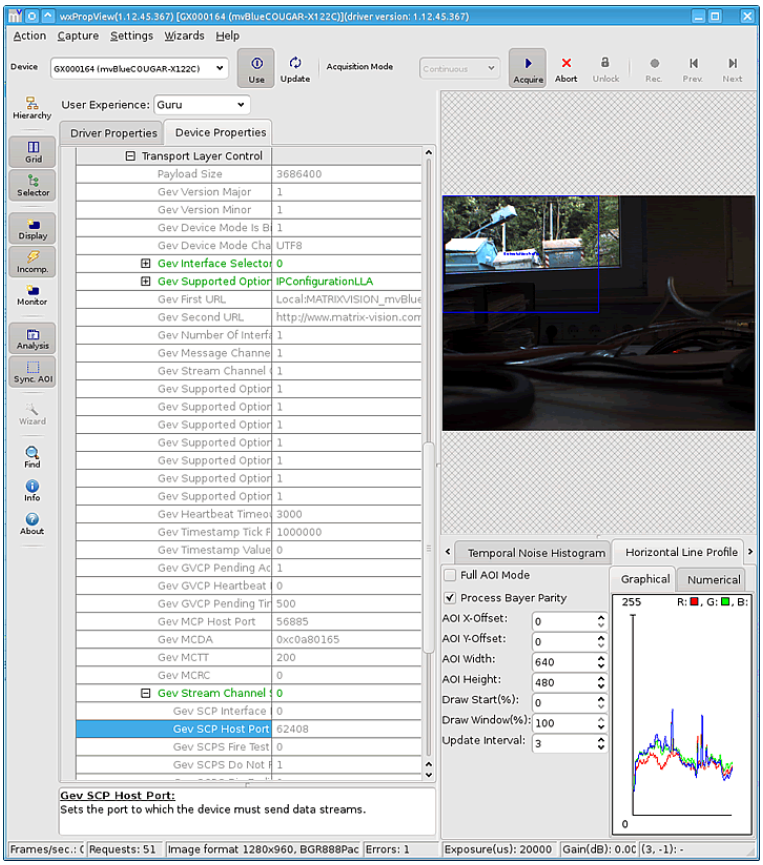


Figure 6: wxPropView - Secondary application receives images from the primary application

Note

The machine that has "Control" access automatically joins the streaming multicast group of the camera it is controlling. If this is not desired, the **"mv Auto Join Multicast Groups"** property has to be set to **false**.

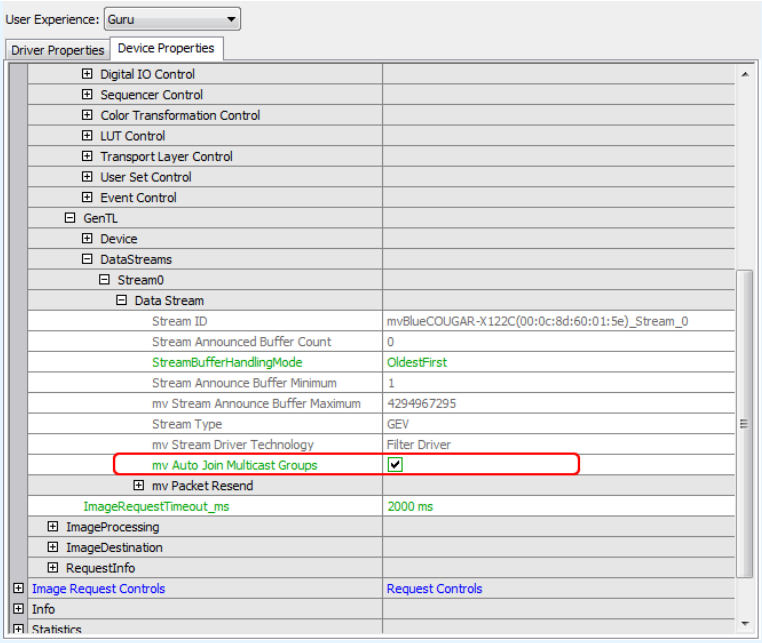


Figure 7: wxPropView - Disable mv Auto Join Multicast Groups to prevent the Control machine to receive multicast streaming data

GigE Vision (p. 215) (GEV) does not allow packet fragmentation for performance reasons. Because of that and the nature of multicasting and network data transfer in general it is crucial to select a packet size every client with respect to its **NIC** (p. 219) s **MTU** (p. 219) can handle as otherwise not all clients will be able to receive the image data. See the chapter

- **Optimizing Network Performance (Windows)** (p. 46)
- **Optimizing Network Performance (Linux)** (p. 57) for details.

1.19.2 Improving the acquisition / image quality

There are several use cases concerning the acquisition / image quality of the camera:

- **Correcting image errors of a sensor** (p. 284)
- **Optimizing the color/luminance fidelity of the camera** (p. 297)
- **Reducing noise by frame averaging** (p. 312)
- **Setting a flicker-free auto expose and auto gain** (p. 315)
- **Working with binning / decimation** (p. 318)
- **Working with Sony's 4 Tap CCD sensors** (p. 321)
- **Minimizing sensor pattern of mvBlueCOUGAR-X1010G** (p. 324)
- **Working with the dual exposure feature ("mvMultiZone") of mvBlueCOUGAR-XD107** (p. 327)
- **Working with the dual gain feature of mvBlueCOUGAR-XD107/XD1031 and mvBlueCOUGAR-X102m/↩ X107** (p. 330)
- **Working with the dual ADC feature of mvBlueCOUGAR-XD107** (p. 334)
- **Working With Gain And Black-Level Values Per Color Channel** (p. 335)

1.19.2.1 Correcting image errors of a sensor

1.19.2.1.1 Introduction Due to random process deviations, technical limitations of the sensors, etc. there are different reasons that image sensors have image errors. MATRIX VISION provides several procedures to correct these errors, by default these are host-based calculations, however some camera families support camera-based corrections, which saves dozens of % CPU load and lowers latency.

Camera Family	Algorithm-↩ Based detection and correction	List-Based correction	Storing facility for defective-pixel list	Flat-Field Correction (Host)	Flat-Field Correction (Camera)
mvBlue↩ COUGAR-X	-	-	X	X	X
mvBlue↩ COUGAR-XD	If binning/decimation is on -> no list is stored	X	X	X	
mvBlue↩ COUGAR-XT	X	-	-	X	X
mvBlueFOX3	If binning/decimation is on -> no list is stored	X	X	X	-

Generally, removing defect pixels requires two sub-tasks:

- Detection of defective pixels
- Correction of defective pixels

Both tasks can performed in different "locations":

- Detection and correction on the host using mvIMPACT Acquire
- Detection on the host using mvIMPACT Acquire, correction on the camera using the camera's `mv↩DefectivePixelCorrectionControl` in the list-based mode
- Detection and correction on the camera using `mvDefectivePixelCorrectionControl` in the algorithm-based mode.

If detection is not happening in real-time, meaning during the image acquisition itself, it is necessary to store the detected defects somewhere. This can be either on the camera or the host or both.

1.19.2.1.2 Defect pixel detection using mvIMPACT Acquire As mentioned, the defect pixel list can be generated using mvIMPACT Acquire. Since there are three types of defects, mvIMPACT Acquire offers three calibration methods for detection:

1. **leaky pixel** (in the dark)
   which indicates pixels that produce a higher read out code than the average
2. **hot pixel** (in standard light conditions)
   which indicates pixels that produce a higher non-proportional read out code when temperatures are rising
3. **cold pixel** (in standard light conditions)
   which indicates pixels that produce a lower read out code than average when the sensor is exposed (e.g. caused by dust particles on the sensor)

Note

Please use either an Mono or RAW Bayer image format when detecting defective pixel data in the image.

1.19.2.1.2.1 Detecting leaky pixels To detect leaky pixels the following steps are necessary:

1. Set gain ("`Setting -> Base -> Camera -> GenICam -> Analog Control -> Gain = 0 dB`") and exposure time "`Setting -> Base -> Camera -> GenICam -> Acquisition Control -> ExposureTime = 360 msec`" to the given operating conditions
   The total number of defective pixels found in the array depend on the gain and the exposure time.
2. Black out the lens completely
3. Set the (Filter-) "`Mode = Calibrate leaky pixel`"
4. Acquire an image (e.g. by pressing **Acquire** in wxPropView with "`Acquisition Mode = Single↩Frame`")

The filter checks:

```
Pixel > LeakyPixelDeviation_ADCLimit // (default value: 50)
```

All pixels above this value are considered as leaky pixel.

1.19.2.1.2.2 Detecting hot or cold pixels

Note

With "Mode = Calibrate Hot And Cold Pixel" you can execute both detections at the same time.

To detect hot or cold pixels the following steps are necessary:

1. You will need a uniform sensor illumination approx. 50 - 70 % saturation (which means an average gray value between 128 and 180)
2. Set the (Filter-) "`Mode = Calibrate Hot Pixel`" or "`Mode = Calibrate Cold Pixel`" or "`Mode = Calibrate Hot And Cold Pixel`"
3. Acquire an image (e.g. by pressing **Acquire** in wxPropView with "`Acquisition Mode = Single↩ Frame`")

The filter checks:

```
Pixel > T[hot] // (default value: 15 %)

// T[hot] = deviation of the average gray value

Pixel < T[cold] // (default value: 15 %)

// T[cold] = deviation of the average gray value
```

Note

Repeating the defective pixel corrections will accumulate the correction data which leads to a higher value in "`DefectivePixelsFound`". If you want to reset the correction data or repeat the correction process you have to set the filter mode to "`Reset Calibration Data`". In oder to limit the amount of defective pixels detected the "`DefectivePixelsMaxDetectionCount`" property can be used.

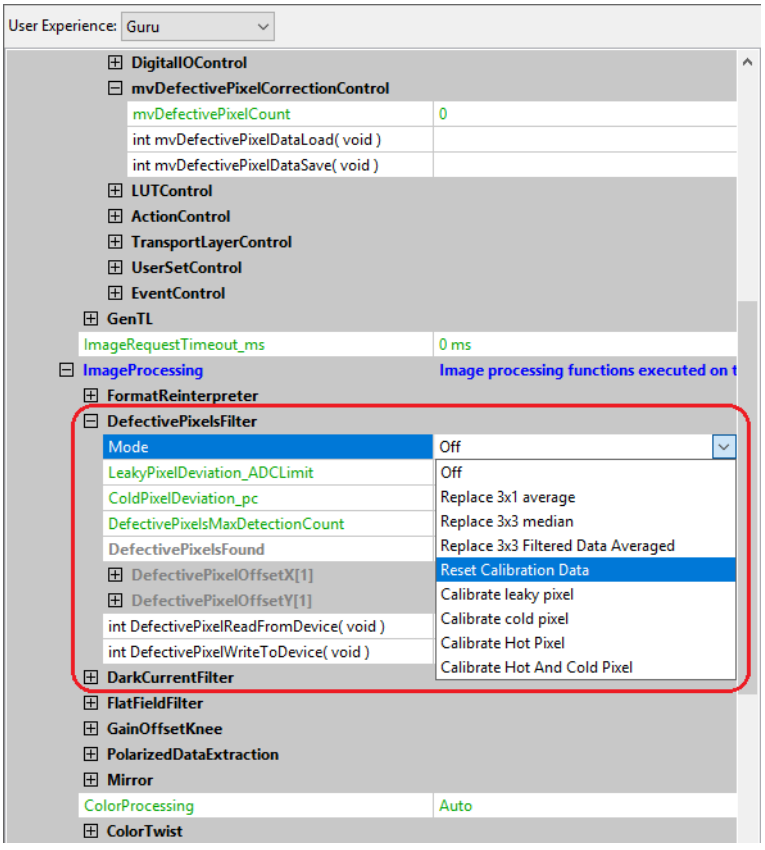


Figure 1: Image corrections: DefectivePixelsFilter

1.19.2.1.3 Storing defective pixel data on the device To save and load the defective pixel data, appropriate functions are available:

- **"int mvDefectivePixelDataLoad(void)"**
- **"int mvDefectivePixelDataSave(void)"**

The section *"Setting -> Base -> ImageProcessing -> DefectivePixelsFilter"* was also extended (see Figure 2). First, the "DefectivePixelsFound" indicates the number of found defective pixels. The coordinates are available through the properties "DefectivePixelOffsetX" and "DefectivePixelOffsetY" now. In addition to that it is possible to edit, add and delete these values manually (via right-click on the "DefectivePixel↩Offset" and select **"Append Value"** or **"Delete Last Value"**). Second, with the functions

- **"int mvDefectivePixelReadFromDevice(void)"**
- **"int mvDefectivePixelWriteToDevice(void)"**

you can exchange the data from the filter with the camera and vice versa.

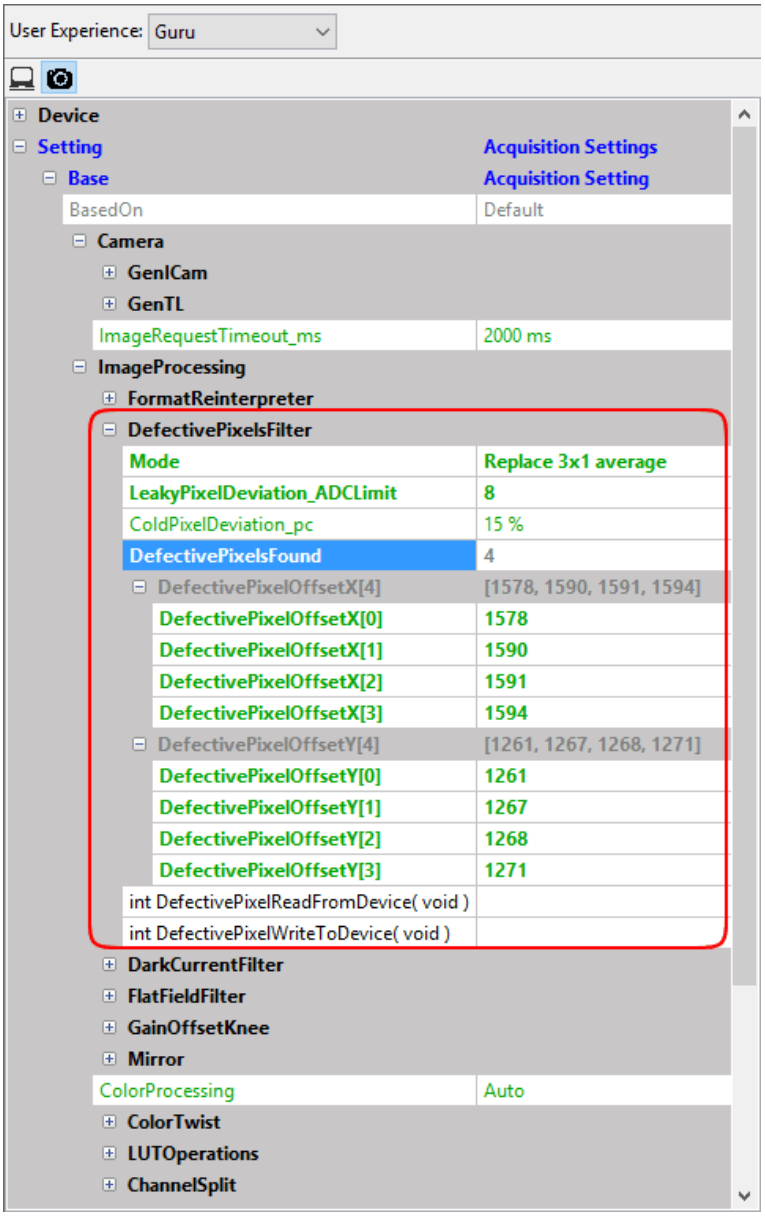


Figure 2: Image corrections: DefectivePixelsFilter (since driver version 2.17.1 and firmware version 2.12.406)

Just right-click on "mvDefectivePixelWriteToDevice" and click on **"Execute"** to write the data to the camera (and hand over the data to the Storing pixel data on the device). To permanently store the data inside the cameras non-volatile memory afterwards "mvDefectivePixelDataSave" must be called as well!

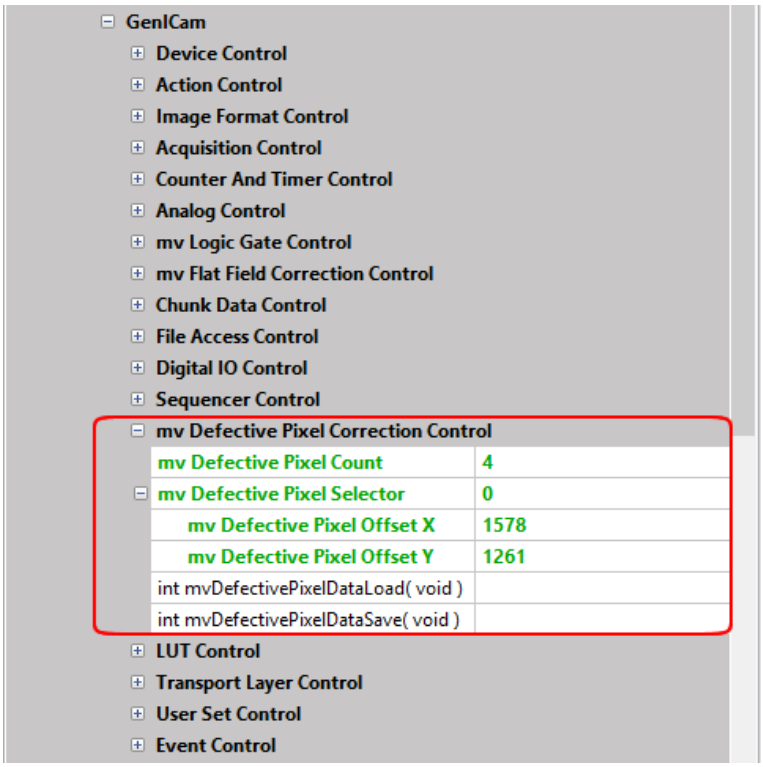


Figure 3: Defective pixel data are written to the camera (since driver version 2.17.1 and firmware version 2.12.406)

While opening the camera, the camera will load the defective pixel data from the camera. If there are pixels in the filter available (via calibration), nevertheless you can load the values from the camera. In this case the values will be merged with the existing ones. I.e., new ones are added and duplicates are removed.

1.19.2.1.4 Defect Pixel correction using mvImpactAcquire After a defect-list is generated, a correction can be performed using mvIMPACT Acquire.

To correct the defective pixels various substitution methods exist:

1. **"Replace 3x1 average"**
   which substitutes the detected defective pixels with the average value from the left and right neighboring pixel (3x1)
2. **"Replace 3x3 median"**
   which substitutes the detected defective pixels with the median value calculated from the nearest neighboring in a 3 by 3 region
3. **"Replace 3x3 Filtered Data Averaged"**
   which substitutes and treats the detected defective pixels as if they have been processed with a 3 by 3 filter algorithm before reaching this filter
   Only recommended for devices which do not offer a defective pixel compensation; packed RGB or packed YUV444 data is needed. See enumeration value dpfmReplaceDefectivePixelAfter3x3Filter in the corresponding API manual for additional details about this algorithm and when and why it is needed

1.19.2.1.5 List-based defect Pixel correction on the camera As described before, it is possible to upload lists of defect pixel onto the camera. Different algorithms can be used to determine whether a pixel is defective or not, which is dependent of how much it is allowed a pixel to deviate, temperature, gain, and exposure time. As described before, the list-based correction is deterministic, meaning it is exactly known which pixels will be corrected.

Anyhow, the list-based correction has some disadvantages:

- A default list is stored in the camera during production, but this might to fit to the target application because of much different temperature / exposure time setting
  → It is necessary to create the list using a detection algorithm (or mvIMPACT Acquire support)
- During time and sensor aging, new defects could/will appear
- It doesn't work in binning/decimation modes
- The memory for storing defective pixels is limited

1.19.2.1.6 Adaptive / algorithm-based correction on the camera In this case, the camera performs detection and correction on-the-fly without using any defect-list.

The adaptive correction addresses the above-mentioned disadvantages of the list-based method. While the correction itself (this is which pixels are used to correct an identified defect) is the same, no static information from a list is used, instead they are detected "on the fly".

To use reasonable thresholds, knowledge of the noise statistics of the sensor is used to detect the outliers. These will be corrected also on the fly. Because this is a dynamic approach, it also works in binning/decimation modes and would also detect new appearing defects.

Nevertheless, there are some disadvantages:

- It is non-deterministic
- Wrong positives can be detected, meaning non-defect pixels could be treated as defect
- If pixels are at the edge of the used thresholds, it could be corrected in one frame, but not in the next

On mvBlueFOX3 cameras, the adaptive correction is always used if:

- There is no list stored on the camera
- Binning or decimation is used

1.19.2.1.7 Flat-Field Correction
Each pixel of a sensor chip is a single detector with its own properties. Particularly, this pertains to the sensitivity as the case may be the spectral sensitivity. To solve this problem (including lens and illumination variations), a plain and equally "colored" calibration plate (e.g. white or gray) as a flat-field is snapped, which will be used to correct the original image. Between flat-field correction and the future application you must not change the optic. To reduce errors while doing the flat-field correction, a saturation between 50 % and 75 % of the flat-field in the histogram is convenient.

Note

Flat-field correction can also be used as a destructive watermark and works for all f-stops.

To make a **flat field correction** following steps are necessary:

1. You need a plain and equally "colored" calibration plate (e.g. white or gray)
2. No single pixel may be saturated - that's why we recommend to set the maximum gray level in the brightest area to max. 75% of the gray scale (i.e., to gray values below 190 when using 8-bit values)
3. Choose a `BayerXY` in *"Setting -> Base -> Camera -> GenICam -> Image Format Control -> PixelFormat"*.
4. Set the (Filter-) **"**`Mode = Calibrate`**"** (Figure 4)
5. Start a Live snap (**"Acquire"** with **"**`Acquisition Mode = Continuous`**"**)
6. Finally, you have to activate the correction: Set the (Filter-) **"**`Mode = On`**"**
7. Save the settings including the correction data via **"**`Action -> Capture Settings -> Save Active Device Settings`**"**
   (Settings can be saved in the Windows registry or in a file)

Note

After having re-started the camera you have to reload the capture settings vice versa.

The filter snaps a number of images (according to the value of the `CalibrationImageCount`, e.g. `5`) and averages the flat-field images to one correction image.

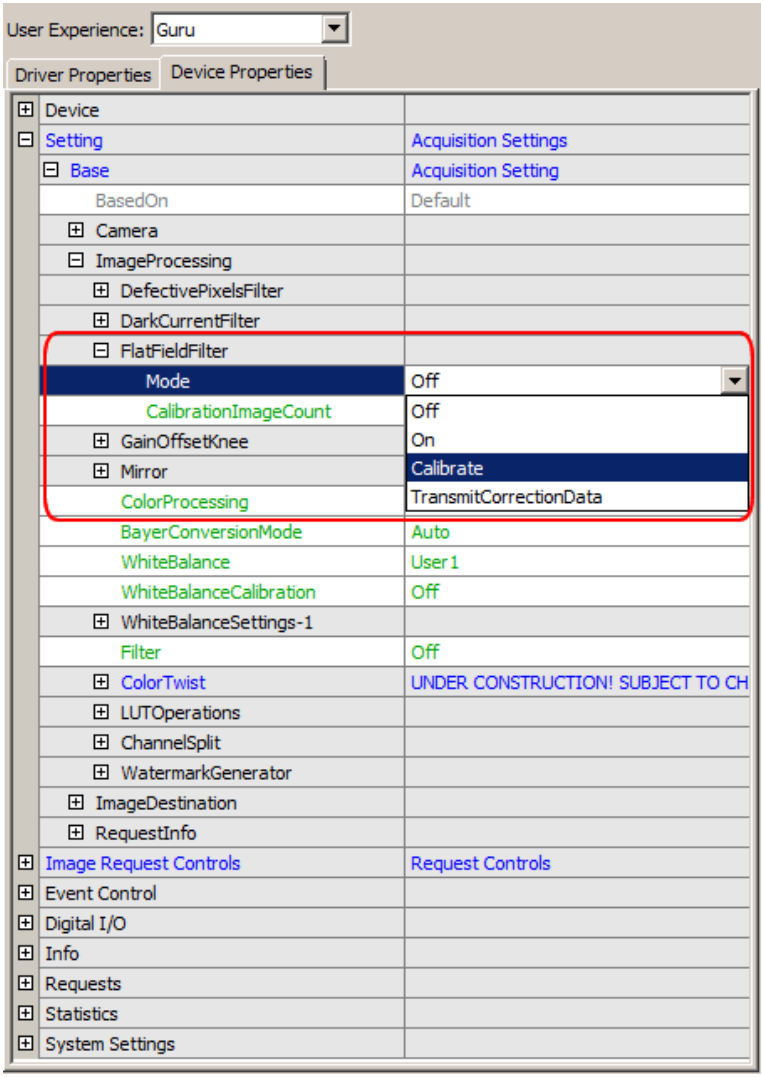


Figure 4: Image corrections: Host-based flat field correction

1.19.2.1.7.1 Host-based Flat-Field Correction With Calibration AOI

In some cases it might be necessary to use just a specific area within the camera's field of view to calculate the correction values. In this case just a specific AOI will be used to calculate the correction factor.

You can set the **"host-based flat field correction"** in the following way:

1. All necessary setting can be found under **"ImageProcessing"->** **"FlatfieldFilter"**.
2. Stop **"Continuous"** acquisition mode.
3. Set **"CalibrationImageCount"** to, for example, 5.
4. Set **"Mode"** to "Calibrate".
5. Set **"CalibrationAoiMode"** to "UseAoi".
6. Set the properties (**"X, Y and W, H"**) appeared under **"CalibrationAOI"** to the desired AOI.
7. Start **"Continuous"** acquisition mode.
8. Finally, you have to activate the correction: Set the **"Mode"** to "On".

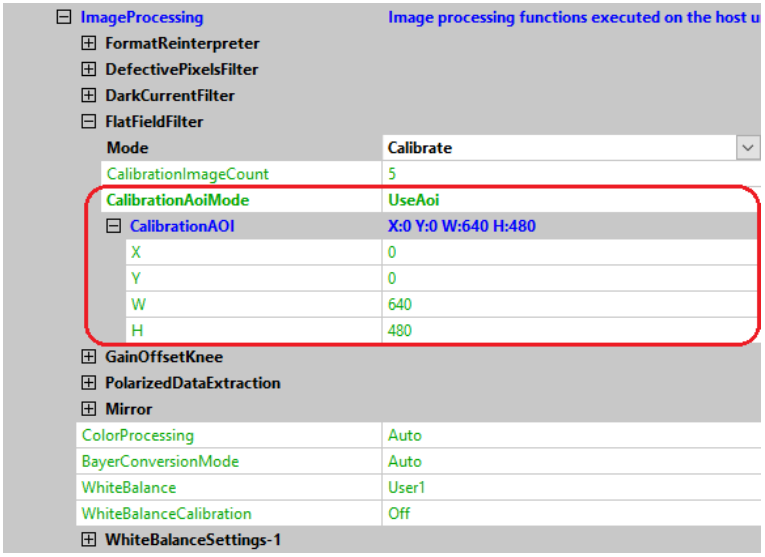


Figure 5: Image corrections: Host-based flat field correction with calibration AOI

1.19.2.1.7.2 Host-based Flat-Field Correction With Correction AOI

In some cases it might be necessary to correct just a specific area in the camera's filed of view. In this case the correction values are only applied to a specific area. For the rest of the image, the correction factor will be just 1.0.

You can set the **"host-based flat field correction"** in the following way:

1. All necessary setting can be found under **"ImageProcessing"** -> **"FlatfieldFilter"**.
2. Stop **"Continuous"** acquisition mode.
3. Set **"CalibrationImageCount"** to, for example, 5.
4. Set **"Mode"** to "Calibrate".
5. Start **"Continuous"** acquisition mode.
6. Now, you have to activate the correction: Set the **"Mode"** to "On".
7. Set **"CorrectionAOIMode"** to "UseAoi".
8. Finally use the properties (**"X, Y and W, H"**) which appeared under **"CorrectionAOI"** to configure the desired AOI.

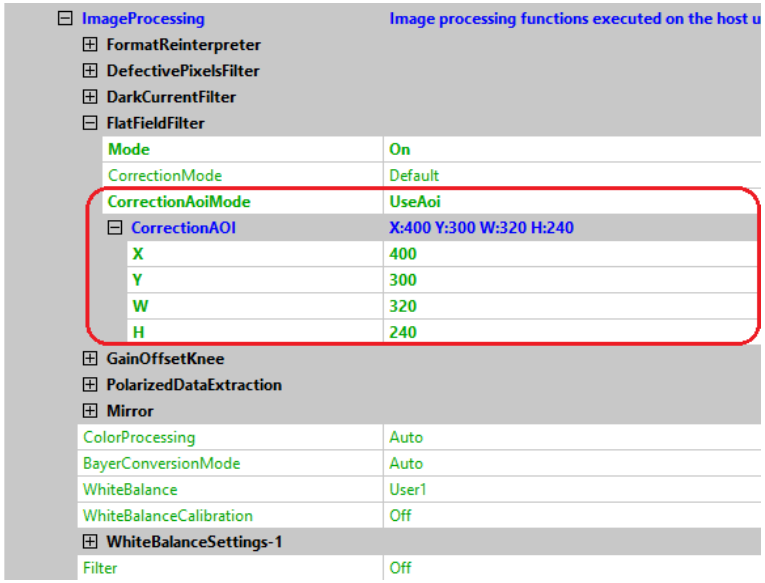


Figure 6: Image corrections: Host-based flat field correction with correction AOI

1.19.2.1.7.3 Camera-based Flat-Field Correction

The camera-based Flat-Field Correction feature supports full AOI and 14 bit to 14 bit correction (12 bit coefficients.). This enables a pixel to pixel correction and saves dozens of % CPU load and lowers latency. To reduce noise averaging of a number of *"mv Flat-Field Correction Calibration Images"* is possible. A correction image in the camera is then calculated. This may take some time / number of images, however, the camera blinks green. One correction image can be stored for all user settings.

The camera-based Flat-Field Correction is independent of the offset and uses a run-mode trigger (e.g. external trigger).

You can set the **"camera-based flat field correction"** in the following way:

1. Set **"mv Flat-Field Correction Calibration Image Count"** to, for example, 5.
2. This will average 5 images before calculating the FFC factors to reduce impact of noise.
3. Stop **"Continuous"** acquisition mode, then right click on "int mvFFCCalibrate()" -> **"Execute"**.
4. Finally, you have to activate the correction: Set the "`mv Flat-Field Correction Enable = 1`".

Depending on the sensor, this need some time, because the data is stored in the internal flash memory (yellow LED lights up).

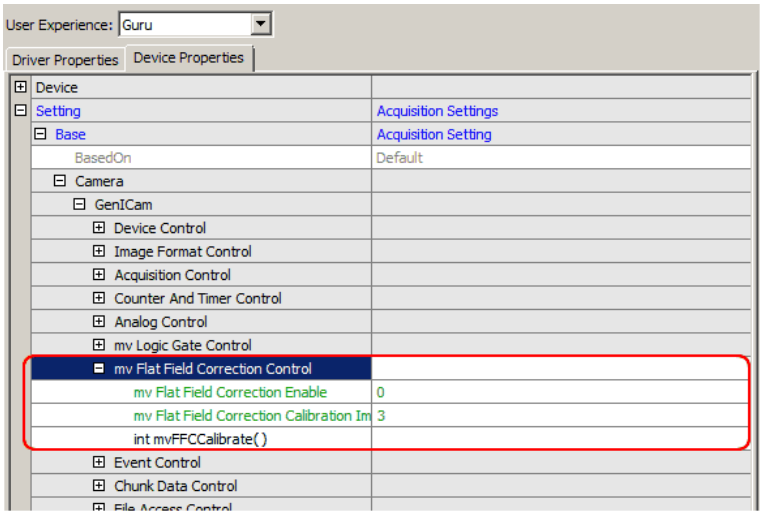


Figure 7: wxPropView - settings

Example

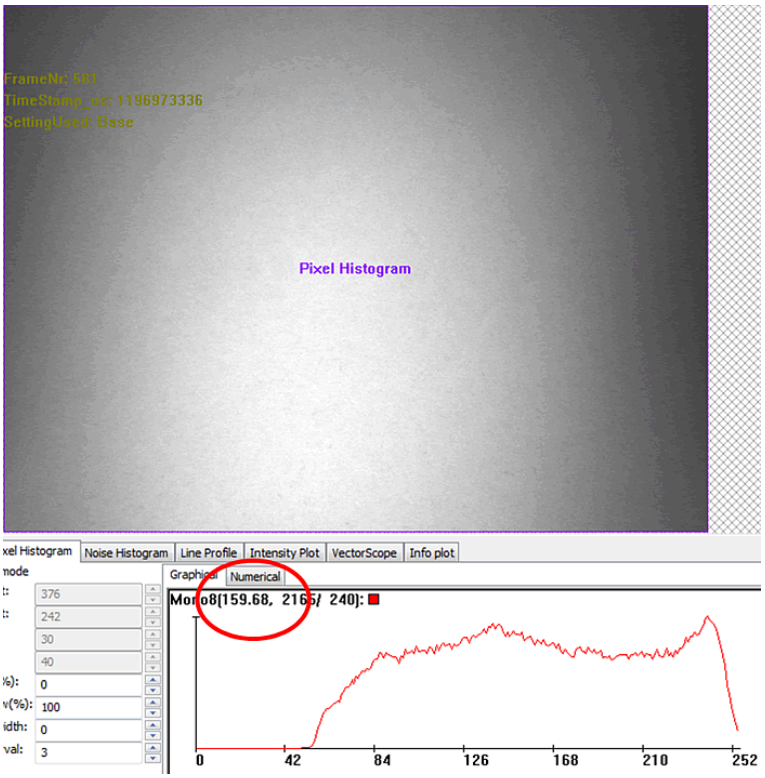


Figure 8: wxPropView - inhomogeneous light / pixel histogram

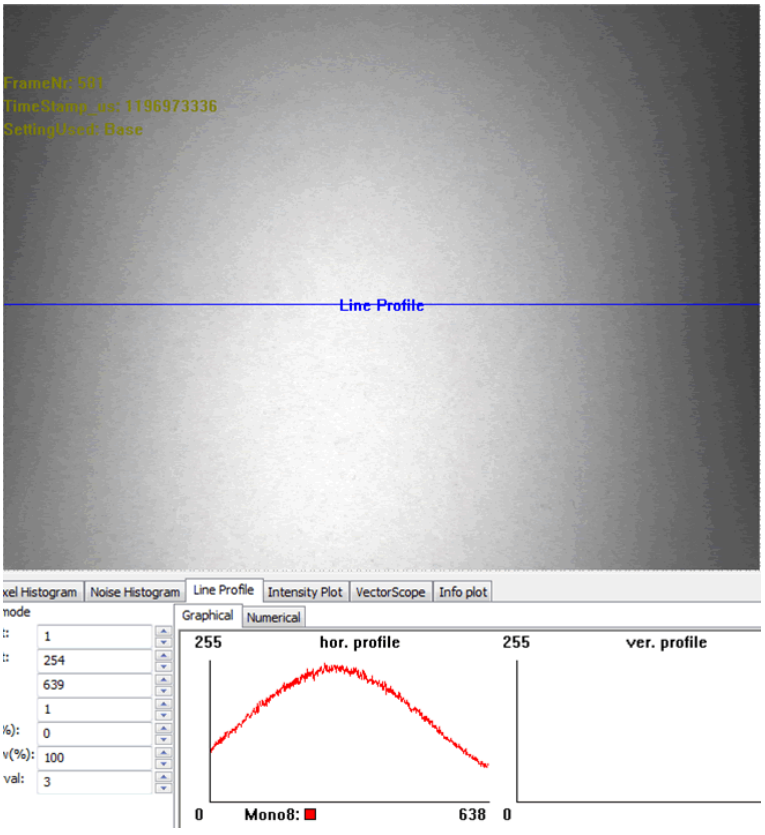


Figure 9: wxPropView - inhomogeneous light / pixel histogram (horizontal)

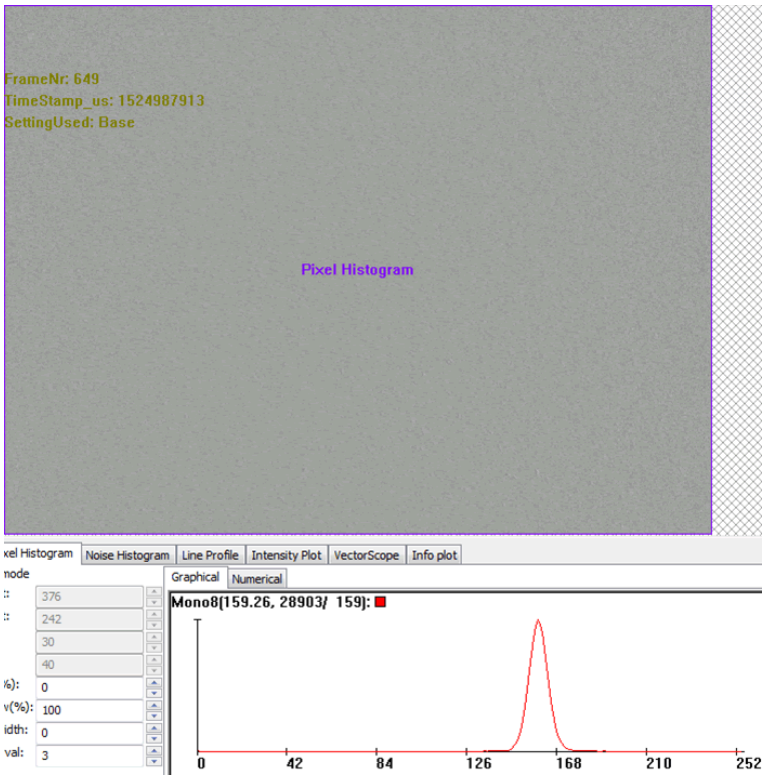


Figure 10: wxPropView - compensated light / pixel histogram

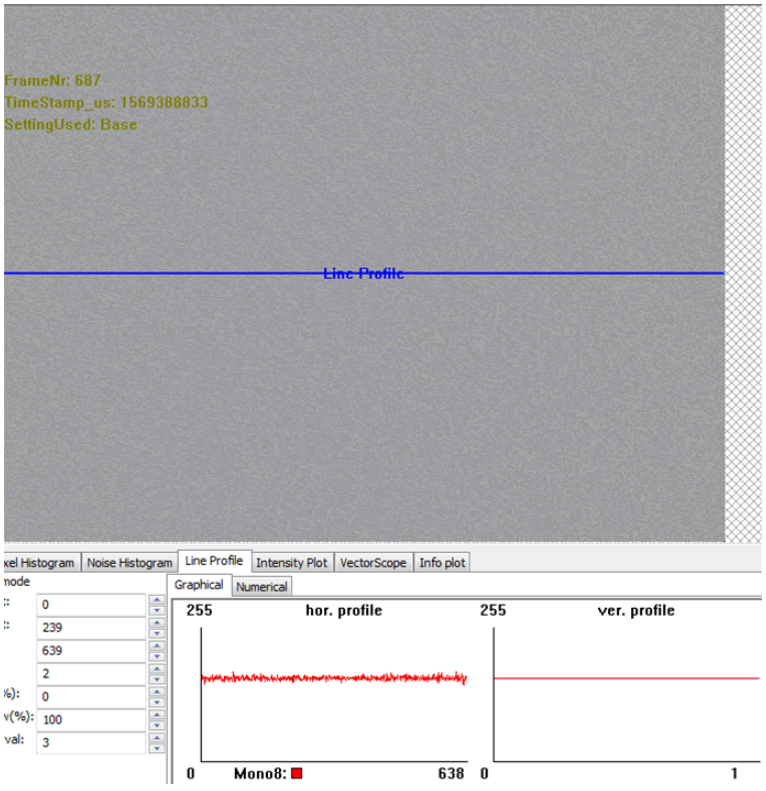


Figure 11: wxPropView - compensated light / pixel histogram (horizontal)

1.19.2.1.7.4 Camera-based Flat-Field Correction With Calibration AOI In some cases it might be necessary to use just a specific area within the camera's field of view to calculate the correction values. In this case just a specific AOI will be used to calculate the correction factor.

You can set the **"camera-based flat field correction with AOI"** in the following way:

1. Start **"Continuous"** acquisition mode.
2. Use the properties (**"X, Y and W, H"**) to configure the desired AOI for FFC calculation.
3. Set **"mv Flat-Field Correction Calibration Image Count"** to, for example, 5.
4. This will average 5 images before calculating the FFC factors to reduce impact of noise.
5. Stop **"Continuous"** acquisition mode, then right click on "int mvFFCCalibrate()" -> **"Execute"**.
6. Use the properties (**"X, Y and W, H"**) to setup full picture size or desired AOI.
7. Finally, you have to activate the correction: Set the "`mv Flat-Field Correction Enable = 1`".

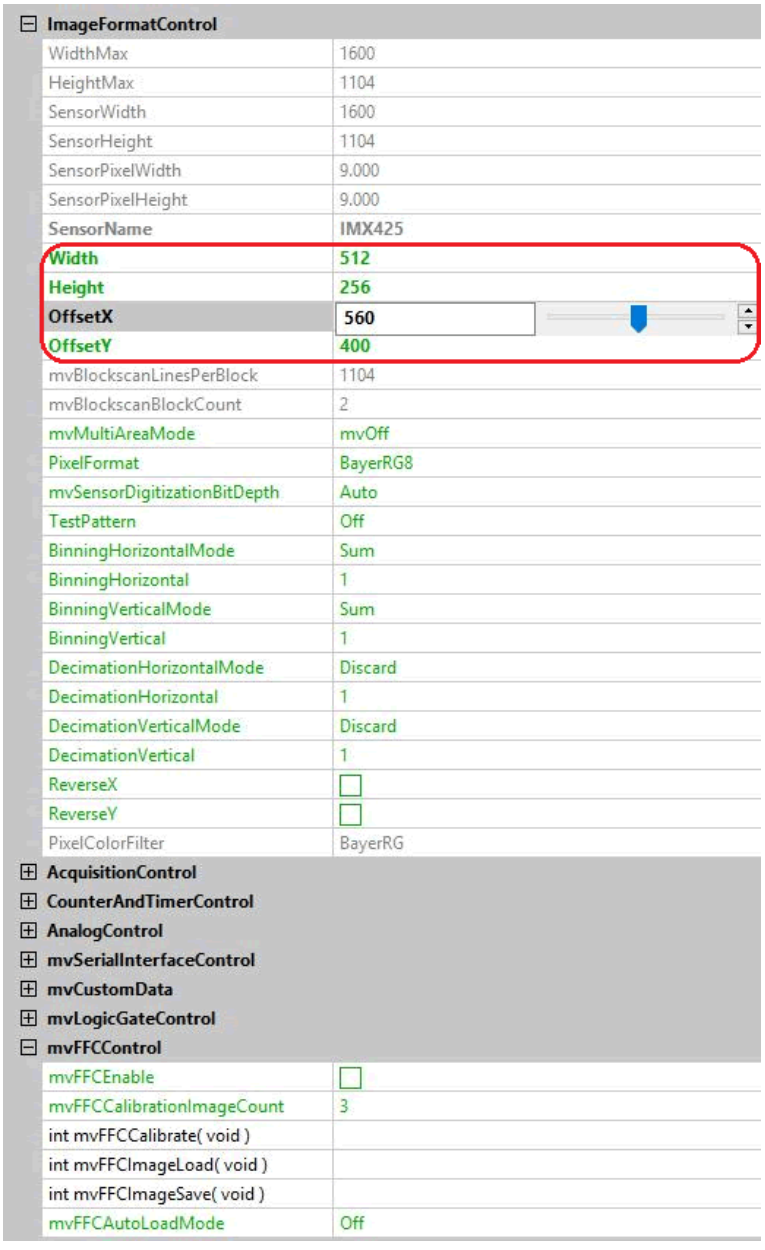


Figure 12: wxPropView - set calculation AOI for flat field correction

If FFC is working as expected it is possible to store the FFC calibration values in the camera if property **mv↩ FFCImageSave** is available. Depending on the sensor, this will need some time, because the data is stored in the internal flash memory (yellow LED lights up). The FFC values can then be loaded from flash with the command **mvFFCImageLoad**. If the values should be loaded every time the camera is powered up this could be done by setting "`mvFFCAutoLoadMode = On`"

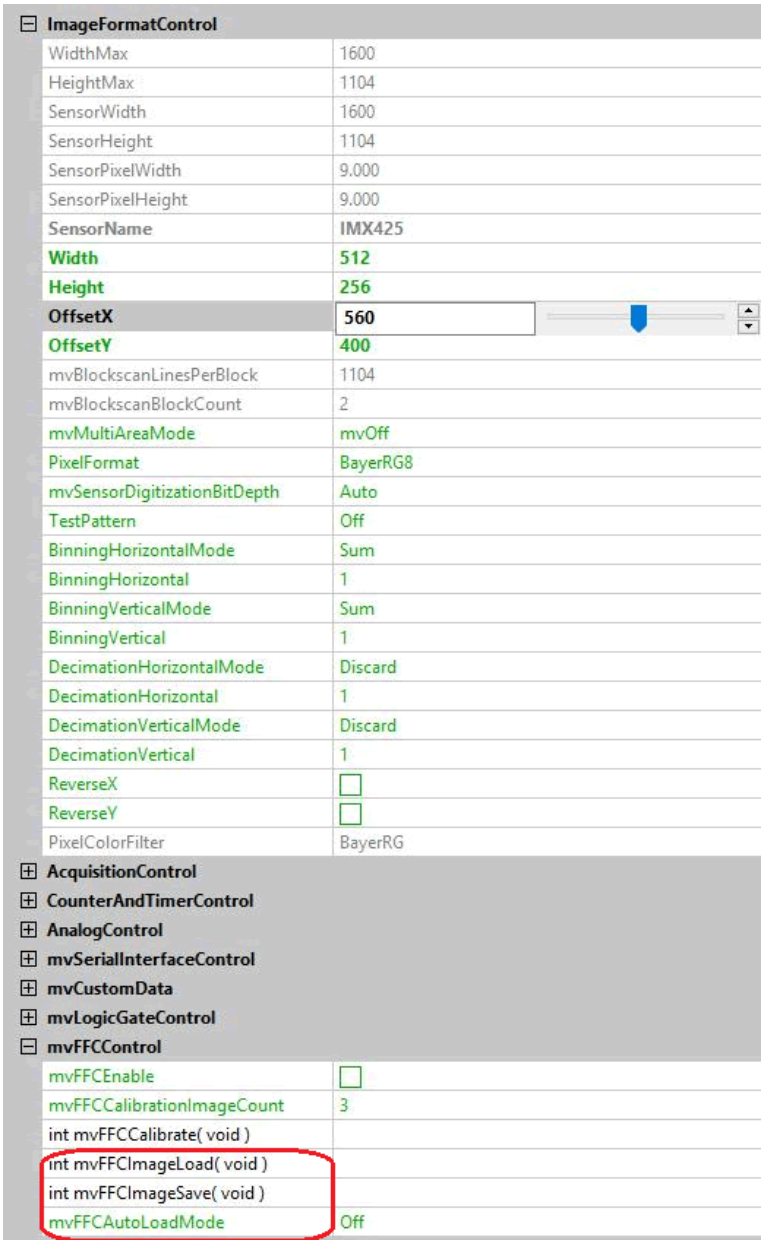


Figure 13: wxPropView - store/load FFC values

Note

Color cameras with FW version smaller than 2.40 use a multiplication factor of 1.3 on the correction factor of each color. This factor minimizes color artifacts when the scene gets brighter. Newer Firmware versions use a multiplication factor of 1.0 and to minimize color artifacts use above described AOI method.

1.19.2.2 Optimizing the color/luminance fidelity of the camera

Purpose of this chapter is to optimize the color image of a camera, so that it looks as natural as possible on different displays and for human vision.

This implies some linear and nonlinear operations (e.g. display color space or Gamma viewing LUT) which are normally not necessary or recommended for machine vision algorithms. A standard monitor offers, for example, several display modes like *sRGB*, *"Adobe RGB"*, etc., which reproduce the very same color of a camera color differently.

It should also be noted that users can choose for either

- camera based settings and adjustments or
- host based settings and adjustments or
- a combination of both.

Camera based settings are advantageous to achieve highest calculating precision, independent of the transmission bit depth, lowest latency, because all calculations are performed in FPGA on the fly and low CPU load, because the host is not invoked with these tasks. These camera based settings are

- **gamma correction** (p. 300)
- **negative gain** / **gain** (p. 300)
- **look-up table (LUT)** (p. 300)
- **white balance** (p. 305)
- **offset** (p. 307)
- **saturation and color correction** (p. 309)

Host based settings save transmission bandwidth at the expense of accuracy or latency and CPU load. Especially performing gain, offset, and white balance in the camera while outputting RAW data to the host can be recommended.

Of course host based settings can be used with all families of cameras (e.g. also mvBlueFOX).

Host based settings are:

- look-up table (*LUTOperations*)
- color correction (*ColorTwist*)

The **block diagram of the camera** (p. 116) shows where the different settings can be found in the data flow of the image data.

To show the different color behaviors, we take a color chart as a starting point:

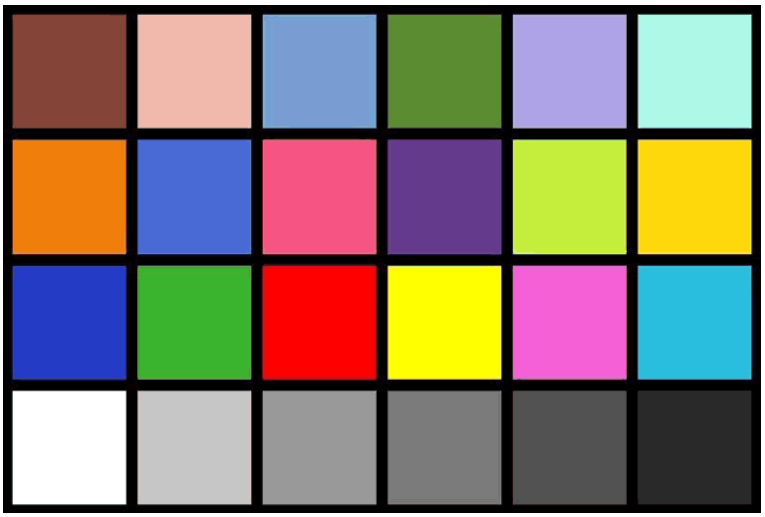


Figure 1: Color chart as a starting point

If we take a *SingleFrame* image without any color optimizations, an image can be like this:



Figure 2: SingleFrame snap without color optimization



Figure 3: Corresponding histogram of the horizontal white to black profile

As you can see,

- saturation is missing,
- white is more light gray,
- black is more dark gray,
- etc.

Note

You have to keep in mind that there are two types of images: the one generated in the camera and the other one displayed on the computer monitor. Up-to-date monitors offer different display modes with different color spaces (e.g. sRGB). According to the chosen color space, the display of the colors is different.

The following figure shows the way to a perfect colored image

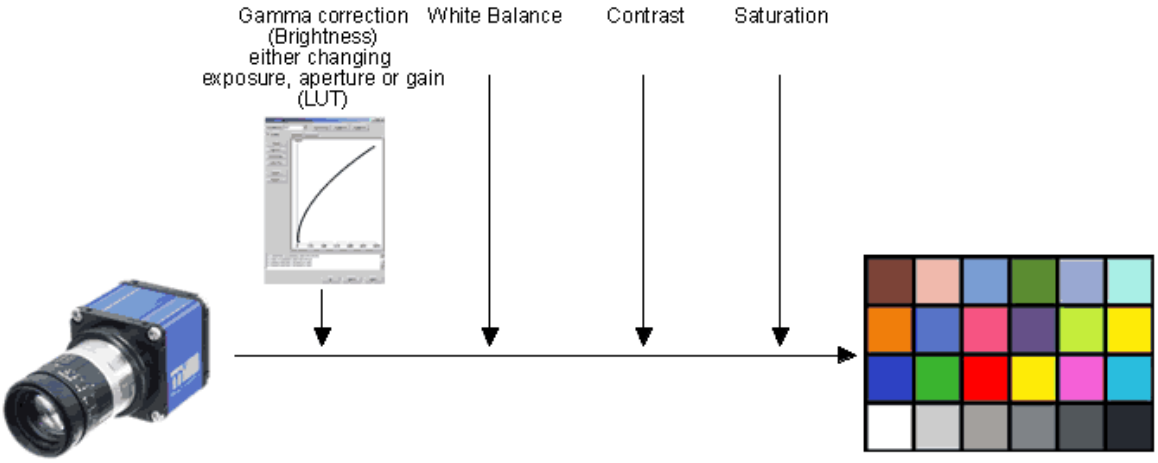


Figure 4: The way to a perfect colored image

including these process steps:

1. Do a **Gamma correction (Luminance)** (p. 300),
2. make a **White balance** (p. 305) and
3. Improve the **Contrast** (p. 307).
4. Improve **Saturation** (p. 309), and use a **"color correction matrix"** for both
   (a) the sensor and / or
   (b) the monitor.

The following sections will describe the single steps in detail.

1.19.2.2.1 Step 1: Gamma correction (Luminance)

First of all, a **Gamma correction** can be performed to change the image in a way how humans perceive light and color.

To do this, you have the following 3 possibilities:

- set Gamma using Gamma feature in AnalogControl,
- set Gamma using a LUT in LUTControl or
- set Gamma using a LUT in ImageProcessing.

Note

The first two possibilities are running on the camera, whereas the third is running on host system

The first and easiest way to set a Gamma is using the gamma feature in AnalogControl (*"Setting -> Base -> Camera -> AnalogControl"*)

mvGammaSelector is used to switch between a fixed mvSRGB gamma curve and mvuser which enables the Gamma property to set gamma between a value of 0.1 and 10.0 The Gamma feature is working on the camera and enabled by setting mvGammaEnable

Note

mvGammaEnable can only be set if no LUT in LUTControl is active

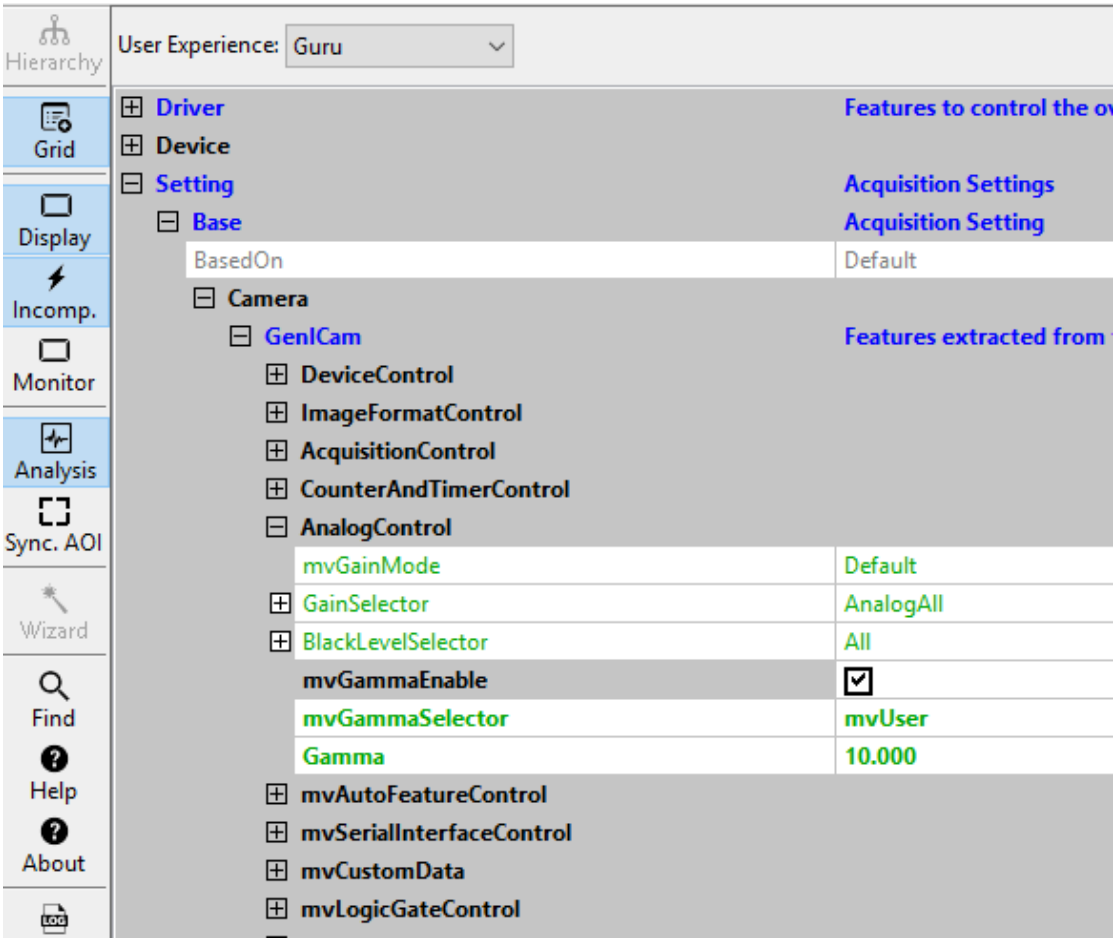


Figure 5: Gamma using AnalogControl

Note

To avoid artifacts while changing the Gamma value, stop Acquisition before changing values and afterwards, start Acquisition

The second way is setting Gamma via LUTControl using **wxPropView** (p. 139), here, the following steps have to be done:

1. Click on *"Setting -> Base -> Camera -> GenICam -> LUT Control -> LUT Selector"*.
2. Afterwards, click on **"Wizard"** to start the LUT Control wizard tool.
   The wizard will load the data from the camera.

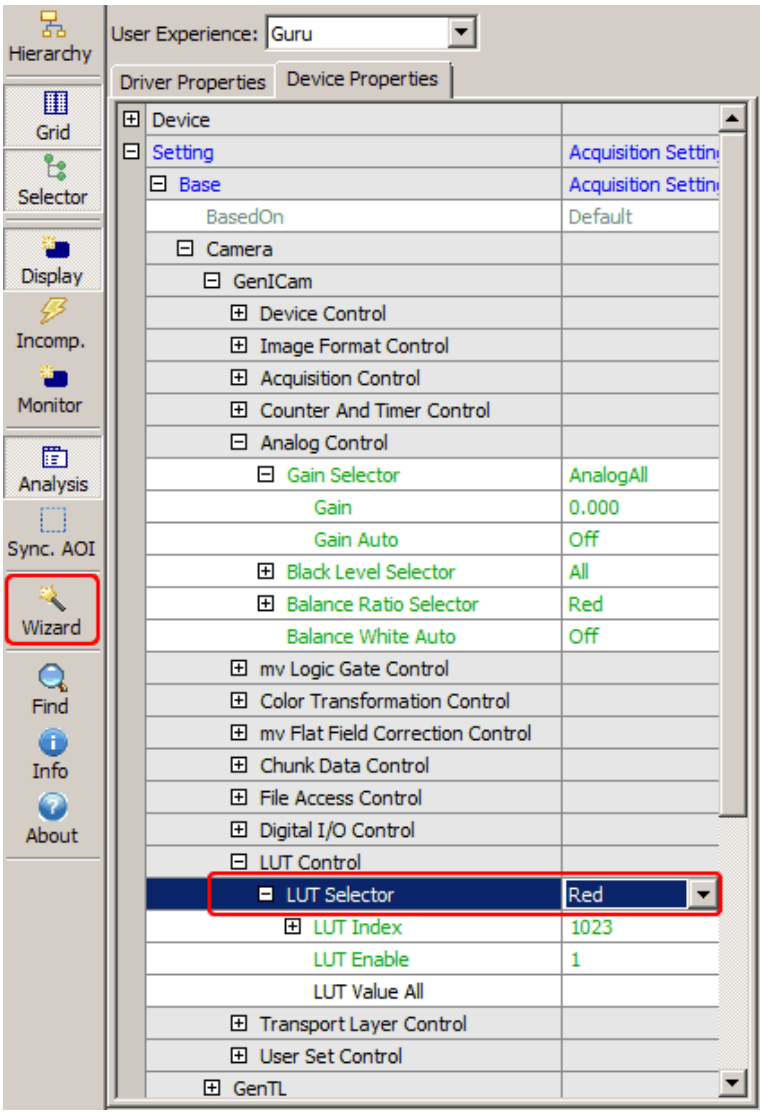


Figure 6: Selected LUT Selector and click on wizard will start wizard tool

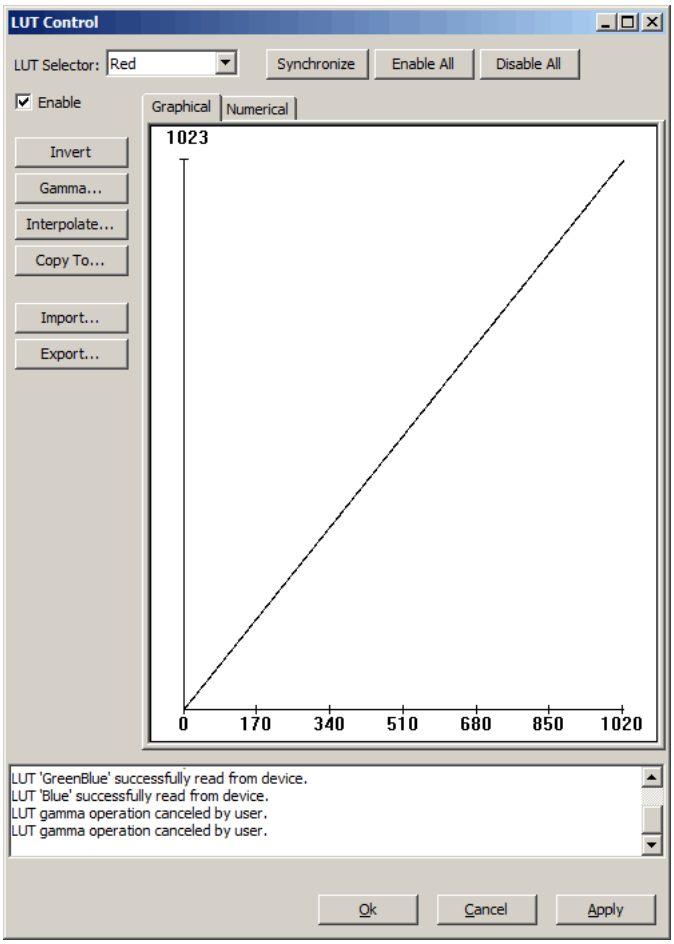


Figure 7: LUT Control

3. Now, click on the **"Gamma..."** button
4. and enter e.g. *"2.2"* as the Gamma value:

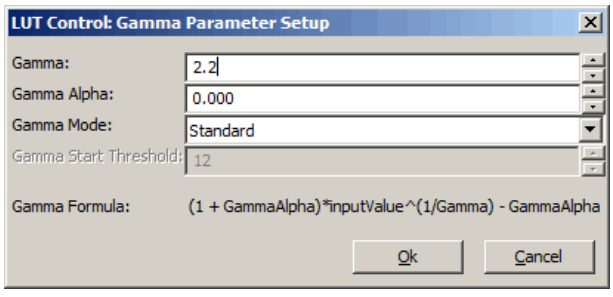


Figure 8: Gamma Parameter Setup

5. Then, click on **"Copy to..."** and select *"All"* and
6. and click on **"Enable All"**.
7. Finally, click on **Synchronize** and play the settings back to the device (via *"Cache -> Device"*).

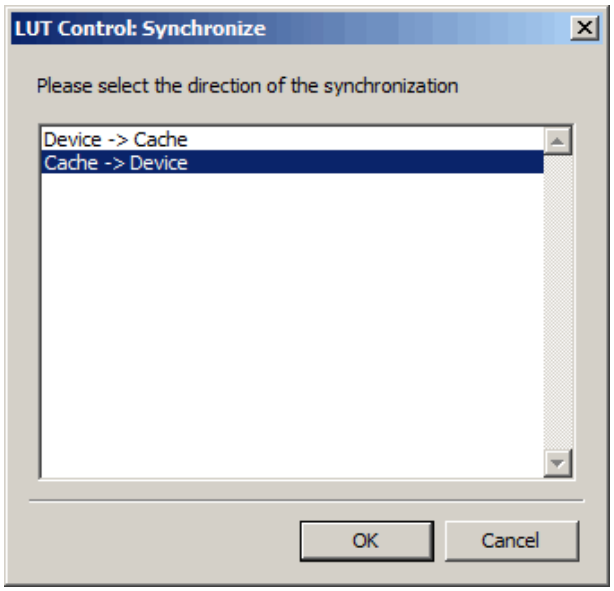


Figure 9: Synchronize

After gamma correction, the image will look like this:

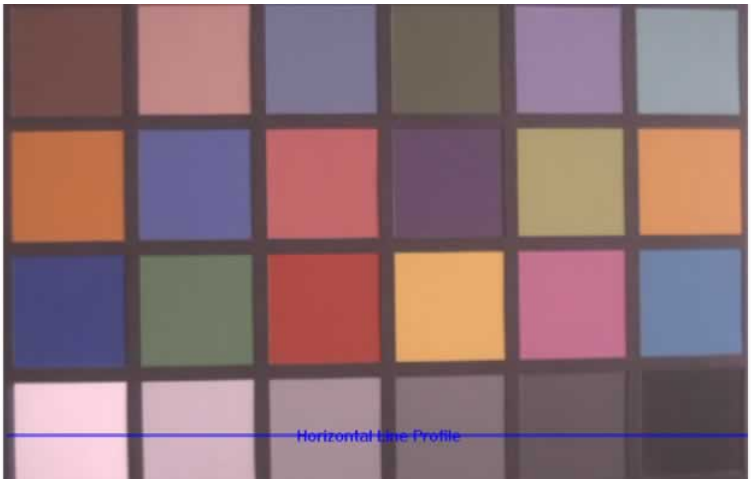


Figure 10: After gamma correction



Figure 11: Corresponding histogram after gamma correction

The third way to set Gamma, is gamma correction via (*"Setting -> Base -> ImageProcessing -> LUTControl"*). Here, the changes will affect the 8 bit image data and the processing is running on the CPU of the host system:

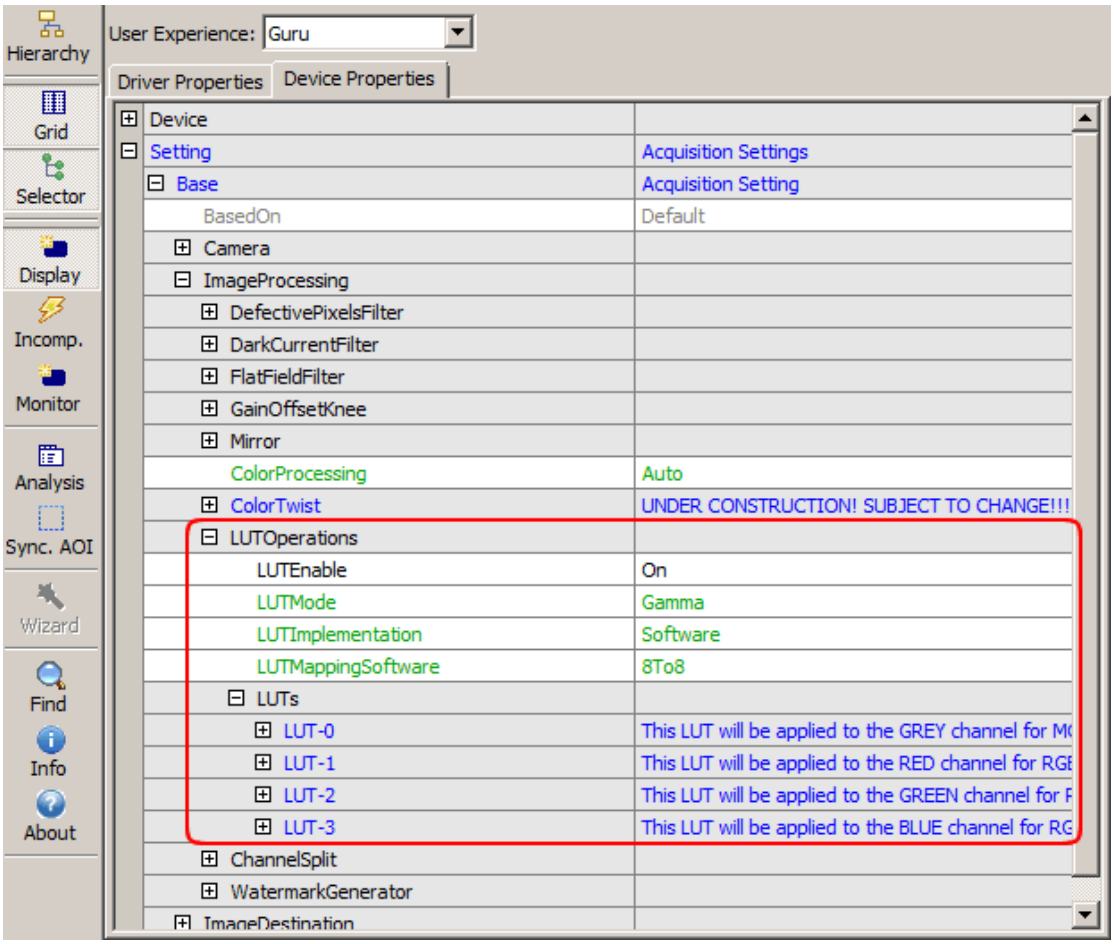


Figure 12: LUTControl dialog

Just set *"LUTEnable"* to **"On"** and adapt the single LUTs like (LUT-0, LUT-1, etc.).

1.19.2.2.2 Step 2: White Balance

As you can see in the histogram, the colors red and blue are above green. Using green as a reference, we can optimize the white balance via *"Setting -> Base -> Camera -> GenICam -> Analog Control -> Balance Ratio Selector"* (*"Balance White Auto"* has to be *"Off"*):

1. Just select *"Blue"* and
2. adjust the *"Balance Ratio"* value until the blue line reaches the green one.

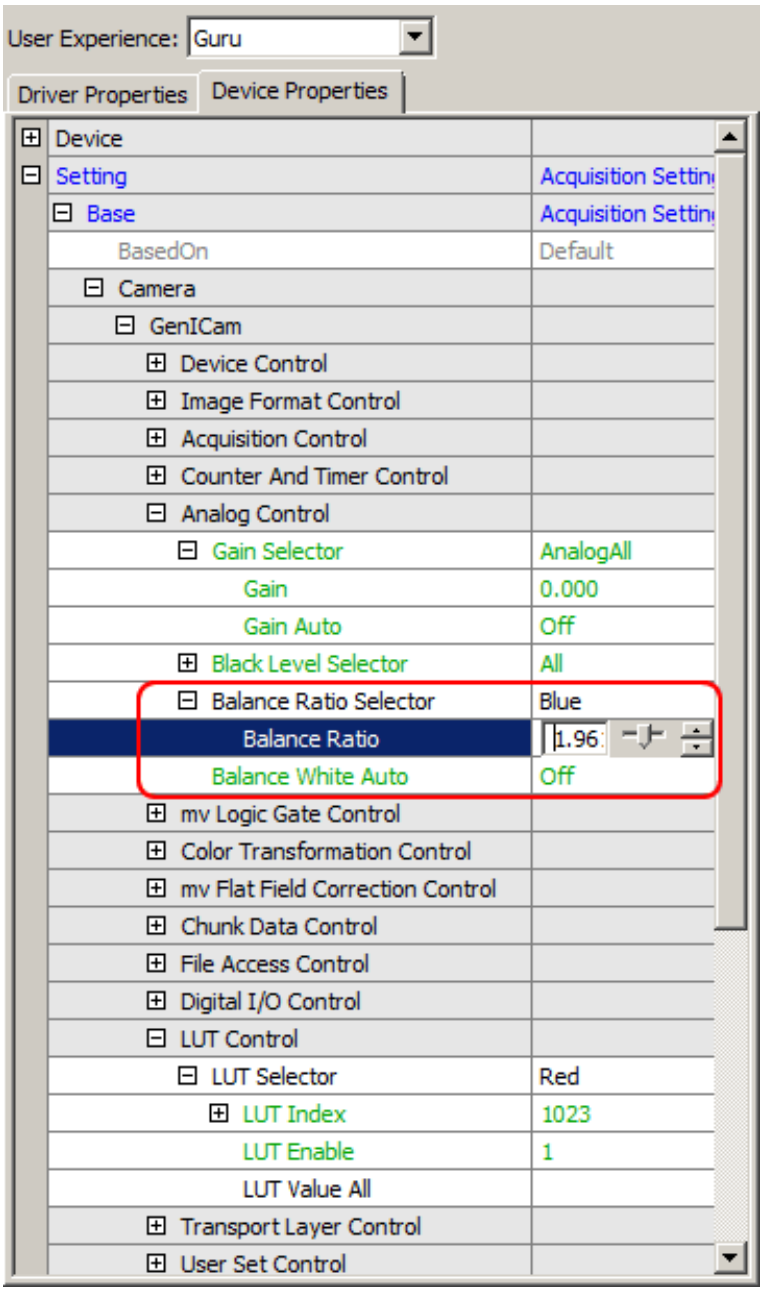


Figure 13: Optimizing white balance

3. Repeat this for *"Red"*.

After optimizing white balance, the image will look like this:

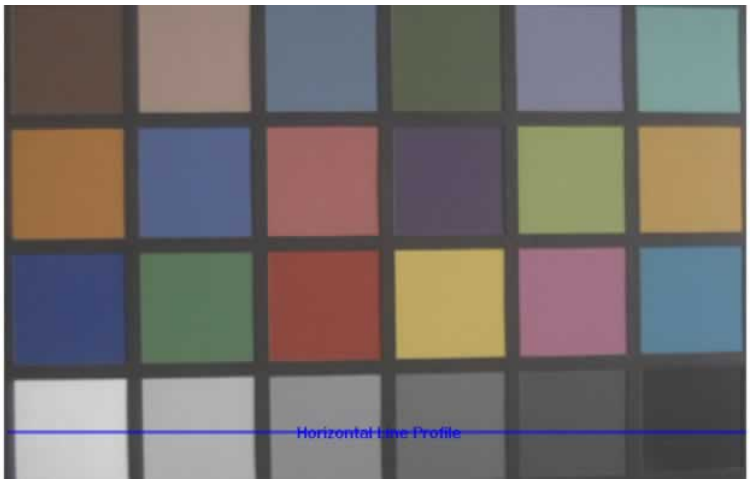


Figure 14: After white balance

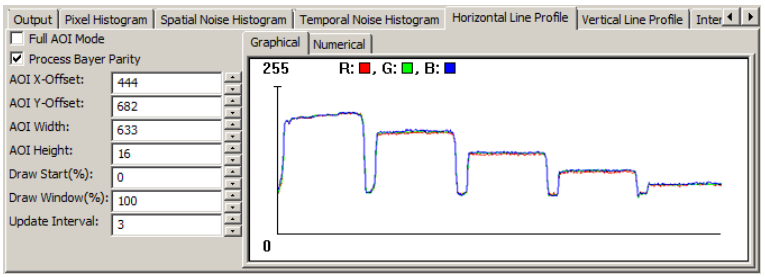


Figure 15: Corresponding histogram after white balance

1.19.2.2.3 Step 3: Contrast

Still, black is more a darker gray. To optimize the contrast you can use *"Setting -> Base -> Camera -> GenICam -> Analog Control -> Black Level Selector"*:

1. Select *"DigitalAll"* and
2. adjust the *"Black Level"* value until black seems to be black.

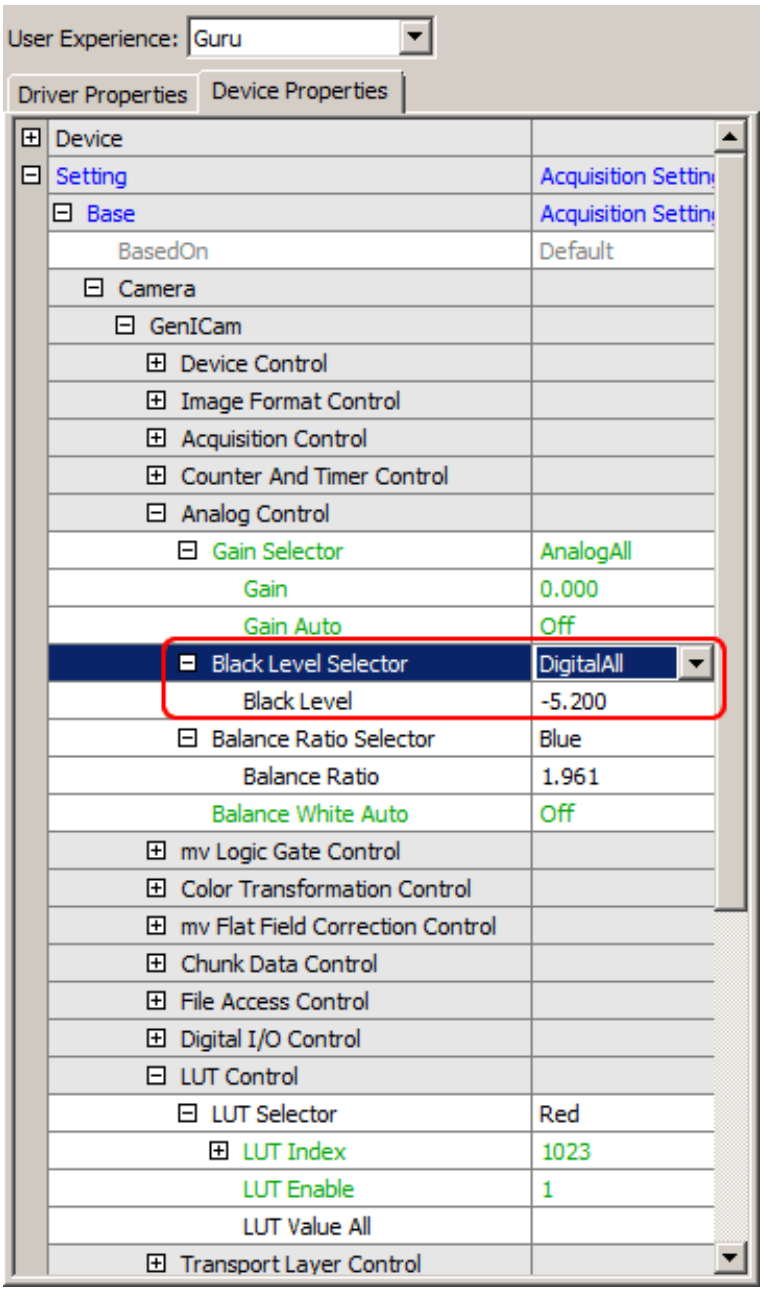


Figure 16: Back level adjustment

The image will look like this now:

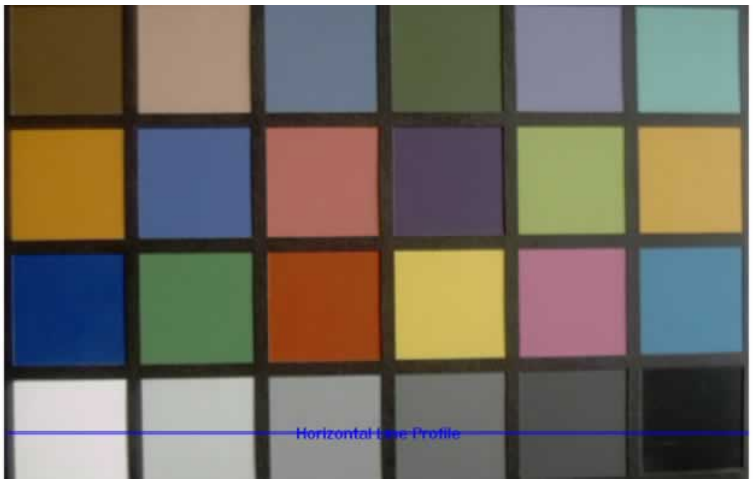


Figure 17: After adapting contrast



Figure 18: Corresponding histogram after adapting contrast

1.19.2.2.4 Step 4: Saturation and Color Correction Matrix (CCM)

Still saturation is missing. To change this, the **"Color Transformation Control"** can be used (*"Setting -> Base -> Camera -> GenICam -> Color Transformation Control"*).

Note

In our optical laboratory we determine the best correction data for the sRGB color space for each sensor type and these parameters are shipped with our cameras. If you want to use these parameters, you have to activate them using the wizard of the Color Transformation Control.

1. Click on **"Color Transformation Enable"** and
2. click on **"Wizard"** to start the saturation via *"Color Transformation Control"* wizard tool (since firmware version 1.4.57).

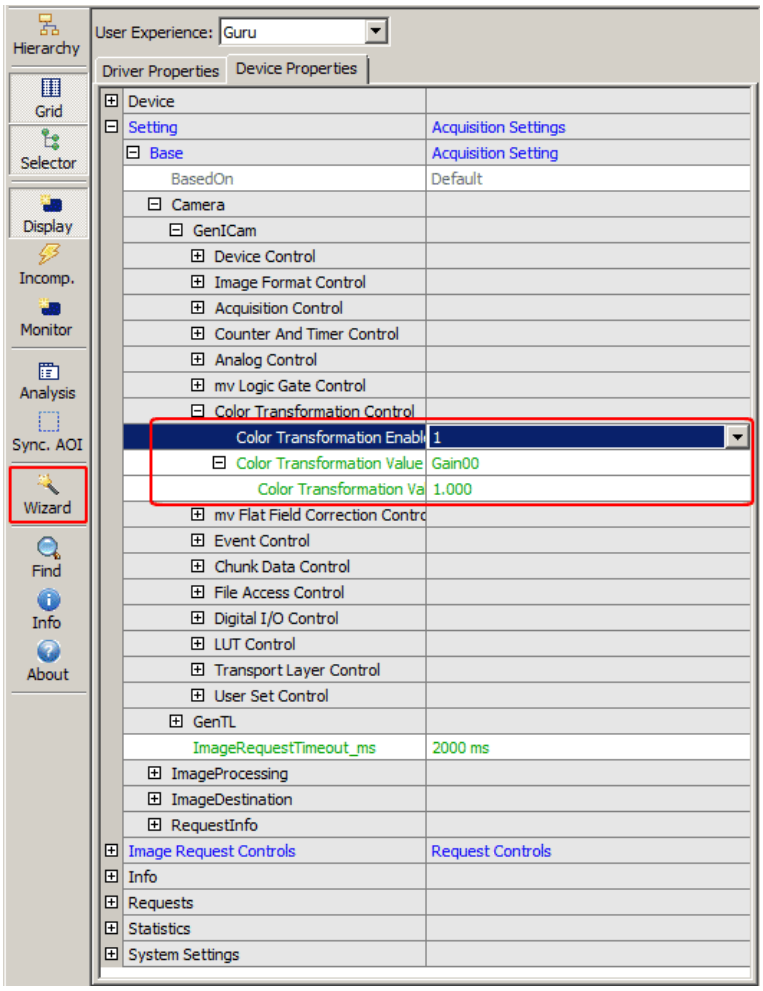


Figure 19: Selected Color Transformation Enable and click on wizard will start wizard tool

3. Now, you can adjust the saturation e.g. *"1.1"*.

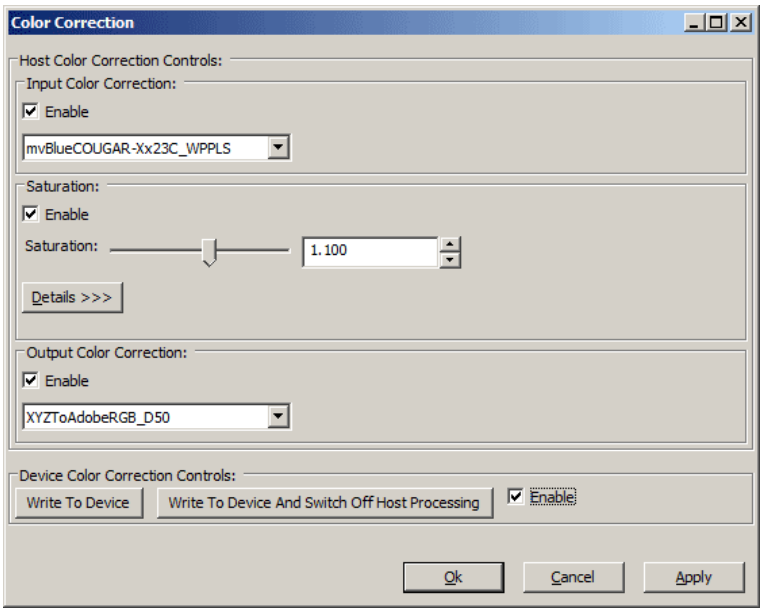


Figure 20: Saturation Via Color Transformation Control dialog

4. Afterwards, click on **"Enable"**.

5. Since driver version 2.2.2, it is possible to set the special color correction matrices at
   (a) the input (sensor),
   (b) the output side (monitor) and
   (c) the saturation itself using this wizard.
6. Select the specific input and output matrix and
7. click on **"Enable"**.
8. As you can see, the correction is done by the host by default (*"Host Color Correction Controls"*). However, you can decide, if the color correction is done by the device by clicking on **"Write To Device And Switch Off Host Processing"**. The wizard will take the settings of the *"Host Color Correction Controls"* and will save it in the device.
9. Finally, click on **"Apply"**.

After the saturation, the image will look like this:

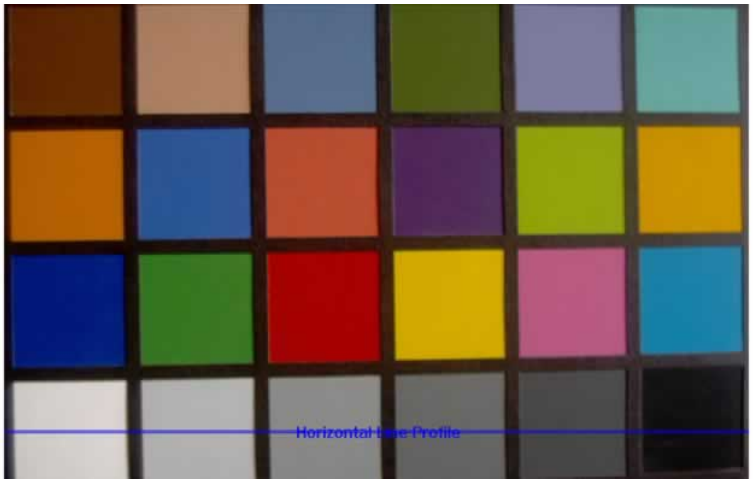


Figure 21: After adapting saturation

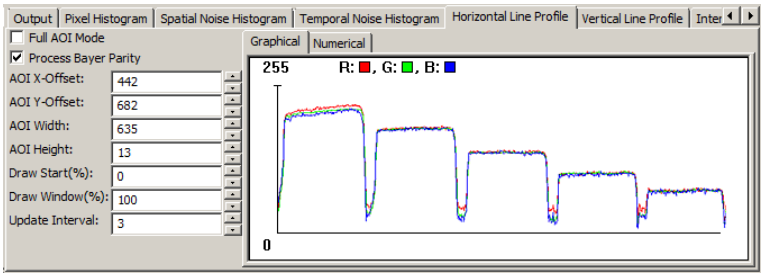


Figure 22: Corresponding histogram after adapting saturation

Note

As mentioned above, you can change the saturation and the color correction matrices via (*"Setting -> Base -> ImageProcessing -> ColorTwist"*). Here, the changes will affect the 8 bit image data and the processing needs the CPU of the host system:

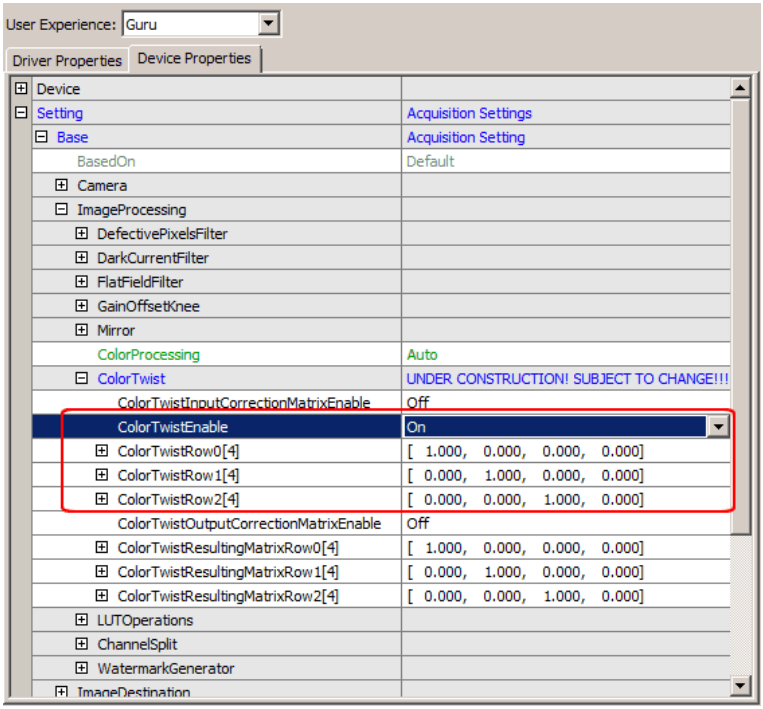


Figure 23: ColorTwist dialog

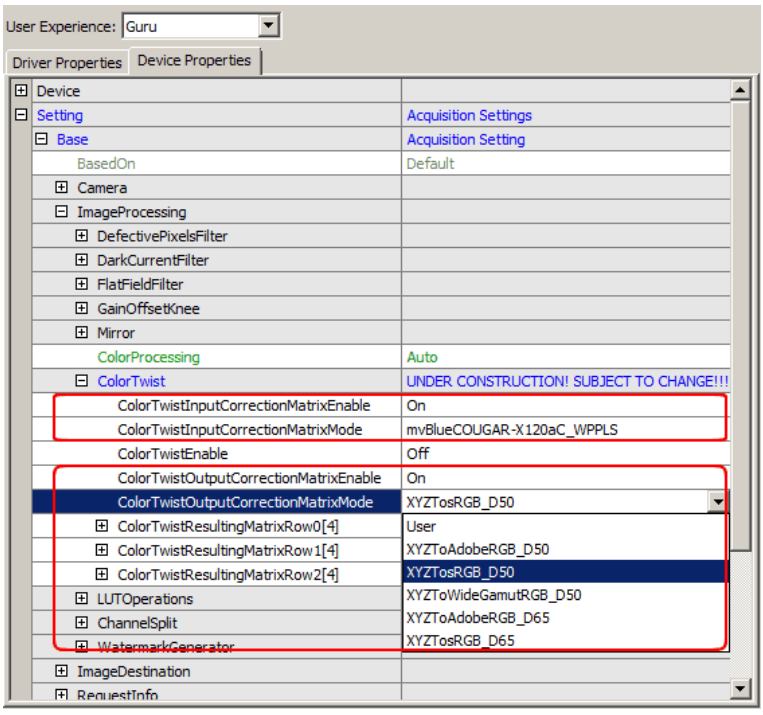


Figure 24: Input and output color correction matrix

1.19.2.3 Reducing noise by frame averaging

1.19.2.3.1 What is frame average?

As the name suggests, the functionality averages the gray values of each pixel using the information of subsequent frames. This can be used to

- reduce the noise in an image and
- compensate motion in an image.

MATRIX VISION implemented a dynamic version of the frame averaging in the FPGA, which not need any CPU of the host system. However, this mode is only available for the

- mvBlueCOUGAR-X2xx cameras with larger FPGAs.

1.19.2.3.2 Dynamic mode of mvBlueCOUGAR-X2xx

This mode uses an adaptive recursive filter with an average slope. The slope sets the amount of new image versus averaged image in relation to the gray scale variation of the pixel. With it, static noise can be removed at full bit depth and full frame rate:

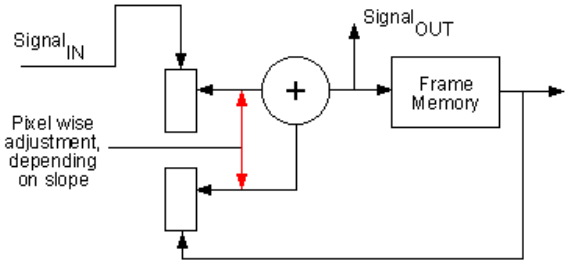


Figure 2: Frame average: Functional principle

This method is well known and is used in the same or a similar way in all flat screen televisions. The amount of de-noising can be set with the slope factor: the smaller the value, the greater the feedback and therefore the de-noising but also the motion blur in the image:

```
Slope: 256 = 100 % pixel_difference = 100 % signal_in
```

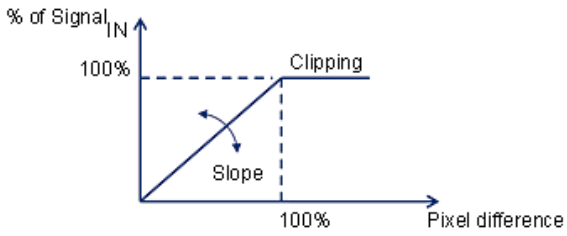


Figure 3: Frame average: Slope

There are no delays with this option because de-noising is recursive and the $Signal_{OUT}$ is extracted before the frame memory.

1.19.2.3.2.1 Using wxPropView

Using the dynamic frame average mode, you have to do the following step:

1. Start **wxPropView** (p. 139) and
2. connect to the camera.
3. Then specify in *"Setting -> Base -> Camera -> GenICam -> Device Control"* which processing unit of the camera should do the frame averaging, e.g. unit 0 should do the frame averaging
   "mv Device Processing Unit Selector = 0"
   "mv Device Processing Unit = mvFrameAverage".
   Afterwards, **"mv Frame Average Control"** is available.
4. Now open *"Setting -> Base -> Camera -> GenICam -> mv Frame Average Control"* and
5. set the slope, e.g. 5000: **"mv Frame Average Slope = 5000"**.
6. Activate frame averaging by setting **"mv Frame Average Enable = 1"**.

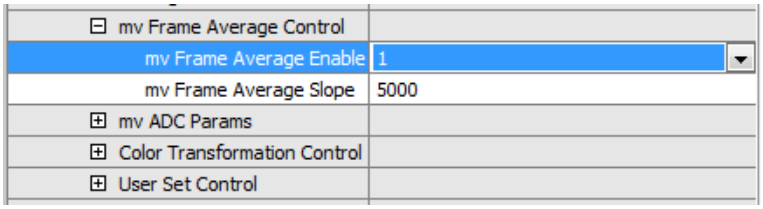


Figure 4: wxPropView: Setting the dynamic frame average

For **"static images"**, setting average slope to small numbers (10 .. 1000) gives the best noise enhancement at the expense of motion blur.

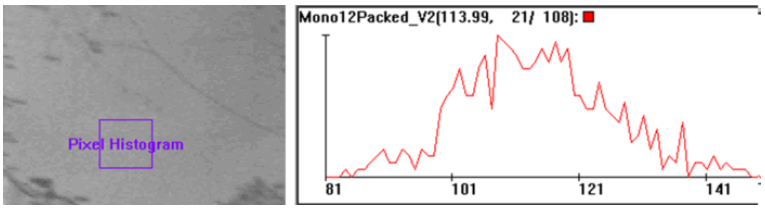


Figure 5: wxPropView - noisy image

For **"dynamic images"**, setting average slope to higher values (1000 .. 5000) reduces motion blur at the expense of noise enhancement. This is especially needed in video application to enhance noise.

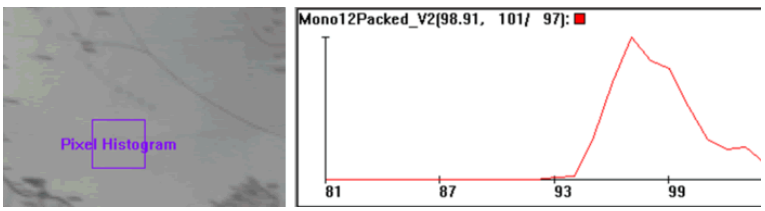


Figure 6: wxPropView - reduced noise in image

1.19.2.4 Setting a flicker-free auto expose and auto gain

1.19.2.4.1 Introduction

In order to prevent oscillations it is important to adapt the camera frequency to the frequency of AC light.

This is, for example, in

- Europe 50 cycles (100 fluctuations/s) whereas in
- USA, Japan and other countries it is 60 Hz.

This means the camera must strictly be coupled to this frequency. In conjunction with auto exposure this can only be maintained by using a timer based generation of external trigger pulses. This is a behavior of both sensor types: CCD and CMOS.

Note

It is not enough to use *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> Acquisition Frame Rate"* for this, as there are small fluctuations in this frame rate if the exposure time changes. These fluctuations lead to oscillations (see settings marked with red boxes in Figure 1). The *"Acquisition Frame Rate"* will only provide the exact frame rate if auto exposure is turned off.

As shown in Figure 1, it is possible to set (*"mv Exposure Auto Mode"*) which part of the camera handles the auto expose (device or the sensor itself; pink boxes). Using *"mvSensor"* as *"mv Exposure Auto Mode"*, it is possible to avoid oscillations in some cases. The reason for this behavior is that you can set more parameters like *"mv Exposure Auto Delay Images"* in contrast to *"mvDevice"*. However, as mentioned above it is recommended to use a timer based trigger when using auto expose together with continuous acquisition.

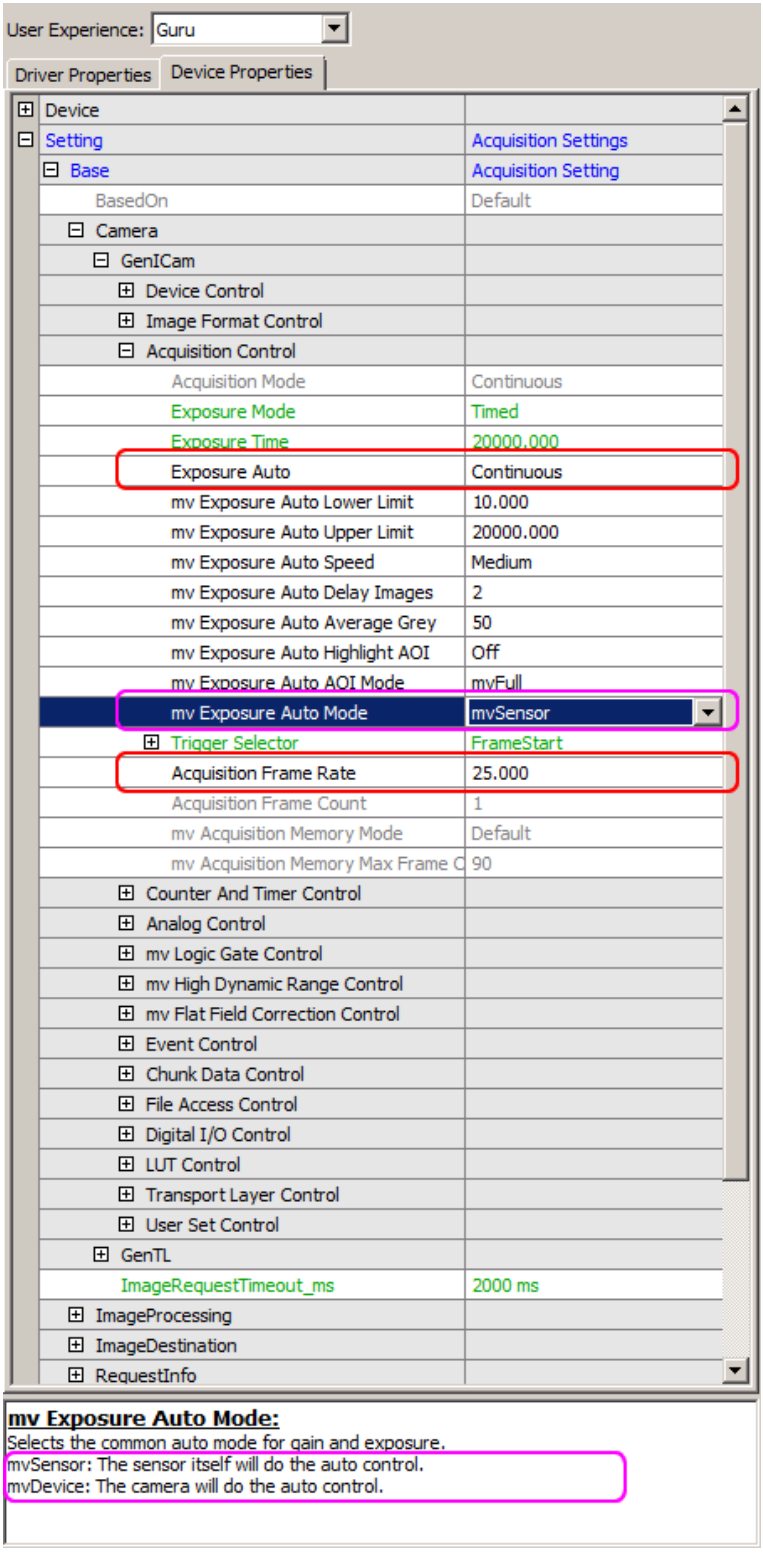


Figure 1: wxPropView - Auto expose is turned on and the frame rate is set to 25 fps

1.19.2.4.2 Example of using a timer for external trigger

Figure 2 shows how to generate a 25 Hz signal, which triggers the camera:

- *"Setting -> Base -> Camera -> GenICam -> Counter & Timer Control -> Timer Selector -> Timer 1"*:

  - *"Timer Trigger Source" = "Timer1End"*
  - *"Timer Duration" = "40000"*

$$FPS_{max} = \frac{1}{40000\ us} = 25$$

- *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> Trigger Selector -> FrameStart"*:
  - *"Trigger Mode" = "On"*
  - *"Trigger Source" = "Timer1End"*

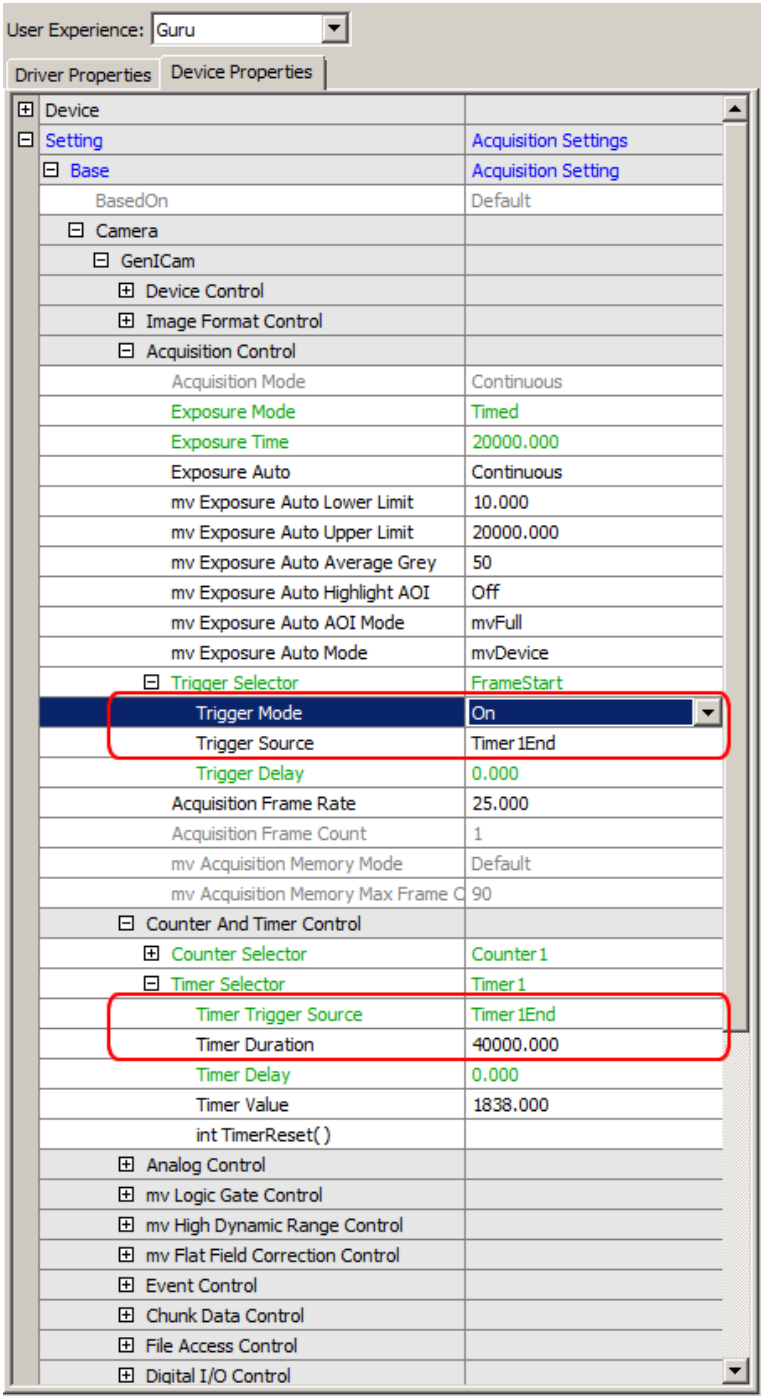


Figure 2: wxPropView - 25 Hz timer for external trigger

No oscillation occurs, regardless of DC ambient vs. AC indoor light.

This operation mode is known as flicker-free or flicker-less operation.

What it mainly does is to adjust the frame frequency to precisely the frequency of the power line. Usually the line frequency is very stable and therefore is the harmonic frequency difference of the two signals are very slow; probably in the range of < 0.1 Hz.

The fact that we do not know the phase relation between the two frequencies means that we scan the alternating ambient light source with our camera. The shorter the exposure time, the more we see a slow change in brightness.

Using AutoExposure/AutoGain can completely eliminate this change because the frequency of change is very low. That means it will be legal if we calculate a brightness difference in one picture and apply it for the next one, because the change is also valid in the next one; as we fulfill the Nyquist theorem.

If we use an arbitrary scanning frequency like 20 fps or whatever your algorithm and data flow is accepting, is wrong in this aspect and leads to oscillations and undesired flicker.

Pointing to a 60 Hz display with flashing backlight an oscillation of 10 Hz can be seen of course.

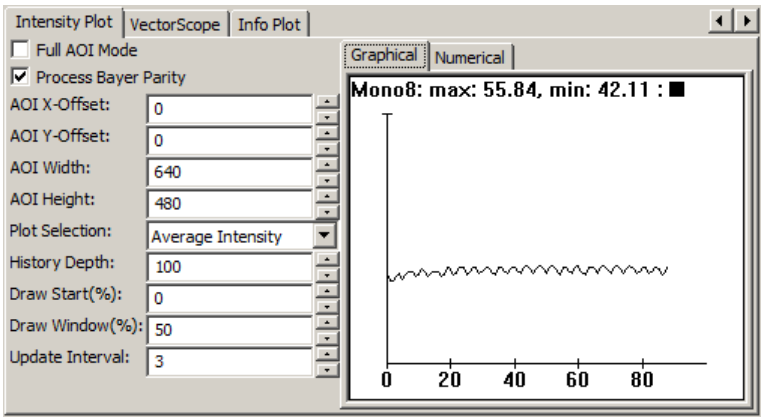


Figure 3: wxPropView - Intensity plot while pointing the camera to a 60 Hz display

1.19.2.4.3 Conclusion

To avoid oscillations, it is necessary to adapt the camera frequency to the frequency of AC light. When using auto expose a flicker-free mode (timer based external trigger) is needed. If the camera is used throughout the world it is necessary that the frequency of AC light can be set in the software and the software adapts the camera to this specific environment.

1.19.2.5 Working with binning / decimation

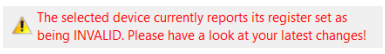
With binning / decimation it is possible to combine / reduce adjacent pixels vertically and/or horizontally. Depending on the sensor, up to 16 adjacent pixels can be combined / reduced.

See also

`https://www.matrix-vision.com/manuals/SDK_CPP/Binning_modes.png`

To optimize the sensor speed, the firmware will check if the binning / decimation functionality is provided by the sensor. If it is available it will use the sensor for binning / decimation, if not, binning / decimation will be processed in the camera's FPGA.

If a binning / decimation combination is not possible, the following note will be displayed:



In this case no acquired images will be displayed until you have changed the settings.

1.19.2.5.1 Binning

Possible binning modes are:

- **Sum:** The response from the combined pixels will be added, resulting in increased sensitivity.
- **Average:** The response from the combined pixels will be averaged, resulting in increased signal/noise ratio.

Binning can be used to lighten the image at the expense of the resolution. This is a neat solution for applications with low light and low noise.

The following results were achieved with the mvBlueFOX3-2124G:

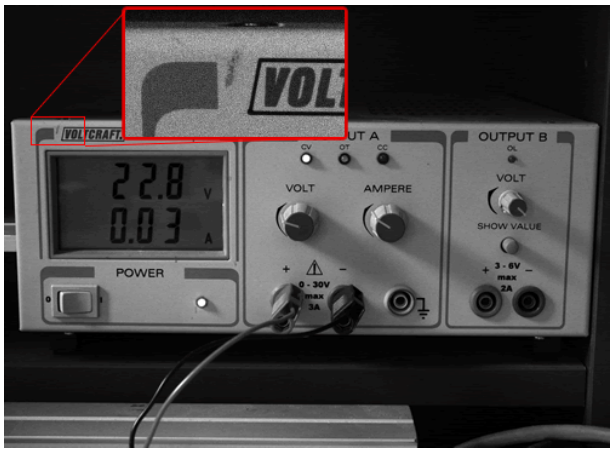
Exposure [in us]	Binning	Gain [in dB]	Averager
2500	-	0	-



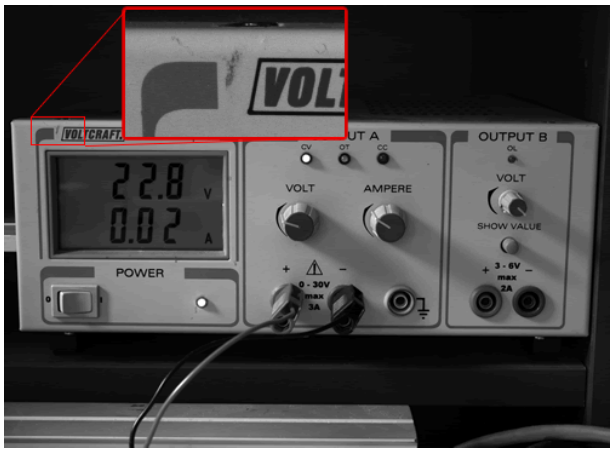
Exposure [in us]	Binning	Gain [in dB]	Averager
2500	-	30	-



Exposure [in us]	Binning	Gain [in dB]	Averager
2500	2H 2V	30	-



Exposure [in us]	Binning	Gain [in dB]	Averager
2500	2H 2V	30	Averaging using 24 frames



The last image shows, that you can reduce the noise, caused by the increased gain, using **frame averaging** (p. 312).

1.19.2.5.2 Decimation

Possible decimation modes are:

- **Discard:** The value of every Nth pixel is kept, others are discarded.
- **Average:** The value of a group of N adjacents pixels are averaged.

1.19.2.6 Working with Sony's 4 Tap CCD sensors

To achieve high frame rates, Sony's high-resolution sensors offer a built-in four channel output a.k.a Taps. I.e., the sensor area is divided into 4 parts in 4-Tap mode and 2 parts in 2-Tap mode:

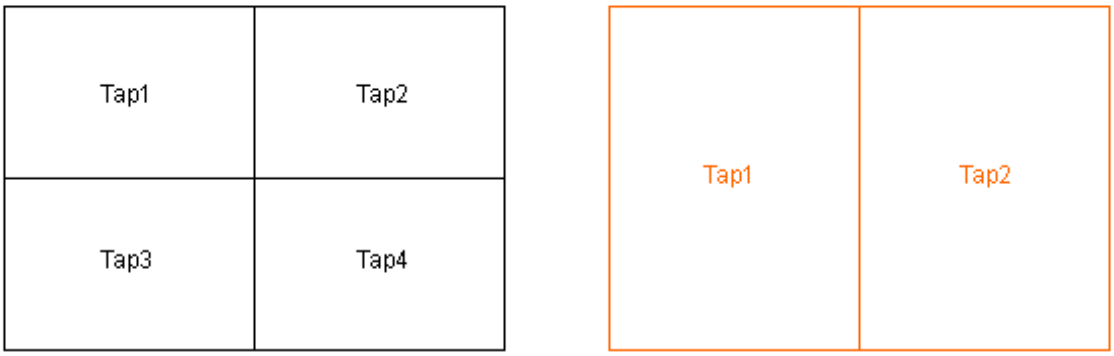


Figure 1: Tap areas in 4-Tap mode (left) and 2-Tap mode (right)

Each Tap has its own gain control and for this reason each image area could behave differently. The following image shows an uncalibrated image where you can see the four different Tap areas of the sensor:

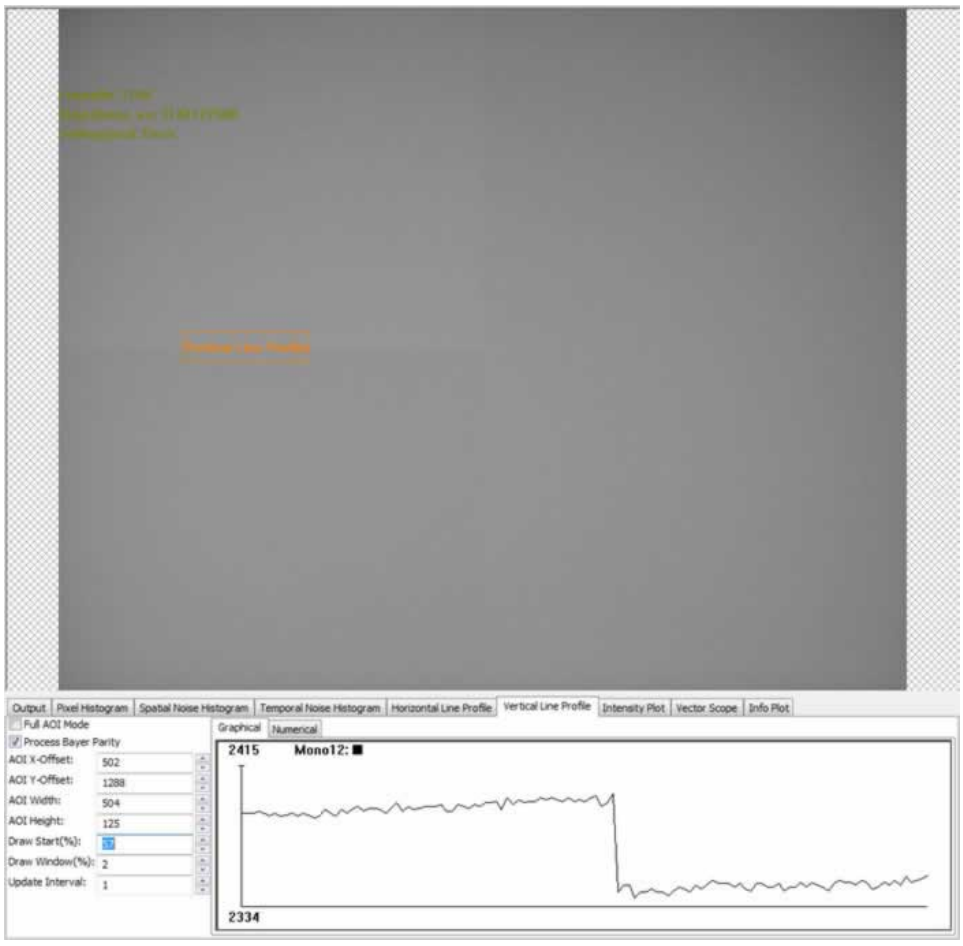


Figure 2: Extreme settings show sensor Tap areas

Therefore it is necessary to calibrate the Taps to get a good image. We can use **wxPropView** (p. 139) to calibrate the camera.

Note

Before calibrating, you should fix the acquisition parameters like number of Taps, device clock frequency, pixel format and exposure time.

1. Open **wxPropView** (p. 139) and
2. open the **Analog Control** (p. 150) (*"Setting -> Base -> Camera -> GenICam -> Analog Control"*).
3. Now, you can adapt the **Gain** of each Tap (**"AnalogTap1"**, **"AnalogTap2"**, **"AnalogTap3"**, **"AnalogTap4"**):

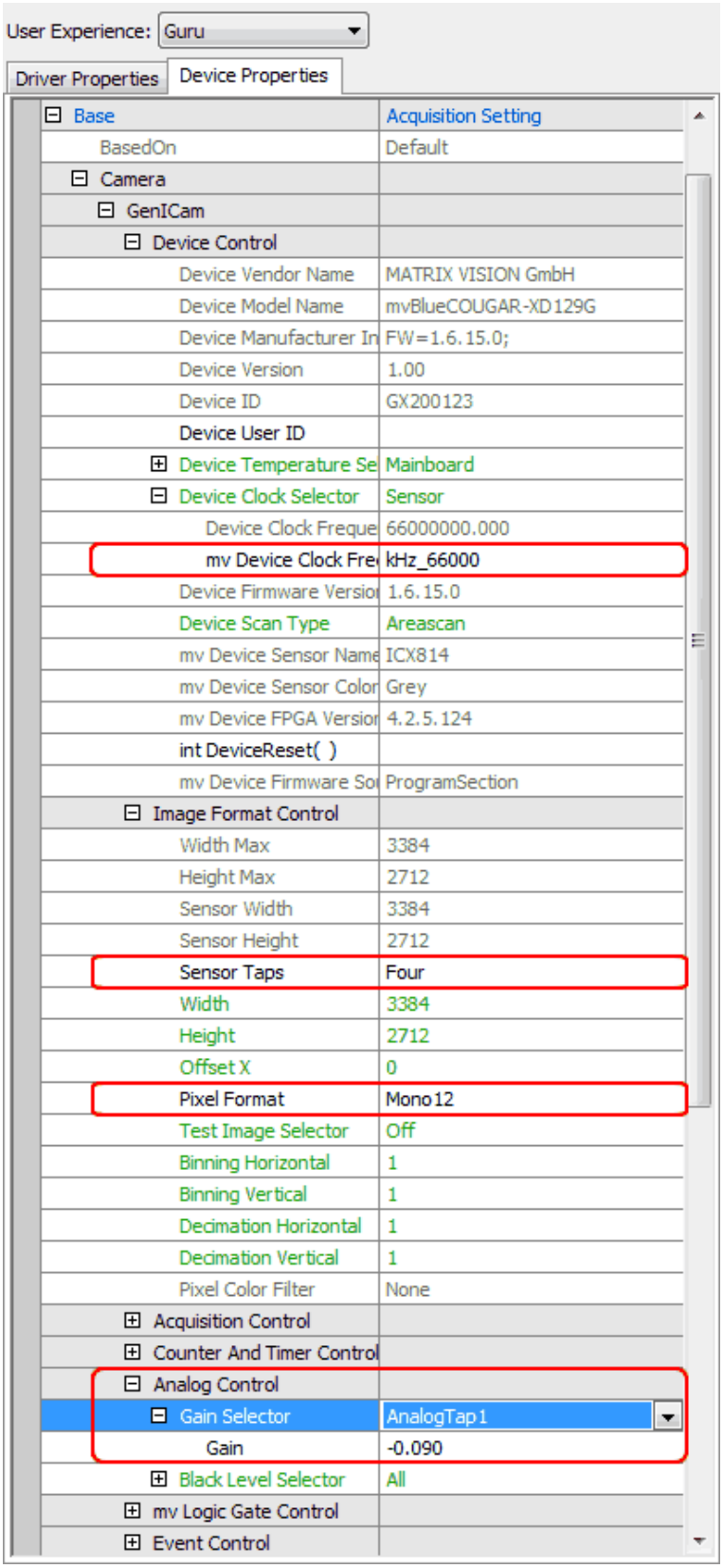


Figure 3: wxPropView - Setting the gain of each Tap area (you will find the exposure time under acquisition control)

In our example the calibrated image would look like this:

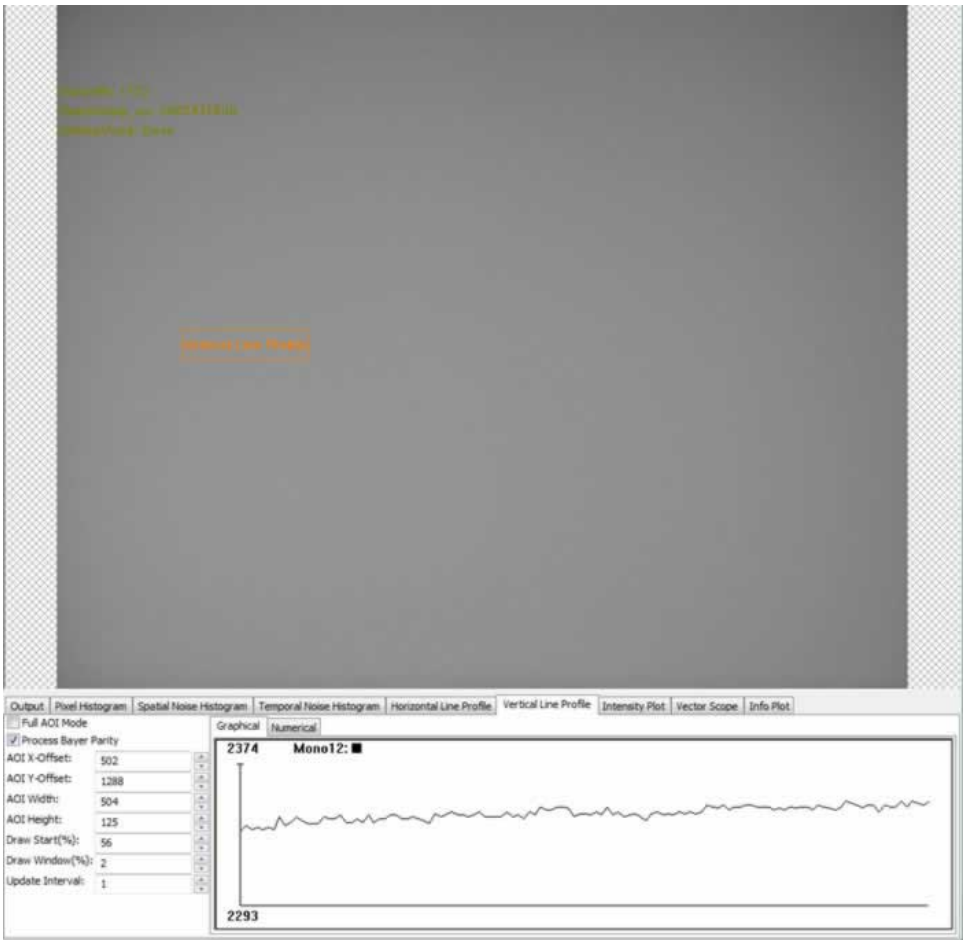


Figure 4: wxPropView - Calibrated Taps

Note

Using 2 Taps, the adapting **"AnalogTap3"** and **"AnalogTap4"** will have no effect.

The mvBlueCOUGAR-XD sensor range contains three 4-Taps sensors by Sony:

- **mvBlueCOUGAR-XD124a (3 Mpix [1936 x 1460])** (p. 630)
- **mvBlueCOUGAR-XD126 (6 Mpix [2752 x 2208])** (p. 649)
- **mvBlueCOUGAR-XD129 (9.2 Mpix [3384 x 2712])** (p. 661)

1.19.2.7 Minimizing sensor pattern of mvBlueCOUGAR-X1010G

Sometimes the gray scale version of Aptinas sensor MT9J003 shows structures comparable with Bayer patterns of color sensors. This pattern is particularly apparent in scaled images:



Figure 1: Bayer pattern like structures of the MT9J003(gray scale version)

To minimized this pattern, you can balance the sensor patterns since Firmware version 2.3.70.0. This procedure works like the white balancing used with color sensors. For this reason the same terms are used (red, blue, green).

The balance reference is the "green" pixel value from the "blue-green" line of the sensor.

See: **Output sequence of color sensors (RGB Bayer)** (p. 133)

I.e. all gray scale values of the these "green" pixels are averaged.

With *"Setting -> Base -> Camera -> GenICam -> Analog Control -> Balance Ration Selector"* you can select each color to set the *"Balance Ratio"*:

- **"Red"** averages the "red" pixel values from the "red-green" line of the sensor.
- **"Green"** averages the "green" pixel values from the "red-green" line of the sensor, too.
- **"Blue"** averages the "blue" pixel values from the "blue-green" line of the sensor.

I.e. there are 4 average values (reference value, red value, green value, blue value). The lowest value will be unchanged, the other values are increased using each *"Balance Ratio"*.

However by using the property *"Balance White Auto"* you can balance the sensor automatically:

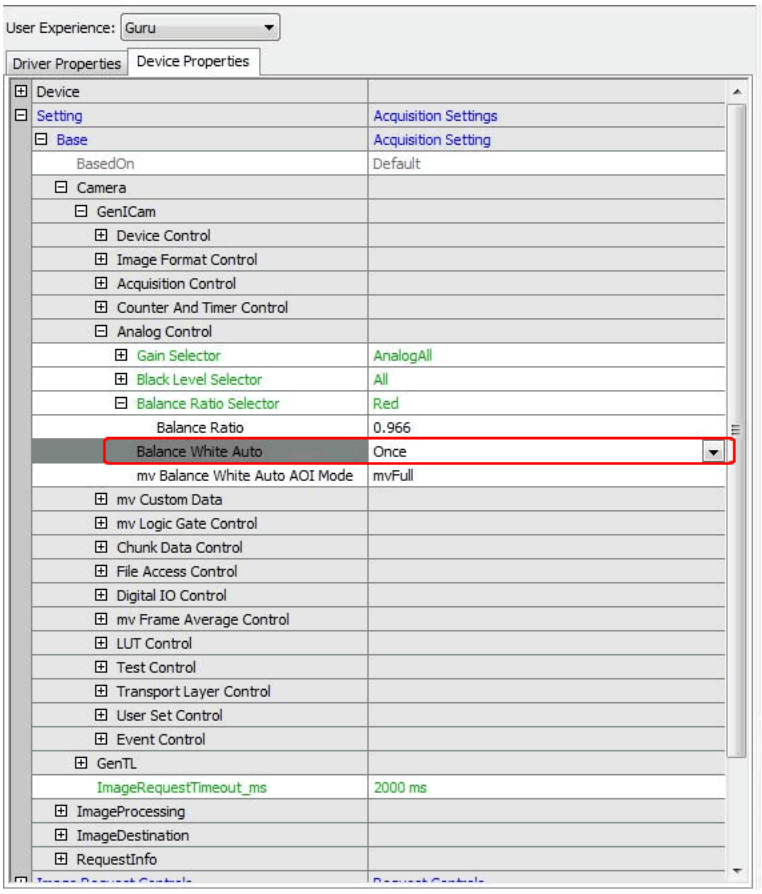


Figure 2: Balance White Auto

After balancing, we recommend to save these settings to a UserSet with **wxPropView** (p. 139) which is described in **"Saving User Settings In The Non-volatile Memory"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual.



Figure 3: Calibrated sensor

1.19.2.8 Working with the dual exposure feature ("mvMultiZone") of mvBlueCOUGAR-XD107

1.19.2.8.1 Introduction

The IMX420 used in the mvBlueCOUGAR-XD107 is a Pregius sensor of the third generation. This sensor features a dual exposure mode, i.e. after a trigger event, for example, different image areas can be exposed to light at the same time differently.

To activate the dual exposure mode we introduced a new exposure mode called **mvMultiZone**.

If this mode is selected you can set the **"mv Exposure Horizontal Zone Divider"**. This property indicates where the zones are divided horizontally. The step size is 12.5%.

The exposure time of the different zones can be specified using the **"Exposure Time Selector"**. Possible zones are

- **mvHorizontalZone0**
- **mvHorizontalZone1**

1.19.2.8.2 Setting the dual exposure using wxPropView

To activate the dual exposure, just

1. Select **mvMultiZone** in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> Exposure Mode"*.
2. Set the *"Exposure Time"* of each zone by selecting the specific **"Exposure Time Selector"**.
3. Finally, specify the *"mv Exposure Horizontal Zone Divider"* (e.g. 50, i.e. you have two equal image halfs).

Note

Regardless of whether the acquisition is already active, changing the *"mv Exposure Horizontal Zone Divider"* takes effect with a delay of at least one additional image. Frames that have already been captured but not used by the application (still in the camera's frame buffer or an internal driver queue) are of course no longer affected by the change, so the overall delay may appear longer and is independent of the time between the captured images.

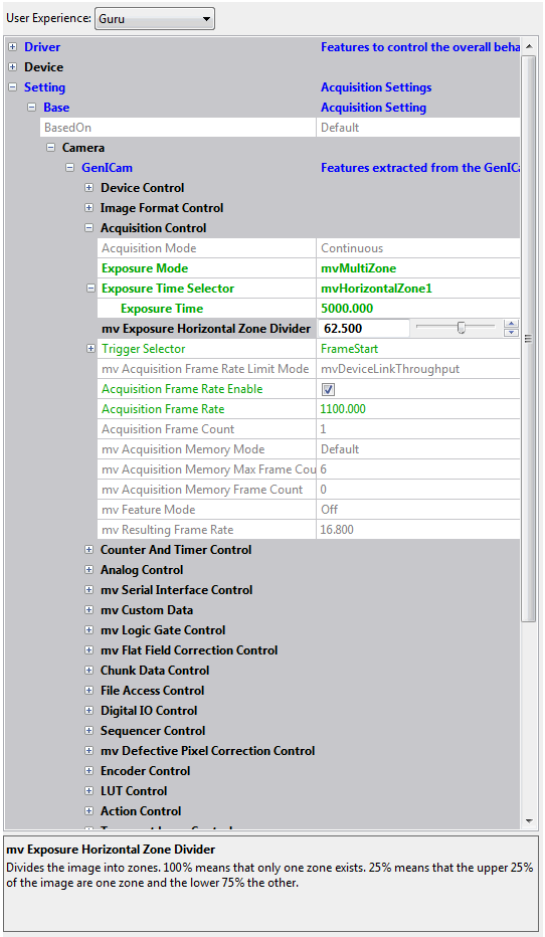


Figure 1: wxPropView - new Exposure Mode

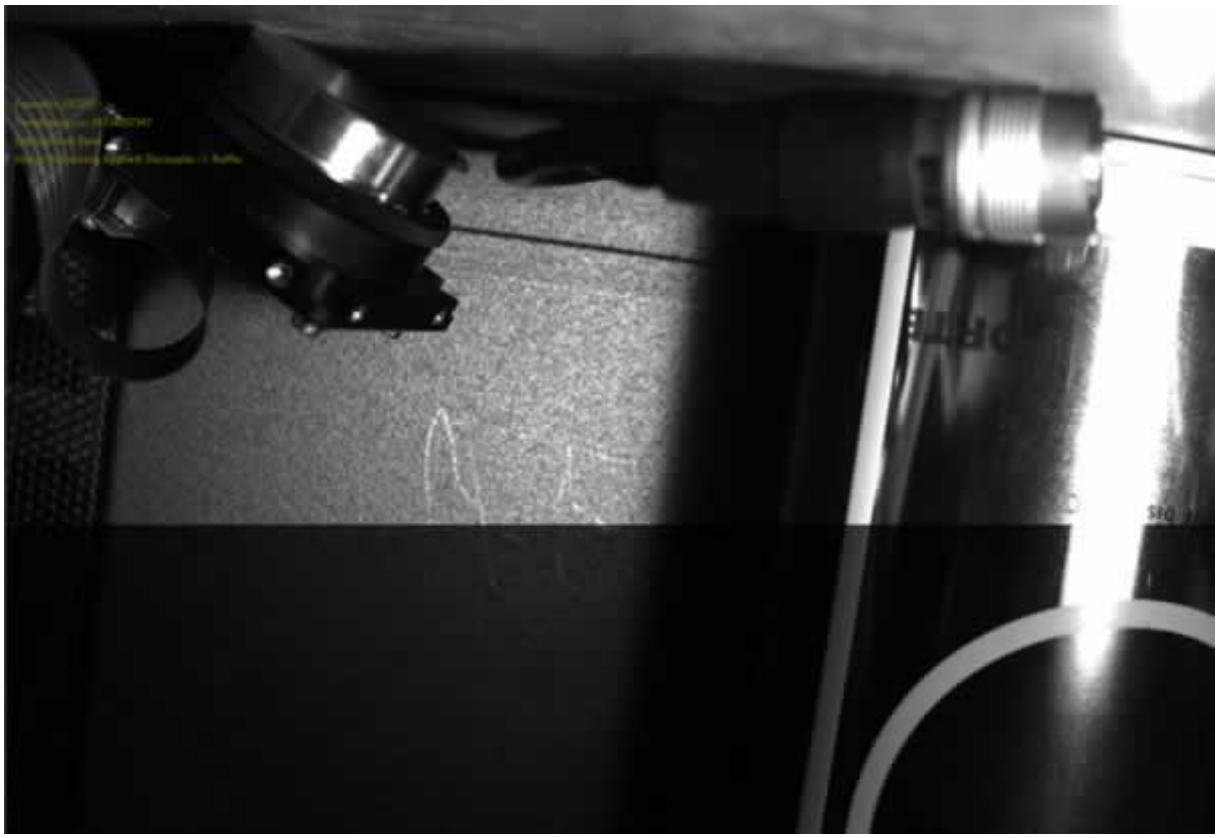


Figure 2: Example image with two different exposed to light zones

1.19.2.8.2.1 Programming the dual exposure

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

  // more code
  GenICam::AcquisitionControl ac( pDev );
  ac.exposureMode.writeS( "mvMultiZone" );
  ac.exposureTimeSelector.writeS( "mvHorizontalZone1" );
  ac.exposureTime.write( 5000 );
  ac.exposureHorizontalDivider.write( 50 );
  // more code
```

1.19.2.9 Working with the dual gain feature of mvBlueCOUGAR-XD107/XD1031 and mvBlueCOUGAR-X102m/X107

1.19.2.9.1 Introduction

The IMX420/425/428 used in the mvBlueCOUGAR-XD107/XD1031 and mvBlueCOUGAR-X102m/X107 are Pregius sensors of the third generation.

Those sensors feature a dual gain mode, i.e. after a trigger event, for example, different image areas can be amplified at the same time differently.

To activate the dual gain mode it is necessary to enable the multi area mode by using the **mvMultiAreaMode** property. At least two different AOIs have to be defined.

Note

Both AOIs must not overlap or touch each other!

Once the AOIs are configured the **GainSelector** has to be set to one of the new implemented options called **mvHorizontalZone0** and **mvHorizontalZone1**.

The gain value of the different zones can be specified using the **"Gain Selector"**. And the corresponding gain property. Possible zones are

- **mvHorizontalZone0**
- **mvHorizontalZone1**

If this mode is selected you can set the **"mv Gain Horizontal Zone Divider"**. This property indicates where the zones are divided horizontally once more than two AOIs are configured. In this case the e.g. 25% means that the upper 25% of the image are defined by the gain value of **mvHorizontalZone0** and the lower 75% are defined by the gain value of **mvHorizontalZone1**.

Note

Some sensors may only allow to change the gain at certain positions e.g. the last line of a defined ROI. In this case the first possible switching point above the actual line will be used.

To activate the dual gain, just

1. Use the Multi AOI Wizard to adjust the different AOIs. (They must not overlap or touch each other!)
2. Select **mvMultiZone** in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> mvGainMode"*.
3. Select **mvHorizontalZone0** in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector"*.
4. Adjust the gain value for the first AOI in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector -> Gain"*.
5. Adjust the gain divider position *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector -> mvGainHorizontalZoneDivider"*.
6. Select **mvHorizontalZone1** in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector"*.
7. Adjust the gain value for the second AOI in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> GainSelector -> Gain"*.

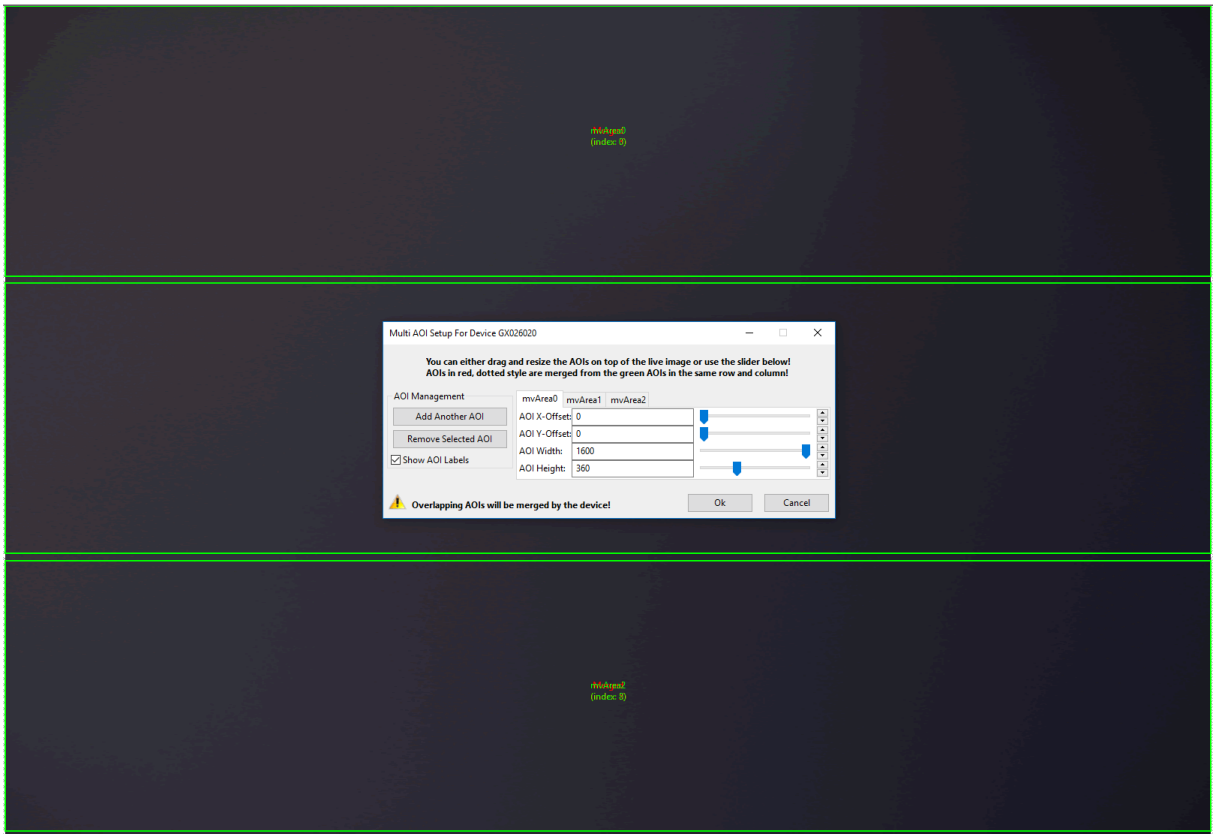


Figure 1: wxPropView - configuring multiple AOIs

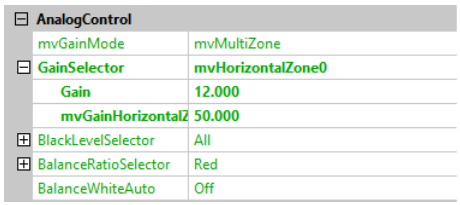


Figure 2: wxPropView - configuring the gain of the first zone



Figure 3: Example image with two different amplified zones

1.19.2.9.2 Programming the dual gain mode

As an example the IMX425 sensor is used for the sample. The goal is to configure three AOIs which have a similar height. Since the AOIs must not overlap or touch each other, it is important to increase the offset of the next AOI by the smallest increment size. Which is 8 in this case.

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

  // more code
    GenICam::ImageFormatControl ifc( pDev );

    ifc.mvMultiAreaMode.writeS( "mvMultiAreasCombined" );

    ifc.mvAreaSelector.writeS( "mvArea0" );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 0 );
    ifc.mvAreaWidth.write( ifc.mvAreaWidth.getMaxValue( ) );
    ifc.mvAreaHeight.write( 360 );
    ifc.mvAreaEnable.writeS( "1" );

    ifc.mvAreaSelector.writeS( "mvArea1" );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 368 );
    ifc.mvAreaWidth.write( ifc.mvAreaWidth.getMaxValue( ) );
    ifc.mvAreaHeight.write( 360 );
    ifc.mvAreaEnable.writeS( "1" );

    ifc.mvAreaSelector.writeS( "mvArea2" );
    ifc.mvAreaOffsetX.write( 0 );
    ifc.mvAreaOffsetY.write( 736 );
```

```
    ifc.mvAreaWidth.write( ifc.mvAreaWidth.getMaxValue( ) );
    ifc.mvAreaHeight.write( 360 );
    ifc.mvAreaEnable.writeS( "1" );

    GenICam::AnalogControl anc( pDev );
    anc.mvGainMode.writeS( "mvMultiZone" );

    anc.gainSelector.writeS( "mvHorizontalZone0" );
    anc.gain.write( 12 );
    anc.mvGainHorizontalZoneDivider.write( 80 );

    anc.gainSelector.writeS( "mvHorizontalZone1" );
    anc.gain.write( 0 );
  // more code
```

1.19.2.10 Working with the dual ADC feature of mvBlueCOUGAR-XD107

1.19.2.10.1 Introduction

The IMX420 used in the mvBlueCOUGAR-XD107C/G is a Pregius sensor of the third generation. This sensor features a dual ADC mode which allows to increase the dynamic range of the sensor.

To activate the dual ADC mode it is sufficient to use the **"mv Dual ADC Mode"** property.

1.19.2.10.2 Enabling the dual ADC mode using wxPropView

To activate the dual ADC mode for a color model, just

1. Change the pixel format to **"BayerRG16"** *"Setting -> Base -> Camera -> GenICam -> Image Format Control -> PixelFormat"*.
2. Enable **"mvDualAdcMode"** *"Setting -> Base -> Camera -> GenICam -> Image Format Control -> mv↩ DualAdcMode"*.

To activate the dual ADC mode for a monochrome model, just

1. Change the pixel format to **"Mono16"** *"Setting -> Base -> Camera -> GenICam -> Image Format Control -> PixelFormat"*.
2. Enable **"mvDualAdcMode"** *"Setting -> Base -> Camera -> GenICam -> Image Format Control -> mv↩ DualAdcMode"*.

Note

Changing the pixel format to a pixel format which utilizes more than one byte per pixel (e.g. BayerRG16 or Mono16) is necessary to present the higher dynamic range in the image. The maximum frame rate will be halved because of the multi byte pixel format. Please be aware that in case of histogram analysis this mode might not be the best option since the transition between both ADCs might show a slight gain offset error.

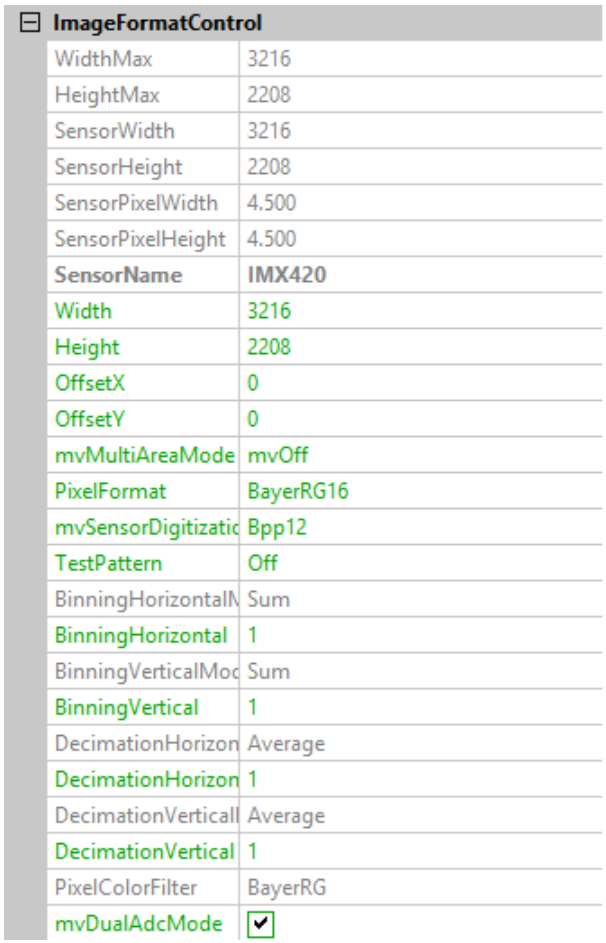


Figure 1: wxPropView - enabling the dual ADC mode

1.19.2.10.2.1 Programming the dual ADC mode

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

  // more code
    GenICam::ImageFormatControl ifc( pDev );
    ifc.pixelFormat.writeS("BayerRG16");
    ifc.mvDualAdcMode.writeS( "1" );
  // more code
```

1.19.2.11 Working With Gain And Black-Level Values Per Color Channel

In many low-lighting applications the gain needs to be increased to enhance the brighness of the images. However, while the image brightness is increased, the black-level of the image is also increased, which in many cases should be avoided. With the help of the GainOffsetKnee filter it is possible to correct/adjust the overall black-level as well as the black-level per color channel, even when the gain is applied. *Figure 1* shows the working principle of the GainOffsetKnee filter.

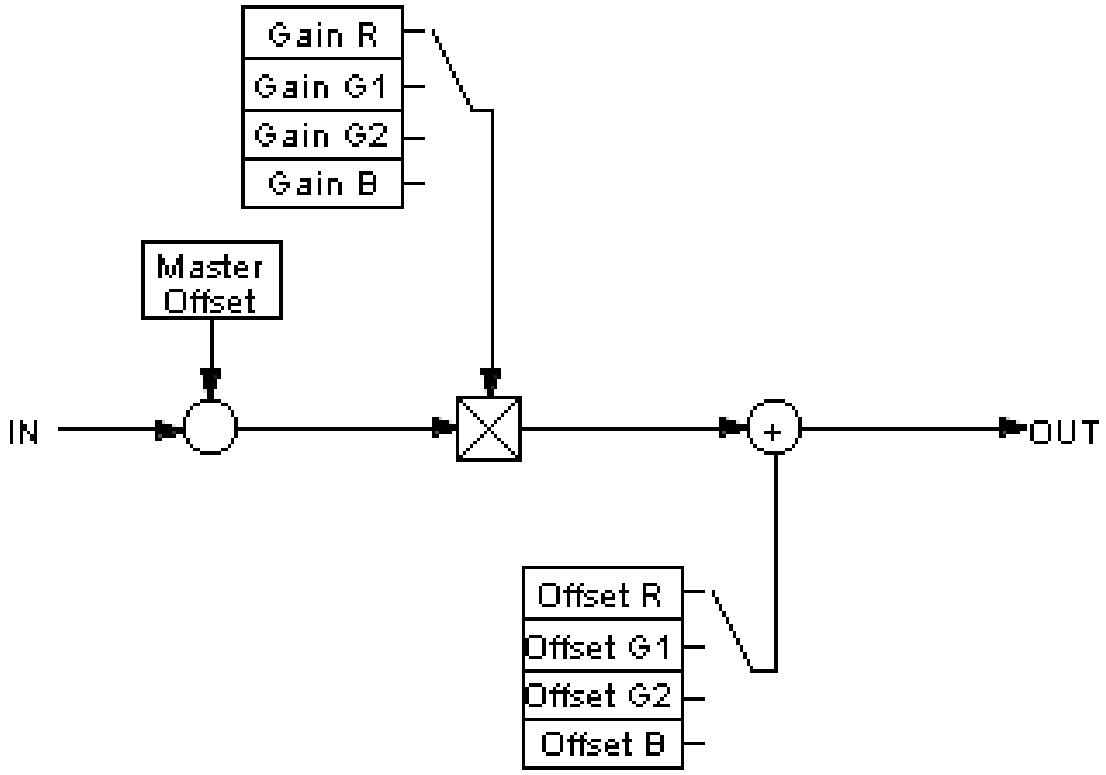


Figure 1: The GainOffsetKnee filter working principle

The GainOffsetKnee filter is one of the image processing methods performed on the host. It allows you to adjust:

- The overall offset (*i.e.* overall black-level) of an image.
- The individual gain per color channel.
- The individual offset (*i.e.* individual black-level) per color channel.

1.19.2.11.1 Configuration in wxPropView Here is how to configure the GainOffsetKnee filter in wxPropView and the impact the filter has on an image:

1. The GainOffsetKnee filter is located under *"Setting -> Base -> ImageProcessing"*.

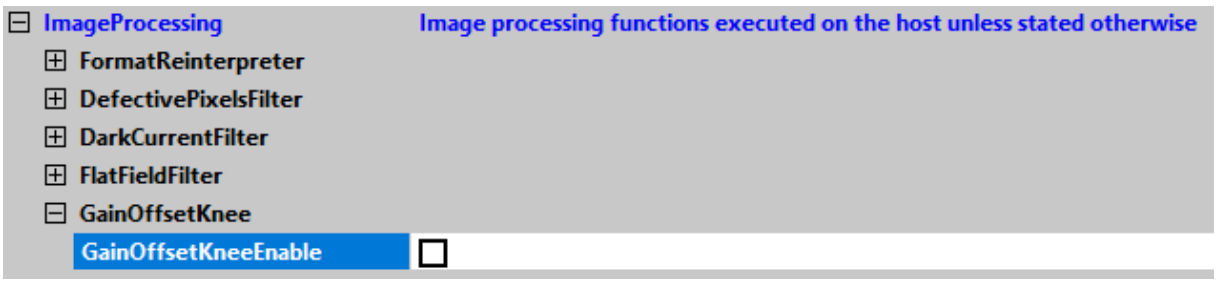


Figure 2: The GainOffsetKnee filter option in wxPropView

2. Once the GainOffsetKnee filter is activated, the configuration field will be displayed (see *Figure 3*). As an example, the current RGB image is shown in *Figure 4* and its histogram in *Figure 5*.

ImageProcessing	Image processing functions executed on the host unless stated otherwise
FormatReinterpreter	
DefectivePixelsFilter	
DarkCurrentFilter	
FlatFieldFilter	
GainOffsetKnee	
GainOffsetKneeEnable	☑
GainOffsetKneeMasterOffset_pc	0.00 %
GainOffsetKneeChannels	
Channel-0	Parameters in this list will be applied to the GREY channel for MONO images, the RED channel for RGB or Bayer images or the Y channel for YUV images
Gain	1.0000
Offset_pc	0.00 %
Channel-1	Parameters in this list will be applied to the GREEN channel for RGB images, the GREEN channel of the RED-GREEN lines for Bayer images or the U channel for YUV images
Gain	1.0000
Offset_pc	0.00 %
Channel-2	Parameters in this list will be applied to the BLUE channel for RGB or Bayer images or the V channel for YUV images
Gain	1.0000
Offset_pc	0.00 %
Channel-3	Parameters in this list will be applied to the GREEN channel of the BLUE-GREEN lines for Bayer images
Gain	1.0000
Offset_pc	0.00 %

Figure 3: The configuration field for the GainOffsetKnee filter



Figure 4: An image without the GainOffsetKnee filter

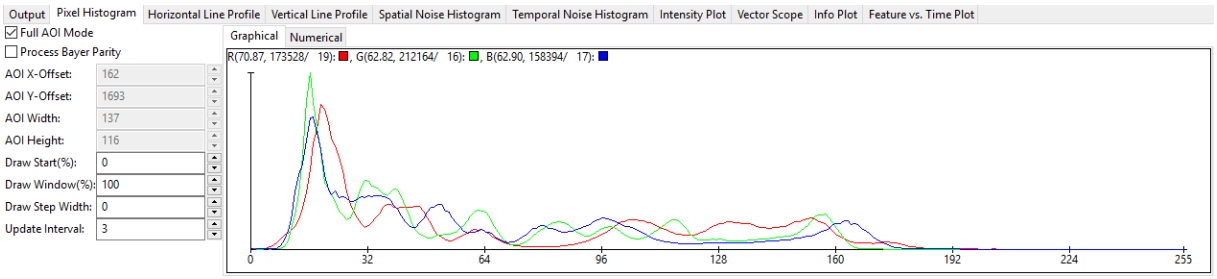


Figure 5: The histogram of *Figure 3*

3. The overall offset can be assigned using the *'GainOffsetKneeMasterOffset_pc'*. A positive offset increases the black-level of the image, whereas a negative offset reduces it. To visualize the effect, an offset of 5% is given as an example, which means that the overall black-level of the image will be increased by 5% of the max. pixel value (*i.e.* 255 in this example). As a result, the overall black-level in the current histogram (see *Figure 8*) has been increase by 12.75 (which is 5% x 255) comparing to the original histogram (see *Figure 5*).

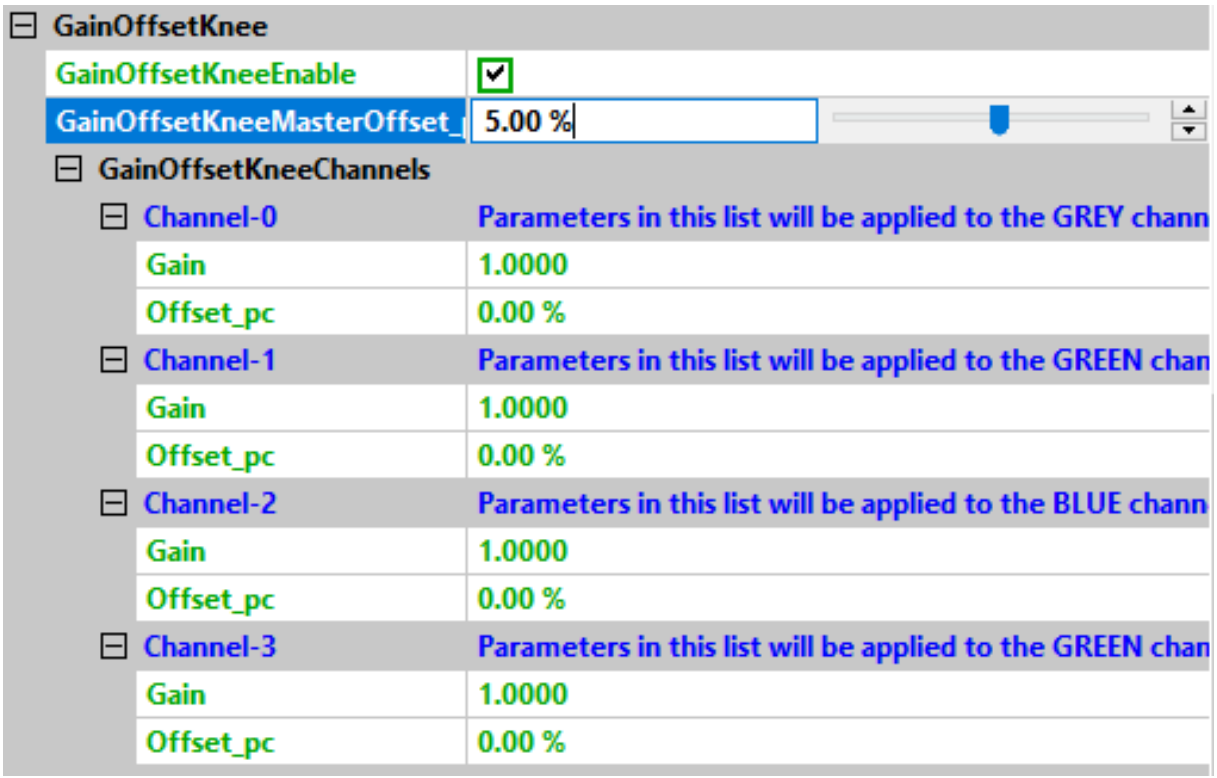


Figure 6: Assign overall/master offset to the image



Figure 7: The image with 5% overall offset

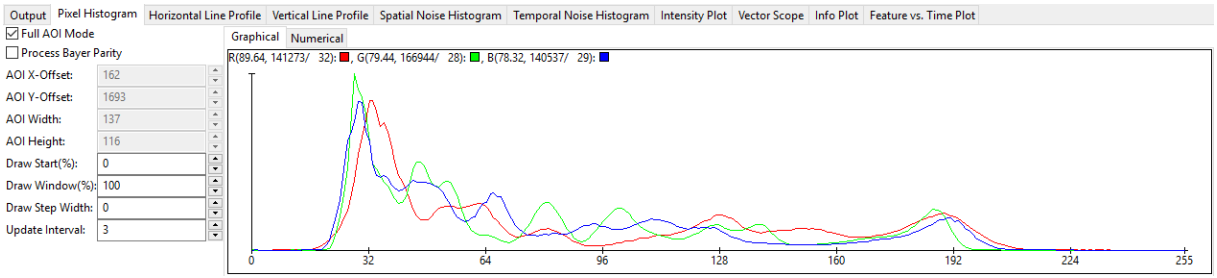


Figure 8: The histogram with 5% overall offset

4. Among the GainOffsetKneeChannels there are 4 channels. For mono images, only channel 0 is used. For RGB images, channel 0-2 are used: red channel, green channel and blue channel respectively. For Bayer images, channel 0-3 are used. For more description please refer to *Figure 3*. As an example, a gain of 1.0625dB is applied to the red channel. As shown in *Figure 10* and *Figure 11*, the grey-level of the red channel is increased while the other two channels remain the same.

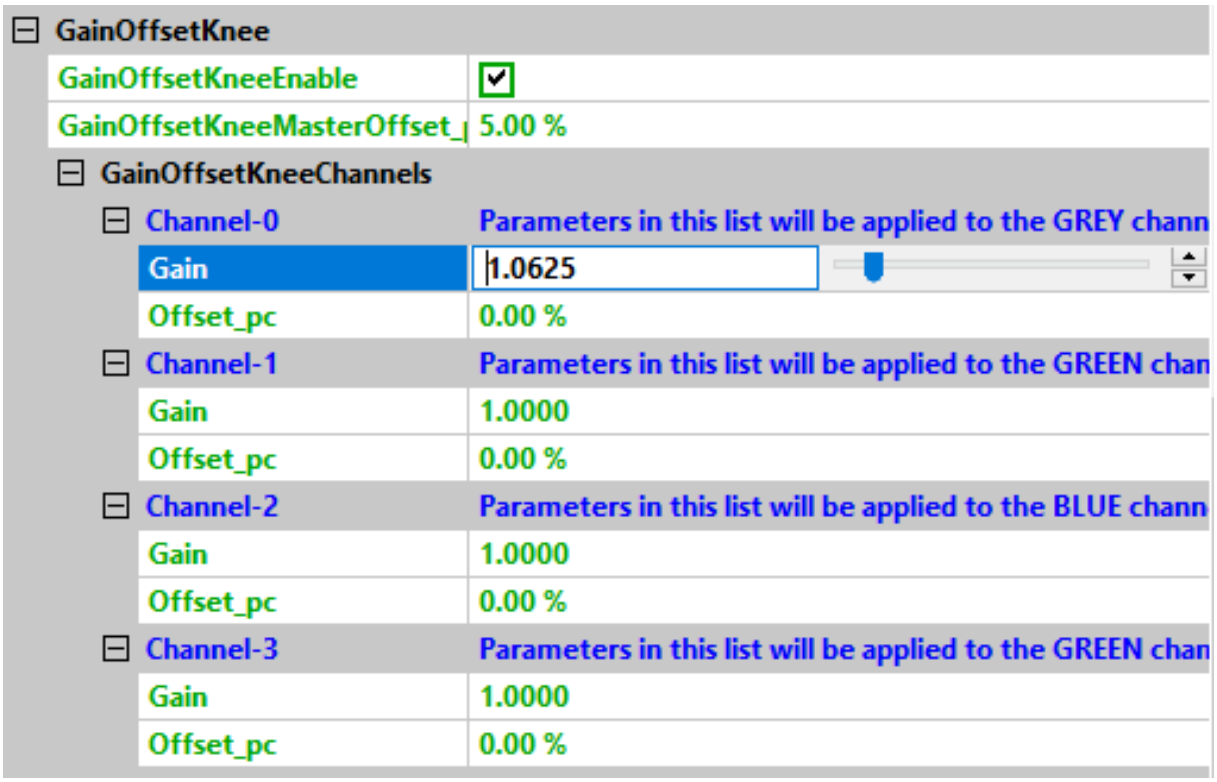


Figure 9: Assign individual gain to the red channel



Figure 10: The image with 1.0625dB gain in the red channel

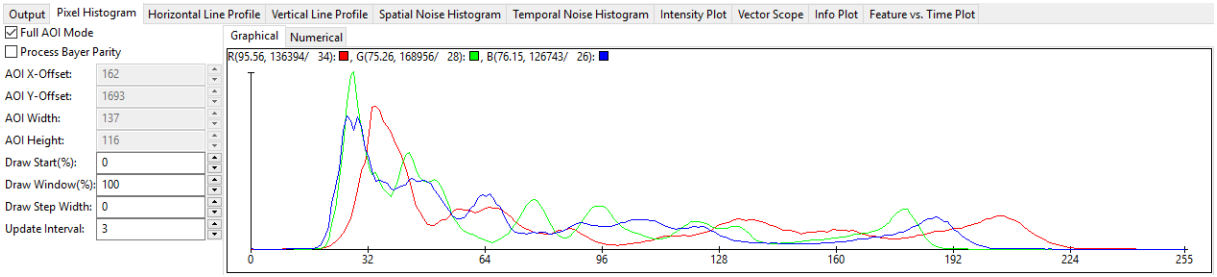


Figure 11: The histogram with 1.0625dB gain in the red channel

5. The individual black-level can be assigned using the channel specific *'Offset_pc'*. Analogous to 'GainOffset↩ KneeMasterOffset_pc', a positive offset increases the black-level of the channel, whereas a negative offset reduces it. To visualize the effect, an offset of 5% is given as an example in the red channel. The histogram (see *Figure 14*) shows therefore a 12.75 (which is 5% x 255) offset increase in the red channel.

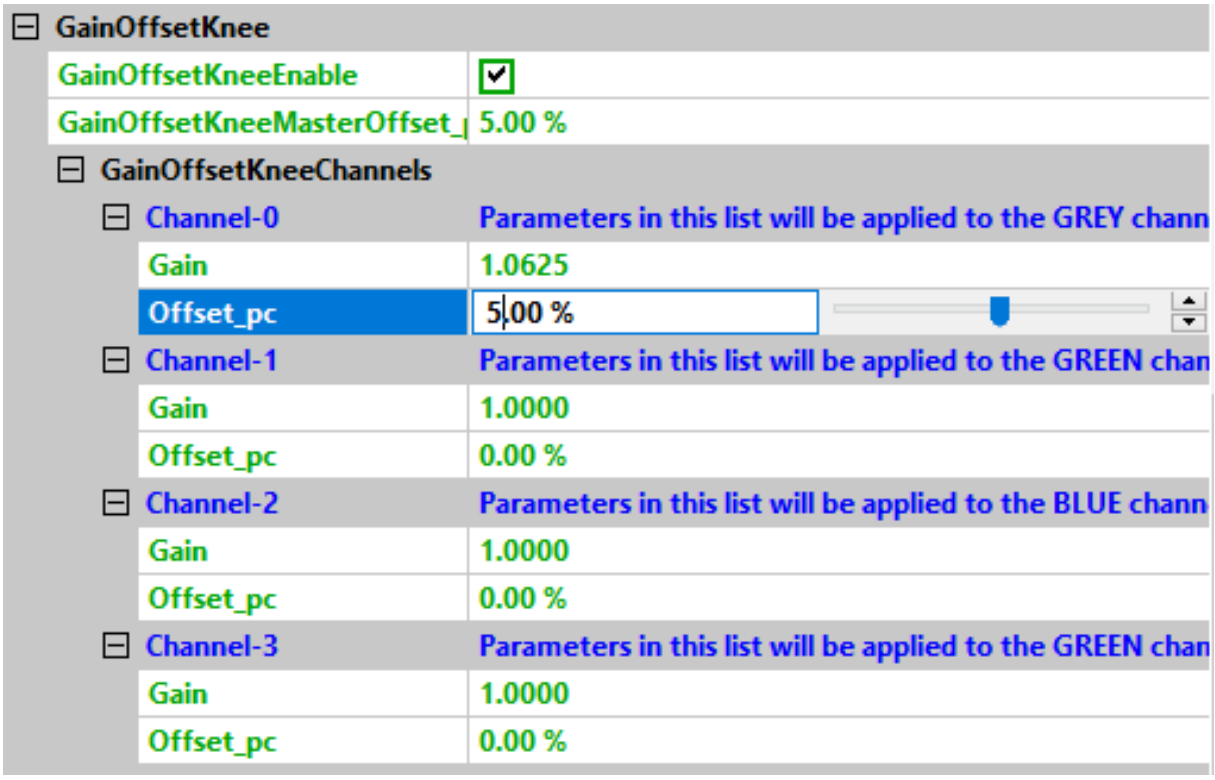


Figure 12: Assign individual offset to the red channel



Figure 13: The image with 5% offset in the red channel

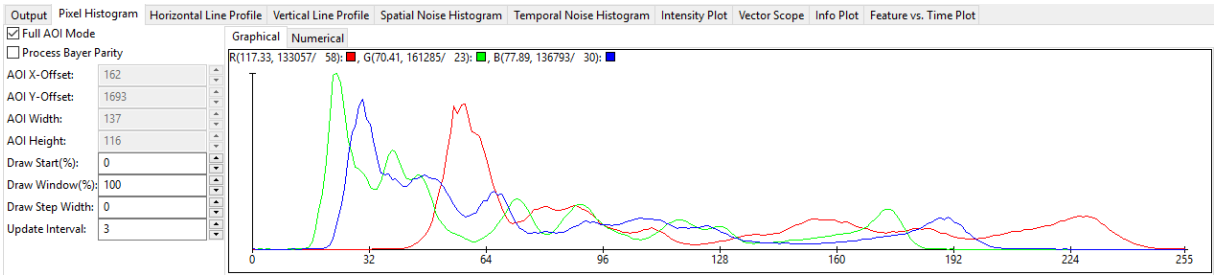


Figure 14: The histogram with 5% offset in the red channel

1.19.2.11.2 Configuration Using The API Depending on the programming language you are working with the names of classes, namespaces and properties might vary slightly. For C++ please refer to the `GainOffset↩KneeChannelParameters` class and the `Image Processing` class for some guidance, for other languages when searching for the *offset* or *knee* properties similar things can be found.

1.19.3 Improving the communication

There are several use cases concerning communication and bandwidth issues:

- **Optimizing the bandwidth** (p. 343)
- **Discovering Devices In Different Subnets** (p. 346)

1.19.3.1 Optimizing the bandwidth

1.19.3.1.1 Calculating the needed bandwidth

In some applications, for example, when multiple cameras share a single network path, you have to keep the bandwidth in mind.

Note

Depending on the available link speed different bandwidth are available:

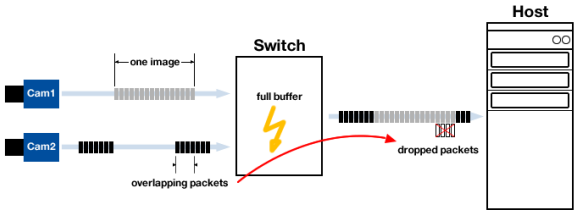
- 10 GBit/s: 1250 MByte/s
- 2 GBit/s: 250 MByte/s
- 1 GBit/s: 125 MByte/s
- 100 MBit/s: 12,5 MByte/s

The result of the formula from above provides a rough guideline only. Some additional bandwidth is needed by the communication protocol and some other non GigE Vision™ related network traffic. Apart from that not every network controller can cope with the full bandwidth offered by the link, thus real world results may vary.

1.19.3.1.2 Limiting the bandwidth of the imaging device

For a setup of multiple streaming devices connected to one host, it is highly recommended to consider the information of this chapter.
Even if the connected links and devices are able to handle the average throughput of a streaming setup situations might be encountered where the data throughput temporarily exceeds the capabilities of involved network components for a very short period of time. This might result in packet loss with GigE Vision™ devices or overflowing buffers with USB3 Vision™ devices.

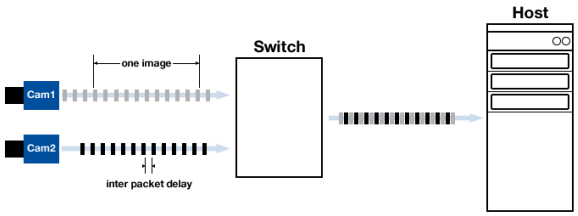
1.19.3.1.2.1 How the Device Link Throughput Limit works If multiple data packets from multiple network devices arrive at the same time in a switch and therefore the incoming bandwidth temporarily exceeds what the outgoing link can deal with, the switch needs to buffer some of the packets. This is due to the fact, that at full speed, each network packet is sent one after the other, with a minimum space of 12 Bytes between two packets (the so called **"inter-packet delay"**). That means a data stream will be sent as a series of consecutive packets, separated by the inter-packet delay, until the whole data is transmitted. Since a typical GigE Vision™ streaming network packet is multiple times bigger than a default packet, the switch internal buffer might not be able to store these data for a longer period of time. Consider the following setup:



Two streams from two devices arrive in the switch at maximum link speed. This example shows, how packets could get dropped by the switch if its internal buffer is not able to hold them all until they could be forwarded. The moment both ports of the switch are receiving data at full link speed at exactly the same time some data **must** be buffered since the switch can only forward the data at maximum link speed to the receiver. So if the switch has less internal buffer than data that cannot be forwarded is **must** be dropped.

The GigE Vision™ standard specifies the feature GevSCPD (Stream Channel Packet Delay), so that the user could have control over the link throughput at packet level. The method how the delay is calculated could vary on different device types from different vendors since the unit of the inter-packet delay is measured in `device ticks`. From version 1.5.2 and above, the GenICam™ SFNC defines the features **DeviceLinkThroughputLimit** and **Device↩ LinkThroughputLimitMode**, which are meant to provide a standardized way to control the delay by defining a throughput limit rather than an explicit delay.
All MATRIX VISION devices support the GenICam™ SFNC features **DeviceLinkThroughputLimitMode** and **DeviceLinkThroughputLimit** to limit the bandwidth used by a device in a convenient way since firmware version 2.25.0.

On GigE Vision™ devices, setting the **DeviceLinkThroughputLimit** results in a longer delay between two consecutive network packets. Now a switch more likely is able to forward all from all incoming links because each link transfers using a reduced data rate, which means less packets need to be buffered reducing the chance for packet loss. However, a larger gap between network packets means, the time to transmit one specific bunch of data could increase. Keep that in mind if you plan an application with hard latency requirements.

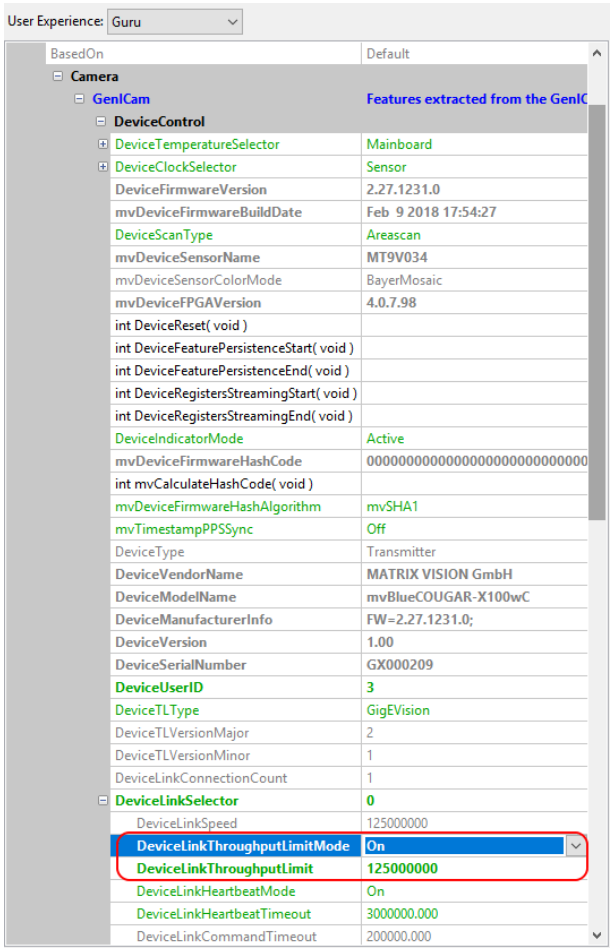


Zipper principle if DeviceLinkThroughputLimit is set. The incoming traffic is equally distributed and so the switch does not need to buffer many packets before they can be forwarded.

If it is necessary to limit the outgoing link throughput of a device, this can be accomplished the following way:

Available since firmware version 2.25.0

1. In `"Setting -> Base -> Camera -> GenICam -> Device Control -> Device Link Selector"` set property **"Device Link Throughput Limit Mode"** to **"On"**.
2. Now, you can set the bandwidth with **"Device Link Throughput Limit"** to your desired bandwidth in bits per second

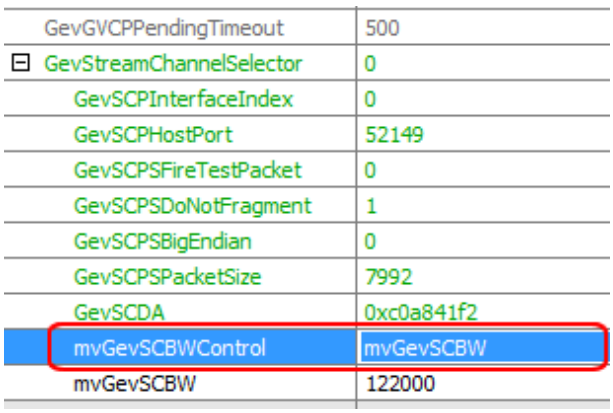


wxPropView - Setting Device Link Throughput Limit

Note

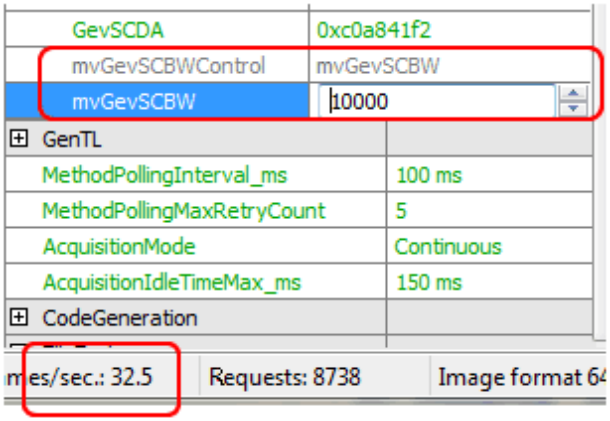
Firmware version $<$ 2.25.0
The following features are deprecated and should not be used anymore!
On devices with an older firmware version, limiting the link throughput was done this way.

1. In `"Setting -> Base -> Camera -> GenICam -> Transport Layer Control -> Gev Stream Channel Selector"` set property **"mv Gev SCBW Control"** to **"mvGevSCBW"**.



wxPropView - Setting mvGevSCBW to mvGevSCBWControl

2. Now, you can set the bandwidth with **"mvGevSCBW"**. E.g. 10000 for 10 MB.
   According to the image size and acquisition settings, the frame rate will be adjusted.



wxPropView - Setting bandwidth size

In contrast to this convenient bandwidth control mechanism of mvBlueCOUGAR-X cameras, with other cameras you have to know and optimize the Inter-Packet Delay of the camera to avoid congestion in the switch (the loss of packages is an indicator of congestion). You can get the Inter-Packet Delay with following calculator:

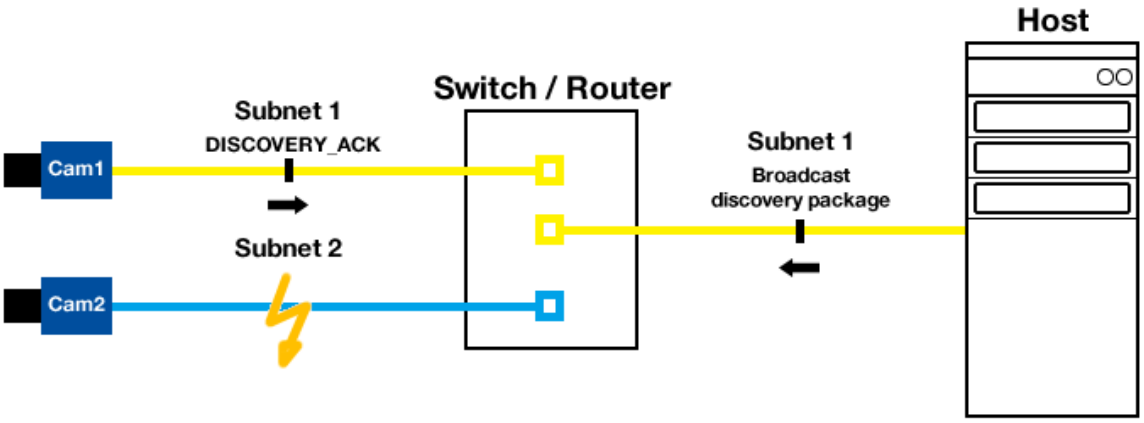
1.19.3.2 Discovering Devices In Different Subnets

1.19.3.2.1 What is Unicast Device Discovery?

With Unicast Device Discoveries it is possible to discover GigE Vision™ devices that reside in a different subnet than the host.

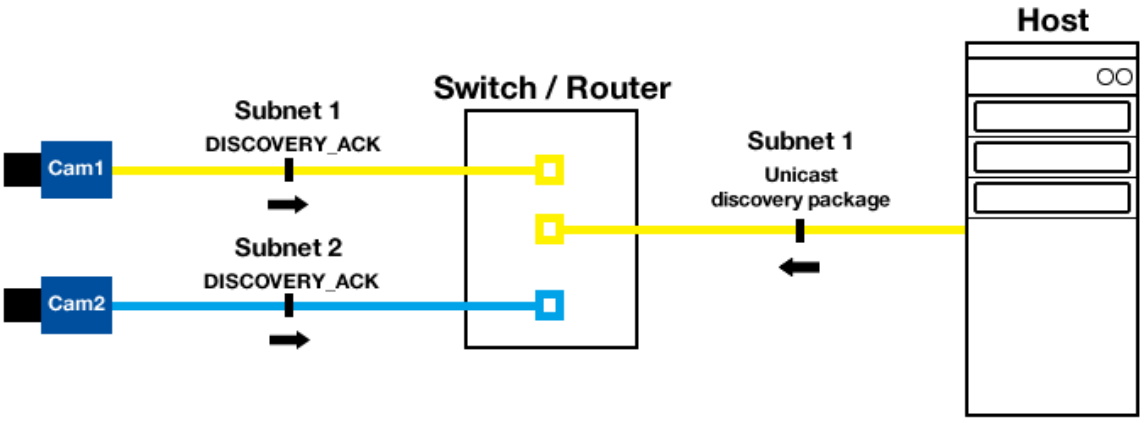
During the standard device discovery process the host sends a broadcast discovery packet into each subnet that the host itself is directly connected to via it's network interfaces. All GigE Vision™ devices receiving this discovery packet will send an acknowledge packet (DISCOVERY_ACK) back to the host.

In most cases this is sufficient to discover every GigE Vision™ devices as they are connected to host either directly or via switch to the host. Sometimes however the host does **not** reside within the same subnet as the GigE Vision™ device which makes it impossible to discover it via a broadcast packet as switches or routers typically don't forward these packets into other subnets.



Device discovery using only a broadcast discovery

In order to allow using devices residing in different subnets as well, so called "Unicast Discovery Packets" can be sent to a GigE Vision™ device in addition to the usual broadcast discovery packets. In order to do this the IPv4 address of the device to discover must be known upfront.



Device discovery using a broadcast discovery and an additional unicast

1.19.3.2.2 How it works

Please have a look at the **"Set Up Device Discoveries In Different Subnets"** chapter of the **mvIPConfigure** (p. 139) manual for a detailed description.

1.19.4 Working with triggers

There are several use cases concerning trigger:

- **Processing triggers from an incremental encoder** (p. 348)
- **Generating a pulse width modulation (PWM)** (p. 351)
- **Outputting a pulse at every other external trigger** (p. 354)
- **Creating different exposure times for consecutive images** (p. 355)
- **Detecting overtriggering** (p. 358)
- **Triggering of an indefinite sequence with precise starting time** (p. 362)
- **Low latency triggering** (p. 365)

1.19.4.1 Processing triggers from an incremental encoder

1.19.4.1.1 Basics

The following figure shows the principle of an incremental encoder:

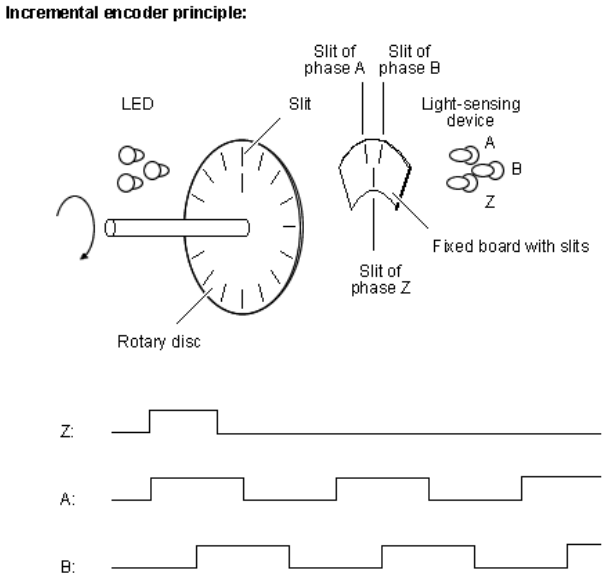


Figure 1: Principle of an incremental encoder

This incremental encoder will send a A, B, and Z pulse. With these pulses there are several ways to synchronize an image with an incremental encoder.

1.19.4.1.2 Using Encoder Control

To create an external trigger event by an incremental encoder, please follow these steps:

1. Connect the incremental encoder output signal A, for example, to the digital input 0 (**"Line4"**) of the mv↩ BlueCOUGAR-X .
   This line counts the forward pulses of the incremental encoder.
2. Depending on the signal quality, it could be necessary to **set a debouncing filter at the input** (p. 370) (red box in Figure 3):
   Adapt in *"Setting -> Base -> Camera -> GenICam -> Digital I/O Control"* the *"Line Selector"* to **"Line4"** and set *"mv Line Debounce Time Falling Edge"* and *"mv Line Debounce Time Rising Edge"* according to your needs.

3. Set the trigger *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> Trigger Selector"* **"FrameStart"** to the **"Encoder0"** (*"Trigger Source"*) signal.
4. Then set *"Setting -> Base -> Camera -> GenICam -> Encoder Control -> Encoder Selector"* to **"Encoder0"** and
5. adapt the parameters to your needs.

See also

Encoder Control (p. 156)

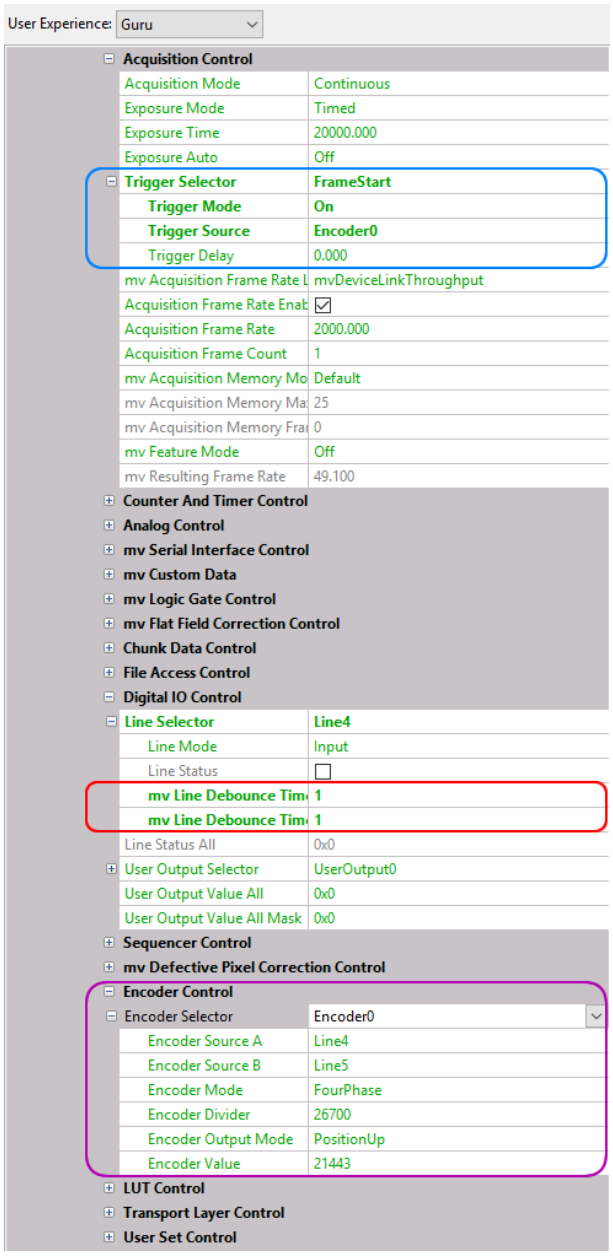


Figure 2: wxPropView settings

Note

The max. possible frequency is 5 KHz.

1.19.4.1.2.1 Programming the Encoder Control

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

  // more code
  GenICam::EncoderControl ec( pDev );
  ec.encoderSelector.writeS( "Encoder0" );
  ec.encoderSourceA.writeS( "Line4" );
  ec.encoderMode.writeS( "FourPhase" );
  ec.encoderOutputMode.writeS( "PositionUp" );
  // more code
```

1.19.4.1.3 Using Counter

It is also possible to use **Counter** and **CounterEnd** as the trigger event for synchronizing images with an incremental encoder

To create an external trigger event by an incremental encoder, please follow these steps:

1. Connect the incremental encoder output signal A, for example, to the digital input 0 (**"Line4"**) of the mv↩ BlueCOUGAR-X .
   This line counts the forward pulses of the incremental encoder.
2. Set *"Setting -> Base -> Camera -> GenICam -> Counter and Timer Control -> Counter Selector"* to **"↩ Counter1"** and
3. **"Counter Event Source"** to **"Line4"** to count the number of pulses e.g. as per revolution (e.g. **"Counter Duration"** to **3600**).
4. Then set the trigger *"Setting -> Base -> Camera -> GenICam -> Acquisition Control -> Trigger Selector"* **"FrameStart"** to the **"Counter1End"** (*"Trigger Source"*) signal.

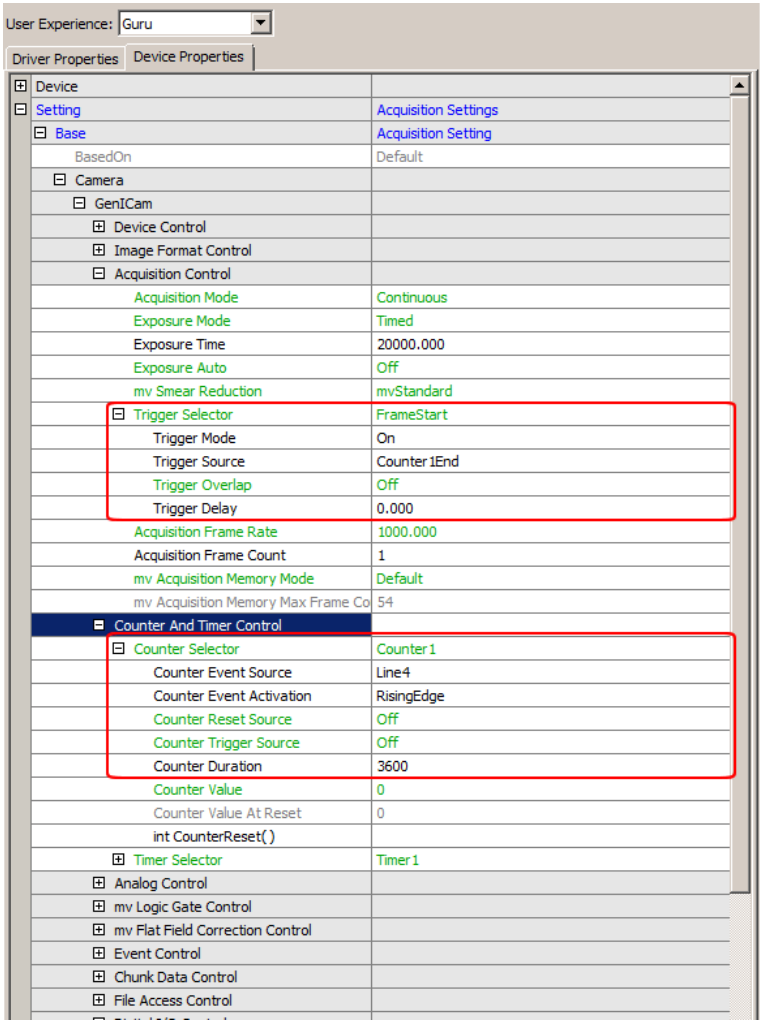


Figure 3: wxPropView setting

To reset **"Counter1"** at Zero degrees, you can connect the digital input 1 (**"Line5"**) to the encoder Z signal.

1.19.4.2 Generating a pulse width modulation (PWM)

1.19.4.2.1 Basics

To dim a laser line generator, for example, you have to generate a pulse width modulation (PWM).

For this, you will need

- 2 timers and
- the active signal of the second timer at an output line

1.19.4.2.2 Programming the pulse width modulation

You will need two timers and you have to set a trigger.

- *Timer1* defines the interval between two triggers.
- *Timer2* generates the trigger pulse at the end of *Timer1*.

The following sample shows a trigger

- which is generated every second and
- the pulse width is 10 ms:

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...

// Master: Set timers to trig image: Start after queue is filled
    GenICam::CounterAndTimerControl catcMaster(pDev);
    catcMaster.timerSelector.writeS( "Timer1" );
    catcMaster.timerDelay.write( 0. );
    catcMaster.timerDuration.write( 1000000. );
    catcMaster.timerTriggerSource.writeS( "Timer1End" );

    catcMaster.timerSelector.writeS( "Timer2" );
    catcMaster.timerDelay.write( 0. );
    catcMaster.timerDuration.write( 10000. );
    catcMaster.timerTriggerSource.writeS( "Timer1End" );
```

See also

Counter And Timer Control (p. 148)

Note

Make sure the *Timer1* interval must be larger than the processing time. Otherwise, the images are lost.

Now, the two timers will work like the following figure illustrates, which means

- *Timer1* is the trigger event and
- *Timer2* the trigger pulse width:

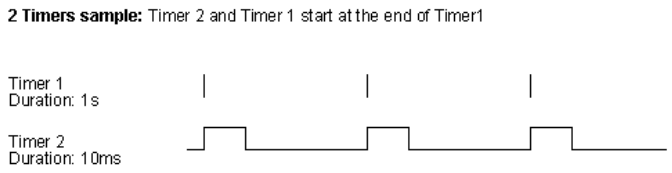


Figure 1: Timers

The timers are defined, now you have to set the digital output, e.g. **"Line 0"**:

```
// Set Digital I/O
    GenICam::DigitalIOControl io(pDev);
    io.lineSelector.writeS( "Line0" );
    io.lineSource.writeS( "Timer2Active" );
```

See also

Digital I/O Control (p. 155)

This signal has to be connected with the digital inputs of the application.

1.19.4.2.3 Programming the pulse width modulation with wxPropView

The following figures show, how you can set the timers using the GUI tool **wxPropView** (p. 139)

1. Setting of *Timer1* (blue box) on the master camera:

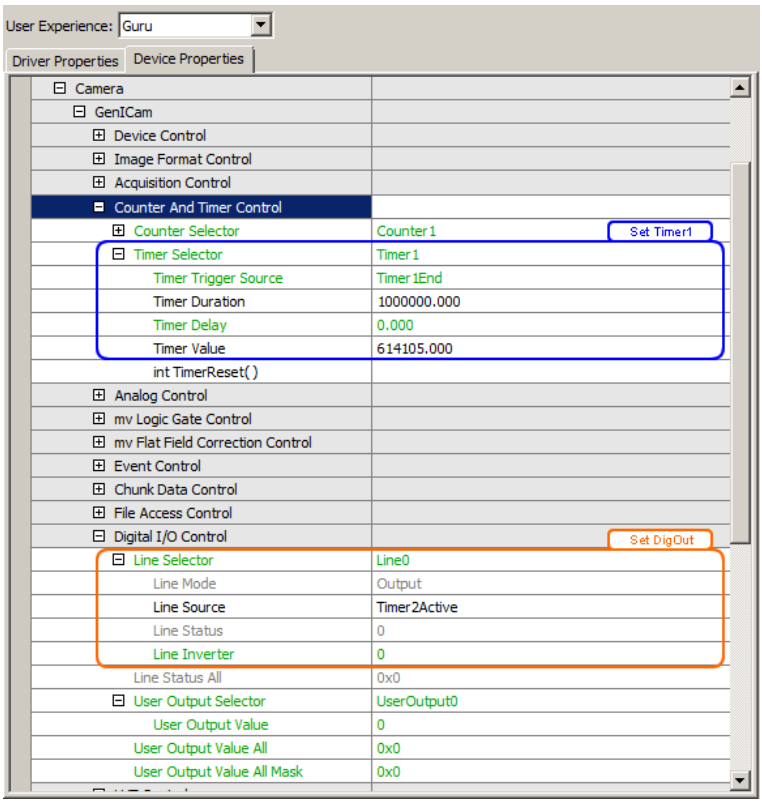


Figure 2: wxPropView - Setting of *Timer1*

2. Setting of *Timer2* (purple on the master camera):

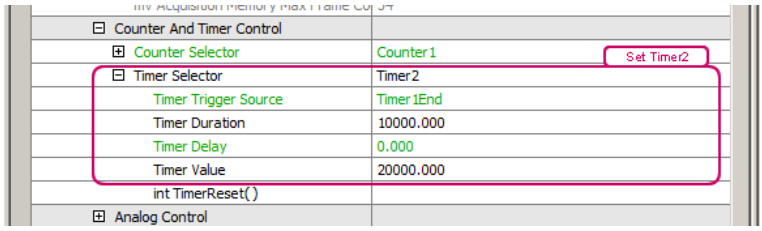


Figure 3: wxPropView - Setting of *Timer2*

3. Assigning timer to DigOut (orange box in Figure 2).

1.19.4.3 Outputting a pulse at every other external trigger

To do this, please follow these steps:

1. Switch *"Trigger Mode"* to **"On"** and
2. Select the *"Trigger Source"*, e.g. **"Line5"**.
3. Use *"Counter1"* and count the number of input trigger by setting the *"Counter Duration"* to **"2"**.
4. Afterwards, start *"Timer1"* at the end of *"Counter1"*:

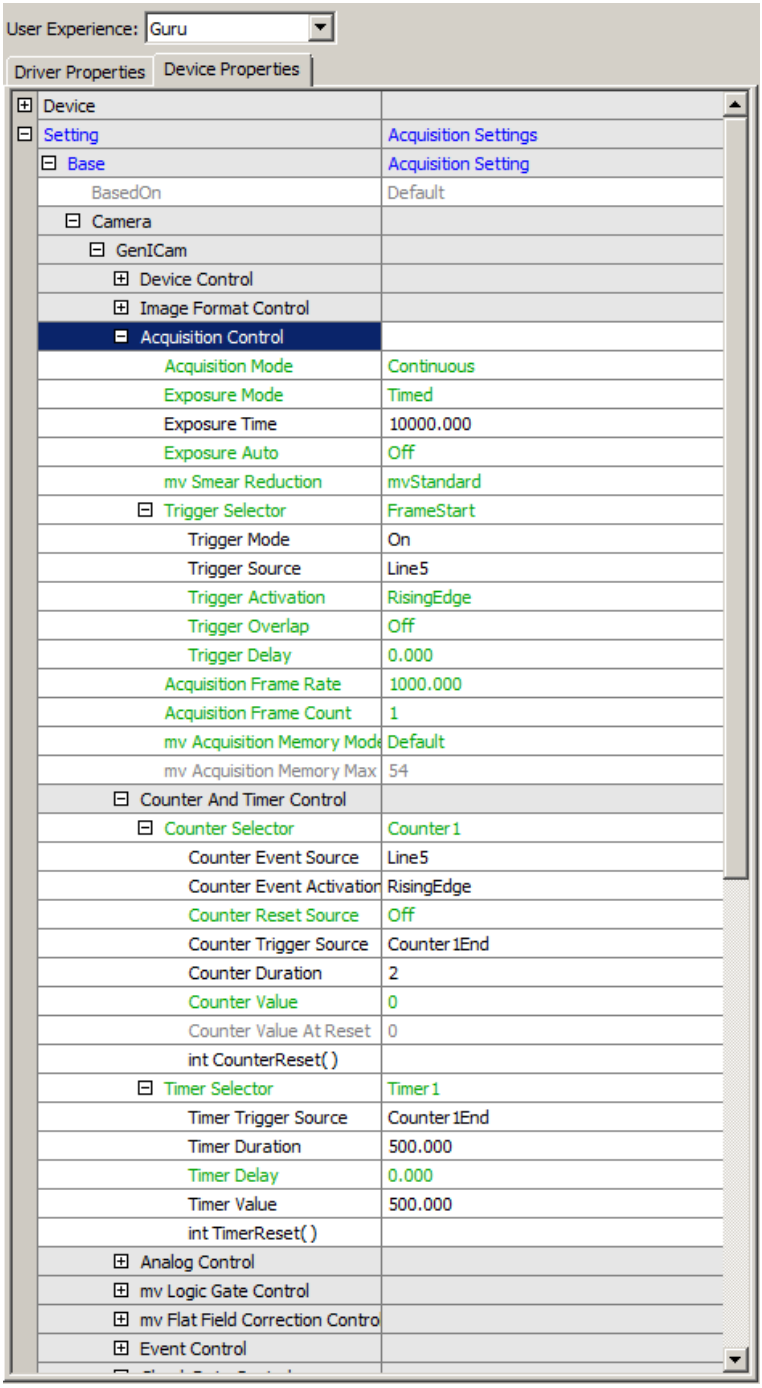


Figure 1: wxPropView - Setting the sample

The *"Timer1"* appears every second image.

Now, you can assign *"Timer1Active"* to a digital output e.g. **"Line3"**:

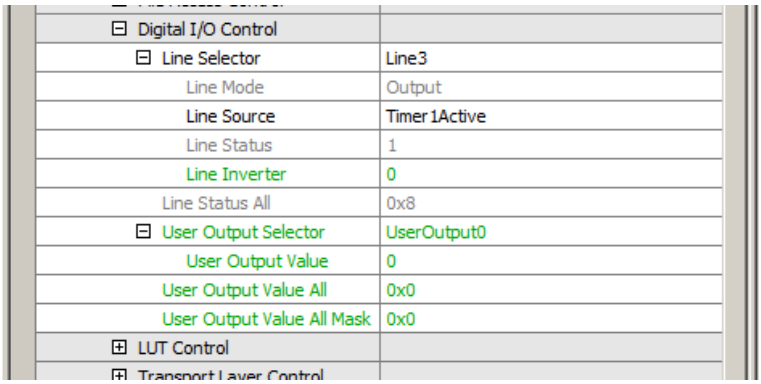


Figure 2: Assigning the digital output

Note

You can delay the pulse if needed.

1.19.4.4 Creating different exposure times for consecutive images

If you want to create a sequence of exposure times, you have to trigger the camera "externally" via pulse width:

1. Use **Timer** and **Counter** to build a sequence of different pulse widths.
2. Use the **Counter** for the time between the exposures (with respect to the readout times).
3. Afterwards, use an **AND** gate followed by **OR** gate to combine different exposure times.

Note

Please be sure that the sensor can output the complete image during **Counter1** or **Counter2**. Otherwise, only one integration time will be used.

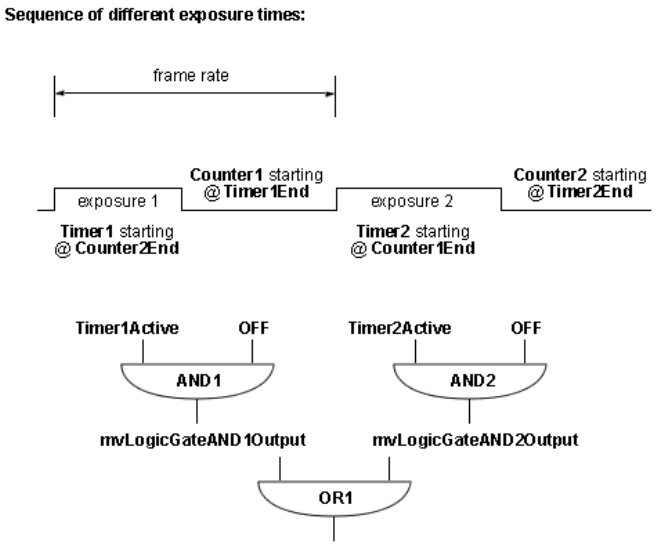


Figure 1: wxPropView - Logic gate principle

You can set this sample in **wxPropView** (p. 139). E.g. the sensor makes 22.7 frames per second in **Continuous Mode** (p. 144). This means that the sensor needs 44 ms to output the complete image.

$$\frac{1}{22.7} = \text{approx. } 44 \text{ ms}$$

We take 55 ms to be sure. Now, as different exposure times we take 1 ms (**Timer1**) and 5 ms (**Timer2**). To get the 55 ms, we have to add 54000 us (**Counter1**) and 50000 us (**Counter2**).

Finally, you have to set the logic gate as shown in the figure:

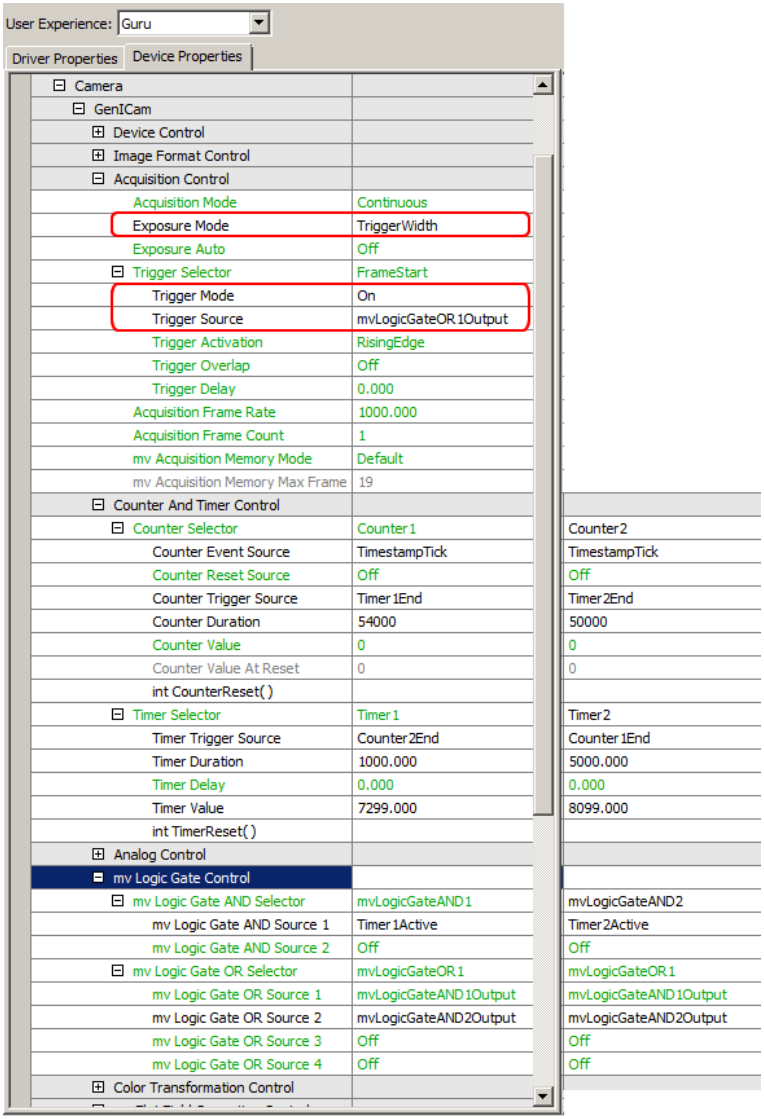


Figure 2: wxPropView - Logic gate setting

Note

Because there are 4 counters and 2 timers you can only add one further exposure time using one counter as a timer.

So if you want other sequences, you have to use the counters and timers in a flexible way as show in the next sample:

1.19.4.4.1 Sequence with 4 times exposure A followed by 1 time exposure B

If you have an external trigger, you can use the counter and timer to create longer exposure sequences.

For example, if you want a sequence with 4 times exposure A followed by 1 time exposure B you can count the trigger events. That means practically:

1. Use **Counter1** to count 5 trigger signals then
2. issuing **Timer2** for the long exposure time (re-triggered by **Counter1End**).
3. Every trigger issues **Timer1** for the short exposure.
4. Afterwards, an **AND** gate followed by **OR** gate combines the different exposure times.

In **wxPropView** (p. 139) it will look like this:

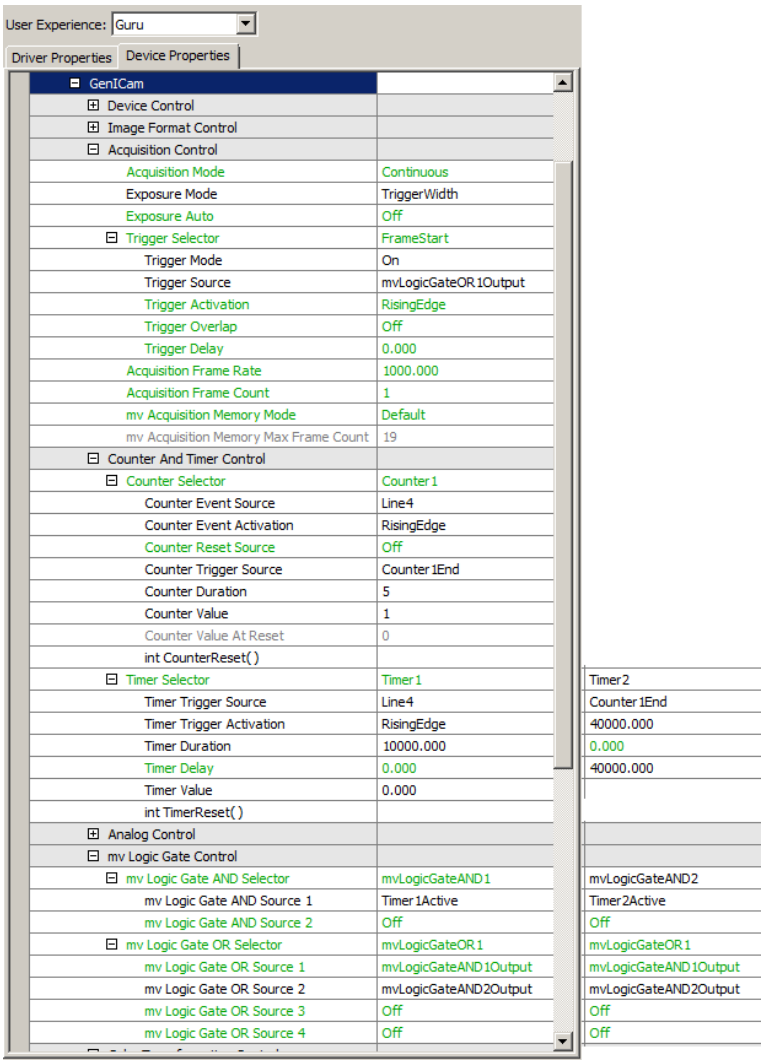


Figure 3: wxPropView - Logic gate setting 2

1.19.4.5 Detecting overtriggering

1.19.4.5.1 Scenario

The image acquisition of a camera consists of two steps:

- exposure of the sensor and
- readout of the sensor data

During these steps, a trigger signal will be skipped:

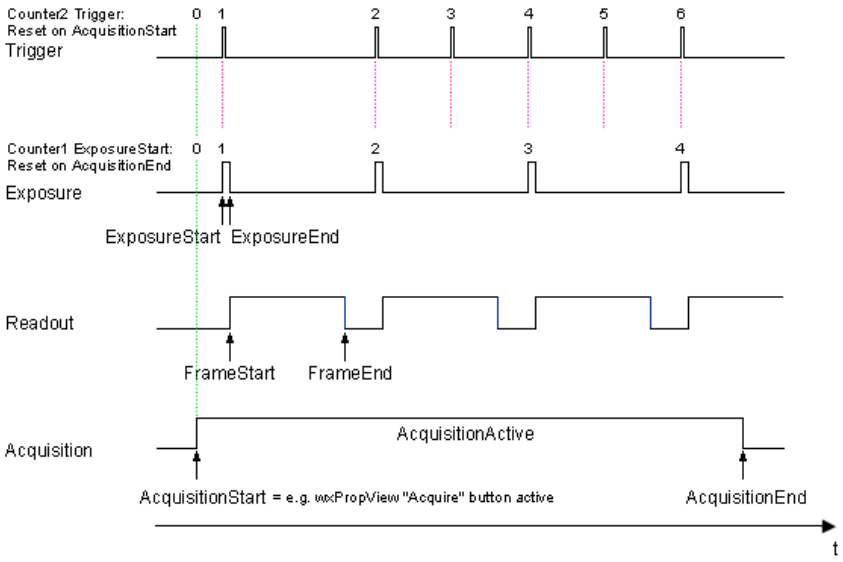


Figure 1: Trigger counter increases but the start exposure counter not

To notice overtriggering, you can use **counters** (p. 148):

- One counter counts the incoming trigger signals, the
- second counter counts the `ExposureStart` signals.

Using the **chunk data** (p. 154) you can overlay the counters in the live image.

1.19.4.5.2 Setting the overtrigger detector using wxPropView

First of all, we have to set the **trigger** in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"* with following settings:

Property name wxPropView	Setting
Trigger Selector	`FrameStart`
Trigger Mode	`On`
Trigger Source	`Line4`
Trigger Activation	`RisingEdge`
Exposure Mode	`Timed`

This trigger will start an acquisition after a rising edge signal on line 4 (= **DigIn0** (p. 84)).

Now, set the two counters. Both counters (`Counter1` and `Counter2`) will be reset and start after the acquisition (`AcquisitionStart`) has started.

While `Counter1` increases with every `ExposureStart` event (see figure above for the event and acquisition details) ...

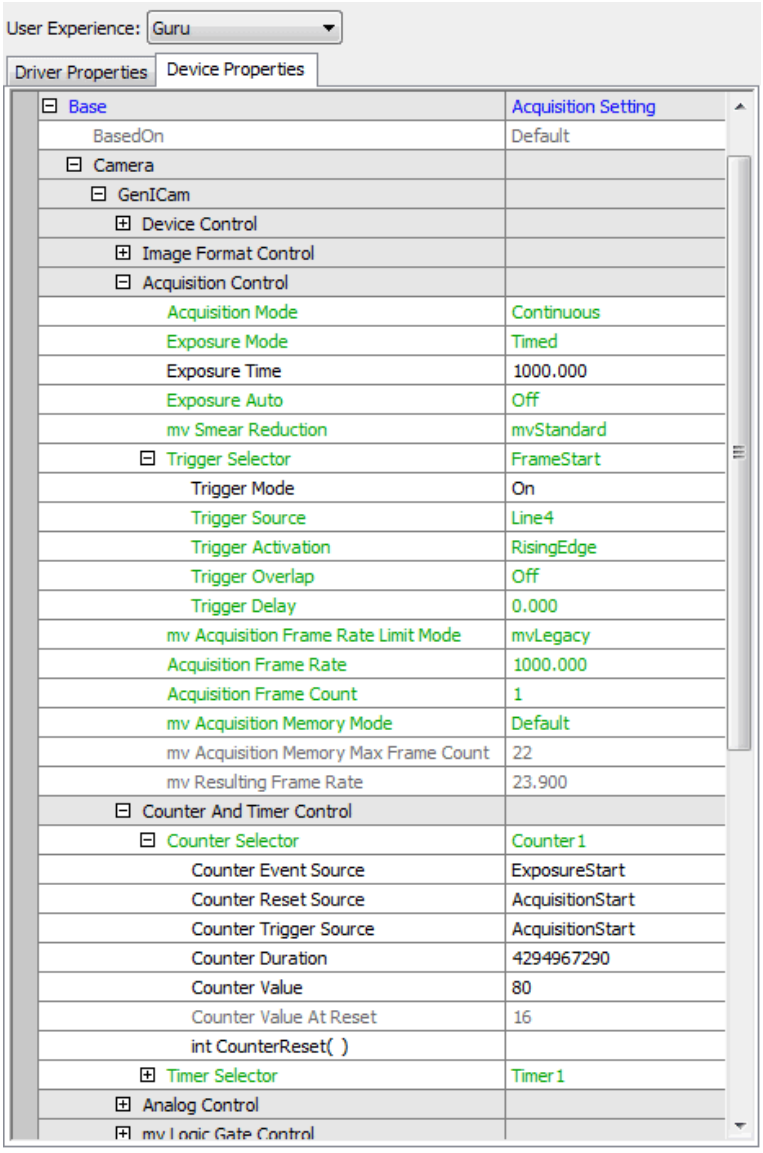


Figure 2: Setting Counter1

... `Counter2` increases with every `RisingEdge` of the trigger signal:

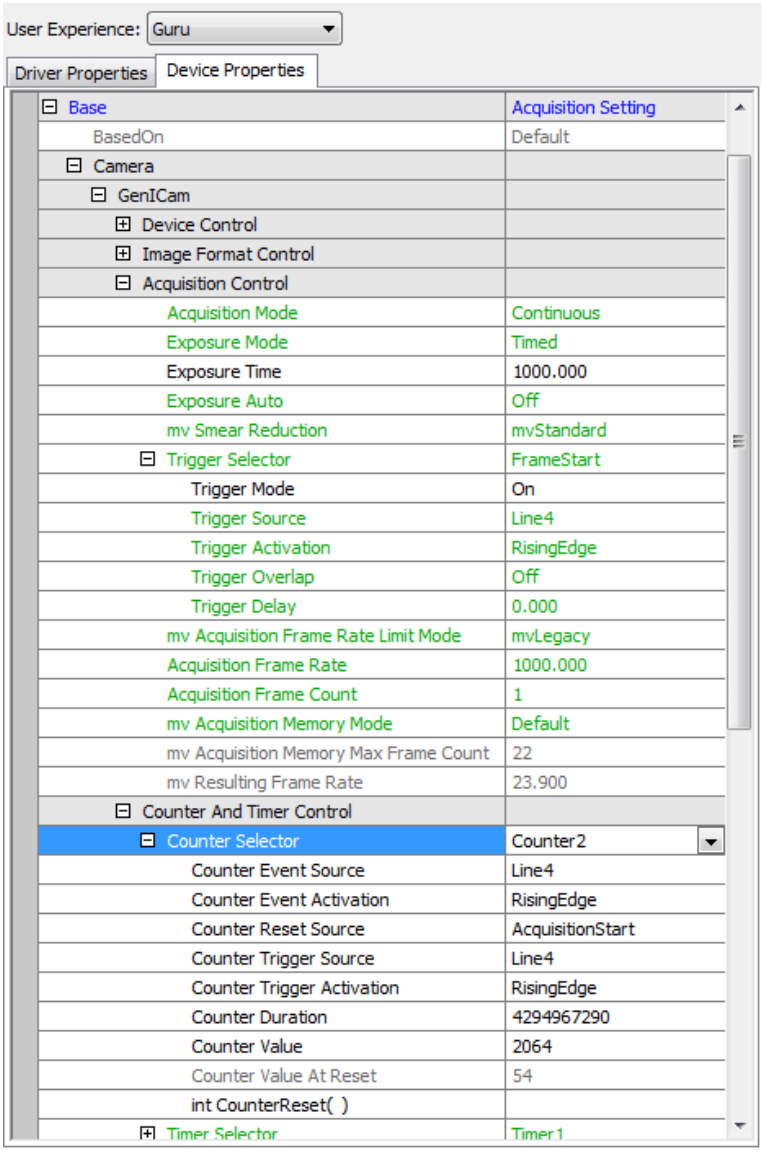


Figure 3: Setting Counter2

Now, you can check if the trigger signal is skipped (when a rising edge signal is active during readout) or not by comparing the two counters.

Enable the inclusion of the selected chunk data ("`Chunk Mode Active = 1`") in the payload of the image in *"Setting -> Base -> Camera -> GenICam -> Chunk Data Control"*:

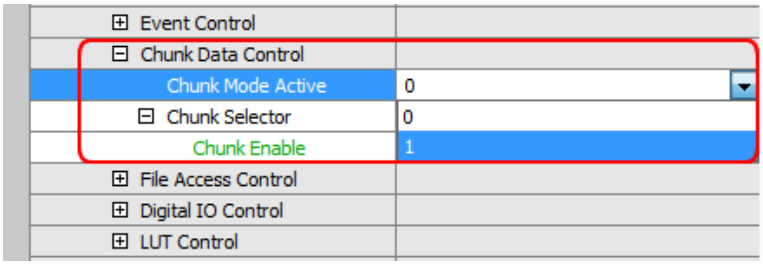


Figure 4: Enable chunk data

Activate the info overlay in the display area. Right-click on the live display and select: *"Request Info Overlay"*

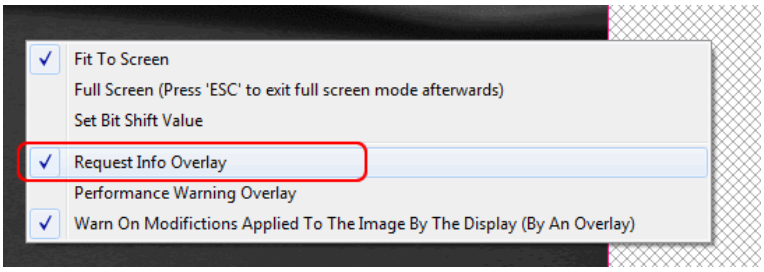


Figure 5: Show chunk data

The following figure shows that no trigger signal is skipped:



Figure 6: Trigger Signal counter equals ExposureStart counter

The following figure shows that the acquisition is overtriggered:



Figure 7: Trigger Signal counter is higher than ExposureStart counter

1.19.4.6 Triggering of an indefinite sequence with precise starting time

1.19.4.6.1 Scenario

Especially in the medical area, there are applications where a triggered acquisition is started, for example, with a foot switch. Following challenges have to be solved in combination with these applications:

- The user wants the acquired image immediately (precise starting time).
- It is not known, when the user stops the acquisition (indefinite sequence).

Using *AcquisitionStart* as the trigger source, it could take between 10 and 40 ms until the camera acquires the first frame. That's not really an immediately acquisition start. It is recommended to use *FrameStart* as the trigger source instead. However, according to the time the trigger event occurs, there will be a timeout during the first frame in nearly all cases.

You can avoid this using a timer which generates a "high" every 100 us and which is connected to the trigger input *Line4* using a logical *AND* gate. I.e. if the timer is "high" and there is a trigger signal at *Line4* then the logical conjunction is true. The AND gate result is then connected as *TriggerSource* of the *FrameStart* trigger using a logical *OR* gate. I.e. as soon as the logical AND conjunction is true, the trigger source is true and the image acquisition will start.

The following figure illustrates the settings:

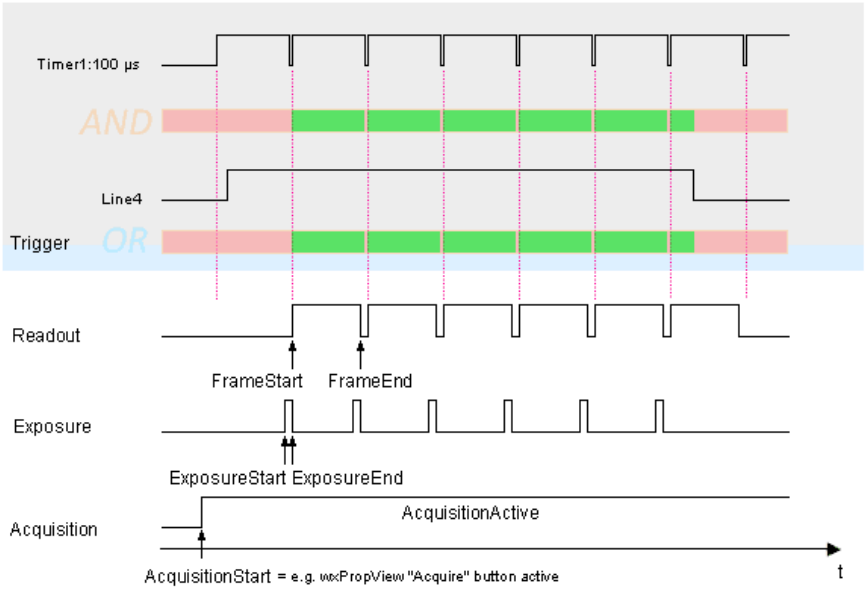


Figure 1: Schematic illustration of the settings

With this setting, there is still an acceptable time delay of approx. 100 to 130 us possible.

1.19.4.6.2 Creating the use case using wxPropView

First of all, we have to set the **timer** in *"Setting -> Base -> Camera -> GenICam -> Counter And Timer Control"* with following settings:

Property name wxPropView	Setting
Timer Selector	`Timer1`
Timer Trigger Source	`Timer1End`
Timer Duration	`100.000`

Afterwards, we have to set the **logical gates** in *"Setting -> Base -> Camera -> GenICam -> mv Logic Gate Control"* with following settings:

Property name wxPropView	Setting
mv Logic Gate AND Selector	`mvLogicGateAND1`
mv Logic Gate AND Source 1	`Line4`
mv Logic Gate AND Source 2	`Time1Active`
mv Logic Gate OR Selector	`mvLogicGateOR1`
mv Logic Gate OR Source 1	`mvLogicGateAND1Output`

Finally, we have to set the **trigger** in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"* with following settings:

Property name wxPropView	Setting
Trigger Selector	`FrameStart`
Trigger Mode	`On`
Trigger Source	`mvLogicGateAND1Output`
Trigger Activation	`RisingEdge`
Exposure Mode	`Timed`

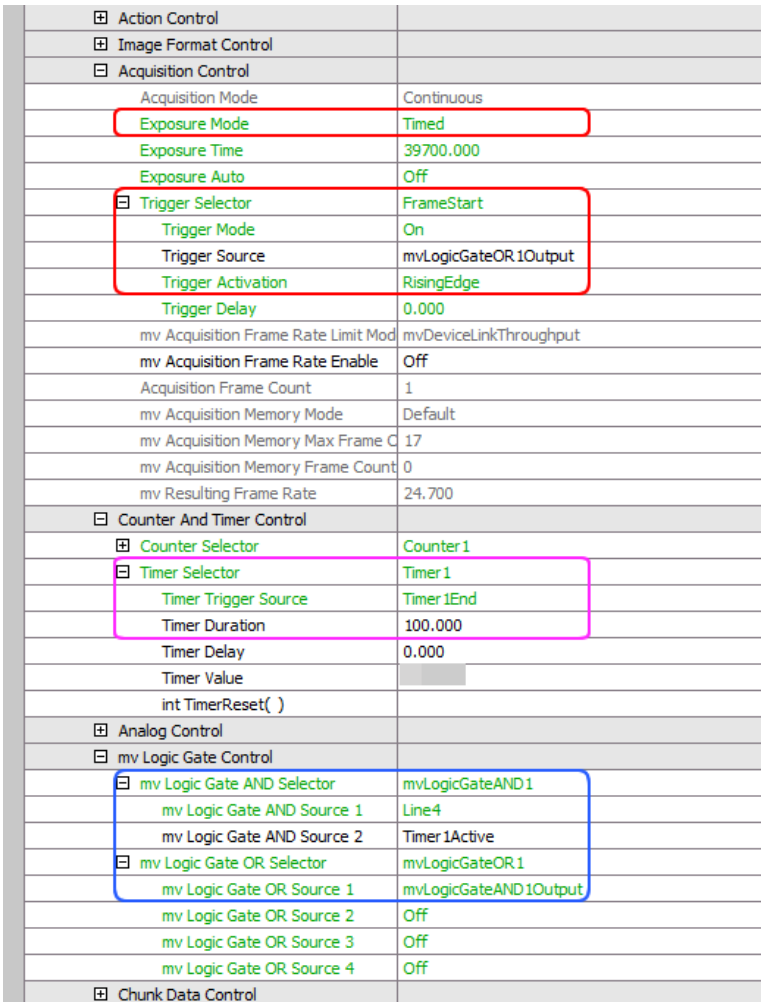


Figure 2: Sample settings

1.19.4.7 Low latency triggering

Since

Firmware revision 2.45

1.19.4.7.1 Introduction

Because the exposure start is synced with the line-period, there is a jitter of the start of up to one line. Additionally, there is a latency of 2 or 4 line-periods (depending on the sensor) until the exposure actually starts. The following image shows this visually:

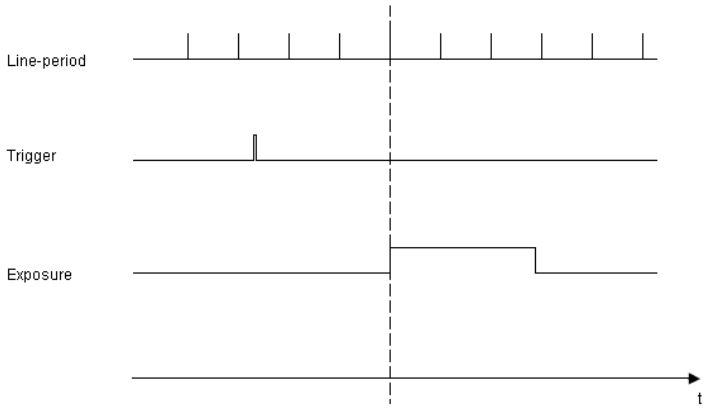


Figure 1: Normal trigger mode

The exposure time increment is a multiple of the line-period. This trigger behavior works also with overlapped exposure and readout.

With the Pregius global shutter sensors of the 2nd, 3rd, and 4th generation from Sony, it is possible to start the exposure on the trigger. There is no latency and no jitter. The length of exposure is also not based on the line-period, because the sensor does not depend on it in this mode. The minimum exposure time is limited by several line periods though. Because during readout the sensor obviously requires the line-periods this mode is only supported in non-overlapped exposure and readout:

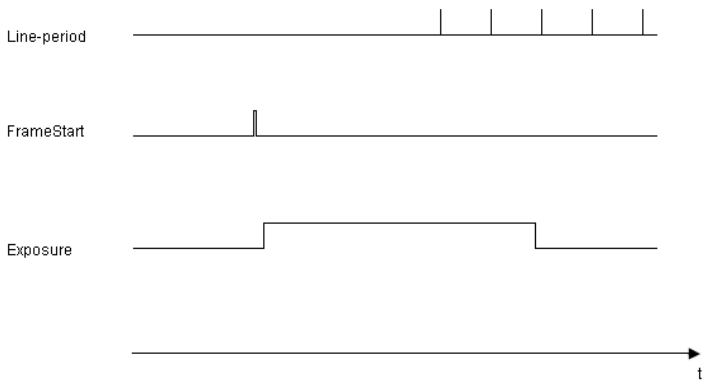


Figure 2: LowLatency trigger mode

1.19.4.7.2 Using wxPropView

To activate the LowLatency trigger mode, please follow these steps:

1. Set the **TriggerSelector** in *"Setting → Base → Camera → GenICam → AcquisitionControl"* to **FrameStart** .
2. Afterwards, change to **TriggerMode** and
3. Select **mvLowLatency**.

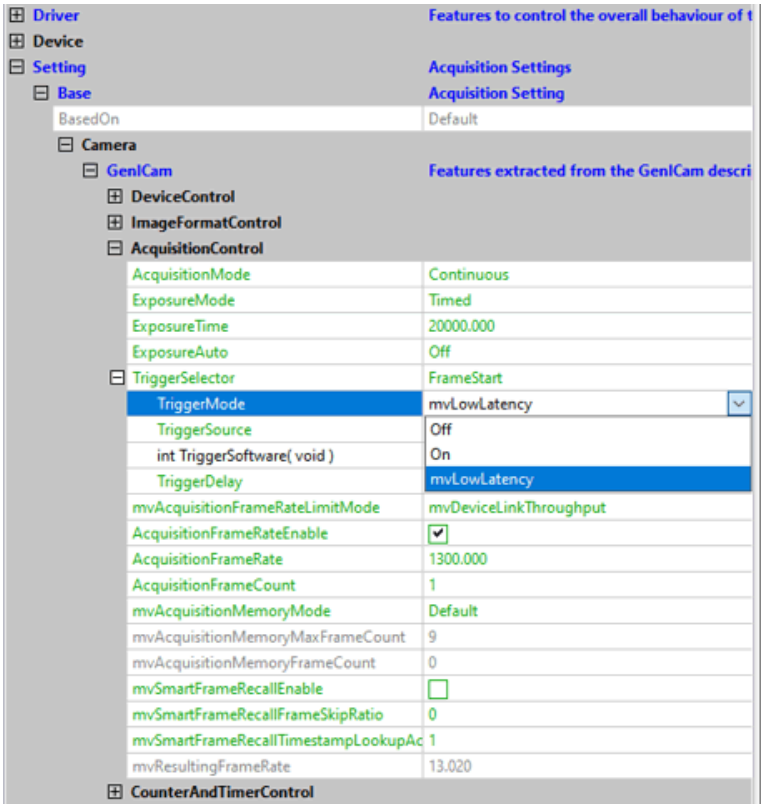


Figure 3: mvLowLatency

1.19.5 Working with I/Os

There are several use cases concerning I/Os:

- **Controlling strobe or flash at the outputs** (p. 367)
- **Creating a debouncing filter at the inputs** (p. 370)
- **Using motorized lenses with mvBlueCOUGAR-XD** (p. 373)

1.19.5.1 Controlling strobe or flash at the outputs

Of course, MATRIX VISION devices support strobe or flash lights. However, there are several things you have to keep in mind when using strobes or flash:

1. Be sure that the illumination fits with the movement of the device under test.
2. Bright illumination and careful control of exposure time are usually required.
3. To compensate blur in the image, short exposure times are needed.

Alternatively, you can use flash with short burn times. For this, you can control the flash using the camera. The following figures show, how you can do this using **wxPropView** (p. 139)

1. Select in *"Setting -> Base -> Camera -> Digital I/O Control"* the output line with the **"Line Selector"** to which the strobe or flash is connected.
2. Now, set the **"Line Source"** to **"mvExposureAndAcquisitionActive"**.
   This means that the signal will be high for the exposure time and only while acquisition of the camera is active.

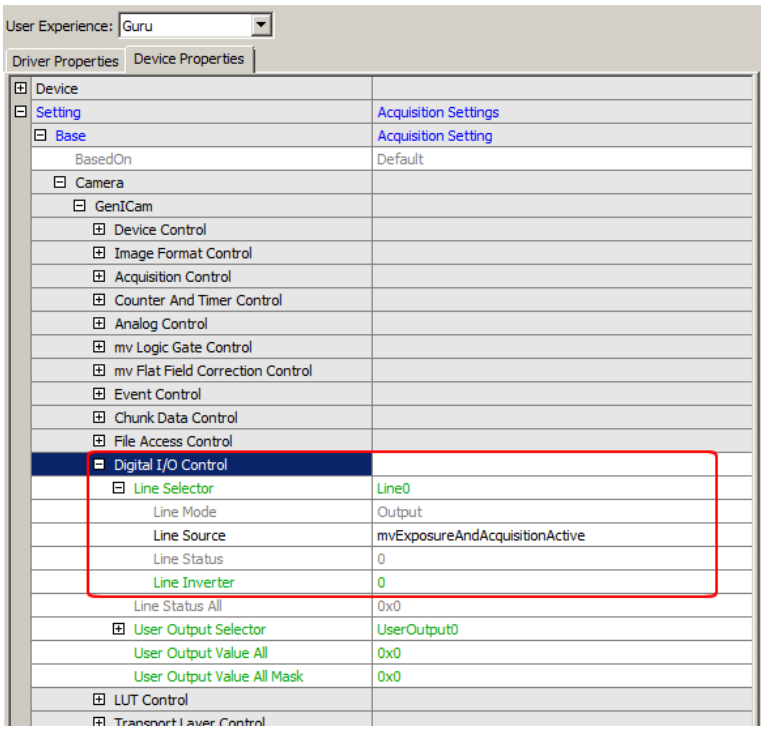


Figure 1: Setting the **"Line Source"** to **"mvExposureAndAcquisitionActive"**

Additionally, you can shift the signal controlling the flash or strobe using the camera. The camera will send the signal at the selected output line either sooner or later. The following figure demonstrates the procedure using **wxPropView** (p. 139)

1. Enable in *"Setting -> Base -> Camera -> AcquisitionControl"* the property **"mvExposureActiveShift↩ Enable"**.
   The property **"mvExposureActiveShift"** will appear.

2. Now you can select a shift value and **"mvExposureAndAcquisitionActive"** will be available either later (with positive values) or sooner (with negative values).

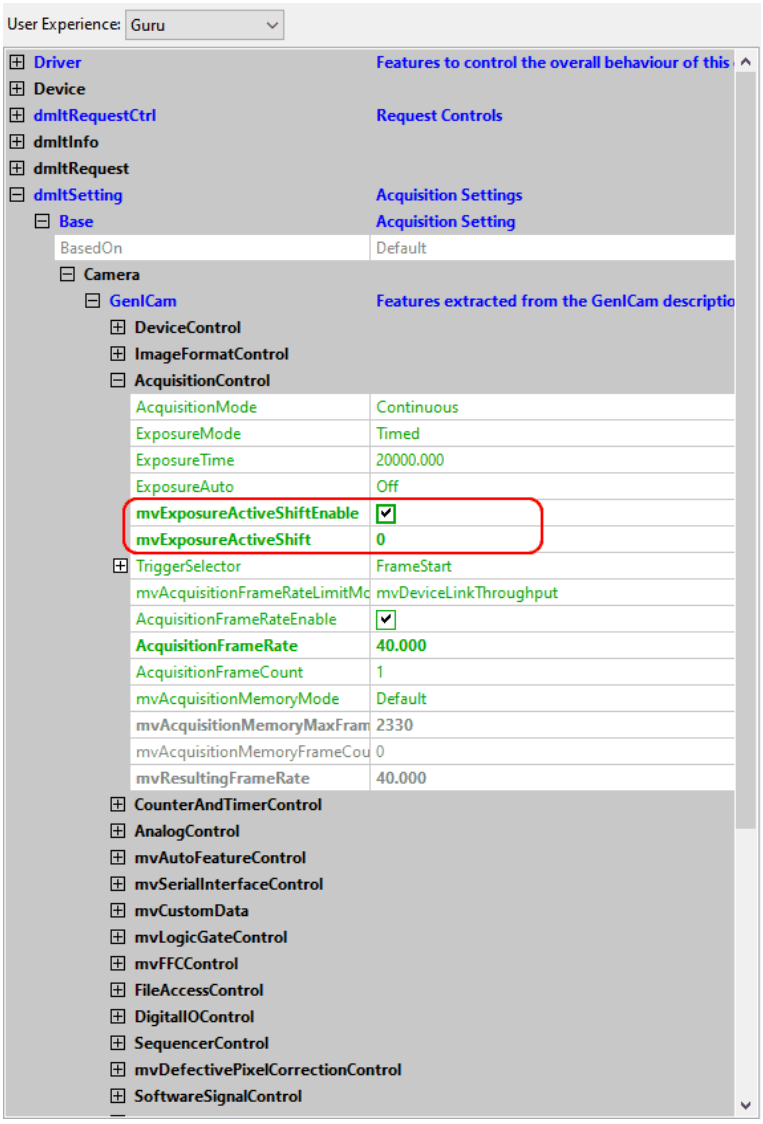


Figure 2: Controlling the shift of **"mvExposureAndAcquisitionActive"** at the output line

Note

This can be combined using an external trigger.

1.19.5.1.1 Special case: Rolling shutter Starvis sensors

Since

Firmware version 2.24.975

With the rolling shutter flash mode, the exposure time which is set corresponds to the time where all lines are exposed simultaneously. The exposure signal corresponds to the time where the last line starts until the first line ends. The rolling shutter flash mode avoids motion effects and is suitable for exposure times $>$ 300 usec.

Additional to the digital output settings in **Controlling strobe or flash at the outputs** (p. 367) you have to set

1. the **"mvShutterMode"** to **"mvRollingShutterFlash"** and
2. the desired **"ExposureTime"**.

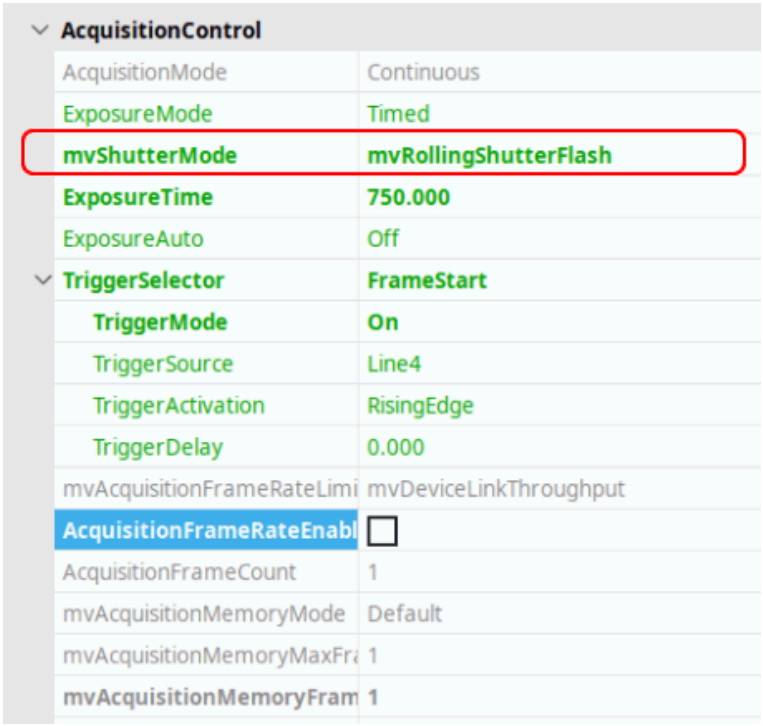


Figure 3: Setting the **"mvShutterMode"** to **"mvRollingShutterFlash"**

Note

If you have an external trigger, please adjust the settings according to **this trigger sample** (p. 358).

1.19.5.1.2 Compensating delay of strobe or flash

Normally, the input circuitry of flash has a delay (e.g. low pass filtering). Using **"ExposureActive"** to fire strobe would actually illuminate delayed with respect to exposure of the sensor. Figure 4 shows the problem:

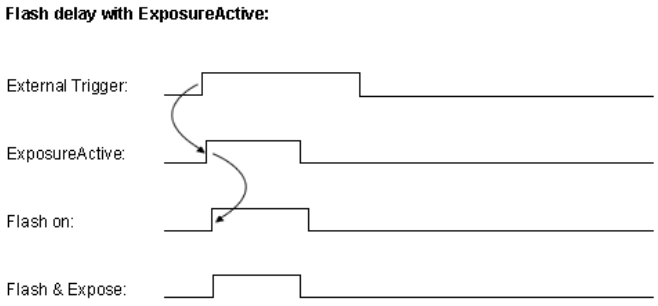


Figure 4: Flash delay with **"ExposureActive"**

To solve this issue, you can use following procedure:

1. Do not use **"ExposureActive"** for triggering strobe.

2. Build flash signal with **Timer**,
3. trigger **Timer** with external trigger (e.g. **"Line5"**).
4. Use **"Trigger Delay"** to delay exposure of the sensor accordingly.

In **wxPropView** (p. 139) it will look like this:

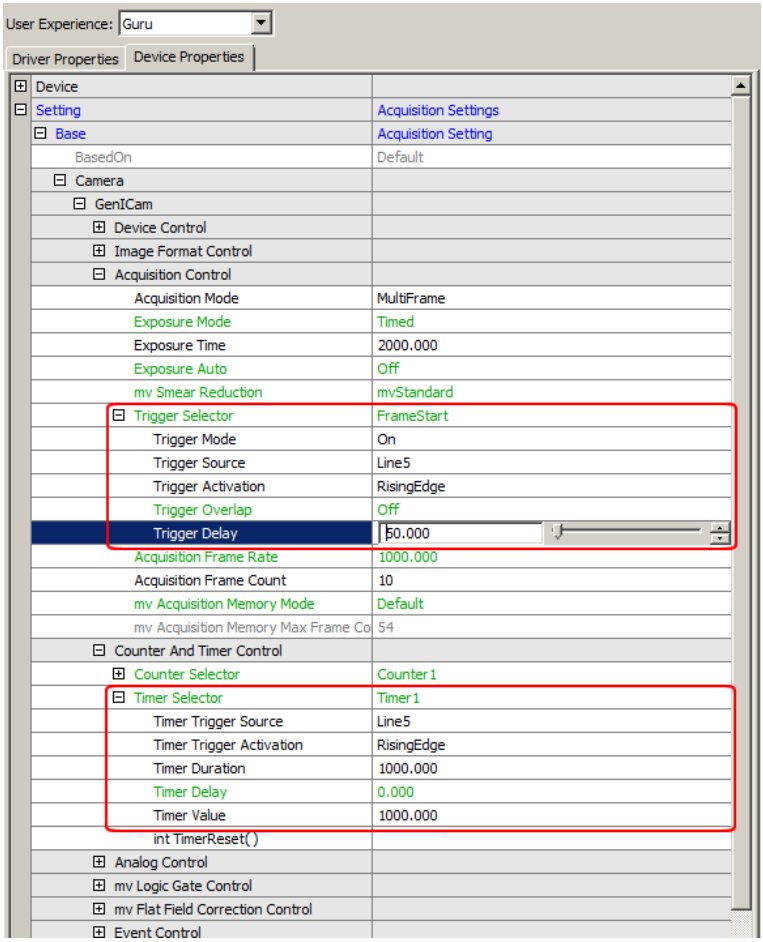


Figure 5: Working with **Timer** and **"Trigger Delay"**

1.19.5.2 Creating a debouncing filter at the inputs

In some cases, it is necessary to eliminate noise on trigger lines. This can become necessary when either

- the edges of a trigger signal are not perfect in terms of slope or
- if, because of the nature of the trigger signal source, multiple trigger edges are generated within a very short period of time even if there has just been a single trigger event.

The latter one is also called *bouncing*.

Bouncing is the tendency of any two metal contacts in an electronic device to generate multiple signals as the contacts close or open; now debouncing is any kind of hardware device or software that ensures that only a single signal will be acted upon for a single opening or closing of a contact.

To address problems that can arise from these kinds of trigger signals MATRIX VISION offers debouncing filters at the digital inputs of a device.

The debouncing filters can be found under *"Setting -> Base -> Camera -> GenICam -> Digital I/O Control -> LineSelector"* (red box in Figure 1) for each digital input:

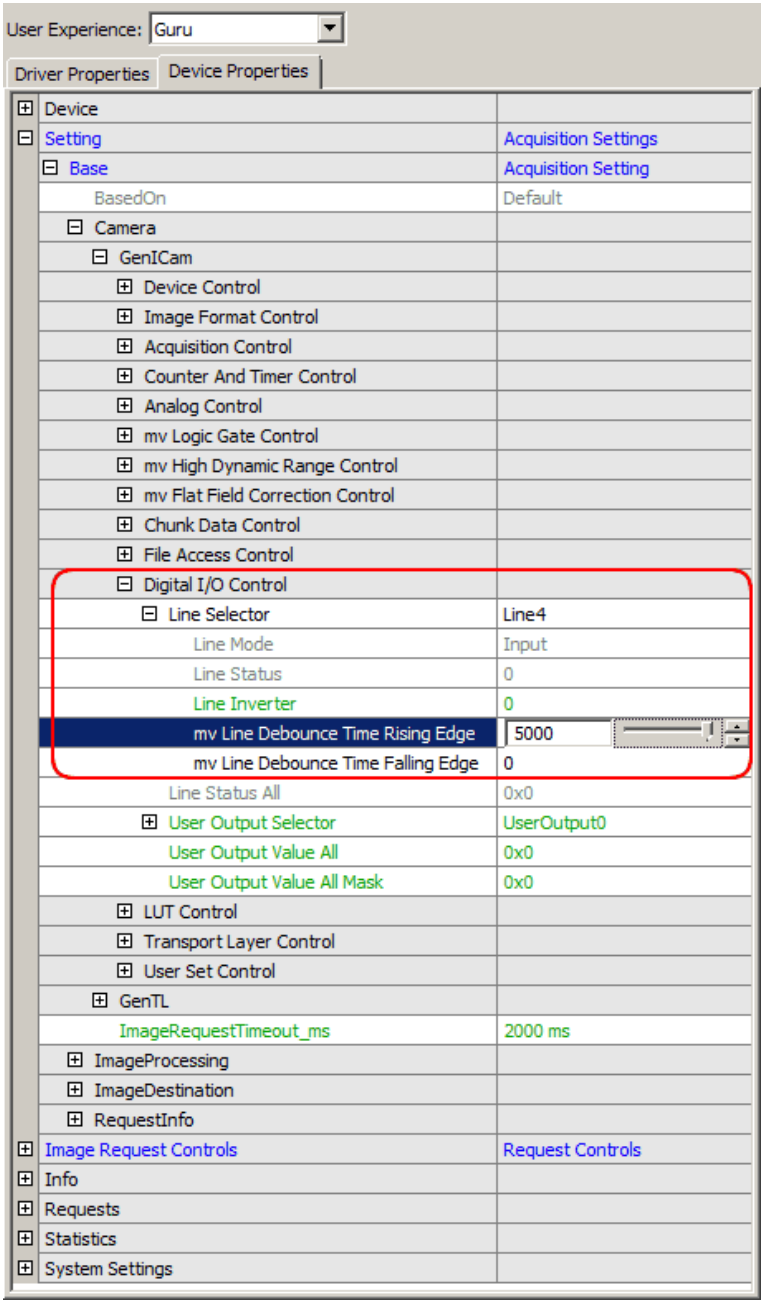


Figure 1: wxPropView - Configuring Digital Input Debounce Times

Each digital input (*LineMode* equals *Input*) that can be selected via the *LineSelector* property will offer its own property to configure the debouncing time for falling edge trigger signals (*"mv Line Debounce Time Falling Edge"*) and rising edge (*"mv Line Debounce Time Rising Edge"*) trigger signals.

The line debounce time can be configured in micro-seconds with a maximum value of up to 5000 micro-seconds. Internally each time an edge is detected at the corresponding digital input a timer will start (orange $*$ in figure 2 and 3), that is reset whenever the signal applied to the input falls below the threshold again. Only if the signal stays at a constant level for a full period of the defined *mvLineDebounceTime* the input signal will be considered as a valid trigger signal.

Note

Of course this mechanism delays the image acquisition by the debounce time.

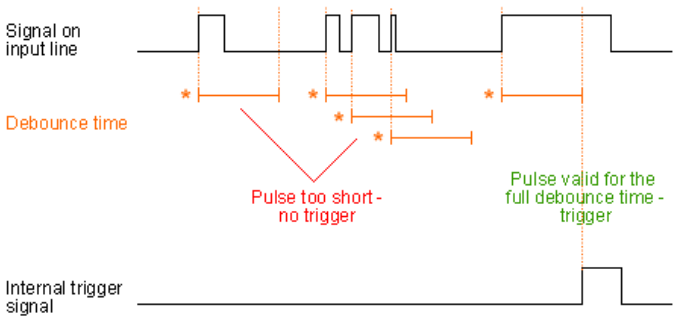


Figure 2: mvLineDebounceTimeRisingEdge Behaviour

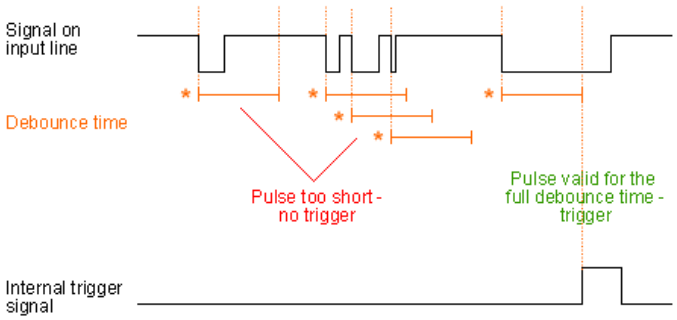


Figure 3: mvLineDebounceTimeFallingEdge Behaviour

1.19.5.3 Using motorized lenses with mvBlueCOUGAR-XD

1.19.5.3.1 Introduction

In contrast to machine vision applications with constant lightings and fixed focal lengths, outdoor applications such as traffic monitoring, security, or sports demand to control image brightness, field of view, zoom, focus, or iris. Lenses with motors offer the possibility to remotely manipulate these settings and the mvBlueCOUGAR-XD offers the possibility to control motorized lenses.

1.19.5.3.2 Types of motorized lenses and their controls

Motorized lenses differ by the fact what element is motorized and how:

- Zoom
- Focus
- Iris
  - Motor
  - Video
  - DC

Lenses with motors differ by the voltage they accept and by certain wiring specialties. Driving voltages may be between 3 and 12V DC, wiring may be 2 wires per motor (a.k.a. bipolar) or with one wire per motor and common ground. Some lenses offer potentiometers so that the actual position can be measured by a resistance. These potentiometers are not supported by the mvBlueCOUGAR-XD camera.

1.19.5.3.2.1 List of usable lenses

Motorized lenses differ by the max. sensor diameter they support and by resolution limits. mvBlueCOUGAR-XD cameras use sensors with 2/3" to 1" diameter. Lens/camera combinations must be selected having these properties in mind. The following is a list of usable lenses. It is provided for reference only. Exclusion from this list does not mean that the product is not usable with the camera per se.

Manufacturer	Details	Motorized Iris	Motorized Focus	Motorized Zoom	Video Iris
KOWA	Motorized LMZ-series up to 1" and 5 MPix resolution	X	X	X	X
Goyo Optical	GAZ series 2/3" – 1"	X		X	X
Linos	Mevis motorized	X	X		
Schneider Optics	Cinegon/Xenoplan: motorized iris	X			X
Computar	2/3" M6Z series	X	X	X	
Fujinon	2/3" and 1" lenses	X	X	X	X

1.19.5.3.3 Connecting the motorized lens to a camera

1.19.5.3.3.1 Connecting the direct drive lens motors

mvBlueCOUGAR-XD offers two connectors at the back. Use the female one on the right side for lens connection. Pinning is shown in the table on the left side below:

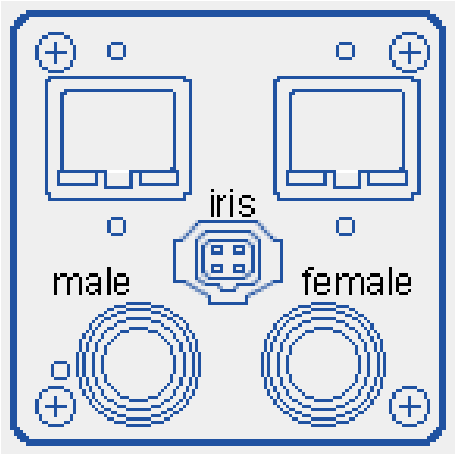


Figure 1: mvBlueCOUGAR-XD connectors

	Female connector
Pin.	Signal
1	Opto DigIn2 (line6)
2	Opto GND
3	Opto DigIn3 (line7)
4	Focus+
5	Focus-
6	Zoom+
7	Zoom-
8	Iris+
9	Iris-
10	Channel4+
11	Channel4-
12	GND

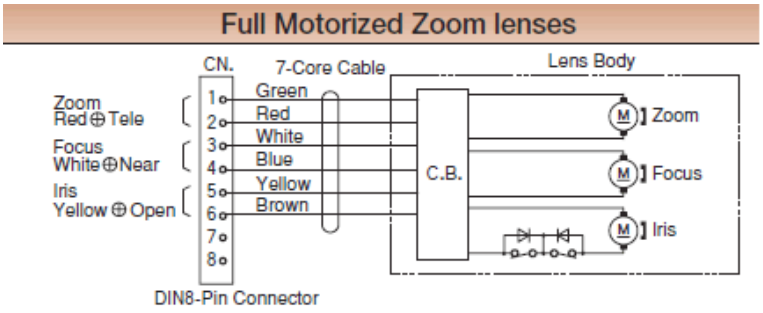


Figure 2: Wiring of full motorized zoom lens

Image above shows a typical lens wiring. The three independent motors can be seen.

- Connect Pin4 (Focus+) to CN Pin4.
- Connect Pin5 (Focus-) to CN Pin3.
- etc.

mvBlueCOUGAR-XD can deliver up to 100mA current with a selectable voltage to the outputs Focus, Zoom and Iris. Please note that the voltage applied is independent of the supply voltage of the camera. Channel 4 can be left open.

1.19.5.3.3.2 Connecting the video iris

mvBlueCOUGAR-XD camera generates a video like signal containing average brightness information and sync signals to drive a video iris type lens. The diaphragm of the lens closes at increasing brightness keeping the resulting overall brightness reaching the sensor constant.

Advantage of video iris against *AutoExposure:* Bigger range of brightness variation avoids smear in CCD as it blocks extreme bright light to hit the sensor; but is slower than *AutoExposure* and *AutoGain* which is also supported by the camera. Pining of the standardized video iris connector (4 pin EIAJ) is shown below.

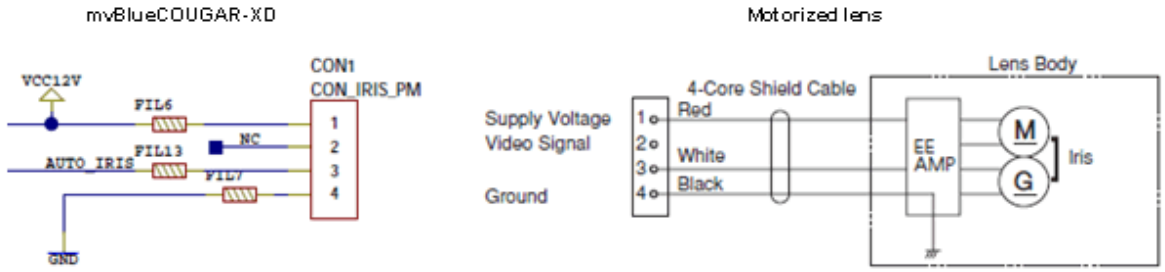


Figure 3: Pining video iris

Use the square 4 pin iris connector of the camera to directly connect the video iris.

1.19.5.3.4 Controlling the lens via viewer or the API (aka mvIMPACT Acquire)

Usage of the lens control wizard of **wxPropView** (p. 139) is recommended for setup:



Figure 4: Lens control wizard

Select the **"Drive Level"** voltage according to the lens type. Focus, Zoom and Iris buttons drive the motors at a selectable speed.

"Video Iris" can be selected to open or completely close the Iris (for setup) and for auto mode.

Note

Additional settings such as level (sensitivity) and/or ALC (peak or average) may be possible directly at the lens (via poti).
ALC settings do not have effect due to digital video signal!
Consult the manual of the lens for more details.

Using AGC/AEC & mvIrisAuto may lead to oscillating brightness.

1.19.5.3.5 Setting the video iris (example)

Purpose is to bring the video iris into a usable range so that during operation it can open if brightness goes down and further close if brightness goes up.

1. Open iris by using **mvIrisOpen** command: This opens iris to the min. F number (f/N) supported by the lens, e.g. f/1.2 (see lens manual).
2. Set exposure of camera so that the image is not saturated.
3. Set exp = 4 x min.
4. Iris auto will move the lens to f/2.4.
5. Set working exposure = 16 X min will move the Iris auto to f/4.8.

1.19.5.3.6 Controlling the lens via 3rd party libraries or APIs

The properties for **"mv Lens Control"** are MATRIX VISION specific but appear in the camera's XML-file according to **GigE Vision** (p. 215) standards and **SFNC** (p. 221) thanks to the standard. This makes it possible to use the features from third party applications or programs without problems.

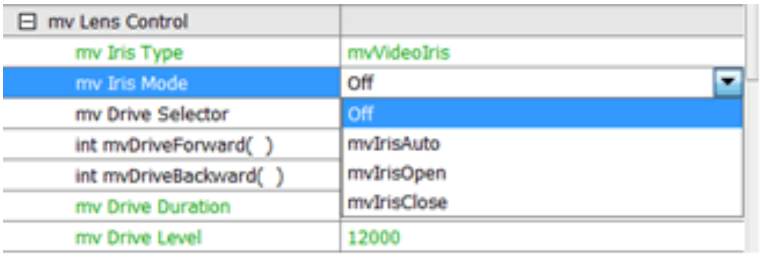


Figure 5: wxPropView - mv Lens Control

The screenshot below shows how the properties appear under MVTec HALCON's image acquisition assistant:

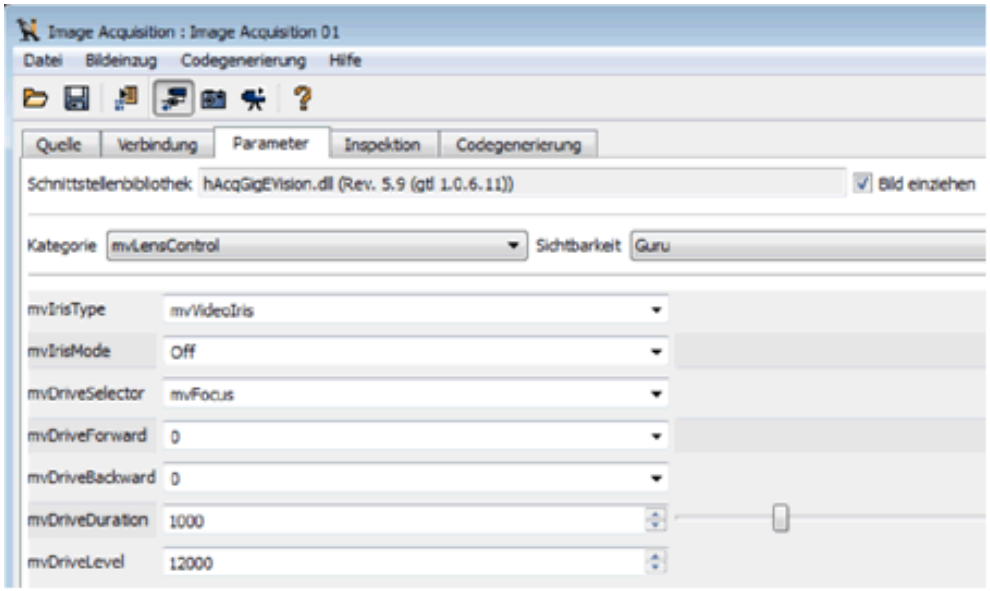


Figure 6: HALCON's image acquisition assistant

The next screenshot shows the respective HDevelop example under Halcon for the same settings:

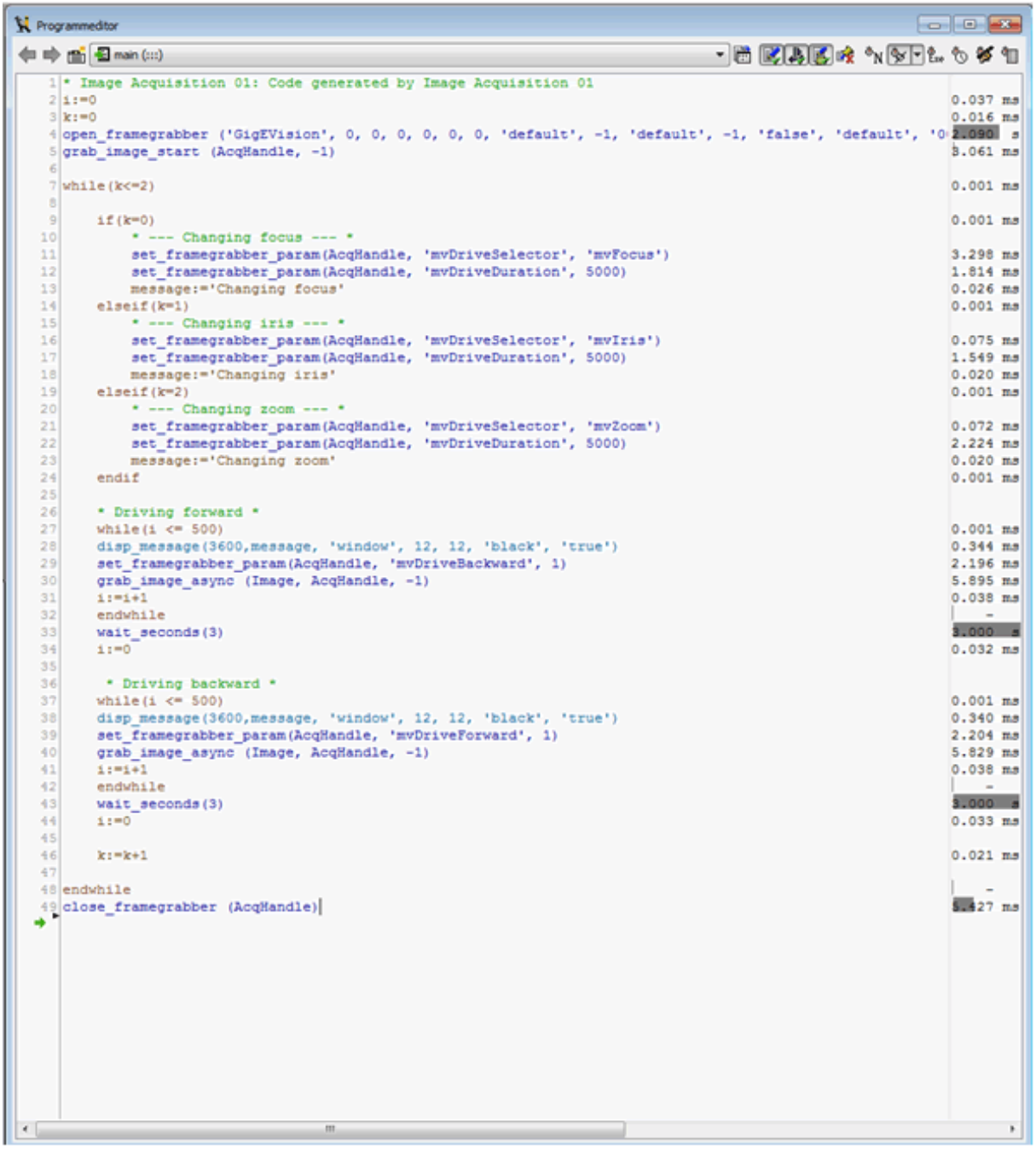


Figure 7: HDevelop example

1.19.6 Working with HDR (High Dynamic Range Control)

There are several use cases concerning High Dynamic Range Control:

- **Adjusting sensor of camera models -x00w** (p. 378)
- **Adjusting sensor of camera models -x02d (-1012d)** (p. 381)
- **Adjusting sensor of camera models -x02e (-1013)** / **-x04e (-1020)** (p. 383)

1.19.6.1 Adjusting sensor of camera models -x00w

1.19.6.1.1 Introduction

The HDR (High Dynamic Range) mode of the sensor -x00w increases the usable contrast range. This is achieved by dividing the integration time in two or three phases. The exposure time proportion of the three phases can be set independently. Furthermore, it can be set, how much signal of each phase is charged.

1.19.6.1.2 Functionality

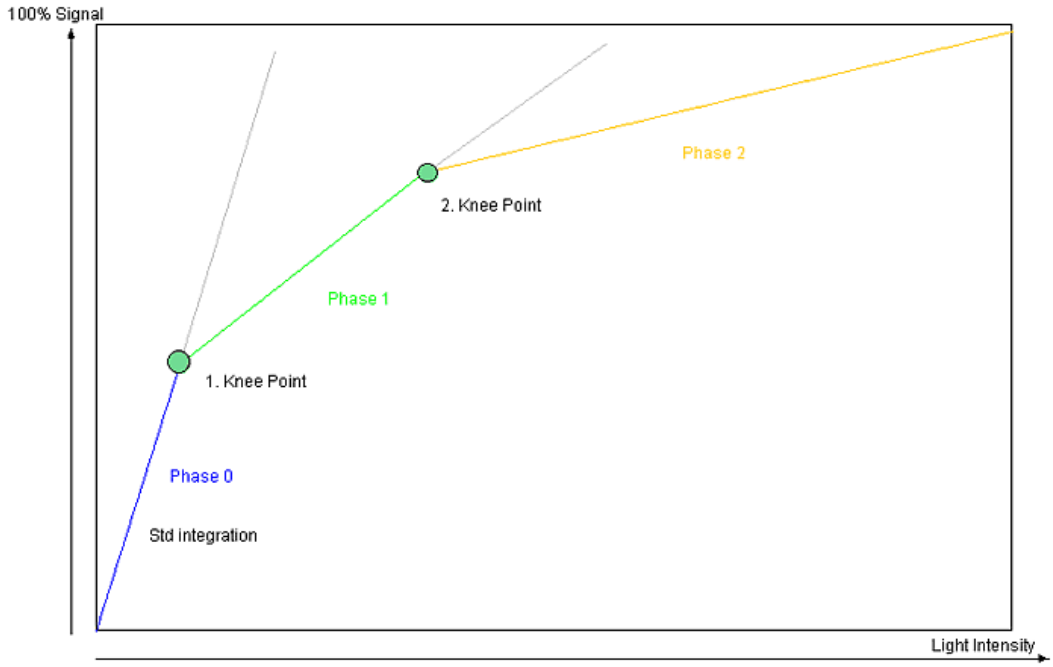


Figure 1: Diagram of the -x00w sensor's HDR mode

1.19.6.1.2.1 Description

- **"Phase 0"**
  - During T1 all pixels are integrated until they reach the defined signal level of Knee Point 1.
  - If one pixel reaches the level, the integration will be stopped.
  - During T1 no pixel can reached a level higher than P1.

- **"Phase 1"**
  - During T2 all pixels are integrated until they reach the defined signal level of Knee Point 2.
  - T2 is always smaller than T1 so that the percentage compared to the total exposure time is lower.
  - Thus, the signal increase during T2 is lower as during T1.
  - The max. signal level of Knee Point 2 is higher than of Knee Point 1.
- **"Phase 2"**
  - During T2 all pixels are integrated until the possible saturation.
  - T3 is always smaller than T2, so that the percentage compared to the total exposure time is again lower here.
  - Thus, the signal increase during T3 is lower as during T2.

For this reason, darker pixels can be integrated during the complete integration time and the sensor reaches its full sensitivity. Pixels, which are limited at each Knee Points, lose a part of their integration time - even more, if they are brighter.

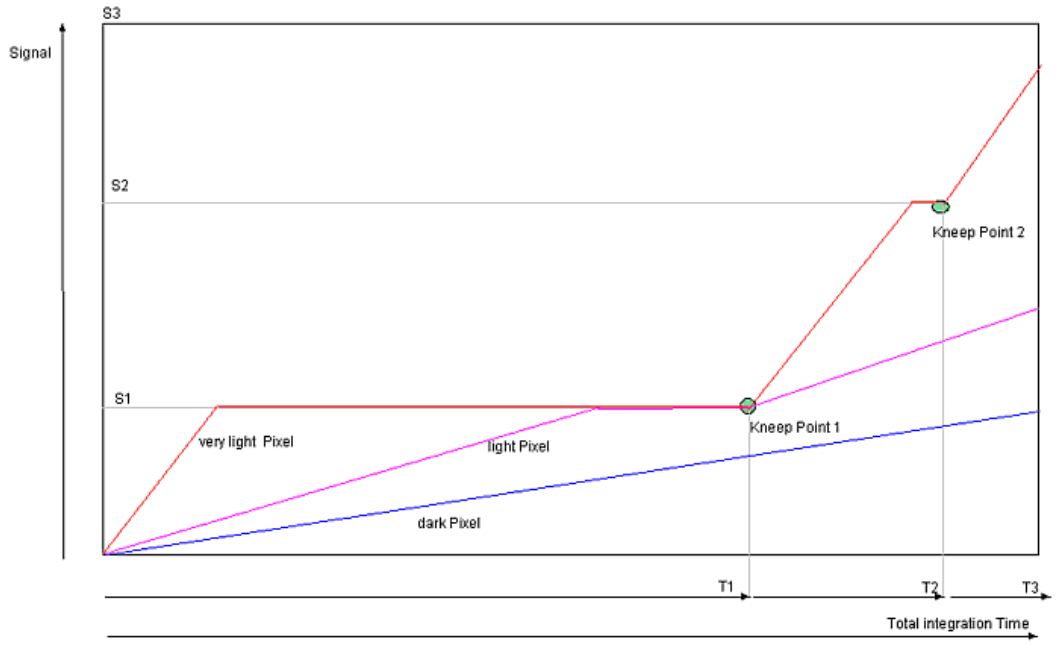


Figure 2: Integration time of different bright pixels

In the diagram you can see the signal line of three different bright pixels. The slope depends of the light intensity , thus it is per pixel the same here (granted that the light intensity is temporally constant). Given that the very light pixel is limited soon at the signal levels S1 and S2, the whole integration time is lower compared to the dark pixel. In practice, the parts of the integration time are very different. T1, for example, is 95% of T_{total}, T2 only 4% and T3 only 1%. Thus, a high decrease of the very light pixels can be achieved. However, if you want to divide the integration thresholds into three parts that is S2 = 2 x S1 and S3 = 3 x S1, a hundredfold brightness of one pixel's step from S2 to S3, compared to the step from 0 and S1 is needed.

1.19.6.1.3 Using HDR with mvBlueCOUGAR-X-x00w

Figure 3 is showing the usage of the HDR mode. You can reach the HDR settings via *"Setting -> Base -> Camera -> GenICam -> mv High Dynanmic Range Control"*.

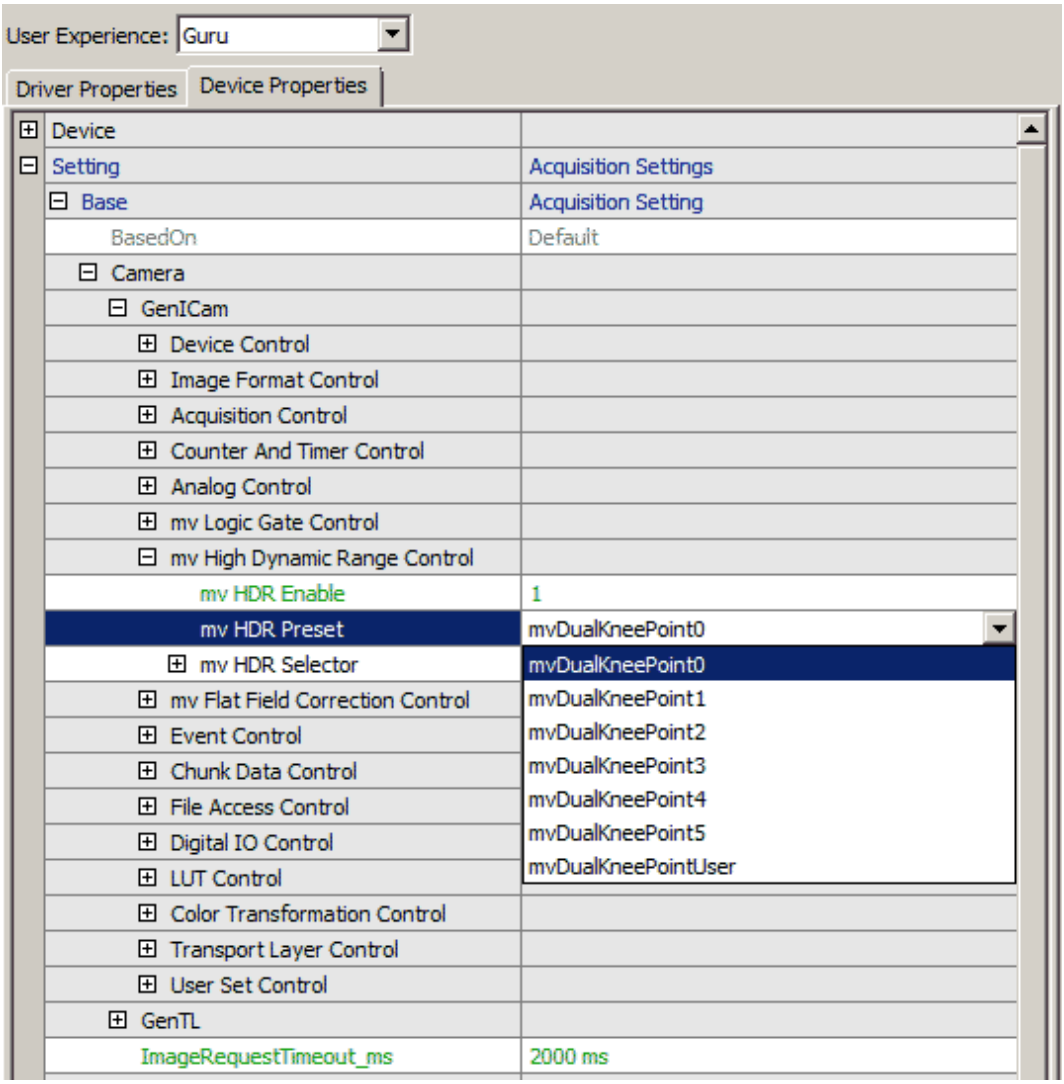


Figure 3: wxPropView HDR screenshot

1.19.6.1.3.1 Notes about the usage of the HDR mode with mvBlueCOUGAR-X-x00w

- In the HDR mode, the basic amplification is reduced by approx. 0.7, to utilize a huge, dynamic area of the sensor.
- If the manual gain is raised, this effect will be reversed.
- Exposure times, which are too low, make no sense. During the third phase, if the exposure time reaches a possible minimum (one line period), a sensible lower limit is reached.

1.19.6.1.3.2 Possible settings using mvBlueCOUGAR-X-x00w

Possible settings of the mvBlueCOUGAR-X-x00w in HDR mode are:

"mv HDR Enable":

- **"Off"**: Standard mode
- **"On"**: HDR mode on, reduced amplification:
- **"mv HDR Preset"**:
  - **"mvDualKneePoint0"**: Fixed setting with 2 Knee Points. modulation Phase 0 .. 33% / 1 .. 66% / 2 .. 100%
  - **"mvDualKneePoint1"**: Phase 1 exposure 12.5% , Phase 2 31.25% of total exposure
  - **"mvDualKneePoint2"**: Phase 1 exposure 6.25% , Phase 2 1.56% of total exposure
  - **"mvDualKneePoint3"**: Phase 1 exposure 3.12% , Phase 2 0.78% of total exposure
  - **"mvDualKneePoint4"**: Phase 1 exposure 1.56% , Phase 2 0.39% of total exposure
  - **"mvDualKneePoint5"**: Phase 1 exposure 0.78% , Phase 2 0.195% of total exposure
  - **"mvDualKneePoint6"**: Phase 1 exposure 0.39% , Phase 2 0.049% of total exposure
  - **"mvDualKneePointUser"**: Variable setting of the Knee Point (1..2), threshold and exposure time proportion
- **"mv HDR Selector"**:
  - **"mv HDR Voltage first knee point"**: Control voltage for exposure threshold of first Knee Point (3030mV is equivalent to approx. 33%)
  - **"mv HDR Voltage second knee point"**: Control voltage for exposure threshold of first Knee Point (2630mV is equivalent to approx. 66%)
  - **"mv HDR Exposure first knee point"**: Proportion of Phase 0 compared to total exposure in parts per million (ppm)
  - **"mv HDR Exposure second knee point"**: Proportion of Phase 1 compared to total exposure in parts per million (ppm)

1.19.6.2 Adjusting sensor of camera models -x02d (-1012d)

1.19.6.2.1 Introduction

The HDR (High Dynamic Range) mode of the Aptina sensor increases the usable contrast range. This is achieved by dividing the integration time in three phases. The exposure time proportion of the three phases can be set independently.

1.19.6.2.2 Functionality

To exceed the typical dynamic range, images are captured at 3 exposure times with given ratios for different exposure times. The figure shows a multiple exposure capture using 3 different exposure times.

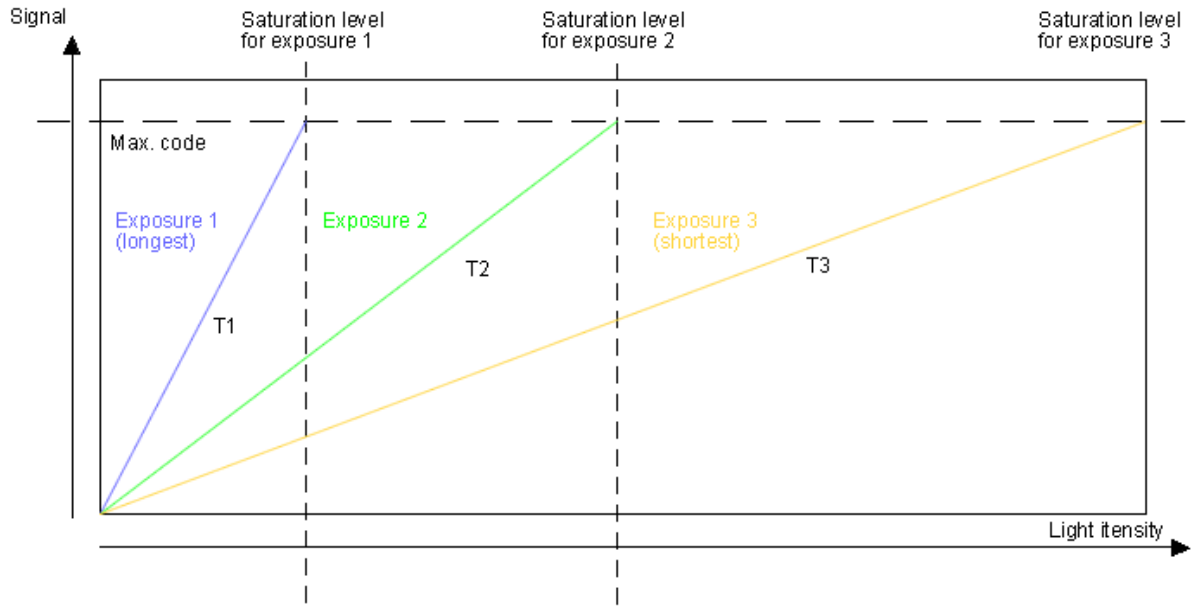


Figure 1: Multiple exposure capture using 3 different exposure times

Note

The longest exposure time (T1) represents the *Exposure_us* parameter you can set in wxPropView.

Afterwards, the signal is fully linearized before going through a compander to be output as a piece-wise linear signal. the next figure shows this.

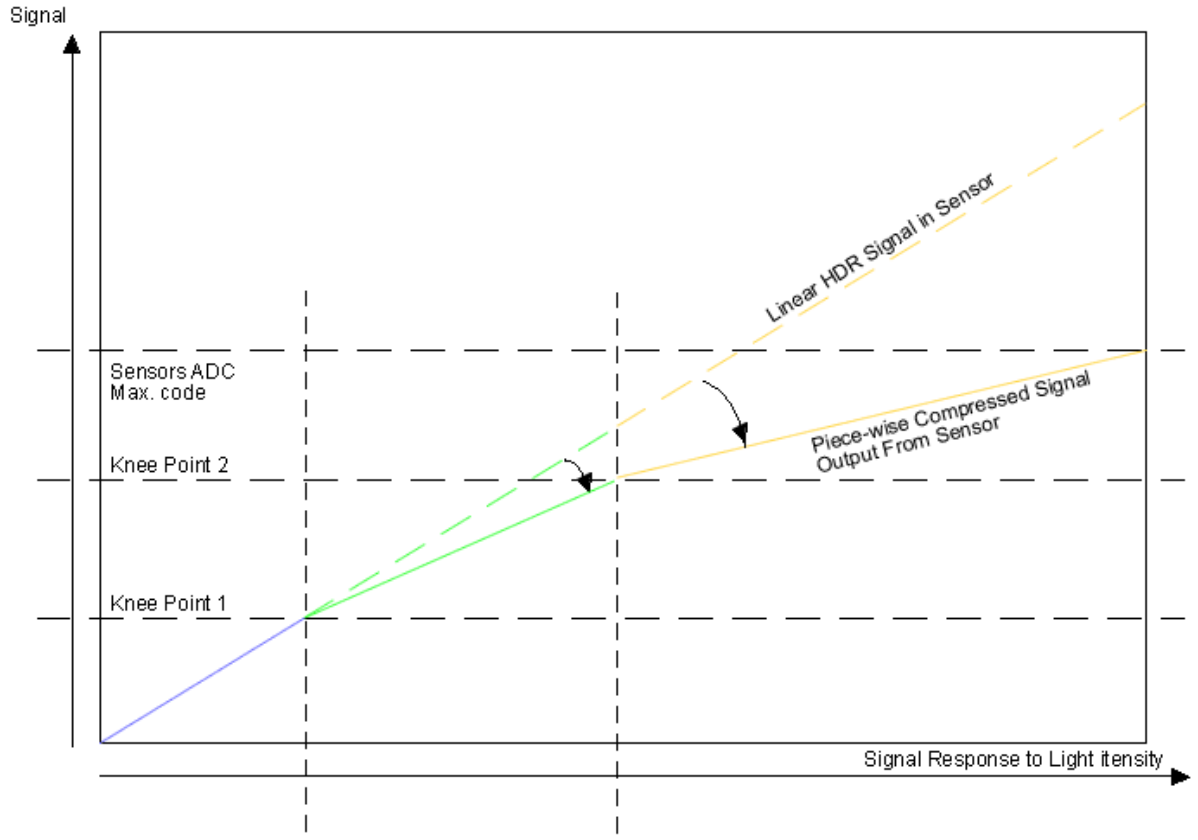


Figure 2: Piece-wise linear signal

1.19.6.2.2.1 Description

Exposure ratios can be controlled by the program. Two rations are used: R1 = T1/T2 and R2 = T2/T3.

Increasing R1 and R2 will increase the dynamic range of the sensor at the cost of lower signal-to-noise ratio (and vice versa).

1.19.6.2.2.2 Possible settings

Possible settings of the mvBlueCOUGAR-X-x02d in HDR mode are:

- **"mv HDR Enable"**:
  - **"Off"**: Standard mode
  - **"On"**: HDR mode on, reduced amplification
    - **"mv HDR Preset"**:
      - **"mvDualKneePoint0"**: Fixed setting with exposure-time-ratios: T1 -> T2 ratio / T2 -> T3 ratio
      - **"mvDualKneePoint1"**: 8 / 4
      - **"mvDualKneePoint2"**: 4 / 8
      - **"mvDualKneePoint3"**: 8 / 8
      - **"mvDualKneePoint4"**: 8 / 16
      - **"mvDualKneePoint5"**: 16 / 16
      - **"mvDualKneePoint6"**: 16 / 32

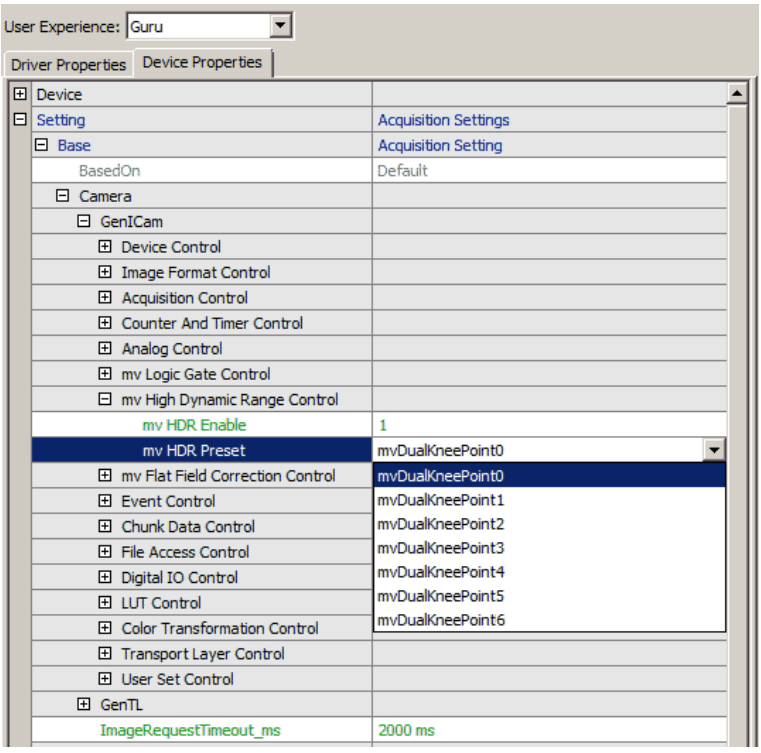


Figure 3: wxPropView - Working with the HDR mode

1.19.6.3 Adjusting sensor of camera models -x02e (-1013) / -x04e (-1020)

1.19.6.3.1 Introduction

The HDR (High Dynamic Range) mode of the e2v sensors increases the usable contrast range. This is achieved by adjusting the logarithmic response of the pixels.

1.19.6.3.2 Functionality

MATRIX VISION offers the *"mv Linear Logarithmic Mode"* to use the HDR mode of the e2v sensors. With this mode you can set the low voltage of the reset signal at pixel level.

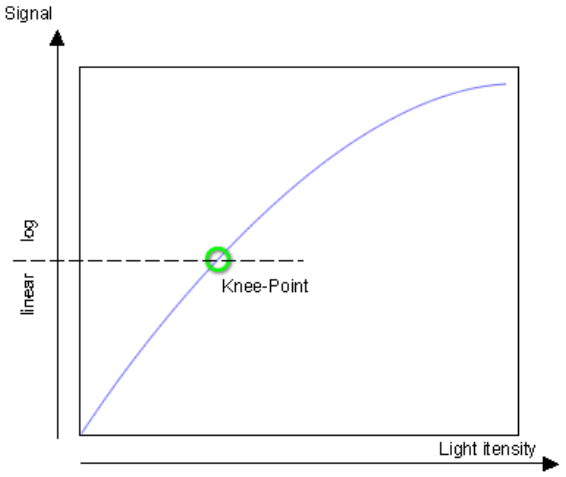


Figure 1: Knee-Point of the e2v HDR mode shifts the linear / logarithmic level

You can find the *"mv Linear Logarithmic Mode"* in "`Setting -> Base -> Camera -> GenICam -> Analog Control`":

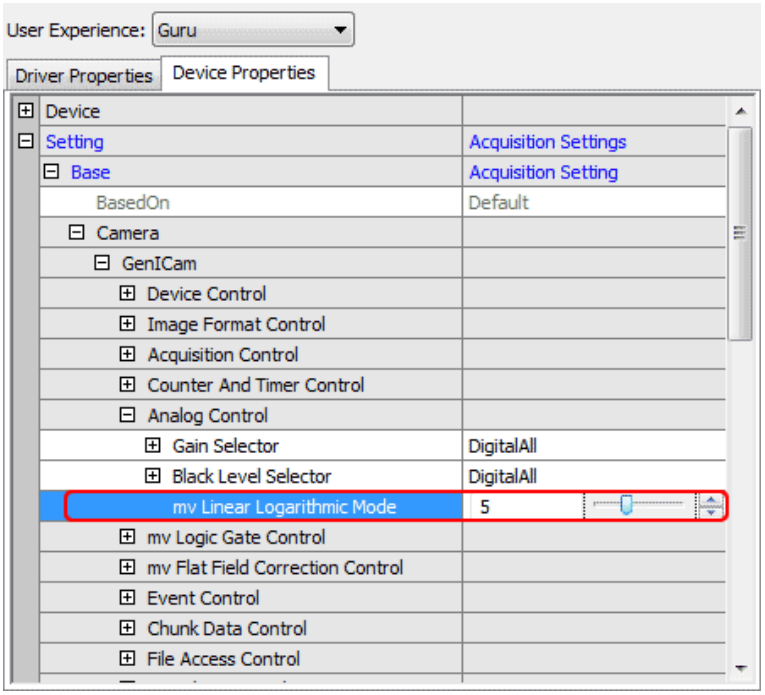


Figure 2: wxPropView - mv Linear Logarithmic Mode

The following figure shows the measured curves at 2 ms and 20 ms exposure time with four different *"mv Linear Logarithmic Mode"* settings:

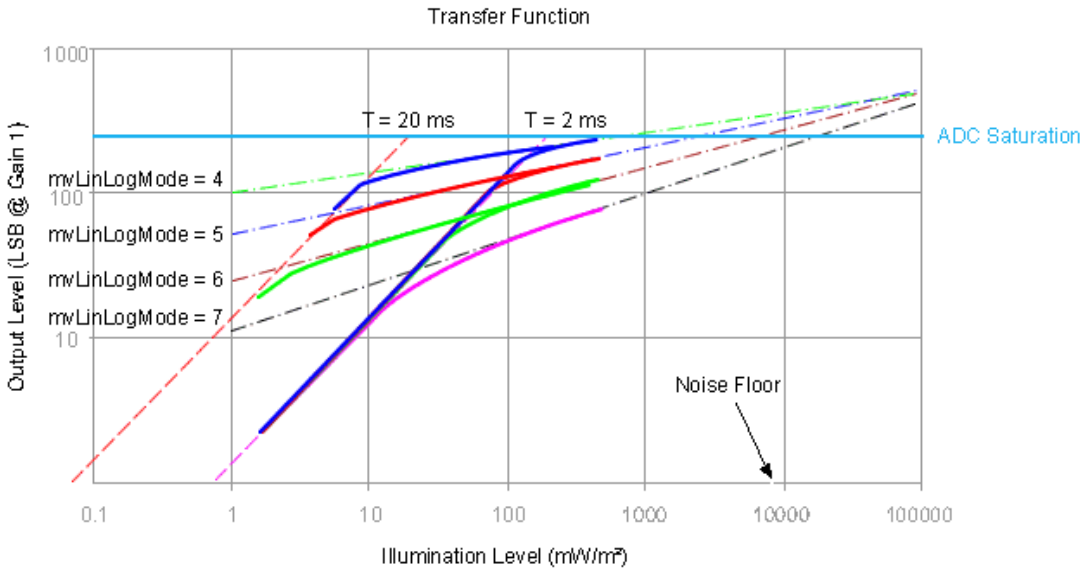


Figure 3: Measured curves

The curves are at Gain 1, lambda = 670 nm (40 nm width), room temperature, nominal power supply values, on a 100 x 100 pixel cantered area.

"mv Linear Logarithmic Mode" value	Dynamic max (dB)	
	T = 2 ms	T = 20 ms
4	47	65
5	74	93
6	85	104
7	92	111

1.19.7 Working with LUTs

There are several use cases concerning LUTs (Look-Up-Tables):

- **Introducing LUTs** (p. 386)
- **Working with LUTValueAll** (p. 390)
- **Implementing a hardware-based binarization** (p. 392)

1.19.7.1 Introducing LUTs

1.19.7.1.1 Introduction

Look-Up-Tables (LUT) are used to transform input data into a desirable output format. For example, if you want to invert an 8 bit image, a Look-Up-Table will look like the following:

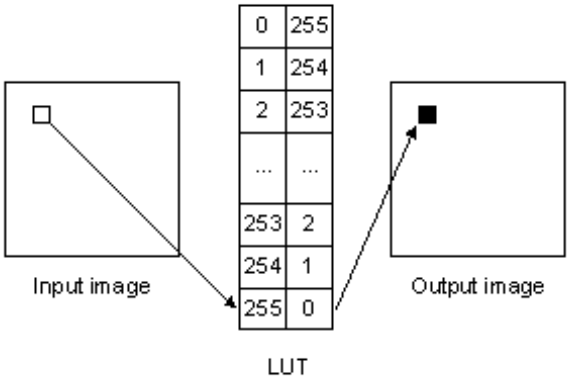


Figure 1: Look-Up-Table which inverts a pixel of an 8 bit mono image

I.e., a pixel which is white in the input image (value 255) will become black (value 0) in the output image.

All MATRIX VISION devices use a hardware based LUT which means that

- no host CPU load is needed and
- the LUT operations are independent of the transmission bit depth.

1.19.7.1.2 Setting the hardware based LUTs via LUT Control

On the mvBlueCOUGAR-X using **wxPropView** (p. 139), you will find the **LUT Control** (p. 170) via *"Setting -> Base -> Camera -> GenICam -> LUT Control"*.

wxPropView (p. 139) offers a wizard for the **LUT Control** (p. 170) usage:

1. Click on *"Setting -> Base -> Camera -> GenICam -> LUT Control"*.
   Now, the *"Wizard"* button becomes active.

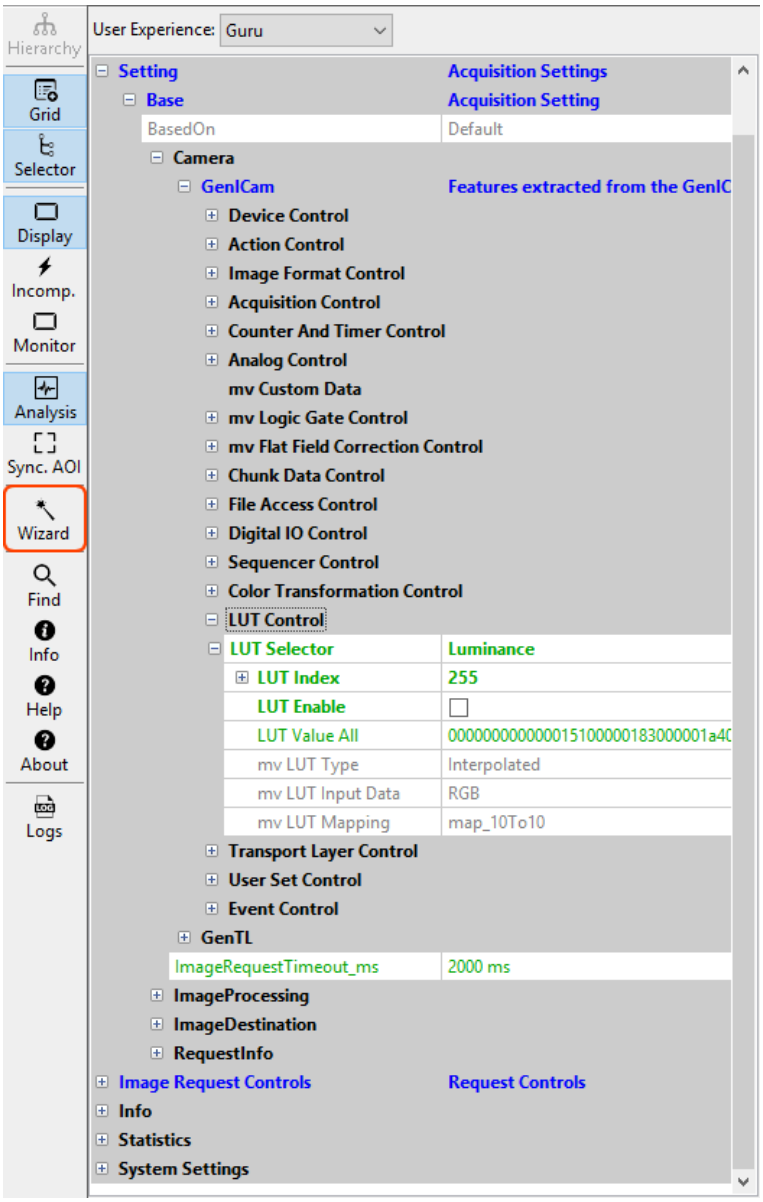


Figure 2: wxPropView - LUT Control wizard button

2. Click on the *"Wizard"* button to start the LUT Control wizard tool.
   The wizard will load the LUT data from the camera.

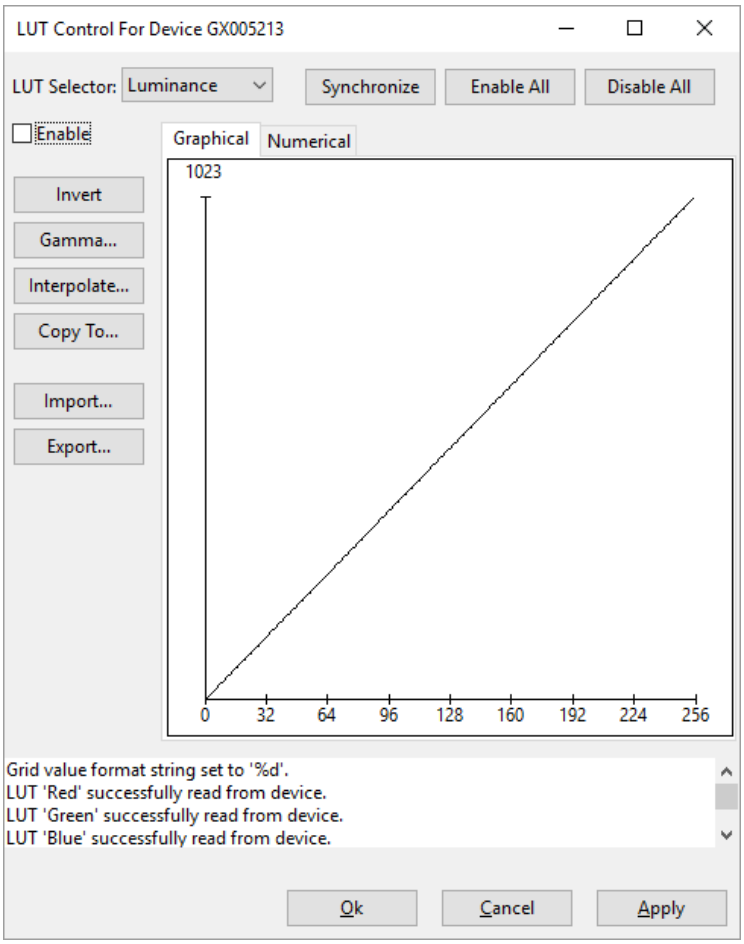


Figure 3: wxPropView - LUT Control wizard dialog

It is easy to change settings like the Gamma value of the Luminance or of each color channel (in combination with a color sensor of course) with the help of the wizard. You can also invert the values of each pixel with the wizard. It is not possible to set a LUT mode and the **"mv LUT Mapping"** is fixed.

Make your changes and do not forget to

1. click on **"Copy to..."** and select *"All"* or the color channel you need, to
2. click on **"Enable All"**, and finally, to
3. click on **Synchronize** and play the settings back to the device (via *"Cache -> Device"*).

Note

If you select *"Enable All"* without entering any value the image will be inverted.

1.19.7.1.3 Setting the Host based LUTs via LUTOperations

Host based LUTs are also available via *"Setting -> Base -> ImageProcessing -> LUTOperations"*. Here, the changes will affect the 8 bit image data and the processing needs the CPU of the host system.

Three *"LUTMode"s* are available:

- **"Gamma"**
  You can use "Gamma" to lift darker image areas and to flatten the brighter ones. This compensates the contrast of the object. The calculation is described here. It makes sense to set the *"GammaStartThreshold"* higher than 0 to avoid a too extreme lift or noise in the darker areas.

- **"Interpolated"**
  With "Interpolated" you can set the key points of a characteristic line. You can defined the number of key points. The following figure shows the behavior of all 3 LUTInterpolationModes with 3 key points:

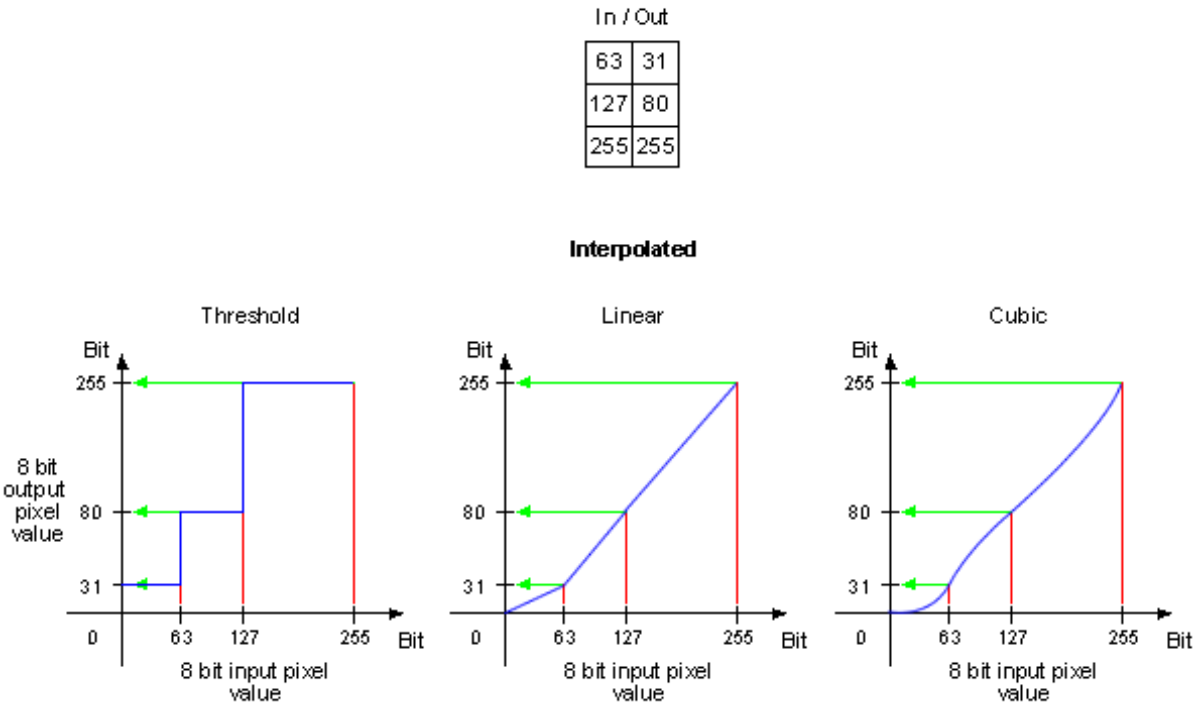


Figure 4: LUTMode "Interpolated" -> LUTInterpolationMode

- **"Direct"**
  With "Direct" you can set the LUT values directly.

1.19.7.1.3.1 Example 1: Inverting an Image

To get an inverted 8 bit mono image like shown in Figure 1, you can set the LUT using **wxPropView** (p. 139). After starting **wxPropView** (p. 139) and using the device,

1. Set *"LUTEnable"* to **"On"** in *"Setting -> Base -> ImageProcessing -> LUTOperations"*.
2. Afterwards, set *"LUTMode"* to **"Direct"**.
3. Right-click on *"LUTs -> LUT-0 -> DirectValues[256]"* and select **"Set Multiple Elements... -> Via A User Defined Value Range"**.
   This is one way to get an inverted result. It is also possible to use the *"LUTMode"* - *"Interpolated"*.
4. Now you can set the range from 0 to 255 and the values from 255 to 0 as shown in Figure 2.

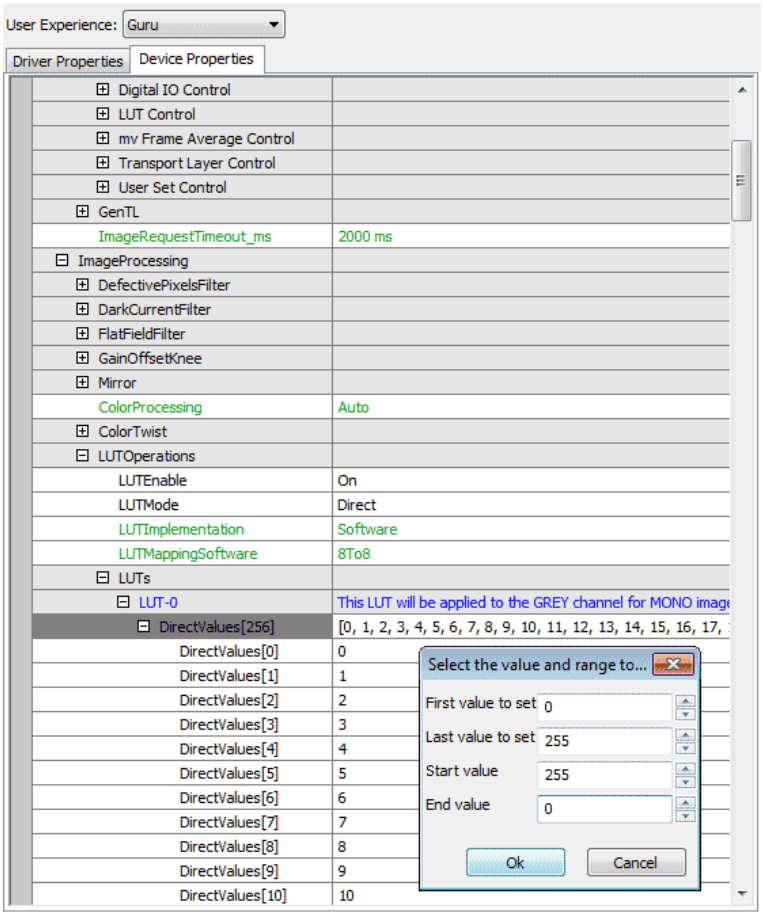


Figure 5: Inverting an image using wxPropView with LUTMode "Direct"

1.19.7.2 Working with LUTValueAll

Working with the **LUTValueAll** feature requires a detailed understanding on both Endianess and the cameras internal format for storing LUT data. **LUTValueAll** typically references the same section in the cameras memory as when accessing the LUT via the features **LUTIndex** and **LUTValue**.

LUT data can either be written to a device like this (C++ syntax):

```
const size_t LUT_VALUE_COUNT = 256;
int64_type LUTData[LUT_VALUE_COUNT] = getLUTDataToWriteToTheDevice();
mvIMPACT::acquire::GenICam::LUTControl lut(getDevicePointerFromSomewhere());
for(int64_type i=0; i< static_cast<int64_type>(LUT_VALUE_COUNT); i++ )
{
  lut.LUTIndex.write( i );
  lut.LUTValue.write( LUTData[i] );
}
```

When using this approach all the Endianess related issues will be handled completely by the **GenICam** (p. 213) runtime library. So this code is straight forward and easy to understand but might be slower than desired as it requires a lot of direct register accesses to the device.

In order to allow a fast efficient way to read/write LUT data from/to a device the **LUTValueAll** feature has been introduced. When using this feature the complete LUT can be written to a device like this:

```
const size_t LUT_VALUE_COUNT = 256;
int LUTData[LUT_VALUE_COUNT] = getLUTDataToWriteToTheDevice();
mvIMPACT::acquire::GenICam::LUTControl lut(getDevicePointerFromSomewhere());
std::string buf(reinterpret_cast<std::string::value_type*>(&LUTData), sizeof(LUTData));
lut.LUTValueAll.writeBinary( buf );
```

BUT as this simply writes a raw block of memory to the device it suddenly becomes important to know exactly how the LUT data is stored inside the camera. This includes:

- The size of one individual LUT entry (this could be anything from 1 up to 8 bytes)
- The Endianess of the device
- The Endianess of the host system used for sending/receiving the LUT data

The first item has impact on how the memory must be allocated for receiving/sending LUT data. For example when the LUT data on the device uses a 'one 32-bit integer per LUT entry with 256 entries' layout then of course this is needed on the host system:

```
const size_t LUT_VALUE_COUNT = 256;
int LUTData[LUT_VALUE_COUNT];
```

When the Endianess of the host system differs from the Endianess used by the device the application communicates with, then before sending data assembled on the host might require Endianess swapping. For the example from above this would e.g. require something like this:

```
#define SWAP_32(l) \
  ((((l) & 0xff000000) >> 24) | \
   (((l) & 0x00ff0000) >> 8)  | \
   (((l) & 0x0000ff00) << 8)  | \
   (((l) & 0x000000ff) << 24))

void fn()
{
 const size_t LUT_VALUE_COUNT = 256;
 int LUTData[LUT_VALUE_COUNT] = getLUTDataToWriteToTheDevice();
 mvIMPACT::acquire::GenICam::LUTControl lut(getDevicePointerFromSomewhere());
 for( size_t i=0; i<LUT_VALUE_COUNT; i++ )
 {
  LUTData[i] = SWAP_32(LUTData[i]);
 }
 std::string buf(reinterpret_cast<std::string::value_type*>(&LUTData), sizeof(LUTData));
 lut.LUTValueAll.writeBinary( buf );
}
```

For details on how the LUT memory is organized for certain sensors please refers to the **Sensor Overview** (p. 116). Please note that all **mvBlueCOUGAR-S**, **mvBlueCOUGAR-X** and **mvBlueCOUGAR-XD** devices are using **Big Endian** while almost any Windows or Linux distribution on the market uses **Little Endian**, thus the swapping of the data will most certainly be necessary when using the **LUTValueAll** feature.

1.19.7.3 Implementing a hardware-based binarization

If you like to have binarized images from the camera, you can use the **hardware-based Look-Up-Tables (LUT)** (p. 170) which you can access via *"Setting -> Base -> Camera -> GenICam -> LUT Control"*.

To get binarized images from the camera, please follow these steps:

1. Set up the camera and the scenery, e.g.



Figure 1: Scenery

2. Open the LUT wizard via the menu *"Wizards -> LUT Control..."*.
3. Export the current LUT as a "*.csv" file.

The "*.csv" file contains just one column for the output gray scale values. Each row of the "*.csv" represents the input gray scale value. In our example, the binarization threshold is 1024 in a 12-to-9 bit LUT. I.e., we have 4096 (= 12 bit) input values (= rows) and 512 (= 9 bit) output values (column values). To binarize the image according to the threshold, you have to

1. set all values below the binarization threshold to 0.
2. Set all values above the binarization threshold to 511:

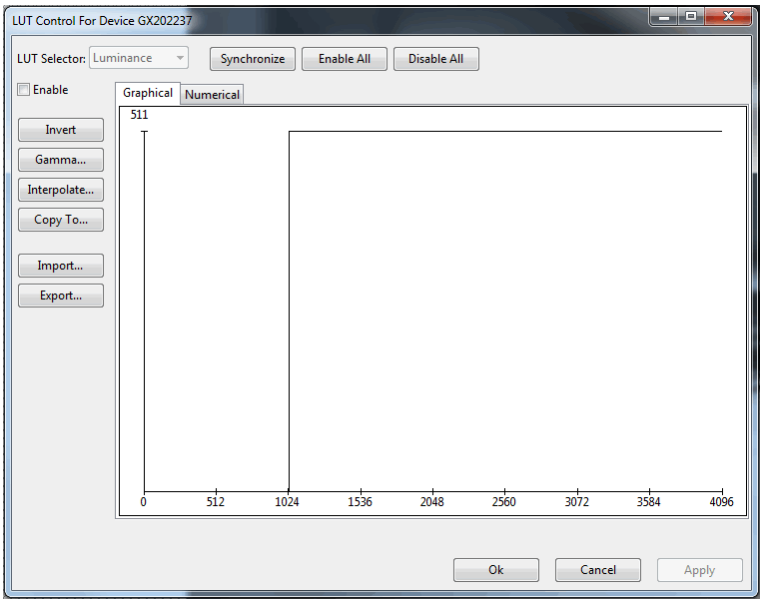


Figure 2: The binarization LUT

3. Now, save the "`*.csv`" file and
4. import it via the LUT Control wizard.
5. Click on synchronize and
6. finally check **"Enable"**.

Afterwards the camera will output binarized images like the following:



Figure 3: Binarized image

1.19.8 Saving data on the device

Note

As described in **"Storing, Restoring And Managing Settings"** in the **"mvIMPACT Acquire SDK GUI Applications"** manual, it is also possible to save the settings as an XML file on the host system. You can find further information about for example the XML compatibilities of the different driver versions in the mvIMPACT Acquire SDK manuals and the according setting classes: `https://www.matrix-vision.com/manuals/SDK_CPP/classmvIMPACT_1_1acquire_1_1FunctionInterface.html` (C++)

- **Creating user data entries** (p. 394)
- **Creating user set entries** (p. 396)
- **Working with the UserFile section (Flash memory)** (p. 400)

1.19.8.1 Creating user data entries

1.19.8.1.1 Basics about user data

It is possible to save arbitrary user specific data on the hardware's non-volatile memory. The amount of possible entries depends on the length of the individual entries as well as the size of the devices non-volatile memory reserved for storing:

- mvBlueFOX,
- mvBlueFOX-M,
- mvBlueFOX-MLC,
- mvBlueFOX3,
- mvBlueCOUGAR-X,
- mvBlueCOUGAR-XD,
- mvBlueCOUGAR-XT and
- mvBlueNAOS

currently offer 512 bytes of user accessible non-volatile memory of which 12 bytes are needed to store header information leaving 500 bytes for user specific data.

```
One entry will currently consume:
1 + <length_of_name (up to 255 chars)> + 2 + <length_of_data (up to 65535 bytes)> + 1 (access mode) bytes

as well as an optional:
1 + <length_of_password> bytes per entry if a password has been defined for this particular entry
```

It is possible to save either **String** or **Binary** data in the *data* property of each entry. When storing binary data please note, that this data internally will be stored in `Base64` format thus the amount of memory required is 4/3 time the binary data size.

The UserData can be accessed and created using **wxPropView** (p. 139) (the device has to be closed). In the section `"UserData"` you will find the entries and following methods:

- `"CreateUserDataEntry"`

- "DeleteUserDataEntry"
- "WriteDataToHardware"

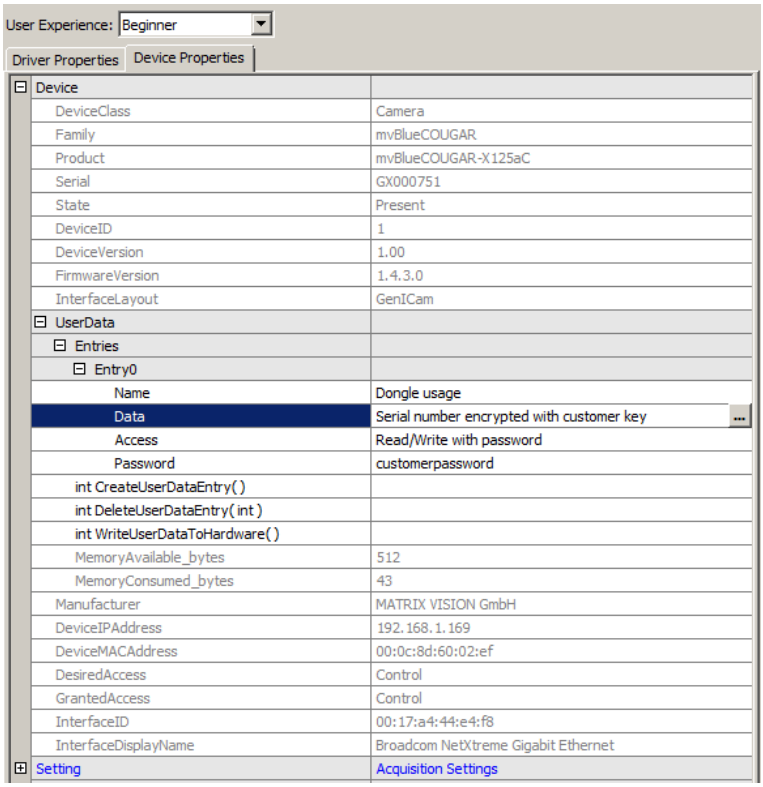


Figure 1: wxPropView - section "UserData -> Entries"

To create a user data entry, you have to

- Right click on "CreateUserDataEntry"
- Select **"Execute"** from the popup menu.
  An entry will be created.
- In "Entries" click on the entry you want to adjust and modify the data fields.
  To permanently commit a modification made with the keyboard the **ENTER** key **must be pressed**.
- To save the data on the device, you have to execute "WriteDataToHardware". Please have a look at the "Output" tab in the lower right section of the screen as shown in Figure 2, to see if the write process returned with no errors. If an error occurs a message box will pop up.

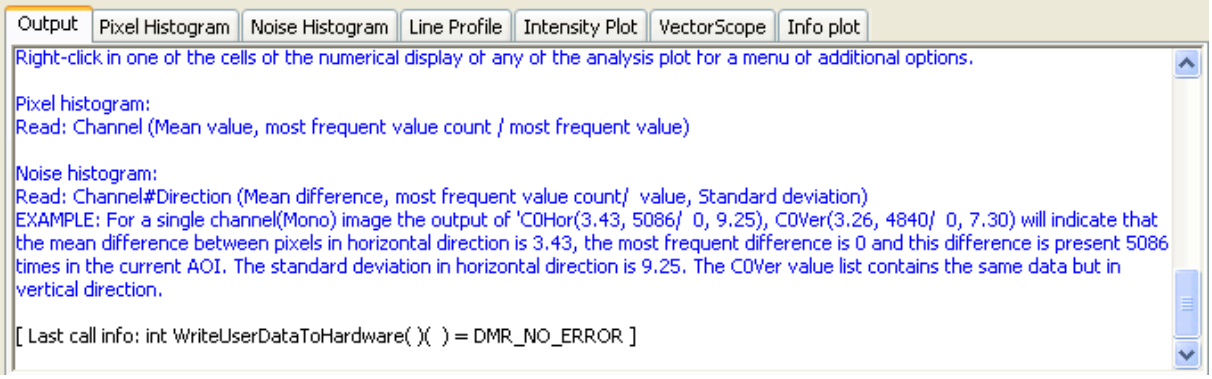


Figure 2: wxPropView - analysis tool "Output"

1.19.8.1.2 Coding sample

If you e.g. want to use the UserData as dongle mechanism (with binary data), it is not suggested to use **wxPropView** (p. 139). In this case you have to program the handling of the user data.

See also

mvIMPACT::acquire::UserDataEntry in mvIMPACT_Acquire_API_CPP_manual.chm.

1.19.8.2 Creating user set entries

With MATRIX VISION GenICam compliant devices it is possible to store up to five configuration sets (4 user plus one factory default) in the camera.

This feature is similar to the **storing settings functionality** (p. 394), which saves the settings in the registry. However, as mentioned before the user sets are stored in the camera.

The user set stores

- exposure,
- gain,
- AOI,
- frame rate,
- **LUT** (p. 170),
  - one **Flat-Field Correction** (p. 289),
- **defective pixels** (p. 285),
  - etc.

permanently and is independent of the computer which in used.

Additionally, you can select, which user set comes up after hard reset.

Note

The storage of user data in the registry can still override user set data!
User sets are cleared after firmware change.

1.19.8.2.1 List of ignored properties

Following properties are not stored in the user set:

```
- ActionUnconditionalMode
- DeviceLinkHeartbeatMode
- DeviceLinkHeartbeatTimeout
- DeviceStreamChannelEndianness
- DeviceTLType
- EventDeviceTemperatureStateCriticalData
- EventDeviceTemperatureStateNormalData
- EventDeviceTemperatureStateOverTemperatureData
- EventExposureEndData
- EventFrameEndData
- EventLine10AnyEdgeData
- EventLine10FallingEdgeData
- EventLine10RisingEdgeData
- EventLine11AnyEdgeData
```

```
- EventLine11FallingEdgeData
- EventLine11RisingEdgeData
- EventLine4AnyEdgeData
- EventLine4FallingEdgeData
- EventLine4RisingEdgeData
- EventLine5AnyEdgeData
- EventLine5FallingEdgeData
- EventLine5RisingEdgeData
- EventLine6AnyEdgeData
- EventLine6FallingEdgeData
- EventLine6RisingEdgeData
- EventLine7AnyEdgeData
- EventLine7FallingEdgeData
- EventLine7RisingEdgeData
- EventLine8AnyEdgeData
- EventLine8FallingEdgeData
- EventLine8RisingEdgeData
- EventLine9AnyEdgeData
- EventLine9FallingEdgeData
- EventLine9RisingEdgeData
- FileAccessBuffer
- FileAccessLength
- FileAccessOffset
- FileOpenMode
- GevCCP
- GevCurrentIPConfigurationDHCP
- GevCurrentIPConfigurationPersistentIP
- GevDiscoveryAckDelay
- GevGVCPExtendedStatusCodes
- GevGVCPHeartbeatDisable
- GevGVCPPendingAck
- GevGVSPExtendedIDMode
- GevHeartbeatTimeout
- GevMCDA
- GevMCPHostPort
- GevMCRC
- GevMCTT
- GevPersistentDefaultGateway
- GevPersistentIPAddress
- GevPersistentSubnetMask
- GevPhysicalLinkConfiguration
- GevPrimaryApplicationSwitchoverKey
- GevSCCFGAllInTransmission
- GevSCCFGExtendedChunkData
- GevSCCFGLargeLeaderTrailer
- GevSCCFGMultiPart
- GevSCCFGPacketResendDestination
- GevSCCFGUnconditionalStreaming
- GevSCDA
- GevSCPD
- GevSCPHostPort
- GevSCPInterfaceIndex
- GevSCPSBigEndian
- GevSCPSDoNotFragment
- GevSCPSFireTestPacket
- GevSCPSPacketSize
- LUTIndex
- LUTValueAll
- PtpControl
- PtpEnable
- UserSetDefault
- mvADCGain
- mvCyclicSignalDelay
- mvCyclicSignalPeriod
- mvDefectPixelSuppressionThreshold
- mvDefectPixelThreshold
- mvDefectivePixelCount
- mvDefectivePixelOffsetX
- mvDefectivePixelOffsetY
- mvDigitalGainOffset
- mvExposureActiveShiftEnable
- mvFFCAutoLoadMode
- mvI2cInterfaceASCIIBuffer
```

- mvI2cInterfaceBinaryBuffer
- mvI2cInterfaceBytesToRead
- mvI2cInterfaceBytesToWrite
- mvPreGain
- mvSerialInterfaceASCIIBuffer
- mvSerialInterfaceBinaryBuffer
- mvSerialInterfaceBytesToRead
- mvSerialInterfaceBytesToWrite
- mvTimestampPPSTriggerEdge
- mvTimestampResetValue
- mvUserData
- mvVRamp

1.19.8.2.2 Working with the user sets

You can find the user set control in *"Setting -> Base -> Camera -> GenICam -> User Set Control"*:

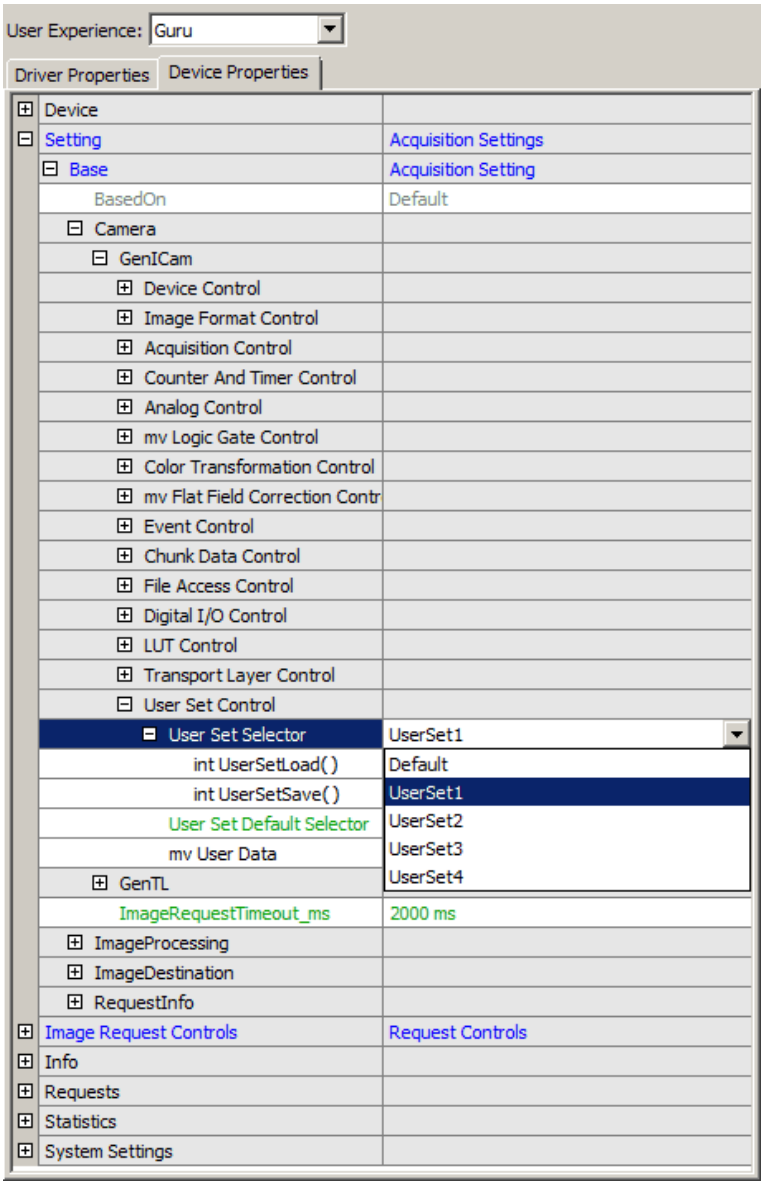


Figure 1: User Set Control

With *"User Set Selector"* you can select the user set (*"Default"*, *"UserSet1 - UserSet4"*). To save or load the specific user set, you have two functions:

- *"int UserSetLoad()"* and
- *"int UserSetSave()"*.

"User Set Default" is the property, where you can select the user set, which comes up after hard reset.

Finally, with *"mv User Data"* you have the possibility to store arbitrary user data.

1.19.8.3 Working with the UserFile section (Flash memory)

The mvBlueCOUGAR-X offers a 64 KByte section in the Flash memory that can be used to upload a custom file to (**UserFile**).

To read or write this file you can use the following GenICam **File Access Control** (p. 155) and its interfaces:

- **IDevFileStream** (read)
- **ODevFileStream** (write)

Note

The **UserFile** is lost each time a firmware update is applied to the device.

1.19.8.3.1 Using wxPropView

wxPropView (p. 139) offers a wizard for the **File Access Control** (p. 155) usage:

1. Click on *"Setting -> Base -> Camera -> GenICam -> File Access Control -> File Selector -> File Operator Selector"*.
   Now, the *"Wizard"* button becomes active.

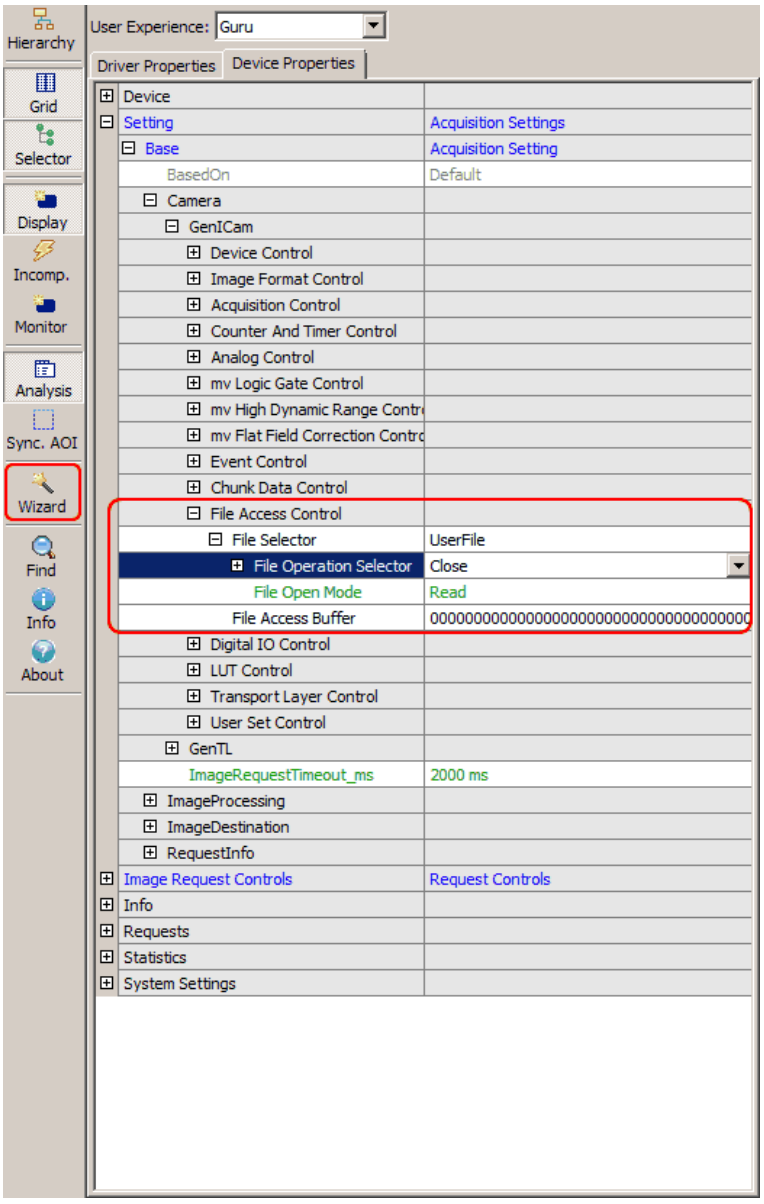


Figure 1: wxPropView - UserFile wizard

2. Click on the *"Wizard"* button.
   Now, a dialog appears where you can choose either to upload or download a file.

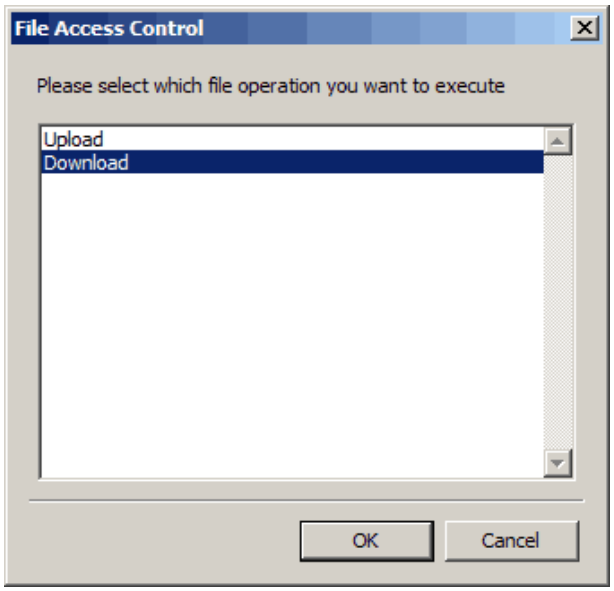


Figure 2: wxPropView - Download / Upload dialog

3. Make your choice and click on *"OK"*.
   Now, a dialog appears where you can select the File.

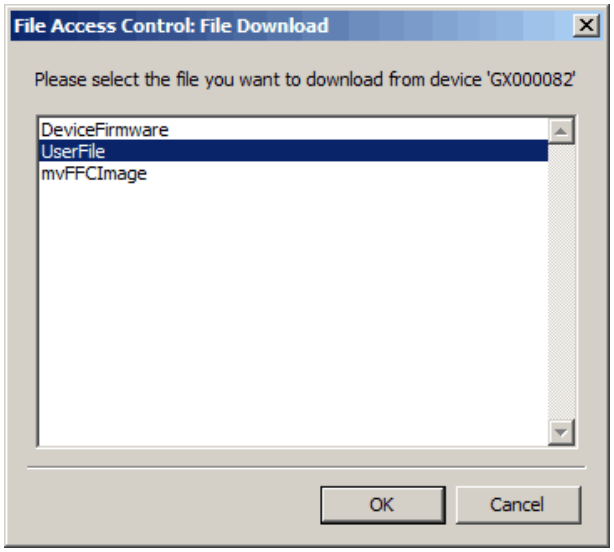


Figure 3: wxPropView - Download / Upload dialog

4. Select *"UserFile"* follow the instructions.

1.19.8.3.2 Manually control the file access from an application (C++)

The header providing the file access related classes must be included into the application:

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam_FileStream.h>
```

A write access then will look like:

```
const string fileNameDevice("UserFile");

// uploading a file
mvIMPACT::acquire::GenICam::ODevFileStream file;
file.open( pDev, fileNameDevice.c_str() );

if( !file.fail() )
{
  // Handle the successful upload.
}
else
{
  // Handle the error.
}
```

A read access will look like:

```
const string fileNameDevice("UserFile");

// downloading a file works in a similar way
mvIMPACT::acquire::GenICam::IDevFileStream file;
file.open( pDev, fileNameDevice.c_str() );

if( !file.fail() )
{
  // Handle the successful upload.
}
else
{
  // Handle the error.
}
```

You can find a detailed code example in the C++ API manual in the documentation of the classes **mvIMPACT::acquire::GenICam::IDevFileStream** and **mvIMPACT::acquire::GenICam::ODevFileStream**

1.19.9 Working with device features

- **Working with the temperature sensors** (p. 404)
- **Disabling the heartbeat** (p. 406)
- **Reset timestamp by hardware** (p. 407)
- **Synchronizing camera timestamps** (p. 408)
- **Working with the 3 head model** (p. 413)
- **Working With The Serial Interface (mv Serial Interface Control)** (p. 414)

1.19.9.1 Working with the temperature sensors

The MATRIX VISION devices offer two different temperature sensors:

- on the sensor board (typically lower)
- on FGPA board (typically higher)

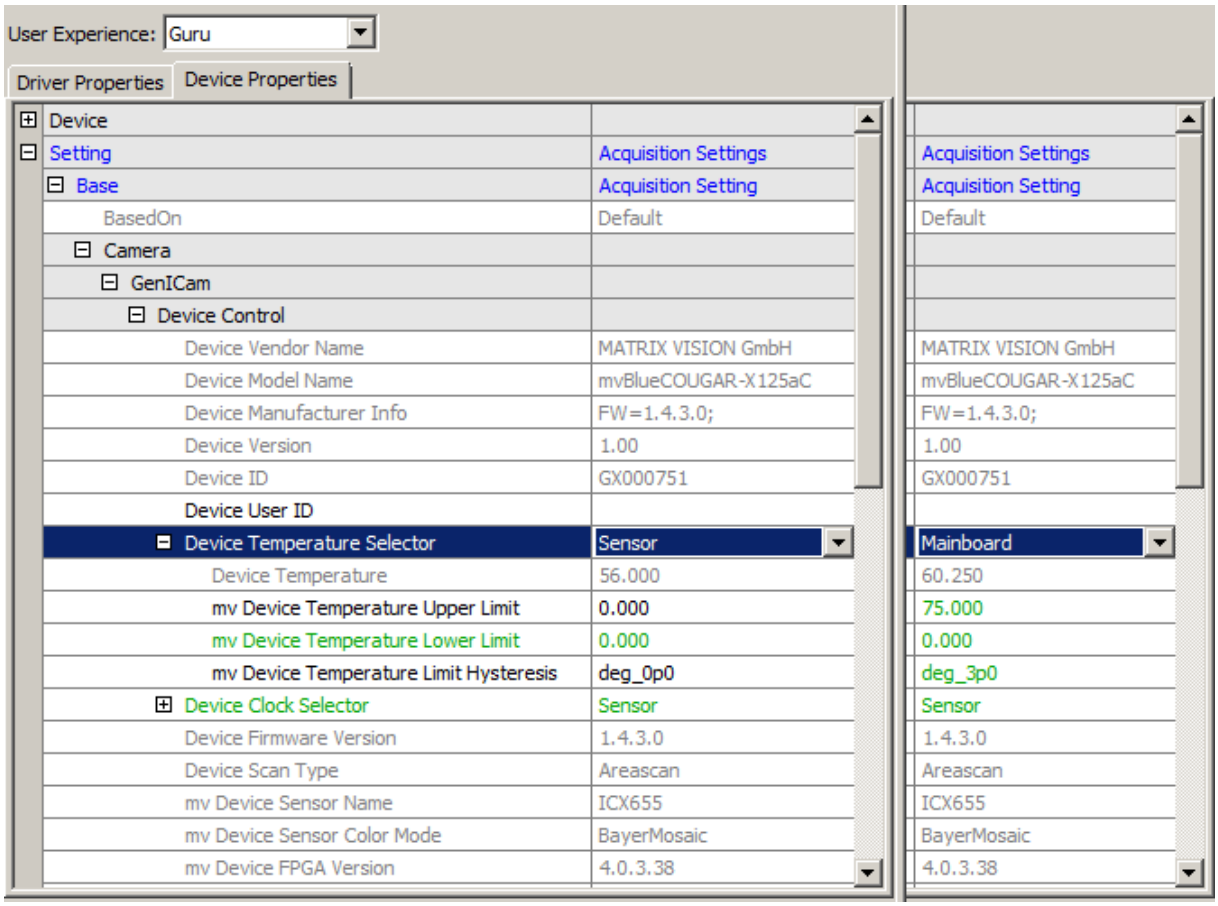


Figure 1: wxPropView - Device Temperature Selector

Note

Avoid temperatures higher than 80 C by lowering thermal resistance of camera housing to connecting structure active cooling means.

It is possible to regulate the temperature of the camera. The limits of this feature are

- upper limit (<= 255 C) and
- lower limit (>= 0 C).

Furthermore, the hysteresis (0,1.5,3,6 C) difference between on/off prevents oscillation. The temperature out of range will set selected output high (see Figure 2). The output can directly drive a fan or a (modest) heating system (but not both due to limit of IC).

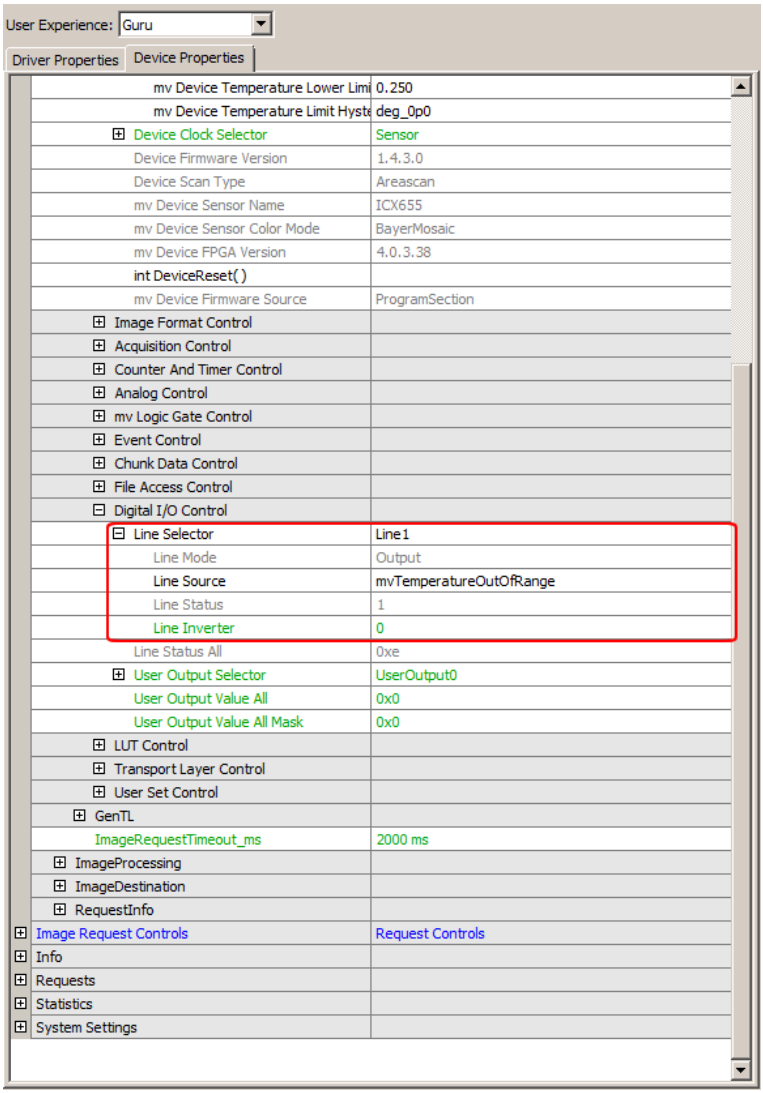


Figure 2: wxPropView - Temperature I/O setting

1.19.9.2 Disabling the heartbeat

The GigE Vision Control Protocol (GVCP) is an application protocol relying on the **UDP** (p. 221) transport layer protocol. Because **UDP** (p. 221) is a connectionless protocol, a recovery mechanism has been defined to detect that one of the two participants has been detached without closing the control channel before. A **heartbeat** message is used to achieve this.

In the default operation mode, these **heartbeat** messages are exchanged with a user selectable frequency (*"Setting -> Base -> Camera -> GenICam -> Device Control -> Device Link Heartbeat Timeout"*; orange box in Figure 1). As a result, during this interval a certain register (GevCCP = Control Channel Privilege) is read several times from the device to reset the heartbeat timer interval in order to ensure the control channel stays open.

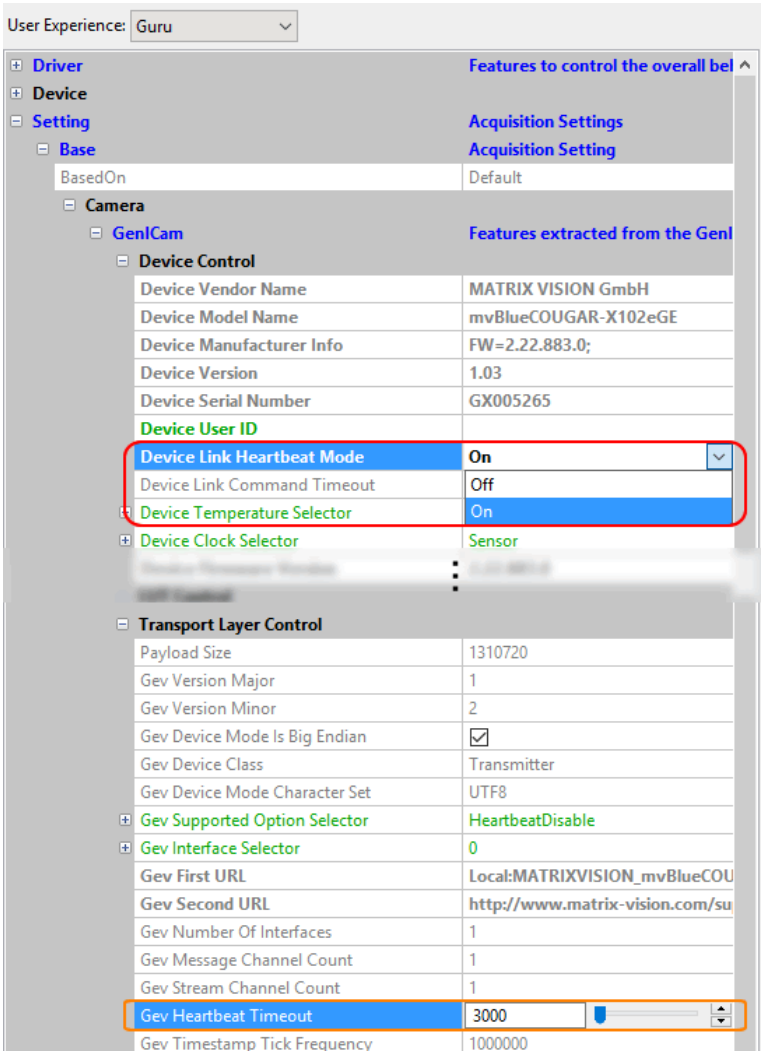


Figure 1: wxPropView - Transport Layer Control

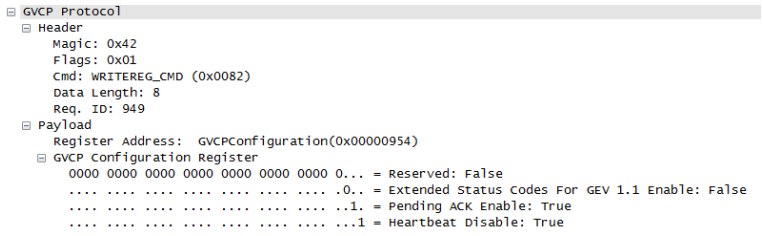


Figure 2: Screenshot of a network analyzer - GVCP Protocol

If the heartbeat bit is set to disable (`"Device Link Heartbeat Mode = On"`; red box in Figure 1), the transport layer lib will stop reading the GevCCP register periodically. To disable the heartbeat makes sense for at least two scenarios:

- while debugging a program and
- in **Triggered line scan acquisition with pausing trigger signals** (p. 253).

In both scenarios, one participant pauses the connection on purpose.

If the heartbeat disable bit is reset again, the transport layer lib will start reading the CCP register periodically again.

1.19.9.3 Reset timestamp by hardware

This feature can be used

- for precise control of timestamp
  - for one camera or
  - to synchronize timestamp of multitude of cameras.

The latter sample, can be achieved by following steps:

1. Define the input line (*"TriggerSource"*) to reset the timestamp, e.g. **"Line5"** and
2. set the *"Trigger Selector"* to **"mvTimestampReset"**.
3. Connect all input lines of all cameras together.
4. Finally, use one output of one camera to generate reset edge:

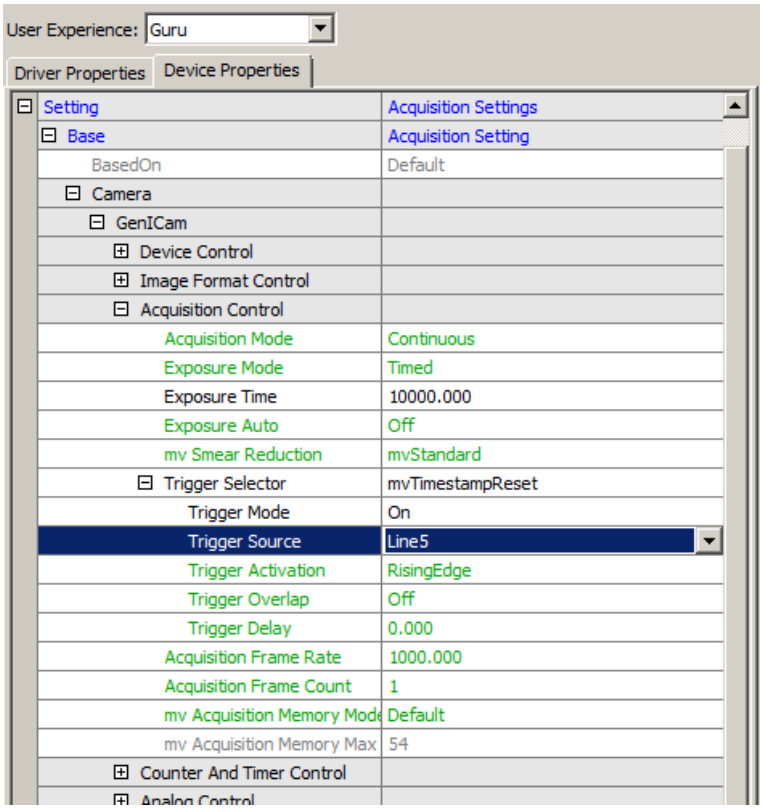


Figure 1: wxPropView - Setting the sample

Note

Be aware of the drift of the individual timestamps.

The timestamp is generated via FPGA in the camera which itself is clocked by a crystal oscillator. This is done independently in each camera and by default not synchronized among cameras or the host system.

Typical stability of crystal oscillators is in the range 100ppm (parts per million).

I.e. for longer operation times (say in excess of hours) there is a tendency that timestamps of individual cameras drift against each other and against the time in the operating system of the host.

Customers wishing to use the individual camera timestamps for synchronization and identification of images via timestamps for multi-camera systems will have in the meantime
- to reset all timestamps either by hardware signal or by command and regularly resynchronize or check the drift algorithmically
- in order to make sure that the drift is less half an image frame time.

1.19.9.4 Synchronizing camera timestamps

1.19.9.4.1 Introduction

Camera timestamps are a recommended **Genicam** (p. 213) / **SFNC** (p. 221) feature to add the information when an image was taken (exactly: when the exposure of the image started).

Without additional synchronization it is merely a camera individual timer with a vendor specific increment and implementation dependent accuracy. Each camera starts its own timestamp beginning with zero and there are no means to adjust or synchronize them among cameras or host PCs. MATRIX VISION cameras offer several ways of synchronizing:

- **mvTimestampReset** (p. 408)
- **Pulse-per-second signal (PPS)** (p. 410)
- **Precision Time Protocol (PTP) IEEE1588** (p. 412)

The usage is described below.

1.19.9.4.2 Without Precision Time Protocol (PTP) IEEE1588

There are many applications which do not or cannot profit from **"IEEE1588"** but have certain synchronization needs. Solutions for these scenarios are describes as follows.

1.19.9.4.3 Synchronizing using mvTimestampReset

First of all the standard does not provide hardware means to reset the timestamp in a camera other than plug off and on again. Therefore MATRIX VISION has created its own mechanism *mvTimestampReset* to reset the timestamp by a hardware input.

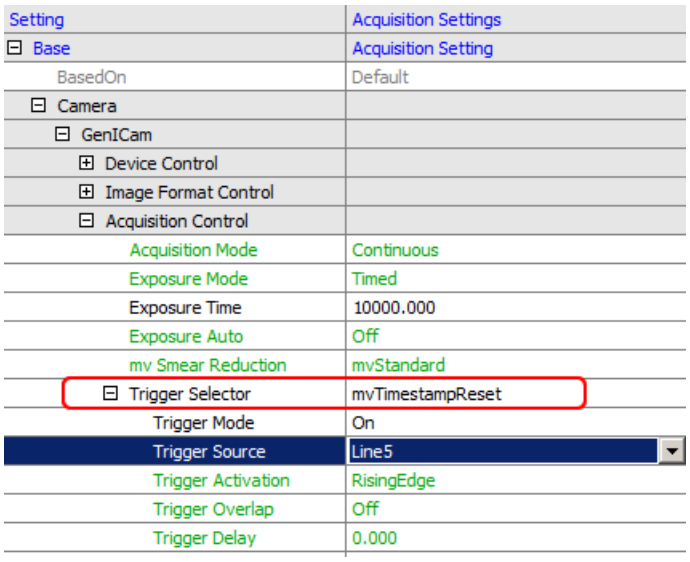


Figure 1: mvTimestampReset

This can be elegantly used for synchronization purposes by means of wiring an input of all cameras together and reset all camera timestamps at the beginning by a defined signal edge from the process. From this reset on all cameras start at zero local time and will increment independently their timestamp so that we achieve a basic accuracy only limited by drift of the clock main frequency (e.g. a 1 MHz oscillator in the FPGA) over time.

In order to compensate for this drift we can in addition reset the timestamp every second or minute or so and count the reset pulse itself by a counter in each camera. Assuming this reset pulse is generated by the master camera itself by means of a timer and output as the hardware reset signal for all cameras, we now can count the reset pulse with all cameras and put both the count and the reset timestamp as so called chunk data in the images.

We thus have achieved a synchronized timestamp with the precision of the master camera among all connected cameras.

Settings required are shown using MATRIX VISION's **wxPropView** (p. 139) tool:

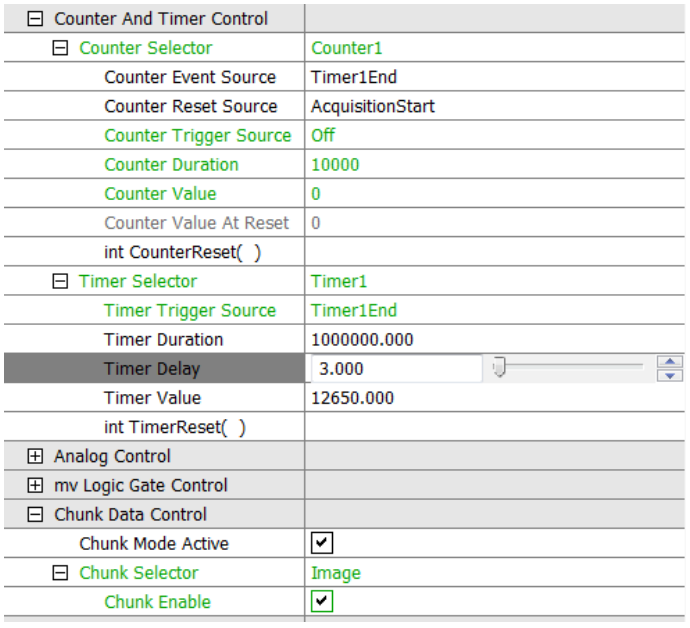


Figure 2: Reset the timestamp every second

An example of the chunk data attached to the image can be seen below. The timestamp is in µs and *Counter1* counts the reset pulses, in this case itself generated by the camera via *Timer1*.

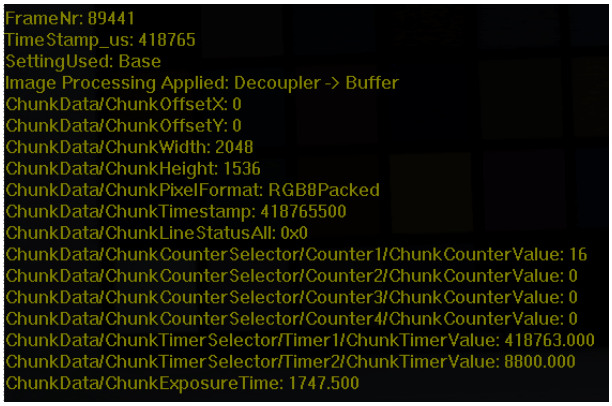


Figure 3: ChunkData

The task of resetting the counter at the beginning of the acquisition can be done by setting the reset property accordingly. Of course is this all independent whether the camera is acquiring images in triggered or continuous mode.

1.19.9.4.4 Synchronizing using a pulse-per-second signal

In order to eliminate the unknown drifts of different devices, a PPS (pulse per second) signal can be fed into each camera using a PC with NTP (network time protocol software), GPS devices or even a looped-back camera timer.

From these pulses the device can extract *how long one second is*. When a device detects that it is no longer running precisely it will adapt its internal clock leading to a "stabilized oscillator".

The device would then maintain their timestamp-differences over long times and stay synchronized. The initial difference between the timers - before the PPS was used - remains however. If you aim to eliminate that as well, you can use the `mvTimestampReset` up front with the same PPS input signal. In an application this can be configured like this (C# syntax):

```
// -----------------------------------------------------------------------
bool waitForNextPulse(Device pDev, String triggerLine)
// -----------------------------------------------------------------------
{
    GenICam.CounterAndTimerControl ctc = new GenICam.CounterAndTimerControl(pDev);
    ctc.counterEventSource.writeS(triggerLine);
    long momentaryValue = ctc.counterValue.read();
    for (int i=0; i<12; i++)
    {
        System.Threading.Thread.Sleep(100);
        if (momentaryValue != ctc.counterValue.read())
        {
            return true;
        }
    }
    return false;
}

// -----------------------------------------------------------------------
void SetupPPS(Device[] pDevs)
// -----------------------------------------------------------------------
{
    string TriggerLine = "Line4";

    if (!waitForNextPulse(pDevs[0],TriggerLine))
    {
```

```
        Console.WriteLine("No pulse seems to be present");
        return;
    }

    // No configure all the devices to reset their timestamp with each pulse coming
    // on the trigger line that the PPS signal is connected to.
    foreach(Device aDevice in pDevs)
    {
        GenICam.AcquisitionControl ac = new GenICam.AcquisitionControl(aDevice);
        ac.triggerSelector.writeS("mvTimestampReset");
        ac.triggerSource.writeS(TriggerLine);
        ac.triggerMode.writeS("On");
    }

    // wait for the next pulse that will then reset the timestamps of all the devices
    if (!waitForNextPulse(pDevs[0],TriggerLine))
    {
        Console.WriteLine("the pulses aren't coming fast enough ...");
        return;
    }

    // Now switch off the reset of the timestamp again. All devices did restart their
    // timestamp counters and will stay in sync using the PPS signal now
    foreach(Device aDevice in pDevs)
    {
        GenICam.AcquisitionControl ac = new GenICam.AcquisitionControl(aDevice);
        ac.triggerMode.writeS("Off");
        GenICam.DeviceControl dc = new GenICam.DeviceControl(aDevice);
        dc.mvTimestampPPSSync.writeS("Line4");
    }
}
```

1.19.9.4.4.1 Using a looped-back camera timer for the PPS signal

To reduce the amount of hardware needed you might want to sacrifice some timestamp validity and use one of the cameras as a master clock. This can be done like this:

- setting *Timer1* to duration 1s
- starting *Timer2* with every *Timer1End* and generating a short pulse (duration = 1000 us)
- placing *Timer2Active* on one of the digital I/O lines and using that as the source for PPS signal

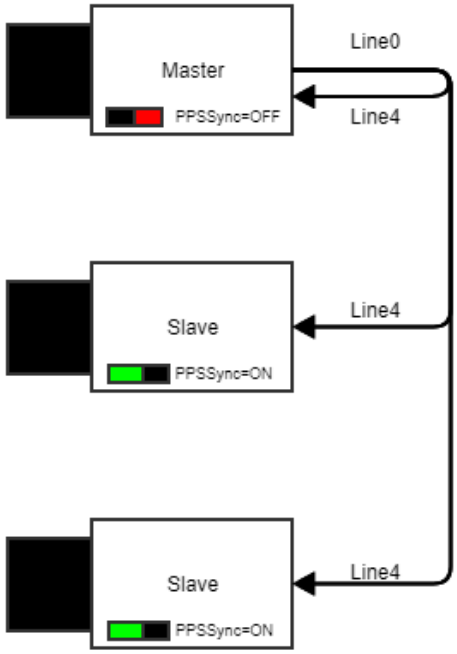


Figure 4: Setup for looped-back Timer

Setting the *Timer1* to 1 s seems like an easy task but due to some internal dependencies you should be carefully here. At the moment two different timer implementations are present in MATRIX VISION products.

- **Type 1:** For cameras with sensors other than the IMX family from Sony please set the duration to the theoretical value of 1000000.
- **Type 2:** For all the other cameras please use 999997 us duration since the self-triggering will consume the other 3 us
- Please refrain from switching on PPSSync inside the Master camera since (at least in Type 1 cameras) since this will lead to an instable feedback loop.

Note

For synchronization you can either use PTP or PPS exclusively.

1.19.9.4.5 Synchronizing using Precision Time Protocol (PTP) IEEE1588

Since

Firmware version 2.37.2285.0

The Precision Time Protocol (PTP) IEEE1588 is a standard for synchronizing devices via Ethernet network. PTP is part of the GigE Vision 2.0 standard and offers a microsecond accuracy. We implemented the two-step mode , i.e. the timestamp is sent in a separate message after the sync message. I.e. MATRIX VISION cameras are able to perform the required synchronization. However, specific network hardware and driver software and procedures can do and maintain the synchronization as well.

You can get the current state via the **PtpDataSetLatch** command which will we written in the **PtpStatus** register.

By default, the camera will work as **slave** with the following camera status transitions (after activating the PTP): *"Disabled → Listening → Uncalibrated → Slave"*

If the camera does not find a better synchronization source, it will change to master with the following status transitions: *"Disabled → Listening → PreMaster → Master"*

To use the PTP, just activate it like this:

1. Check the **PtpEnable** register in *"Setting → Base → Camera → GenICam → TransportLayerControl → PtpControl"*.

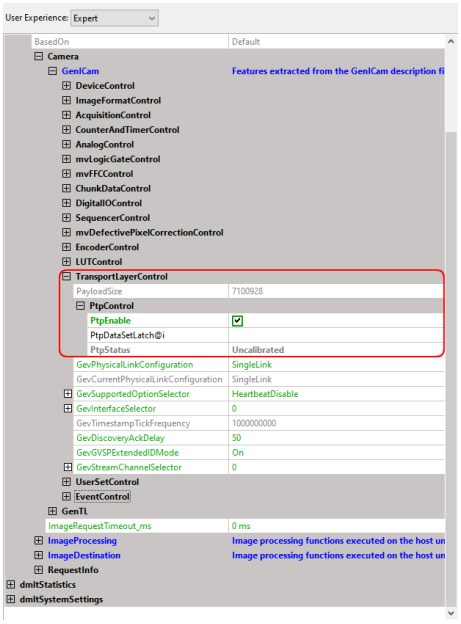


Figure 5: PtpControl

1.19.9.5 Working with the 3 head model

The 3 head model behaves like a mvBlueCOUGAR-X standard camera and for this reason it can be used with every GigE Vision compliant software. The information of the three sensors are returned as a packed pseudo RGB image, with each color channel representing one sensor.

While working with **trigger signals** (p. 347), a trigger signal will trigger all sensors simultaneously.

Because of the three sensor heads, you can set the gain and the offset (black level) of each sensor separately in *"Setting -> Base -> Camera -> GenICam -> Analog Control -> "*

- **"Gain Selector"**
- **"Black Level Selector"**

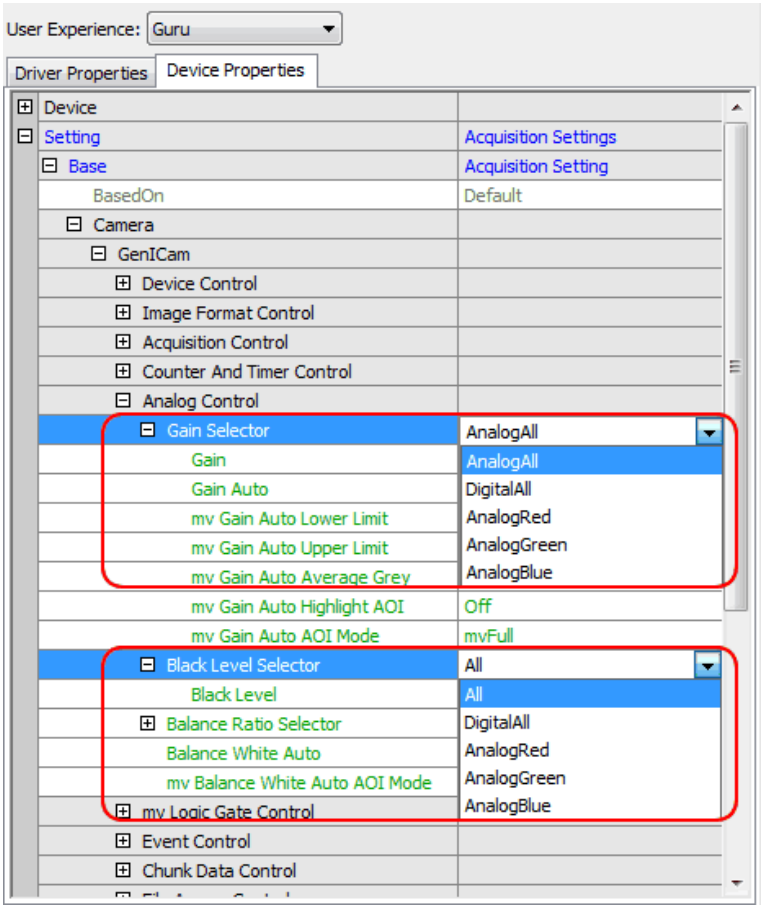


Figure 1: wxPropView - Gain and Black Level Selector

1.19.9.6 Working With The Serial Interface (mv Serial Interface Control)

1.19.9.6.1 Introduction

As mentioned in **GenICam And Advanced Features** section of this manual, the **mv Serial Interface Control** (p. 167) is a feature which allows an easy integration of motor lenses or other peripherals based on RS232.

- Available message buffer size: 1 KByte

Note

Use the Power GND for the RS232 signal.

1.19.9.6.2 Setting up the device

Follow these steps to prepare the camera for the communication via RS232:

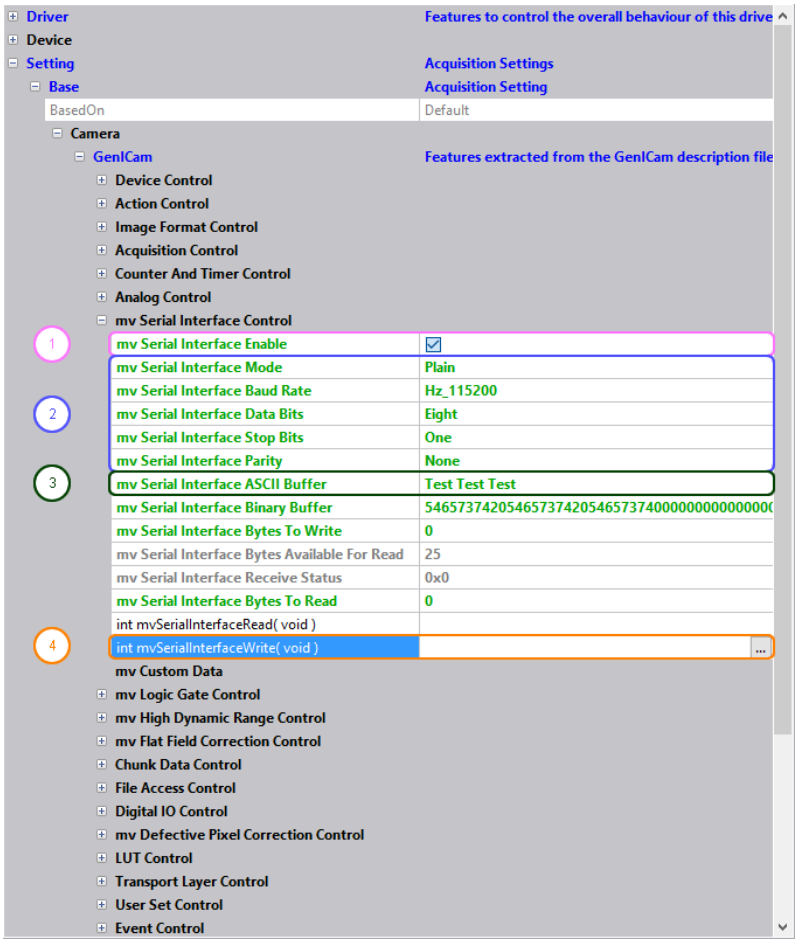


Figure 1: wxPropView - mv Serial Interface Control

1. Start **wxPropView** (p. 139)
2. Connect to the camera
3. Under *"Setting -> Base -> Camera -> GenICam -> mv Serial Interface Control"* activate the serial interface by enabling
   "mv Serial Interface Enable" (1).
   Afterwards **"mv Serial Interface Control"** is available.
4. Set up the connection settings to your needs (2).
5. To test the settings you can send a test message (3).
6. Send messages by executing the function *"int mvSerialInterfaceWrite(void)"* by either clicking on the 3 dots next to the function name or by right-clicking on the command and then selecting *Execute* from the context menu.(4)

Note

Please enter a hexadecimal value in **mvSerialInterfaceBinaryBuffer**.

If you are listening to the RS232 serial line using a tool like *PuTTY* with matching settings...

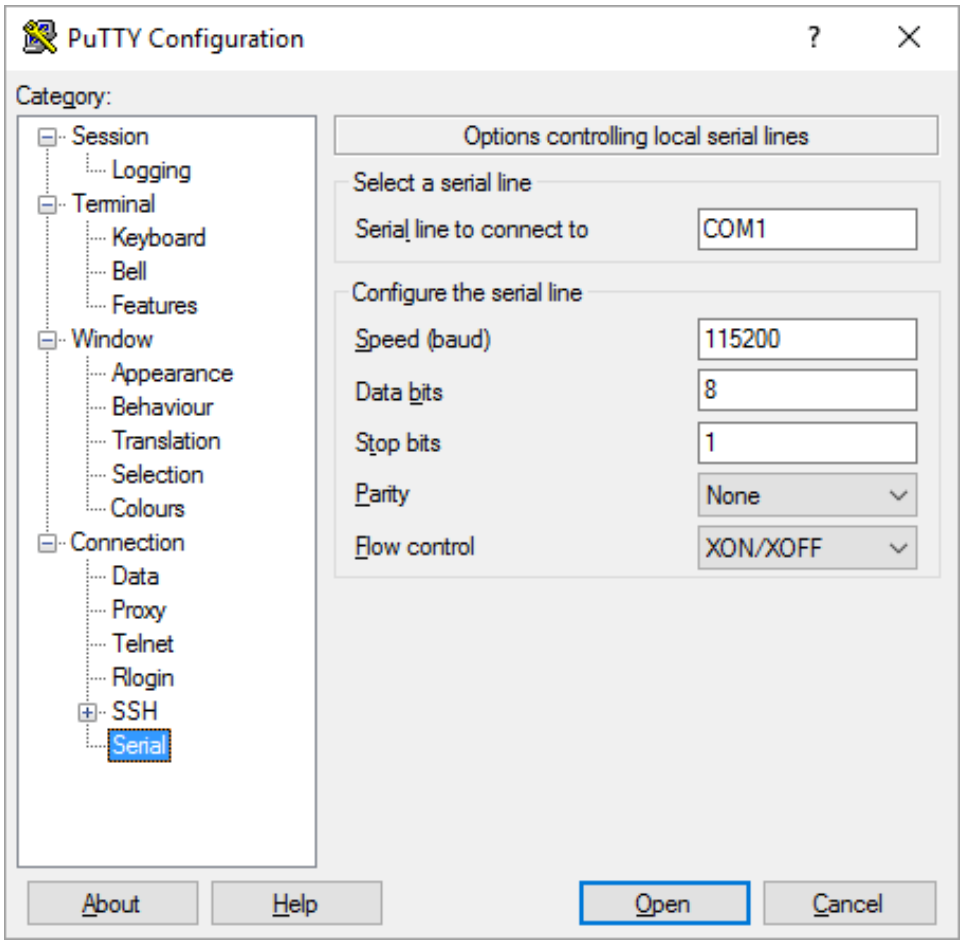


Figure 2: PuTTY - Setting up the serial interface

you will see the test message:



Figure 3: PuTTY - Receiving the test message

1.19.9.6.3 Programming the serial interface

```
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

  // more code
  GenICam::mvSerialInterfaceControl sic( pDev );
  sic.mvSerialInterfaceBaudRate.writeS( "Hz_115200" );
  sic.mvSerialInterfaceASCIIBuffer.writeS( "Test Test Test" );
  sic.mvSerialInterfaceWrite();
  // more code
```

1.19.10 Working with several cameras simultaneously

- **Using Action Commands** (p. 418)
- **Creating synchronized acquisitions using timers** (p. 424)
- **Using the primary application switchover functionality** (p. 430)

1.19.10.1 Using Action Commands

GigE Vision (p. 215) specifies so called **Action Commands** to trigger an action in multiple devices at roughly the same time.

> **Note**
>
> Due to the nature of Ethernet, this is not as synchronous as a hardware trigger since different network segments can have different latencies. Nevertheless in a switched network, the resulting jitter is acceptable for a broad range of applications and this scheme provides a convenient way to synchronize devices by means of a software command.

Action commands can be unicasted or broadcasted by applications with either `exclusive`, `write` or `read` (only when the device is configured accordingly) access to the device. They can be used e.g. to

- increment or reset counters
- reset timers
- act as trigger sources
- ...

The most typical scenario is when an application desires to trigger a simultaneous action on multiple devices. This case is showed by the figure below. The application fires a broadcast action command that will reach all the devices on the subnet.

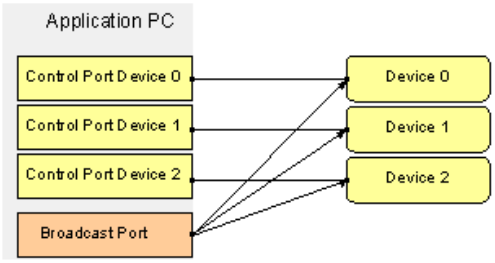


Figure 1: Action command sent as a broadcast to all devices in the subnet

Attention

Due to the nature of Ethernet an action command at a given time can only leave a single network interface thus sending action commands via several different network interfaces to different devices that are e.g. directly connected to a given network interface card in a PC only works sequential and therefore these commands will **NOT** reach the devices at the same time. This can only be achieved using `Scheduled Action Commands`. Depending on whether an action command is unicasted or broadcasted the command will either reach a single device on a given subnet or multiple devices.

Note

The following diagrams assume the connecting line between the PC and the devices to be the GVCP(↩ GigE Vision control protocol)s control port socket. Therefore these diagram do no make any assumption about the physical connection between the devices and the PC. However for the examples to work they must all be connected to the same network interface of the PC using a network switch as otherwise as stated above it would not be possible to sent the very same action command using a single network packet to all the devices.

But action commands can also be used by secondary applications. This can even be another device on the same subnet. This is depicted in the following figure.

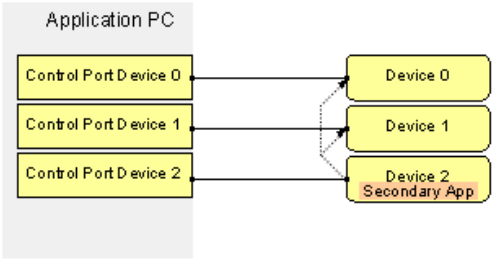


Figure 2: Action command sent as a broadcast by one device to all other devices in the subnet

Upon reception of an action command, the device will decode the information to identify which internal action signal is requested. An action signal is a device internal signal that can be used as a trigger for functional units inside the device (ex: a frame trigger). It can be routed to all signal sinks of the device.

Each action command message contains information for the device to validate the requested operation:

1. `device_key` to authorize the action on this device.
2. `group_key` to define a group of devices on which actions have to be executed.
3. `group_mask` to be used to filter out some of these devices from the group.

Action commands can only be asserted if the device has an open primary control channel (so if an application has established `write` or `exclusive` access to a device) or when `unconditional action` mode is enabled.

A device can define several action commands which can be selected via the `ActionSelector` in the device features property tree.

The conditions for an action command to be asserted by the device are:

1. the device has an open primary control channel or unconditional action mode is enabled.
2. the device_key in the action command sent by the application and the ActionDeviceKey property of the device must be equal.
3. the group_key in the action command sent by the application and the ActionGroupKey property for the corresponding action of the device must be equal.
4. the logical AND-wise operation of group_mask in the action command sent by the application against the ActionGroupMask for the corresponding action of the device must be non-zero. Therefore, they must have at least one common bit set at the same position in the register.

When these 4 conditions are met for at least one action signal configured on the device, then the device internally asserts the requested action. As these conditions could be met on more than one action, the device could assert more than one action signal in parallel. When one of the 4 conditions is not met for any supported action signal, then the device ignores the action command.

This first condition asks for the write or exclusive access being established between the application and the device unless the ActionUnconditionalMode has been enabled. When this bit is set, the device can assert the requested action even if no application has established write or exclusive access as long as the 3 other conditions are met.

1.19.10.1.1 Scheduled Action Command Scheduled Action Commands provide a way to trigger actions in a device at a specific time in the future. The typical use case is depicted in the following diagram:

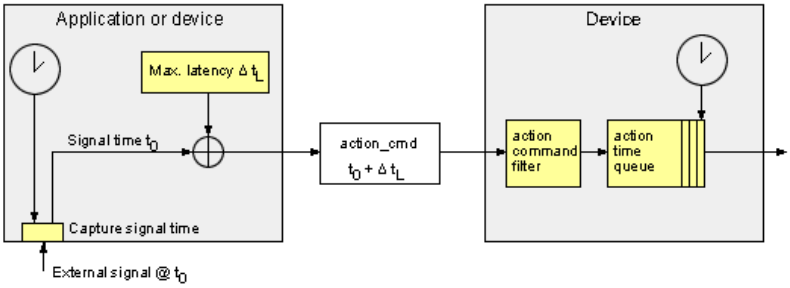


Figure 3: Principle of scheduled action commands

The transmitter of an action command records the exact time, when the source signal is asserted (External signal). This time t 0 is incremented by a delta time DELTA L and transmitted in an action command to the receivers. The delta time DELTA L has to be larger than the longest possible transmission and processing latency of the action command in the network.

If the packet passes the action command filters in the receiver, then the action signal is put into a time queue (the depth of this queue is indicated by the ActionQueueSize property).

When the time of the local clock is greater or equal to the time of an action signal in the queue, the signal is removed from the queue and asserted. Combined with the timestamp precision of IEEE 1588 which can be sub-microseconds, a Scheduled Action Command provides a way to allow low-jitter software trigger. If the sender of an action command is not capable to set a future time into the packet, the action command has a flag to fall back to legacy mode (bit 0 of the flag field). In this mode the signal is asserted in the moment the packet passes the action command filters.

Note

Scheduled action commands are not supported by every device. A device supporting scheduled action commands should also support time stamp synchronization based on IEEE1588. MATRIX VISION GigE Vision devices support **IEEE1588** (p. 412) since firmware version 2.37.2285.0.

1.19.10.1.2 Examples

The following examples illustrate the behavior of action commands in various scenarios. The figure below shows 4 different action commands sent by an application. The content of the action command packet is illustrated on the left side of the figure.

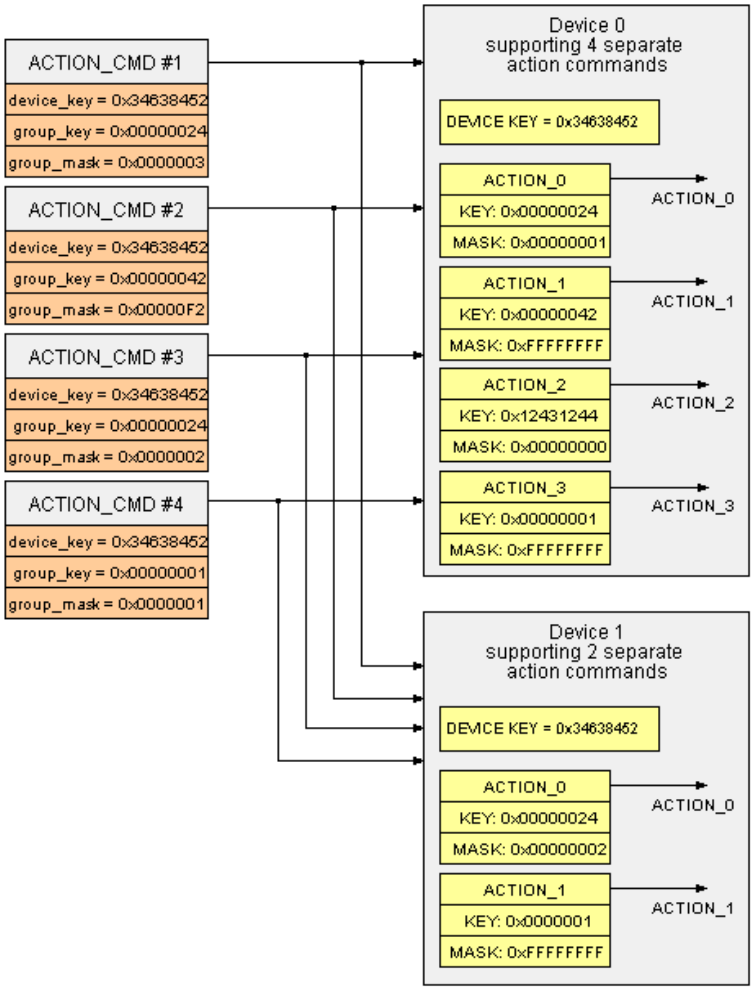


Figure 4: Action command examples

The content of the action command must be examined against the conditions listed above for each supported action signal.

For the first request (ACTION_CMD #1)

	Action Command 1	Device 0			Device 1
		ACTION 0	ACTION 1	ACTION 2	ACTION 0
device_key	0x34638452	0x34638452	0x34638452	0x34638452	0x34638452
group_key	0x00000024	0x00000024	**0x00000042**	**0x12341244**	0x00000024
group_mask	0x00000003	0x00000001	0xFFFFFFFF	**0x00000000**	0x00000002

Device 0 receives the request and looks for the 4 conditions.

1. `exclusive` or `write` access has been established between the application and the device (or unconditional action is enabled)
2. `device_key` matches
3. `group_key` matches
4. Logical AND-wise comparison of requested `group_mask` is non-zero

All 4 conditions are met only for ACTION_0, thus the device immediately asserts the internal signal represented by ACTION_0. The same steps are followed by Device 1. Only the `group_mask` is different, but nevertheless the logical bitwise AND operation produces a non-zero value, leading to the assertion of ACTION_0 by Device 1.

For the second request (ACTION_CMD #2)

	Action Command 2	Device 0			Device 1
		ACTION 0	ACTION 1	ACTION 2	ACTION 0
device_key	0x34638452	0x34638452	0x34638452	0x34638452	0x34638452
group_key	0x00000042	**0x00000024**	0x00000042	**0x12341244**	**0x00000024**
group_mask	0x000000F2	**0x00000001**	0xFFFFFFFF	**0x00000000**	0x00000002

Looking for the 4 conditions, Device 0 will assert ACTION_1 while Device 1 will not assert any signal because the `group_key` condition is not met. Therefore, Device 1 ignores the request.

For the third request (ACTION_CMD #3)

	Action Command 3	Device 0			Device 1
		ACTION 0	ACTION 1	ACTION 2	ACTION 0
device_key	0x34638452	0x34638452	0x34638452	0x34638452	0x34638452
group_key	0x00000024	0x00000024	**0x00000042**	**0x12341244**	0x00000024
group_mask	0x00000002	**0x00000001**	0xFFFFFFFF	**0x00000000**	0x00000002

In the third example, the `group_mask` and `group_key` of Device 0 do not match with ACTION_CMD #3 for any of the ACTION_0 to ACTION_2. Therefore, Device 0 ignores the request. Device 1 asserts ACTION_0 since the 4 conditions are met.

The ACTION_CMD is flexible enough to accommodate “simultaneous” triggering of the same functional action in functionally different devices.

For instance, let’s assume the software trigger of Device 0 can only be associated to its ACTION_3 and that the software trigger of Device 1 can only be associated to its ACTION_1. And the action command supports to trigger the same functional action provided that their respective action group key and masks are set in order to meet the conditions from the previous list.

For the fourth request (ACTION_CMD #4)

	Action Command 4	Device 0	Device 1
		ACTION 3	ACTION 1
device_key	0x34638452	0x34638452	0x34638452
group_key	0x00000001	0x00000001	0x00000001
group_mask	0x00000001	0xFFFFFFFF	0xFFFFFFFF

In this case, Device 0 asserts ACTION_3 and Device 1 asserts ACTION_1 since the conditions are met. As a result of this, the software trigger of both devices can be “simultaneously” triggered even though they are associated to different action numbers.

1.19.10.1.3 Writing Code Using Action Commands

The following section uses C# code snippets but the same thing can be done using a variety of other programming languages as well.

To set up an action command on the device something like this is needed:

```
private static void setupActionCommandOnDevice(GenICam.ActionControl ac, Int64 deviceKey,
    Int64 actionNumber, Int64 groupKey, Int64 groupMask)
{
  if ((deviceKey == 0) && (groupKey == 0) && (groupMask == 0))
  {
    Console.WriteLine("Switching off action {0}.", actionNumber);
  }
  else
  {
    Console.WriteLine("Setting up action {0}. Device key: 0x{1:X8}, group key: 0x{2:X8},
        group mask: 0x{3:X8}", actionNumber, deviceKey, groupKey, groupMask);
  }
  ac.actionDeviceKey.write(deviceKey);
  ac.actionSelector.write(actionNumber);
  ac.actionGroupKey.write(groupKey);
  ac.actionGroupMask.write(groupMask);
}
```

Note

In a typical scenario the `deviceKey` parameter will only be set up once as there is only one register for it on each device while there can be various action commands.

Now to send action commands to devices connected to a certain interface it might be necessary to locate the correct instance of the **InterfaceModule** class first. One way to do this would be like this:

```
private static List<GenICam.InterfaceModule> getGenTLInterfaceListForDevice(Device pDev)
{
  // first get a list of ALL interfaces in the current system
  GenICam.SystemModule systemModule = new GenICam.SystemModule(pDev);
  Dictionary<String, Int64> interfaceIDToIndexMap = new Dictionary<string, long>();
  Int64 interfaceCount = systemModule.interfaceSelector.maxValue + 1;
  for (Int64 i = 0; i < interfaceCount; i++)
  {
    systemModule.interfaceSelector.write(i);
    interfaceIDToIndexMap.Add(systemModule.interfaceID.read(), i);
  }

  // now try to get access to the interfaces the device in question is connected to
  PropertyI64 interfaceID = new PropertyI64();
  DeviceComponentLocator locator = new DeviceComponentLocator(pDev.hDev);
  locator.bindComponent(interfaceID, "InterfaceID");
  if (interfaceID.isValid == false)
  {
    return null;
  }
  ReadOnlyCollection<String> interfacesTheDeviceIsConnectedTo = interfaceID.listOfValidStrings;

  // create an instance of the GenICam.InterfaceModule class for each interface the device is
  // connected to
  List<GenICam.InterfaceModule> interfaces = new List<GenICam.InterfaceModule>();
  foreach (String interfaceIDString in interfacesTheDeviceIsConnectedTo)
  {
    interfaces.Add(new GenICam.InterfaceModule(pDev, interfaceIDToIndexMap[interfaceIDString]));
  }
  return interfaces;
}
```

Once the desired interface has been located it could be configured to send an action command like this:

```
private static void setupActionCommandOnInterface(GenICam.InterfaceModule im, Int32 destinationIP,
   Int64 deviceKey, Int64 groupKey, Int64 groupMask, bool boScheduledAction)
{
  im.mvActionDestinationIPAddress.write(destinationIP);
  im.mvActionDeviceKey.write(deviceKey);
  im.mvActionGroupKey.write(groupKey);
  im.mvActionGroupMask.write(groupMask);
  im.mvActionScheduledTimeEnable.write(boScheduledAction ? TBoolean.bTrue : TBoolean.bFalse);
  // here the desired execution time must also be configured for scheduled action commands
  // if desired by writing to the im.mvActionScheduledTime property
}
```

Now the interface is set up completely and sending an action command works like this:

```
private static void sendActionCommand(GenICam.InterfaceModule im)
{
  im.mvActionSend.call();
}
```

Depending on the value of `destinationIP` when actually firing the action command either a broadcast or a unicast message will be generated and send to either one or all devices in the subnet and depending on whether one or more action commands on the device are set up to assert the command they will react appropriately or will silently ignore the command.

1.19.10.2 Creating synchronized acquisitions using timers

1.19.10.2.1 Basics

Getting images from several cameras exactly at the same time is a major task in

- 3D image acquisitions
  (the images must be acquired at the same time using two cameras) or
- acquisitions of larger objects
  (if more than one camera is required to span over the complete image, like in the textile and printing industry).

To solve this task MATRIX VISION devices offer timers that can be used to generate pulse at regular intervals. This pulse can be connected to a digital output. The digital output can be connected digital to the digital input of one or more cameras to use it as a trigger.

1.19.10.2.2 Connecting the hardware

Note

We recommend to use the cable *"KS-BCX-HR12"* to connect an external power supply (at pin 2 and 10) and the I/Os.

1.19.10.2.2.1 On the master camera

1. Connect power supply GND to **"pin 1"** and **"pin 7"**.
2. Connect power supply +12 V to **"pin 2"** and **"pin 10 (power supply for the outputs)"**.
3. Connect DigOut0 of master (**"pin 6"**) to DigIn0 of master (**"pin 4"**).

1.19.10.2.2.2 On each slave camera

1. Connect power supply GND to **"pin 1"** and **"pin 7"**.
2. Connect power supply +12 V to **"pin 2"** and **"pin 10 (power supply for the outputs)"**.

1.19.10.2.2.3 Between the cameras

1. Connect DigOut0 of master (**"pin 6"**) to DigIn0 of slave (**"pin 4"**).

Note

If more than one slave is used, connect the same **"pin 6"** of master to all **"pin 4"** of the slaves.

If each camera has its own power supply, then connect all grounds (GND) together.

For the master camera, there are 2 possibilities how it is triggered:

1. The master camera triggers itself logically (so called **"Master - Slave"**, see Figure 1), or
2. the master camera uses the external trigger signal, which was created by itself, via digital input (so called **"Slave - Slave"**, see Figure 2).

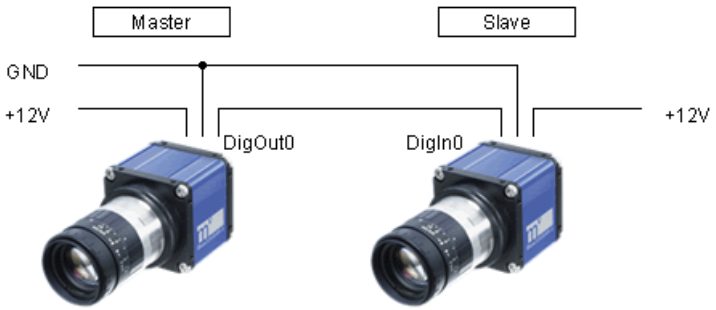


Figure 1: Master - Slave connecting

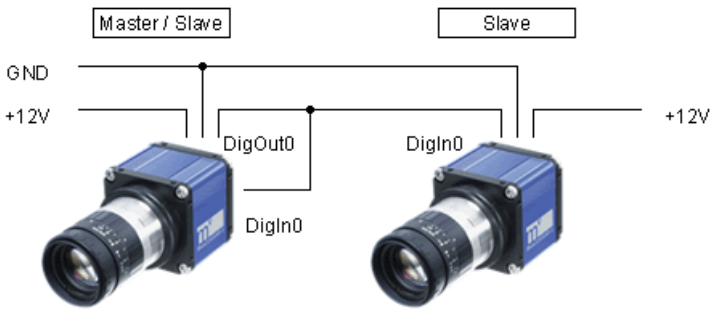


Figure 2: Slave - Slave connecting

Note

With **"Master - Slave"** and according to the delay of the opto-isolated inputs of the slave cameras, you have to adapted the property *"Trigger delay"* of the master camera to synchronize the cameras exactly.

1.19.10.2.3 Programming the acquisition

You will need two timers and you have to set a trigger.

1.19.10.2.3.1 Start timer

Two timers are used for the **"start timer"**. *Timer1* defines the interval between two triggers. *Timer2* generates the trigger pulse at the end of *Timer1*.

The following sample shows a trigger

- which is generated every second and
- the pulse width is 10 ms:

```
#include <mvIMPACT_CPP/mvIMPACT_acquire.h>
#include <mvIMPACT_CPP/mvIMPACT_acquire_GenICam.h>

...

// Master: Set timers to trig image: Start after queue is filled
    GenICam::CounterAndTimerControl catcMaster(pDev);
    catcMaster.timerSelector.writeS( "Timer1" );
    catcMaster.timerDelay.write( 0. );
    catcMaster.timerDuration.write( 1000000. );
    catcMaster.timerTriggerSource.writeS( "Timer1End" );

    catcMaster.timerSelector.writeS( "Timer2" );
    catcMaster.timerDelay.write( 0. );
    catcMaster.timerDuration.write( 10000. );
    catcMaster.timerTriggerSource.writeS( "Timer1End" );
```

See also

Counter And Timer Control (p. 148)

Note

Make sure the *Timer1* interval must be larger than the processing time. Otherwise, the images are lost.

The timers are defined, now you have to do following steps:

1. Set the digital output, e.g. **"Line 0"**,
2. connect the digital output with the inputs of the slave cameras (and master camera if using **"Slave - Slave"**), and finally
3. set the trigger source to the digital input, e.g. **"Line4"**.

1.19.10.2.3.2 Set digital I/O

In this step, the signal has to be connected to the digital output, e.g. **"Line0"**:

```
// Set Digital I/O
    GenICam::DigitalIOControl io(pDev);
    io.lineSelector.writeS( "Line0" );
    io.lineSource.writeS( "Timer2Active" );
```

See also

Digital I/O Control (p. 155)

This signal has to be connected with the digital inputs of the slave cameras as shown in Figure 1 and 2.

1.19.10.2.3.3 Set trigger

"If you want to use Master - Slave":

```
// Set Trigger of Master camera
    GenICam::AcquisitionControl ac(pDev);
    ac.triggerSelector.writeS( "FrameStart" );
    ac.triggerMode.writeS( "On" );
    ac.triggerSource.writeS( "Timer1Start" );
// or ac.triggerSource.writeS( "Timer1End" );

// Set Trigger of Slave camera
    GenICam::AcquisitionControl ac(pDev);
    ac.triggerSelector.writeS( "FrameStart" );
    ac.triggerMode.writeS( "On" );
    ac.triggerSource.writeS( "Line4" );
    ac.triggerActivation.writeS( "RisingEdge" );
```

"If you want to use Slave - Slave":

```
// Set Trigger of Master and Slave camera
    GenICam::AcquisitionControl ac(pDev);
    ac.triggerSelector.writeS( "FrameStart" );
    ac.triggerMode.writeS( "On" );
    ac.triggerSource.writeS( "Line4" );
    ac.triggerActivation.writeS( "RisingEdge" );
```

See also

Acquisition Control (p. 144)

Now, the two timers will work like the following figure illustrates, which means

- *Timer1* is the trigger event and
- *Timer2* the trigger pulse width:

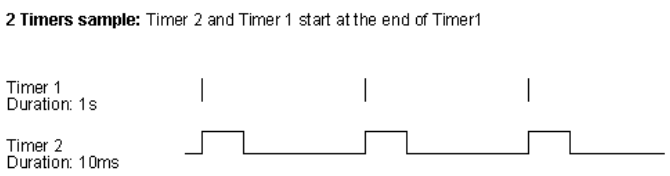


Figure 3: Timers

By the way, this is a simple **"pulse width modulation (PWM)"** example.

1.19.10.2.4 Setting the synchronized acquisition using wxPropView

The following figures show, how you can set the timers and trigger using the GUI tool **wxPropView** (p. 139)

1. Setting of *Timer1* (blue box) on the master camera:

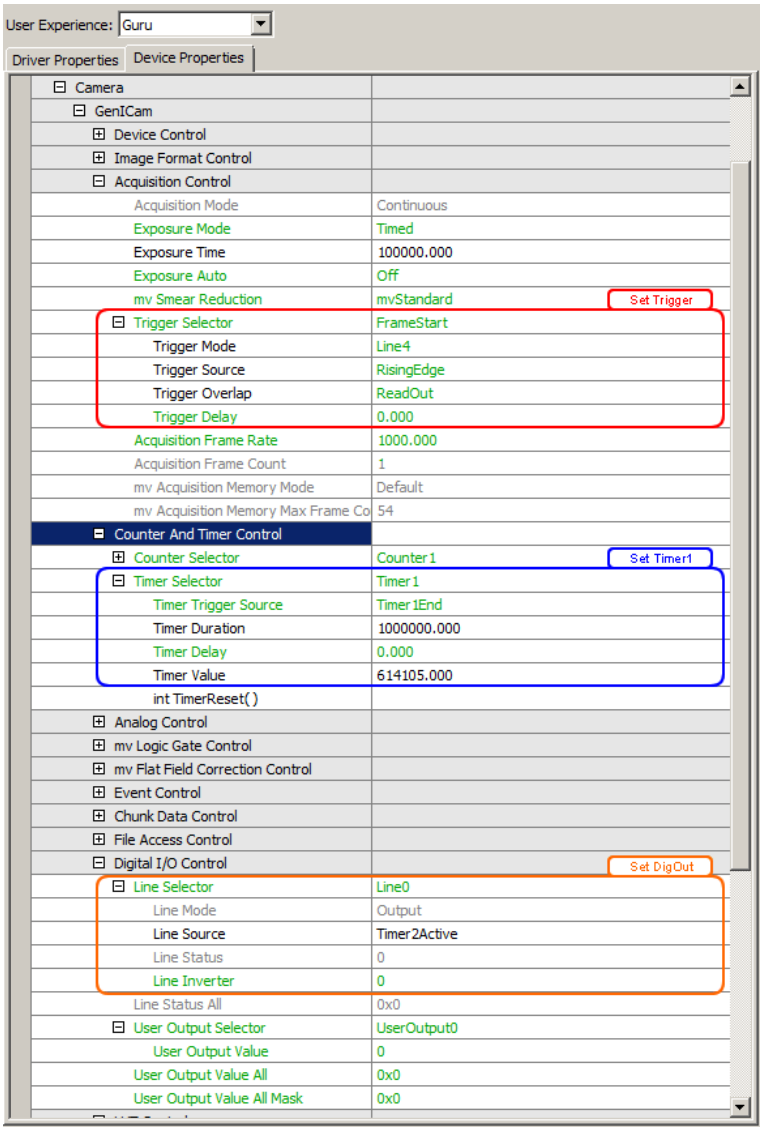


Figure 4: wxPropView - Setting of *Timer1* on the master camera

2. Setting of *Timer2* (purple box) on the master camera:

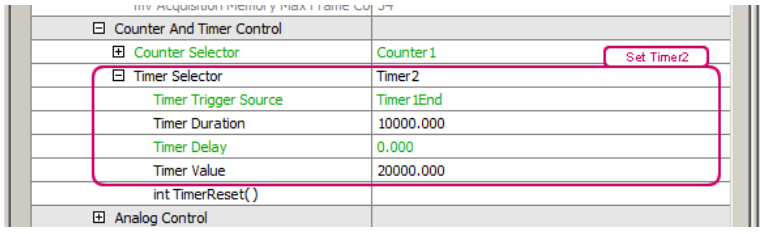


Figure 5: wxPropView - Setting of *Timer2* on the master camera

3. Setting the trigger slave camera(s)
   - The red box in Figure 4 is showing **"Slave - Slave"**, which means that both master and slave camera are connected via digital input.
   - The red box in Figure 6 is showing **"Master - Slave"**), which means that the master is triggered internally and the slave camera is set as shown in Figure 4.

4. Assigning timer to DigOut (orange box in Figure 4).

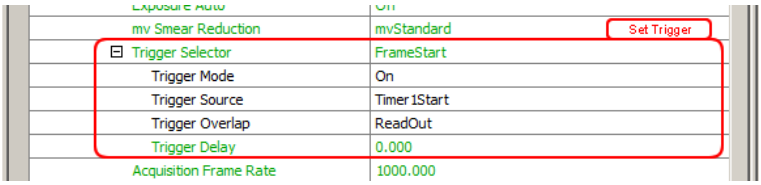


Figure 6: Trigger setting of the master camera using **"Master - Slave"**

1.19.10.3 Using the primary application switchover functionality

There are scenarios where a second application should take control over a device that is already under control of another (primary) application (e.g. systems which requires redundancy, fault recovery or systems with a higher level management entity).

The switchover procedure will look like this: The primary application

1. requests (and gets granted) exclusive access,
2. verifies that the device supports switchover consulting GVCP capability register,
3. sets the control switchover key register,
4. requests (and gets granted) control access with enabled switchover (this is done without closing the control channel),

Another application that knows the key, can request (and gets granted) device control.

You can enable the switchover via *"Device -> PrimaryApplicationSwitchoverEnable"*. Set this register to *"On"* to allow other applications to take control over the device.

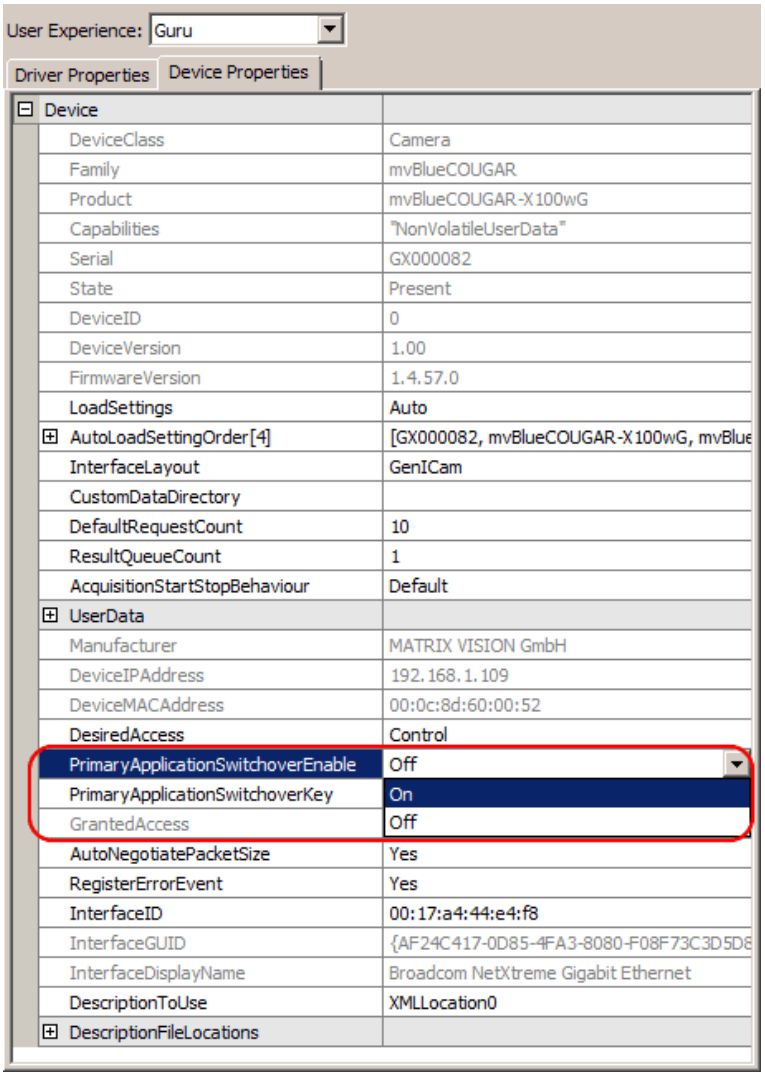


Figure 1: wxPropView - Device PrimaryApplicationSwitchoverEnable

If the control access has been granted, *"DesiredAccess"*, *"PrimaryApplicationSwitchoverEnable"* and *"PrimaryApplicationSwitchoverKey"* will become read-only.

Now, in *"Setting -> Base -> Camera -> GenICam -> Transport Layer Control"* the property *"Gev Primary Application Switchover Key"* can be used by the control application to define a value that must be specified by an application that wants to take over control over the device. E.g. *"666"*:

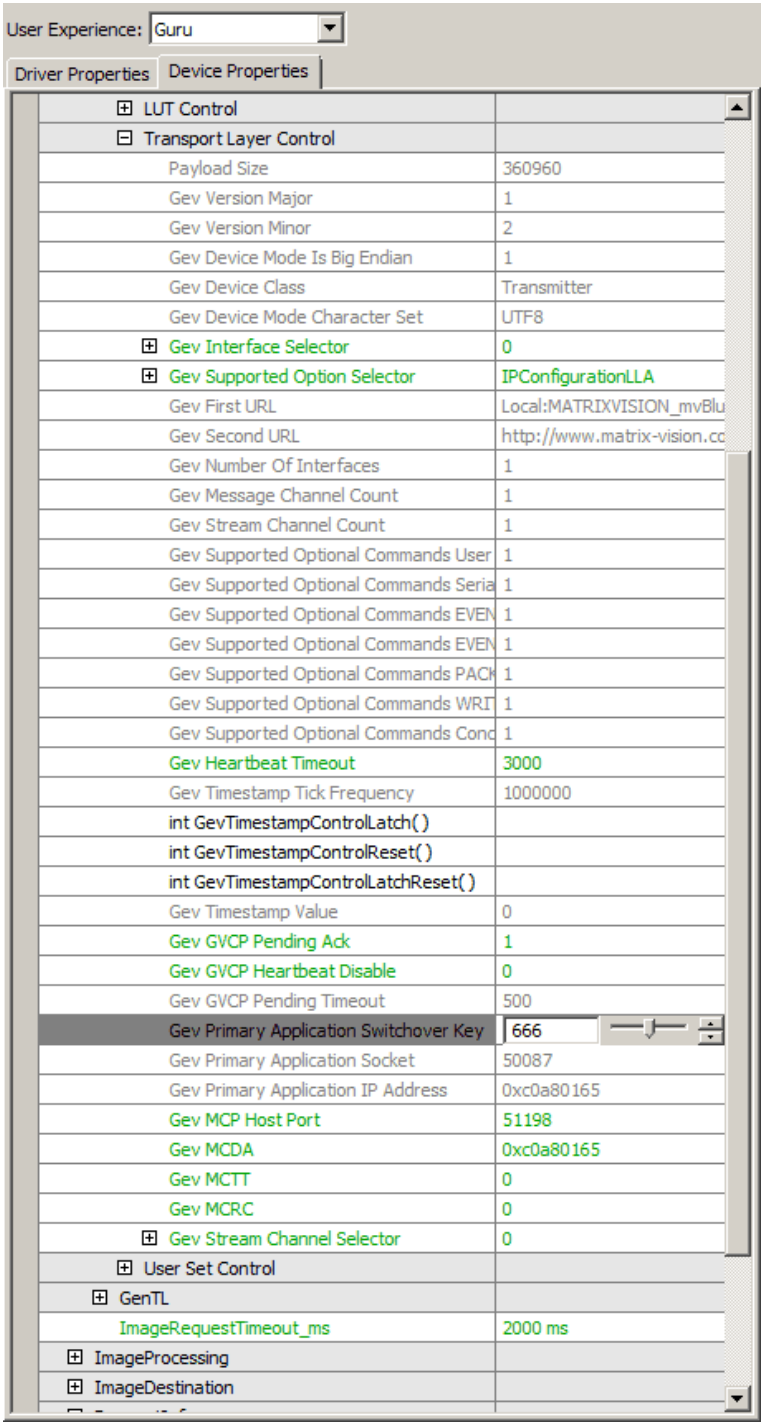


Figure 2: wxPropView - Gev Primary Application Switchover Key

The other application now tries to take over control with the correct switchover key and this access is granted. As a result the first application can no longer write to the device (executing *"int timestampLatch()"* fails with *"DEV_ACCESS_DENIED"*).

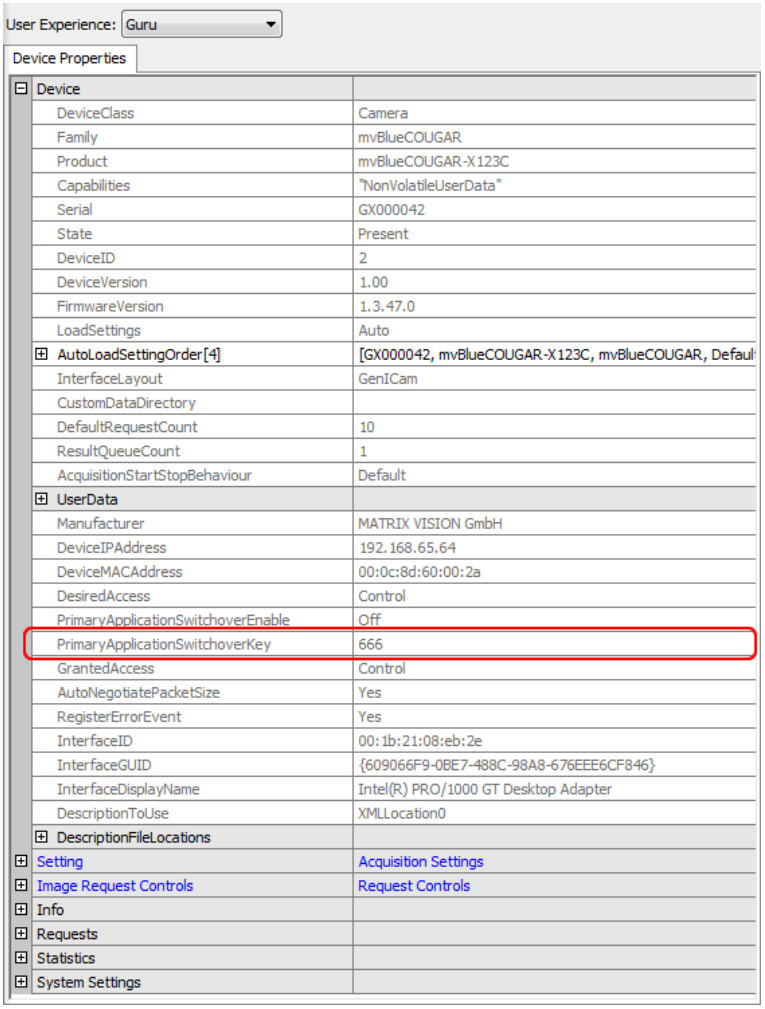


Figure 3: wxPropView - Application Switchover

Note

If the other application tries to take over control without specifying the correct switchover key, an *"DEV_ACCESS_DENIED"* error will appear.

1.19.10.3.1 Code samples

The following code samples show, how to create functions which

- bind and set a property,
- how to configure the switchover access and
- how to take over the control.

```
//-----------------------------------------------------------------------------
template<typename _Ty, typename _Tx>
void bindAndSetProperty( _Ty& prop, ComponentLocatorBase& locator, const string& propName,
const _Tx& value )
//-----------------------------------------------------------------------------
{
  locator.bindComponent( prop, propName );
  if( prop.isValid() )
  {
    try
```

```
    {
      prop.write( value );
      cout << "Property '" << propName << "' set to '" << prop.readS() << "'." << endl;
      // read back the value from the property in order to notice any rounding(e.g. for
      // properties defining a setp width)
    }
    catch( const ImpactAcquireException& e )
    {
      cout << "Failed to write '" << value << "' to property '" << propName << "'. Error message: "
      << e.getErrorString() << endl;
      displayPropertyData( prop );
    }
  }
  else
  {
    cout << "Property '" << propName << "' is not supported by this device or could not be located."
    << endl;
  }
}

//-----------------------------------------------------------------------------
void configureControlAccessWithSwitchoverEnabled( Device* pDev, int64_type key )
//-----------------------------------------------------------------------------
{
  try
  {
    ComponentLocator locator(pDev->hDev());
    PropertyIBoolean primaryApplicationSwitchoverEnable;
    bindAndSetProperty( primaryApplicationSwitchoverEnable, locator,
     "PrimaryApplicationSwitchoverEnable", bTrue );
    pDev->interfaceLayout.write( dilGenICam );
    pDev->desiredAccess.write( damControl );
    pDev->open();
    DeviceComponentLocator devLocator(pDev, dltSetting);
    PropertyI64 gevPrimaryApplicationSwitchoverKey;
    bindAndSetProperty( gevPrimaryApplicationSwitchoverKey, locator,
     "GevPrimaryApplicationSwitchoverKey", key );
  }
  catch( const ImpactAcquireException& e )
  {
    cout << string(__FUNCTION__) << ": " << e.getErrorString() << "("
    << e.getErrorCodeAsString() << ") occurred at line " << e.getErrorOrigin() << endl;
  }
}

//-----------------------------------------------------------------------------
void takeOverControlAccess( Device* pDev, int64_type key, bool boKeepSwitchoverPossible )
//-----------------------------------------------------------------------------
{
  try
  {
    ComponentLocator locator(pDev->hDev());
    PropertyIBoolean primaryApplicationSwitchoverEnable;
    bindAndSetProperty( primaryApplicationSwitchoverEnable, locator,
    "PrimaryApplicationSwitchoverEnable", boKeepSwitchoverPossible ? bTrue : bFalse );
    PropertyI64 primaryApplicationSwitchoverKey;
    bindAndSetProperty( primaryApplicationSwitchoverKey, locator,
    "PrimaryApplicationSwitchoverKey", key );
    pDev->interfaceLayout.write( dilGenICam );
    pDev->desiredAccess.write( damControl );
    pDev->open();
  }
  catch( const ImpactAcquireException& e )
  {
    cout << string(__FUNCTION__) << ": " << e.getErrorString() << "("
    << e.getErrorCodeAsString() << ") occurred at line " << e.getErrorOrigin() << endl;
  }
}
```

1.19.11 Working with 3rd party tools

- **Using VLC Media Player** (p. 434)
- **Working with ROS (Robot Operating System)** (p. 436)
- **Using GigE Vision™ Devices In A Docker Container** (p. 439)

1.19.11.1 Using VLC Media Player

With the **DirectShow interface** (p. 178) MATRIX VISION devices become a (acquisition) video device for the VLC Media Player.

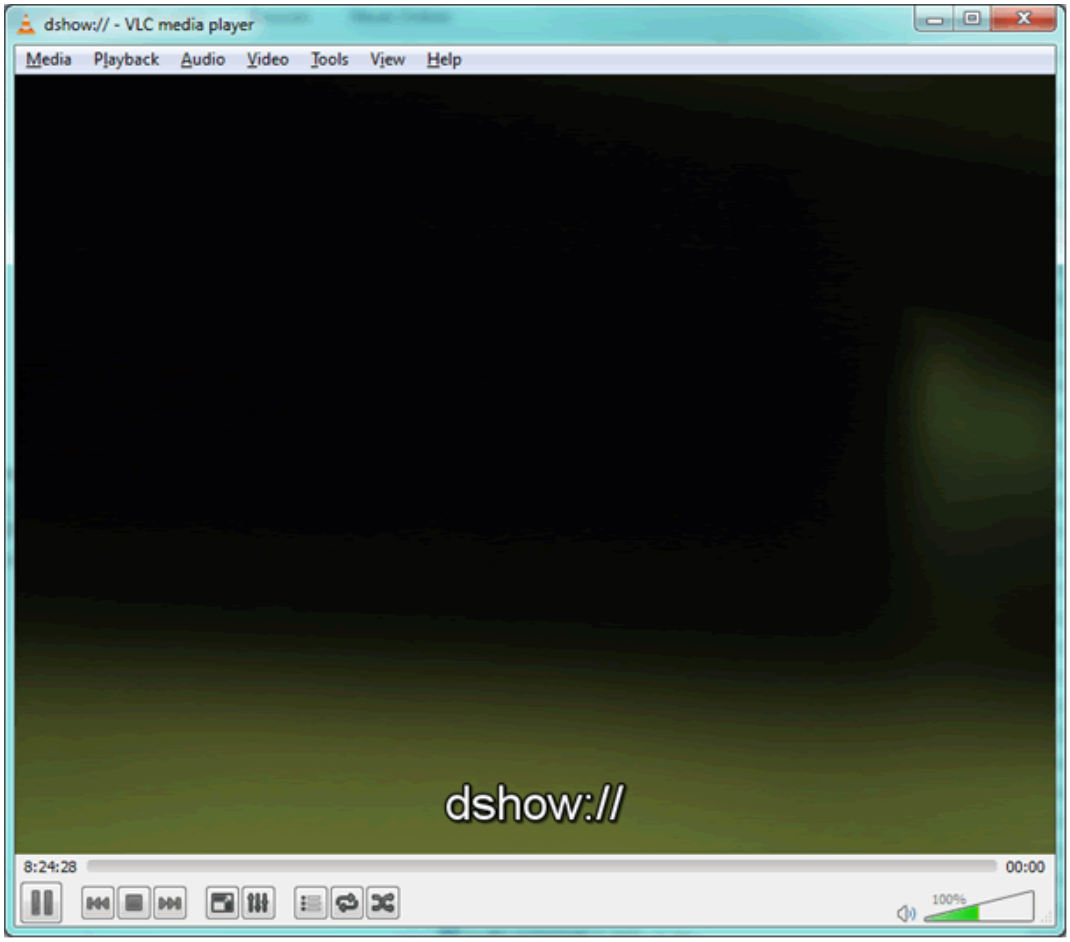


Figure 1: VLC Media Player with a connected device via DirectShow

1.19.11.1.1 System requirements

It is necessary that following drivers and programs are installed on the host device (laptop or PC):

- Windows 7 or higher, 32-bit or 64-bit
- up-do-date VLC Media Player, 32-bit or 64-bit (here: version 2.0.6)
- up-do-date MATRIX VISION driver, 32-bit or 64-bit (here: version 2.5.6)

Note

Using **Windows 10** or **Windows 7**: VLC Media Player with versions 2.2.0 have been tested successfully with older versions of mvIMPACT Acquire. Since version 3.0.0 of VLC at least mvIMPACT Acquire 2.34.0 will be needed to work with devices through the DirectShow interface!

1.19.11.1.2 Installing VLC Media Player

1. Download a suitable version of the VLC Media Player from the VLC Media Player website mentioned below.
2. Run the setup.
3. Follow the installation process and use the default settings.

A restart of the system is not required.

See also

`http://www.videolan.org/`

1.19.11.1.3 Setting up MV device for DirectShow

Note

Please be sure to register the MV device for DirectShow with the right version of **mvDeviceConfigure** (p. 139) . I.e. if you have installed the 32 bit version of the VLC Media Player, you have to register the MV device with the 32 bit version of **mvDeviceConfigure** (p. 139) (*"C:/Program Files/MATRIX VISION/mv↩ IMPACT Acquire/bin"*) !

1. Connect the MV device to the host device directly or via GigE switch using an Ethernet cable.
2. Power the camera using a power supply at the power connector.
3. Wait until the status LED turns blue.
4. Open the tool **mvDeviceConfigure** (p. 139) ,
5. **set a friendly name** (p. 181) ,
6. and **register the MV device for DirectShow** (p. 178) .

Note

In some cases it could be necessary to repeat step 5.

1.19.11.1.4 Working with VLC Media Player

1. Start VLC Media Player.
2. Click on **"Media ->** **Open Capture Device..."** .

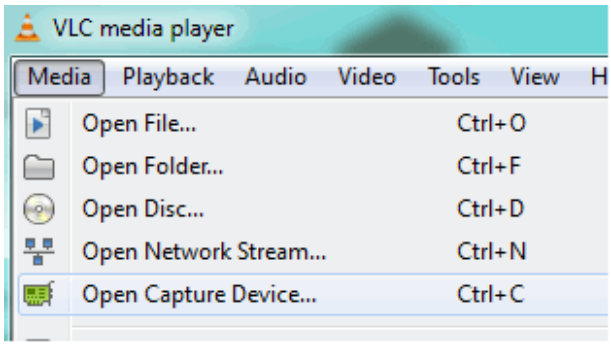


Figure 2: Open Capture Device...

3. Select the tab **"Device Selection"** .
4. In the section **"Video device name"** , select the friendly name of the MV device:

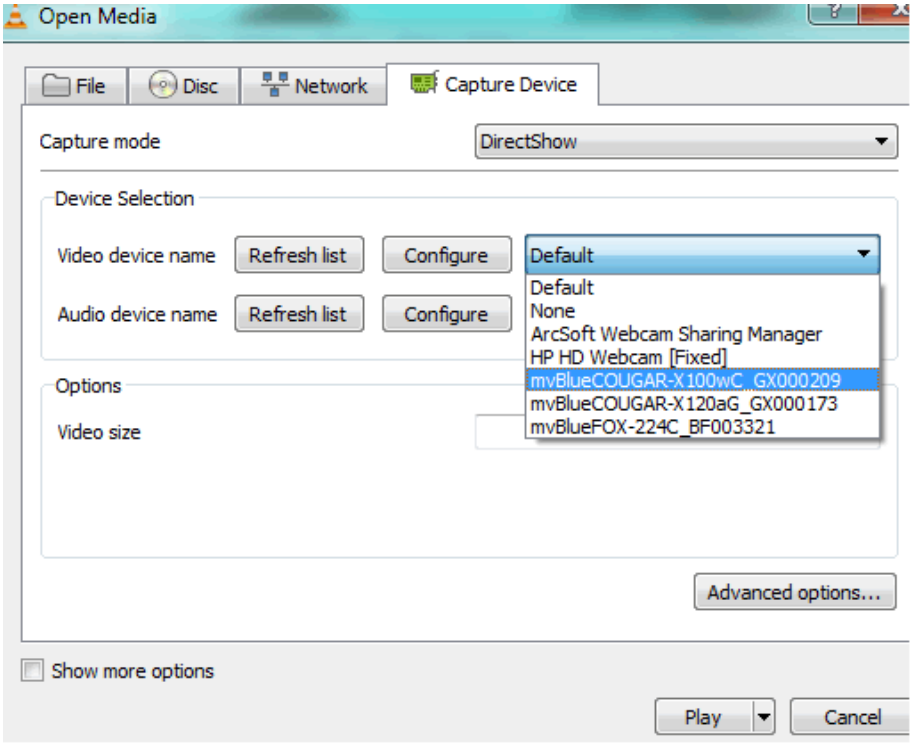


Figure 3: Video device name

5. Finally, click on **"Play"** .
   After a short delay you will see the live image of the camera.

1.19.11.2 Working with ROS (Robot Operating System)

1.19.11.2.1 Recommended ROS Packages If you are planning to use MATRIX VISION GenICam™ devices in a ROS-application, we recommend to use two packages, maintained by the `Roboception GmbH`:

- *rc_genicam_api*, which offers a GenICam™-based access to GenTL™ producers

- *rc_genicam_camera*, which builds on the GenICam™ access provided by *rc_genicam_api* and provides a ROS-node for acquiring images from a camera

Since these packages can be a solid foundation for your ROS project, the following chapters describe how you can use them.

1.19.11.2.2 Initial Setup

1. Make sure to install the mvIMPACT Acquire driver package. Afterwards, the GENICAM_GENTL64_PATH environment variable should point to a directory that contains the mvGenTLProducer.cti library.
2. Install your ROS environment. Please refer to the `ROS Project` if you need more detailed advice.
3. Install or download and build the Roboception packages.
   - For Ubuntu and Debian systems, use the package manager to install:
     (a) **ros-DISTRIBUTION-rc-genicam-api**
     (b) **ros-DISTRIBUTION-rc-genicam-camera** (e.g. `apt install ros-noetic-rc-genicam-api ros-noetic-rc-genicam-camera` if you are using ROS Noetic Ninjemys)
   - If you want to build it yourself, the packages are available on GitHub:
     (a) `rc_genicam_api` like mentioned on the ROS description page `http://wiki.ros.org/rc_genicam_api`
     (b) `rc_genicam_camera`

1.19.11.2.3 Testing The Setup To test your setup, you may use the **gc_info** command line tool, that comes with the *rc_genicam_api* package. This tool will use the **GenICam** (p. 213) Transport Layer that is found within the GENICAM_GENTL64_PATH. A first step might be to list all available devices:

```
gc_info -l
```

The output should look similar to this:



Example of the output from the 'gc_info -l' command

The **GenICam** (p. 213) Transport Layer assigns an individual DeviceID to all devices, which can be used to address the device.

1.19.11.2.4 Streaming From A Device Start the node with the command:
`rosrun rc_genicam_camera rc_genicam_camera _device:=DeviceID_from_step_before`

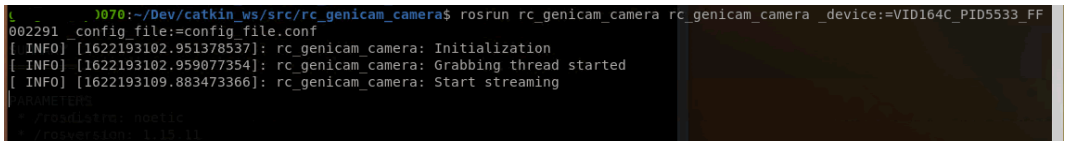
> **Note**
>
> If the *DeviceID* contains one or more colons, it must be preceded by a colon (e.g. :deviceName_MAC:ADDRESS:PART).

With this setup, the node will publish images to the topic */image_raw*. You can now use the tool of your choice to view the image data, for example `RQT`, that comes with the ROS distribution.

1.19.11.2.5 Initial Device Configuration The *rc_genicam_camera* node accepts an optional config file on startup, to launch the device with a defined setup. There is some more information about this file `on the GitHub page of rc_genicam_camera`, but in short it is a list of <property>=<value>, using one line per property. The contents of this file could look like this:

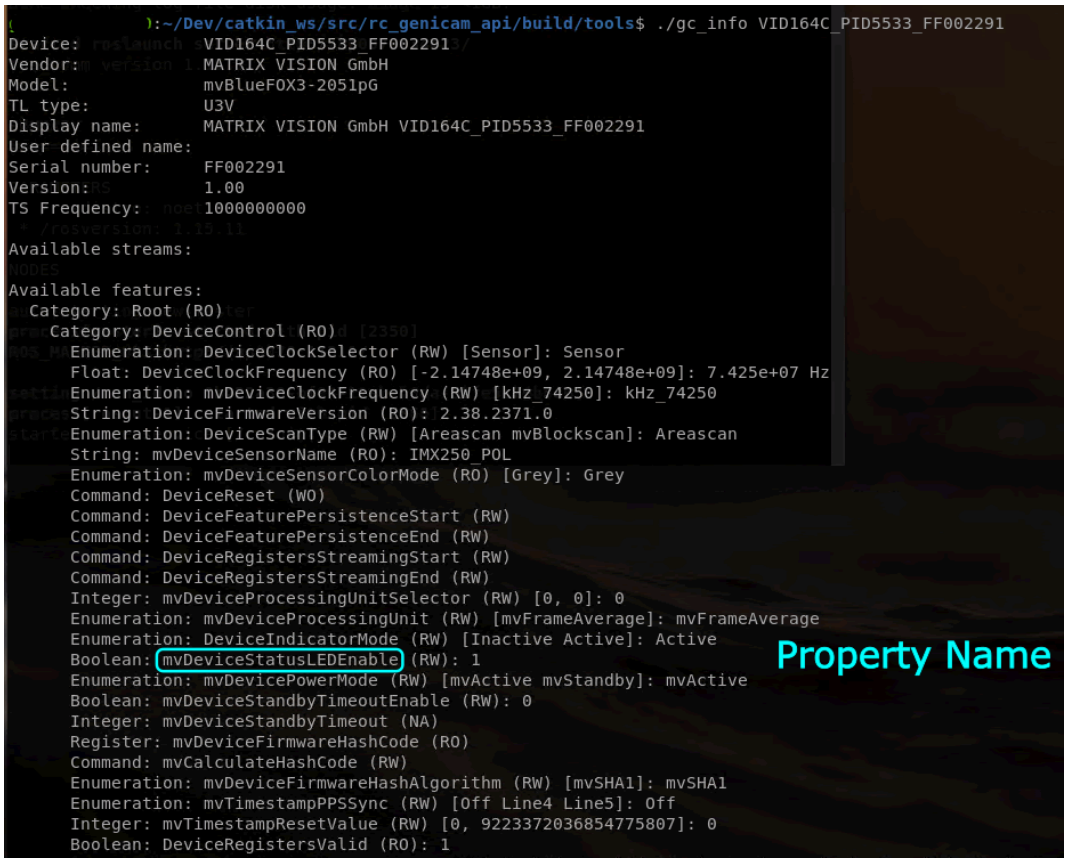
```
# A comment line
Width=720
Height=480
ExposureAuto=Continuous
mvExposureAutoUpperLimit=20000
```

To launch the node with the config file, simply append it to the rosrun call:
`rosrun rc_genicam_camera rc_genicam_camera _device:=DeviceID _config_file:=/path/to/config/file`



Starting the node with DeviceID and config file.

To request a list of all available properties that are accepted by the device, use the *gc_info* tool again. Calling
`gc_info DeviceID`
will return a list, that contains all registered properties:



Listing of available properties from 'gc_info DeviceID'

You may also use **wxPropView** (p. 139) to search for available properties. There, you can find the possible options within the **GenICam** (p. 213) tree. If you are uncertain about valid values for a property, right click on it and choose 'Detailed Feature Information'.

1.19.11.2.6 Device Configuration While Streaming Many properties can be changed while the device is already streaming. This can be accomplished with the *set_genicam_parameter*-service, offered by the *rc_genicam_camera* node. For testing, you may try to use the *rosservice* command. Call *rosservice list* to see all available services.
To set a property, e.g. the *mvExposureAutoUpperLimit*, execute:

```
rosservice call /rc_genicam_camera/set_genicam_parameter 'mvExposureAutoUpperLimit=10000'
```

This will attempt to set the *mvExposureAutoUpperLimit* property to the provided value and return 0 if successful.

Another service, *get_genicam_parameter* is available to read values from properties. E.g. if you want to get the image width, you could execute

```
rosservice call /rc_genicam_camera/get_genicam_parameter 'Width'
```

For a more detailed description of the services, have a look at the `rc_genicam_camera Github page`.

1.19.11.3 Using GigE Vision™ Devices In A Docker Container

When developing machine vision applications using Docker containers, chances are that you would like to access GigE Vision™ devices inside the container. With the mvIMPACT Acquire driver stack this can be achieved fairly easy and this chapter will show you how to build a basic Docker container where you can use GigE Vision™ devices. The current sample Docker container runs on a native Linux machine.

Note

The following chapter is documented only for a native Linux host system.

1.19.11.3.1 Host Preparation

Note

For this demo Docker container the operating system of the host machine is Linux.

The host **network interface card** (p. 219) has to be first configured for an optimized GigE Vision™ transmission: **Checklist for Linux** (p. 57)

1.19.11.3.2 Building A Docker Image The following demo Dockerfile builds a basic Docker image based on a slim version of Debian, where the mvIMPACT Acquire GenTL driver package and its sample programs are installed. This Dockerfile can be used in many ways:

- Use it directly to test your device in a Docker container.
- Use it as a base image for your device applications.
- Use it as an inspiration for building your own Dockerfile.

Before building the Dockerfile, please download the mvIMPACT Acquire GenTL driver installation files from MATRIX VISION GmbH website (`https://www.matrix-vision.com/treiber-software.html`) (user login is required):

- The installation script: *install_mvGenTL_Acquire.sh*
- The installation package: *mvGenTL_Acquire-x86_64_ABI2-*.tgz* (* should be replaced by the version number)

Create a directory called *mvIMPACT_Acquire* (as used in this demo Dockerfile) and move both installation files into this directory. In this example, both files are downloaded into the *Downloads* directory and the *mvIMPACT_Acquire* directory is created inside the *Downloads* directory:

```
$ cd ~/Downloads
$ mkdir mvIMPACT_Acquire
$ mv install_mvGenTL_Acquire.sh mvGenTL_Acquire-x86_64_ABI2-*.tgz mvIMPACT_Acquire/
```

Make the installation script *install_mvGenTL_Acquire.sh* executable:

```
$ cd mvIMPACT_Acquire
$ chmod a+x install_mvGenTL_Acquire.sh
```

Navigate back into the directory where *mvIMPACT_Acquire* resides (e.g. *Downloads*) and create your Dockerfile:

```
$ cd ~/Downloads
$ touch Dockerfile
```

Create the content of your Dockerfile. Our demo Dockerfile (for Linux x86_64) looks as follows:

```
# start with slim version of actual Debian
FROM debian:9-slim

ENV LC_ALL C
ENV DEBIAN_FRONTEND noninteractive

# entrypoint of Docker
CMD ["/bin/bash"]

# set environment variables
ENV TERM linux
ENV MVIMPACT_ACQUIRE_DIR /opt/mvIMPACT_Acquire
ENV MVIMPACT_ACQUIRE_DATA_DIR /opt/mvIMPACT_Acquire/data
ENV GENICAM_GENTL64_PATH /opt/mvIMPACT_Acquire/lib/x86_64
ENV GENICAM_ROOT /opt/mvIMPACT_Acquire/runtime
ENV container docker

# update packets and install minimal requirements
# after installation it will clean apt packet cache
RUN apt-get update && apt-get -y install build-essential && \
    iproute2 && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*

# move the directory mvIMPACT_Acquire with *.tgz and *.sh files to the container
COPY mvIMPACT_Acquire /var/lib/mvIMPACT_Acquire

# execute the setup script in an unattended mode
RUN cd /var/lib/mvIMPACT_Acquire && \
    ./install_mvGenTL_Acquire.sh -u && \
    rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
```

Note

In case of ARM architectures, all occurrences of "x86_64" in this demo dockerfile have to be replaced by the correct platform e.g. "arm64" and the install script to use will be *install_mvGenTL_Acquire_ARM.sh* then.

Finally, build a Docker image using this Dockerfile:

```
$ sudo docker build -t [image_name] .
```

Note

Please make sure to call *docker build* from within the directory where the Dockerfile resides. Note that Internet access is required for the *docker build*.

If built successfully, you will be able to see [image_name] being listed when calling:

```
$ sudo docker images
```

1.19.11.3.3 Starting The Docker Container Though the Docker container is isolated from the host system, we can connect the container to the host network by using *–net=host* flag when starting the container:

```
$ sudo docker run -ti --net=host [image_name] /bin/bash
```

1.19.11.3.4 Validation After starting the container the correct operation of GigE Vision™ devices can be validated by running one of the sample programs provided by the mvIMPACT Acquire (e.g. *SingleCapture*):

```
$ cd /opt/mvIMPACT_Acquire/apps/SingleCapture/x86_64
$ ./SingleCapture
```

If the attached GigE Vision™ device appears in the device list of the program's output, congratulations, you've managed to access GigE Vision™ devices in the container by using the mvIMPACT Acquire. Now you can use them inside the Docker container for your machine vision applications.

1.20 Appendix A. Specific Camera / Sensor Data

- **A.1 Pregius CMOS** (p. 442)
- **A.2 Starvis CMOS** (p. 526)
- **A.3 Polarsens CMOS** (p. 537)
- **A.4 CMOS** (p. 541)
- **A.5 CCD** (p. 592)

1.20.1 A.1 Pregius CMOS

- **Pregius S** (p. 442)
- **Pregius** (p. 467)

1.20.1.1 Pregius S

- **mvBlueCOUGAR-X[D]105d (5.1 Mpix [2472 x 2064])** (p. 442)
- **mvBlueCOUGAR-X[D]108a (8.1 Mpix [2856 x 2848])** (p. 446)
- **mvBlueCOUGAR-X108u (8.1 Mpix [2848 x 2848])** (p. 449)
- **mvBlueCOUGAR-X[D]1012d (12.4 Mpix [4128 x 3008])** (p. 452)
- **mvBlueCOUGAR-XD1016 (16.2 Mpix [5328 x 3040])** (p. 456)
- **mvBlueCOUGAR-XD1020a (20.4 Mpix [4512 x 4512])** (p. 460)
- **mvBlueCOUGAR-XD1025 (24.6 Mpix [5328 x 4608])** (p. 463)

1.20.1.1.1 mvBlueCOUGAR-X[D]105d (5.1 Mpix [2472 x 2064])

1.20.1.1.1.1 Introduction

The CMOS sensor module (IMX547) incorporates the following features:

- resolution to 2472 x 2064 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.1.1.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.1.1.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	859 (mvBlueCOUGAR-X) / 450 (mvBlueCOUGAR-XD)
VerticalBlankLines	139
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4 (mvBlueCOUGAR-X) / 8 (mvBlueCOUGAR-XD)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 11.57 us (mvBlueCOUGAR-X) or 6.06 us (mvBlueCOUGAR-XD) and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

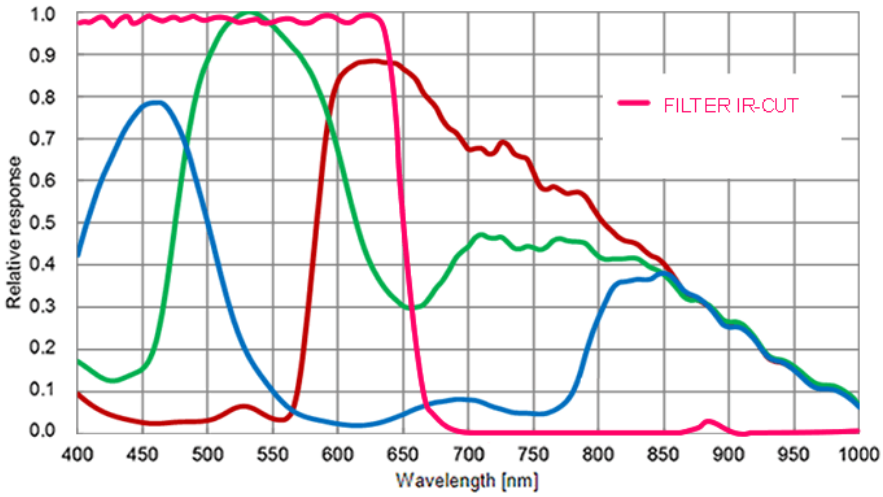
1.20.1.1.1.4 Sensor Data

Device Structure

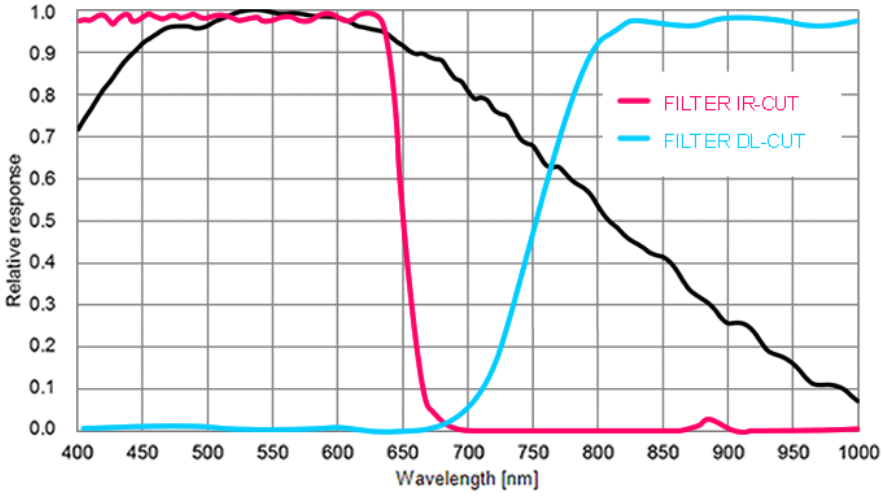
- CMOS image sensor (Type 1/1.8")
- Number of effective pixels: 2472 (H) x 2064 (V)
- Unit cell size: 2.74um (H) x 2.74um (V)

1.20.1.1.1.5 Characteristics

1.20.1.1.1.6 Color version



1.20.1.1.1.7 Gray scale version



Device Feature And Property List (p. 445)

1.20.1.1.1.8 Device Feature And Property List

- **mvBlueCOUGAR-X105dG Features** (p. 446)
- **mvBlueCOUGAR-X105dC Features** (p. 446)
- **mvBlueCOUGAR-XD105dG Features** (p. 446)
- **mvBlueCOUGAR-XD105dC Features** (p. 446)

1.20.1.1.1.9 mvBlueCOUGAR-X105dG Features

1.20.1.1.1.10 mvBlueCOUGAR-X105dC Features

1.20.1.1.1.11 mvBlueCOUGAR-XD105dG Features

1.20.1.1.1.12 mvBlueCOUGAR-XD105dC Features

1.20.1.1.2 mvBlueCOUGAR-X[D]108a (8.1 Mpix [2856 x 2848])

1.20.1.1.2.1 Introduction

The CMOS sensor module (IMX546) incorporates the following features:

- resolution to 2856 x 2848 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.1.2.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.1.2.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	979 (mvBlueCOUGAR-X) / 510 (mvBlueCOUGAR-XD)
VerticalBlankLines	139
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4 (mvBlueCOUGAR-X) / 8 (mvBlueCOUGAR-XD)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 13.19 us (mvBlueCOUGAR-X) or 6.87 us (mvBlueCOUGAR-XD) and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = LevelLow"` `"ExposureMode = Timed"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = LevelHigh"` `"ExposureMode = Timed"`	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = FallingEdge"` `"ExposureMode = Timed"`	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

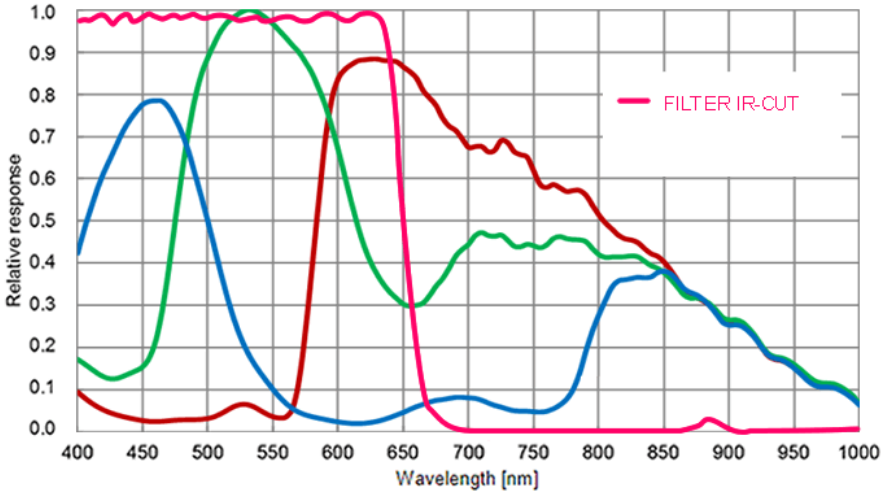
1.20.1.1.2.4 Sensor Data

Device Structure

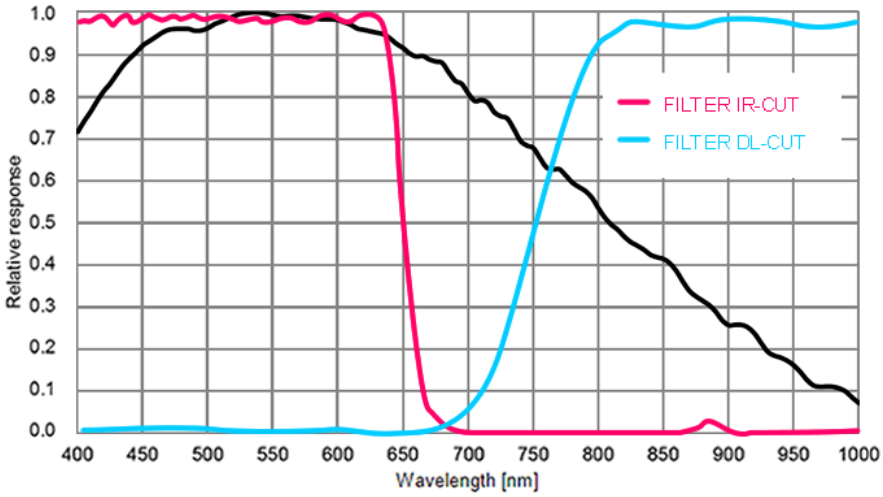
- CMOS image sensor (Type 2/3")
- Number of effective pixels: 2856 (H) x 2848 (V)
- Unit cell size: 2.74um (H) x 2.74um (V)

1.20.1.1.2.5 Characteristics

1.20.1.1.2.6 Color version



1.20.1.1.2.7 Gray scale version



Device Feature And Property List (p. 449)

1.20.1.1.2.8 Device Feature And Property List

- **mvBlueCOUGAR-X108aG Features** (p. 449)
- **mvBlueCOUGAR-X108aC Features** (p. 449)
- **mvBlueCOUGAR-XD108aG Features** (p. 449)
- **mvBlueCOUGAR-XD108aC Features** (p. 449)

1.20.1.1.2.9 mvBlueCOUGAR-X108aG Features

1.20.1.1.2.10 mvBlueCOUGAR-X108aC Features

1.20.1.1.2.11 mvBlueCOUGAR-XD108aG Features

1.20.1.1.2.12 mvBlueCOUGAR-XD108aC Features

1.20.1.1.3 mvBlueCOUGAR-X108u (8.1 Mpix [2848 x 2848])

1.20.1.1.3.1 Introduction

The CMOS sensor module (IMX487) incorporates the following features:

- resolution to 2848 x 2848 gray scale
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.1.3.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.1.3.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	979
VerticalBlankLines	148
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 13.19 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

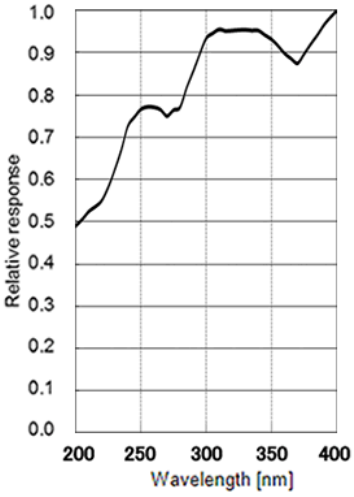
1.20.1.1.3.4 Sensor Data

Device Structure

- CMOS image sensor (Type 2/3")
- Number of effective pixels: 2848 (H) x 2848 (V)
- Unit cell size: 2.74um (H) x 2.74um (V)

1.20.1.1.3.5 Characteristics

1.20.1.1.3.6 Gray scale version



Device Feature And Property List (p. 452)

1.20.1.1.3.7 Device Feature And Property List

- **mvBlueCOUGAR-X108uG Features** (p. 452)

1.20.1.1.3.8 mvBlueCOUGAR-X108uG Features

1.20.1.1.4 mvBlueCOUGAR-X[D]1012d (12.4 Mpix [4128 x 3008])

1.20.1.1.4.1 Introduction

The CMOS sensor module (IMX545) incorporates the following features:

- resolution to 4128 x 3008 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.1.4.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.1.4.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	1366 (mvBlueCOUGAR-X) / 704 (mvBlueCOUGAR-XD)
VerticalBlankLines	139
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4 (mvBlueCOUGAR-X) / 8 (mvBlueCOUGAR-XD)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 18.4 us (mvBlueCOUGAR-X) or 9.49 us (mvBlueCOUGAR-XD) and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

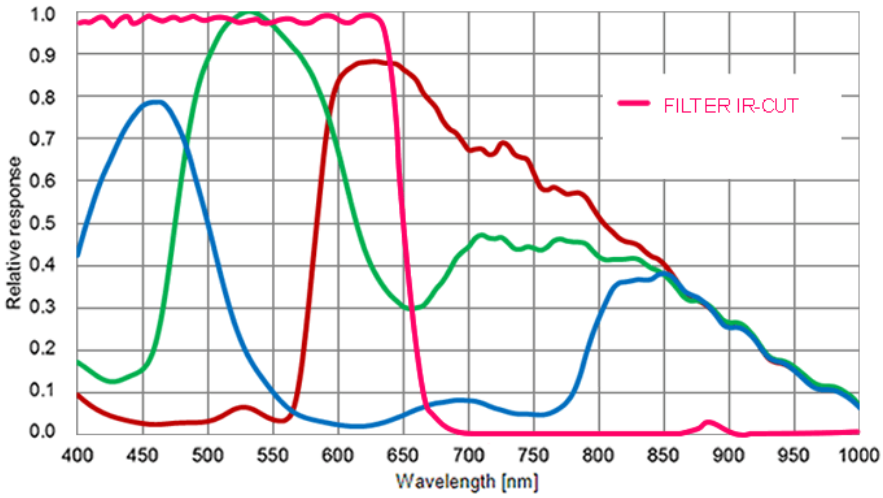
1.20.1.1.4.4 Sensor Data

Device Structure

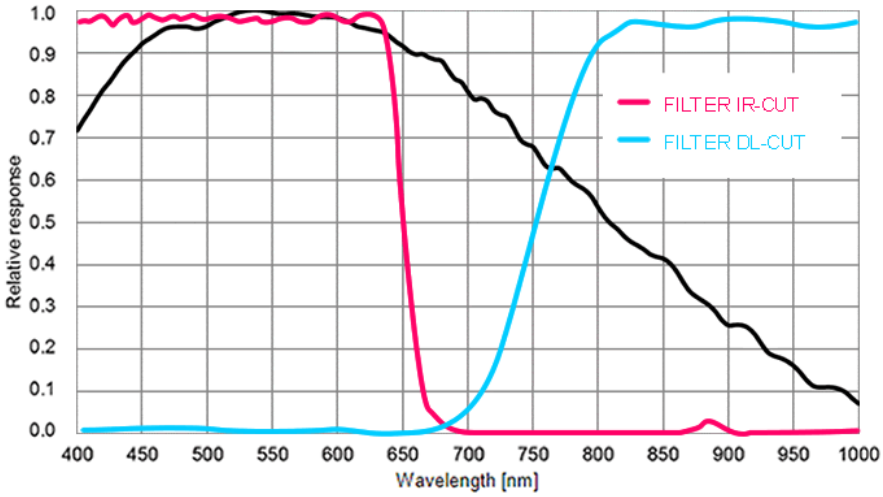
- CMOS image sensor (Type 1/1.1")
- Number of effective pixels: 4128 (H) x 3008 (V)
- Unit cell size: 2.74um (H) x 2.74um (V)

1.20.1.1.4.5 Characteristics

1.20.1.1.4.6 Color version



1.20.1.1.4.7 Gray scale version



Device Feature And Property List (p. 456)

1.20.1.1.4.8 Device Feature And Property List

- **mvBlueCOUGAR-X1012dG Features** (p. 456)
- **mvBlueCOUGAR-X1012dC Features** (p. 456)
- **mvBlueCOUGAR-XD1012dG Features** (p. 456)
- **mvBlueCOUGAR-XD1012dC Features** (p. 456)

1.20.1.1.4.9 mvBlueCOUGAR-X1012dG Features

1.20.1.1.4.10 mvBlueCOUGAR-X1012dC Features

1.20.1.1.4.11 mvBlueCOUGAR-XD1012dG Features

1.20.1.1.4.12 mvBlueCOUGAR-XD1012dC Features

1.20.1.1.5 mvBlueCOUGAR-XD1016 (16.2 Mpix [5328 x 3040])

1.20.1.1.5.1 Introduction

The CMOS sensor module (IMX542) incorporates the following features:

- resolution to 5328 x 3040 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.1.5.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.1.5.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	649 (10 bit) / 776 (12 bit)
VerticalBlankLines	129
SensorInClock	53.977 (@54 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 12.02 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = AnyEdge"`
`"ExposureMode = Timed"`
	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

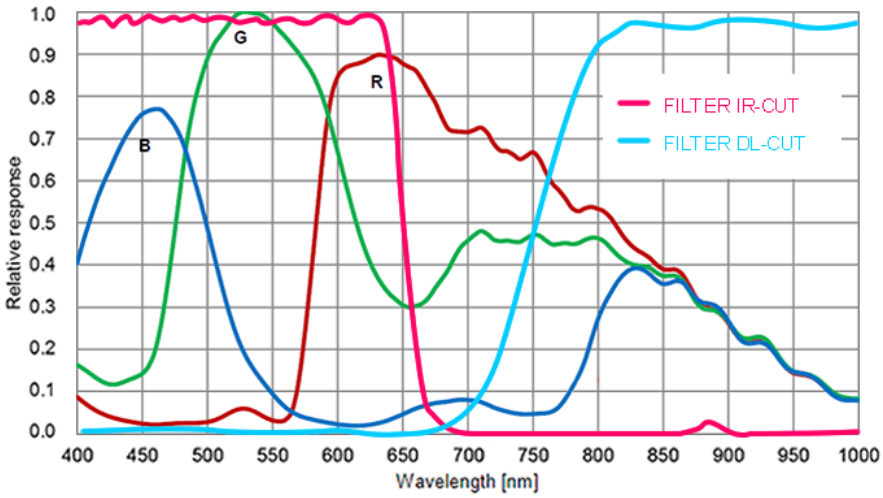
1.20.1.1.5.4 Sensor Data

Device Structure

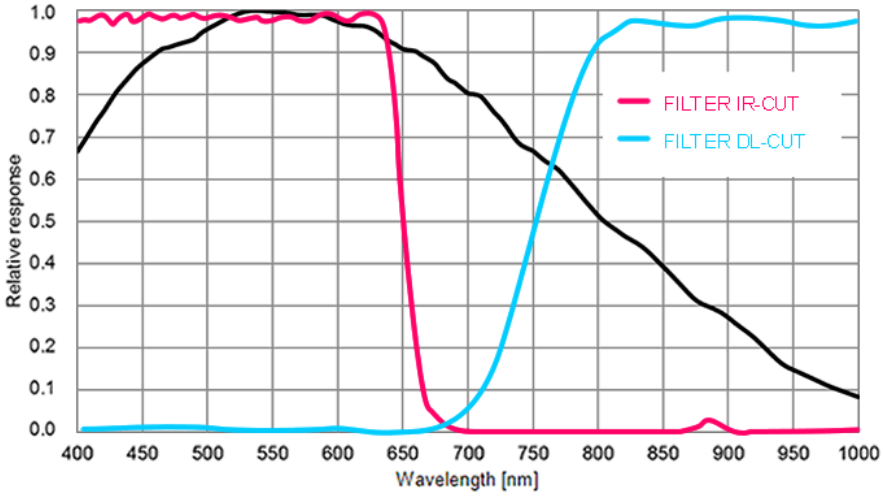
- CMOS image sensor (Type 1.1)
- Number of effective pixels: 5328 (H) x 3040 (V)
- Unit cell size: 2.74um (H) x 2.74um (V)

1.20.1.1.5.5 Characteristics

1.20.1.1.5.6 Color version



1.20.1.1.5.7 Gray scale version



Device Feature And Property List (p. 460)

1.20.1.1.5.8 Device Feature And Property List

- **mvBlueCOUGAR-XD1016G Features** (p. 460)
- **mvBlueCOUGAR-XD1016C Features** (p. 460)

1.20.1.1.5.9 mvBlueCOUGAR-XD1016G Features

1.20.1.1.5.10 mvBlueCOUGAR-XD1016C Features

1.20.1.1.6 mvBlueCOUGAR-XD1020a (20.4 Mpix [4512 x 4512])

1.20.1.1.6.1 Introduction

The CMOS sensor module (IMX541) incorporates the following features:

- resolution to 4512 x 4512 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.1.6.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.1.6.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	556 (10 bit) / 665 (12 bit)
VerticalBlankLines	133
SensorInClock	53.977 (@54 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 10.30 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

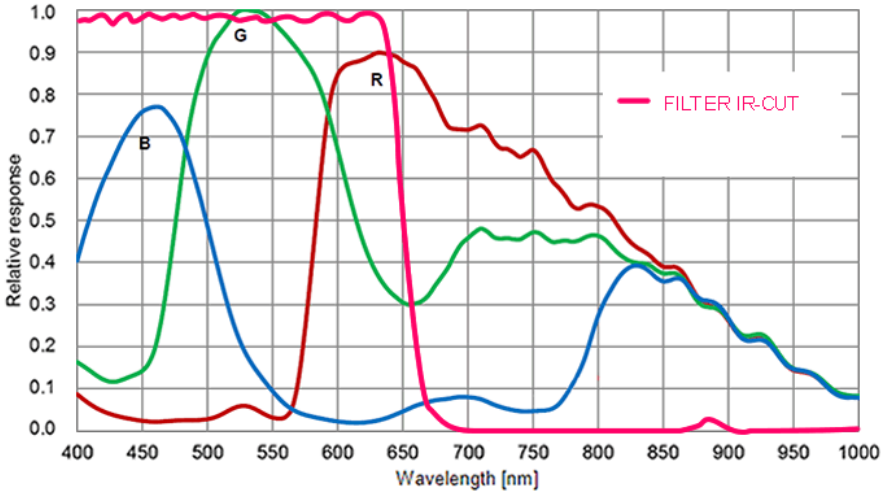
1.20.1.1.6.4 Sensor Data

Device Structure

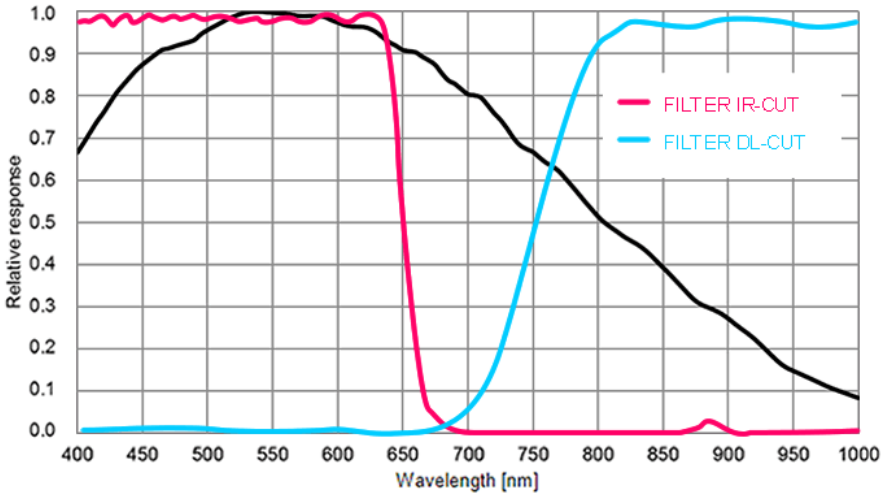
- CMOS image sensor (Type 1.1)
- Number of effective pixels: 4512 (H) x 4512 (V)
- Unit cell size: 2.74um (H) x 2.74um (V)

1.20.1.1.6.5 Characteristics

1.20.1.1.6.6 Color version



1.20.1.1.6.7 Gray scale version



Device Feature And Property List (p. 463)

1.20.1.1.6.8 Device Feature And Property List

- **mvBlueCOUGAR-XD1020aG Features** (p. 463)
- **mvBlueCOUGAR-XD1020aC Features** (p. 463)

1.20.1.1.6.9 mvBlueCOUGAR-XD1020aG Features

1.20.1.1.6.10 mvBlueCOUGAR-XD1020aC Features

1.20.1.1.7 mvBlueCOUGAR-XD1025 (24.6 Mpix [5328 x 4608])

1.20.1.1.7.1 Introduction

The CMOS sensor module (IMX540) incorporates the following features:

- resolution to 5328 x 4608 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.1.7.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.1.7.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	649 (10 bit) / 776 (12 bit)
VerticalBlankLines	129
SensorInClock	53.977 (@54 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 12.02 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
`"TriggerActivation = LevelLow"` `"ExposureMode = Timed"`		

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

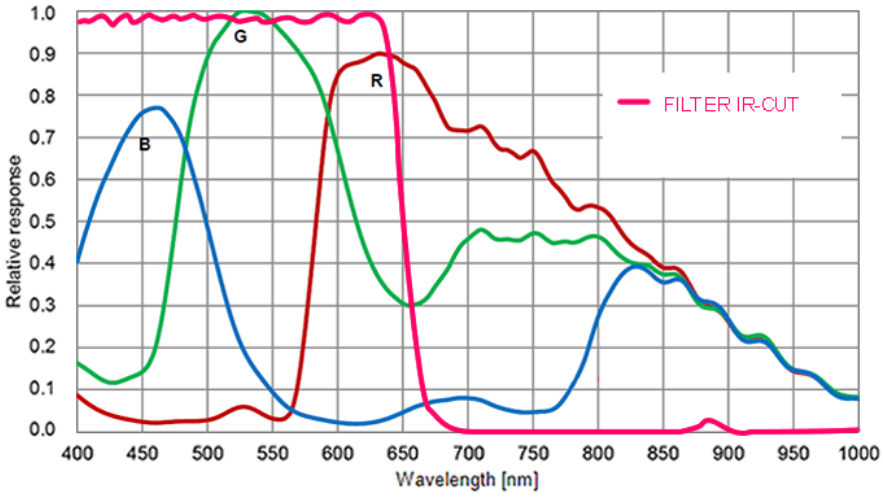
1.20.1.1.7.4 Sensor Data

Device Structure

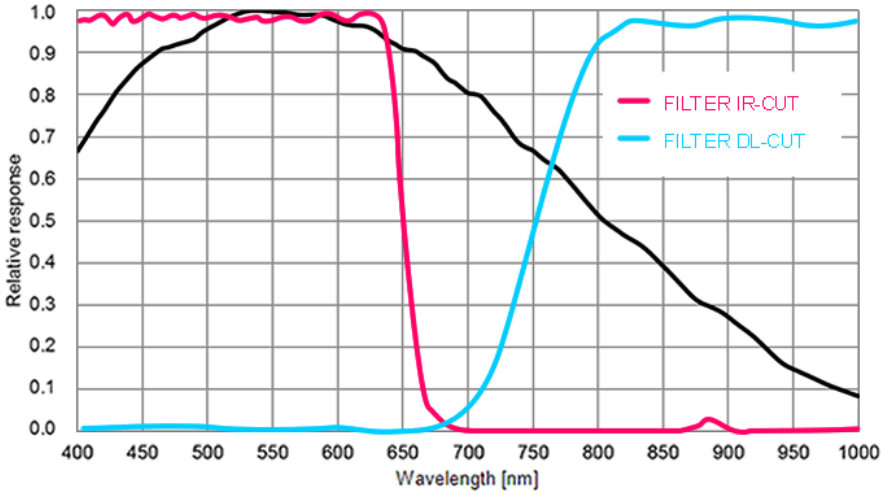
- CMOS image sensor (Type 1.2)
- Number of effective pixels: 5328 (H) x 4608 (V)
- Unit cell size: 2.74um (H) x 2.74um (V)

1.20.1.1.7.5 Characteristics

1.20.1.1.7.6 Color version



1.20.1.1.7.7 Gray scale version



Device Feature And Property List (p. 466)

1.20.1.1.7.8 Device Feature And Property List

- **mvBlueCOUGAR-XD1025G Features** (p. 466)
- **mvBlueCOUGAR-XD1025C Features** (p. 466)

1.20.1.1.7.9 mvBlueCOUGAR-XD1025G Features

1.20.1.1.7.10 mvBlueCOUGAR-XD1025C Features

1.20.1.2 Pregius

- **mvBlueCOUGAR-X100f (0.4 Mpix [728 x 544])** (p. 468)
- **mvBlueCOUGAR-X[D]102f (1.6 Mpix [1456 x 1088])** (p. 471)
- **mvBlueCOUGAR-X102k (1.8 Mpix [1600 x 1104])** (p. 475)
- **mvBlueCOUGAR-X102m (1.8 Mpix [1600 x 1104])** (p. 478)
- **mvBlueCOUGAR-X102n (2 Mpix [1632 x 1248])** (p. 482)
- **mvBlueCOUGAR-XD104d (2.4 Mpix [1936 x 1216])** (p. 485)
- **mvBlueCOUGAR-X104f (2.4 Mpix [1936 x 1216])** (p. 490)
- **mvBlueCOUGAR-XD104h (3.2 Mpix [2064 x 1544])** (p. 493)
- **mvBlueCOUGAR-X104i (3.2 Mpix [2064 x 1544])** (p. 496)
- **mvBlueCOUGAR-XD105a (5.1 Mpix [2464 x 2056])** (p. 500)
- **mvBlueCOUGAR-X105b (5.1 Mpix [2464 x 2056])** (p. 503)
- **mvBlueCOUGAR-XD107 (7.1 Mpix [3216 x 2208])** (p. 507)
- **mvBlueCOUGAR-X[D]107b (7.1 Mpix [3216 x 2208])** (p. 511)
- **mvBlueCOUGAR-X[D]109b (8.9 Mpix [4112 x 2176])** (p. 515)
- **mvBlueCOUGAR-X[D]1012b (12.4 Mpix [4112 x 3008])** (p. 519)
- **mvBlueCOUGAR-XD1031 (31.5 Mpix [6480 x 4856])** (p. 523)

1.20.1.2.1 mvBlueCOUGAR-X100f (0.4 Mpix [728 x 544])

1.20.1.2.1.1 Introduction

The CMOS sensor module (IMX287) incorporates the following features:

- resolution to 728 x 544 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.1.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.1.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	290 (10 bit) / 396 (12 bit)
VerticalBlankLines	42
SensorInClock	74.25 (@50 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 3.9 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

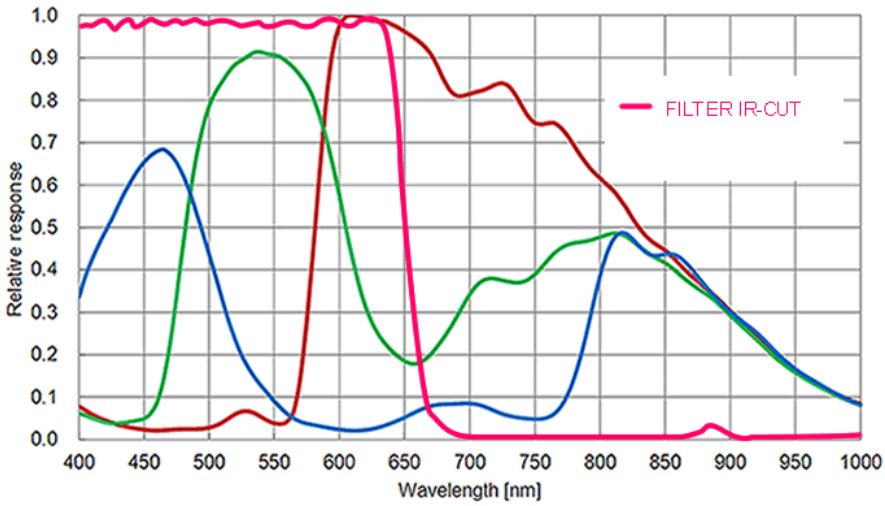
1.20.1.2.1.4 Sensor Data

Device Structure

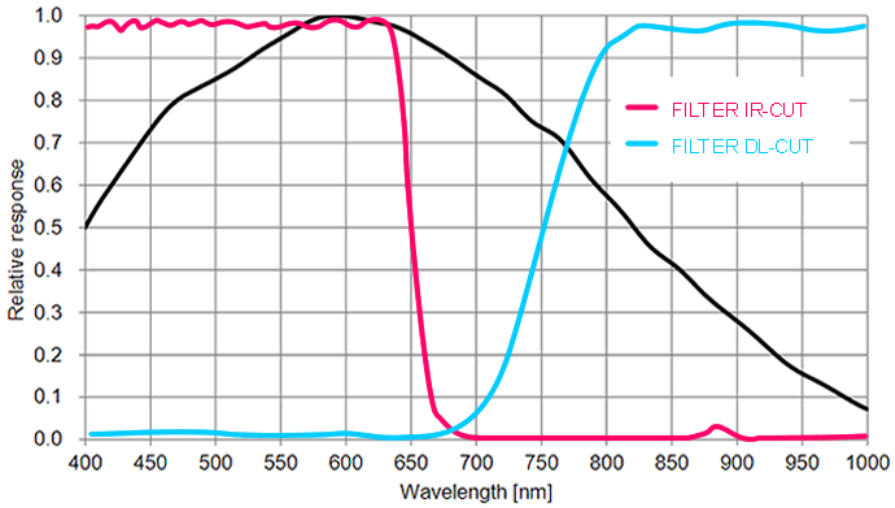
- CMOS image sensor (Type 1/2.9")
- Number of effective pixels: 728 (H) x 544 (V)
- Unit cell size: 6.9um (H) x 6.9um (V)

1.20.1.2.1.5 Characteristics

1.20.1.2.1.6 Color version



1.20.1.2.1.7 Gray scale version



Device Feature And Property List (p. 471)

1.20.1.2.1.8 Device Feature And Property List

- **mvBlueCOUGAR-X100fG Features** (p. 471)
- **mvBlueCOUGAR-X100fC Features** (p. 471)

1.20.1.2.1.9 mvBlueCOUGAR-X100fG Features

1.20.1.2.1.10 mvBlueCOUGAR-X100fC Features

1.20.1.2.2 mvBlueCOUGAR-X[D]102f (1.6 Mpix [1456 x 1088])

1.20.1.2.2.1 Introduction

The CMOS sensor module (IMX273) incorporates the following features:

- resolution to 1456 x 1088 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.2.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.2.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	520 (mvBlueCOUGAR-X) / 290 (mvBlueCOUGAR-XD)
VerticalBlankLines	42
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4 (mvBlueCOUGAR-X) / 8 (mvBlueCOUGAR-XD)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 7.01 us (mvBlueCOUGAR-X) or 3.91 us (mvBlueCOUGAR-XD) and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

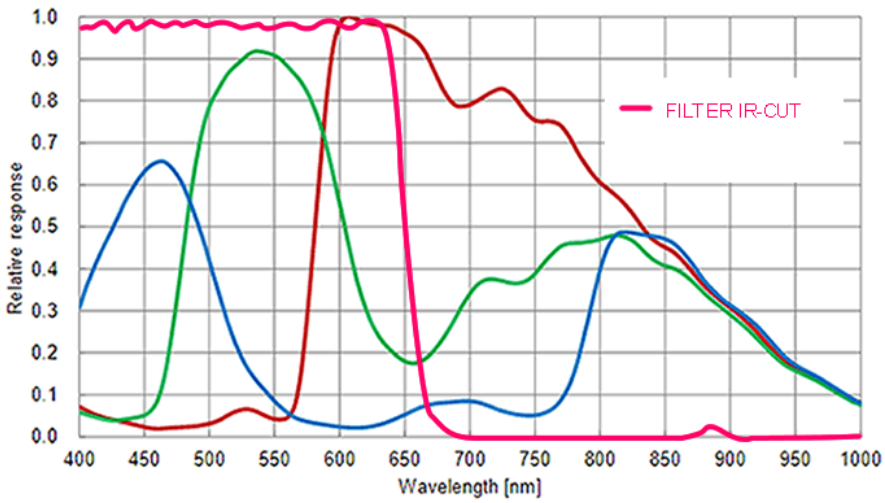
1.20.1.2.2.4 Sensor Data

Device Structure

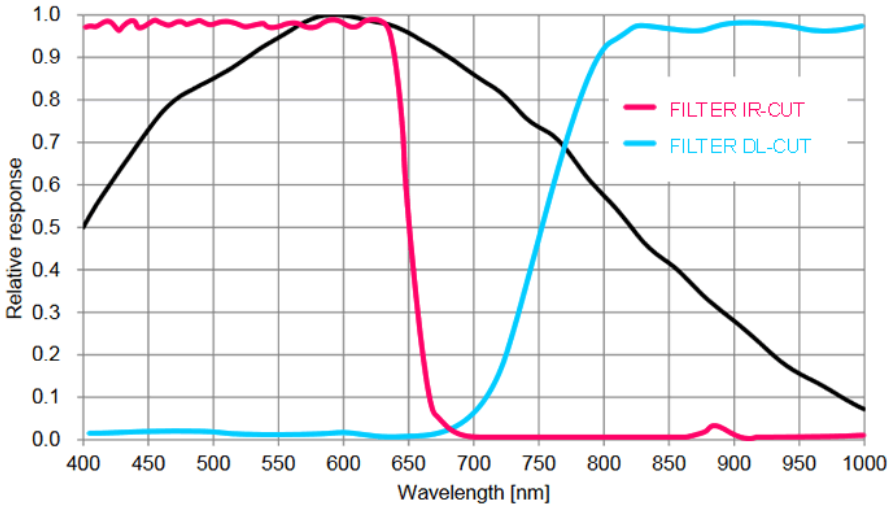
- CMOS image sensor (Type 1/2.9")
- Number of effective pixels: 1456 (H) x 1088 (V)
- Unit cell size: 3.45um (H) x 3.45um (V)

1.20.1.2.5 Characteristics

1.20.1.2.6 Color version



1.20.1.2.7 Gray scale version



Device Feature And Property List (p. 474)

1.20.1.2.8 Device Feature And Property List

- **mvBlueCOUGAR-X102fG Features** (p. 475)
- **mvBlueCOUGAR-X102fC Features** (p. 475)
- **mvBlueCOUGAR-XD102fG Features** (p. 475)
- **mvBlueCOUGAR-XD102fC Features** (p. 475)

1.20.1.2.2.9 mvBlueCOUGAR-X102fG Features

1.20.1.2.2.10 mvBlueCOUGAR-X102fC Features

1.20.1.2.2.11 mvBlueCOUGAR-XD102fG Features

1.20.1.2.2.12 mvBlueCOUGAR-XD102fC Features

1.20.1.2.3 mvBlueCOUGAR-X102k (1.8 Mpix [1600 x 1104])

1.20.1.2.3.1 Introduction

The CMOS sensor module (IMX432) incorporates the following features:

- resolution to 1600 x 1104 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.3.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.3.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	466
VerticalBlankLines	74
SensorInClock	53.977 (@54 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 8.63 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

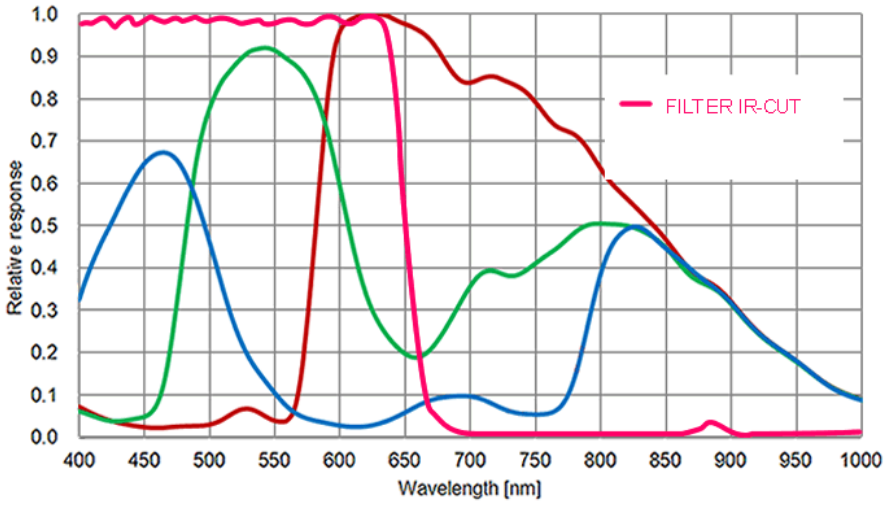
1.20.1.2.3.4 Sensor Data

Device Structure

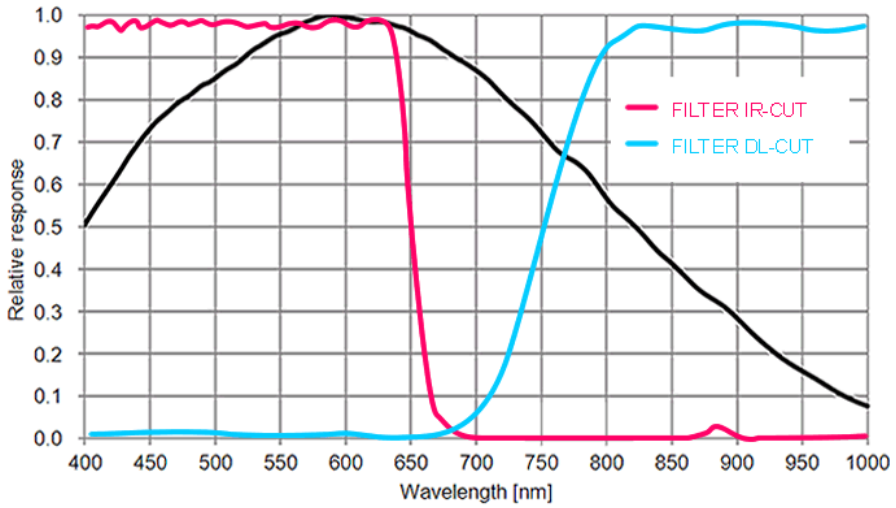
- CMOS image sensor (Type 1.1")
- Number of effective pixels: 1600 (H) x 1104 (V)
- Unit cell size: 9.0um (H) x 9.0um (V)

1.20.1.2.3.5 Characteristics

1.20.1.2.3.6 Color version



1.20.1.2.3.7 Gray scale version



Device Feature And Property List (p. 478)

1.20.1.2.3.8 Device Feature And Property List

- **mvBlueCOUGAR-X102kG Features** (p. 478)
- **mvBlueCOUGAR-X102kC Features** (p. 478)

1.20.1.2.3.9 mvBlueCOUGAR-X102kG Features

1.20.1.2.3.10 mvBlueCOUGAR-X102kC Features

1.20.1.2.4 mvBlueCOUGAR-X102m (1.8 Mpix [1600 x 1104])

1.20.1.2.4.1 Introduction

The CMOS sensor module (IMX425) incorporates the following features:

- resolution to 1600 x 1104 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.4.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.4.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	390 (10 bit) / 466 (12 bit)
VerticalBlankLines	98
SensorInClock	53.977 (@54 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 7.23 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
`"TriggerActivation = LevelLow"` `"ExposureMode = Timed"`		

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

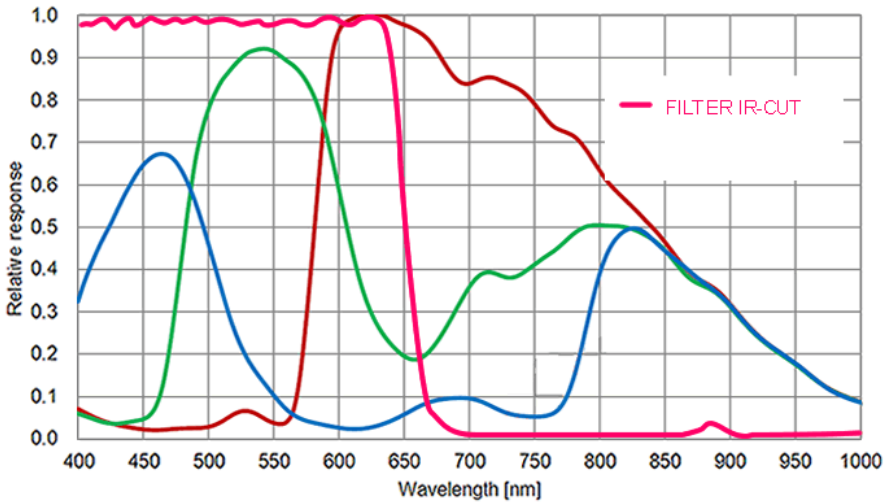
1.20.1.2.4.4 Sensor Data

Device Structure

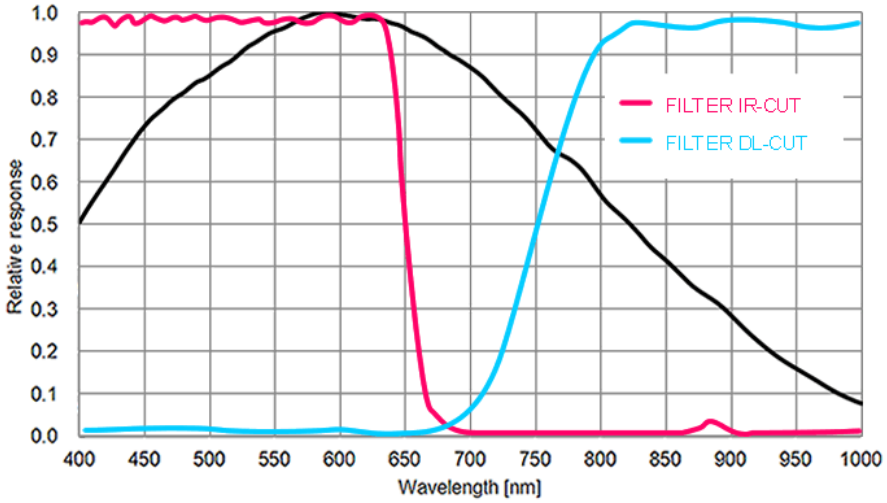
- CMOS image sensor (Type 1.1")
- Number of effective pixels: 1600 (H) x 1104 (V)
- Unit cell size: 9.0um (H) x 9.0um (V)

1.20.1.2.4.5 Characteristics

1.20.1.2.4.6 Color version



1.20.1.2.4.7 Gray scale version



Device Feature And Property List (p. 481)

1.20.1.2.4.8 Device Feature And Property List

- **mvBlueCOUGAR-X102mG Features** (p. 481)
- **mvBlueCOUGAR-X102mC Features** (p. 481)

1.20.1.2.4.9 mvBlueCOUGAR-X102mG Features

1.20.1.2.4.10 mvBlueCOUGAR-X102mC Features

1.20.1.2.5 mvBlueCOUGAR-X102n (2 Mpix [1632 x 1248])

1.20.1.2.5.1 Introduction

The CMOS sensor module (IMX430) incorporates the following features:

- resolution to 1632 x 1248 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.5.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.5.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	472
VerticalBlankLines	55
SensorInClock	53.977 (@54 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 8.74 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

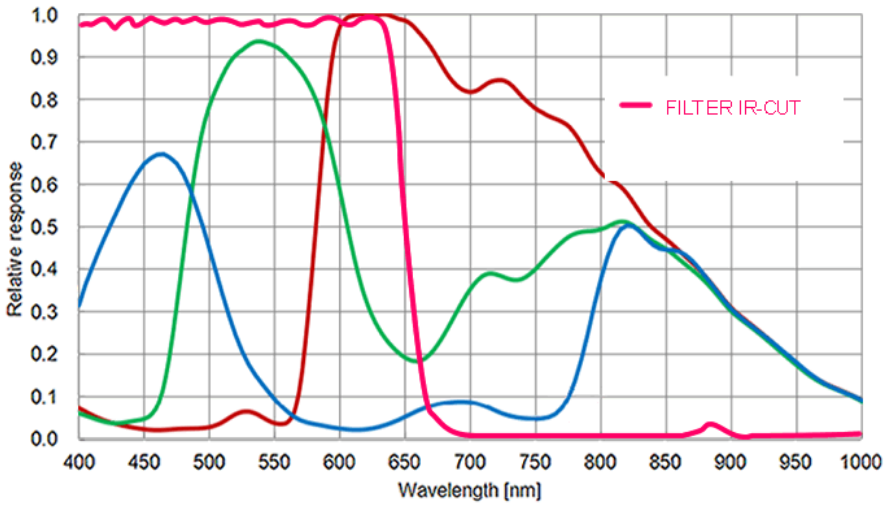
1.20.1.2.5.4 Sensor Data

Device Structure

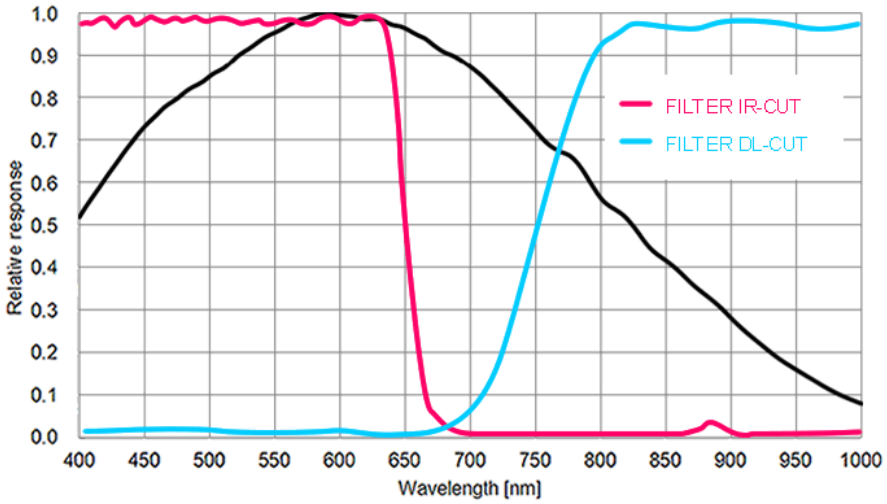
- CMOS image sensor (Type 1/1.7")
- Number of effective pixels: 1632 (H) x 1248 (V)
- Unit cell size: 4.5um (H) x 4.5um (V)

1.20.1.2.5.5 Characteristics

1.20.1.2.5.6 Color version



1.20.1.2.5.7 Gray scale version



Device Feature And Property List (p. 485)

1.20.1.2.5.8 Device Feature And Property List

- **mvBlueCOUGAR-X102nG Features** (p. 485)
- **mvBlueCOUGAR-X102nC Features** (p. 485)

1.20.1.2.5.9 mvBlueCOUGAR-X102nG Features

1.20.1.2.5.10 mvBlueCOUGAR-X102nC Features

1.20.1.2.6 mvBlueCOUGAR-XD104d (2.4 Mpix [1936 x 1216])

1.20.1.2.6.1 Introduction

The CMOS sensor module (IMX174) incorporates the following features:

- resolution to 1936 x 1216 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- **high dynamic range** (p. 378)
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.6.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.6.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	360 (10 bit) / 462 (12 bit)
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 4.86 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
`"TriggerActivation = LevelLow"` `"ExposureMode = Timed"`		

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

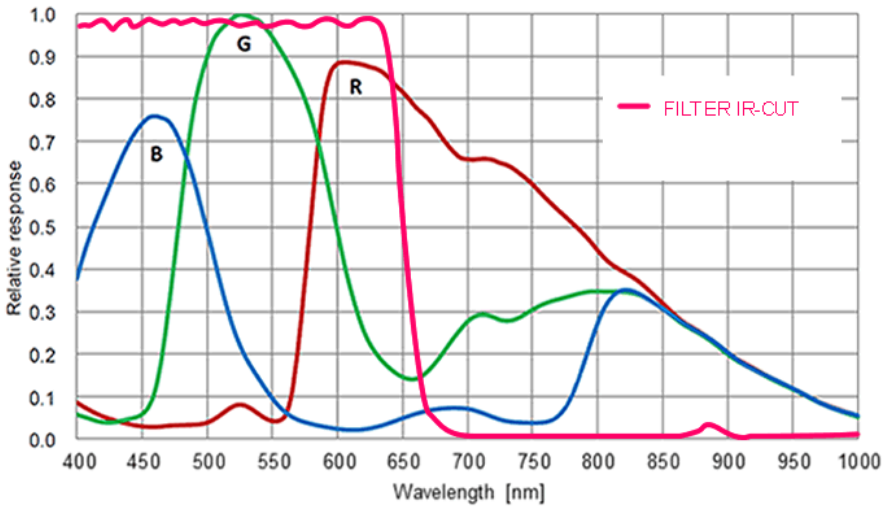
1.20.1.2.6.4 Sensor Data

Device Structure

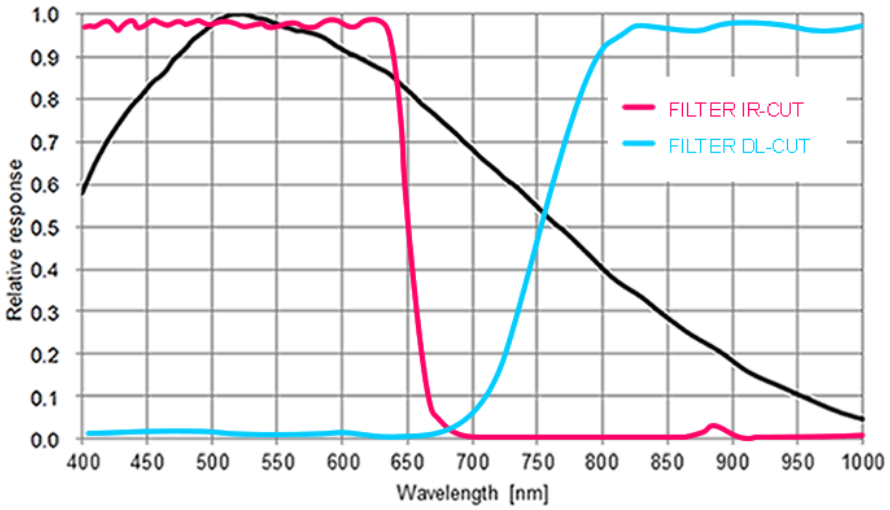
- CMOS image sensor (Type 1/1.2")
- Number of effective pixels: 1936 (H) x 1216 (V)
- Unit cell size: 5.86um (H) x 5.86um (V)

1.20.1.2.6.5 Characteristics

1.20.1.2.6.6 Color version



1.20.1.2.6.7 Gray scale version



Device Feature And Property List (p. 489)

1.20.1.2.6.8 Device Feature And Property List

- **mvBlueCOUGAR-XD104dG Features** (p. 489)
- **mvBlueCOUGAR-XD104dC Features** (p. 489)

1.20.1.2.6.9 mvBlueCOUGAR-XD104dG Features

1.20.1.2.6.10 mvBlueCOUGAR-XD104dC Features

1.20.1.2.7 mvBlueCOUGAR-X104f (2.4 Mpix [1936 x 1216])

1.20.1.2.7.1 Introduction

The CMOS sensor module (IMX249) incorporates the following features:

- resolution to 1936 x 1216 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.7.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.7.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	360 (10 bit) / 462 (12 bit)
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	2

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 19.4 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = FallingEdge"` `"ExposureMode = Timed"`	OnFallingEdge	Each falling edge of trigger signal acquires one image.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = RisingEdge"` `"ExposureMode = Timed"`	OnRisingEdge	Each rising edge of trigger signal acquires one image.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = AnyEdge"` `"ExposureMode = Timed"`	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

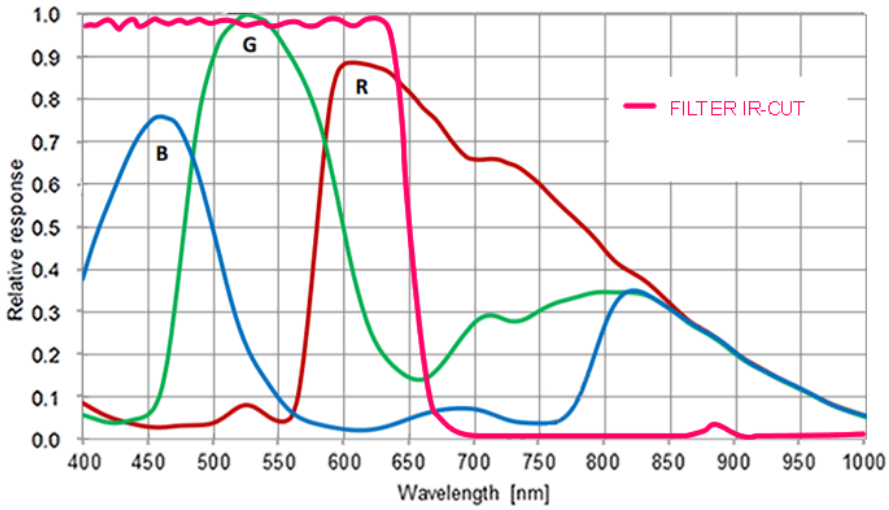
1.20.1.2.7.4 Sensor Data

Device Structure

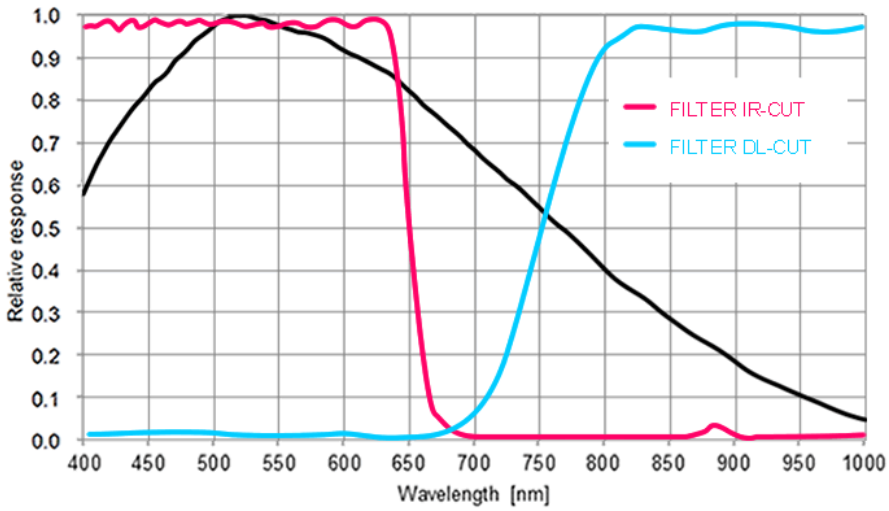
- CMOS image sensor (Type 1/1.2")
- Number of effective pixels: 1936 (H) x 1216 (V)
- Unit cell size: 5.86um (H) x 5.86um (V)

1.20.1.2.7.5 Characteristics

1.20.1.2.7.6 Color version



1.20.1.2.7.7 Gray scale version



Device Feature And Property List (p. 492)

1.20.1.2.7.8 Device Feature And Property List

- **mvBlueCOUGAR-X104fG Features** (p. 493)
- **mvBlueCOUGAR-X104fC Features** (p. 493)

1.20.1.2.7.9 mvBlueCOUGAR-X104fG Features

1.20.1.2.7.10 mvBlueCOUGAR-X104fC Features

1.20.1.2.8 mvBlueCOUGAR-XD104h (3.2 Mpix [2064 x 1544])

1.20.1.2.8.1 Introduction

The CMOS sensor module (IMX252) incorporates the following features:

- resolution to 2064 x 1544 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.8.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.8.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	380 (10 bit) / 444 (12 bit)
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 5.13 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

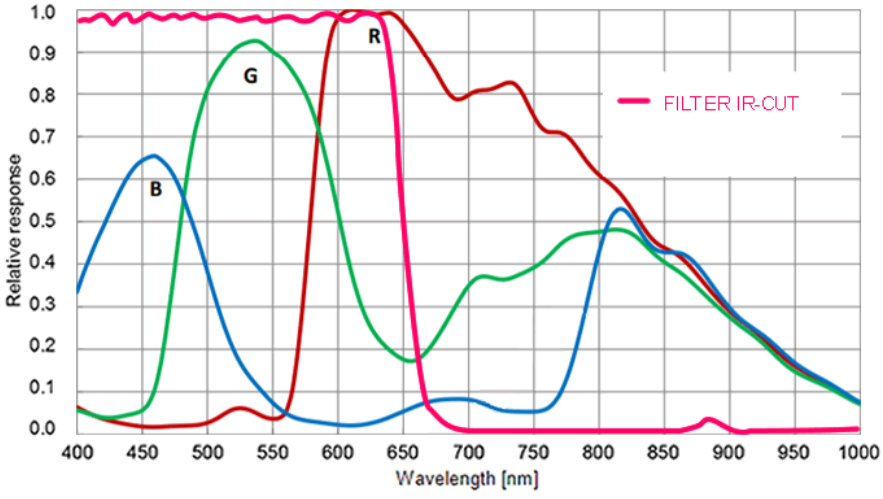
1.20.1.2.8.4 Sensor Data

Device Structure

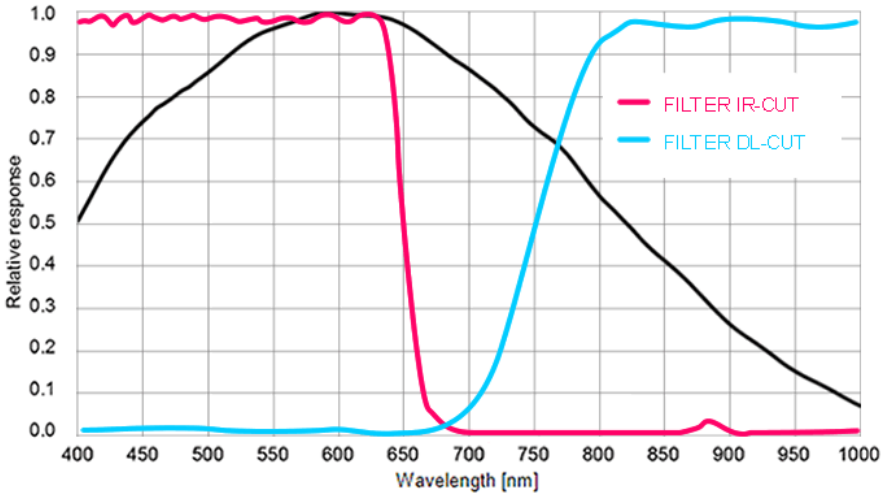
- CMOS image sensor (Type 1/1.8")
- Number of effective pixels: 2064 (H) x 1544 (V)
- Unit cell size: 3.45um (H) x 3.45um (V)

1.20.1.2.8.5 Characteristics

1.20.1.2.8.6 Color version



1.20.1.2.8.7 Gray scale version



Device Feature And Property List (p. 496)

1.20.1.2.8.8 Device Feature And Property List

- **mvBlueCOUGAR-XD104hG Features** (p. 496)
- **mvBlueCOUGAR-XD104hC Features** (p. 496)

1.20.1.2.8.9 mvBlueCOUGAR-XD104hG Features

1.20.1.2.8.10 mvBlueCOUGAR-XD104hC Features

1.20.1.2.9 mvBlueCOUGAR-X104i (3.2 Mpix [2064 x 1544])

1.20.1.2.9.1 Introduction

The CMOS sensor module (IMX265) incorporates the following features:

- resolution to 2064 x 1544 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.9.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.9.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	423
VerticalBlankLines	32
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 11.4 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

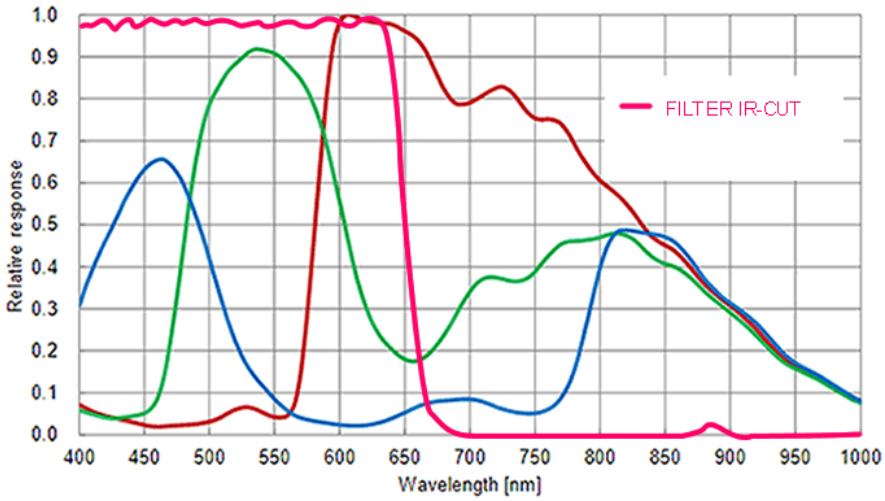
1.20.1.2.9.4 Sensor Data

Device Structure

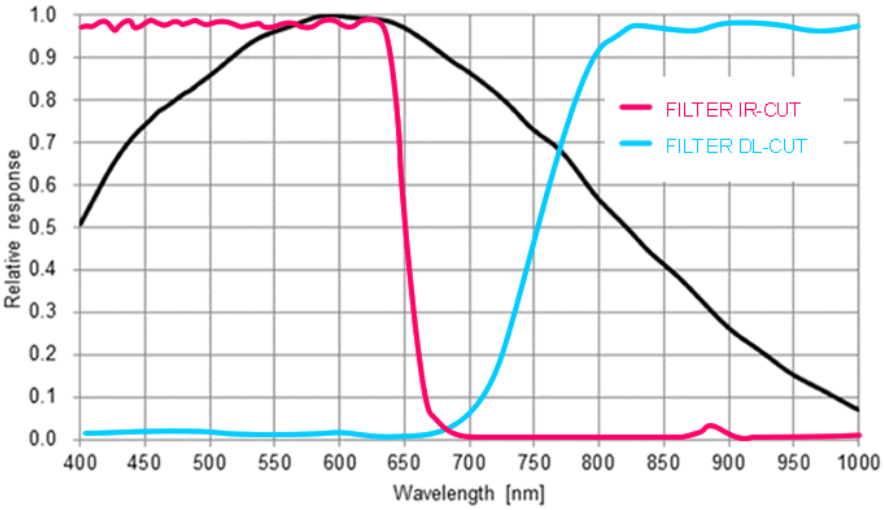
- CMOS image sensor (Type 1/1.8")
- Number of effective pixels: 2064 (H) x 1544 (V)
- Unit cell size: 3.45um (H) x 3.45um (V)

1.20.1.2.9.5 Characteristics

1.20.1.2.9.6 Color version



1.20.1.2.9.7 Gray scale version



Device Feature And Property List (p. 499)

1.20.1.2.9.8 Device Feature And Property List

- **mvBlueCOUGAR-X104iG Features** (p. 499)
- **mvBlueCOUGAR-X104iC Features** (p. 500)

1.20.1.2.9.9 mvBlueCOUGAR-X104iG Features

1.20.1.2.9.10 mvBlueCOUGAR-X104iC Features

1.20.1.2.10 mvBlueCOUGAR-XD105a (5.1 Mpix [2464 x 2056])

1.20.1.2.10.1 Introduction

The CMOS sensor module (IMX250) incorporates the following features:

- resolution to 2464 x 2056 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.10.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.10.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	440 (10 bit) / 519 (12 bit)
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 5.94 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

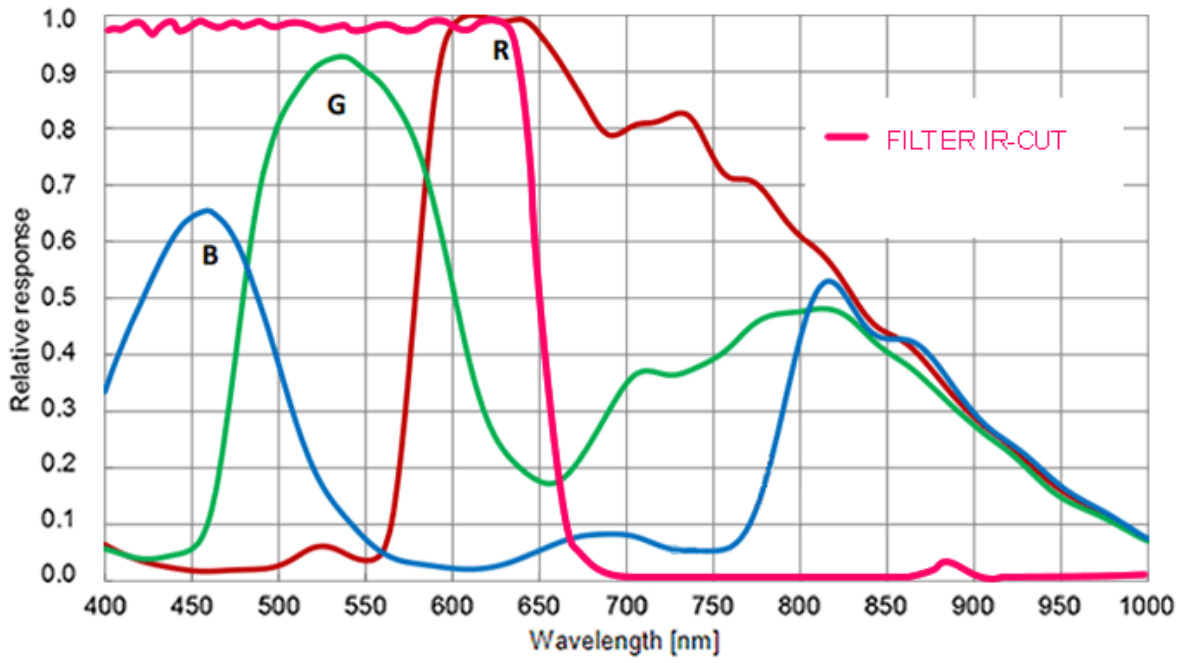
1.20.1.2.10.4 Sensor Data

Device Structure

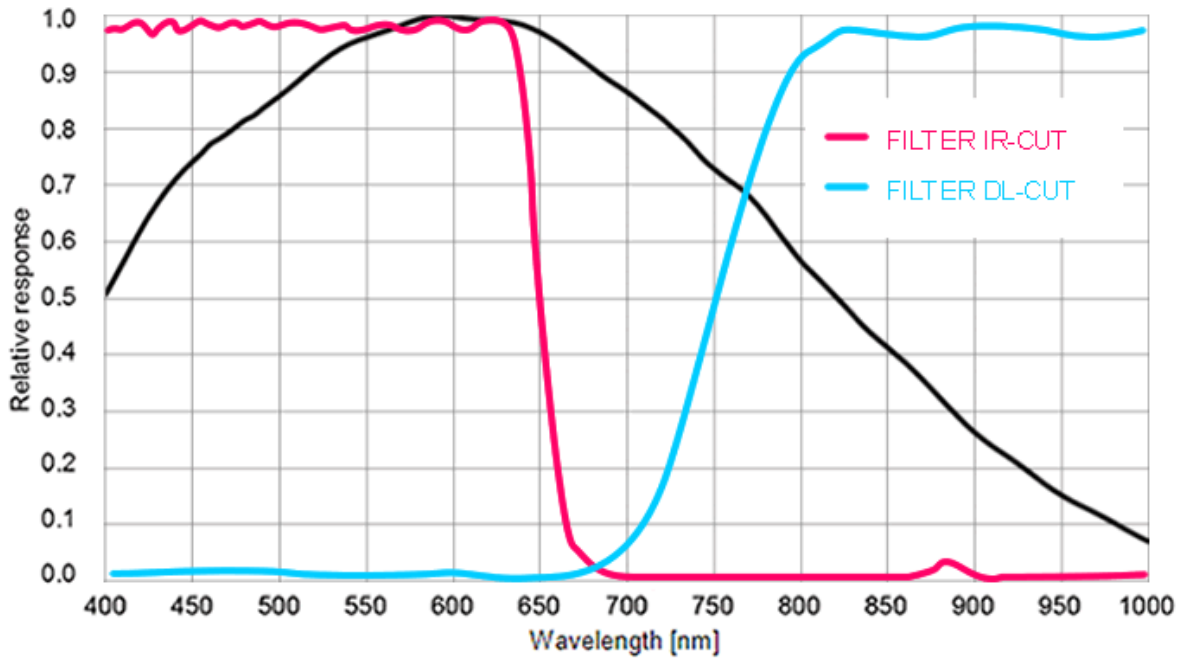
- CMOS image sensor (Type 2/3")
- Number of effective pixels: 2464 (H) x 2056 (V)
- Unit cell size: 3.45um (H) x 3.45um (V)

1.20.1.2.10.5 Characteristics

1.20.1.2.10.6 Color version



1.20.1.2.10.7 Gray scale version



Device Feature And Property List (p. 503)

1.20.1.2.10.8 Device Feature And Property List

- **mvBlueCOUGAR-XD105aG Features** (p. 503)
- **mvBlueCOUGAR-XD105aC Features** (p. 503)

1.20.1.2.10.9 mvBlueCOUGAR-XD105aG Features

1.20.1.2.10.10 mvBlueCOUGAR-XD105aC Features

1.20.1.2.11 mvBlueCOUGAR-X105b (5.1 Mpix [2464 x 2056])

1.20.1.2.11.1 Introduction

The CMOS sensor module (IMX264) incorporates the following features:

- resolution to 2464 x 2056 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.11.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.11.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	498
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4

$$FrameTime = \frac{InternalLineLength * 8}{SensorInClock * NumberOfLVDS} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 13.4 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

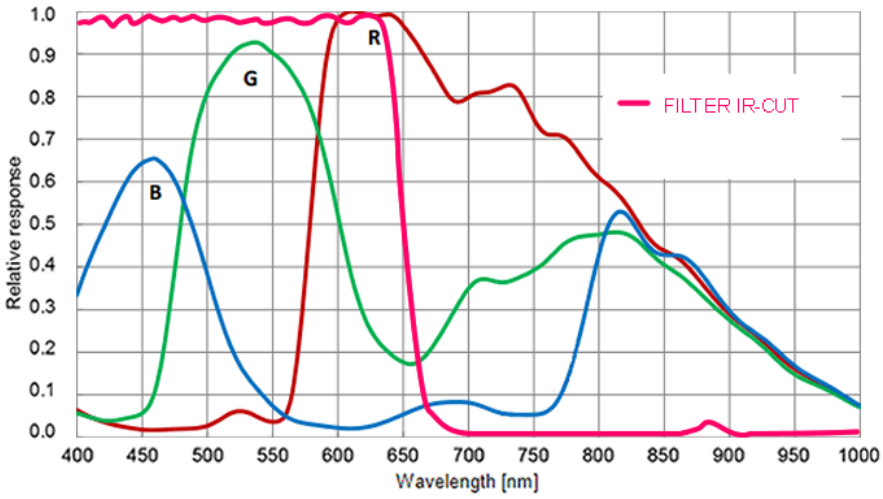
1.20.1.2.11.4 Sensor Data

Device Structure

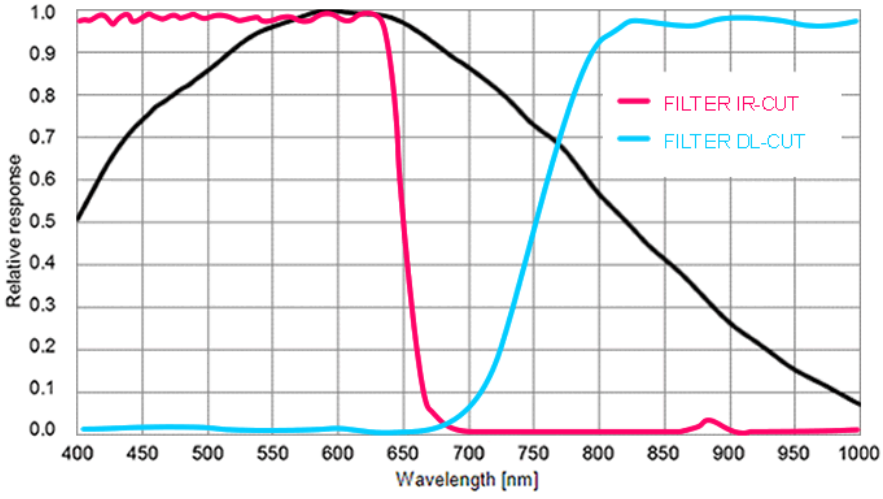
- CMOS image sensor (Type 2/3")
- Number of effective pixels: 2464 (H) x 2056 (V)
- Unit cell size: 3.45um (H) x 3.45um (V)

1.20.1.2.11.5 Characteristics

1.20.1.2.11.6 Color version



1.20.1.2.11.7 Gray scale version



Device Feature And Property List (p. 507)

1.20.1.2.11.8 Device Feature And Property List

- **mvBlueCOUGAR-X105bG Features** (p. 507)
- **mvBlueCOUGAR-X105bC Features** (p. 507)

1.20.1.2.11.9 mvBlueCOUGAR-X105bG Features

1.20.1.2.11.10 mvBlueCOUGAR-X105bC Features

1.20.1.2.12 mvBlueCOUGAR-XD107 (7.1 Mpix [3216 x 2208])

1.20.1.2.12.1 Introduction

The CMOS sensor module (IMX420) incorporates the following features:

- resolution to 3216 x 2208 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.12.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.12.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	390 (10 bit) / 466 (12 bit)
VerticalBlankLines	75
SensorInClock	53.977 (@54 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 7.23 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

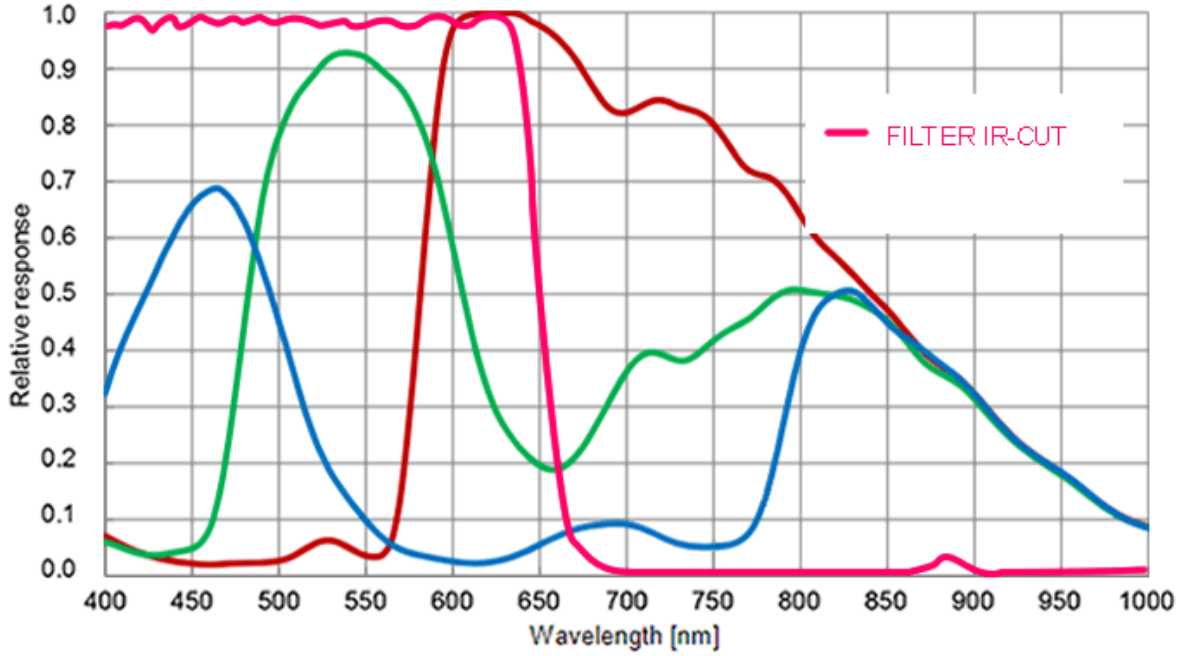
1.20.1.2.12.4 Sensor Data

Device Structure

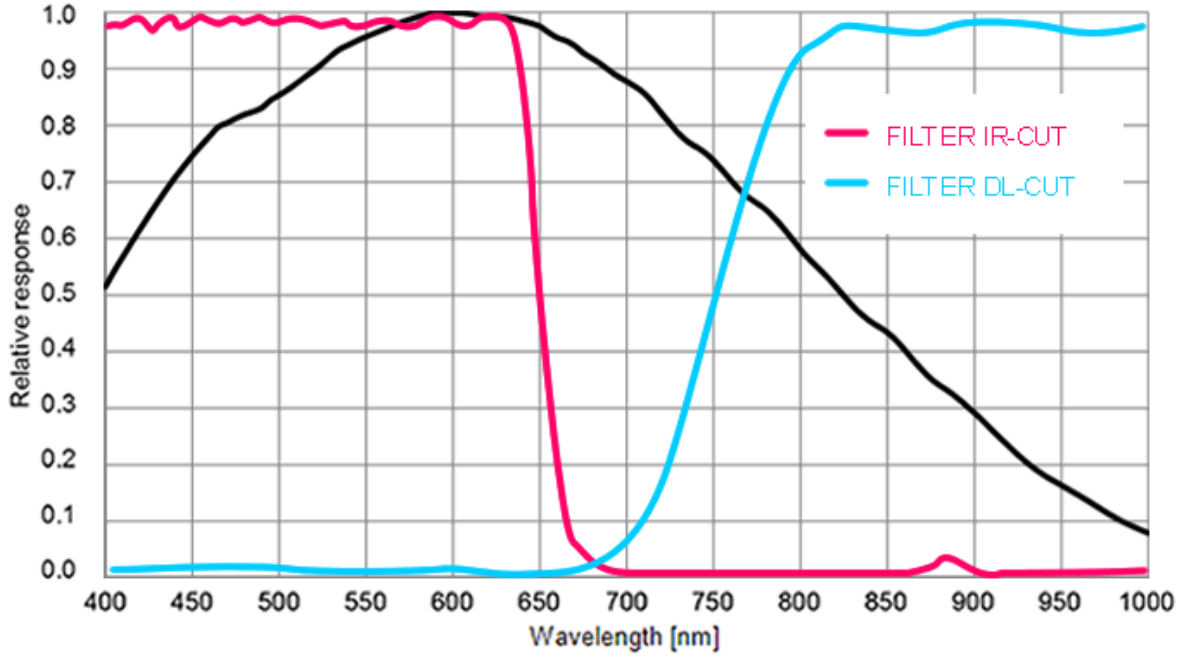
- CMOS image sensor (Type 1.1")
- Number of effective pixels: 3216 (H) x 2208 (V)
- Unit cell size: 4.5um (H) x 4.5um (V)

1.20.1.2.12.5 Characteristics

1.20.1.2.12.6 Color version



1.20.1.2.12.7 Gray scale version



Device Feature And Property List (p. 510)

1.20.1.2.12.8 Device Feature And Property List

- **mvBlueCOUGAR-XD107G Features** (p. 511)
- **mvBlueCOUGAR-XD107C Features** (p. 511)

1.20.1.2.12.9 mvBlueCOUGAR-XD107G Features

1.20.1.2.12.10 mvBlueCOUGAR-XD107C Features

1.20.1.2.13 mvBlueCOUGAR-X[D]107b (7.1 Mpix [3216 x 2208])

1.20.1.2.13.1 Introduction

The CMOS sensor module (IMX428) incorporates the following features:

- resolution to 3216 x 2208 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.13.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.13.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	466
VerticalBlankLines	75
SensorInClock	53.977 (@54 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 8.63 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

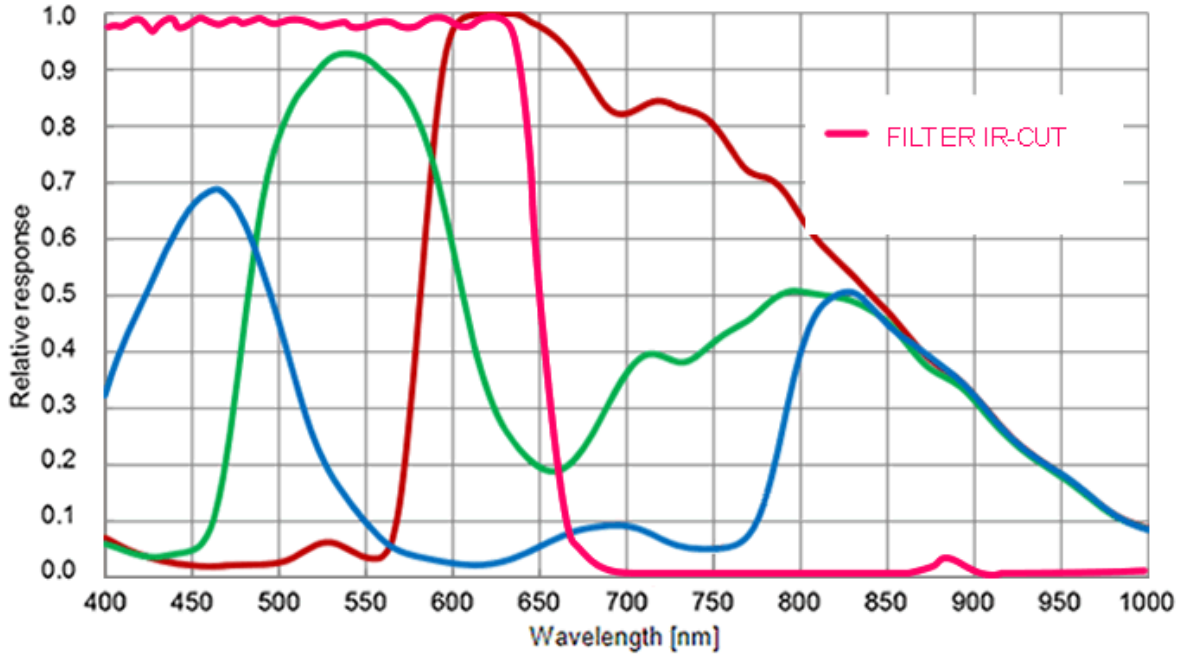
1.20.1.2.13.4 Sensor Data

Device Structure

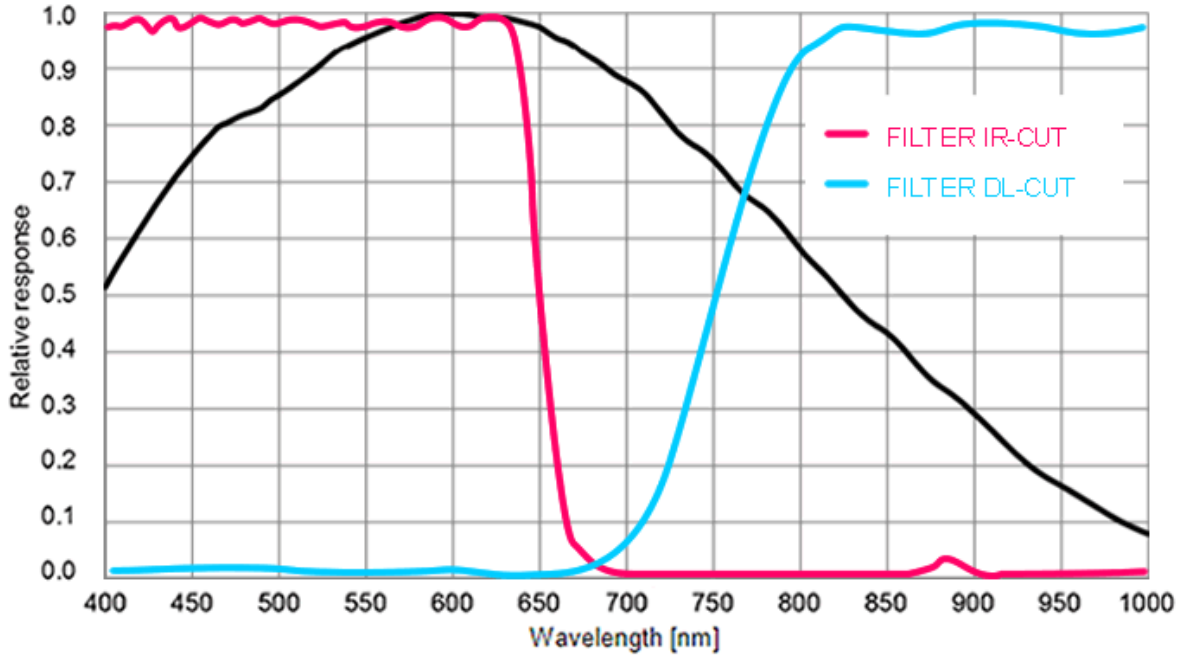
- CMOS image sensor (Type 1.1")
- Number of effective pixels: 3216 (H) x 2208 (V)
- Unit cell size: 4.5um (H) x 4.5um (V)

1.20.1.2.13.5 Characteristics

1.20.1.2.13.6 Color version



1.20.1.2.13.7 Gray scale version



Device Feature And Property List (p. 514)

1.20.1.2.13.8 Device Feature And Property List

- **mvBlueCOUGAR-X107bG Features** (p. 515)

- **mvBlueCOUGAR-X107bC Features** (p. 515)
- **mvBlueCOUGAR-XD107bG Features** (p. 515)
- **mvBlueCOUGAR-XD107bC Features** (p. 515)

1.20.1.2.13.9 mvBlueCOUGAR-X107bG Features

1.20.1.2.13.10 mvBlueCOUGAR-X107bC Features

1.20.1.2.13.11 mvBlueCOUGAR-XD107bG Features

1.20.1.2.13.12 mvBlueCOUGAR-XD107bC Features

1.20.1.2.14 mvBlueCOUGAR-X[D]109b (8.9 Mpix [4112 x 2176])

1.20.1.2.14.1 Introduction

The CMOS sensor module (IMX267) incorporates the following features:

- resolution to 4112 x 2176 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.14.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.14.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	807 (mvBlueCOUGAR-X) / 1041 (mvBlueCOUGAR-XD)
VerticalBlankLines	54 (mvBlueCOUGAR-X) / 54 (mvBlueCOUGAR-XD)
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4 (mvBlueCOUGAR-X) / 8 (mvBlueCOUGAR-XD)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 21.78 us (mvBlueCOUGAR-X) or 14.04 us (mvBlueCOUGAR-XD) and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = LevelLow"` `"ExposureMode = Timed"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = LevelHigh"` `"ExposureMode = Timed"`	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

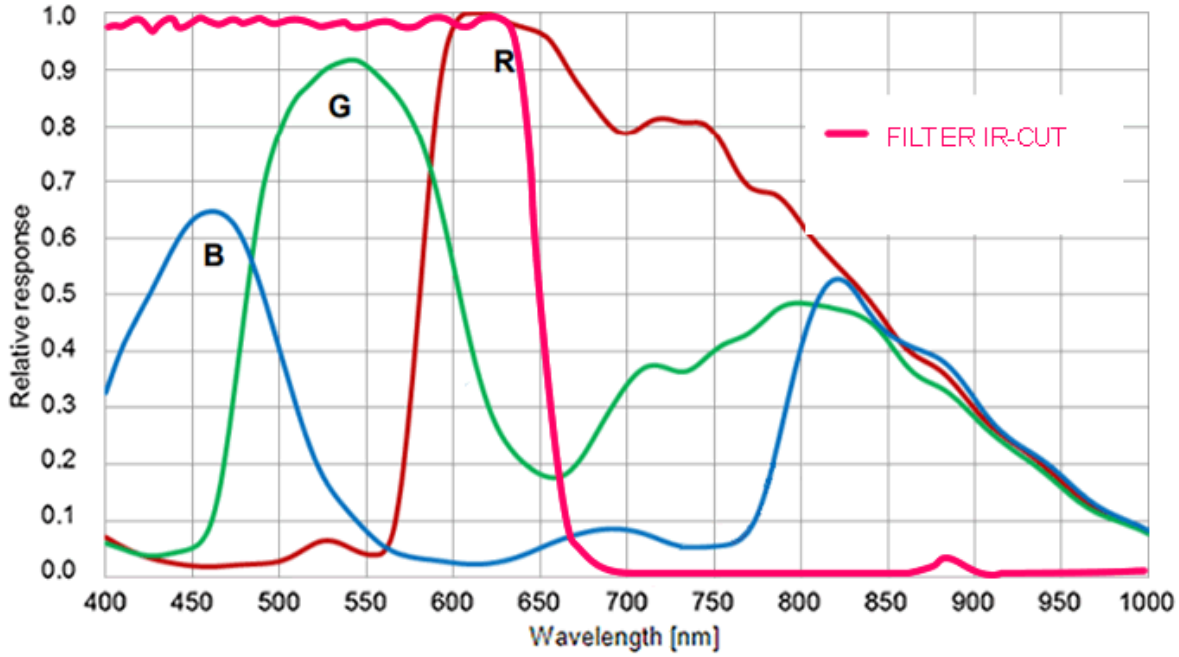
1.20.1.2.14.4 Sensor Data

Device Structure

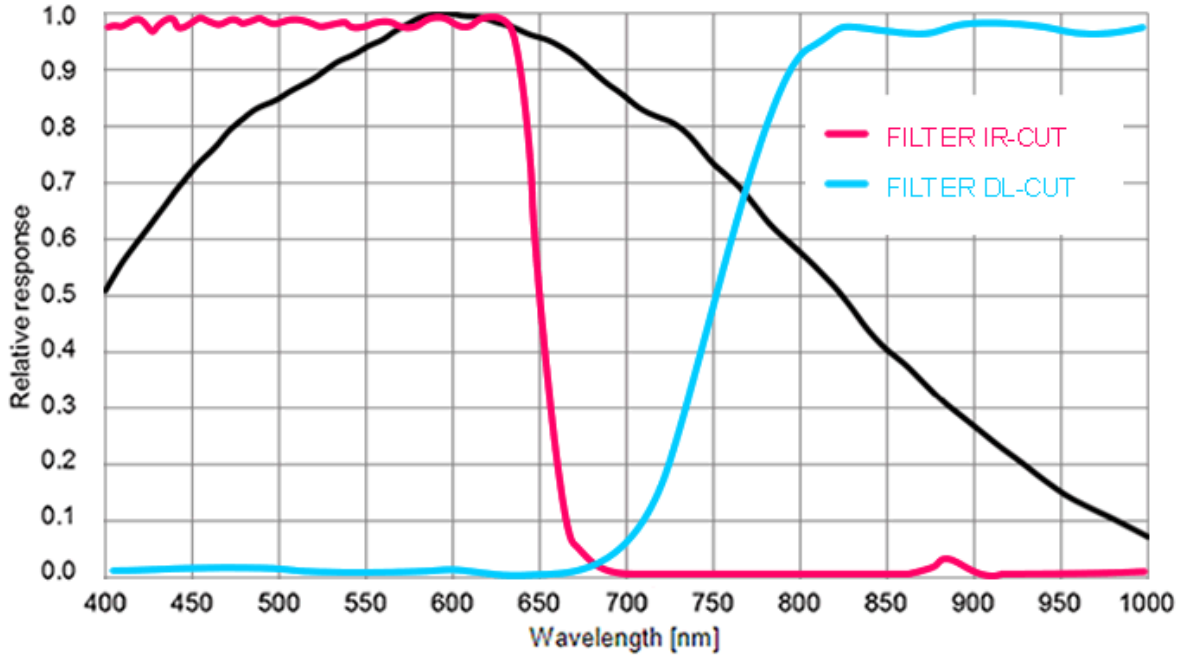
- CMOS image sensor (Type 1")
- Number of effective pixels: 4112 (H) x 2176 (V)
- Unit cell size: 3.45um (H) x 3.45um (V)

1.20.1.2.14.5 Characteristics

1.20.1.2.14.6 Color version



1.20.1.2.14.7 Gray scale version



Device Feature And Property List (p. 518)

1.20.1.2.14.8 Device Feature And Property List

- **mvBlueCOUGAR-X109bG Features** (p. 519)

- **mvBlueCOUGAR-X109bC Features** (p. 519)
- **mvBlueCOUGAR-XD109bG Features** (p. 519)
- **mvBlueCOUGAR-XD109bC Features** (p. 519)

1.20.1.2.14.9 mvBlueCOUGAR-X109bG Features

1.20.1.2.14.10 mvBlueCOUGAR-X109bC Features

1.20.1.2.14.11 mvBlueCOUGAR-XD109bG Features

1.20.1.2.14.12 mvBlueCOUGAR-XD109bC Features

1.20.1.2.15 mvBlueCOUGAR-X[D]1012b (12.4 Mpix [4112 x 3008])

1.20.1.2.15.1 Introduction

The CMOS sensor module (IMX304) incorporates the following features:

- resolution to 4112 x 3008 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.15.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.15.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	807 (mvBlueCOUGAR-X) / 1041 (mvBlueCOUGAR-XD)
VerticalBlankLines	54 (mvBlueCOUGAR-X) / 54 (mvBlueCOUGAR-XD)
SensorInClock	74.25 (@50 MHz Pixel clock)
NumberOfLVDS	4 (mvBlueCOUGAR-X) / 8 (mvBlueCOUGAR-XD)

$$FrameTime = \frac{InternalLineLength * 8}{SensorInClock * NumberOfLVDS} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 21.78 us (mvBlueCOUGAR-X) or 14.04 us (mvBlueCOUGAR-XD) and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation =	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
LevelHigh" "ExposureMode = Timed"		

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

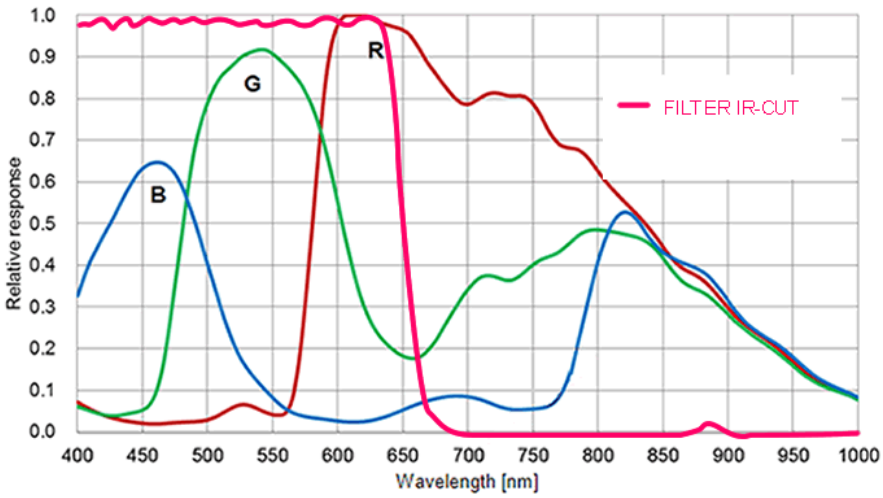
1.20.1.2.15.4 Sensor Data

Device Structure

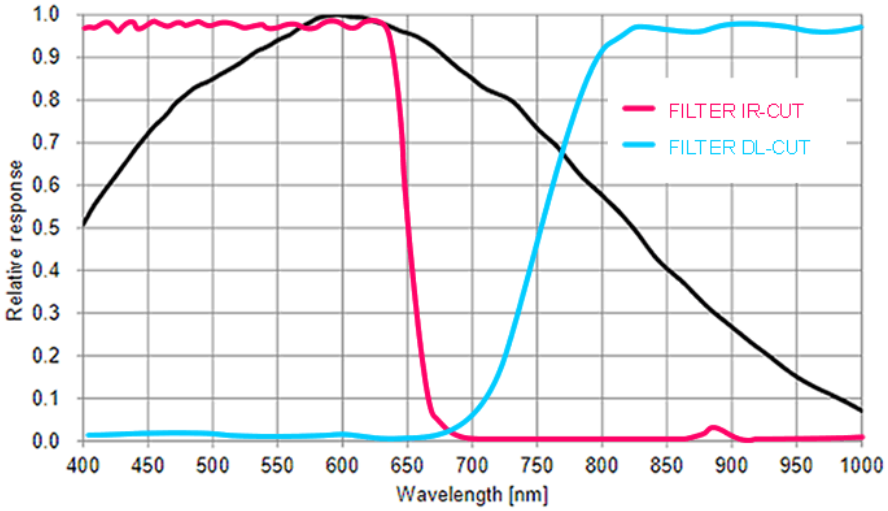
- CMOS image sensor (Type 1.1")
- Number of effective pixels: 4112 (H) x 3008 (V)
- Unit cell size: 3.45um (H) x 3.45um (V)

1.20.1.2.15.5 Characteristics

1.20.1.2.15.6 Color version



1.20.1.2.15.7 Gray scale version



Device Feature And Property List (p. 522)

1.20.1.2.15.8 Device Feature And Property List

- **mvBlueCOUGAR-X1012bG Features** (p. 522)
- **mvBlueCOUGAR-X1012bC Features** (p. 522)
- **mvBlueCOUGAR-XD1012bG Features** (p. 523)
- **mvBlueCOUGAR-XD1012bC Features** (p. 523)

1.20.1.2.15.9 mvBlueCOUGAR-X1012bG Features

1.20.1.2.15.10 mvBlueCOUGAR-X1012bC Features

1.20.1.2.15.11 mvBlueCOUGAR-XD1012bG Features

1.20.1.2.15.12 mvBlueCOUGAR-XD1012bC Features

1.20.1.2.16 mvBlueCOUGAR-XD1031 (31.5 Mpix [6480 x 4856])

1.20.1.2.16.1 Introduction

The CMOS sensor module (IMX342) incorporates the following features:

- resolution to 6480 x 4856 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.1.2.16.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.1.2.16.3 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	910 (12 bit)
VerticalBlankLines	50
SensorInClock	53.977 (@54 MHz Pixel clock)

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 16.86 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

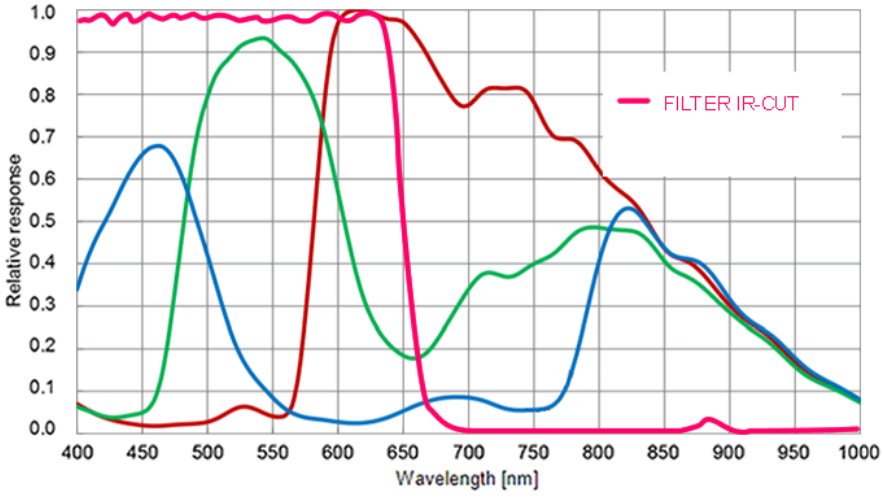
1.20.1.2.16.4 Sensor Data

Device Structure

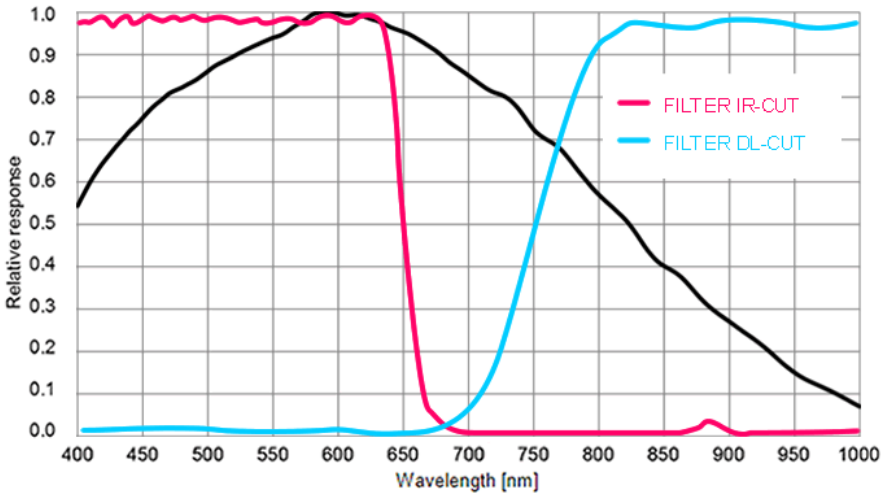
- CMOS image sensor (Type APS-C)
- Number of effective pixels: 6480 (H) x 4860 (V)
- Unit cell size: 3.45um (H) x 3.45um (V)

1.20.1.2.16.5 Characteristics

1.20.1.2.16.6 Color version



1.20.1.2.16.7 Gray scale version



Device Feature And Property List (p. 526)

1.20.1.2.16.8 Device Feature And Property List

- **mvBlueCOUGAR-XD1031G Features** (p. 526)
- **mvBlueCOUGAR-XD1031C Features** (p. 526)

1.20.1.2.16.9 mvBlueCOUGAR-XD1031G Features

1.20.1.2.16.10 mvBlueCOUGAR-XD1031C Features

1.20.2 A.2 Starvis CMOS

- **mvBlueCOUGAR-X106 (6.4 Mpix [3096 x 2080])** (p. 526)
- **mvBlueCOUGAR-X1012r (12.4 Mpix [4072 x 3044])** (p. 530)
- **mvBlueCOUGAR-X1020 (20.5 Mpix [5544 x 3692])** (p. 533)

1.20.2.1 mvBlueCOUGAR-X106 (6.4 Mpix [3096 x 2080])

1.20.2.1.1 Introduction

The CMOS sensor module (IMX178) incorporates the following features:

- resolution to 3096 x 2080 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- rolling shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.2.1.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.2.1.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	1005
VerticalBlankLines	32
SensorInClock	74.25
NumberOfLVDS	4

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 13.54 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

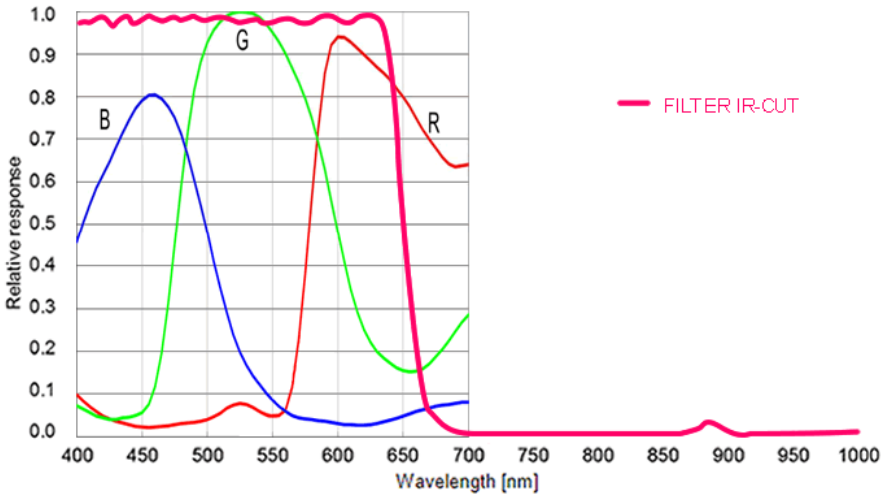
1.20.2.1.3 Sensor Data

Device Structure

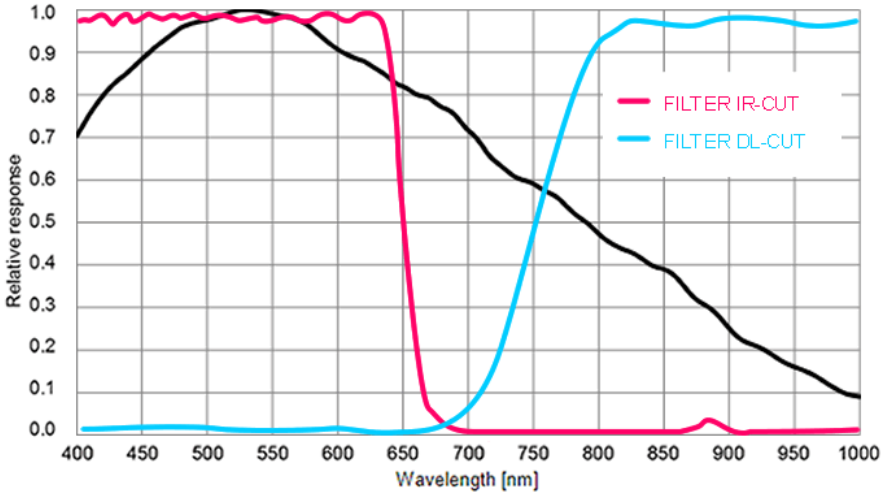
- CMOS image sensor (Type 1/1.8")
- Number of effective pixels: 3096 (H) x 2080 (V)
- Unit cell size: 1.85um (H) x 1.85um (V)

1.20.2.1.3.1 Characteristics

1.20.2.1.3.2 Color version



1.20.2.1.3.3 Gray scale version



Device Feature And Property List (p. 530)

1.20.2.1.4 Device Feature And Property List

- **mvBlueCOUGAR-X106G Features** (p. 530)
- **mvBlueCOUGAR-X106C Features** (p. 530)

1.20.2.1.4.1 mvBlueCOUGAR-X106G Features

1.20.2.1.4.2 mvBlueCOUGAR-X106C Features

1.20.2.2 mvBlueCOUGAR-X1012r (12.4 Mpix [4072 x 3044])

1.20.2.2.1 Introduction

The CMOS sensor module (IMX226) incorporates the following features:

- resolution to 4072 x 3044 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- rolling shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.2.2.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.2.2.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	1465
VerticalBlankLines	38
SensorInClock	72
NumberOfLVDS	4

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 20.35 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"ExposureMode = Timed"		

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

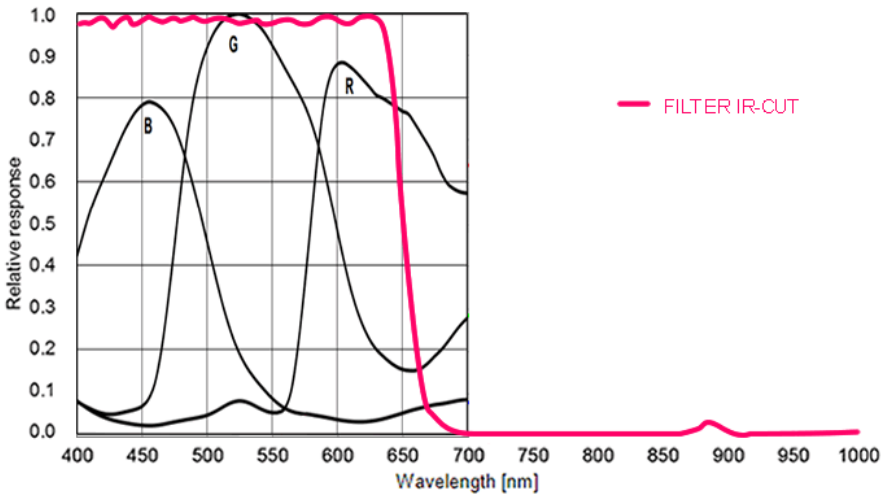
1.20.2.2.3 Sensor Data

Device Structure

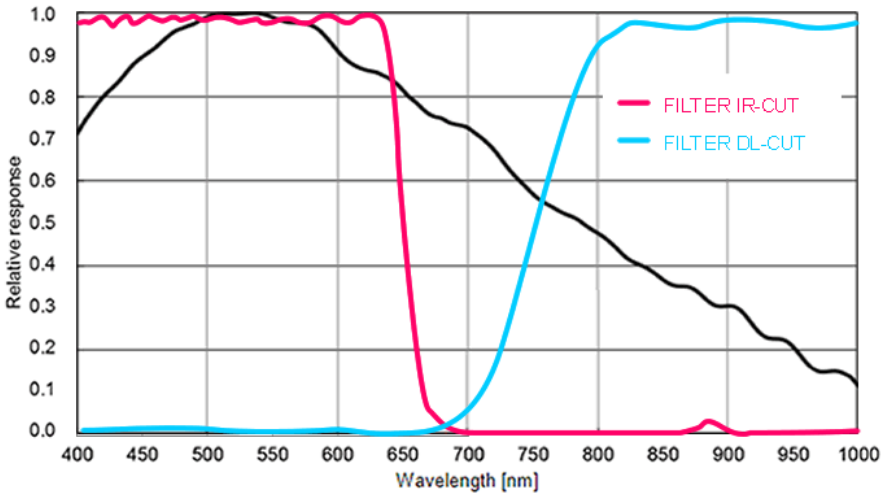
- CMOS image sensor (Type 1/1.7")
- Number of effective pixels: 4072 (H) x 3044 (V)
- Unit cell size: 1.85um (H) x 1.85um (V)

1.20.2.2.3.1 Characteristics

1.20.2.2.3.2 Color version



1.20.2.2.3.3 Gray scale version



Device Feature And Property List (p. 533)

1.20.2.2.4 Device Feature And Property List

- **mvBlueCOUGAR-X1012rG Features** (p. 533)
- **mvBlueCOUGAR-X1012rC Features** (p. 533)

1.20.2.2.4.1 mvBlueCOUGAR-X1012rG Features

1.20.2.2.4.2 mvBlueCOUGAR-X1012rC Features

1.20.2.3 mvBlueCOUGAR-X1020 (20.5 Mpix [5544 x 3692])

1.20.2.3.1 Introduction

The CMOS sensor module (IMX183) incorporates the following features:

- resolution to 5544 x 3692 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- rolling shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.2.3.2 Details of operation

The sensor uses a global shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.

1.20.2.3.2.1 Free running mode

The sensor uses a **rolling shutter**, i.e. all pixels of a given line are exposed to light the same time or **synchronous shutter**, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential. Therefore the sensor offers two different modes of operation:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout)

1.20.2.3.2.2 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	2190
VerticalBlankLines	37
SensorInClock	72 MHz
NumberOfLVDS	4

$$FrameTime = \frac{(InternalLineLength / SensorInClock) * VerticalBlankLines}{1000} + \frac{ImageHeight * (InternalLength}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 25.27 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

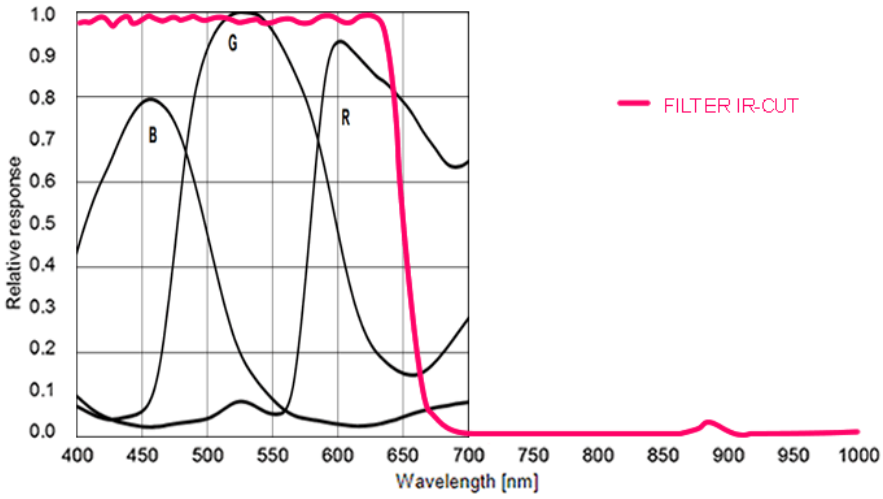
1.20.2.3.3 Sensor Data

Device Structure

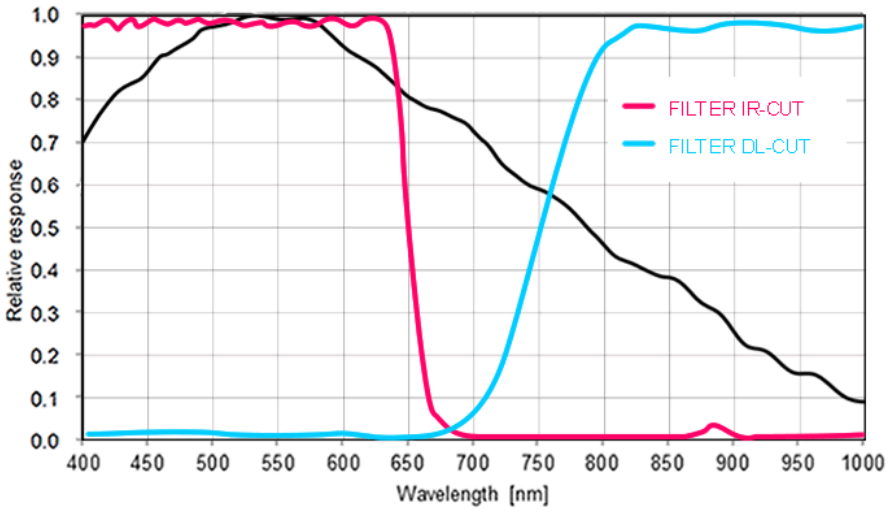
- CMOS image sensor (Type 1")
- Number of effective pixels: 5544 (H) x 3692 (V)
- Unit cell size: 2.4um (H) x 2.4um (V)

1.20.2.3.3.1 Characteristics

1.20.2.3.3.2 Color version



1.20.2.3.3.3 Gray scale version



Device Feature And Property List (p. 537)

1.20.2.3.4 Device Feature And Property List

- **mvBlueCOUGAR-X1020G Features** (p. 537)
- **mvBlueCOUGAR-X1020C Features** (p. 537)

1.20.2.3.4.1 mvBlueCOUGAR-X1020G Features

1.20.2.3.4.2 mvBlueCOUGAR-X1020C Features

1.20.3 A.3 Polarsens CMOS

- **mvBlueCOUGAR-X105p (5.1 Mpix [2464 x 2056])** (p. 537)

1.20.3.1 mvBlueCOUGAR-X105p (5.1 Mpix [2464 x 2056])

1.20.3.1.1 Introduction

The sensor can acquire a four directional polarization image in one shot. Because the polarizer is under the on-chip lens layer

- reflectances are reduced and
- flare and ghost characteristics are avoided.

The pixel array of the sensor is as follows:

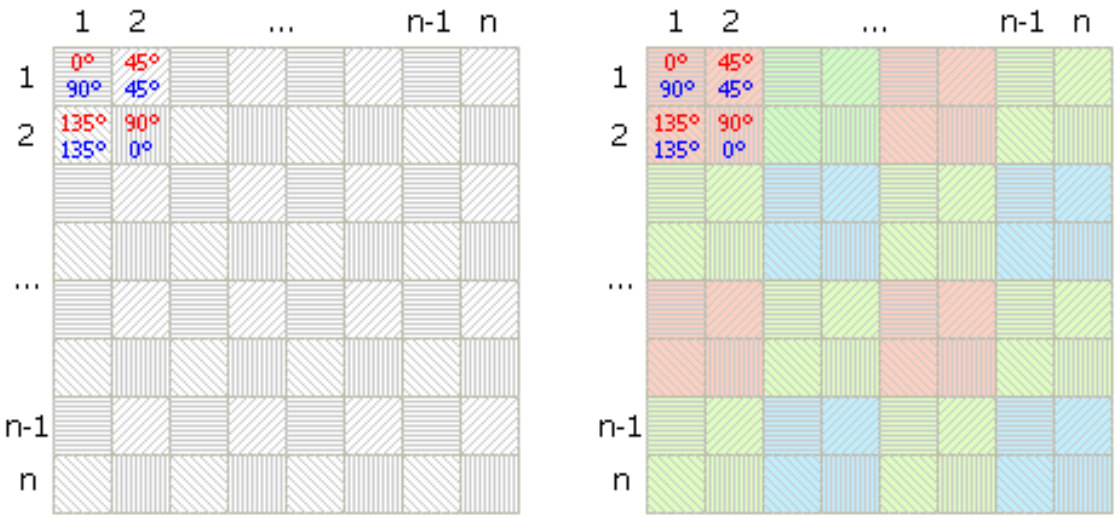


Figure 1: Pixel array gray scale / color

Feature	Description
Manufacturer	Sony
Sensor name	IMX250_POL
Max. frames per second	80
Device Structure	CMOS image sensor
SNR_{max} [dB][1/]	40.2
DNR (normal / HDR) [dB][1/]	71.1 / -
Image size	2/3
Number of effective pixels	2464 (H) x 2056 (V)
Unit cell size	3.45µm (H) x 3.45µm (V)
ADC resolution / output	12 bit → 8/10/(12)

[1] Measured accord. to EMVA1288 with gray scale version of the camera

1.20.3.1.2 Spectral Sensitivity

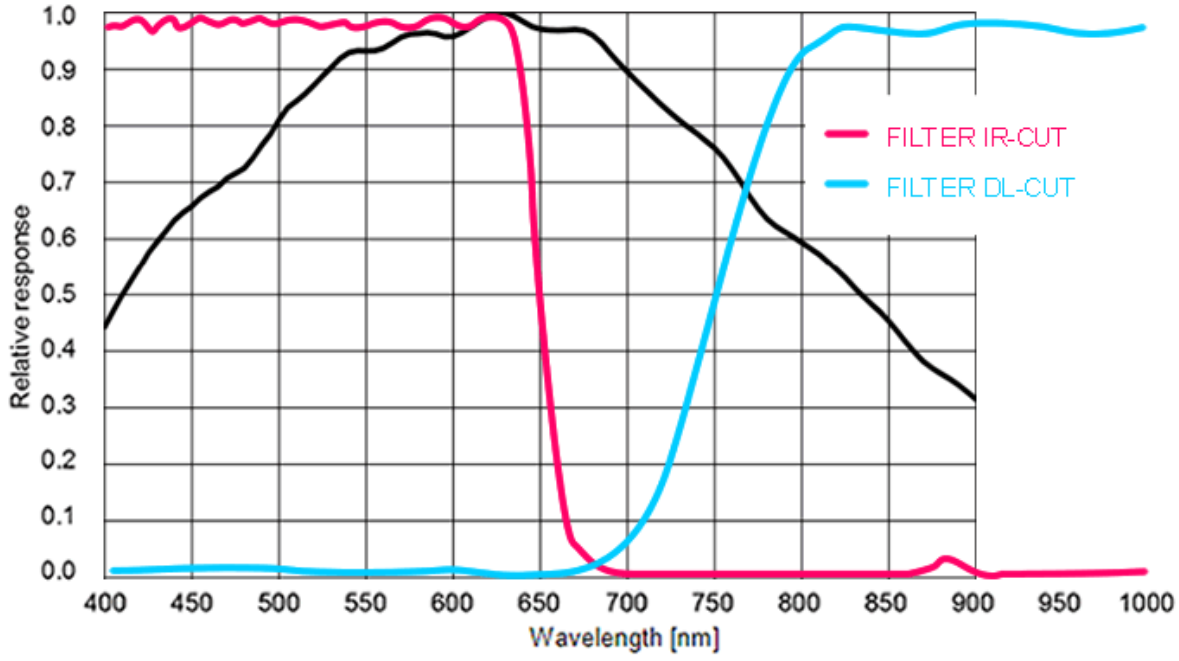


Figure 2: Spectral sensitivity mvBlueCOUGAR-X105pG

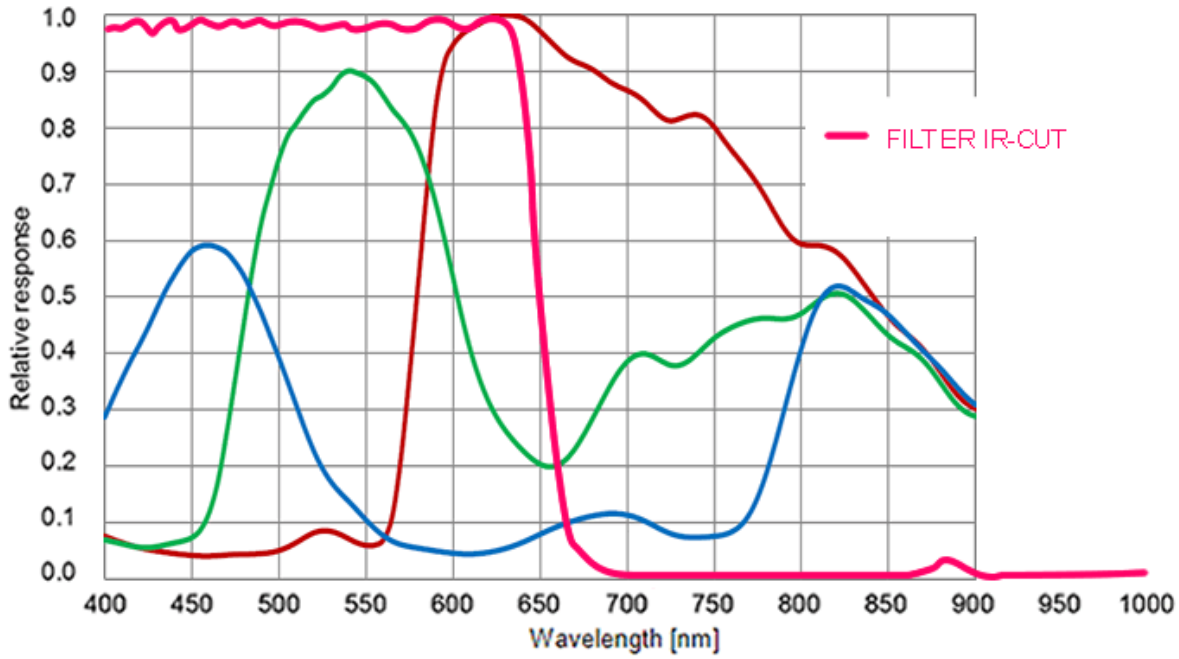


Figure 3: Spectral sensitivity mvBlueCOUGAR-X105pC

Name	Value
InternalLineLength	835 (10 bit) / 996 (12 bit)
VerticalBlankLines	37
SensorInClock	74.25 (@50 MHz Pixel clock)

1.20.3.1.3 Timings

1.20.3.1.3.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

$$FrameTime = \frac{InternalLineLength}{SensorInClock} * \frac{ImageHeight + VerticalBlankLines}{1000}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

Note

The exposure time step width is limited to the sensor's row time of 11.27 us and therefore
- auto exposure with very low exposure times will perform with relatively large increments and
- exposure mode = TriggerWidth (if available) will perform with a jitter corresponding to the row time.

The following trigger modes are available:

Setting (GenICam)	Mode / Setting (obsolete "Device Specific")	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Device Feature And Property List (p. 541)

1.20.3.1.4 Device Feature And Property List

- **mvBlueCOUGAR-X105pG Features** (p. 541)
- **mvBlueCOUGAR-X105pC Features** (p. 541)

1.20.3.1.4.1 mvBlueCOUGAR-X105pG Features

1.20.3.1.4.2 mvBlueCOUGAR-X105pC Features

1.20.4 A.4 CMOS

- **mvBlueCOUGAR-X100w (0.4 Mpix [752 x 480])** (p. 542)
- **mvBlueCOUGAR-X102b (1.2 Mpix [1280 x 960])** (p. 546)
- **mvBlueCOUGAR-X102d (1.2 Mpix [1280 x 960])** (p. 549)
- **mvBlueCOUGAR-X102e (1.3 Mpix [1280 x 1024])** (p. 554)
- **mvBlueCOUGAR-X102eGE (1.3 Mpix [1280 x 1024])** (p. 560)
- **mvBlueCOUGAR-X104e (1.9 Mpix [1600 x 1200])** (p. 567)
- **mvBlueCOUGAR-X[D]x04 (2.2 Mpix [2048 x 1088])** (p. 573)
- **mvBlueCOUGAR-X[D]x04b (4.2 Mpix [2048 x 2048])** (p. 577)
- **mvBlueCOUGAR-X105 (5 Mpix [2592 x 1944])** (p. 581)
- **mvBlueCOUGAR-X1010 (10 Mpix [3856 x 2764])** (p. 586)

1.20.4.1 mvBlueCOUGAR-X100w (0.4 Mpix [752 x 480])

1.20.4.1.1 Introduction

The CMOS sensor module (MT9V034) incorporates the following features:

- resolution to 752 x 480 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- **high dynamic range** (p. 378) 110 dB
- programmable analog gain (0..12 dB)
- progressive scan sensor (no interlaced problems!)
- full frame shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.1.2 Details of operation

The sensor uses a full **frame shutter** (ShutterMode = "`FrameShutter`"), i.e. all pixels are reset at the same time and the exposure commences. It ends with the charge transfer of the voltage sampling.
Furthermore, the sensor offers two different modes of operation:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout)

1.20.4.1.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

```
FrameTime = (ImageWidth + 61) * ((ImageHeight + 45) / PixelClock)
```

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

1.20.4.1.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

1.20.4.1.2.3 Trigger

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.4.1.2.4 Erase, exposure and readout

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

Note

Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

```
FrameTime = (ImageWidth + 61) * ((ImageHeight + 45) / PixelClock)

FPS_max =                  1
          ------------------------------------
                FrameTime + ExposureTime
```

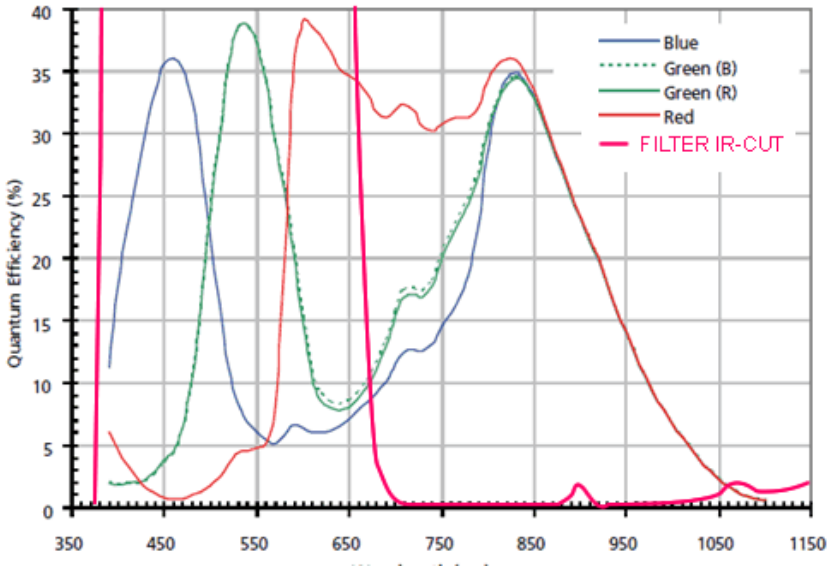
1.20.4.1.3 Sensor Data

Device Structure

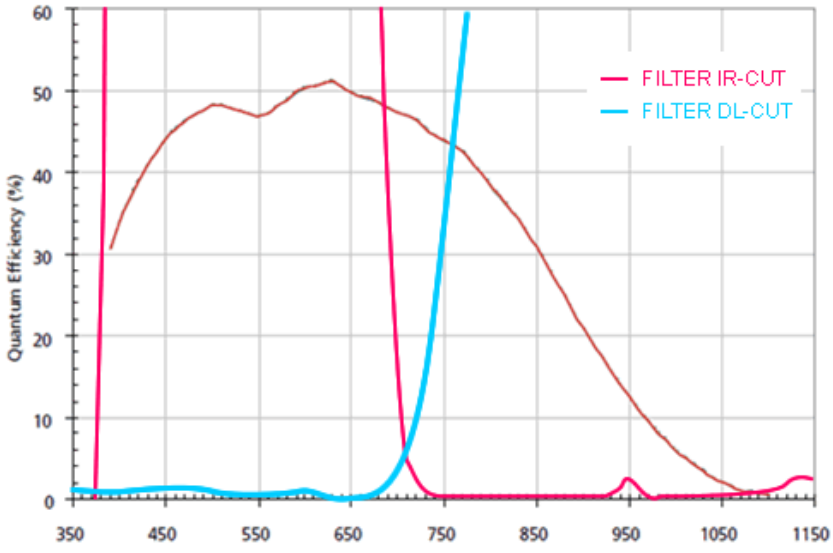
- Progressive scan CMOS image sensor
- Image size: 4.51(H)x2.88(V)mm (Type 1/3")
- Number of effective pixels: 752 (H) x 480 (V)
- Unit cell size: 6um (H) x 6um (V)

1.20.4.1.3.1 Characteristics

1.20.4.1.3.2 Color version



1.20.4.1.3.3 Gray scale version



Device Feature And Property List (p. 545)

1.20.4.1.4 Device Feature And Property List

- **mvBlueCOUGAR-X100wG Features** (p. 546)
- **mvBlueCOUGAR-X100wC Features** (p. 546)

1.20.4.1.4.1 mvBlueCOUGAR-X100wG Features

1.20.4.1.4.2 mvBlueCOUGAR-X100wC Features

1.20.4.2 mvBlueCOUGAR-X102b (1.2 Mpix [1280 x 960])

1.20.4.2.1 Introduction

The CMOS sensor module (MT9M021) incorporates the following features:

- resolution to 1280 x 960 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- programmable analog gain (0..12 dB)
- progressive scan sensor (no interlaced problems!)
- pipelined global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.2.2 Details of operation

The sensor uses a pipelined global snapshot shutter , i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.
Therefore the sensor offers two different modes of operation:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout)

1.20.4.2.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

```
FrameTime = (ImageHeight * (1650 / PixelClock)) + (25 * (1650 / PixelClock))
```

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

1.20.4.2.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

1.20.4.2.2.3 Trigger

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.4.2.2.4 Erase, exposure and readout

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

> **Note**
>
> Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

```
FrameTime = (ImageHeight * (1650 / PixelClock)) + (25 * (1650 / PixelClock))

FPS_max =                  1
          ------------------------------------
                FrameTime + ExposureTime
```

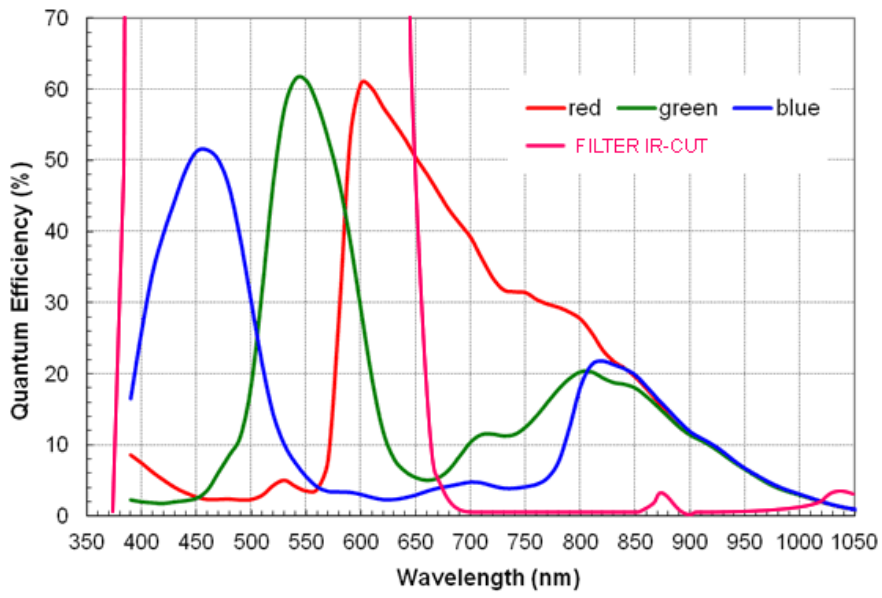
1.20.4.2.3 Sensor Data

Device Structure

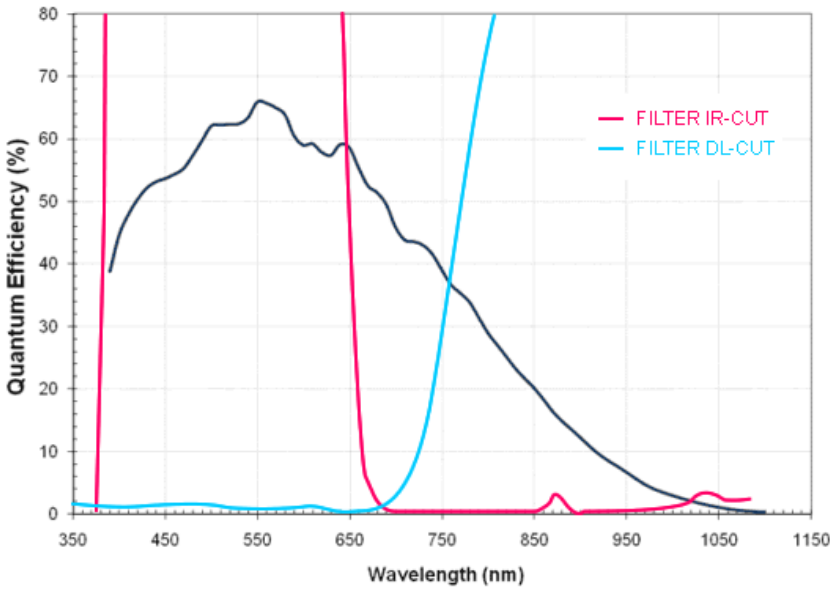
- CMOS image sensor (Type 1/3")
- Number of effective pixels: 1280 (H) x 960 (V)
- Unit cell size: 3.75um (H) x 3.75um (V)

1.20.4.2.3.1 Characteristics

1.20.4.2.3.2 Color version



1.20.4.2.3.3 Gray scale version



Device Feature And Property List (p. 549)

1.20.4.2.4 Device Feature And Property List

- **mvBlueCOUGAR-X102bG-POEI Features** (p. 549)
- **mvBlueCOUGAR-X102bC Features** (p. 549)

1.20.4.2.4.1 mvBlueCOUGAR-X102bG-POEI Features

1.20.4.2.4.2 mvBlueCOUGAR-X102bC Features

1.20.4.3 mvBlueCOUGAR-X102d (1.2 Mpix [1280 x 960])

1.20.4.3.1 Introduction

The CMOS sensor module (MT9M034) incorporates the following features:

- resolution to 1280 x 960 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- programmable analog gain (0..12 dB)
- progressive scan sensor (no interlaced problems!)
- **high dynamic range** (p. 381) 115 dB (with gray scale version)
- rolling shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.3.2 Details of operation

The sensor uses following acquisition mode:

- **rolling shutter**

With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time.

Note

Moving objects together with a rolling shutter can cause a shear in moving objects.

Furthermore, the sensor offers following operating modes:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout)

1.20.4.3.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

```
FrameTime = (ImageHeight * (1650 / PixelClock)) + (25 * (1650 / PixelClock))
```

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

1.20.4.3.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

1.20.4.3.2.3 Trigger

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.4.3.2.4 Erase, exposure and readout

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

Note

Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

```
FrameTime = (ImageHeight * (1650 / PixelClock)) + (25 * (1650 / PixelClock))
```

$$FPS_max = \frac{1}{FrameTime + ExposureTime}$$

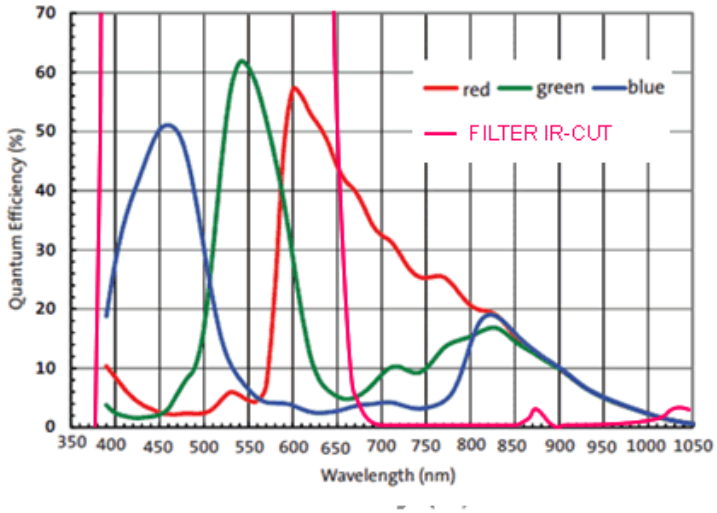
1.20.4.3.3 Sensor Data

Device Structure

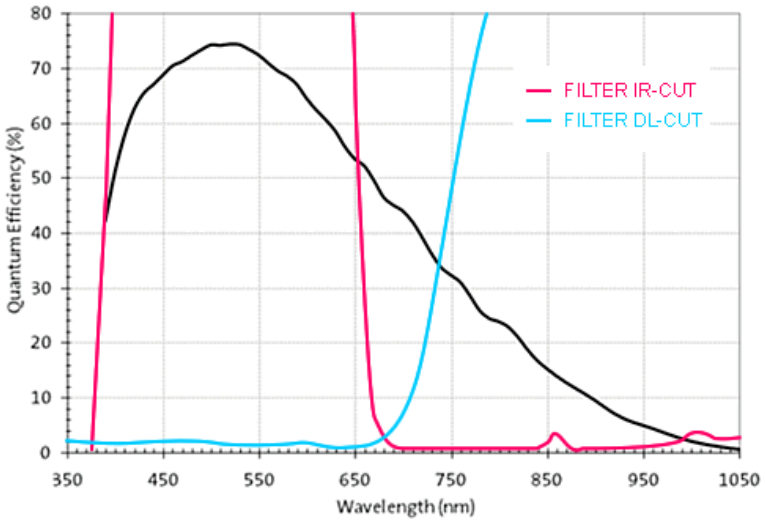
- CMOS image sensor (Type 1/3")
- Number of effective pixels: 1280 (H) x 960 (V)
- Unit cell size: 3.75um (H) x 3.75um (V)

1.20.4.3.3.1 Characteristics

1.20.4.3.3.2 Color version



1.20.4.3.3.3 Gray scale version



Device Feature And Property List (p. 553)

1.20.4.3.4 Device Feature And Property List

- **mvBlueCOUGAR-X102dG Features** (p. 554)
- **mvBlueCOUGAR-X102dC Features** (p. 554)

1.20.4.3.4.1 mvBlueCOUGAR-X102dG Features

1.20.4.3.4.2 mvBlueCOUGAR-X102dC Features

1.20.4.4 mvBlueCOUGAR-X102e (1.3 Mpix [1280 x 1024])

1.20.4.4.1 Introduction

The CMOS sensor module (EV76C560) incorporates the following features:

- resolution to 1280 x 1024 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- **high dynamic range** (p. 383)
- pipelined global snapshot shutter
- **Line scan mode** (p. 556)
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.4.2 Details of operation

The sensor uses a **rolling shutter**, i.e. all pixels of a given line are exposed to light the same time or **synchronous shutter**, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.
Therefore the sensor offers two different modes of operation:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout)

1.20.4.4.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	944
InternalADCClock us	120 MHz

$$FrameTime(ms) = InternalLineLength * \frac{2 * (ImageHeight + 10)}{InternalADCClock * 1000}$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1000}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

1.20.4.4.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

1.20.4.4.2.3 Trigger

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX) T
GP-IN0	Line4
GP-IN1	Line5

1.20.4.4.2.4 Erase, exposure and readout

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

Note

Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$FrameTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

$$FPS_max = \frac{1}{FrameTime + ExposureTime}$$

1.20.4.4.2.5 Line scan mode

The sensor offers also a usage as a line scan sensor. One (gray scale sensor) or two lines (in terms of color sensor) can be selected to be read out of the full line height of 1024 lines. This or these lines are grouped to a pseudo frame of selectable height in the internal buffer of the camera.

The camera then outputs these frames which contain multiples of the same scan line(s) without gaps or interruptions.

To operate in line scan mode, use the following properties:

1. In *"Setting -> Base -> Camera -> GenICam -> Device Control"*, please change the *"Device Scan Type"* to **"line scan"**.

2. In *"Setting -> Base -> Camera -> GenICam -> Image Format Control"*, please set *"Width"* and *"Height"*. *"Height"* specifies how often the same line(s) will be used to reach the height of the image.
   Use *"Offset X"* to shift horizontally the starting point of the exposed line. Use *"Offset Y"* to shift the scan line vertically (typically to 512 which represents the optical center).

   Note

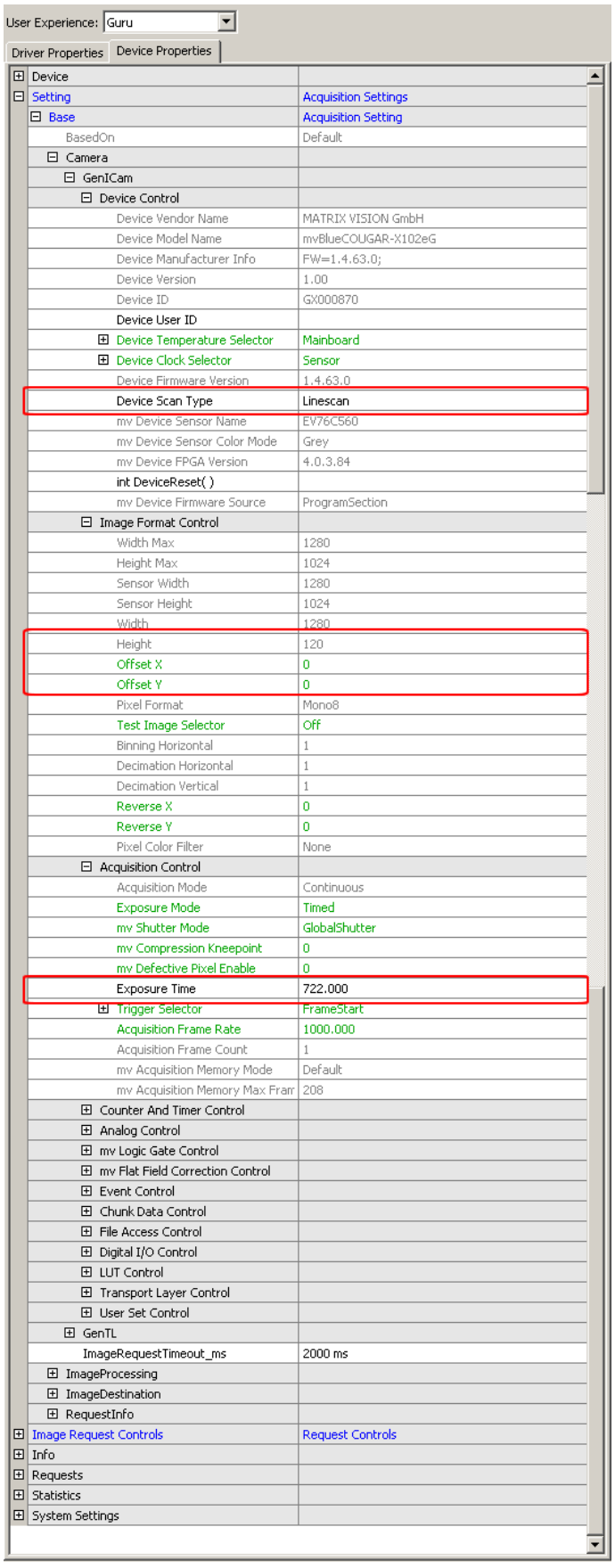
   The sensor will not get faster by windowing in x direction.

3. Finally, in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"*, please adapt the *"Exposure Time"*. For high line scan rates the exposure time has to be proportionally low. To achieve the maximum line scan rate of 12.6 kHz (gray scale sensor with rolling shutter) or 10.6 kHz (gray scale sensor with global shutter) the exposure time has to be set to lower than 62us.

You may use longer exposure times at the expense of scanning frequency.

Note

Using more than one line e.g. 20, is like having an area scan with ImageHeight = 20. In line scan mode, the min. ImageHeight is at least 16.



wxPropView settings for line scan usage

Note

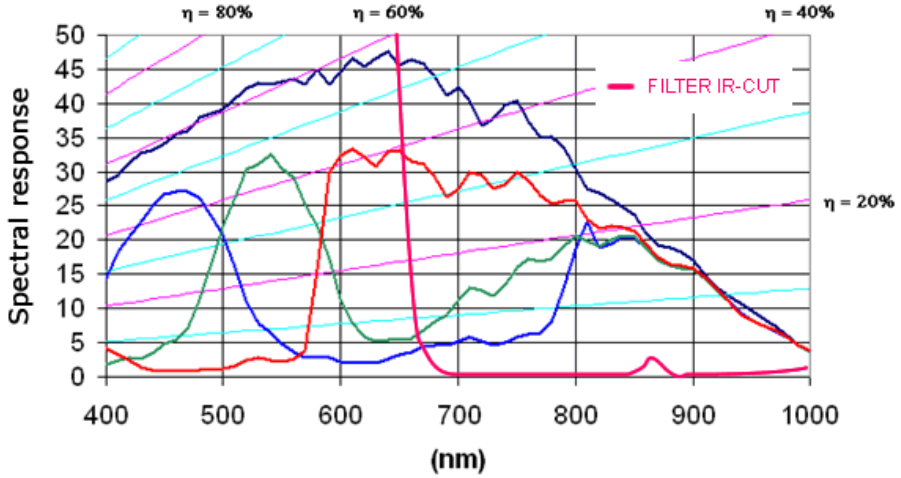
You can use either "`Continuous`" or a trigger mode as *"Acquisition Mode"* in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"*. However, if an external (line) trigger will have to be used, it is absolutely required that the external trigger is always present. During a trigger interruption, controlling or communication to the camera is not possible.

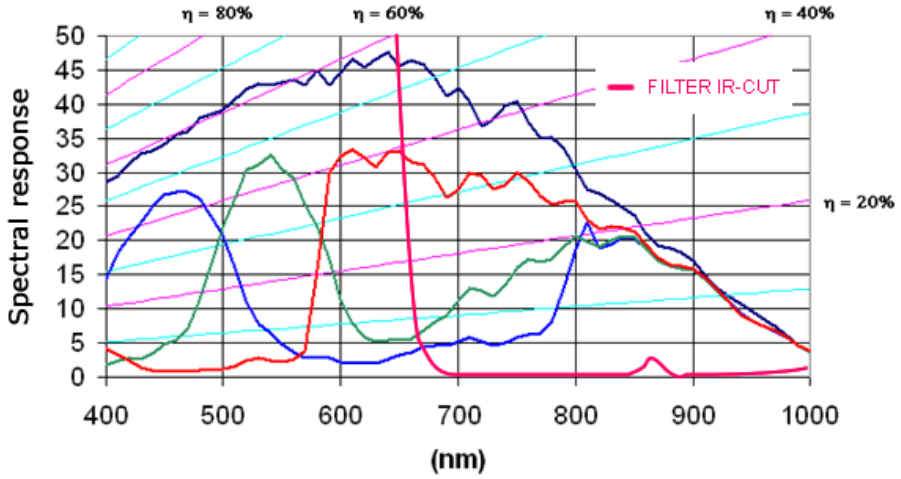
1.20.4.4.3 Sensor Data

Device Structure

- CMOS image sensor (Type 1/1.8")
- Number of effective pixels: 1280 (H) x 1024 (V)
- Unit cell size: 5.3um (H) x 5.3um (V)

1.20.4.4.3.1 Characteristics





Device Feature And Property List (p. 560)

1.20.4.4.4 Device Feature And Property List

- **mvBlueCOUGAR-X102eG Features** (p. 560)
- **mvBlueCOUGAR-X102eC Features** (p. 560)

1.20.4.4.4.1 mvBlueCOUGAR-X102eG Features

1.20.4.4.4.2 mvBlueCOUGAR-X102eC Features

1.20.4.5 mvBlueCOUGAR-X102eGE (1.3 Mpix [1280 x 1024])

1.20.4.5.1 Introduction

The CMOS sensor module (EV76C661) incorporates the following features:

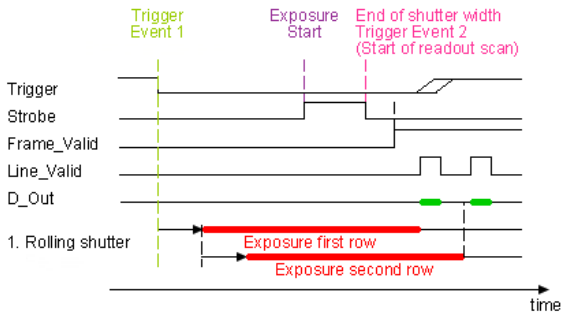
- resolution to 1280 x 1024 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- **high dynamic range** (p. 383)
- enhanced infrared sensitivity
- global snapshot shutter
- **Line scan mode** (p. 563)
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.5.2 Details of operation

The sensor uses two acquisition modes:

- **rolling shutter** (mvShutterMode = "`RollingShutter`") and
- **global reset release shutter** (mvShutterMode = "`mvGlobalShutter`").

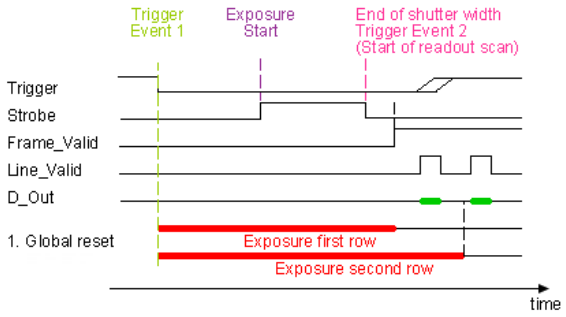
With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time:



Note

Moving objects together with a rolling shutter can cause a shear in moving objects.

The **global reset release shutter**, which is only available in triggered operation, starts the exposure of all rows simultaneously and the reset to each row is released simultaneously, too. However, the readout of the lines is equal to the readout of the rolling shutter: line by line:



Note

This means, the bottom lines of the sensor will be exposed to light longer! For this reason, this mode will only make sense, if there is no extraneous light and the flash duration is shorter or equal to the exposure time.

Furthermore, the sensor offers two operating modes:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout) in triggered operation

1.20.4.5.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	944
InternalADCClock	120

$$FrameTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

1.20.4.5.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

1.20.4.5.2.3 Trigger

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.4.5.2.4 Erase, exposure and readout

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

Note

Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$FrameTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

$$FPS_max = \frac{1}{FrameTime + ExposureTime}$$

1.20.4.5.2.5 Line scan mode

The sensor offers also a usage as a line scan sensor. One (gray scale sensor) or two lines (in terms of color sensor) can be selected to be read out of the full line height of 1024 lines. This or these lines are grouped to a pseudo frame of selectable height in the internal buffer of the camera.

The camera then outputs these frames which contain multiples of the same scan line(s) without gaps or interruptions.

To operate in line scan mode, use the following properties:

1. In *"Setting -> Base -> Camera -> GenICam -> Device Control"*, please change the *"Device Scan Type"* to **"line scan"**.

2. In *"Setting -> Base -> Camera -> GenICam -> Image Format Control"*, please set *"Width"* and *"Height"*. *"Height"* specifies how often the same line(s) will be used to reach the height of the image.
   Use *"Offset X"* to shift horizontally the starting point of the exposed line. Use and *"Offset Y"* to shift the scan line vertically.

   Note

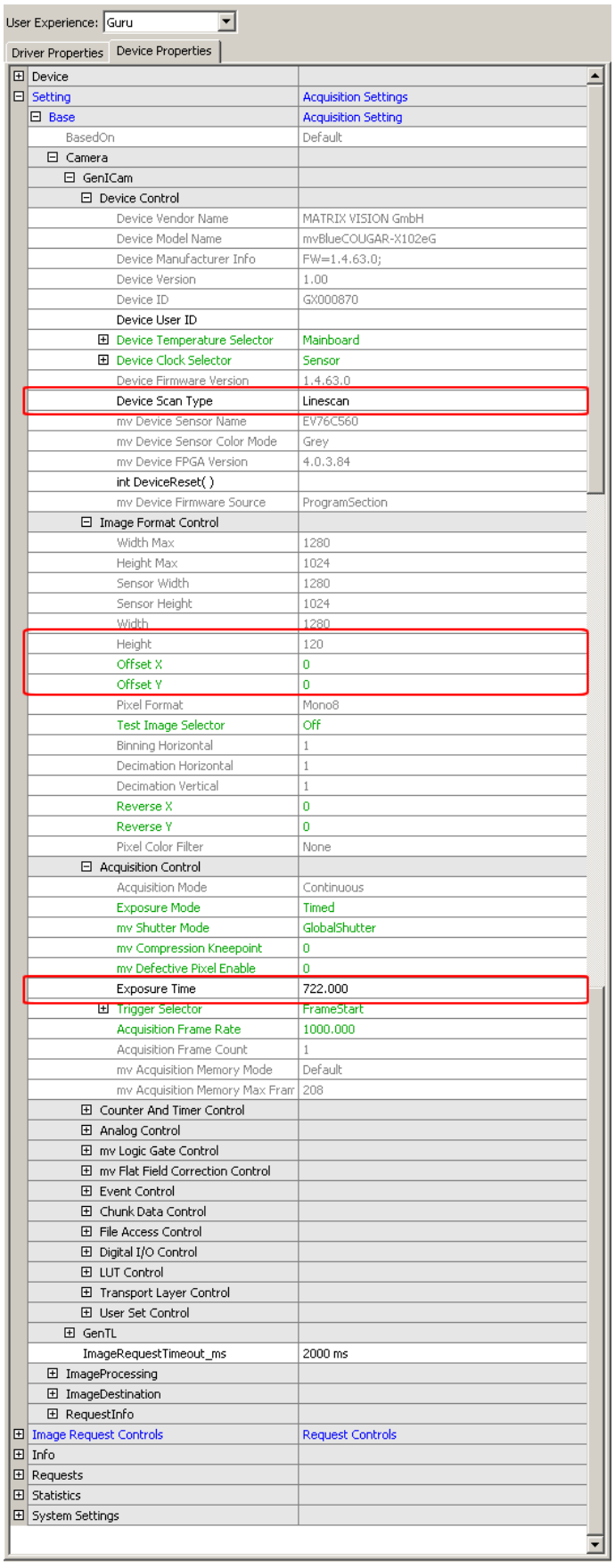
   The sensor will not get faster by windowing in x direction.

3. Finally, in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"*, please adapt the *"Exposure Time"*. For high line scan rates the exposure time has to be proportionally low. To achieve the maximum line scan rate of 12.6 kHz (gray scale sensor with rolling shutter) or 10.6 kHz (gray scale sensor with global shutter) the exposure time has to be set to lower than 62us.

You may use longer exposure times at the expense of scanning frequency.

Note

Using more than one line e.g. 20, is like having an area scan with ImageHeight = 20. In line scan mode, the min. ImageHeight is at least 16.



wxPropView settings for line scan usage

Note

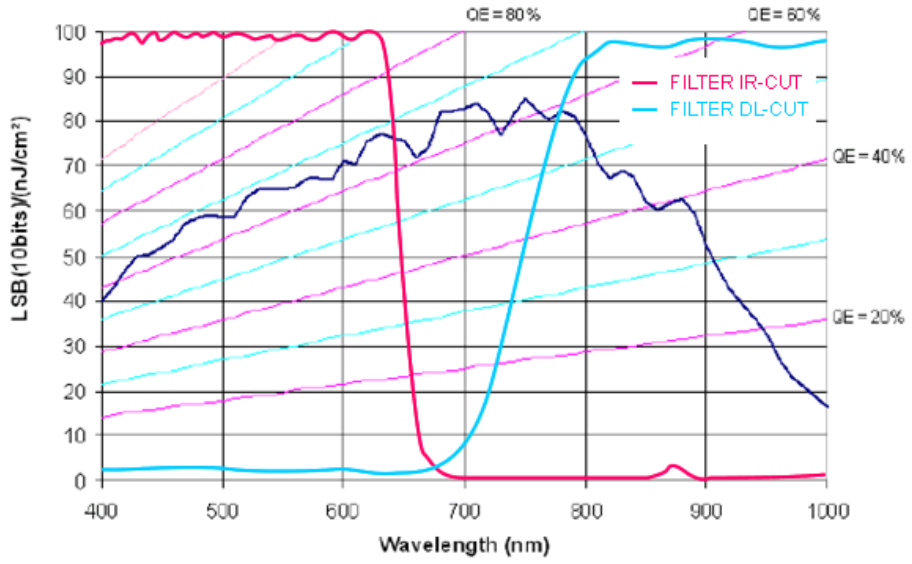
You can use either "`Continuous`" or a trigger mode as *"Acquisition Mode"* in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"*. However, if an external (line) trigger will have to be used, it is absolutely required that the external trigger is always present. During a trigger interruption, controlling or communication to the camera is not possible.

1.20.4.5.3 Sensor Data

Device Structure

- CMOS image sensor (Type 1/1.8")
- Number of effective pixels: 1280 (H) x 1024 (V)
- Unit cell size: 5.3um (H) x 5.3um (V)

1.20.4.5.3.1 Characteristics



Device Feature And Property List (p. 566)

1.20.4.5.4 Device Feature And Property List

- **mvBlueCOUGAR-X102eGE Features** (p. 566)

1.20.4.5.4.1 mvBlueCOUGAR-X102eGE Features

1.20.4.6 mvBlueCOUGAR-X104e (1.9 Mpix [1600 x 1200])

1.20.4.6.1 Introduction

The CMOS sensor module (EV76C570) incorporates the following features:

- resolution to 1600 x 1200 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- **high dynamic range** (p. 378)
- pipelined global snapshot shutter
- **Line scan mode** (p. 569)
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.6.2 Details of operation

The sensor uses a pipelined global snapshot shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.
Therefore the sensor offers two different modes of operation:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout)

1.20.4.6.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
InternalLineLength	1184
InternalADCClock	120

$$FrameTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1}{ExposureTime}$$

1.20.4.6.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

"Trigger"

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

"Erase, exposure and readout"

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

Note

Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$FrameTime = InternalLineLength * \frac{2}{InternalADCClock} * \left(\frac{ImageHeight + 10}{1000}\right)$$

$$FPS_max = \frac{1}{FrameTime + ExposureTime}$$

1.20.4.6.2.3 Line scan mode

The sensor offers also a usage as a line scan sensor. One (gray scale sensor) or two lines (in terms of color sensor) can be selected to be read out of the full line height of 1024 lines. This or these lines are grouped to a pseudo frame of selectable height in the internal buffer of the camera.

The camera then outputs these frames which contain multiples of the same scan line(s) without gaps or interruptions.

To operate in line scan mode, use the following properties:

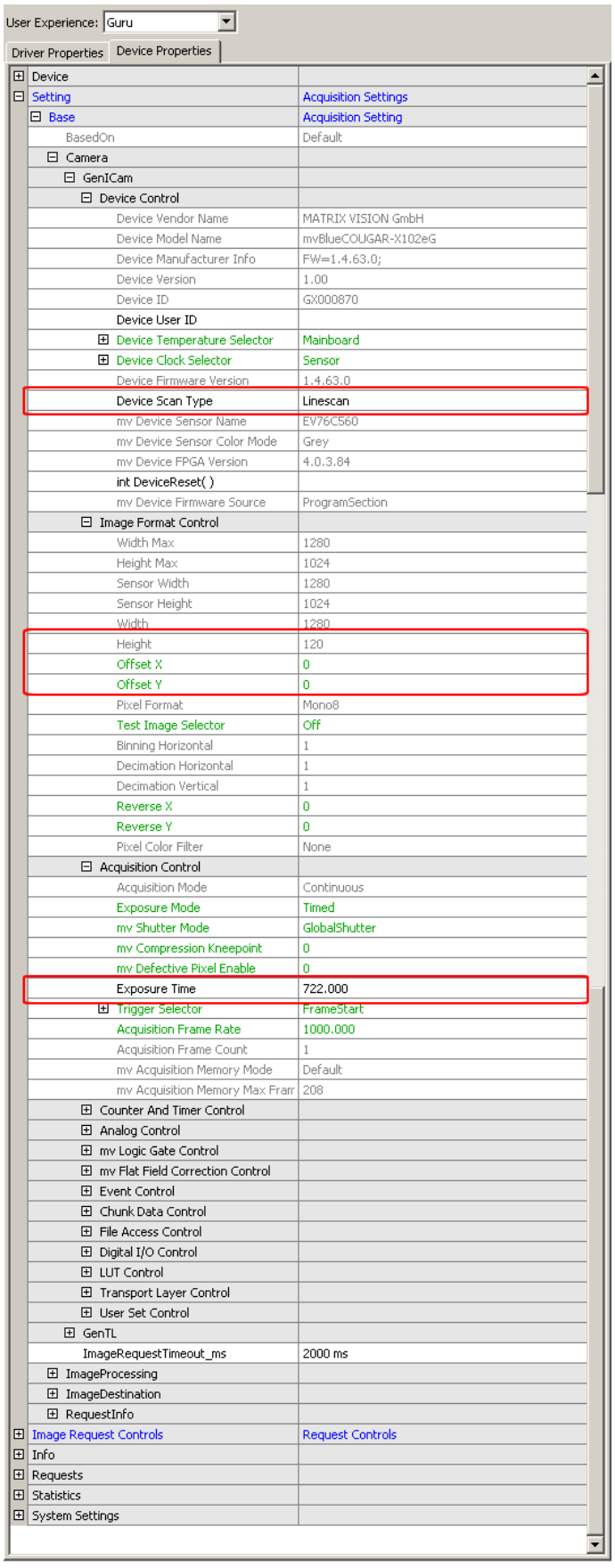
1. In *"Setting -> Base -> Camera -> GenICam -> Device Control"*, please change the *"Device Scan Type"* to **"line scan"**.
2. In *"Setting -> Base -> Camera -> GenICam -> Image Format Control"*, please set *"Width"* and *"Height"*. *"Height"* specifies how often the same line(s) will be used to reach the height of the image.
   Use *"Offset X"* to shift horizontally the starting point of the exposed line. Use *"Offset Y"* to shift the scan line vertically (typically to 512 which represents the optical center).

Note

The sensor will not get faster by windowing in x direction.

3. Finally, in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"*, please adapt the *"Exposure Time"*. For high line scan rates the exposure time has to be proportionally low. To achieve the maximum line scan rate of 12.6 kHz (gray scale sensor) the exposure time has to be set to approx. 80us.

You may use longer exposure times at the expense of scanning frequency.



wxPropView settings for line scan usage (screenshot of -x02e but the properties are similar to -x04e)

Note

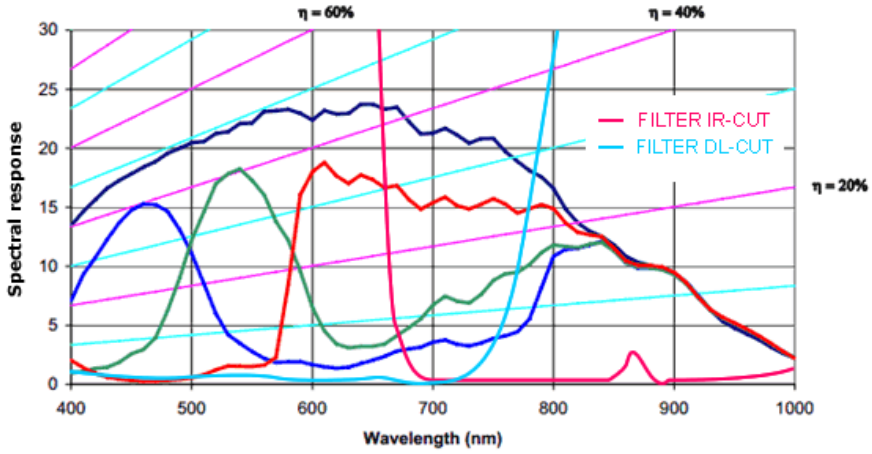
You can use either "`Continuous`" or a trigger mode as *"Acquisition Mode"* in *"Setting -> Base -> Camera -> GenICam -> Acquisition Control"*. However, if an external (line) trigger will have to be used, it is absolutely required that the external trigger is always present. During a trigger interruption, controlling or communication to the camera is not possible.

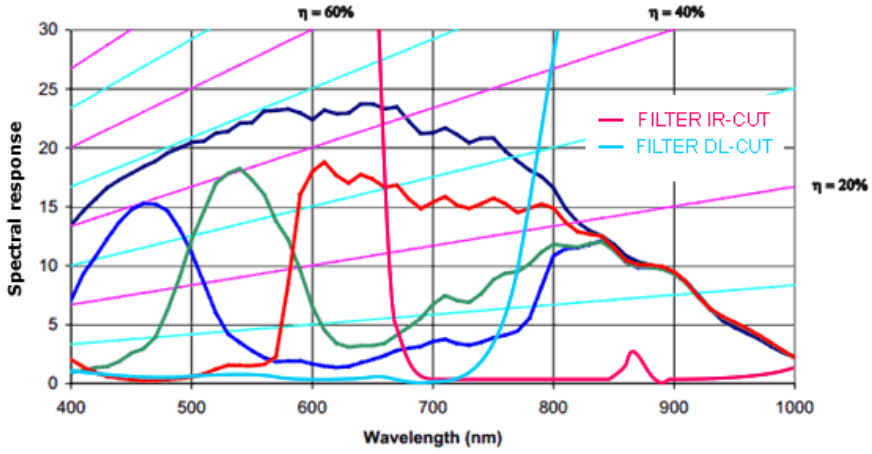
1.20.4.6.3 Sensor Data

Device Structure

- CMOS image sensor (Type 1/1.8")
- Number of effective pixels: 1600 (H) x 1200 (V)
- Unit cell size: 4.5um (H) x 4.5um (V)

1.20.4.6.3.1 Characteristics





Device Feature And Property List (p. 573)

1.20.4.6.4 Device Feature And Property List

- **mvBlueCOUGAR-X104eG Features** (p. 573)
- **mvBlueCOUGAR-X104eC Features** (p. 573)

1.20.4.6.4.1 mvBlueCOUGAR-X104eG Features

1.20.4.6.4.2 mvBlueCOUGAR-X104eC Features

1.20.4.7 mvBlueCOUGAR-X[D]x04 (2.2 Mpix [2048 x 1088])

1.20.4.7.1 Introduction

The CMOS sensor module (CMV2000) incorporates the following features:

- resolution to 2048 x 1088 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- programmable analog gain (0..18 dB)
- progressive scan sensor (no interlaced problems!)
- pipelined global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.7.2 Details of operation

The sensor uses a pipelined global snapshot shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.
Therefore the sensor offers two different modes of operation:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout)

1.20.4.7.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula (in us):

Name	Value
LVDSOutputs	16 (mvBlueCOUGAR-XD)
LVDSOutputs	2 (mvBlueCOUGAR-X)

129 32

$$FrameOverheadTime = \frac{}{PixelClock} * (10 + \frac{}{LVDSOutputs})$$

$$ReadoutTime = \frac{129}{PixelClock} * \frac{16 * ImageHeight}{LVDSOutputs}$$

$$FrameTime = FrameOverheadTime + ReadoutTime$$

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1000000}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1000000}{ExposureTime + FrameOverheadTime}$$

1.20.4.7.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

1.20.4.7.2.3 Trigger

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge"		

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.4.7.2.4 Erase, exposure and readout

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

Note

Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula (in us):

$$FrameTime = \frac{129}{PixelClock} * \left(10 + \frac{32 + (16 * ImageHeight)}{LVDSOutputs} \right)$$

$$FPS_max = \frac{1000000}{FrameTime + ExposureTime}$$

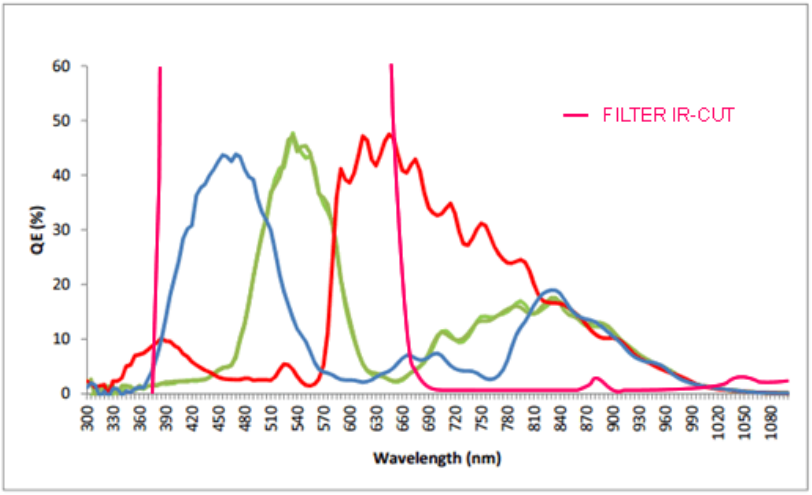
1.20.4.7.3 Sensor Data

Device Structure

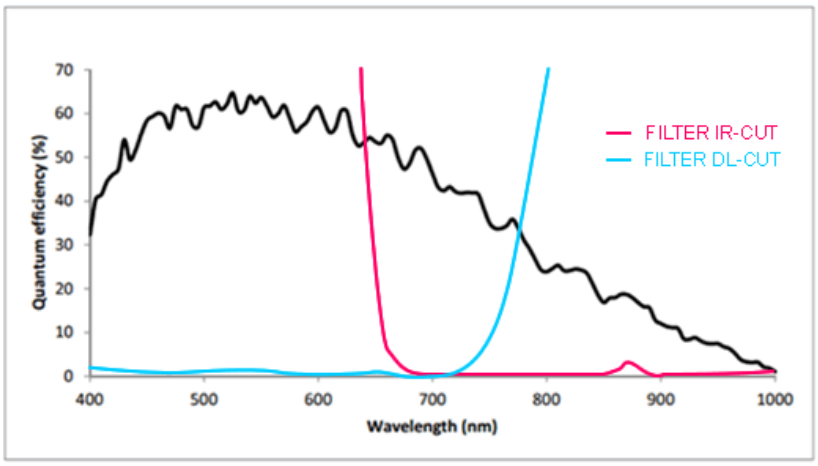
- CMOS image sensor (Type 2/3")
- Unit cell size: 5.5um (H) x 5.5um (V)

1.20.4.7.3.1 Characteristics

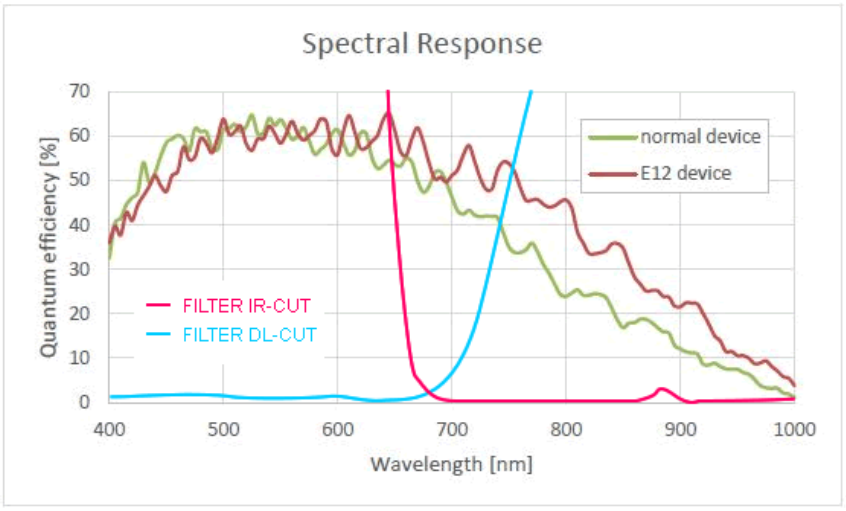
1.20.4.7.3.2 Color version



1.20.4.7.3.3 Gray scale version



1.20.4.7.3.4 Gray scale version (-XD104a12)



Device Feature And Property List (p. 576)

1.20.4.7.4 Device Feature And Property List

- **mvBlueCOUGAR-X104G Features** (p. 577)
- **mvBlueCOUGAR-X104C Features** (p. 577)
- **mvBlueCOUGAR-XD104G Features** (p. 577)
- **mvBlueCOUGAR-XD104C Features** (p. 577)

1.20.4.7.4.1 mvBlueCOUGAR-X104G Features

1.20.4.7.4.2 mvBlueCOUGAR-X104C Features

1.20.4.7.4.3 mvBlueCOUGAR-XD104G Features

1.20.4.7.4.4 mvBlueCOUGAR-XD104C Features

1.20.4.8 mvBlueCOUGAR-X[D]x04b (4.2 Mpix [2048 x 2048])

1.20.4.8.1 Introduction

The CMOS sensor module (CMV4000) incorporates the following features:

- resolution to 2048 x 2048 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- programmable analog gain (0..18 dB)
- progressive scan sensor (no interlaced problems!)
- pipelined global shutter
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.8.2 Details of operation

The sensor uses a pipelined global snapshot shutter, i.e. light exposure takes place on all pixels in parallel, although subsequent readout is sequential.
Therefore the sensor offers two different modes of operation:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout)

1.20.4.8.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula (in us):

Name	Value
LVDSOutputs	16 (mvBlueCOUGAR-XD)
LVDSOutputs	2 (mvBlueCOUGAR-X)

$$FrameOverheadTime = \frac{129}{PixelClock} * \left(20 + \frac{32}{LVDSOutputs} \right)$$

```
                  129          16 * ImageHeight
ReadoutTime = ------------ * ------------------
               PixelClock        LVDSOutputs


FrameTime = FrameOverheadTime + ReadoutTime
```

If exposure time is lower than frame time:

```
             1000000
FPS_max = -----------
            FrameTime
```

If exposure time is greater than frame time:

```
                      1000000
FPS_max = ----------------------------------
            ExposureTime + FrameOverheadTime
```

1.20.4.8.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

1.20.4.8.2.3 Trigger

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.4.8.2.4 Erase, exposure and readout

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

Note

Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula (in us):

$$FrameTime = \frac{129}{PixelClock} * (20 + \frac{32 + (16 * ImageHeight)}{LVDSOutputs})$$

$$FPS_max = \frac{1000000}{FrameTime + ExposureTime}$$

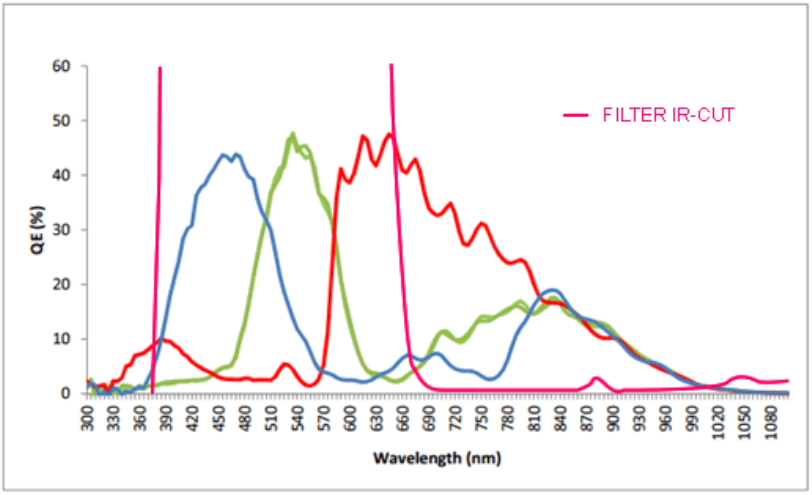
1.20.4.8.3 Sensor Data

Device Structure

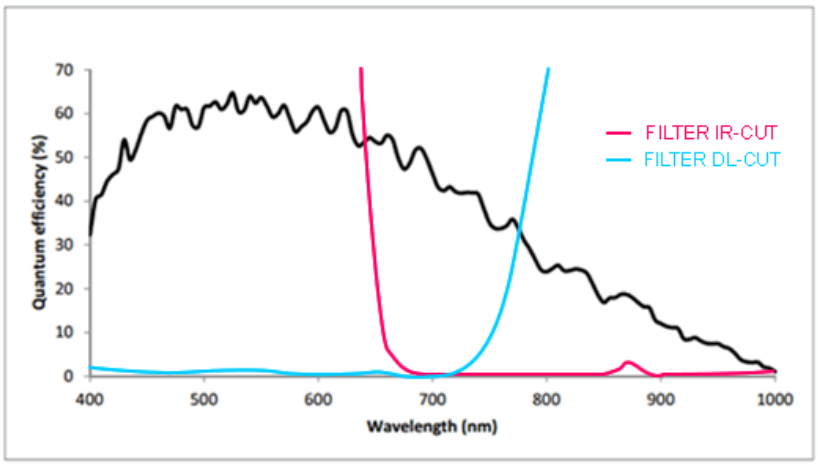
- CMOS image sensor (Type 1")
- Unit cell size: 5.5um (H) x 5.5um (V)

1.20.4.8.3.1 Characteristics

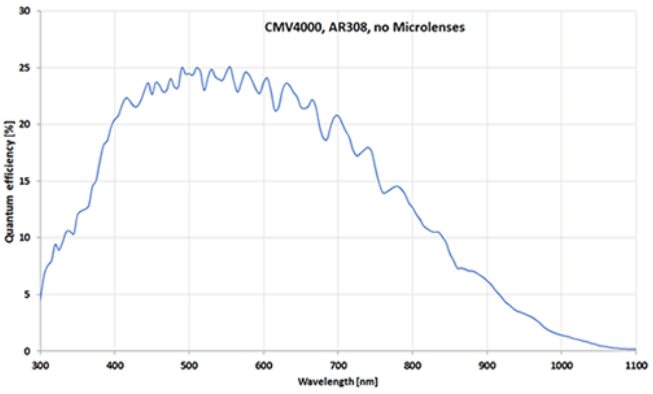
1.20.4.8.3.2 Color version



1.20.4.8.3.3 Gray scale version



1.20.4.8.3.4 Gray scale version (-X104bUV)



Device Feature And Property List (p. 580)

1.20.4.8.4 Device Feature And Property List

- **mvBlueCOUGAR-X104bG Features** (p. 581)
- **mvBlueCOUGAR-X104bC Features** (p. 581)
- **mvBlueCOUGAR-XD104bG Features** (p. 581)
- **mvBlueCOUGAR-XD104bC Features** (p. 581)

1.20.4.8.4.1 mvBlueCOUGAR-X104bG Features

1.20.4.8.4.2 mvBlueCOUGAR-X104bC Features

1.20.4.8.4.3 mvBlueCOUGAR-XD104bG Features

1.20.4.8.4.4 mvBlueCOUGAR-XD104bC Features

1.20.4.9 mvBlueCOUGAR-X105 (5 Mpix [2592 x 1944])

1.20.4.9.1 Introduction

The CMOS sensor module (MT9P031) incorporates the following features:

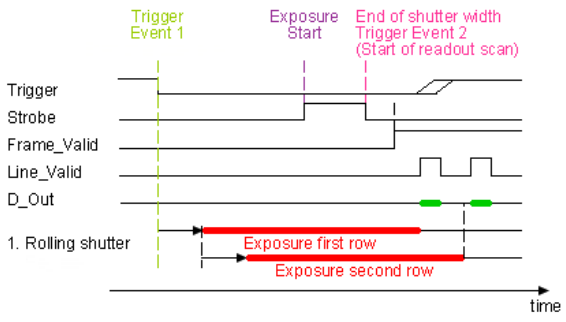
- resolution to 2592 x 1944 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- programmable analog gain (0..12 dB)
- progressive scan sensor (no interlaced problems!)
- rolling shutter / global reset
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.9.2 Details of operation

The sensor uses two acquisition modes:

- **rolling shutter** (mvShutterMode = "`RollingShutter`") and
- **global reset release shutter** (mvShutterMode = "`GlobalResetRelease`").

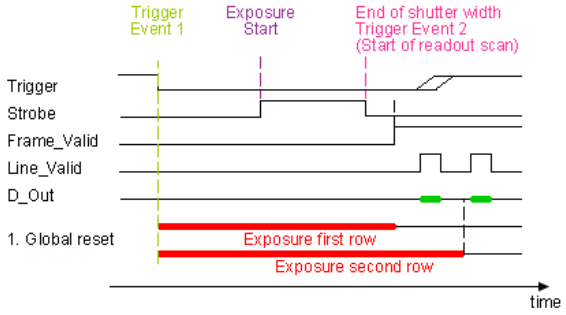
With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time:



Note

Moving objects together with a rolling shutter can cause a smear in moving objects.

The **global reset release shutter**, which is only available in triggered operation, starts the exposure of all rows simultaneously and the reset to each row is released simultaneously, too. However, the readout of the lines is equal to the readout of the rolling shutter: line by line:



Note

This means, the bottom lines of the sensor will be exposed to light longer! For this reason, this mode will only make sense, if there is no extraneous light and the flash duration is shorter or equal to the exposure time.

Furthermore, the sensor offers two operating modes:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout) in triggered operation

1.20.4.9.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

```
FrameTime = (ImageHeight * RowTime) + (25 * RowTime)
```

If exposure time is lower than frame time:

$$FPS_{max} = \frac{1}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_{max} = \frac{1}{ExposureTime}$$

1.20.4.9.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

1.20.4.9.2.3 Trigger

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.4.9.2.4 Erase, exposure and readout

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

Note

Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

```
FrameTime = (ImageHeight * RowTime) + (25 * RowTime)
```

$$FPS_max = \frac{1}{FrameTime + ExposureTime}$$

Name	Value
RowTime	35.63 us @ 98 MHz
RowTime	43.65 us @ 80 MHz

1.20.4.9.2.5 Use Cases

As mentioned before, **"Global reset release"** will only make sense, if a flash is used which is brighter than the ambient light. The settings in **wxPropView** (p. 139) will look like this:

FlashMode	"Digout0"
FlashType	RollingShutterFlash
TestMode	Off
ShutterMode	GlobalResetRelease

In this case, *DigOut0* gets a high signal as long as the exposure time (which is synchronized with the GlobalReset↩ Release). This signal can start a flash light.

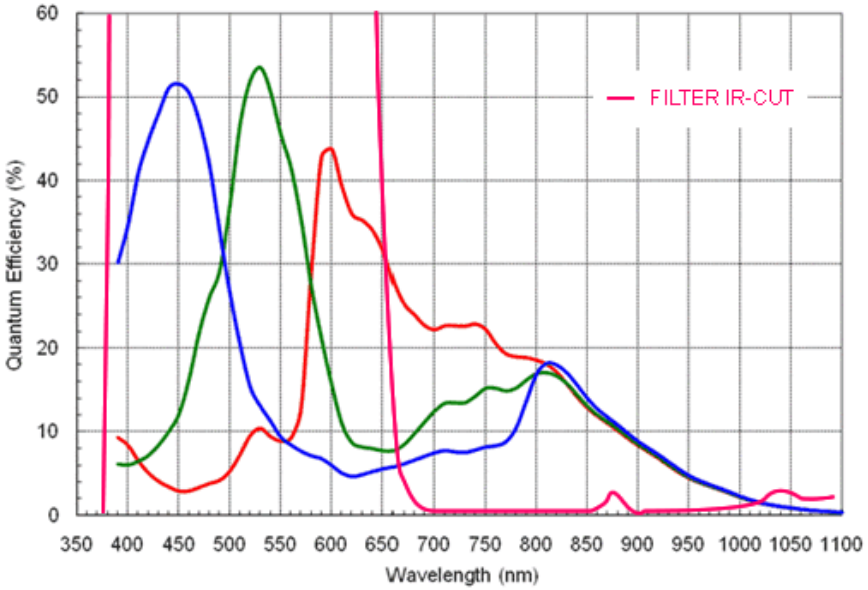
1.20.4.9.3 Sensor Data

Device Structure

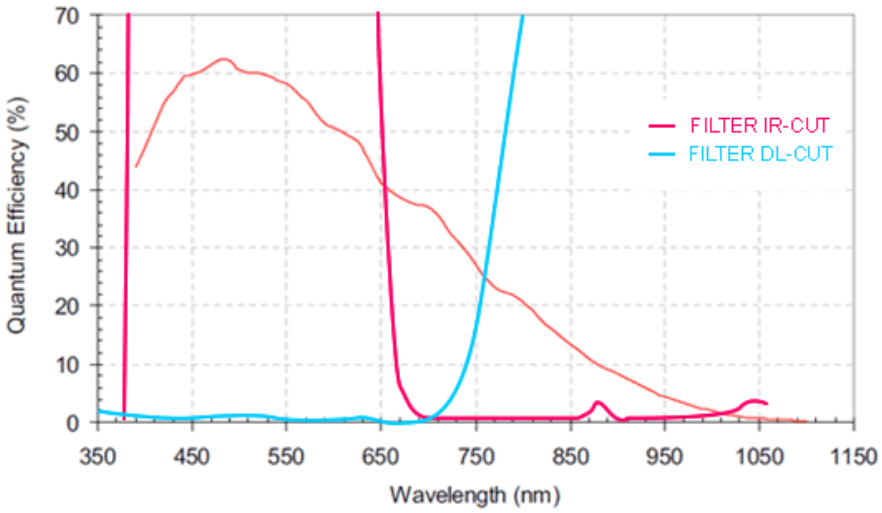
- CMOS image sensor
- Image size: 5.70(H)x4.28(V)mm (Type 1/2.5")
- Number of effective pixels: 2592 (H) x 1944 (V)
- Unit cell size: 2.2um (H) x 2.2um (V)

1.20.4.9.3.1 Characteristics

1.20.4.9.3.2 Color version



1.20.4.9.3.3 Gray scale version



Device Feature And Property List (p. 585)

1.20.4.9.4 Device Feature And Property List

- **mvBlueCOUGAR-X105G Features** (p. 585)
- **mvBlueCOUGAR-X105C Features** (p. 586)

1.20.4.9.4.1 mvBlueCOUGAR-X105G Features

1.20.4.9.4.2 mvBlueCOUGAR-X105C Features

1.20.4.10 mvBlueCOUGAR-X1010 (10 Mpix [3856 x 2764])

1.20.4.10.1 Introduction

The CMOS sensor module (MT9J003) incorporates the following features:

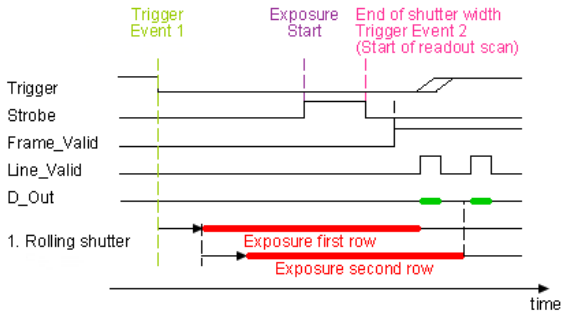
- resolution to 3856 x 2764 gray scale or RGB Bayer mosaic
- supports window AOI mode with faster readout
- programmable analog gain (0..12 dB)
- progressive scan sensor (no interlaced problems!)
- rolling shutter / global reset
- programmable readout timing with free capture windows and partial scan
- many trigger modes (free-running, hardware-triggered)

1.20.4.10.2 Details of operation

The sensor uses two acquisition modes:

- **rolling shutter** (mvShutterMode = "`ElectronicRollingShutter`") and
- **global reset release shutter** (mvShutterMode = "`GlobalResetRelease`").

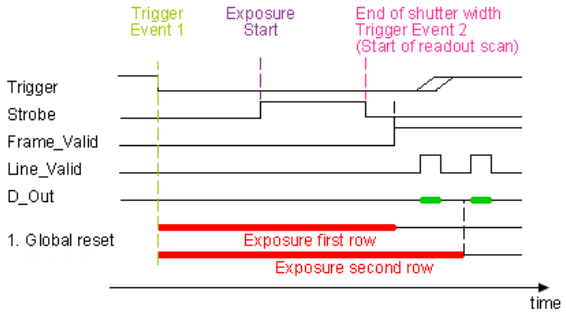
With the **rolling shutter** the lines are exposed for the same duration, but at a slightly different point in time:



Note

Moving objects together with a rolling shutter can cause a shear in moving objects.

The **global reset release shutter**, which is only available in triggered operation, starts the exposure of all rows simultaneously and the reset to each row is released simultaneously, too. However, the readout of the lines is equal to the readout of the rolling shutter: line by line:



Note

This means, the bottom lines of the sensor will be exposed to light longer! For this reason, this mode will only make sense, if there is no extraneous light and the flash duration is shorter or equal to the exposure time.

Furthermore, the sensor offers two operating modes:

- free running mode (Overlapping exposure and readout)
- snapshot mode (Sequential exposure and readout) in triggered operation

1.20.4.10.2.1 Free running mode

In free running mode, the sensor reaches its maximum frame rate. This is done by overlapping erase, exposure and readout phase. The sensor timing in free running mode is fixed, so there is no control when to start an acquisition. This mode is used with trigger mode **Continuous**.

To calculate the maximum frames per second (FPS_{max}) in free running mode you will need following formula:

Name	Value
PixelClock	81.25
VerticalBlank	143

$$FrameTime = \frac{(ImageHeight * (1648)) + (VerticalBlank * 1648)}{PixelClock}$$

If exposure time is lower than frame time:

$$FPS_max = \frac{1000000}{FrameTime}$$

If exposure time is greater than frame time:

$$FPS_max = \frac{1000000}{ExposureTime}$$

1.20.4.10.2.2 Snapshot mode

In snapshot mode, the image acquisition process consists off several sequential phases:

1.20.4.10.2.3 Trigger

Snapshot mode starts with a trigger. This can be either a hardware or a software signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.4.10.2.4 Erase, exposure and readout

All pixels are light sensitive at the same period of time. The whole pixel core is reset simultaneously and after the exposure time all pixel values are sampled together on the storage node inside each pixel. The pixel core is read out line-by-line after exposure.

Note

Exposure and read out cycle is carry-out in serial; that causes that no exposure is possible during read out.

The step width for the exposure time is 1 us.

Image data is then shifted out line-by-line and transferred to memory.

To calculate the maximum frames per second (FPS_{max}) in snapshot mode you will need following formula:

$$FrameTime = \frac{(ImageHeight * (3980)) + (VerticalBlank * 3980)}{PixelClock}$$

$$FPS_max = \frac{1}{FrameTime}$$

Note

The maximum duration of the signal *ExposureActive* used in *Counter*, *Timer*, or as digital output signal is limited to 172ms (96.8 MHz pixel clock) or 207ms (80 MHz pixel clock). These limits also apply to the **chunk data** (p. 154) parameter *ChunkExposureTime*. It does not matter which exposure time is set, the value of this parameter will always be below or equal to 172ms (96.8 MHz pixel clock) or 207ms (80 MHz pixel clock). ExposureEndEvents are generated with the help of this signal, too and for this reason they have too small timestamps above this limit.

1.20.4.10.2.5 Use Cases

As mentioned before, **"Global reset release"** will only make sense, if a flash is used which is brighter than the ambient light. The settings in **wxPropView** (p. 139) will look like this:

FlashMode	"Digout0"
FlashType	RollingShutterFlash
TestMode	Off
ShutterMode	GlobalResetRelease

In this case, *DigOut0* gets a high signal as long as the exposure time (which is synchronized with the GlobalReset↩ Release). This signal can start a flash light.

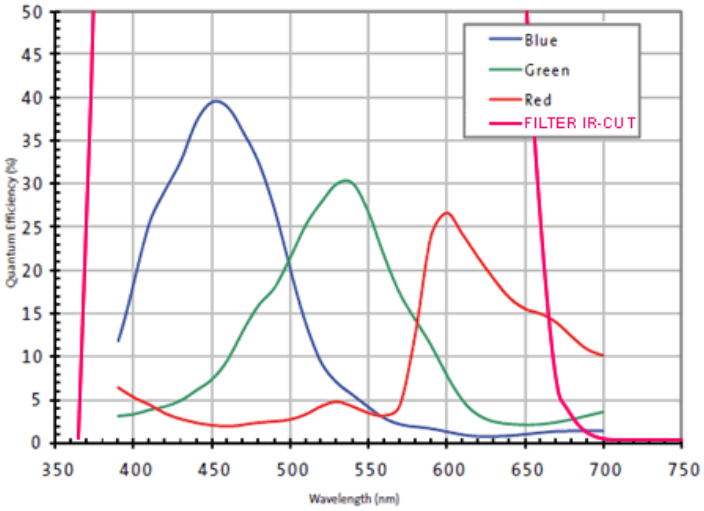
1.20.4.10.3 Sensor Data

Device Structure

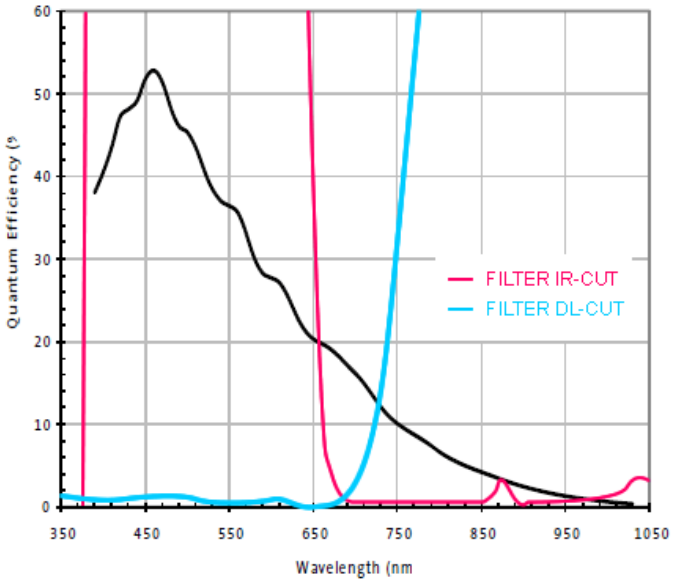
- CMOS image sensor
- Image size: 6.119(H)x4.589(V)mm (Type 1/2.3")
- Number of effective pixels: 3856 (H) x 2764 (V)
- Unit cell size: 1.67um (H) x 1.67um (V)

1.20.4.10.3.1 Characteristics

1.20.4.10.3.2 Color version



1.20.4.10.3.3 Gray scale version



Device Feature And Property List (p. 592)

1.20.4.10.4 Device Feature And Property List

- **mvBlueCOUGAR-X1010G Features** (p. 592)
- **mvBlueCOUGAR-X1010C Features** (p. 592)

1.20.4.10.4.1 mvBlueCOUGAR-X1010G Features

1.20.4.10.4.2 mvBlueCOUGAR-X1010C Features

1.20.5 A.5 CCD

- **mvBlueCOUGAR-X120a (0.3 Mpix [640 x 480])** (p. 593)
- **mvBlueCOUGAR-X120b (0.3 Mpix [640 x 480])** (p. 599)
- **mvBlueCOUGAR-X120d (0.5 Mpix [776 x 580])** (p. 605)
- **mvBlueCOUGAR-X122 (1.2 Mpix [1280 x 960])** (p. 611)
- **mvBlueCOUGAR-X123 (1.4 Mpix [1360 x 1024])** (p. 617)
- **mvBlueCOUGAR-X124 (1.9 Mpix [1600 x 1200])** (p. 624)
- **mvBlueCOUGAR-XD124a (3 Mpix [1936 x 1460])** (p. 630)
- **mvBlueCOUGAR-X225 (5.1 Mpix [2448 x 2050])** (p. 637)
- **mvBlueCOUGAR-X125a (5.1 Mpix [2448 x 2050])** (p. 642)
- **mvBlueCOUGAR-XD126 (6 Mpix [2752 x 2208])** (p. 649)
- **mvBlueCOUGAR-XD126a (6 Mpix [2752 x 2208])** (p. 655)
- **mvBlueCOUGAR-XD129 (9.2 Mpix [3384 x 2712])** (p. 661)
- **mvBlueCOUGAR-XD129a (9.2 Mpix [3384 x 2712])** (p. 667)
- **mvBlueCOUGAR-XD1212a (12.1 Mpix [4248 x 2836])** (p. 673)

1.20.5.1 mvBlueCOUGAR-X120a (0.3 Mpix [640 x 480])

1.20.5.1.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image integration start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame integration, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time integration for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.1.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.1.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = LevelLow"` `"ExposureMode = Timed"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

See also

For detailed description about the trigger modes (`https://www.matrix-vision/manuals/` [mv↩ IMPACT Acquire API])

- **C: TCameraTriggerMode**
- **C++**: **mvIMPACT::acquire::TCameraTriggerMode**

Note

Trigger modes which use an external input (**ctmOnLowLevel**, **ctmOnHighLevel**, **ctmOnRisingEdge**, **ctmOnFallingEdge**) will use digital input 0 as input for the trigger signal. Input 0 is not restricted to the trigger function. It can always also be used as general purpose digital input. The input switching threshold of all inputs can be programmed with write_dac(level_in_mV). The best is to set this to the half of the input voltage. So for example if you apply a 24V switching signal to the digital inputs set the threshold to 12000 mV.

1.20.5.1.2.2 Exposure aka Integration

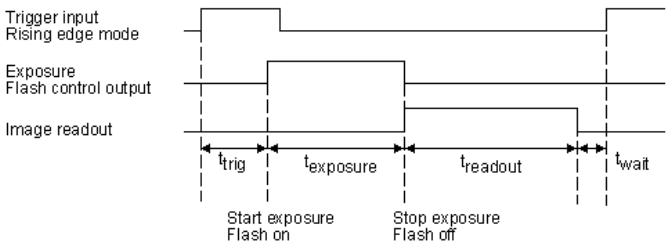
After an active trigger, the integration phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.1.2.3 Readout

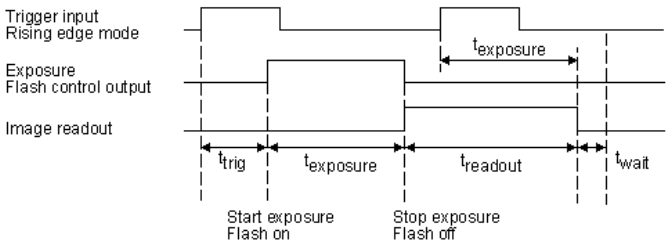
When integration is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes tvshift, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.1.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period ealier than $t_{readout}$ is finished.

Name	Description	Pixel clock		
		24.54 MHz	32 MHz	40 MHz
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)		
t_{wait}	Time after a new trigger is accepted	$<$ 1us		
t_{trans}	Image transfer time (move image to readout cells in CCD)	7.99us	6.13us	4.90us
$t_{readline}$	time needed to readout a line	31.95us	24.50us	19.60us
t_{vshift}	time needed to shift unused lines away	2.93us	2.25us	1.80us
$t_{exposure}$	Exposure time	10us..10s	10us..10s	10us..10s
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout} = ((ActiveLines + 1) * t_{readline})) + ((504 - ActiveLines) * t_{vshift}) + t_{trans}$		

1.20.5.1.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 480 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

$$FPS_max = \frac{1}{t_readout + t_exposure}$$

(Expose mode: Overlapped):

$$t_readout < t_exposure: \quad FPS_max = \frac{1}{t_exposure}$$

$$t_readout > t_exposure: \quad FPS_max = \frac{1}{t_readout}$$

1.20.5.1.3.2 Example: Frame rate as function of lines & exposure time

Now, when we insert the values using exposure time of, for example, 8000 us, 480 lines and 40MHz pixel clock (Expose mode: No overlap):

$$FPS_max = \frac{1}{(((480 + 1) * 19.60\ us) + ((504 - 480) * 1.80\ us) + 4.90\ us) + 8000\ us}$$

$$= 0.00005722231441372878 \quad 1 / us$$

$$= 57.2$$

1.20.5.1.3.3 Frame rate calculator

Note

The calculator returns the max. frame rate supported by the sensor. Please keep in mind that it will depend on the interface and the used image format if this frame rate can be transferred.

1.20.5.1.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing Trigger Modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.1.5 CCD Sensor Data

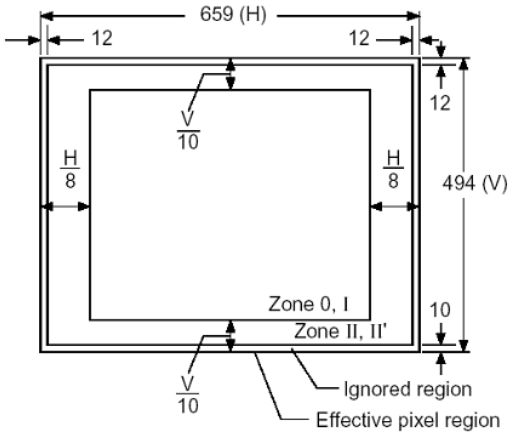
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 6mm (Type 1/3)
- Number of effective pixels: 659 (H) x 494 (V) approx. 330K pixels
- Total number of pixels: 692 (H) x 504 (V) approx. 350K pixels
- Chip size: 5.79mm (H) x 4.89mm (V)
- Unit cell size: 7.4um (H) x 7.4um (V)
- Optical black:
  - Horizontal (H) direction: Front 2 pixels, rear 31 pixels
  - Vertical (V) direction: Front 8 pixels, rear 2 pixels
- Number of dummy bits: Horizontal 16 Vertical 5
- Substrate material: Silicon

1.20.5.1.5.1 Characteristics

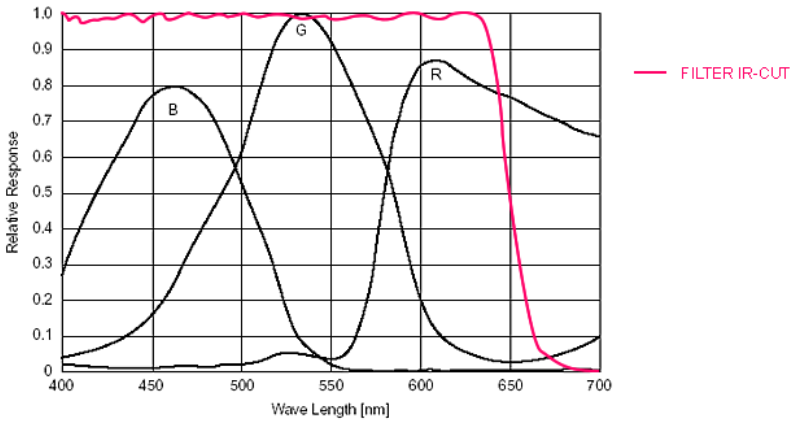
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading



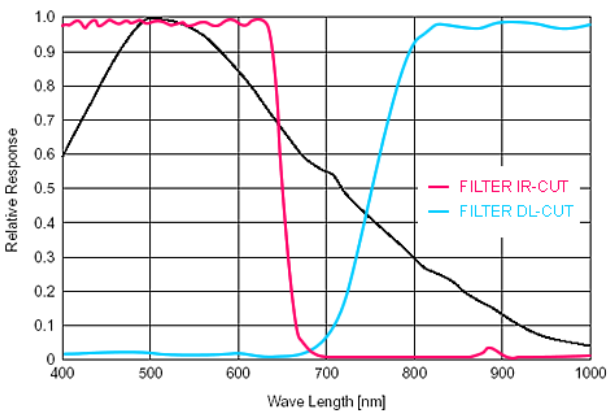
1.20.5.1.5.2 Color version

Spectral Sensitivity Characteristics (Includes lens characteristics, excludes light source characteristics)



1.20.5.1.5.3 Gray scale version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



Device Feature And Property List (p. 599)

1.20.5.1.6 Device Feature And Property List

- **mvBlueCOUGAR-X120aG Features** (p. 599)
- **mvBlueCOUGAR-X120aC Features** (p. 599)

1.20.5.1.6.1 mvBlueCOUGAR-X120aG Features

1.20.5.1.6.2 mvBlueCOUGAR-X120aC Features

1.20.5.2 mvBlueCOUGAR-X120b (0.3 Mpix [640 x 480])

1.20.5.2.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.2.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.2.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.2.2.2 Exposure aka Integration

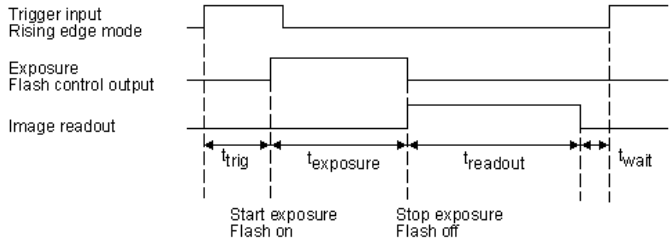
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.2.2.3 Readout

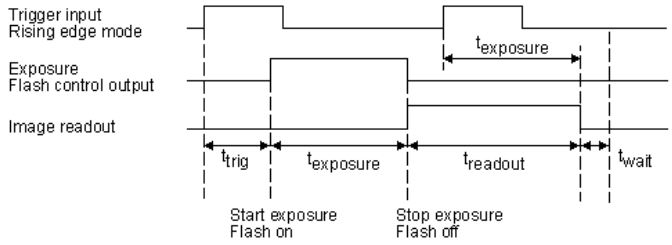
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.2.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock		
		24.54 MHz	32 MHz	40 MHz
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)		
t_{wait}	Time after a new trigger is accepted	$<$ 1us		
t_{trans}	Image transfer time (move image to readout cells in CCD)	7.99us	6.13us	4.90us
$t_{readline}$	time needed to readout a line	31.95us	24.50us	19.60us
t_{vshift}	time needed to shift unused lines away	2.93us	2.25us	1.80us
$t_{exposure}$	Exposure time	10us..20s	10us..20s	10us..20s
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout} = ((ActiveLines + 1) * t_{readline})) + ((504 - ActiveLines) * t_{vshift}) + t_{trans}$		

1.20.5.2.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 480 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                         t_readout
```

1.20.5.2.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 480 lines and 40MHz pixel clock:

```
FPS_max =                                  1
          ------------------------------------------------------------------------
          (((480 + 1) * 19.60 us) + ((504 - 480) * 1.80 us) + 4.90 us) + 8000 us
        = 0.00005722231441372878   1 / us
        = 57.2
```

1.20.5.2.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing trigger modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.2.5 CCD Sensor Data

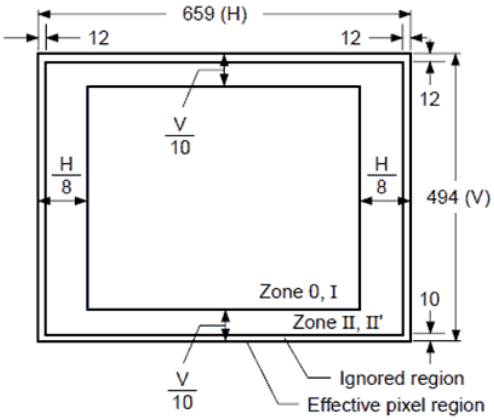
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 6mm (Type 1/2)
- Number of effective pixels: 659 (H) x 494 (V) approx. 330 K pixels
- Total number of pixels: 692 (H) x 504 (V) approx. 350 K pixels
- Chip size: 7.48mm (H) x 6.15mm (V)
- Unit cell size: 9.9um (H) x 9.9um (V)
- Optical black:
  - Horizontal (H) direction: Front 2 pixels, rear 31 pixels
  - Vertical (V) direction: Front 8 pixels, rear 2 pixels
- Number of dummy bits: Horizontal 16 Vertical 5
- Substrate material: Silicon

1.20.5.2.5.1 Characteristics

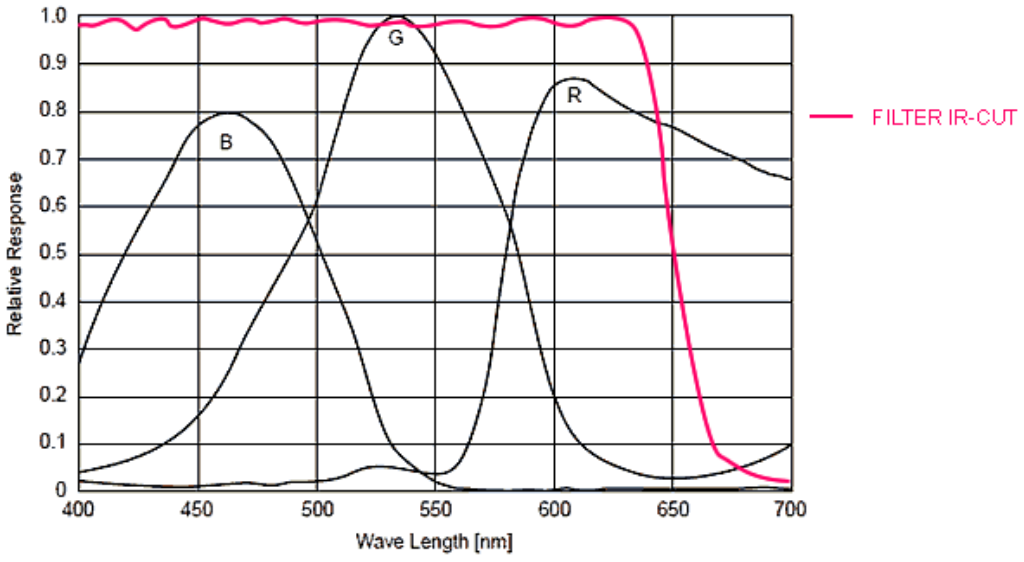
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading



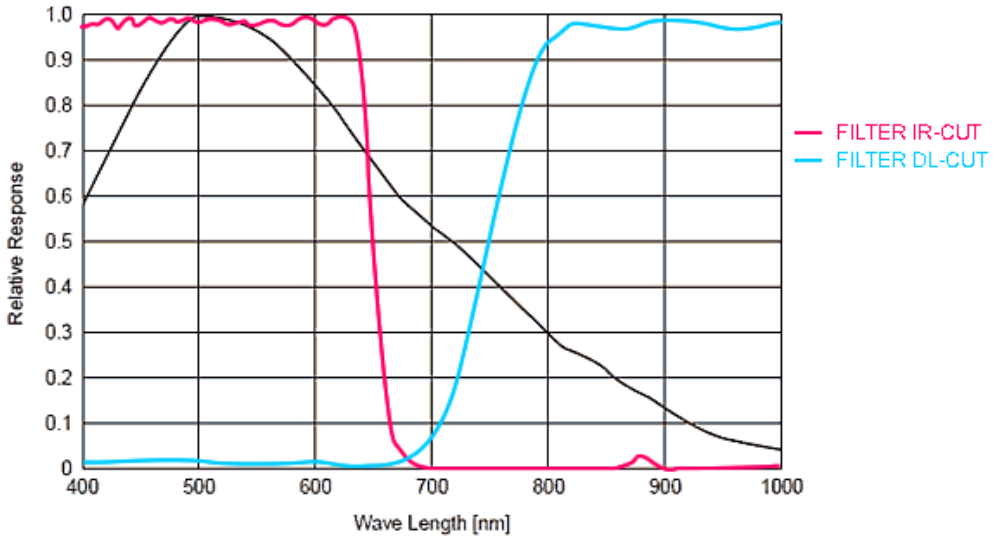
1.20.5.2.5.2 Color version

Spectral Sensitivity Characteristics (Includes lens characteristics, excludes light source characteristics)



1.20.5.2.5.3 Gray scale version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



Device Feature And Property List (p. 605)

1.20.5.2.6 Device Feature And Property List

- **mvBlueCOUGAR-X120bG Features** (p. 605)
- **mvBlueCOUGAR-X120bC Features** (p. 605)

1.20.5.2.6.1 mvBlueCOUGAR-X120bG Features

1.20.5.2.6.2 mvBlueCOUGAR-X120bC Features

1.20.5.3 mvBlueCOUGAR-X120d (0.5 Mpix [776 x 580])

1.20.5.3.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.3.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.3.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.3.2.2 Exposure aka Integration

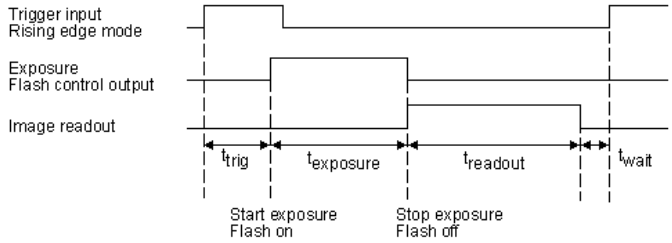
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.3.2.3 Readout

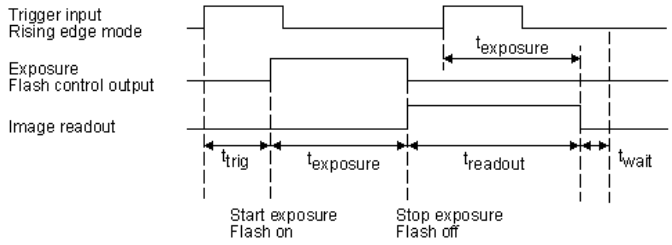
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.3.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock		
		29.5 MHz	48 MHz	
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)		
t_{wait}	Time after a new trigger is accepted	$<$ 1us		
t_{trans}	Image transfer time (move image to readout cells in CCD)	30.51us	18.75us	
$t_{readline}$	time needed to readout a line	32us	19.67us	
t_{vshift}	time needed to shift unused lines away	3.25us	2us	
$t_{exposure}$	Exposure time	10us..20s	10us..20s	
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout} = ((ActiveLines + 1) * t_{readline})) + ((592 - ActiveLines) * t_{vshift}) + t_{trans}$		

1.20.5.3.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 580 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                         t_readout
```

1.20.5.3.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 580 lines and 48MHz pixel clock:

```
FPS_max =                          1
          ------------------------------------------------------------
          (((580 + 1) * 19.67 us) + ((592 - 580) * 2 us) + 18.75 us)
        = 0.00008717620577769021  1 / us
        = 87.2
```

1.20.5.3.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing trigger modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.3.5 CCD Sensor Data

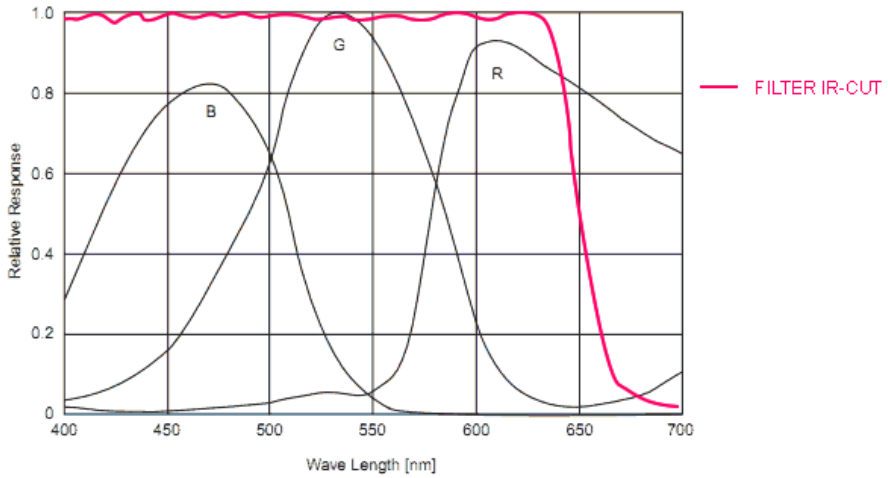
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 6mm (Type 1/2)
- Number of effective pixels: 782 (H) x 582 (V) approx. 460 K pixels
- Total number of pixels: 823 (H) x 592 (V) approx. 490 K pixels
- Chip size: 7.48mm (H) x 6.15mm (V)
- Unit cell size: 8.3um (H) x 8.3um (V)
- Optical black:
  - Horizontal (H) direction: Front 3 pixels, rear 38 pixels
  - Vertical (V) direction: Front 8 pixels, rear 2 pixels
- Number of dummy bits: Horizontal 19 Vertical 5
- Substrate material: Silicon

1.20.5.3.5.1 Characteristics

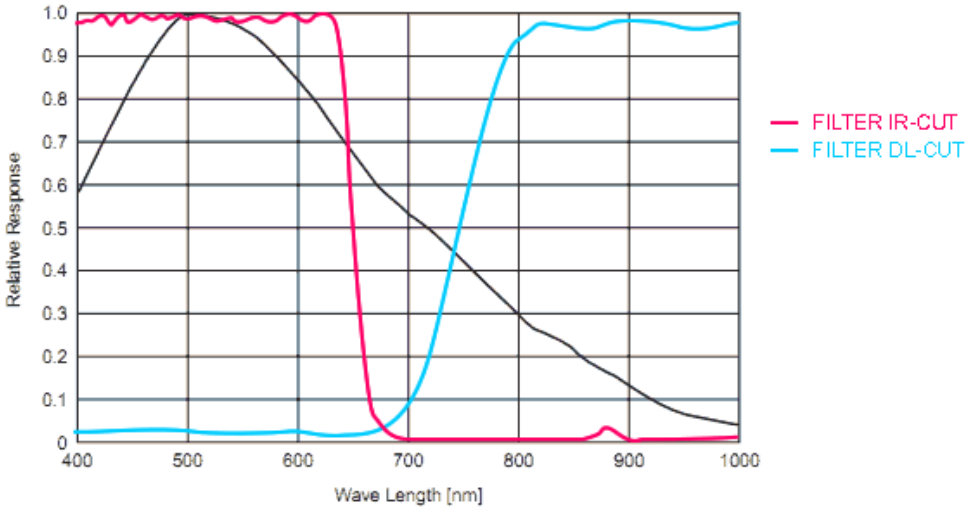
These zone definitions apply to both the color and gray scale version of the sensor.

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



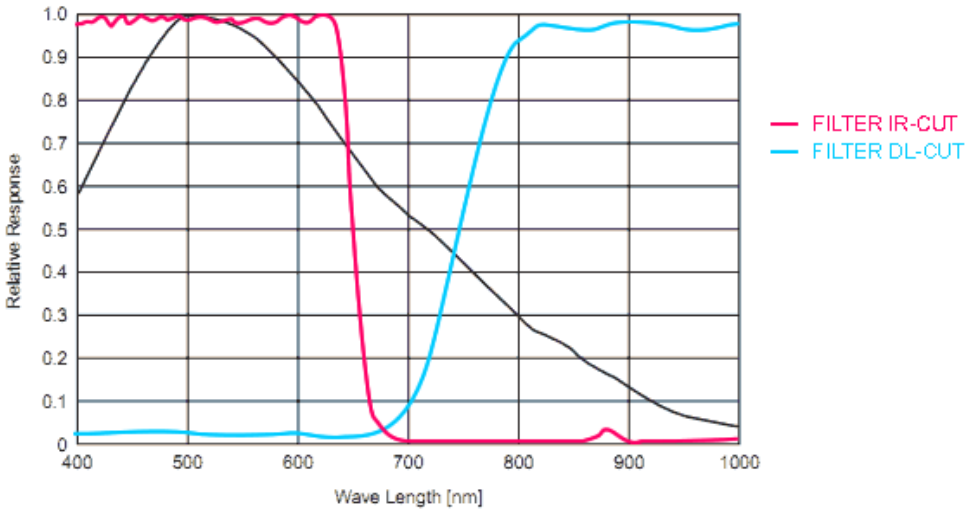
1.20.5.3.5.2 Color version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



1.20.5.3.5.3 Gray scale version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



Device Feature And Property List (p. 611)

1.20.5.3.6 Device Feature And Property List

- **mvBlueCOUGAR-X120dG Features** (p. 611)
- **mvBlueCOUGAR-X120dC Features** (p. 611)

1.20.5.3.6.1 mvBlueCOUGAR-X120dG Features

1.20.5.3.6.2 mvBlueCOUGAR-X120dC Features

1.20.5.4 mvBlueCOUGAR-X122 (1.2 Mpix [1280 x 960])

1.20.5.4.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.4.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.4.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.4.2.2 Exposure aka Exposure

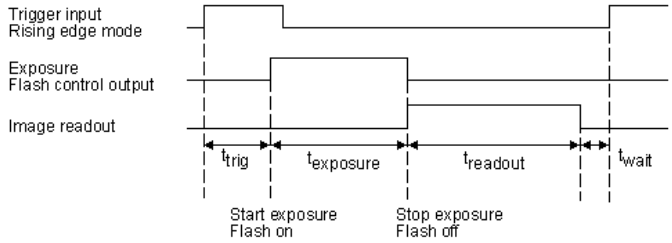
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.4.2.3 Readout

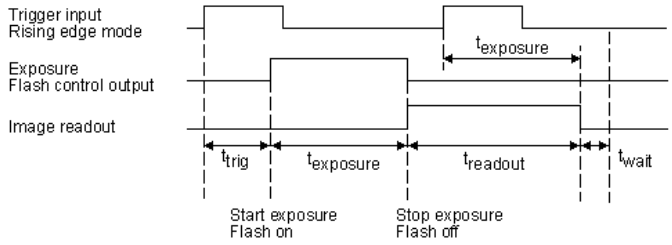
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes tvshift, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.4.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock	
		36 MHz	50 MHz
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)	
t_{wait}	Time after a new trigger is accepted	$<$ 1us	
t_{trans}	Image transfer time (move image to readout cells in CCD)	16.25us	11.70us
$t_{readline}$	time needed to readout a line	45.83us	33.00us
t_{vshift}	time needed to shift unused lines away	2.06us	1.48us
$t_{exposure}$	Exposure time	10us..10s	10us..10s
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout} = ((ActiveLines + 1) * t_{readline})) + ((976 - ActiveLines) * t_{vshift}) + t_{trans}$	

1.20.5.4.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 964 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                    ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                    ---------------
                                        t_readout
```

1.20.5.4.3.2 Example: Frame rate as function of lines & exposure time Now, when we insert the values using exposure time of, for example, 8000 us and 960 lines and 50MHz pixel clock (Expose mode: No overlap):

```
FPS_max =                                      1
          ------------------------------------------------------------------------
          (((960 + 1) * 33.00 us) + ((976 - 960) * 1.48 us) + 11.70 us) + 8000 us
        = 0.00002515825802208794   1 / us
        = 25.2
```

1.20.5.4.4 Reprogramming CCD Timing Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing Trigger Modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

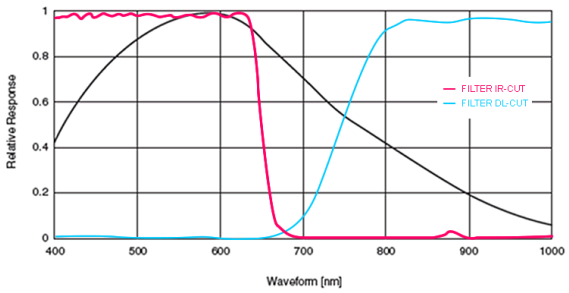
1.20.5.4.5 CCD Sensor Data
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 6mm (Type 1/3)
- Number of effective pixels: 1296 (H) x 966 (V) approx. 1.25 M pixels
- Total number of pixels: 1348 (H) x 976 (V) approx. 1.32 M pixels
- Chip size: 6.26mm (H) x 5.01mm (V)
- Unit cell size: 3.75um (H) x 3.75um (V)
- Optical black:
  - Horizontal (H) direction: Front 12 pixels, rear 40 pixels
  - Vertical (V) direction: Front 8 pixels, rear 2 pixels
- Number of dummy bits: Horizontal 4 Vertical 2
- Substrate material: Silicon

1.20.5.4.5.1 Characteristics

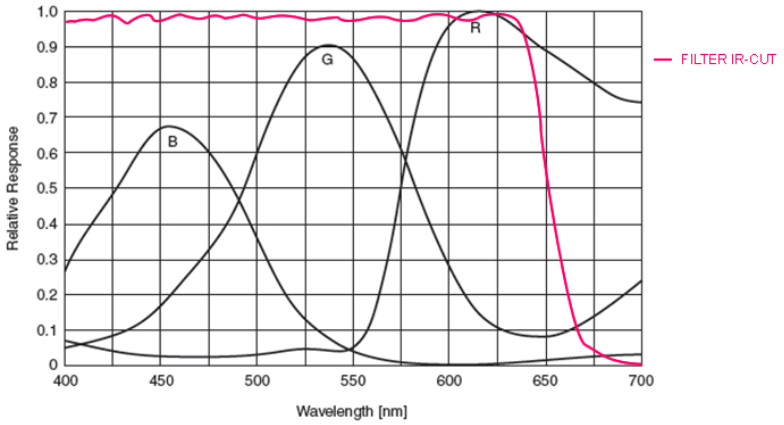
These zone definitions apply to both the color and gray scale version of the sensor.

Spectral Sensitivity Characteristics (excludes lens characteristics and light source characteristics)



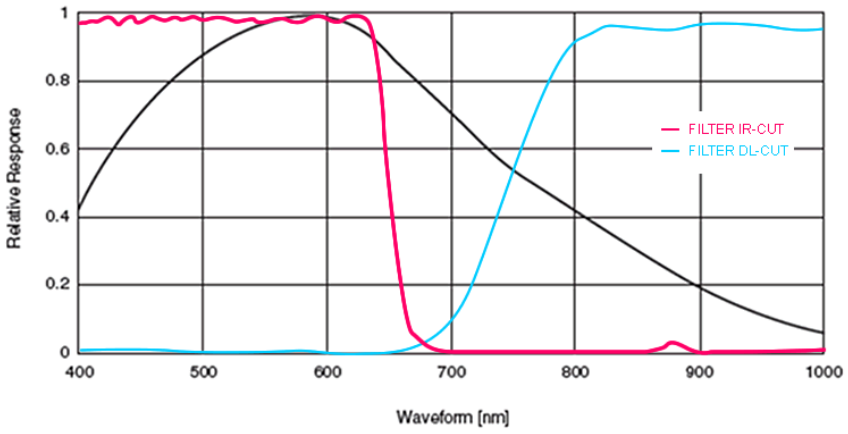
1.20.5.4.5.2 Color version

Spectral Sensitivity Characteristics (excludes lens characteristics and light source characteristics)



1.20.5.4.5.3 Gray scale version

Spectral Sensitivity Characteristics (excludes lens characteristics and light source characteristics)



Device Feature And Property List (p. 617)

1.20.5.4.6 Device Feature And Property List

- **mvBlueCOUGAR-X122G Features** (p. 617)
- **mvBlueCOUGAR-X122C Features** (p. 617)

1.20.5.4.6.1 mvBlueCOUGAR-X122G Features

1.20.5.4.6.2 mvBlueCOUGAR-X122C Features

1.20.5.5 mvBlueCOUGAR-X123 (1.4 Mpix [1360 x 1024])

1.20.5.5.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.5.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.5.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart" "TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
`"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"`	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
`"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"`	OnFallingEdge	Each falling edge of trigger signal acquires one image.
`"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"`	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

See also

For detailed description about the trigger modes (https://www.matrix-vision/manuals/ [mv↩ IMPACT Acquire API])

- **C: TCameraTriggerMode**
- **C++**: **mvIMPACT::acquire::TCameraTriggerMode**

1.20.5.5.2.2 Exposure aka Integration

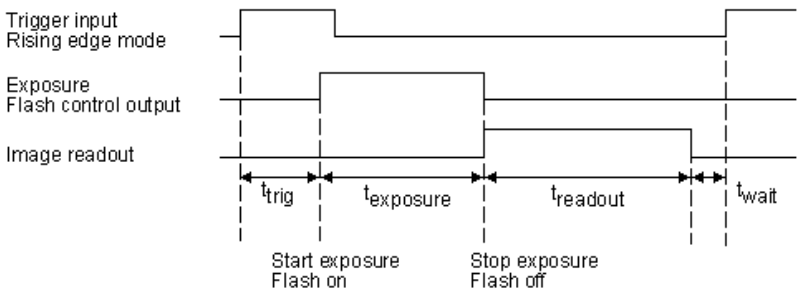
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.5.2.3 Readout

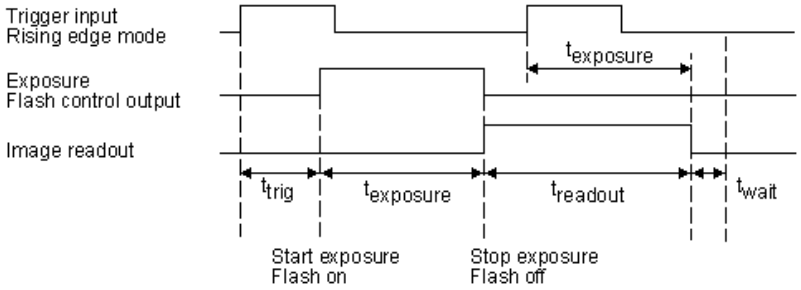
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes tvshift, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.5.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock	
		28 MHz	56 MHz
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)	
t_{wait}	Time after a new trigger is accepted	$<$ 1us	
t_{trans}	Image transfer time (move image to readout cells in CCD)	42.86us	21.43us
$t_{readline}$	Time needed to readout a line	63.93us	31.96us
t_{vshift}	Time needed to shift unused lines away	8.57us	4.29us
$t_{exposure}$	Exposure time	20us..10s	20us..10s
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout} = ((ActiveLines + 1) * t_{readline})) + ((1050 - ActiveLines) * t_{vshift}) + t_{trans}$	

1.20.5.5.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 1024 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                         t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                         t_readout
```

1.20.5.5.3.2 Example: Frame rate as function of lines & exposure time Now, when we insert the values using exposure time of, for example, 8000 us, 1024 lines and 56MHz pixel clock (Expose mode: No overlap):

```
FPS_max =                                    1
          ------------------------------------------------------------------------
          (((1024 + 1) * 31.96 us) + ((1050 - 1024) * 4.29 us) + 8.57 us) + 8000 us
        = 0.000024454678999324316   1 / us
        = 24.5
```

1.20.5.5.3.3 Frame rate calculator

> **Note**
>
> The calculator returns the max. frame rate supported by the sensor. Please keep in mind that it will depend on the interface and the used image format if this frame rate can be transferred.

1.20.5.5.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing Trigger Modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed **exposure** : abt 2..3ms **window:** abt 4..6ms **trigger mode:** from 5..90ms, varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.5.5 CCD Sensor Data

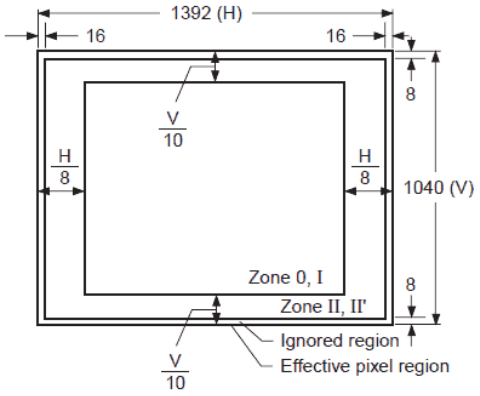
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 8mm (Type 1/2)
- Number of effective pixels: 1392 (H) x 1040 (V) approx. 1.45M pixels
- Total number of pixels: 1434 (H) x 1050 (V) approx. 1.5M pixels
- Chip size: 7.60mm (H) x 6.2mm (V)
- Unit cell size: 4.65um (H) x 4.65um (V)
- Optical black:
  - Horizontal (H) direction: Front 2 pixels, rear 40 pixels
  - Vertical (V) direction: Front 8 pixels, rear 2 pixels
- Number of dummy bits: Horizontal 20 Vertical 3
- Substrate material: Silicon

1.20.5.5.5.1 Characteristics

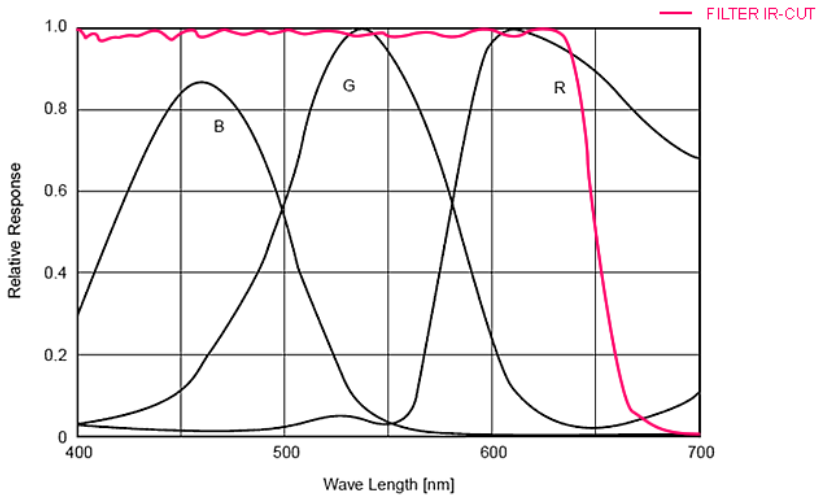
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading



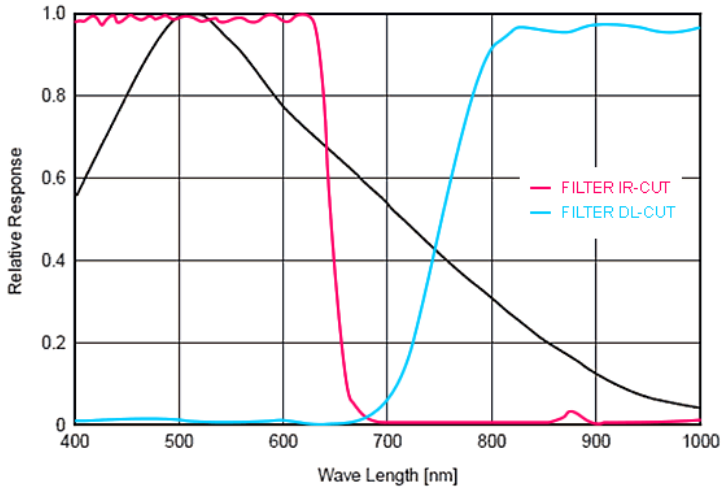
1.20.5.5.5.2 Color version

Spectral Sensitivity Characteristics (excludes lens characteristics and light source characteristics)



1.20.5.5.5.3 Gray scale version

Spectral Sensitivity Characteristics (excludes lens characteristics and light source characteristics)



Device Feature And Property List (p. 623)

1.20.5.5.6 Device Feature And Property List

- **mvBlueCOUGAR-X123G Features** (p. 623)
- **mvBlueCOUGAR-X123C Features** (p. 623)

1.20.5.5.6.1 mvBlueCOUGAR-X123G Features

1.20.5.5.6.2 mvBlueCOUGAR-X123C Features

1.20.5.6 mvBlueCOUGAR-X124 (1.9 Mpix [1600 x 1200])

1.20.5.6.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.6.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.6.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = LevelLow"` `"ExposureMode = Timed"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

See also

For detailed description about the trigger modes (`https://www.matrix-vision/manuals/` [mv↩ IMPACT Acquire API])

- **C: TCameraTriggerMode**
- **C++**: **mvIMPACT::acquire::TCameraTriggerMode**

1.20.5.6.2.2 Exposure aka Integration

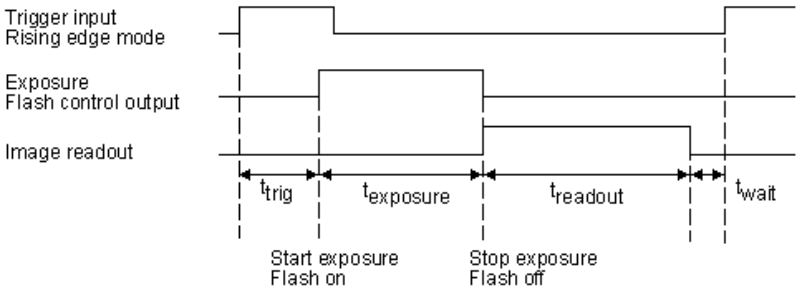
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.6.2.3 Readout

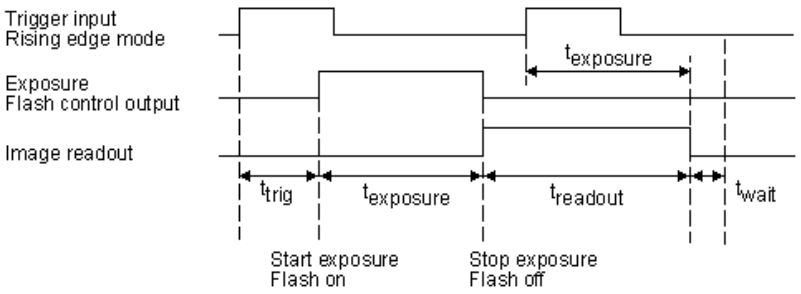
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes tvshift, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.6.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock		
		36 MHz	56 MHz	65 MHz
t_{trig}	Time from trigger (internal or external) to exposure start	t_{trig} = t_{vshift} + $t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx.		
t_{wait}	Time after a new trigger is accepted	$<$ 1us		
t_{trans}	Image transfer time (move image to readout cells in CCD)	35.56us	22.86us	19.69us
$t_{readline}$	time needed to readout a line	53.33us	34.29us	29.54us
t_{vshift}	time needed to shift unused lines away	5.56us	3.57us	3.08us
$t_{exposure}$	Exposure time	20us..10s	20us..10s	20us..10s
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout}$ = ((ActiveLines + 1) $*$ $t_{readline}$)) + ((1248 - Activel		

1.20.5.6.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 1200 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

$$FPS_max = \frac{1}{t_readout + t_exposure}$$

(Expose mode: Overlapped):

$$t_readout < t_exposure: \quad FPS_max = \frac{1}{t_exposure}$$

$$t_readout > t_exposure: \quad FPS_max = \frac{1}{t_readout}$$

1.20.5.6.3.2 Example: Frame rate as function of lines & exposure time

Now, when we insert the values using exposure time of, for example, 8000 us, 1200 lines and 65MHz pixel clock (Expose mode: No overlap):

$$FPS_max = \frac{1}{(((1200 + 1) * 29.54\ us) + ((1248 - 1200) * 3.08\ us) + 19.69\ us) + 8000\ us}$$

$$= 0.0000229120952263 \quad 1 / us$$

$$= 22.9$$

1.20.5.6.3.3 Frame rate calculator

Note

The calculator returns the max. frame rate supported by the sensor. Please keep in mind that it will depend on the interface and the used image format if this frame rate can be transferred.

1.20.5.6.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing Trigger Modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed **exposure** : abt 2..3ms **window:** abt 4..6ms **trigger mode:** from 5..90ms, varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.6.5 CCD Sensor Data

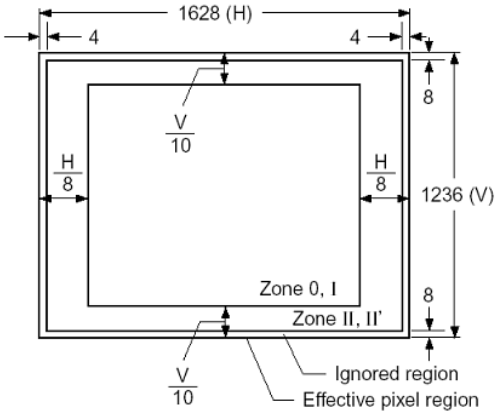
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 8.923mm (Type 1/1.8)
- Number of effective pixels: 1600 (H) x 1200 (V) approx. 1.92M pixels
- Total number of pixels: 1688 (H) x 1248 (V) approx. 2.11M pixels
- Chip size: 8.50mm (H) x 6.8mm (V)
- Unit cell size: 4.4um (H) x 4.4um (V)
- Optical black:
  - Horizontal (H) direction: Front 12 pixels, rear 48 pixels
  - Vertical (V) direction: Front 10 pixels, rear 2 pixels
- Number of dummy bits: Horizontal 28 Vertical 1
- Substrate material: Silicon

1.20.5.6.5.1 Characteristics

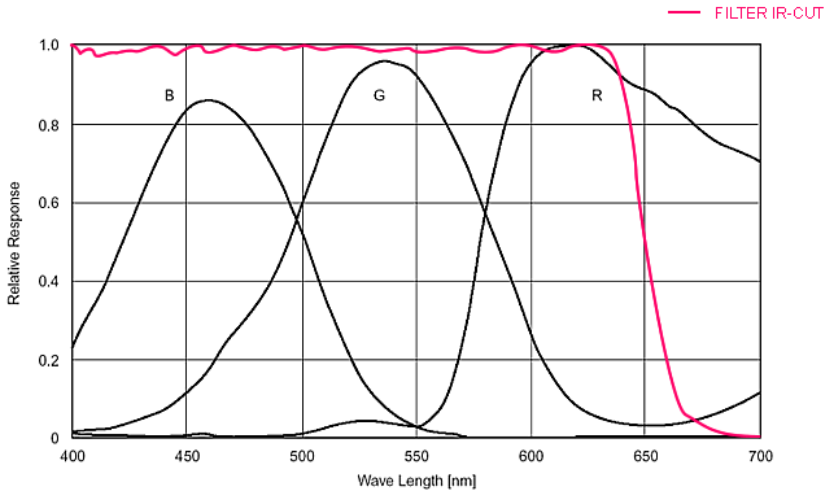
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading



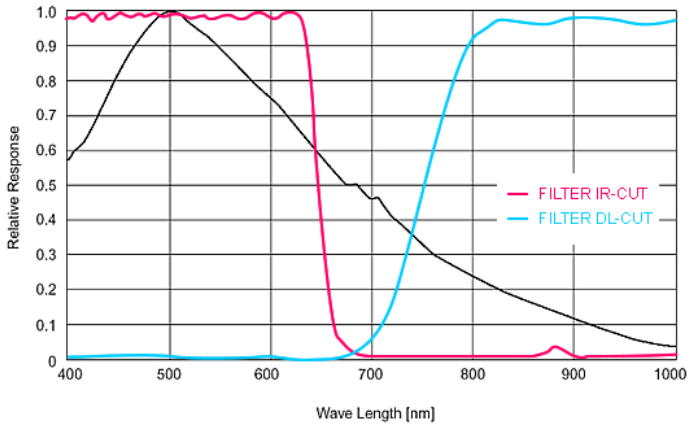
1.20.5.6.5.2 Color version

Spectral Sensitivity Characteristics (excludes lens characteristics and light source characteristics)



1.20.5.6.5.3 Gray scale version

Spectral Sensitivity Characteristics (excludes lens characteristics and light source characteristics)



Device Feature And Property List (p. 630)

1.20.5.6.6 Device Feature And Property List

- **mvBlueCOUGAR-X124G Features** (p. 630)
- **mvBlueCOUGAR-X124C Features** (p. 630)

1.20.5.6.6.1 mvBlueCOUGAR-X124G Features

1.20.5.6.6.2 mvBlueCOUGAR-X124C Features

1.20.5.7 mvBlueCOUGAR-XD124a (3 Mpix [1936 x 1460])

1.20.5.7.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.7.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.7.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = LevelLow"` `"ExposureMode = Timed"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = LevelHigh"` `"ExposureMode = Timed"`	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = FallingEdge"` `"ExposureMode = Timed"`	OnFallingEdge	Each falling edge of trigger signal acquires one image.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = RisingEdge"` `"ExposureMode = Timed"`	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.7.2.2 Exposure aka Integration

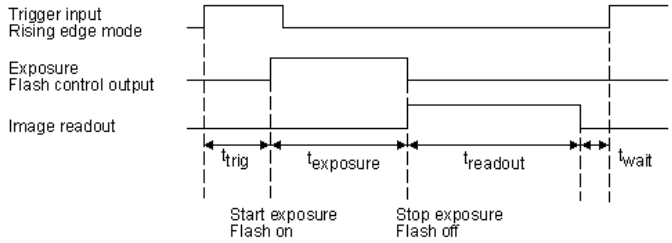
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.7.2.3 Readout

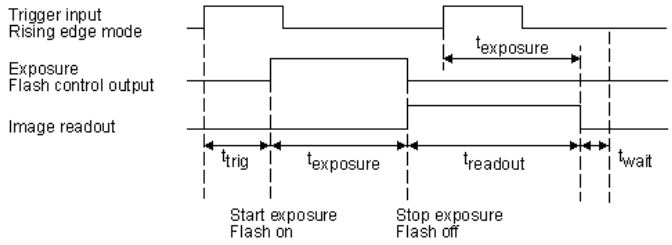
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.7.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock		
		56 MHz	66 MHz	
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)		
t_{wait}	Time after a new trigger is accepted	$<$ 1us		
t_{trans}	Image transfer time (move image to readout cells in CCD)	42.30us	35.89us	
$t_{readline}$	time needed to readout a line	24.91us	21.14us	
t_{vshift}	time needed to shift unused lines away	4.57us	3.88us	
$t_{exposure}$	Exposure time	30us..20s	30us..20s	
taps	Number of Taps	2 or 4	2 or 4	
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout}$ = ((ActiveLines + 1) $*$ $t_{readline}$ / (taps / 2))) + (((1476 - ActiveLines) / (		

1.20.5.7.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 1460 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                         t_readout
```

1.20.5.7.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 1460 lines, 4 taps and 66MHz pixel clock:

```
FPS_max =                                                1
          ------------------------------------------------------------------------------------
          (((1460 + 1) * 21.14 us / (4 / 2)) + (((1476 - 1460)  / (4 / 2)) * 3.88 us) + 35.89 us)
        =  0.00006453836353943874  1 / us
        =  64.5
```

1.20.5.7.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing trigger modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.7.5 CCD Sensor Data

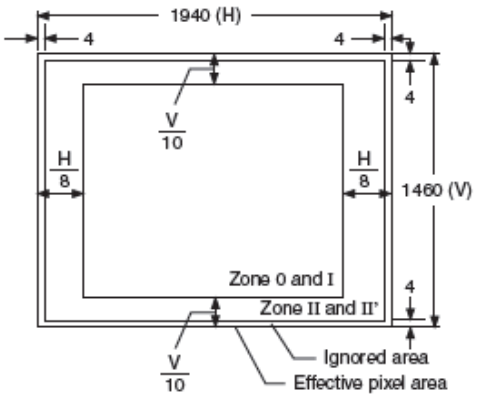
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 10.972mm (Type 2/3)
- Number of effective pixels: 1940 (H) x 1460 (V) approx. 2.83 Mpixels
- Total number of pixels: 2020 (H) x 1476 (V) approx. 2.98 Mpixels
- Chip size: 10.7mm (H) x 9.2mm (V)
- Unit cell size: 4.54um (H) x 4.54um (V)
- Optical black:
  - Horizontal (H) direction: Front 40 pixels each channel
  - Vertical (V) direction: Front 8 pixels each channel
- Number of dummy bits: Front 1 each channel
- Substrate material: Silicon

1.20.5.7.5.1 Characteristics

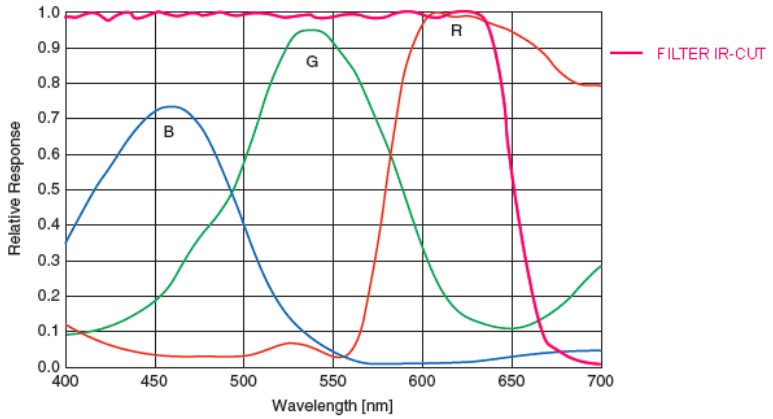
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading



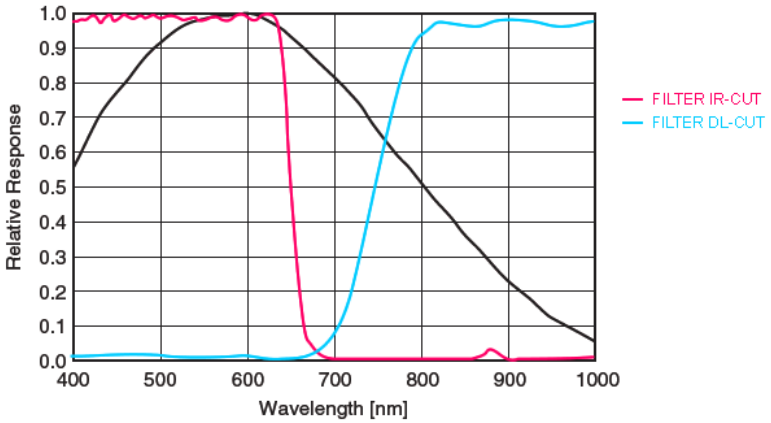
1.20.5.7.5.2 Color version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



1.20.5.7.5.3 Gray scale version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



Device Feature And Property List (p. 636)

1.20.5.7.6 Device Feature And Property List

- **mvBlueCOUGAR-XD124aG Features** (p. 636)
- **mvBlueCOUGAR-XD124aC Features** (p. 636)

1.20.5.7.6.1 mvBlueCOUGAR-XD124aG Features

1.20.5.7.6.2 mvBlueCOUGAR-XD124aC Features

1.20.5.8 mvBlueCOUGAR-X225 (5.1 Mpix [2448 x 2050])

1.20.5.8.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.8.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.8.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.
`"TriggerSelector = FrameStart"` `"TriggerMode = On"` `"TriggerSource = Software"` `"ExposureMode = Timed"` To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
`"TriggerSelector = AcquisitionActive"` `"TriggerMode = On"` `"TriggerSource = <desired Line>"` `"TriggerActivation = LevelLow"` `"ExposureMode = Timed"`	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.8.2.2 Exposure aka Integration

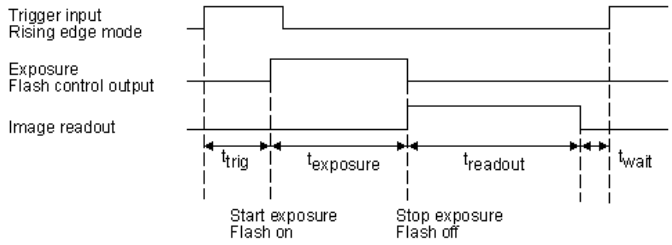
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.8.2.3 Readout

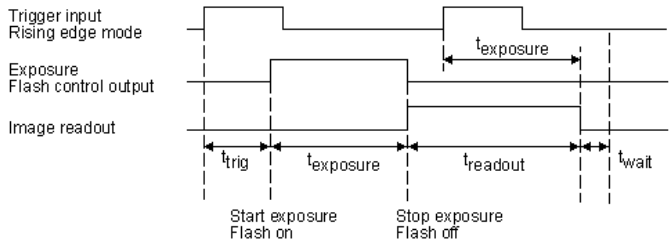
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.8.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock	
		60 MHz	66 MHz
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)	
t_{wait}	Time after a new trigger is accepted	$<$ 1us	
t_{trans}	Image transfer time (move image to readout cells in CCD)	30.87us	28.06us
$t_{readline}$	time needed to readout a line	32.07us	29.15us
t_{vshift}	time needed to shift unused lines away	10.00us	9.09us
$t_{exposure}$	Exposure time	20us..20s	20us..20s
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout}$ = ((ActiveLines + 1) $*$ $t_{readline}$)) + ((2068 - ActiveLines) $*$ t_{vshift}) + t_{trans}	

1.20.5.8.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 2050 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

$$FPS_max = \frac{1}{t_readout + t_exposure}$$

(Expose mode: Overlapped):

$$t_readout < t_exposure: \quad FPS_max = \frac{1}{t_exposure}$$

$$t_readout > t_exposure: \quad FPS_max = \frac{1}{t_readout}$$

1.20.5.8.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 2050 lines and 60MHz pixel clock:

$$FPS_max = \frac{1}{(((2050 + 1) * 32.07\ us) + ((2068 - 2050) * 10.00\ us) + 30.87\ us) + 8000\ us}$$

$$= 0.0000135159902274\ \ 1\ /\ us$$

$$= 13.5$$

1.20.5.8.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing Trigger Modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.8.5 CCD Sensor Data

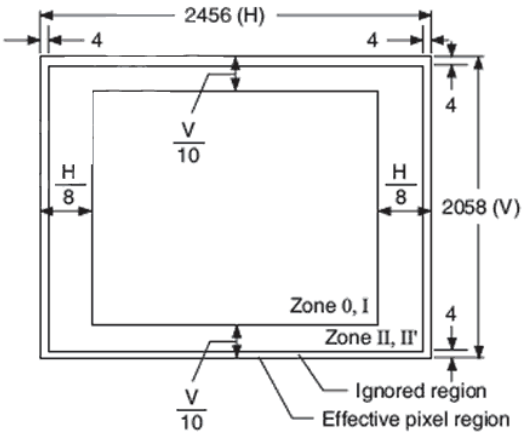
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 11.016mm (Type 2/3)
- Number of effective pixels: 2456 (H) x 2058 (V) approx. 5.05 M pixels
- Total number of pixels: 2536 (H) x 2068 (V) approx. 5.24 M pixels
- Chip size: 9.93mm (H) x 8.70mm (V)
- Unit cell size: 3.45um (H) x 3.45um (V)
- Optical black:
  - Horizontal (H) direction: Front 40 pixels per channel
  - Vertical (V) direction: Front 8 pixels, rear 2 pixels
- Number of dummy bits: Horizontal 1 per channel Vertical 1
- Substrate material: Silicon

1.20.5.8.5.1 Characteristics

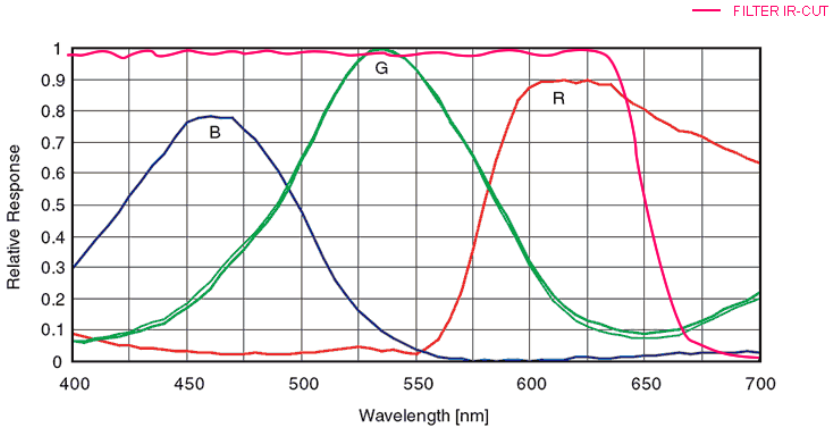
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading



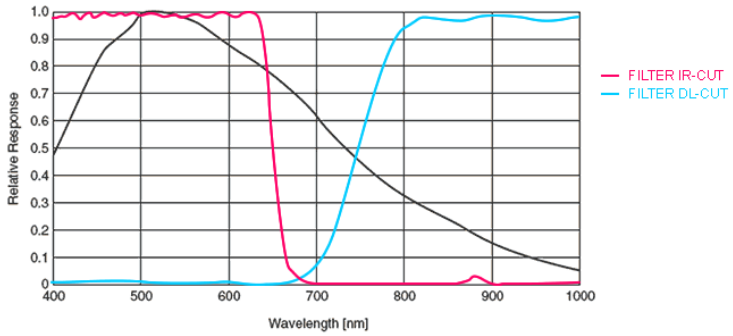
1.20.5.8.5.2 Color version

Spectral Sensitivity Characteristics (Includes lens characteristics, excludes light source characteristics)



1.20.5.8.5.3 Gray scale version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



Device Feature And Property List (p. 642)

1.20.5.8.6 Device Feature And Property List

- **mvBlueCOUGAR-X225G Features** (p. 642)
- **mvBlueCOUGAR-X225C Features** (p. 642)

1.20.5.8.6.1 mvBlueCOUGAR-X225G Features

1.20.5.8.6.2 mvBlueCOUGAR-X225C Features

1.20.5.9 mvBlueCOUGAR-X125a (5.1 Mpix [2448 x 2050])

1.20.5.9.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.9.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.9.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
"TriggerSelector = FrameStart" "TriggerMode = Off"	Continuous	Free running, no external trigger signal needed.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = Software" "ExposureMode = Timed" To trigger one frame execute the *TriggerSoftware@i* command then.	OnDemand	Image acquisition triggered by command (software trigger).
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = Timed"	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = AcquisitionActive" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = Timed"	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = FallingEdge" "ExposureMode = Timed"	OnFallingEdge	Each falling edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = RisingEdge" "ExposureMode = Timed"	OnRisingEdge	Each rising edge of trigger signal acquires one image.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.9.2.2 Exposure aka Integration

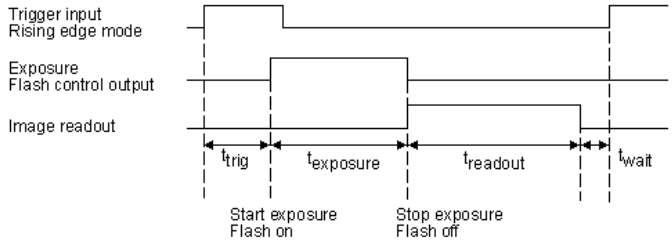
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.9.2.3 Readout

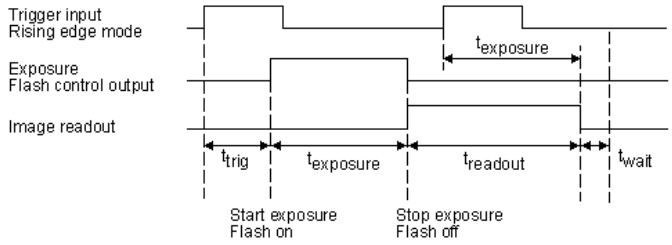
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.9.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock	
		60 MHz	66 MHz
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)	
t_{wait}	Time after a new trigger is accepted	$<$ 1us	
t_{trans}	Image transfer time (move image to readout cells in CCD)	30.87us	28.06us
$t_{readline}$	time needed to readout a line	53.20us	48.36us
t_{vshift}	time needed to shift unused lines away	10.00us	9.09us
$t_{exposure}$	Exposure time	20us..20s	20us..20s
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout}$ = ((ActiveLines + 1) $*$ $t_{readline}$)) + ((2068 - ActiveLines) $*$ t_{vshift}) + t_{trans}	

1.20.5.9.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 2050 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                        t_readout
```

1.20.5.9.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 2050 lines and 60MHz pixel clock:

```
FPS_max =                                  1
          ------------------------------------------------------------------------
          (((2050 + 1) * 53.20 us) + ((2068 - 2050) * 10.00 us) + 30.87 us) + 8000 us
        = 0.00000852340018549 1 / us
        = 8.5
```

1.20.5.9.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing Trigger Modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

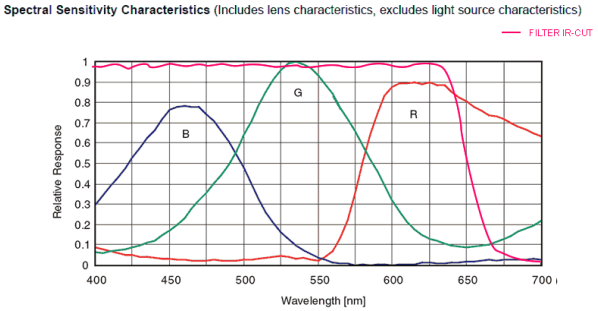
1.20.5.9.5 CCD Sensor Data

Device Structure

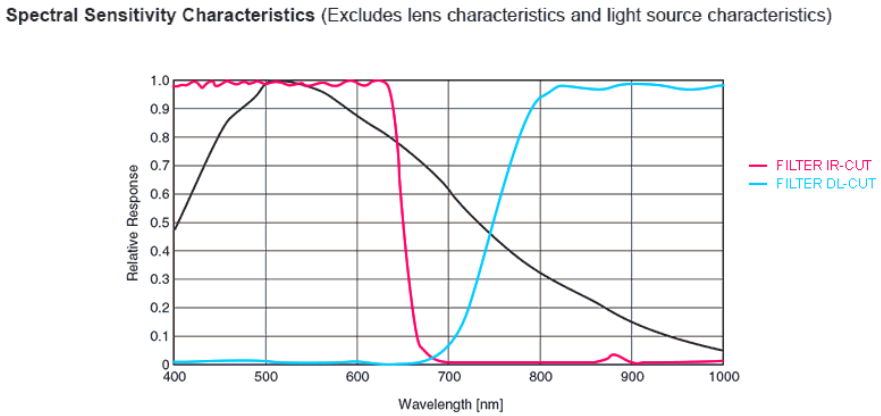
- Interline CCD image sensor
- Image size: Diagonal 11.016mm (Type 2/3)
- Number of effective pixels: 2456 (H) x 2058 (V) approx. 5.05 M pixels
- Total number of pixels: 2536 (H) x 2068 (V) approx. 5.24 M pixels
- Chip size: 9.93mm (H) x 8.70mm (V)
- Unit cell size: 3.45um (H) x 3.45um (V)
- Optical black:
  - Horizontal (H) direction: Front 40 pixels, rear 40 pixels
  - Vertical (V) direction: Front 8 pixels, rear 2 pixels
- Number of dummy bits: Horizontal 1 per channel Vertical 1
- Substrate material: Silicon

1.20.5.9.5.1 Characteristics

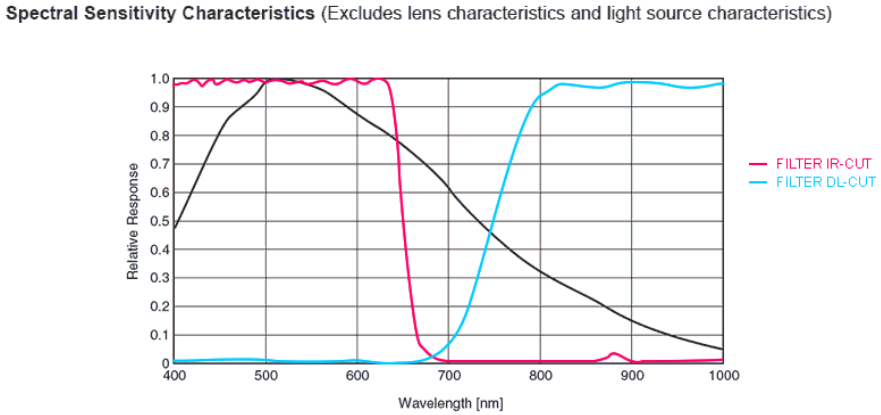
These zone definitions apply to both the color and gray scale version of the sensor.



1.20.5.9.5.2 Color version



1.20.5.9.5.3 Gray scale version



Device Feature And Property List (p. 649)

1.20.5.9.6 Device Feature And Property List

- **mvBlueCOUGAR-X125aG Features** (p. 649)
- **mvBlueCOUGAR-X125aC Features** (p. 649)

1.20.5.9.6.1 mvBlueCOUGAR-X125aG Features

1.20.5.9.6.2 mvBlueCOUGAR-X125aC Features

1.20.5.10 mvBlueCOUGAR-XD126 (6 Mpix [2752 x 2208])

1.20.5.10.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.10.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.10.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

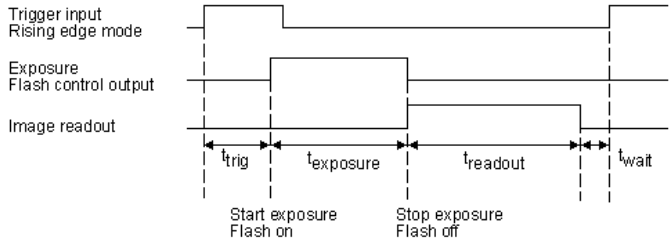
TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.10.2.2 Exposure aka Integration After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

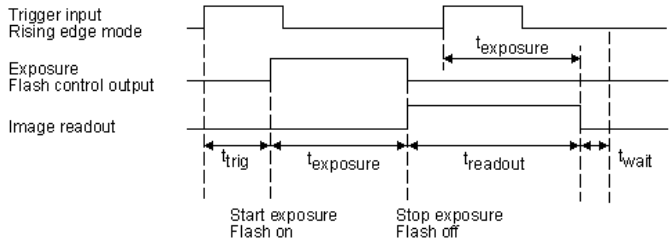
1.20.5.10.2.3 Readout When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.10.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock		
		56 MHz	66 MHz	
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)		
t_{wait}	Time after a new trigger is accepted	$<$ 1us		
t_{trans}	Image transfer time (move image to readout cells in CCD)	42.3us	35.89us	
$t_{readline}$	time needed to readout a line	32.21us	27.33us	
t_{vshift}	time needed to shift unused lines away	4.57us	3.88us	
$t_{exposure}$	Exposure time	30us..20s	30us..20s	
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout}$ = ((ActiveLines + 1) $*$ $t_{readline}$ / (taps / 2))) + (((2224 - ActiveLines) / (		

1.20.5.10.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 2208 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                         t_readout
```

1.20.5.10.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 2208 lines, 4 taps and 66MHz pixel clock:

```
FPS_max =                                                1
          ------------------------------------------------------------------------------------
          (((2208 + 1) * 27.33 us / (4 / 2)) + (((2224 - 2208)  / (4 / 2)) * 3.88 us) + 35.89 us)
        =  0.00003305466597185759  1 / us
        =  33.1
```

1.20.5.10.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing trigger modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.10.5 CCD Sensor Data

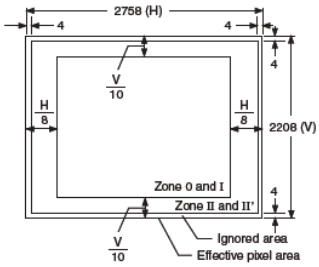
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 15.989mm (Type 1)
- Number of effective pixels: 2758 (H) x 2208 (V) approx. 6.09 Mpixels
- Total number of pixels: 2838 (H) x 2224 (V) approx. 6.31 Mpixels
- Chip size: 14.6mm (H) x 12.8mm (V)
- Unit cell size: 4.54um (H) x 4.54um (V)
- Optical black:
  - Horizontal (H) direction: Front 40 pixels each channel
  - Vertical (V) direction: Front 8 pixels each channel
- Number of dummy bits: Front 1 each channel
- Substrate material: Silicon

1.20.5.10.5.1 Characteristics

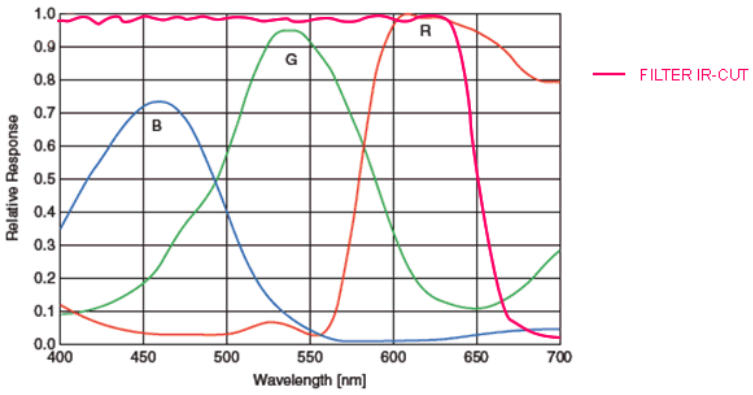
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading

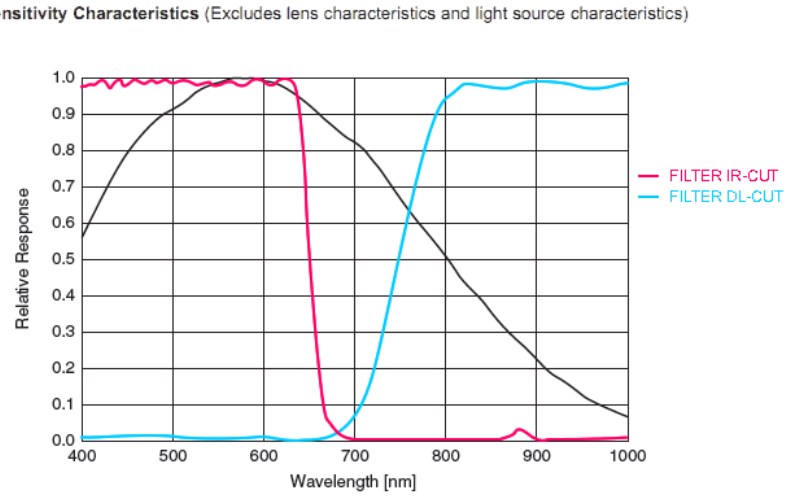


1.20.5.10.5.2 Color version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



1.20.5.10.5.3 Gray scale version



Device Feature And Property List (p. 655)

1.20.5.10.6 Device Feature And Property List

- **mvBlueCOUGAR-XD126G Features** (p. 655)
- **mvBlueCOUGAR-XD126C Features** (p. 655)

1.20.5.10.6.1 mvBlueCOUGAR-XD126G Features

1.20.5.10.6.2 mvBlueCOUGAR-XD126C Features

1.20.5.11 mvBlueCOUGAR-XD126a (6 Mpix [2752 x 2208])

1.20.5.11.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.11.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.11.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.11.2.2 Exposure aka Integration

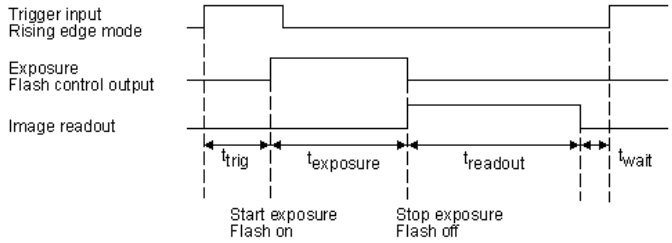
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.11.2.3 Readout

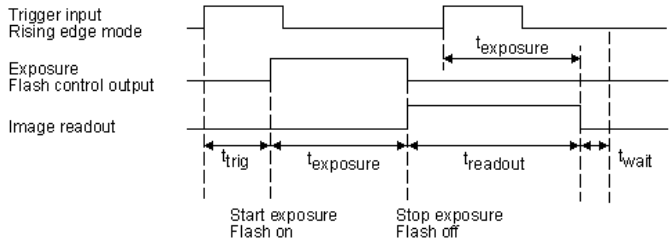
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.11.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock	
		56 MHz	
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)	
t_{wait}	Time after a new trigger is accepted	$<$ 1us	
t_{trans}	Image transfer time (move image to readout cells in CCD)	42.3us	
$t_{readline}$	time needed to readout a line	32.21us	
t_{vshift}	time needed to shift unused lines away	4.57us	
$t_{exposure}$	Exposure time	30us..20s	
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout}$ = ((ActiveLines + 1) $*$ $t_{readline}$ / (taps / 2))) + (((2224 - ActiveLines) / (	

1.20.5.11.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 2208 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                         t_readout
```

1.20.5.11.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 2208 lines, 2 taps and 56MHz pixel clock:

```
FPS_max =                                                1
          ------------------------------------------------------------------------------------
          (((2208 + 1) * 32.21 us / (2 / 2)) + (((2224 - 2208)  / (2 / 2)) * 4.57 us) + 42.3 us)
        =  0.0000140316787599  1 / us
        =  14
```

1.20.5.11.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing trigger modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.11.5 CCD Sensor Data

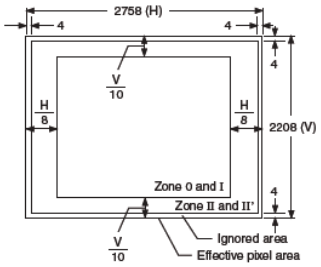
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 15.989mm (Type 1)
- Number of effective pixels: 2758 (H) x 2208 (V) approx. 6.09 Mpixels
- Total number of pixels: 2838 (H) x 2224 (V) approx. 6.31 Mpixels
- Chip size: 14.6mm (H) x 12.8mm (V)
- Unit cell size: 4.54um (H) x 4.54um (V)
- Optical black:
  - Horizontal (H) direction: Front 40 pixels each channel
  - Vertical (V) direction: Front 8 pixels each channel
- Number of dummy bits: Front 1 each channel
- Substrate material: Silicon

1.20.5.11.5.1 Characteristics

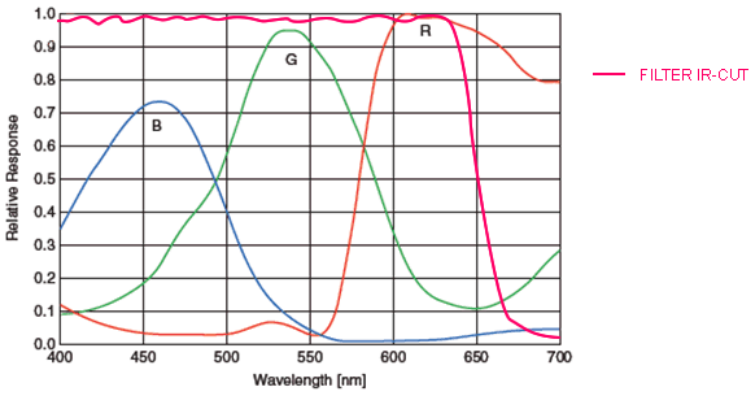
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading

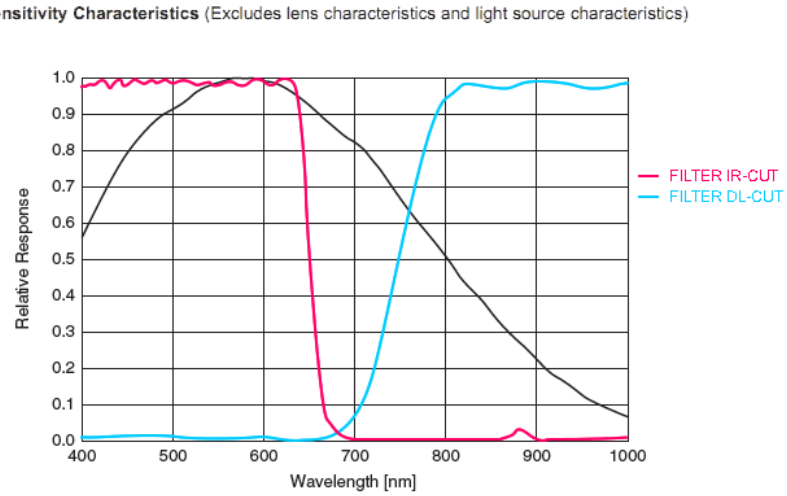


1.20.5.11.5.2 Color version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



1.20.5.11.5.3 Gray scale version



Device Feature And Property List (p. 661)

1.20.5.11.6 Device Feature And Property List

- **mvBlueCOUGAR-XD126aG Features** (p. 661)
- **mvBlueCOUGAR-XD126aC Features** (p. 661)

1.20.5.11.6.1 mvBlueCOUGAR-XD126aG Features

1.20.5.11.6.2 mvBlueCOUGAR-XD126aC Features

1.20.5.12 mvBlueCOUGAR-XD129 (9.2 Mpix [3384 x 2712])

1.20.5.12.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.12.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.12.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.12.2.2 Exposure aka Integration

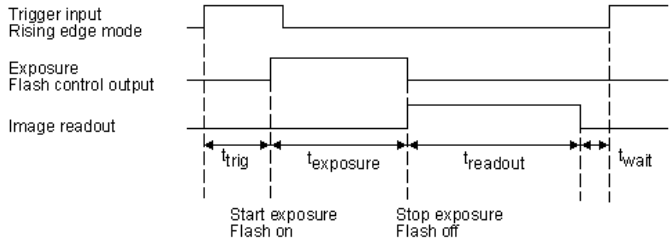
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.12.2.3 Readout

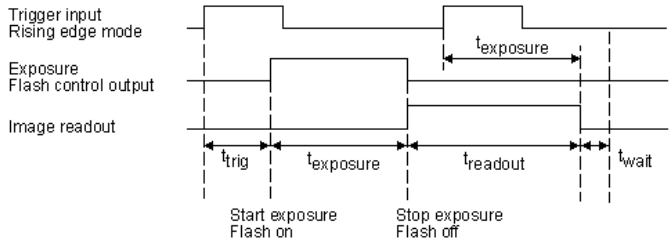
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.12.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock		
		56 MHz	66 MHz	
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)		
t_{wait}	Time after a new trigger is accepted	$<$ 1us		
t_{trans}	Image transfer time (move image to readout cells in CCD)	59.64us	50.61us	
$t_{readline}$	time needed to readout a line	38.88us	32.98us	
t_{vshift}	time needed to shift unused lines away	5.86us	4.97us	
$t_{exposure}$	Exposure time	30us..20s	30us..20s	
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout}$ = ((ActiveLines + 1) $*$ $t_{readline}$ / (taps / 2))) + (((2728 - ActiveLines) / (		

1.20.5.12.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 2712 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                         t_readout
```

1.20.5.12.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 2712 lines, 4 taps and 66MHz pixel clock:

```
FPS_max =                                                1
          ------------------------------------------------------------------------------------
          (((2712 + 1) * 32.98 us / (4 / 2)) + (((2728 - 2712)  / (4 / 2)) * 4.97 us) + 50.61 us)
        =  0.000022307615775410495  1 / us
        =  22.3
```

1.20.5.12.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing trigger modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.12.5 CCD Sensor Data

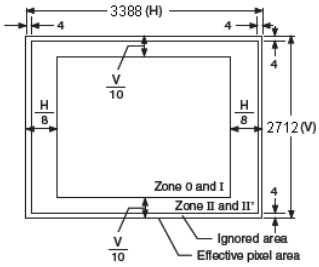
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 15.972mm (Type 1)
- Number of effective pixels: 3388 (H) x 2712 (V) approx. 9.19 Mpixels
- Total number of pixels: 3468 (H) x 2728 (V) approx. 9.46 Mpixels
- Chip size: 14.6mm (H) x 12.8mm (V)
- Unit cell size: 3.69um (H) x 3.69um (V)
- Optical black:
  - Horizontal (H) direction: Front 40 pixels each channel
  - Vertical (V) direction: Front 8 pixels each channel
- Number of dummy bits: Front 1 each channel
- Substrate material: Silicon

1.20.5.12.5.1 Characteristics

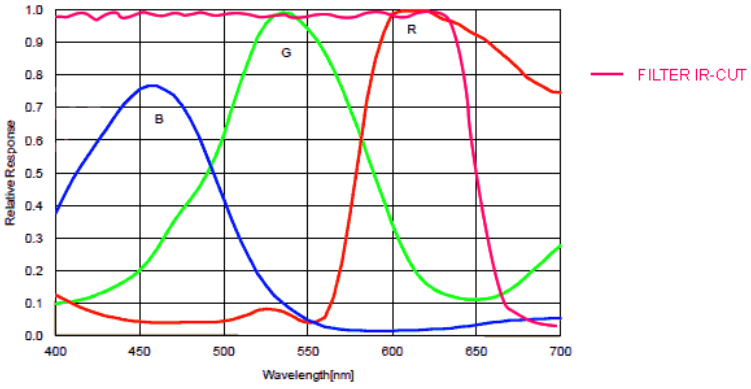
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading

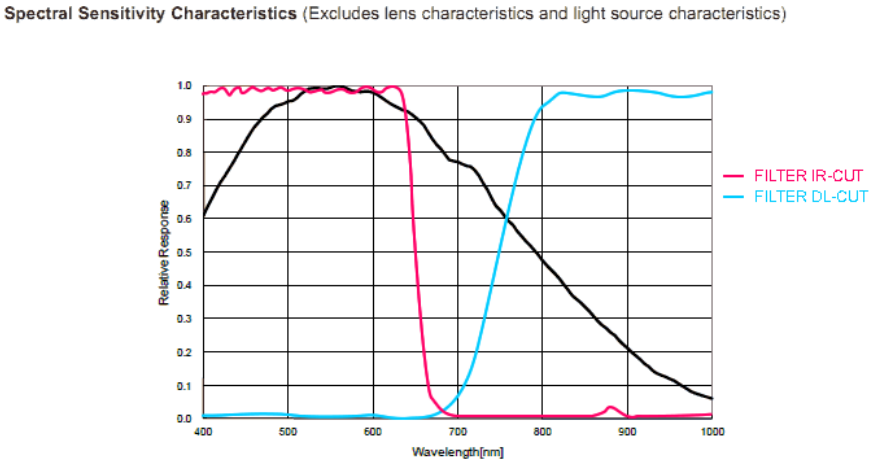


1.20.5.12.5.2 Color version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



1.20.5.12.5.3 Gray scale version



Device Feature And Property List (p. 667)

1.20.5.12.6 Device Feature And Property List

- **mvBlueCOUGAR-XD129G Features** (p. 667)
- **mvBlueCOUGAR-XD129C Features** (p. 667)

1.20.5.12.6.1 mvBlueCOUGAR-XD129G Features

1.20.5.12.6.2 mvBlueCOUGAR-XD129C Features

1.20.5.13 mvBlueCOUGAR-XD129a (9.2 Mpix [3384 x 2712])

1.20.5.13.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.13.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.13.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.13.2.2 Exposure aka Integration

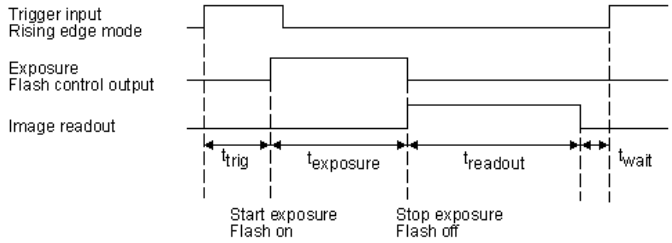
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.13.2.3 Readout

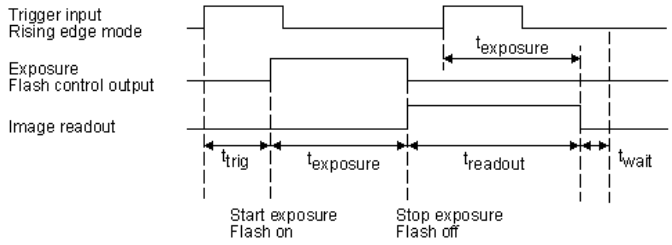
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.13.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock	
		56 MHz	
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)	
t_{wait}	Time after a new trigger is accepted	$<$ 1us	
t_{trans}	Image transfer time (move image to readout cells in CCD)	59.64us	
$t_{readline}$	time needed to readout a line	38.88us	
t_{vshift}	time needed to shift unused lines away	5.86us	
$t_{exposure}$	Exposure time	30us..20s	
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout}$ = ((ActiveLines + 1) $*$ $t_{readline}$ / (taps / 2))) + (((2728 - ActiveLines) / (	

1.20.5.13.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 2712 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                        t_readout
```

1.20.5.13.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 2712 lines, 2 taps and 56MHz pixel clock:

```
FPS_max =                                                1
          ------------------------------------------------------------------------------------
          (((2712 + 1) * 38.88 us / (2 / 2)) + (((2728 - 2712)  / (2 / 2)) * 5.86 us) + 59.64 us)
        = 0.000009466573717  1 / us
        = 9.5
```

1.20.5.13.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing trigger modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.13.5 CCD Sensor Data

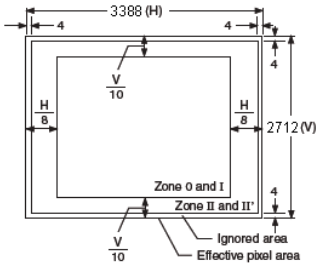
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 15.972mm (Type 1)
- Number of effective pixels: 3388 (H) x 2712 (V) approx. 9.19 Mpixels
- Total number of pixels: 3468 (H) x 2728 (V) approx. 9.46 Mpixels
- Chip size: 14.6mm (H) x 12.8mm (V)
- Unit cell size: 3.69um (H) x 3.69um (V)
- Optical black:
  - Horizontal (H) direction: Front 40 pixels each channel
  - Vertical (V) direction: Front 8 pixels each channel
- Number of dummy bits: Front 1 each channel
- Substrate material: Silicon

1.20.5.13.5.1 Characteristics

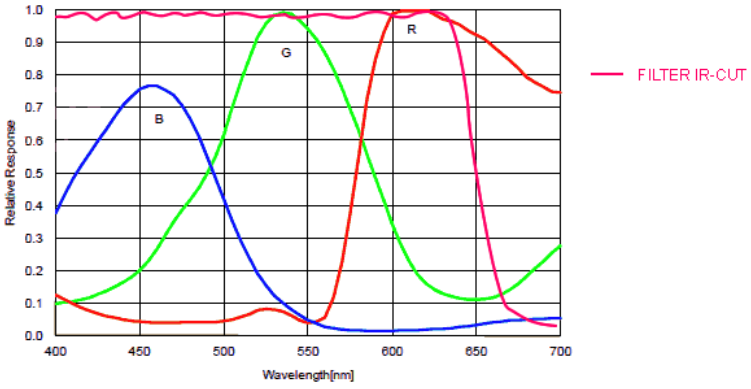
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading

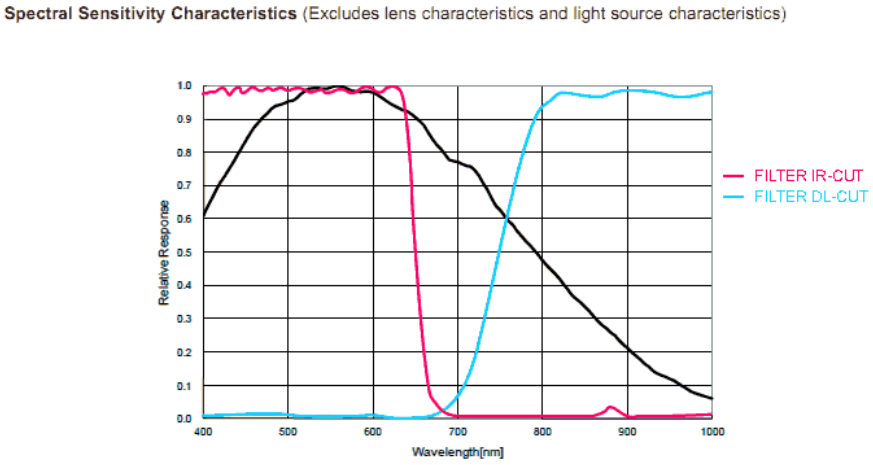


1.20.5.13.5.2 Color version

Spectral Sensitivity Characteristics (Excludes lens characteristics and light source characteristics)



1.20.5.13.5.3 Gray scale version



Device Feature And Property List (p. 673)

1.20.5.13.6 Device Feature And Property List

- **mvBlueCOUGAR-XD129aG Features** (p. 673)
- **mvBlueCOUGAR-XD129aC Features** (p. 673)

1.20.5.13.6.1 mvBlueCOUGAR-XD129aG Features

1.20.5.13.6.2 mvBlueCOUGAR-XD129aC Features

1.20.5.14 mvBlueCOUGAR-XD1212a (12.1 Mpix [4248 x 2836])

1.20.5.14.1 Introduction

The CCD sensor is a highly programmable imaging module which will, for example, enable the following type of applications

Industrial applications:

- triggered image acquisition with precise control of image exposure start by hardware trigger input.
- image acquisition of fast moving objects due to:
  - frame exposure, integrating all pixels at a time in contrast to CMOS imager which typically integrate line-by-line.
  - short shutter time, to get sharp images.
  - flash control output to have enough light for short time.

Scientific applications:

- long time exposure for low light conditions.
- optimizing image quality using the variable shutter control.

1.20.5.14.2 Details of operation

The process of getting an image from the CCD sensor can be separated into three different phases.

1.20.5.14.2.1 Trigger

When coming out of reset or ready with the last readout the CCD controller is waiting for a Trigger signal.

The following trigger modes are available:

Setting (**"GenICam"**)	Mode / Setting (**"Device Specific"**)	Description
`"TriggerSelector = FrameStart"` `"TriggerMode = Off"`	Continuous	Free running, no external trigger signal needed.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = Software"`
`"ExposureMode = Timed"`
 	
To trigger one frame execute the *TriggerSoftware@i* command then.
	OnDemand	Image acquisition triggered by command (software trigger).

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelLow"`
`"ExposureMode = Timed"`
	OnLowLevel	Start an exposure of a frame as long as the trigger input is below the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = AcquisitionActive"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = LevelHigh"`
`"ExposureMode = Timed"`
	OnHighLevel	Start an exposure of a frame as long as the trigger input is above the trigger threshold. (No Frame↩ Trigger!)

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = FallingEdge"`
`"ExposureMode = Timed"`
	OnFallingEdge	Each falling edge of trigger signal acquires one image.

`"TriggerSelector = FrameStart"`
`"TriggerMode = On"`
`"TriggerSource = <desired Line>"`
`"TriggerActivation = RisingEdge"`
`"ExposureMode = Timed"`
	OnRisingEdge	Each rising edge of trigger signal acquires one image.

"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelHigh" "ExposureMode = TriggerWidth"	OnHighExpose	Each rising edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = LevelLow" "ExposureMode = TriggerWidth"	OnLowExpose	Each falling edge of trigger signal acquires one image, exposure time corresponds to pulse width.
"TriggerSelector = FrameStart" "TriggerMode = On" "TriggerSource = <desired Line>" "TriggerActivation = AnyEdge" "ExposureMode = Timed"	OnAnyEdge	Start the exposure of a frame when the trigger input level changes from high to low or from low to high.

Line Mapping (*TriggerSource* mvIMPACT Acquire -> *TriggerSource* GenICam (valid values for <desired Line>)):

TriggerSource mvIMPACT Acquire	TriggerSource GenICam(BCX)
GP-IN0	Line4
GP-IN1	Line5

1.20.5.14.2.2 Exposure aka Integration

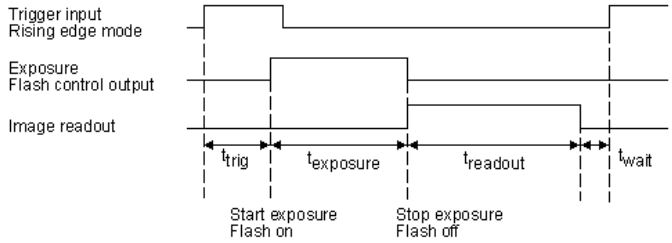
After an active trigger, the exposure phase starts with a maximum jitter of t_{trig}. If flash illumination is enabled in software the flash output will be activated exactly while the sensor chip is integrating light. Exposure time is adjustable by software in increments of $t_{readline}$.

1.20.5.14.2.3 Readout

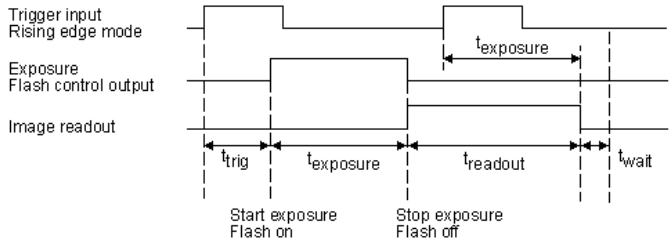
When exposure is finished, the image is transferred to hidden storage cells on the CCD. Image data is then shifted out line-by-line and transferred to memory. Shifting out non active lines takes t_{vshift}, while shifting out active lines will consume $t_{readline}$. The number of active pixels per line will not have any impact on readout speed.

1.20.5.14.3 CCD Timing

Trigger Overlap: No



Trigger Overlap: ReadOut



Note

In the "`Trigger Overlap ReadOut`" mode, the camera accepts a trigger signal the $t_{exposure}$ period earlier than $t_{readout}$ is finished.

Name	Description	Pixel clock		
		36 MHz	56 MHz	
t_{trig}	Time from trigger (internal or external) to exposure start	$t_{trig} = t_{vshift} + t_{turn_on_delay_opto}$ ($t_{turn_on_delay_opto}$ = approx. 2us)		
t_{wait}	Time after a new trigger is accepted	$<$ 1us		
t_{trans}	Image transfer time (move image to readout cells in CCD)	92.78us	59.64us	
$t_{readline}$	time needed to readout a line	71.56us	46.00us	
t_{vshift}	time needed to shift unused lines away	8.22us	5.29us	
$t_{exposure}$	Exposure time	30us..20s	30us..20s	
$t_{readout}$	Image readout time (move image from readout cells to memory	$t_{readout}$ = ((ActiveLines + 1) $*$ $t_{readline}$ / (taps / 2))) + (((2854 - ActiveLines) / (		

1.20.5.14.3.1 Timings

Note

In partial scan mode (readout window ysize $<$ 2838 lines).

To calculate the maximum frames per second (FPS_{max}) you will need following formula (Expose mode: No overlap):

```
FPS_max =            1
          ----------------------
          t_readout + t_exposure
```

(Expose mode: Overlapped):

```
t_readout < t_exposure:    FPS_max =        1
                                      ---------------
                                        t_exposure

t_readout > t_exposure:    FPS_max =        1
                                      ---------------
                                         t_readout
```

1.20.5.14.3.2 Example: Frame rate as function of lines & exposure time (Expose mode: No overlap)

Now, when we insert the values using exposure time of, for example, 8000 us, 2838 lines, 2 taps and 56MHz pixel clock:

```
FPS_max =                                                1
          ------------------------------------------------------------------------------------
          (((2838 + 1) * 46.00 us / (2 / 2)) + (((2854 - 2838)  / (2 / 2)) * 5.29 us) + 59.64 us)
        =  0.0000765156136231  1 / us
        =  7.7
```

1.20.5.14.4 Reprogramming CCD Timing

Reprogramming the CCD Controller will happen when the following changes occur

- Changing the exposure time
- Changing the capture window
- Changing trigger modes

Reprogram time consists of two phases

1. Time needed to send data to the CCD controller depending on what is changed
   exposure : abt 2..3ms
   window: abt 4..6ms
   trigger mode: from 5..90ms,
   varies with oldmode/newmode combination
2. Time to initialize (erase) the CCD chip after reprogramming this is fixed, abt 4.5 ms

So for example when reprogramming the capture window you will need (average values)

$t_{regprog}$ = change_window + init_ccd

$t_{regprog}$ = 5ms + 4.5ms

$t_{regprog}$ = 9.5ms

1.20.5.14.5 CCD Sensor Data

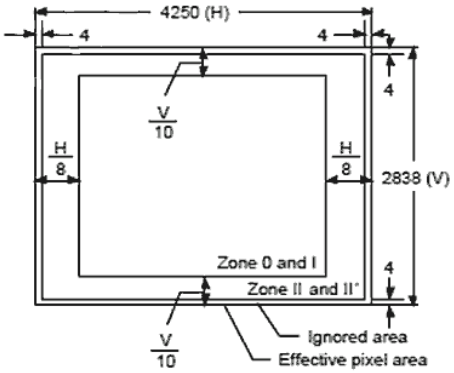
Device Structure

- Interline CCD image sensor
- Image size: Diagonal 15.808mm (Type 1)
- Number of effective pixels: 4248 (H) × 2836 (V) approx. 12.06 Mpixels
- Total number of pixels: 4330 (H) × 2854 (V) approx. 12.36 Mpixels
- Chip size: 15.3mm (H) x 12.0mm (V)
- Unit cell size: 3.1um (H) x 3.1um (V)
- Optical black:
  - Horizontal (H) direction: Front 40 pixels each channel
  - Vertical (V) direction: Front 8 pixels each channel
- Number of dummy bits: Front 1 each channel
- Substrate material: Silicon

1.20.5.14.5.1 Characteristics

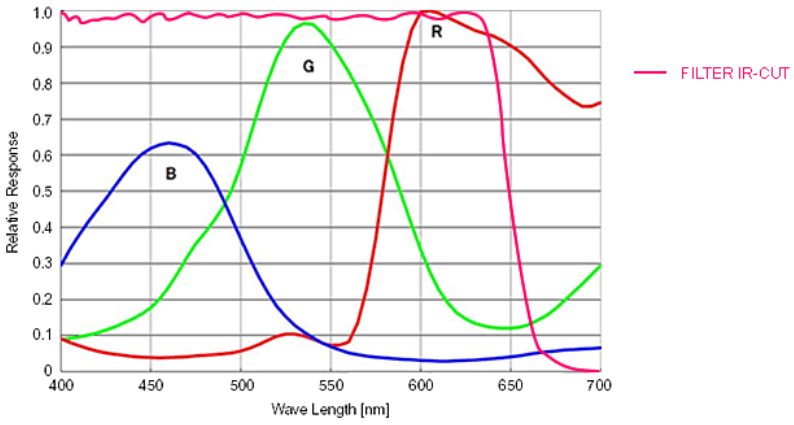
These zone definitions apply to both the color and gray scale version of the sensor.

Zone Definition of Video Signal Shading

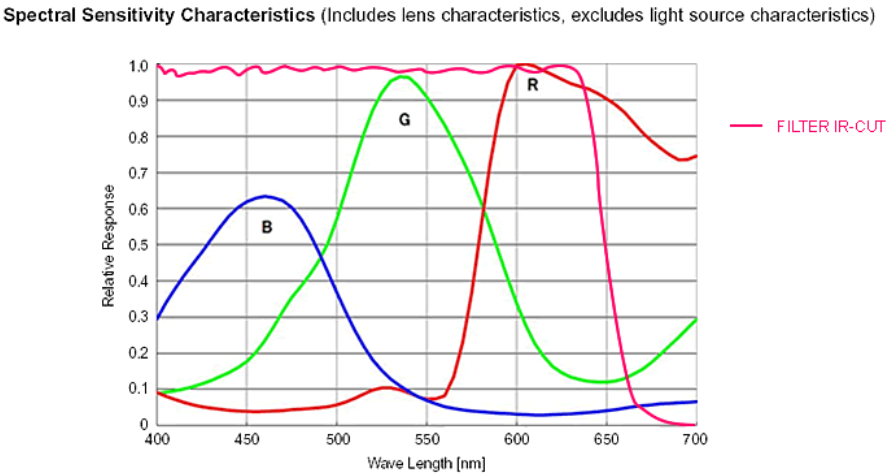


1.20.5.14.5.2 Color version

Spectral Sensitivity Characteristics (Includes lens characteristics, excludes light source characteristics)



1.20.5.14.5.3 Gray scale version



Device Feature And Property List (p. 679)

1.20.5.14.6 Device Feature And Property List

- **mvBlueCOUGAR-XD1212aG Features** (p. 679)
- **mvBlueCOUGAR-XD1212aC Features** (p. 679)

1.20.5.14.6.1 mvBlueCOUGAR-XD1212aG Features

1.20.5.14.6.2 mvBlueCOUGAR-XD1212aC Features

1.21 Appendix C. Tested ARM platforms

MATRIX VISION devices can run on ARM-based Linux platforms without limitations regarding available feature sets or API functions. However, each platform may have its own limits in terms of achievable data throughput, RAM or bus speeds. Apart from that, each platform may also come with its own specific set of challenges. Therefore, certain modifications may need to be adapted in order to get your devices run at maximum performance.

This chapter contains test results from different ARM platforms, as well as the specific information on each platform, especially changes that need to be applied to achieve better performance.

The following platforms have been tested by MATRIX VISION:

System	ARM-Architecture	Technology Test Results					Approx. price	Performance	Suitable for	More information
		USB2.0	USB3.0	GigE	10GigE	PCIe				
NVIDIA Jetson AGX Xavier (p. 681)	NVIDIA Carmel ARMv8.2						ca. 650-700$		Demanding Applications	nvidia.com
NVIDIA Jetson Xavier NX (p. 683)	NVIDIA Carmel ARMv8.2						ca. 700$		Demanding Applications	nvidia.com
NVIDIA Jetson Nano (p. 685)	ARM Cortex-A57						90-100$		Mid-Range Applications	nvidia.com
NVIDIA Jetson TX2 (p. 687)	NVIDIA Denver 2 and ARM Cortex-A57						ca. 400$		Demanding Applications	nvidia.com
i.MX8M Mini (p. 689)	ARM Cortex-A53								Mid-Range Applications	nxp.com
Raspberry Pi 4 (p. 692)	ARM Cortex-A72						35-75$		Price Sensitive Projects	raspberrypi.org
Raspberry Pi Compute Module 4	ARM Cortex-A72						30-80$		Price Sensitive Projects	raspberrypi.org

The system delivers good performance with this device.

The system doesn't work with this device.

The developer kit doesn't work with this device because it provides no PCI Express interface.

The system delivers limited performance with this device.

The system hasn't been tested yet with this device.

Appendices:

- **C.1 ARM64 based devices** (p. 681)
- **C.2 ARMhf based devices** (p. 692)

1.21.1 C.1 ARM64 based devices

- **NVIDIA Jetson AGX Xavier** (p. 681)
- **NVIDIA Jetson Xavier NX** (p. 683)
- **NVIDIA Jetson Nano** (p. 685)
- **NVIDIA Jetson TX2** (p. 687)
- **i.MX8M Mini** (p. 689)

1.21.1.1 NVIDIA Jetson AGX Xavier

CPU	NVIDIA Carmel ARMv8.2 @ 2.26GHz
Cores	8
RAM	32GB
USB2.0 Interfaces	4
USB3.1 Interfaces	3
Ethernet	10/100/1000 MBit
PCIe	1x8 + 1x4 + 1x2 + 2x1 Gen 4.0

1.21.1.1.1 General

Note

The above table describes the specification of the NVIDIA Jetson AGX Xavier Developer Kit.

1.21.1.1.2 Benchmarks

1.21.1.1.2.1 GigE Performance



Test setup

1.21.1.1.2.2 Additional Settings Applied On The System To improve the data transfer between the camera and the ARM device the following Kernel parameters have been modified:

In **/etc/sysctl.d/62-buffers-performance.conf**:

Note

You may have to create this file!

Setting	Value	Description
net.core.wmem_max	16777216	Maximum memory size of a socket buffer for sending in Bytes
net.core.rmem_max	16777216	Maximum memory size of a socket buffer for receiving in Bytes
net.core.udp_mem	10240, 87380, 16777216	Minimum, Default and Maximum memory size of a UDP socket buffer for receiving and sending data in bytes
net.core.netdev_max_backlog	10000	Maximum number of packets which can be buffered if the Kernel does not manage to process them as fast as they are received

NIC Settings

Setting	Value
MTU	8000 Byte

The importance of setting these parameters as above is explained here: **Network Performance Settings** (p. 57).

1.21.1.1.2.3 Results The following scenarios have been tested:

1. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.
2. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlue COUGAR-X102mC	1600 x 1104	BayerRG8 (on camera) -> RGB8 (on host)	68	120.32	∼22%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlue COUGAR-X102mC	1600 x 1104	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	68	120.32	∼20%

1.21.1.2 NVIDIA Jetson Xavier NX

CPU	NVIDIA Carmel ARMv8.2 @ 1.9GHz
Cores	6
RAM	8GB
USB3.1 Interfaces	4
Ethernet	10/100/1000 MBit
PCIe	1x1 + 1x4 Gen 3.0

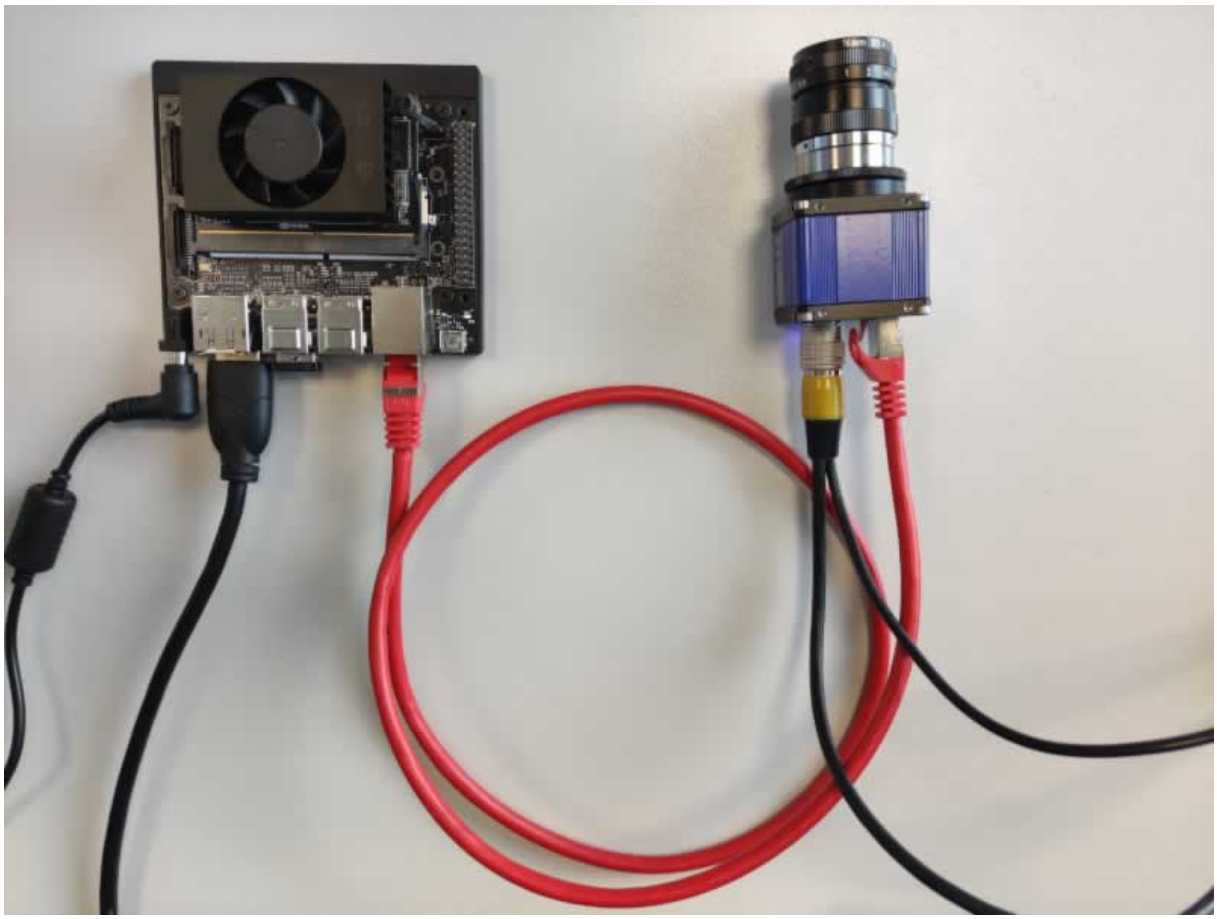
1.21.1.2.1 General

Note

The above table describes the specification of the NVIDIA Jetson Xavier NX Developer Kit.

1.21.1.2.2 Benchmarks

1.21.1.2.2.1 GigE Performance



Test setup

1.21.1.2.2.2 Additional Settings Applied On The System To improve the data transfer between the camera and the ARM device the following Kernel parameters have been modified:

In **/etc/sysctl.d/62-buffers-performance.conf**:

> **Note**
>
> You may have to create this file!

Setting	Value	Description
net.core.wmem_max	16777216	Maximum memory size of a socket buffer for sending in Bytes
net.core.rmem_max	16777216	Maximum memory size of a socket buffer for receiving in Bytes
net.core.udp_mem	10240, 87380, 16777216	Minimum, Default and Maximum memory size of a UDP socket buffer for receiving and sending data in bytes
net.core.netdev_max_backlog	10000	Maximum number of packets which can be buffered if the Kernel does not manage to process them as fast as they are received

NIC Settings

Setting	Value
MTU	8000 Byte

The importance of setting these parameters as above is explained here: **Network Performance Settings** (p. 57).

1.21.1.2.2.3 Results

The following scenarios have been tested:

1. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.
2. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlue COUGAR-X102mC	1600 x 1104	BayerRG8 (on camera) -> RGB8 (on host)	68	120.32	∼34%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlue COUGAR-X102mC	1600 x 1104	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	68	120.32	∼27%

1.21.1.3 NVIDIA Jetson Nano

CPU	Cortex-A57 @ 1.43 GHz
Cores	4
RAM	4GB
USB2.0 Interfaces	1
USB3.0 Interfaces	4
Ethernet	10/100/1000 MBit
PCIe	x1/x2/x4 Gen 2.0

1.21.1.3.1 General

Note

The above table describes the specification of the NVIDIA Jetson Nano Developer Kit.

1.21.1.3.2 Benchmarks

1.21.1.3.2.1 GigE Performance

1.21.1.3.2.2 Additional Settings Applied On The System To improve the data transfer between the camera and the ARM device the following Kernel parameters have been modified:

In **/etc/sysctl.d/62-buffers-performance.conf**:

> **Note**
>
> You may have to create this file!

Setting	Value	Description
net.core.wmem_max	16777216	Maximum memory size of a socket buffer for sending in Bytes
net.core.rmem_max	16777216	Maximum memory size of a socket buffer for receiving in Bytes
net.core.udp_mem	10240, 87380, 16777216	Minimum, Default and Maximum memory size of a UDP socket buffer for receiving and sending data in bytes
net.core.netdev_max_backlog	10000	Maximum number of packets which can be buffered if the Kernel does not manage to process them as fast as they are received

NIC Settings

Setting	Value
MTU	8000 Byte

The importance of setting these parameters as above is explained here: **Network Performance Settings** (p. 57).

1.21.1.3.2.3 Results The following scenarios have been tested:

1. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.
2. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-↩ Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlue↩ COUGAR-↩ X102mC	1600 x 1104	BayerRG8 (on camera) -> RGB8 (on host)	68	120.32	∼59%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlue↩ COUGAR-↩ X102mC	1600 x 1104	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	68	120.32	∼37%

1.21.1.3.3 Remarks

1.21.1.3.3.1 Choose the right power supply

The Jetson Nano has 2 power supply possibilities: via the micro-USB connection or via the Barrel Jack connection.

The power (by default 10W) via the micro-USB connector is not sufficient if you want to connect other peripherals (e.g. keyboard, mouse, cameras, etc...) that draw current from the board. So if when powering the USB/USB3 camera through the USB bus (i.e. no external power supply), please supply the board with power through the Barrel Jack connector (4A@5V).

Otherwise the system will throttle due to over-current.

1.21.1.4 NVIDIA Jetson TX2

CPU	ARM Cortex-A57 @ 2GHz NVIDIA Denver2 @ 2GHz
Cores	4 2
RAM	8GB
USB2.0 Interfaces	1
USB3.0 Interfaces	1
Ethernet	10/100/1000 MBit
PCIe	1x4 + 1x1 | 2x1 + 1x2 Gen 2.0

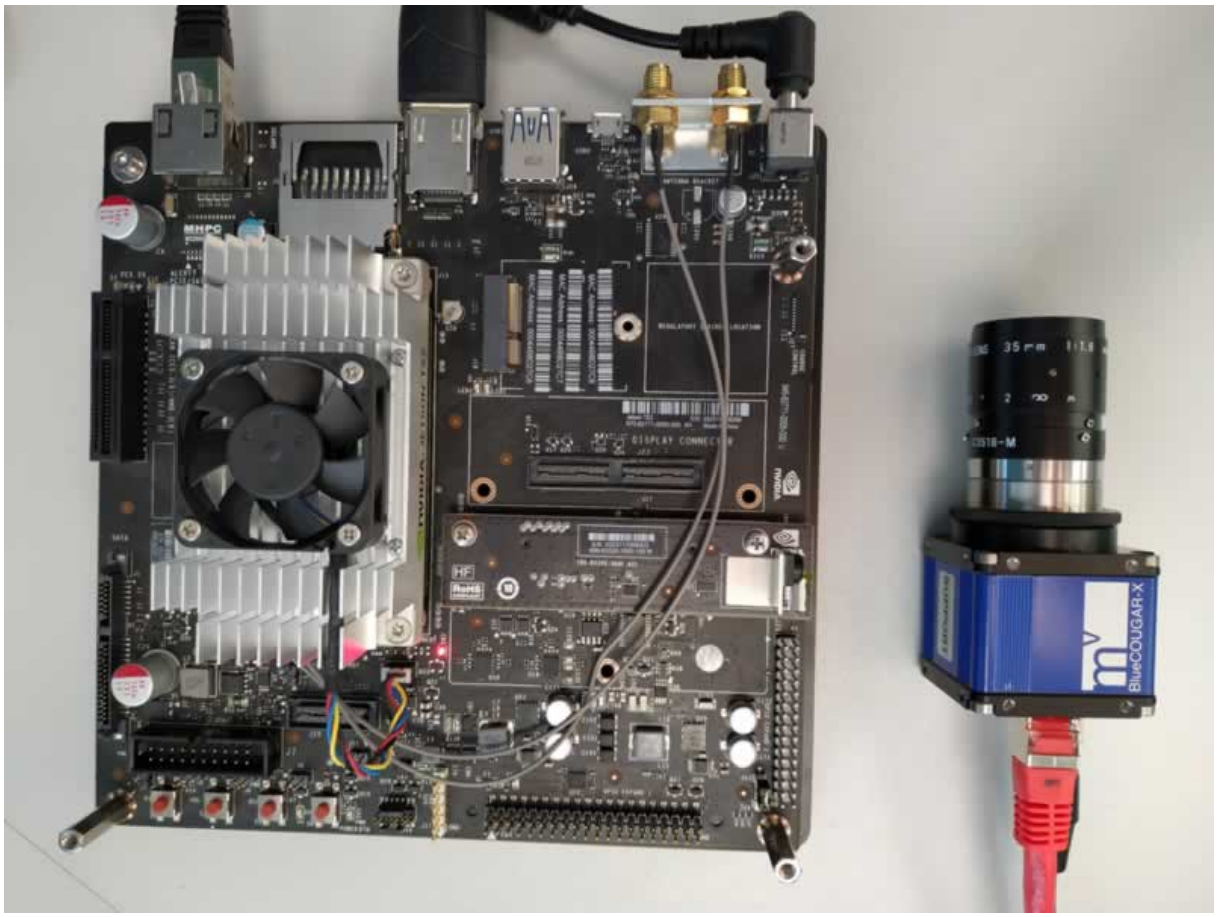
1.21.1.4.1 General

Note

The above table describes the specification of the NVIDIA Jetson TX2 Developer Kit.

1.21.1.4.2 Benchmarks

1.21.1.4.2.1 GigE Performance



Test setup

1.21.1.4.2.2 Additional Settings Applied To The System To improve the data transfer between the camera and the ARM device the following Kernel parameters have been modified:

In **/etc/sysctl.d/62-buffers-performance.conf**:

Note

You may have to create this file!

Setting	Value	Description
net.core.wmem_max	16777216	Maximum memory size of a socket buffer for sending in Bytes
net.core.rmem_max	16777216	Maximum memory size of a socket buffer for receiving in Bytes
net.core.udp_mem	10240, 87380, 16777216	Minimum, Default and Maximum memory size of a UDP socket buffer for receiving and sending data in bytes
net.core.netdev_max_backlog	10000	Maximum number of packets which can be buffered if the Kernel does not manage to process them as fast as they are received

NIC Settings

Setting	Value
MTU	8000 Byte

The importance of setting these parameters as above is explained here: **Checklist for Linux** (p. 57).

1.21.1.4.2.3 Results The following scenarios have been tested:

1. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.
2. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-↩ Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlue↩ COUGAR-↩ X102mC	1600 x 1104	BayerRG8 (on camera) -> RGB8 (on host)	68	120.32	∼53%
Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load
mvBlue↩ COUGAR-↩ X102mC	1600 x 1104	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	68	120.32	∼42%

1.21.1.5 i.MX8M Mini

CPU	ARM Cortex®-A53 @ 1.6GHz
Cores	4
RAM	1 GB
USB2.0 Interfaces	2
USB3.0 Interfaces	None
Ethernet	1000 MBit
PCIe	1 x 1 Lane Gen 2.0

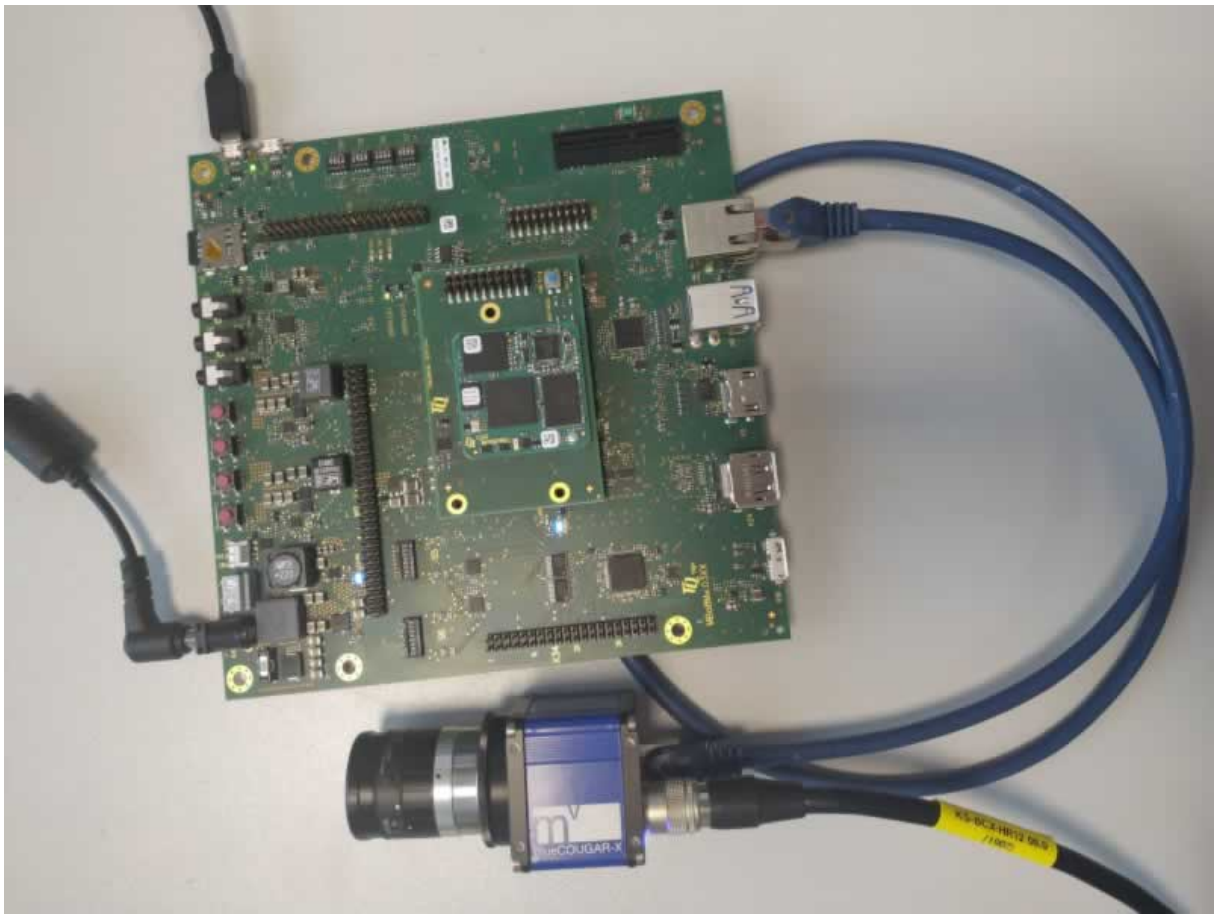
1.21.1.5.1 General The carrier-board used in this test: MBa8Mx from TQ-Systems GmbH

Note

If you are looking for more information and guidance about installing mvIMPACT Acquire driver packages via the Yocto Project, please choose an API-manual suited for your programming language and then go to chapter "Installation From Private Setup Routines -> Embedded Linux -> Yocto Project". All API-manuals can be found under `https://www.matrix-vision.com/manuals/`.

1.21.1.5.2 Benchmarks

1.21.1.5.2.1 GigE Performance Test



Test setup

1.21.1.5.2.2 Additional Settings Applied On The System To improve the data transfer between the camera and the ARM device the network buffers may need to be increased. The modification can be applied at runtime or can be included in Yocto recipes (sample recipes are given in chapter "Installation From Private Setup Routines -> Embedded Linux -> Yocto Project" of mvIMPACT Acquire API manuals).

In **/etc/sysctl.d/62-buffers-performance.conf**:

Setting	Value	Description
net.core.wmem_max	16777216	Maximum memory size of a socket buffer for sending in Bytes
net.core.rmem_max	16777216	Maximum memory size of a socket buffer for receiving in Bytes
net.core.udp_mem	10240, 87380, 16777216	Minimum, Default and Maximum memory size of a UDP socket buffer for receiving and sending data in bytes
net.core.netdev_max_backlog	10000	Maximum number of packets which can be buffered if the Kernel does not manage to process them as fast as they are received

The importance of setting these parameters as above is explained here: **Network Performance Settings** (p. 57).

1.21.1.5.2.3 Results

Note

The tests using GigE Vision™ device were conducted using a MTU value of 1500 Bytes. This limitation is caused by the default kernel which only supports MTU value up to 1500 Bytes. In this case, a reliable data transmission can only be achieved when NO de-Bayering is carried out on the host system. If de-Bayering is required on the host system, the transmission bandwidth (i.e. `DeviceLinkThoughput`) has to be limited (to e.g. 42[MB/s]) and the `RequestCount` has to be increased (to e.g. 30) to avoid frame loss. If the NIC supports jumbo frames, you can also increase the maximum supported MTU (to e.g. 8000 Bytes) by customizing the kernel to improve the performance when de-Bayering on the host system.

A `Request` in the mvIMPACT Acquire API represents a buffer where an image with the current device configuration is captured into. In order to avoid losing images at a high FPS, it's recommended to increase the number of these request buffers (i.e. '`RequestCount`' in '`SystemSettings`', by default the value is set to 10), so that the driver can continue capturing image data even if the host application is slower at processing an image than the camera transferring one.

The following scenarios have been tested:

1. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. Due to the limited max. MTU of i.MX8M Mini, the transmission bandwidth (i.e. `DeviceLinkThoughput`) has to be limited and the `RequestCount` has to be increased to avoid frame loss in this case. Also see the `Note` above.
2. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-↩ Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Camera	Resolution	Pixel Format	Frame Rate [Frames/s]	Bandwidth [MB/s]	CPU Load (averaged over 4 cores)
mvBlue↩ COUGAR-↩ X104iC	2064 x 1544	BayerRG8 (on camera) -> RGB8 (on host)	13.2	42	∼28.25%
mvBlue↩ COUGAR-↩ X104iC	2064 x 1544	BayerRG8 (on camera) -> BayerRG8/Raw (on host)	37	118	∼22.6%

1.21.2 C.2 ARMhf based devices

- **Raspberry Pi 4** (p. 692)

1.21.2.1 Raspberry Pi 4

1.21.2.1.1 General The Raspberry Pi 4 is a well priced platform regarding its performance.

CPU	Cortex-A72 @ 1500MHz
Cores	4
RAM	1/2/4/8 GB
USB2.0 Interfaces	2
USB3.0 Interfaces	2
Ethernet	10/100/1000 MBit

Note

For the following benchmark the 4GB version of the Raspberry Pi 4 with Raspbian OS has been used.

1.21.2.1.2 Benchmarks

1.21.2.1.2.1 GigE Performance

1.21.2.1.2.2 Additional Settings Applied On The System To improve the data transfer between the camera and the ARM device the following Kernel parameters have been modified:

In **/etc/sysctl.d/62-buffers-performance.conf**:

Note

You may have to create this file!

Setting	Value	Description
net.core.wmem_max	16777216	Maximum memory size of a socket buffer for sending in Bytes
net.core.rmem_max	16777216	Maximum memory size of a socket buffer for receiving in Bytes
net.core.udp_mem	10240, 87380, 16777216	Minimum, Default and Maximum memory size of a UDP socket buffer for receiving and sending data in bytes
net.core.netdev_max_backlog	10000	Maximum number of packets which can be buffered if the Kernel does not manage to process them as fast as they are received

NIC Settings

Setting	Value
MTU	1500 Byte

The importance of setting these parameters as above is explained here: **Network Performance Settings** (p. 57).

Attention

All tests using network (GigE Vision™) device have been done using the default MTU value of 1500 Bytes. This limitation is caused by the Raspberry Pi 4's default Kernel supporting MTU value only up to 1500 Bytes. There might be the possibility to compile a custom Kernel to enable the support of lager MTUs but the unmodified Kernel was used for testing.

1.21.2.1.2.3 Results The following scenarios have been tested:

1. When de-Bayering is carried out on the host system: The camera delivers Bayer8 image data to the host system. The Bayer8 image data then get de-Bayered to RGB8 format on the host system. This setting results in a higher frame rate but a higher CPU load as well.
2. When no de-Bayering is performed: The camera delivers Bayer8 image data to the host system. No de-↩ Bayering is performed. This settings results in a lower CPU load and a higher frame rate. The behavior is identical to monochrome cameras.

Attention

For the internal NIC there is no support for MTUs bigger than 1500 Bytes in the standard Kernel as it seems (see above).